



MultiBase

Manual del Programador

BASE100

BASE 100, S.A.
www.base100.com

Índice

CAPÍTULO 1. PROGRAMACIÓN CON MULTIBASE	11
INTRODUCCIÓN.....	12
CTL CON SQL EMBEBIDO	13
COMANDOS ASOCIADOS AL DESARROLLO	14
ELEMENTOS AUXILIARES	15
CAPÍTULO 2. EL ENTORNO DE DESARROLLO	17
TRANS: ENTORNO DE DESARROLLO DE MULTIBASE	18
MANEJO DE UN MENÚ	21
LAS TECLAS DE ACCIÓN	23
LA TECLA DE AYUDA	23
PANTALLAS DE ENTRADA DE DATOS.....	24
MANEJO DE UNA VENTANA DE CONSULTA	25
PERSIANA «BASE DE DATOS»	26
<i>Opción «Seleccionar»</i>	<i>26</i>
<i>Opción «Crear»</i>	<i>27</i>
<i>Opción «Tablas»</i>	<i>29</i>
<i>Opción «Views»</i>	<i>44</i>
<i>Opción «Usuarios».....</i>	<i>45</i>
<i>Opción «Información»</i>	<i>47</i>
<i>Opción «DBSQL».....</i>	<i>49</i>
<i>Opción «Esquema».....</i>	<i>51</i>
<i>Opción «Borrar»</i>	<i>52</i>
<i>Opción «Verif. Diccionario»</i>	<i>52</i>
<i>Opción «Documentar».....</i>	<i>53</i>
PERSIANA «DESARROLLO»	54
<i>Opción «Programas»</i>	<i>54</i>
<i>Opción «Módulos/Generación».....</i>	<i>58</i>
<i>Opción «Ayudas y Mensajes»</i>	<i>65</i>
<i>Opción «Información»</i>	<i>66</i>
PERSIANA «APLICACIÓN»	67
<i>Opción «Definir».....</i>	<i>67</i>
<i>Opción «Compilar»</i>	<i>68</i>
<i>Opción «Generar Prototipo»</i>	<i>69</i>
<i>Opción «Verificar Diccionario»</i>	<i>70</i>
<i>Opción «Modificar Diccionario»</i>	<i>71</i>
PERSIANA «SQL»	72
<i>Opción «Elegir».....</i>	<i>73</i>
<i>Opción «Ejecutar»</i>	<i>73</i>
<i>Opción «Nuevo»</i>	<i>73</i>
<i>Opción «Editor»</i>	<i>73</i>
<i>Opción «Salida».....</i>	<i>73</i>
<i>Opción «Guardar»</i>	<i>74</i>
<i>Opción «Borrar»</i>	<i>74</i>
<i>Opción «Insertar»</i>	<i>74</i>
<i>Opción «Documentar».....</i>	<i>74</i>

PERSIANA «DOCUMENTACIÓN»	75
<i>Opción «Base de Datos»</i>	75
<i>Opción «Desarrollo»</i>	76
<i>Opción «Generar/Editar»</i>	78
PERSIANA «ENTORNO».....	80
<i>Opción «Directorios»</i>	80
<i>Opción «Formatos»</i>	81
<i>Opción «Varios»</i>	81
<i>Opción «Listar»</i>	82
INFORMACIÓN DEL ENTORNO TRANS	83
TRANS INTERACTIVO	84
<i>Persiana «Fichero»</i>	85
<i>Persiana «Salida»</i>	86
<i>Persiana «Otros»</i>	87
CATÁLOGO DEL ENTORNO DE DESARROLLO (TABLAS «EP_*»).....	87
<i>Columnas de las tablas del entorno de desarrollo</i>	95
CAPÍTULO 3. CÓMO EMPEZAR	97
INTRODUCCIÓN	98
EJECUCIÓN DEL ENTORNO DE DESARROLLO	98
CREACIÓN DE UNA BASE DE DATOS	100
PERMISOS SOBRE LA BASE DE DATOS	101
CREACIÓN DE TABLAS.....	101
ESTABLECIMIENTO DE LA INTEGRIDAD REFERENCIAL	105
<i>Claves primarias («Primary Keys»)</i>	105
<i>Claves referenciales («Foreign Keys»)</i>	106
CREACIÓN DE ÍNDICES	107
GENERACIÓN DE MÓDULOS.....	108
CÓMO PROBAR LA APLICACIÓN	108
CAPÍTULO 4. SINTAXIS EMPLEADA EN CTL	111
CONVENCIONES UTILIZADAS	112
MÓDULOS Y PROGRAMAS.....	112
SECCIONES DE UN MÓDULO FUENTE CTL	114
ESTRUCTURA DE UN MÓDULO PRINCIPAL CTL.....	115
ESTRUCTURA DE UN MÓDULO LIBRERÍA CTL	116
COMPONENTES DE UN MÓDULO	117
CAPÍTULO 5. CONCEPTOS BÁSICOS DEL CTL	119
IDENTIFICADOR CTL	120
TIPOS DE DATOS CTL.....	120
ATRIBUTOS	122
<i>AUTONEXT (CTL)</i>	122
<i>CHECK (CTL y CTSQL)</i>	122
<i>COMMENTS (CTL)</i>	124
<i>DEFAULT (CTL y CTSQL)</i>	125
<i>DOWNSHIFT (CTL y CTSQL)</i>	125
<i>FORMAT (CTL y CTSQL)</i>	125
<i>HELP (CTL)</i>	127
<i>INCLUDE (CTL)</i>	128
<i>LABEL (CTL y CTSQL)</i>	128

LEFT (CTL y CTSQL)	128
LOOKUP (CTL)	129
NOENTRY (CTL y CTSQL)	129
NOUPDATE (CTL y CTSQL)	129
PICTURE (CTL y CTSQL)	129
QUERYCLEAR (CTL)	130
REQUIRED (CTL)	130
REVERSE (CTL)	130
RIGHT (CTL y CTSQL)	130
SCROLL (CTL)	130
UNDERLINE (CTL)	131
UPSHIFT (CTL y CTSQL)	131
VERIFY (CTL)	131
ZEROFILL (CTL y CTSQL)	131
EXPRESIONES CTL	131
Expresiones simples	131
Expresiones compuestas	132
Expresiones de STREAMS	132
Expresiones de FORM	133
Expresiones de CURSOR	133
OPERADORES CTL	133
REGLAS DE PRECEDENCIA DE OPERADORES	137
VARIABLES INTERNAS CTL	138
FUNCIONES INTERNAS CTL	139
Funciones de fecha	139
Funciones de hora	140
Funciones decimales	140
Funciones de caracteres	140
Funciones de mensajes	142
Funciones de teclado	142
Funciones de sistema	142
Funciones aritméticas	143
Funciones de manejo de directorios	143
Función de manejo del cursor de la pantalla	143
Funciones de manejo de «Queries» en un FORM	144
Funciones de manejo de «Cursores» SQL	145
Funciones de evaluación	145
Funciones de manejo de ficheros	145
Funciones trigonométricas y de conversión	146
Funciones financieras	147
Función de control de página en un FORM o FRAME	148
Funciones de asignación dinámica de variables	148
Funciones diversas	148
Funciones específicas de la versión para Windows	150
DEFINICIÓN DE OBJETO	155
COMPONENTES DE UN OBJETO	155
CAPÍTULO 6. OBJETOS PROCEDURALES DE LA SECCIÓN DEFINE	157
INTRODUCCIÓN	158
VARIABLES	158

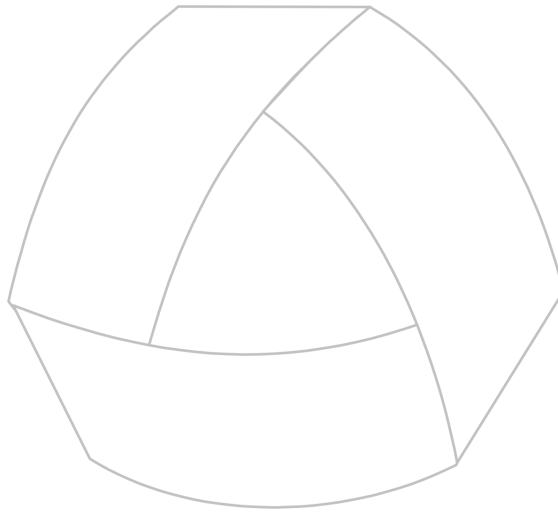
PARÁMETROS.....	160
ARRAYS 162	
CAPÍTULO 7. CONTROL DE FLUJO EN CTL	165
INSTRUCCIONES DE CONTROL DE FLUJO.....	166
Instrucción FOR	167
Instrucción FOREVER	167
Instrucción IF	167
Instrucción SWITCH	168
Instrucción WHILE	169
ANIDACIÓN DE ESTRUCTURAS	169
INSTRUCCIONES DE RUPTURA DE BUCLES	170
Instrucción BREAK	170
Instrucción CONTINUE.....	170
Instrucción EXIT PROGRAM.....	171
Instrucción RETURN.....	171
ASIGNACIÓN DE VALORES A VARIABLES CTL.....	172
Instrucción LET	172
Instrucción PROMPT FOR	173
LLAMADAS A FUNCIONES	173
LLAMADAS A MÓDULOS/PROGRAMAS CTL.....	175
Ejecución desde la línea de comando del sistema operativo	175
Llamada desde otro programa CTL	176
Llamada a objetos de un módulo librería.....	177
Variables de entorno relacionadas.....	177
LLAMADAS A PROGRAMAS DEL SISTEMA OPERATIVO.....	178
Comandos del sistema operativo embebidos.....	179
CAPÍTULO 8. ENTRADA Y SALIDA DE INFORMACIÓN POR PANTALLA	181
LA LISTA DE «DISPLAY»	182
INSTRUCCIONES DE SALIDA.....	182
Instrucción CLEAR.....	183
Instrucción CLEAR AT.....	183
Instrucción CLEAR BOX	184
Instrucción CLEAR FRAME BOX.....	185
Instrucción CLEAR FROM	185
Instrucción CLEAR LINE.....	186
Instrucción CLEAR LIST.....	187
Instrucción DISPLAY.....	187
Instrucción DISPLAY BOX	188
Instrucción DISPLAY FRAME BOX.....	188
Instrucción DISPLAY LINE.....	189
Instrucción MESSAGE	189
Instrucción PAUSE	190
Instrucción REDRAW.....	190
Instrucción VIEW	191
INSTRUCCIONES DE ENTRADA.....	192
Instrucción EDIT.....	192
Instrucción PAUSE	193
Instrucción PROMPT FOR	193

Instrucción <i>READ KEY</i>	194
ALTERACIÓN DE LAS CARACTERÍSTICAS ESTÁNDARES DEL CTL	195
Instrucción <i>OPTIONS</i>	195
OTRAS INSTRUCCIONES.....	196
Instrucción <i>PATH WALK</i>	197
Instrucción <i>TREE WALK</i>	197
Instrucción <i>TSQL</i>	198
Instrucción <i>WINDOW</i>	198
CAPÍTULO 9. USO DEL SQL EMBEBIDO EN CTL.....	201
INTRODUCCIÓN.....	202
VARIABLES «HOSTS».....	202
VARIABLES RECEPTORAS	205
APERTURA Y CIERRE DE UNA BASE DE DATOS	207
MANIPULACIÓN DE DATOS CON INSTRUCCIONES SQL.....	209
Instrucción <i>SELECT</i>	209
Instrucción <i>INSERT</i>	210
Instrucción <i>DELETE</i>	211
Instrucción <i>UPDATE</i>	212
VENTANAS DE CONSULTA Y SELECCIÓN (INSTRUCCIÓN <i>WINDOW</i>)	213
IMPORTACIÓN Y EXPORTACIÓN DE DATOS DESDE/HACIA FICHEROS ASCII	215
Instrucción <i>LOAD</i>	216
Instrucción <i>UNLOAD</i>	217
SQL DINÁMICO.....	218
Instrucción <i>TSQL</i>	219
Instrucción <i>PREPARE</i>	220
Instrucción <i>EXECUTE</i>	222
TABLAS TEMPORALES	223
Tabla temporal creada con la cláusula « <i>INTO TEMP</i> »	224
Tabla temporal creada con la instrucción <i>CREATE TEMP TABLE</i>	225
« <i>VIEWS</i> » ACTUALIZABLES	226
CAPÍTULO 10. EL OBJETO CURSOR	229
EL CURSOR	230
10.2. DECLARACIÓN DE UN CURSOR: DIRECTIVA <i>DECLARE</i>	230
Sección <i>CONTROL</i> del <i>CURSOR</i>	234
APERTURA DEL CURSOR: INSTRUCCIÓN <i>OPEN</i>	239
LECTURA DE FILAS DEL CURSOR.....	240
Instrucción <i>FETCH</i>	241
Instrucción <i>FOREACH</i>	242
CIERRE DEL CURSOR: INSTRUCCIÓN <i>CLOSE</i>	246
INSTRUCCIÓN <i>WINDOW</i> ASOCIADA A UN CURSOR	246
EXPRESIONES DEL CURSOR	247
CAPÍTULO 11. EL FORM: OBJETO NO PROCEDURAL DE <i>DEFINE</i>	249
FORM: DEFINICIÓN Y USOS.....	250
CONCEPTOS BÁSICOS EMPLEADOS EN EL FORM	250
ESTRUCTURA DE UN FORM	251
ASPECTO DE LA PANTALLA: SECCIÓN <i>SCREEN</i>	253
MANTENIMIENTO DE TABLAS: SECCIÓN <i>TABLES</i>	256
MANTENIMIENTO DE COLUMNAS: SECCIÓN <i>VARIABLES</i>	258

<i>Variables asociadas a columnas</i>	258
<i>Variables PSEUDO</i>	262
<i>Variables VARIABLE</i>	263
<i>Variables DISPLAYTABLE</i>	266
<i>Resumen de la sección VARIABLES</i>	268
OPERACIONES A REALIZAR CON UN FORM: SECCIÓN MENU	269
CONTROL DE PROCESOS: SECCIONES CONTROL Y EDITING	271
<i>Grabación de filas nuevas</i>	272
<i>Consulta de filas</i>	275
<i>Modificación de la fila en curso</i>	279
<i>Borrado de la fila en curso</i>	282
<i>Bloques de control activados con el repintado</i>	284
<i>Instrucciones no procedurales activadas con cambio de tabla</i>	285
DISEÑO DE LA PANTALLA: SECCIÓN LAYOUT	290
ENLACE DE TABLAS: SECCIÓN JOINS.....	291
MANTENIMIENTO DE UNA TABLA TEMPORAL	298
EMPLEO DE MÁS DE UN FORM POR PROGRAMA.....	299
INSTRUCCIONES RELACIONADAS CON EL FORM	301
<i>Instrucción ADD</i>	301
<i>Instrucción ADD ONE</i>	302
<i>Instrucción CANCEL</i>	303
<i>Instrucción CLEAR FORM</i>	304
<i>Instrucción DISPLAY FORM</i>	305
<i>Instrucción EXIT EDITING</i>	306
<i>Instrucción GET QUERY</i>	307
<i>Instrucción HEAD</i>	308
<i>Instrucción INTERCALATE</i>	310
<i>Instrucción LINES</i>	311
<i>Instrucción MENU FORM</i>	312
<i>Instrucción MODIFY</i>	313
<i>Instrucción NEXT FIELD</i>	314
<i>Instrucción NEXT ROW</i>	315
<i>Instrucción NEXT TABLE</i>	315
<i>Instrucción OUTPUT</i>	316
<i>Instrucción PREVIOUS ROW</i>	317
<i>Instrucción QUERY</i>	318
<i>Instrucción READ KEY THROUGH FORM</i>	319
<i>Instrucción REMOVE</i>	320
<i>Instrucción RETRY</i>	321
FUNCIONES Y VARIABLES INTERNAS CTL	322
CAPÍTULO 12. EL FRAME: OBJETO DE ENTRADA Y SALIDA DE INFORMACIÓN	323
EL FRAME: DEFINICIÓN Y USOS	324
ESTRUCTURA DE UN FRAME	324
ASPECTO DE LA PANTALLA: SECCIÓN SCREEN	326
VARIABLES COMPONENTES DEL FRAME: SECCIÓN VARIABLES.....	330
MENÚ DEL FRAME: SECCIÓN MENU.....	332
FORMAS DE EDICIÓN: SECCIONES CONTROL Y EDITING	335
<i>Entrada de nuevos valores sobre variables</i>	336
<i>Modificación de valores sobre variables del FRAME</i>	340

<i>Consultas dinámicas con variables del FRAME</i>	344
<i>Bloques de control activados con el repintado</i>	347
DISEÑO DE LA PANTALLA: SECCIÓN LAYOUT.....	347
VARIABLES DEL FRAME EN INSTRUCCIONES SQL	349
INSTRUCCIONES RELACIONADAS	352
<i>Instrucción CANCEL</i>	353
<i>Instrucción CLEAR FRAME</i>	354
<i>Instrucción DISPLAY FRAME</i>	355
<i>Instrucción EXIT EDITING</i>	356
<i>Instrucción INPUT FRAME</i>	357
<i>Instrucción MENU FRAME</i>	360
<i>Instrucción NEXT FIELD</i>	361
<i>Instrucción READ KEY THROUGH FRAME</i>	362
<i>Instrucción RETRY</i>	364
FUNCIONES Y VARIABLES INTERNAS CTL.....	365
CAPÍTULO 13. EL STREAM: OBJETO PARA COMUNICAR CON EL SISTEMA OPERATIVO	367
INTRODUCCIÓN.....	368
DEFINICIÓN DE STREAM	368
TIPOS DE STREAMS	368
STREAMS DE SALIDA.....	369
<i>Formateo de la página de salida</i>	369
<i>Apertura del STREAM</i>	370
<i>Redireccionamiento de la información hacia el STREAM</i>	379
<i>Cierre del STREAM de salida</i>	389
<i>Escritura en binario</i>	390
<i>Manejo del «buffer» de escritura</i>	390
<i>Otras instrucciones relacionadas con STREAM OUTPUT</i>	391
<i>Expresiones relacionadas con STREAMS de salida</i>	394
STREAM DE ENTRADA	394
<i>Apertura del STREAM</i>	395
<i>Recepción de información desde un STREAM de entrada</i>	398
<i>Lectura en binario desde un STREAM</i>	402
<i>Cierre del STREAM de entrada</i>	404
<i>Expresiones relacionadas con STREAMS de lectura</i>	404
STREAMS DE LECTURA/ESCRITURA.....	405
<i>Comunicación con un servicio remoto</i>	406
CAPÍTULO 14. TIPOS DE MENÚS CTL	411
MENÚS CTL.....	412
MENÚS «LOTUS» O DE «ICONOS» (WINDOWS)	413
MENÚS «POP-UP»	415
MENÚS «PULLDOWN»	417
INSTRUCCIONES RELACIONADAS	422
<i>Instrucción EXIT MENU</i>	422
<i>Instrucción EXIT PULLDOWN</i>	423
<i>Instrucción MENU</i>	423
<i>Instrucción NEXT OPTION</i>	424
<i>Instrucción PULLDOWN</i>	425
CAPÍTULO 15. FUNCIONES DE USUARIO.....	427

INTRODUCCIÓN	428
ESTRUCTURA Y CARACTERÍSTICAS DE UNA FUNCIÓN.....	428
INSTRUCCIONES RELACIONADAS	431
Instrucción <i>CALL</i>	431
Instrucción <i>RETURN</i>	433
CAPÍTULO 16. FICHEROS DE AYUDA Y MENSAJES.....	435
INTRODUCCIÓN	436
INCLUSIÓN DE TEXTOS DE AYUDA DE PROGRAMAS.....	436
LITERALES, MENSAJES O ETIQUETAS	438
LITERALES DE PANTALLA DE UN FORM/FRAME	440
LITERALES DE MENÚS	441
CAPÍTULO 17. BLOQUEOS	445
INTRODUCCIÓN	446
BLOQUEO DE LA BASE DE DATOS	446
BLOQUEO DE UNA TABLA	446
BLOQUEO DE UNA FILA.....	447
BLOQUEO DE UN NÚMERO INDETERMINADO DE FILAS.....	449
CAPÍTULO 18. TRANSACCIONES	451
INTRODUCCIÓN	452
CREACIÓN DE UNA BASE DE DATOS TRANSACCIONAL.....	452
CONVERTIR UNA BASE DE DATOS EN TRANSACCIONAL Y VICEVERSA.....	453
CÓMO CAMBIAR DE FICHERO TRANSACCIONAL	453
CONTROL DE TRANSACCIONES	454
RECUPERACIÓN DE DATOS	455
CAPÍTULO 19. CONTROL DE ERRORES E INTERRUPCIONES	457
INTRODUCCIÓN	458
PROPAGACIÓN DEL ERROR	463
CAPÍTULO 20. PARTICULARIDADES DE LA VERSIÓN PARA WINDOWS	465
INTRODUCCIÓN	466
UTILIZACIÓN DEL PORTAPAPELES	466
EMPLEO DE LIBRERÍAS DINÁMICAS (DLLs)	467
PROTOCOLO DE INTERCAMBIO DINÁMICO DE DATOS (DDE)	475



MultiBase
MANUAL DEL PROGRAMADOR

CAPÍTULO 1. Programación con MultiBase

Introducción

A lo largo de los años 80 del siglo XX fueron apareciendo en el mercado diversas herramientas de programación que, de una forma u otra, intentaban facilitar el trabajo en el entorno de aplicaciones de gestión. Unas conscientemente y otras inconscientemente, lo que todas pretendían era disminuir el «gap» (salto) entre los tres personajes involucrados en la realización de una aplicación: el usuario final (operador), el analista y el programador.

MultiBase es una herramienta de programación en la que el lenguaje de programación utilizado hace que el «gap» entre el analista y el programador sea prácticamente imperceptible.

Entre otras, las ventajas más importantes del lenguaje de MultiBase son las siguientes:

- Poseer un alto contenido no procedural. Es decir, disponer de un gran número de estructuras e instrucciones no procedurales que le proporcionan una gran sencillez de manejo, pasando a ser responsabilidad del lenguaje un gran número de características que en otros lenguajes serían responsabilidad del programador.
- Proporcionar un interface de usuario independiente del programador, lo que hace que todas las aplicaciones escritas con MultiBase posean la misma ergonomía.
- Ligero. Por la estructura de implementación empleada, en una plataforma UNIX/Linux cada usuario nunca tendrá más de dos procesos en memoria, independientemente del número de llamadas anidadas a otros programas que haya realizado.
- Multiidioma. Tanto la herramienta como las aplicaciones con ella desarrolladas pueden trabajar en cualquier idioma.
- Modular. Un programa de MultiBase puede tener tantos módulos como sea necesario. Estos módulos son cargados en memoria en tiempo de ejecución en el momento de ser llamados.
- Fácil de aprender. El uso de MultiBase no exige conocimientos especiales. Cualquier programador acostumbrado a los lenguajes de 3ª generación podrá empezar a manejarlo inmediatamente.
- Portable. La programación en ficheros fuentes es compatible en diferentes entornos operativos (UNIX/Linux y Windows). Si desea instalar la aplicación en un sistema operativo distinto del que se ha desarrollado, bastará simplemente con compilar los módulos fuente. En Windows, ya sea en versiones monopuesto o de red local, no es necesaria siquiera la recompilación. Sin embargo, sí será necesaria la recompilación y el enlace de programas entre distintas arquitecturas y plataformas UNIX/Linux.
- Diferentes SQL. MultiBase puede manejar distintos SQL de los existentes en el mercado sin necesidad de modificar la programación, tales como Oracle, Informix.
- Comunicación entre dos SQL. MultiBase puede comunicar dos SQL en una misma máquina, tanto en modo local como en cliente-servidor, incluso tratándose de diferentes gestores de base de datos.
- Cliente-Servidor. Además de las instalaciones en local sobre UNIX/Linux y Windows, MultiBase permite combinar todos ellos en instalaciones con arquitectura cliente-servidor. En este tipo de instalaciones, las máquinas «clientes», sobre las que se ejecutará el lenguaje CTL, pueden tener instalado cualquier sistema operativo, mientras que la máquina «servidor» deberá ser necesariamente UNIX/Linux. Los tipos de instalación soportados por MultiBase son:
 - UNIX/Linux (cualquier plataforma).
 - Windows (monopuesto y en red).
 - Cliente-Servidor:
 - UNIX/Linux-UNIX/Linux.
 - Windows-UNIX/Linux.

CTL con SQL embebido

MultiBase posee dos herramientas básicas sobre las que apoya su estructura: Un lenguaje y un servidor de base de datos relacional.

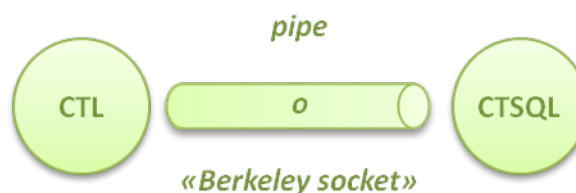
El lenguaje se llama CTL, mientras que el servidor de base de datos empleado por defecto es el CTSQL (SQL propio de MultiBase), si bien, como ya hemos comentado anteriormente, puede manejar distintos gestores de los existentes en el mercado. A su vez, el lenguaje está compuesto por dos comandos, un compilador (**ctlcomp**) y un interpretador (**ctl**).



Estructura de capas de MultiBase.

Alrededor de este núcleo, y desarrollada con CTL, se encuentra la siguiente capa de MultiBase, el entorno de desarrollo de aplicaciones (TRANS). Esta capa representa el interface del programador con la herramienta, proporcionando un entorno de desarrollo de muy fácil manejo y muy elevada productividad. A través de esta capa se proporciona acceso al editor de programas, al compilador, al enlazador («linker»), al lanzador (CTL), al SQL en modo interactivo, etc. Asimismo, se proporciona acceso a los diccionarios, tanto de programación como de la base de datos, y a otro tipo de herramientas de utilidad en el momento de desarrollar una aplicación. Este entorno de desarrollo se explica en detalle en el siguiente capítulo de este mismo volumen.

La comunicación entre el CTL y el CTSQL se efectúa con estructura de cliente-servidor, con paso de mensajes a través de «tuberías» («pipes») en el caso del sistema operativo UNIX/Linux.



Estructura cliente-servidor del CTL y el CTSQL.

Este tipo de comunicación se consigue a través de un protocolo de comunicaciones, de forma que cuando el lenguaje encuentra una instrucción de manejo de la base de datos, en lugar de ejecutarla la envía al servidor (el CTSQL), y éste, una vez procesada, devuelve el resultado al cliente (el CTL).

El lenguaje que se utiliza para acceder a la base de datos es un SQL (Structured Query Language) convertido desde hace años en el estándar para manejar bases de datos relacionales. En el caso del MultiBase, la única forma de ejecutar instrucciones de SQL es a través del CTL, el cual posee todas las instrucciones del SQL en su gramática. Este hecho se conoce comúnmente como lenguaje con SQL embebido, o «empotrado».



El CTL posee el SQL embebido.

Por ello, aunque en realidad existen dos lenguajes diferentes y ejecutados por dos procesos diferentes (el CTL y el CTSQL), a lo largo de este manual serán considerados como un único lenguaje a todos los efectos. Es decir, ambas gramáticas se explicarán como si se tratase de una sola.

Todas las instrucciones de mantenimiento de la base de datos (creación de bases de datos, tablas, índices, etc.) son ejecutables tanto de forma interactiva, a través de alguna de las opciones del menú principal del entorno de desarrollo (TRANS), como a través de CTL.

Comandos asociados al desarrollo

En el desarrollo de una aplicación intervienen diferentes elementos de MultiBase. Estos elementos imprescindibles se encuentran representados en el entorno de desarrollo por ciertas opciones del menú. Para más información acerca de este entorno [consulte el siguiente capítulo](#) de este mismo volumen.

Los elementos que intervienen en el desarrollo de una aplicación están representados a su vez por los siguientes comandos:

trans	Ejecuta el entorno de desarrollo de MultiBase. Asimismo, se utiliza para la ejecución de instrucciones de tipo SQL de forma interactiva. Este comando sólo está disponible en las versiones para UNIX/Linux. Por lo que respecta a las versiones para Windows, la forma de acceder al entorno de desarrollo será ejecutando el programa CTL «ep_trans», que se encuentra en el subdirectorio «ep» de MultiBase.
tword	Disponible únicamente en las versiones para UNIX/Linux, ejecuta el tratamiento de textos (TWORD) incluido en MultiBase. No obstante, si se especifica la cláusula «-e» permite la edición de módulos con código fuente CTL. Por ejemplo: «tword -e fichero.sct».
wedit	Equivalente al comando tword, se encuentra disponible únicamente en las versiones de MultiBase para Windows. Permite la edición de módulos con código fuente CTL, aunque dispone de menos funciones que tword.

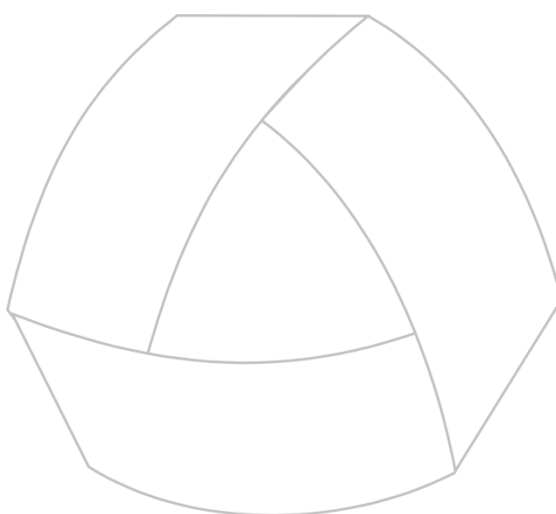
ctlcomp	Común a todos los sistemas operativos, es el compilador de módulos fuentes CTL, los cuales se identifican por tener la extensión «.sct», y cuyo resultado son los programas ejecutables (ficheros objeto con extensión «.oct»).
ctlink	Permite enlazar varios módulos con código objeto CTL (compilados con ctlcomp) para generar un programa (fichero con extensión «.lnk»). Asimismo, verifica las referencias cruzadas entre módulos y permite detectar llamadas a funciones no definidas si se utiliza aunque sea con un solo módulo. Este comando es común a todos los entornos operativos sobre los que funciona MultiBase (UNIX/Linux y Windows).
ctl	Ejecuta los programas compilados. Si en la compilación se generó información para depuración, se podrá optar entre su ejecución normal o bien arrancar en «modo depurador». Este comando puede arrancar la ejecución indicándole como nombre de programa el nombre sin extensión del fichero «.oct» o el del fichero de enlace «.lnk» (programa).

NOTA: Para más información acerca de estos comandos consulte el capítulo «Comandos MultiBase» en el Manual del Administrador.

Elementos auxiliares

Muchas instrucciones del CTL, particularmente las no procedurales, incluyen algún texto que se mostrará en pantalla (por ejemplo, al pedir o mostrar datos, posibles textos de ayuda sobre la aplicación, la opción en curso de un menú o las acciones asociadas al teclado). Todos estos textos se pueden incluir en los ficheros fuente CTL o, más adecuadamente, en los ficheros de mensajes (ficheros con extensión «.shl»). Los mensajes (textos) se referencian desde el programa mediante un número que se indica en el fichero de mensajes. Esto permite una mayor portabilidad de una aplicación, tanto para la modificación de mensajes como para su traducción a otro idioma. Para ser utilizable desde CTL, el fichero de mensajes debe ser compilado mediante el comando thelpcmp, cuya ejecución producirá un fichero objeto con extensión «.ohl». Este fichero de mensajes deberá estar en cualquiera de los directorios indicados en la variable de entorno MSGDIR. Para más información sobre esta variable de entorno y el comando de compilación del fichero de ayuda consulte los capítulos «Variables de Entorno» y «Comandos MultiBase», respectivamente, en el Manual del Administrador.

Asimismo, CTL permite controlar el teclado de forma que se ejecuten porciones de código (programas, funciones, menús, etc.) al pulsar una determinada tecla. Esto se realiza mediante un fichero, denominado «tactions», configurable por el usuario, donde se asocia el nombre de una función con el nombre de una tecla. Estas funciones entran en distintos objetos de CTL (ventanas, menús, FORMS, etc.) y permiten una mayor portabilidad de una aplicación respecto a distintos entornos hardware. Para más información acerca del fichero «tactions» consulte el capítulo sobre «Configuración de Terminales» en el Manual del Administrador.



MultiBase
MANUAL DEL PROGRAMADOR

CAPÍTULO 2. El entorno de desarrollo

TRANS: Entorno de desarrollo de MultiBase

El entorno de desarrollo de MultiBase (TRANS) constituye el interface natural de la herramienta con el programador. Se trata de un entorno de desarrollo de aplicaciones de muy alta productividad que ofrece al programador, en un único conjunto, todos los elementos necesarios para escribir su software.

TRANS engloba en un menú estructurado, de sencillo manejo, todas aquellas funciones que necesita el programador para desarrollar su aplicación de una forma extremadamente automatizada, sin necesidad de acceder a la línea de comando del sistema operativo.

La utilización de menús de tipo «pulldown» (de persianas), unido a los distintos niveles de ayuda permanentes que ofrece MultiBase, permiten al usuario controlar de manera exhaustiva las diferentes operaciones durante el desarrollo de una aplicación.

El entorno de desarrollo de MultiBase constituye una herramienta «Lower-CASE» para ayuda a la programación que incorpora prestaciones muy avanzadas, tales como mantenimiento de estructuras de bases de datos, desarrollo y mantenimiento de aplicaciones, generación de prototipos, generación automática de código y documentación automática. Al incluir toda la funcionalidad necesaria no requiere del programador ningún conocimiento del sistema operativo.

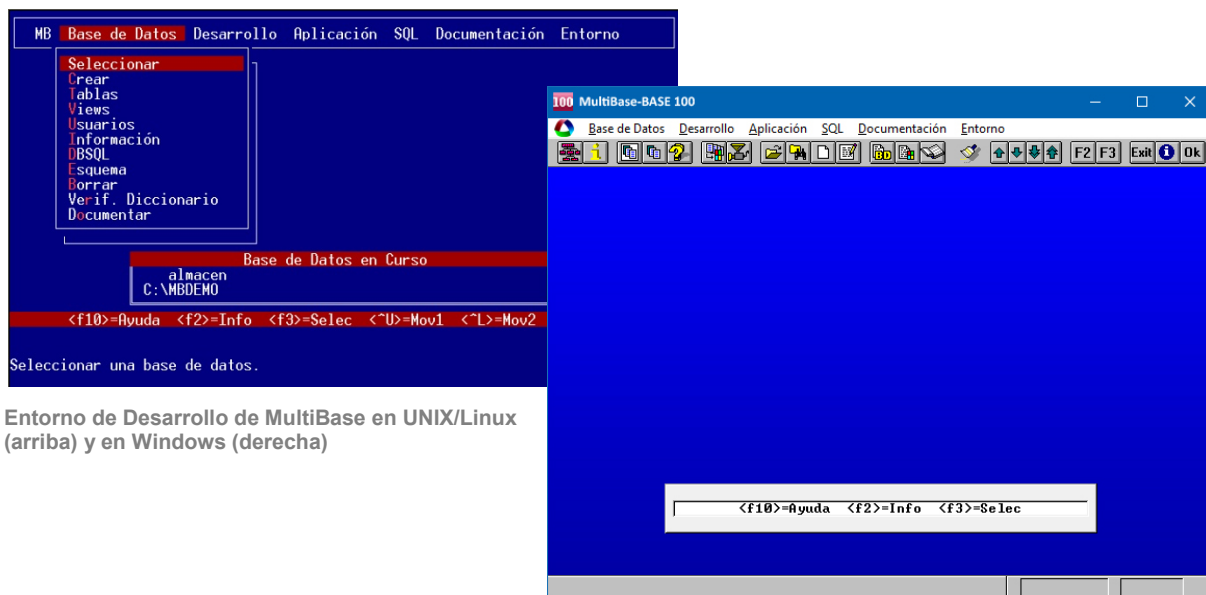
TRANS ofrece un aspecto prácticamente idéntico bajo cualquier plataforma o entorno operativo, si bien las diferencias existentes entre las versiones para UNIX/Linux por un lado, y Windows, por otro, vienen determinadas lógicamente por la utilización de los elementos propios de este último entorno, tales como ventanas de aspecto tridimensional, menús de iconos, empleo del ratón, etc.

Dependiendo del entorno operativo en que nos encontremos, la forma de acceder al entorno de desarrollo de MultiBase es la siguiente:

- En UNIX/Linux: Ejecutando el comando **trans** desde la línea de comando del sistema operativo.
- En Windows: Ejecutando el icono «Trans» del grupo «MultiBase» que se genera al instalar la aplicación.

Para cualquier duda sobre el procedimiento de instalación bajo cualquier entorno operativo consulte los capítulos 2 y 3 del Manual del Administrador).

El aspecto del entorno de desarrollo al arrancar MultiBase es el que se muestra en las siguientes figuras:



Entorno de Desarrollo de MultiBase en UNIX/Linux (arriba) y en Windows (derecha)

Como puede apreciarse, este entorno presenta por defecto un menú de tipo «pulldown» (más adelante se explica la forma de manejar tanto éste como otros tipos de menús utilizados por MultiBase), que incluye las siguientes opciones:

1.— «Base de Datos»

Las opciones incluidas dentro de esta persiana permiten definir la estructura de la base de datos:

«Seleccionar»	Permite elegir una base de datos existente.
«Crear»	Permite la creación de una base de datos.
«Tablas»	Procesos de mantenimiento de las tablas de la base de datos.
«Views»	Procesos de mantenimiento de las «views» de la base de datos.
«Usuarios»	Permite establecer los permisos de acceso a la base de datos a los distintos usuarios del sistema.
«Información»	Consulta de las definiciones de la base de datos.
«DBSQL»	Procesos de mantenimiento de la base de datos.
«Esquema»	Imprime información de la base datos.
«Borrar»	Borra una base de datos.
«Verif. Diccionario»	Verificación de los componentes del entorno de desarrollo.
«Documentar»	Documentación de la base de datos para la elaboración del Manual del Programador.

2.— «Desarrollo»

Esta persiana incluye todas las opciones referentes a la programación. Dichas opciones son las siguientes:

«Programas»	Mediante esta opción es posible definir, editar, compilar, enlazar y ejecutar los diferentes programas que componen una aplicación.
«Módulos/Generación»	Permite definir, editar, compilar, generar y ejecutar los diferentes módulos CTL que conforman la aplicación.
«Ayudas y Mensajes»	Permite definir, editar y compilar los módulos fuentes de mensajes y ayudas que serán utilizados por los módulos fuentes CTL de la aplicación.
«Información»	Permite obtener información sobre las definiciones realizadas de módulos, programas y mensajes.

3.— «Aplicación»

Las opciones incluidas en esta persiana realizan operaciones de carácter general que afectan a las definiciones sobre la base de datos. Dichas opciones son:

«Definir»	Permite definir los directorios de trabajo sobre la base de datos.
«Compilar»	Permite la compilación de los distintos componentes de la base de datos (módulos y programas CTL y módulos de ayuda).
«Generar Prototipo»	Permite la generación de módulos fuentes a partir de las definiciones de tablas de la base de datos.
«Verificar Diccionario»	Permite comprobar los distintos componentes de la aplicación.

«Modificar Diccionario»	Permite modificar los directorios para los componentes de una aplicación.
-------------------------	---

4.— «SQL»

Las opciones incluidas en esta persiana permiten crear, guardar, modificar y ejecutar cualquier instrucción del SQL. Todas las instrucciones del SQL deben incluirse en fichero ASCII con la extensión «.sql». Las opciones de esta persiana son:

«Elegir»	Permite seleccionar un fichero SQL previamente guardado en la base de datos.
«Ejecutar»	Permite ejecutar las instrucciones del fichero de trabajo SQL en curso.
«Nuevo»	Genera un nuevo fichero de trabajo SQL.
«Editor»	Permite modificar el fichero de trabajo SQL.
«Salida»	Permite enviar el resultado de la ejecución del fichero de trabajo SQL a una salida distinta de la pantalla.
«Guardar»	Guarda en un fichero el resultado de la ejecución del fichero de trabajo SQL.
«Borrar»	Elimina de la base de datos la definición de un fichero SQL.
«Insertar»	Inserta un fichero SQL dentro de la definición de la base de datos.
«Documentar»	Documenta el fichero SQL en curso.

5.— «Documentación»

MultiBase permite establecer dos niveles de documentación a la hora de confeccionar el manual de una aplicación:

a) Programador:

El Manual del Programador se genera a partir de las definiciones de la base de datos, así como de la estructura de los módulos y programas asociados a ella.

En el proceso de generación del manual hay que distinguir entre el texto constante, que es incorporado por el entorno de desarrollo, y el texto variable, que es el incluido por el usuario en las diferentes opciones de documentación incluidas en el propio entorno de desarrollo.

b) Usuario:

El Manual de Usuario de una aplicación se genera a partir de los módulos CTL, tanto fuentes como compilados, y teniendo en cuenta las llamadas que se realizan entre sí mediante las instrucciones CTL correspondientes.

En la generación automática hay que distinguir entre el texto constante, incorporado por el CTL, y el texto variable, incorporado por el propio usuario como ayudas y definido a través de los ficheros de ayuda (ficheros con extensión «.shl»).

Las opciones incluidas dentro de la persiana «Documentación» son:

«Base de Datos»	Permite documentar los diferentes elementos pertenecientes a la base de datos.
«Desarrollo»	Permite documentar los distintos elementos pertenecientes a la aplicación.
«Generar/Editar»	Permite la generación, modificación e impresión de la documentación generada automáticamente para los manuales del usuario y del programador.

6.— «Entorno»

El contenido de las diferentes variables de entorno se va a guardar junto a cada base de datos. Así, al ser seleccionar las variables de entorno asociadas éstas se definen automáticamente. Si no se ha seleccionado la base de datos, la modificación de cualquiera de los valores afectará únicamente al entorno que se esté ejecutando en ese momento, sin almacenar dicha modificación en la base de datos.

Las opciones de esta persiana son las siguientes:

«Directorios»	Permite definir las variables de entorno asociadas a directorios.
«Formatos»	Permite definir las variables de entorno asociadas a formatos.
«Varios»	Permite definir las restantes variables de entorno.
«Listar»	Permite consultar información de los valores ya definidos.

Más adelante en este mismo capítulo se comenta en detalle cada una de las persianas y sus respectivas opciones del entorno de desarrollo de MultiBase.

Manejo de un menú

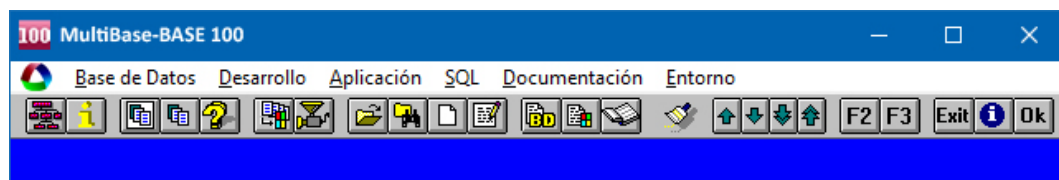
MultiBase utiliza tres tipos de menús diferentes:

- Menús «pulldown» o de «persiana».
- Menús «Pop-up».
- Menús «Lotus» (denominados de «iconos» en las versiones para Windows).

A continuación se explican las principales características de cada uno de ellos.

a) Menús «pulldown» o de «persiana»

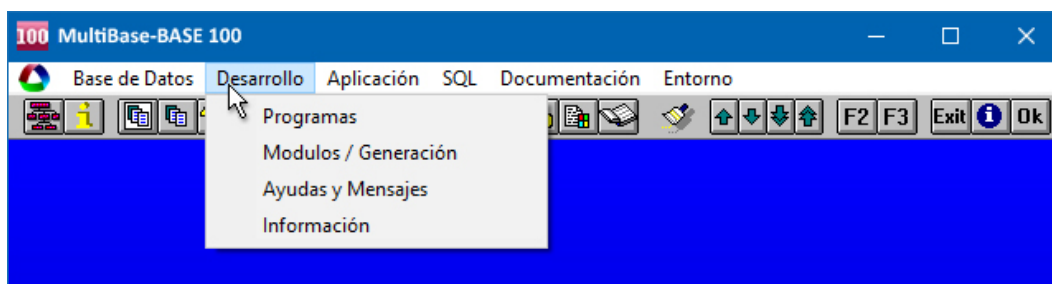
Este tipo de menú se caracteriza por presentar sus opciones dispuestas horizontalmente a lo largo de la parte superior de la pantalla. Por lo general, suelen ser el punto de entrada a una aplicación o a un módulo importante de la misma. El aspecto de estos menús es el que se muestra en la Figura 3.



Menú «pulldown» de MultiBase.

En las versiones de MultiBase para UNIX/Linux estos menús son cíclicos. Así, cuando un «pulldown» incluye más opciones (persianas) de las que caben en una pantalla, éstas se sitúan en pantallas sucesivas. Por su parte, en las versiones para Windows, aquellas opciones que no caben en el ancho de la pantalla se sitúan en líneas sucesivas inmediatamente debajo de la que contiene el primer nivel del menú.

La forma de activar las distintas persianas de un «pulldown» es mediante las teclas [FLECHA DERECHA] y [FLECHA IZQUIERDA]. Una vez situados sobre cualquiera de ellas, pulsando [FLECHA ABAJO], [FLECHA ARRIBA] o [RETURN] desplegaríamos la persiana correspondiente (ver figura), activando de esta forma el segundo nivel del «pulldown».



Segundo nivel de un menú «pulldown».

Otra forma de seleccionar una opción de un menú «pulldown» es pulsando directamente la letra correspondiente al carácter que figura en distinta intensidad de vídeo, como ocurre en UNIX/Linux, o subrayado, en caso del Windows, en combinación con la tecla [ALT]. Bajo este último entorno se podrá asimismo seleccionar las diferentes opciones mediante el empleo del ratón.

Aquellas opciones que figuren en una intensidad de vídeo atenuada no podrán ser activadas en ese momento.

b) Menús «Pop-up»

Por regla general, este tipo de menús son utilizados como submenús de un menú de nivel superior para seleccionar una opción entre varias.

A diferencia de los menús «pulldown» y «Lotus» (que veremos a continuación), los menús «Pop-up» se caracterizan por presentar sus opciones verticalmente.

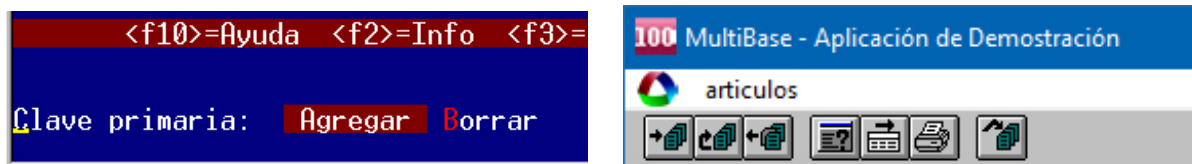


Menú «Pop-up».

La forma de seleccionar una opción en un menú «Pop-up» es idéntica a la del segundo nivel del «pulldown».

c) Menús «Lotus» o «de Iconos»

Los menús de tipo «Lotus», denominados de iconos en las versiones para Windows, se utilizan fundamentalmente como menús de uso de las pantallas o formularios de entrada de datos, y su forma de presentación difiere sustancialmente en función del entorno operativo en que nos encontremos (ver Figura 6). Así, mientras en las versiones para UNIX/Linux su presentación se realiza generalmente en la última línea al pie de la pantalla, en el caso del Windows estos menús se presentan en la parte superior (al igual que un «pulldown») y llevan asociados una serie de «botones» (iconos), correspondientes a las distintas opciones del menú.



Menú «Lotus» en UNIX/Linux (izquierda) y de iconos en Windows (derecha).

El manejo de este tipo de menús es similar a los anteriores.

Las teclas de acción

Siempre que esté utilizando MultiBase podrá utilizar distintas teclas de acción que le permitirán realizar diversas operaciones. Como norma general, le sugerimos que no trate de memorizar de entrada un gran número de teclas de acción, ya que las irá conociendo a medida que vaya manejando la aplicación. En cualquier caso, y siempre que lo necesite, podrá consultar las teclas asignadas a las diferentes acciones pulsando la tecla correspondiente a la acción de <ayuda>, que normalmente será la tecla [F10] de su teclado.

No obstante, existen tres acciones que, por su frecuencia de uso en el manejo de la aplicación, sí es conveniente conocer de antemano. Estas acciones son las siguientes:

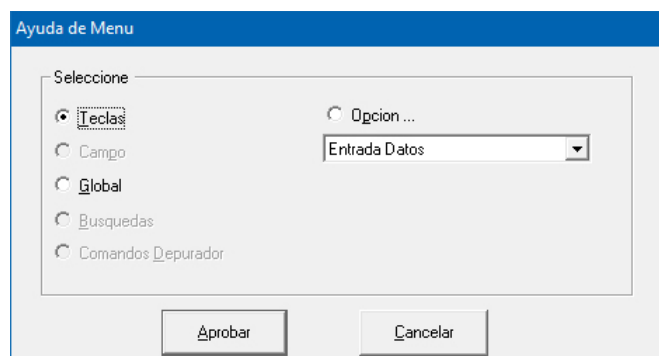
Aprobar, <faccept>	Permite validar el contenido de la pantalla. Normalmente esta acción está asignada a la tecla [F1].
Ayuda, <fhelp>	Como ya hemos comentado anteriormente, esta acción estará asignada por lo general a la tecla [F10] de su teclado y le proporcionará ayuda dependiendo del contexto en el que se encuentre.
Cancelar, <fquit>	Permite interrumpir cualquier operación que se haya iniciado y que, por la razón que sea, no se quiere continuar. Esta acción sirve también para abandonar un programa o un menú, etc. Normalmente, la tecla asignada a esta acción será [F2].

Estas teclas y otras que se mencionarán a lo largo de este capítulo podrán ser distintas de las citadas, dado que el teclado que utiliza MultiBase es programable por el propio usuario, quien podrá asignar libremente las teclas a aquellas acciones que desee del modo de que más le convenga.

La tecla de ayuda

Siempre que esté utilizando MultiBase dispondrá de una tecla de ayuda (por lo general la [F10] de su teclado), que le permitirá obtener información adicional sobre la operación a realizar en cualquier punto del programa. Dependiendo del lugar de la aplicación en que se encuentre esta tecla facilitará distintos tipos de ayuda.

Así, en la versión de MultiBase para Windows, al pulsar la tecla de ayuda aparecerá un elemento de diálogo como el que se muestra en la Figura 7.



Cuadro de diálogo de la opción de «Ayuda» bajo Windows.

En dicho elemento podrá seleccionar el tipo de ayuda deseado. Las opciones que podrá elegir para la obtención de la ayuda son las siguientes:

«Teclas»	Proporciona una descripción de las acciones que se ejecutarán al pulsar las teclas de acción. Dependiendo del punto del programa en que se haya pulsado la
----------	--

tecla de ayuda, esta opción mostrará únicamente las teclas de acción disponibles en ese momento..

- «Opción» Proporciona ayuda sobre la opción del menú en la que estuviese situado en el momento de pulsar la tecla de ayuda. Sólo funcionará si el programa dispone de texto de ayuda para esa opción.
- «Global» Facilita ayuda global sobre el programa que esté utilizando en el momento de pulsar la tecla de ayuda. Sólo funcionará si el programa dispone de un texto de ayuda para este caso.
- «Campo» Proporciona ayuda sobre el campo en el que estuviese situado el cursor en el momento de pulsar la tecla de ayuda. Sólo funcionará si existe texto de ayuda para ese campo.
- «Búsquedas» Esta opción sólo estará disponible cuando se estén realizando búsquedas sobre la base de datos a través de un formulario de entrada de datos. Su finalidad es explicar de qué forma se pueden realizar estas búsquedas.

Pantallas de entrada de datos

Una pantalla de entrada de datos está compuesta siempre por dos partes. La primera de ellas es un «formulario» sobre el que se deberán introducir los datos, mientras que la segunda es un menú de tipo «Lotus» o de iconos, según el entorno operativo en que nos encontremos, que incluirá las distintas opciones relativas a las diferentes operaciones que podremos llevar a cabo sobre dicho formulario. El aspecto típico de una pantalla de entrada de datos es el que se muestra en la siguiente figura. Como puede observarse, en ella se presenta un formulario para la introducción de datos junto a un menú de iconos compuesto por cinco «botones», cada uno de los cuales corresponde a una opción de la persiana «Cabecera»: «Agregar», «Modificar», «Borrar», «Cabece-
ras» y «Líneas».

MultiBase - Aplicación de Demostración

Cabecera

Edición de Albaranes

Albaran: [] Cliente: []
Fec. Albaran: [] Fec. Envio: []
Fec. Pago: [] Form. Pago: []

Total Albaran: [] Facturado: ☐

Lin	Arti	Prov.	Un.	Descripción	Des.	Precio	Total

Aspecto de una pantalla de entrada de datos en Windows.

Si el programador no hubiese diseñado un menú específico para el programa, éste tendría un menú por defecto («Lotus» o de iconos, según el entorno operativo en que nos encontrásemos) cuyas opciones serían las siguientes:

«Agregar»	Permite añadir nuevas filas (registros) a la tabla con la que esté trabajando el programa. Al seleccionar esta opción se entrará en edición de los campos del formulario. Una vez editados todos los campos deberá pulsar la tecla correspondiente a «aprobar» (que normalmente será la [F1]) para dar validez a los datos introducidos. Cuando desee abandonar el modo de edición de campos del formulario deberá pulsar la tecla de «cancelar» (normalmente [F2]).
«Modificar»	Permite realizar cambios sobre el registro mostrado en ese momento en el formulario. Una vez realizados los cambios, éstos deberán validarse mediante la tecla correspondiente a «aprobar» ([F1]).
«Borrar»	Permite eliminar de la base de datos el registro que tenga en pantalla en ese momento. Una vez seleccionada esta opción el programa pedirá conformidad al operador antes de proceder a la operación de borrado.
«Encontrar»	Permite realizar búsquedas condicionales sobre la tabla de la base de datos que esté manejando el formulario. Si pulsa la tecla correspondiente a «ayuda» (normalmente [F10]), el elemento de diálogo que aparece incluirá una opción que le indicará cómo introducir condiciones de búsqueda sobre los campos. Una vez indicadas las condiciones para el formulario en curso, éstas deberán aceptarse mediante la tecla de «aprobar» ([F1]) para que el programa comience la búsqueda. Al finalizar ésta, las teclas [FLECHA ARRIBA] y [FLECHA ABAJO] nos permitirán movernos a lo largo de la lista seleccionada, mostrándose en el formulario los distintos elementos de dicha lista. Asimismo, las teclas [INICIO] y [FIN] nos permitirán desplazarlos respectivamente al primero y al último elemento de la lista.
«Tabla»	En el caso de que existiese más de una tabla en el formulario, esta opción nos permitirá ir de una a otra. Al seleccionarla aparecerá un submenú que incluirá las opciones: «Siguiente», para acceder a la siguiente tabla; «Cabecera», para ir a la tabla de cabecera, y «Líneas», para ir a la tabla de líneas (estas dos últimas siempre que las tablas estén enlazadas por una relación del tipo «cabecera-líneas»).
«Salida»	Esta opción del menú le permitirá enviar a un fichero o a un programa la lista de fichas resultado de una selección realizada por medio de la opción «Encontrar». Sólo funcionará en el caso en que la lista contenga algún registro.
«Insertar»	Esta opción es similar a la de «Añadir», con la diferencia de que el registro que se añade queda insertado delante del registro que aparece en el formulario en el momento de ejecutar la opción.

Manejo de una ventana de consulta

En algunas ocasiones, al ejecutar determinadas acciones, MultiBase podrá mostrar una ventana de consulta (provocada por la instrucción WINDOW) en la que, según cada circunstancia concreta, deberá seleccionar o no un elemento entre varios. Las ventanas de consulta tienen el aspecto que se muestra en la siguiente figura. La forma de movernos a través de estas ventanas es mediante las teclas de control de movimiento del cursor. Una vez situados sobre la opción deseada, para seleccionarla bastará con pulsar [RETURN]. Cuando haya terminado la consulta pulse la tecla correspondiente a «cancelar» ([F2]) para cerrar la ventana.



Ventana de consulta.

Persiana «Base de Datos»

Esta persiana es la primera opción propiamente dicha del entorno de desarrollo de MultiBase, ya que la persiana «MB» es común a cualquier «pulldown» de la herramienta.

Las opciones incluidas dentro de esta persiana permiten definir la estructura de la base de datos. Dichas opciones son las siguientes:

- «Seleccionar».
- «Crear».
- «Tablas».
- «Views».
- «Usuarios».
- «Información».
- «DBSQL».
- «Esquema».
- «Borrar».
- «Verif. Diccionario».
- «Documentar».

En los siguientes apartados se comenta en detalle cada una de ellas.

Opción «Seleccionar»

Esta opción permite elegir una base de datos existente. Para poder realizar cualquier operación sobre una base de datos es necesario haberla seleccionado previamente. Si se intenta realizar cualquier acción que afectee a una base de datos sin tener ninguna activa, automáticamente aparecerá una ventana para solicitar el nombre de la base de datos a seleccionar.

La instrucción SQL que se ejecuta para seleccionar una base de datos es:

DATABASE base_datos

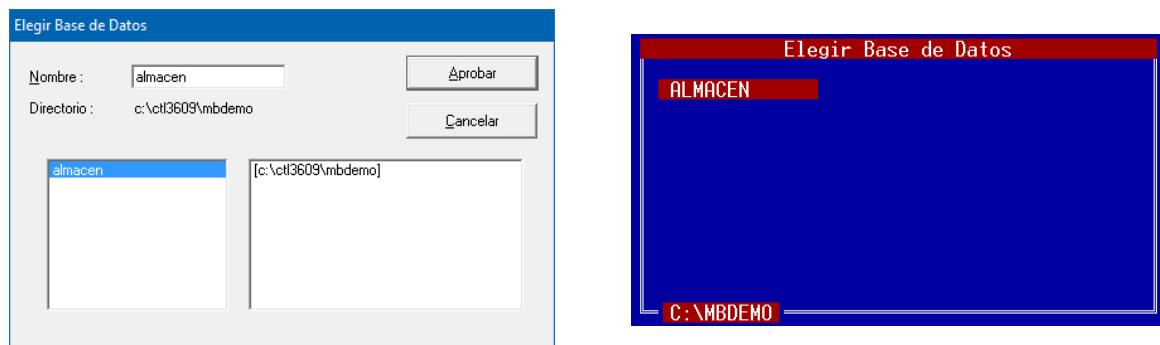
Siendo «base_datos» el nombre de la base de datos que se desea seleccionar.

Como puede comprobarse en la siguiente figura, el resultado por pantalla de la ejecución de esta opción varía sustancialmente dependiendo del entorno operativo en que nos encontremos:

- En UNIX/Linux: Al ejecutar la opción «Seleccionar» se mostrará una ventana en la que figurará el primer directorio de los definidos en la variable de entorno DBPATH, así como las bases de datos existentes en él. Si hubiese más directorios definidos en dicha variable de entorno, éstos se mostrar-

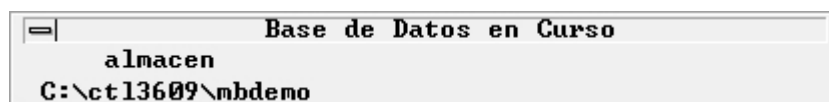
ían en páginas sucesivas de la misma ventana, a las que accederíamos mediante la tecla [AV-PÁG]. Si la variable DBPATH no estuviese definida se asumirá por defecto el directorio en curso.

- En Windows: Aparecerá una ventana, generada por la instrucción PATH WALK, conteniendo dos cajas: en la de la derecha se mostrarán los diferentes directorios definidos en la variable de entorno DBPATH y en la de la izquierda las bases de datos existentes dentro del directorio seleccionado en la ventana de la derecha. En este caso, si la variable de entorno DBPATH no estuviese definida se asumirá como directorio por defecto el indicado en el campo «Directorio de trabajo» del cuadro de diálogo de las propiedades del icono.
- En instalaciones con arquitectura cliente-servidor y «gateways» la ejecución de esta opción mostrará una ventana de edición (FRAME) para solicitar el nombre de la base de datos que se desea seleccionar.



Resultado por pantalla de ejecutar la opción «Seleccionar» de la persiana «Base de Datos» en Windows (izquierda) y UNIX/Linux (derecha).

Una vez seleccionada una base de datos aparecerá en pantalla una ventana indicando su nombre y su «path» correspondiente (ver siguiente figura). En instalaciones con arquitectura cliente-servidor, en lugar del «path» se mostrará información relativa al usuario, a la máquina «servidor» (UNIX/Linux) y al «gateway» que se está utilizando.



Ventana en pantalla tras la selección de una base de datos.

Si al seleccionar una base de datos aparece el mensaje «Verificar diccionario» en la parte inferior de la pantalla, es posible que existan inconsistencias entre la información contenida en las tablas del catálogo de la base de datos (tablas «sys*») y las del entorno de desarrollo (tablas «ep_*»). Para solventar estas inconsistencias habrá que ejecutar la opción «Verif. Diccionario» de la persiana «Base de Datos» del menú principal de MultiBase.

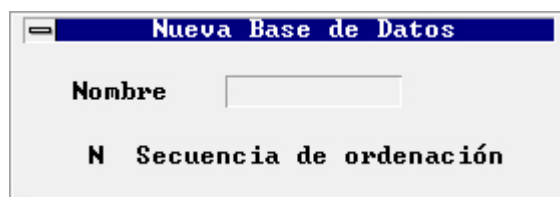
Opción «Crear»

Esta opción permite generar una base de datos nueva. Todos los elementos que componen una aplicación (datos, programas, etc.) se agrupan en torno al concepto de «base de datos».

La instrucción SQL que se ejecuta al activar esta opción es la siguiente:

```
CREATE DATABASE base_datos [WITH LOG IN "archivo de ordenación"]
```

Al ejecutar esta opción aparecerá en primer lugar una ventana en la que se mostrarán los diferentes directorios definidos en la variable de entorno DBPATH, debiendo elegir uno de ellos como destino para la base de datos que se va a crear. Si la variable DBPATH no estuviese definida se asumirá por defecto el directorio en curso en UNIX/Linux, mientras que en Windows se asumiría el directorio de trabajo indicado en el cuadro de diálogo de las propiedades del icono. Una vez seleccionado el directorio se mostrará otra ventana, cuyo aspecto (similar en UNIX/Linux respecto a Windows) es el que se muestra en la siguiente figura, para indicar el nombre de la nueva base de datos.



Ventana para indicar el nombre de la base de datos a crear.

La descripción de los campos que aparecen en esta ventana es la siguiente:

Nombre	Nombre de la base de datos que se va a crear. Este nombre deberá ser un literal alfanumérico de 18 caracteres de longitud como máximo y deberá cumplir las reglas de los identificadores de elementos SQL (campo de tipo carácter con longitud 10 que requiere la introducción de un valor).
--------	--

Secuencia de ordenación	La ordenación de los datos sigue el criterio de orden establecido en la tabla ASCII. Sin embargo, en algunos casos, este orden no se corresponde con el orden alfabético en nuestro idioma, como ocurre por ejemplo con las vocales acentuadas, la letra ñ, etc. En la creación de la base de datos podemos establecer los cambios de orden necesarios a través de un fichero de ordenación («collating sequence»), en el que se especifican estas diferencias. Mediante este campo podremos indicar si queremos utilizar o no esta característica.
-------------------------	---

Los valores posibles son los siguientes:

- | | |
|---|--|
| Y | Si se especifica este valor aparecerá una ventana de selección («TREE WALK») en la que se mostrarán los posibles ficheros de ordenación (ficheros con extensión «.ocs»). En el directorio «\$TRANSDIR/msg» se incluye un fichero de ejemplo para nuestro idioma:

Fichero fuente: «spanish.scs». Compilador de ficheros «.scs»: «tcoll-comp». Fichero compilado: «spanish.ocs».

Una vez seleccionado este fichero, al crear la base de datos se tendrán en cuenta las excepciones que en él se indican. |
| N | Éste es el valor por defecto. Permite el uso de la base de datos sin tener en cuenta las excepciones indicadas en el fichero de secuencia de ordenación. |

Una vez aceptados todos los valores indicados se crea la base de datos, la cual pasa a ser automáticamente la base de datos activa. El nombre de esta base de datos y su «path» se mostrarán en pantalla en una ventana como la expuesta en la Figura 11 de la opción anterior.

Al crear una base de datos se crea también paralelamente un directorio, de extensión «.dbs», en el que se incluirán una serie de ficheros, correspondientes tanto al catálogo de la base de datos (ficheros «sys*.dat» y «sys*.idx»), como al catálogo del entorno de desarrollo (ficheros «ep_*.idx» y «ep_*.dat»).

El catálogo de la base de datos está compuesto por un conjunto de tablas que, asociadas a una base de datos, contienen toda la información referida a ella.

NOTAS:

- 1.— En instalaciones con arquitectura cliente-servidor la base de datos se creará siempre en el primer directorio de los incluidos en la variable de entorno DBPATH, si es que ésta se ha definido. De no ser así, se creará en el directorio en curso.
- 2.— En instalaciones con «gateways» la variable de entorno DBAPTH no tiene efecto, asumiendo entonces la correspondiente al gestor de base de datos que se esté utilizando.

Opción «Tablas»

La estructura de una base de datos se conforma en torno a tres elementos: el catálogo de la base de datos, las tablas y las «views». Mediante esta opción podremos llevar a cabo todas las operaciones relativas a la definición de una tabla (creación, modificación, borrado, comprobación y reparación), así como a su mantenimiento (claves primarias, claves referenciales e índices).

Para ejecutar esta opción es necesario haber seleccionado previamente la base de datos. Una vez ejecutada se mostrará un submenú de tipo «Pop-up» (ver figura) que incluye las siguientes opciones:

«Definición»	Permite la definición de tablas (creación, modificación, borrado y consulta).
«Renombrar»	Permite cambiar de nombre a la tabla en curso.
«Primarias»	Permite la definición de la clave primaria para la tabla en curso.
«Claves»	Permite la definición de las claves referenciales para la tabla en curso.
«Índices»	Permite la definición de índices para la tabla en curso.
«Permisos»	Permite modificar los permisos de acceso a la tabla en curso.
«Chequear/Reparar»	Permite la comprobación y, en su caso, la reparación de los índices correspondientes a la tabla en curso.
«Borrar»	Borra la tabla en curso o alguno de sus elementos.
«Documentar»	Documenta la información contenida en la tabla en curso para la elaboración del correspondiente manual.



Menú «Pop-up» de la opción «Tablas».

Situados en este submenú se podrá seleccionar la tabla en curso pulsando la tecla asociada a la opción «Selec» que aparece en la línea de teclas de ayuda en la parte inferior de la pantalla (bajo entorno Windows estas teclas se muestran dentro de una ventana).

A continuación se comenta en detalle cada una de las opciones de este submenú.

«Definición»

Mediante esta opción se podrán llevar a cabo las siguientes operaciones:

- Crear tablas.
- Modificar o borrar atributos de columnas.
- Borrar y documentar columnas.

Las instrucciones SQL que se ejecutan al activar esta opción son CREATE TABLE y ALTER TABLE.

Al ejecutar la opción «Definición» aparecerá la siguiente ventana:

Ventana de introducción de datos para la creación de una tabla.

La descripción de los campos que aparecen en esta ventana es la siguiente:

- «Nombre de la tabla» En este campo se deberá indicar el nombre de la tabla de la base de datos que se va a crear o modificar. Si lo que se desea es crear una tabla, deberemos indicar un nombre que no coincida con el de ninguna tabla existente en la base de datos y que cumpla las reglas de los identificadores del SQL. Por el contrario, si deseásemos modificar una tabla ya existente podremos seleccionarla de dos maneras, bien eligiéndola antes de ejecutar la opción mediante la tecla asociada a la acción <Selecc> que se indica en la línea de teclas de ayuda en la parte inferior de la pantalla (en Windows estas teclas se muestran dentro de una ventana), o bien tecleando directamente su nombre en este campo, con lo cual los datos correspondientes a los demás campos de la ventana se cumplimentarían automáticamente (la tabla seleccionada se mostrará en la parte superior derecha de la pantalla).
- «Propietario» Nombre del propietario de la tabla (este nombre es asignado automáticamente).
- «Path» En este campo se deberá indicar el directorio donde se ubicará la tabla que se va a crear. Si no se especifica ningún directorio la tabla se creará en el mismo que la base de datos a la que pertenece. Cualquier otro valor ha de corresponder necesariamente con un directorio existente. Si lo que se desea es modificar una tabla ya existente, este campo aparecerá ya cumplimentado con el «path» correspondiente a la tabla seleccionada.

En cualquier caso, tanto si se trata de crear como de modificar una tabla, al ejecutar la opción «Definición» accederemos al menú «Tablas», que será de tipo «Lotus» en UNIX/Linux y de iconos en Windows.

El menú «Tablas» incluye dos opciones:

- «Sólo esquema» Esta opción permite realizar todas las operaciones relativas al mantenimiento de las columnas pertenecientes a la tabla en curso (creación, modificación, borrado, cambio de nombre etc.). Estas operaciones afectarán únicamente a los atributos generales e imprescindibles de la tabla, tales como tipo de dato, aceptación o no de valores nulos y etiqueta. Al ejecutar esta opción accederemos nuevamente a la ventana mostrada anteriormente para elegir una tabla. Una vez seleccionada ésta, pulsando la tecla correspondiente a «aceptar» (normalmente [F1]), aparecerá en pantalla el formulario que se muestra en la siguiente figura para establecer los atributos de la tabla, junto con un nuevo menú («Esquema»).

Columna	Tipo	Long/Prec	Req	Etiqueta
albaran	2 integer	4	☒	Num. Albaran
cliente	2 integer	4	☒	Cod. Cliente
fecha_albaran	7 date	4	☐	Fecha Albaran
fecha_envio	7 date	4	☐	Fecha Envio
fecha_pago	7 date	4	☐	Fecha Pago
formpago	0 char	2	☐	Cod. F. Pago
estado	0 char	1	☐	Facturado

Formulario para establecer el esquema de la tabla.

La descripción de los campos que aparecen en este formulario es la siguiente:

- «Columna» Nombre de una columna de la tabla. Este nombre deberá ser un identificador SQL único en la tabla donde se define.

«Tipo»

Número que identifica el tipo de dato de la columna donde se define. Sus posibles valores y sus tipos de datos correspondientes son:

VALOR	TIPO DE DATO	SE APLICA A
0	CHAR	Caracteres alfanuméricos
1	SMALLINT	Números enteros
2	INTEGER	Números enteros
3	TIME	Hora
5	DECIMAL	Números decimales
6	SERIAL	Contador automático
7	DATE	Fechas
8	MONEY	Números decimales

Los tipos de datos cuya longitud no puede ser modificada, y que aparecerá automáticamente después de su selección, son los siguientes:

TIPO DE DATO	LONGITUD
SMALLINT	2
INTEGER	4
TIME	4
SERIAL	4
DATE	4

Por su parte, los tipos de datos en los que habrá que indicar su longitud a la hora de definirlos son:

CHAR	Número de caracteres.
DECIMAL	Número de cifras significativas, enteras y decimales.
MONEY	Número de cifras significativas, enteras y decimales.

«Long/Prec»

Número de decimales que se van a utilizar para los tipos de datos DECIMAL y MONEY.

«Req»

En este campo podremos determinar si una columna acepta o no valores nulos. Sus posibles valores son:

Y	No acepta valores nulos.
N	Acepta valores nulos.

«Etiqueta»

Siempre que se ejecute una instrucción SQL de consulta en la que aparezca esta columna se utilizará la etiqueta definida como encabezamiento de los datos.

«Esquema y atributos» Esta opción permite realizar las diferentes operaciones de mantenimiento sobre las columnas de la tabla en curso (creación, modificación, borrado, cambio de nombre, etc.). A diferencia de la opción anterior, ésta afecta a todos los atributos que se pueden aplicar sobre una columna. Al ejecutar esta opción aparecerá la ventana de edición mostrada anteriormente para seleccionar la tabla. Una vez seleccionada ésta, pulsando la tecla correspondiente a «aceptar» (normalmente [F1]), aparecerá en pantalla el formulario que se muestra en la siguiente

figura para establecer las características de las columnas de la tabla en curso, junto con un nuevo menú («Atributos»).

Formulario para definir características de columnas.

La descripción de cada uno de los campos que aparecen en esta pantalla es la siguiente:

- «Tabla» Nombre de la tabla en curso a la que pertenece la columna cuyas características se van establecer.
- «Columna» Nombre de la columna. Este nombre deberá ser un identificador SQL único en la tabla donde se define.
- «Trae atr.» Mediante este campo podremos definir los atributos de la columna importándolos de otra ya existente. Sus posibles valores son:
- Y Presenta una ventana para elegir una tabla y a continuación otra para elegir una columna de esa tabla. Una vez seleccionada la columna el formulario se rellenará automáticamente con los atributos de la columna elegida.
 - N No se importarán atributos de otra columna (éste es el valor por defecto).
- «Ed. descrip.» Mediante este campo se podrán introducir unas líneas descriptivas de la columna en curso. Estas líneas se utilizarán posteriormente al confeccionar el Manual del Programador con el documentador automático. Sus posibles valores son:
- Y Presenta una ventana con el texto descriptivo asociado a la columna (si lo hubiese), pudiendo modificarlo o agregar más texto. Una vez finalizada la edición, pulsando la tecla asociada a «aceptar» aparecerá un menú de confirmación para dar validez o no a los cambios realizados.
 - N No se documenta la columna en curso (valor por defecto).
- «Tipo» Número que identifica el tipo de dato de la columna que se está definiendo. Sus posibles valores son:

VALOR	TIPO DE DATO	SE APLICA A
0	CHAR	Caracteres alfanuméricos

VALOR	TIPO DE DATO	SE APLICA A
1	SMALLINT	Números enteros
2	INTEGER	Números enteros
3	TIME	Hora
5	DECIMAL	Números decimales
6	SERIAL	Contador automático
7	DATE	Fechas
8	MONEY	Números decimales

Este campo incluye dos ventanas, una para el número del tipo de dato y otra donde se mostrará su nombre.

«Longitud»

Longitud de ocupación o definición del tipo de dato elegido en el campo anterior. Los tipos de datos cuya longitud no puede ser modificada, y que aparecerá automáticamente después de su selección son los siguientes:

TIPO DE DATO	LONGITUD
SMALLINT	2
INTEGER	4
TIME	4
SERIAL	4
DATE	4

Por su parte, los tipos de datos en los que habrá que indicar su longitud a la hora de definirlos son:

CHAR Número de caracteres.

DECIMAL Número de cifras significativas, enteras y decimales.

MONEY Número de cifras significativas, enteras y decimales.

«Precisión»

Número de decimales a utilizar para los tipos de datos DECIMAL y MONEY.

«Etiqueta»

Siempre que se ejecute una instrucción SQL de consulta en la que aparezca esta columna se utilizará la etiqueta definida como encabezamiento de los datos.

«Requerido»

Mediante este atributo podremos determinar si una columna acepta o no valores nulos. Sus posibles valores son:

Y No acepta valores nulos.

N Acepta valores nulos.

«No agregar»

En este campo indicaremos si la columna podrá incluirse o no en la sintaxis de una instrucción INSERT. Sus posibles valores son:

Y No podrá incluirse en la sintaxis de la instrucción INSERT.

N Sí podrá incluirse (valor por defecto).

«No actualizar»	En este campo indicaremos si la columna podrá incluirse o no en la sintaxis de una instrucción UPDATE. Sus posibles valores son: Y No podrá incluirse en la sintaxis de la instrucción UPDATE. N Sí podrá incluirse (valor por defecto).
«May./Min.»	En este campo indicaremos si una columna de tipo CHAR admitirá únicamente mayúsculas, minúsculas o ambas indistintamente. Sus posibles valores son: U Sólo admitirá mayúsculas. L Sólo admitirá minúsculas. N Admitirá mayúsculas y minúsculas (valor por defecto).
«Alineación»	En este campo indicaremos la alineación de la columna. Sus posibles valores son: R Alineación a la derecha. L Alineación a la izquierda. N Valor por defecto. Los tipos de datos numéricos (SMALLINT, INTEGER, DECIMAL y MONEY) se alinearán a la derecha.
«Llenar a ceros»	Permite indicar si en la edición de las columnas (CHAR o numéricas) la alineación será a la derecha y rellenando con ceros a la izquierda. Sus valores son: Y Rellena con ceros por la izquierda y alinea a la derecha. N No rellena ni alinea.
«formato»	En este campo podremos indicar si se va aplicar una máscara a las columnas de tipo CHAR a la hora de su edición y presentación en pantalla, o bien un formato determinado en los demás tipos de datos (para más información consulte los atributos FORMAT y PICTURE en el Manual del SQL).
«V.defec.»	En este campo podremos indicar el valor por defecto que se utilizará en caso de que la columna no participe en la instrucción INSERT. Este valor por defecto será utilizado igualmente en los módulos de Entrada de Datos que se generen. Los valores que podrán especificarse son constantes y variables de SQL («today», para tipos DATE, y «now», para tipos TIME). Las constantes de tipo CHAR, TIME y DATE irán entre comillas.
«Condición»	Mediante este campo podremos indicar una condición válida de SQL que se deberá cumplir en cualquiera de las operaciones de INSERT o UPDATE, cuyo resultado deberá ser «True». El nombre de la columna en la condición puede ser sustituido por el carácter «\$». Por ejemplo: \$ in ("A", "B", "C") \$ between 1 and 10 \$ is not null and \$ >= 0
«Ret. auto.»	Mediante este campo podremos alterar la forma de edición por defecto en los módulos de entrada de datos, de forma que al editar la columna no sea necesario pulsar [RETURN] para va-

	lidarla, sino que al rellenar la zona de edición se produzca automáticamente el salto al siguiente campo de edición. Sus posibles valores son: Y Salta al siguiente campo en edición sin necesidad de pulsar [RETURN]. N Se deberá pulsar [RETURN] para aceptar el valor del campo (valor por defecto).
«Atr. vídeo»	Mediante este campo podremos alterar la forma de edición y presentación en pantalla en los módulos de entrada de datos generados, de forma que el atributo empleado por defecto (UNDERLINE, subrayado) pueda ser sustituido por REVERSE (vídeo inverso). Sus posibles valores son: R Vídeo inverso. U Subrayado. N Valor por defecto (subrayado).
«Atr. título»	Mediante este campo podremos modificar la presentación por pantalla de los literales (etiquetas) que acompañarán a la columna en los módulos de entrada de datos que se generen. Sus posibles valores son: R Vídeo inverso. U Subrayado. N Valor por defecto (subrayado).
«Borr.auto.»	Permite indicar si se limpiará o no el campo de cruce al comenzar la edición de las variables con la instrucción QUERY del FORM. Sus valores son: Y Limpia el campo de cruce en QUERY. N No lo limpia (valor por defecto).
«Verificar»	Mediante este campo podremos indicar si se repetirá o no la edición de la columna, de forma que el valor introducido la segunda vez coincida con el editado en primer lugar. En caso contrario emitirá un mensaje de error y continuará la edición. Sus posibles valores son: Y Repite la edición del campo. N No repite (valor por defecto).
«Núm. mensaje»	En este campo podremos indicar un número de mensaje de ayuda. De este modo, una vez construido el fichero de ayudas y mensajes, éstos se utilizarán desde los módulos de entrada de datos a través del atributo HELP generado en aquellos. Sus posibles valores son: 0 No utiliza número de mensaje (valor por defecto). N Número entre -32.767 y 32.767.
«Ayuda»	En este campo se podrá indicar una línea de texto descriptiva de la columna, de forma que durante la edición de ésta dicho texto aparezca en la línea de mensajes, facilitando así su edición. Su valor por defecto es en blanco.

- «Cruce: Tabla» En este campo podremos indicar el nombre de una tabla existente para ver sus datos en pantalla a partir del cruce indicado en la secuencia de opciones «Tabla» — «Columna» — «Columnas a mostrar». La tabla de enlace se puede seleccionar pulsando la tecla asociada a la acción <Selec> que aparece en la línea de teclas de ayuda en la parte inferior de la pantalla. Su valor por defecto es en blanco.
- «Cruce: Columna» En este campo podremos indicar el nombre de una columna perteneciente a la tabla especificada en el campo anterior con la que se realizará el cruce. La columna de enlace se puede seleccionar pulsando la tecla asociada a la acción <Selec> que aparece en la línea de teclas de ayuda en la parte inferior de la pantalla. Su valor por defecto es en blanco.
- «Columnas a mostrar» En este campo se podrán indicar los nombres de las columnas de la tabla seleccionada que se mostrarán en pantalla. Dichos nombres deberán ir separados por comas. Las columnas a mostrar se podrán seleccionar mediante la tecla asociada a la acción <Selec> que aparece en la línea de teclas de ayuda en la parte inferior de la pantalla. Su valor por defecto es en blanco.

Las opciones «Sólo esquema» y «Esquema y atributos» presentan unos menús (de tipo «Lotus» en UNIX/Linux y de iconos en Windows), denominados «Esquema» y «Atributos», respectivamente, cuyas opciones, comunes a ambos, son las siguientes:

- «Agregar» Esta opción permite añadir una columna a la tabla en curso. Una vez seleccionada comenzará el proceso de edición de campos. Cumplimentados éstos y aceptados sus valores mediante la tecla «aceptar», se ejecutará la instrucción de SQL correspondiente para agregar la columna a la tabla en curso. En el caso de que la tabla a la que se va a agregar la columna no hubiese sido creada previamente, habrá que ejecutar la opción «Aprobar» para hacer efectiva la generación de la misma.
- «Modificar» Mediante esta opción se podrán modificar los valores existentes en los campos de la columna en curso. Si se modifica el nombre de la columna aparecerá un menú para solicitar conformidad del nuevo nombre asignado. Una vez seleccionada esta opción comenzará el proceso de edición de campos. Cumplimentados éstos y aceptados sus valores mediante la tecla «aceptar», se ejecutará la instrucción de SQL correspondiente para modificar los campos de la columna en curso. Si la tabla cuya columna se desea modificar no hubiese sido creada previamente, habrá que ejecutar la opción «Aprobar» para hacer efectiva la creación de la misma.
- «Borrar» Esta opción permite eliminar la columna en curso. En caso de que existiesen varias columnas podremos acceder a ellas mediante las teclas [FLECHA ARRIBA] y [FLECHA ABAJO], siendo la columna en curso aquella en la que el cursor aparece en la parte izquierda de la ventana. Si el número de columnas es superior a nueve, podremos emplear también las teclas [AV-PÁG] y [RE-PÁG]. Una vez seleccionada esta opción aparecerá un menú para pedir conformidad al usuario antes de proceder al borrado de la columna.

«Encontrar»	Mediante esta opción podremos consultar las columnas existentes en la tabla en curso. En caso de que existiesen varias columnas podremos acceder a ellas mediante las teclas [FLECHA ARRIBA] y [FLECHA ABAJO], siendo la columna en curso aquella en la que el cursor aparezca en la parte izquierda de la ventana. Si el número de columnas existentes es superior a nueve podremos emplear también las teclas [AV-PÁG] y [RE-PÁG].
«Documentar»	El texto incluido en la documentación de esta opción aparecerá en el fichero «idtabla.dta» como parte de la información de la tabla a la hora de generar el Manual del Programador. Una vez seleccionada aparecerá una ventana de edición con varias líneas para incluir el texto relativo a la documentación de la columna en curso.
«Aprobar»	Esta opción sólo estará disponible si la columna que se genera pertenece a una tabla que no existe. La ejecución de esta opción es necesaria para hacer efectivas todas las definiciones realizadas sobre las columnas. Si la tabla ya estuviese creada, esta opción no será accesible, ya que todas las acciones posibles (agregar, modificar o borrar) se aplican automáticamente sobre ella. En caso de cancelar el menú con alguna definición ya realizada, aparecerá un nuevo menú para confirmar si se procede o no a la creación de la tabla, que se realizará una vez seleccionada esta opción.

«Renombrar»

Esta opción permite cambiar el nombre (identificador) de la tabla en curso. Si ésta no se ha seleccionado antes de ejecutar la opción se mostrará una ventana de selección para elegir la tabla que se desea renombrar.

La instrucción SQL que se ejecuta es RENAME TABLE.

Al activar la opción «Renombrar» aparecerá una ventana de edición para indicar el nuevo nombre que se desea asignar a la tabla en curso.

«Primarias»

Una vez definidas todas las tablas de la base de datos, el siguiente paso a realizar será la definición de la clave primaria para cada una de ellas. La clave primaria, junto a las claves referenciales, es uno de los componentes de la integridad referencial.

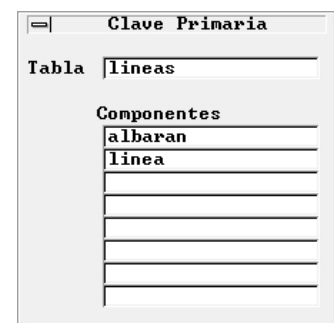
Una clave primaria es un conjunto de columnas, ninguna de las cuales ha de admitir valores nulos (NOT NULL), que va a identificar de forma unívoca cada una de las filas de la tabla. Una tabla admite únicamente una clave primaria.

La definición de una clave primaria lleva implícita la creación de un índice único para cada columna componente de dicha clave, siendo 8 el número máximo de columnas.

Si no se ha seleccionado ninguna tabla, al ejecutar esta opción aparecerá una ventana de selección para elegir la tabla a la que se desea definir la clave primaria.

Una vez seleccionada la tabla aparecerá una ventana (ver figura) donde se mostrarán los nombres de la tabla en curso y de las columnas componentes de la clave primaria (máximo 8), así como un menú que, bajo el título «Clave Primaria», incluirá las siguientes opciones:

«Agregar»	Permite la definición de la clave primaria para la tabla en curso. Esta opción sólo podrá ejecutarse si la tabla en curso no tiene definida la clave primaria.
-----------	--



Ventana para definición o borrado de la clave primaria.

«Borrar» Permite eliminar la clave primaria de la tabla en curso.

Pulsando la tecla asignada a la opción <Selec> en cualquiera de los ocho campos correspondientes a las columnas se mostrarán todas aquellas que podrán ser seleccionadas.

«Claves»

Mediante esta opción podremos definir las claves referenciales para la tabla en curso. Como ya hemos comentado anteriormente, las claves referenciales son, junto con la clave primaria, uno de los componentes de la integridad referencial. La definición de claves referenciales es el siguiente paso a realizar una vez definida la clave primaria. Mediante la definición de claves referenciales se establecen relaciones de integridad entre las tablas, ya que de lo contrario sólo existiría una relación «lógica».

Para definir una clave referencial a una tabla es necesario haber seleccionado ésta previamente, de no ser así, al ejecutar esta opción aparecerá una ventana de selección para elegir la tabla a la que se desean definir las claves referenciales.

Tabla	lines
Referencia a	albaranes
Nombre	For2_lin
ON DELETE	R restrict
ON UPDATE	R restrict
Componentes	albaran

Ventana para definición o borrado de claves referenciales.

Una vez seleccionada la tabla aparecerá una ventana (ver figura) junto con un menú que, bajo el título «Claves Referenciales», incluye las siguientes opciones:

«Agregar» Permite definir una clave referencial para la tabla en curso.
 «Borrar» Permite eliminar la clave referencial en curso de la tabla activa.
 «Encontrar» Selecciona todas las claves referenciales de la tabla en curso.

Si existiesen varias claves referenciales se podrá seleccionar una de ellas desplazándose con las teclas de movimiento del cursor ([FLECHA ARRIBA] y [FLECHA ABAJO]). La clave referencial seleccionada será aquella cuyos datos aparezcan en la ventana.

La descripción de cada uno de los campos que aparecen en la ventana es la siguiente:

«Tabla» Nombre de la tabla en curso.

«Referencia a» Nombre de la tabla a la que hará referencia la tabla en curso. Este nombre deberá ser el de una tabla existente en la base de datos, la cual deberá tener definida además la clave primaria, ya que ésta es la clave por la cual se establece la relación.

«Nombre» Identificador único de SQL que indica el nombre de la relación. Su definición es obligatoria.

«ON DELETE» Indica la acción que se deberá ejecutar en caso de borrar filas de la tabla referenciada. Sus posibles valores son los siguientes:

R (RESTRICT) Impide el borrado de filas de la tabla referenciada en el caso de que exista alguna fila con los mismos valores dentro de la tabla en curso.

N (SET NULL) Permite el borrado de las filas de la tabla referenciada y modifica las columnas de relación con el valor «NULL» en todas aquellas filas que tengan los mismos valores dentro de la tabla en curso.

«ON UPDATE»	Indica la acción que se deberá ejecutar en caso de modificar el contenido de las columnas que forman la clave primaria de la tabla referenciada. Sus posibles valores son los siguientes:
R (RESTRICT)	Impide la modificación de las filas de la tabla referenciada en el caso de que exista alguna fila con los mismos valores sobre la tabla en curso.
N (SET NULL)	Permite la modificación de las filas de la tabla referenciada, modificando a su vez las columnas de relación con el valor «NULL» en todas aquellas filas que tengan los mismos valores dentro de la tabla en curso.
«Componentes»	Columnas que formarán la relación entre la tabla en curso y la clave primaria de la tabla referenciada (máximo 8). Para seleccionar una columna de la tabla en curso podremos utilizar la tecla asociada a la opción <Selec>.

«Índices»

La creación de índices para una tabla permite mejorar el tiempo de acceso a los datos. Un índice podrá ser simple o compuesto (según esté formado por una sola columna o por varias —máximo 8—) y único o duplicado (dependiendo respectivamente de que no admita valores duplicados o de que sí lo haga).

Para crear un índice de una tabla es preciso seleccionar ésta previamente. De no ser así, al ejecutar esta opción aparecerá una ventana para seleccionarla. Una vez seleccionada la tabla aparecerá una ventana de mantenimiento de índices (ver figura) junto con un menú que, bajo el título «Índices», incluye las siguientes opciones:

Columna	Orden <A/D>
1	A
2	A
3	A
4	A
5	A
6	A
7	A
8	A

Ventana de mantenimiento de índices.

«Agregar»	Permite la definición de un índice sobre la tabla en curso.
«Encontrar»	Selecciona todos los índices de la tabla en curso.
«Borrar»	Borra el índice en curso de la tabla activa.
«Documentar»	Documenta el índice en curso de la tabla activa.

Si existiesen varios índices definidos podremos seleccionar aquel que nos interese desplazándonos con las teclas de movimiento del cursor ([FLECHA ARRIBA] y [FLECHA ABAJO]). El índice seleccionado será aquel cuyos datos figuren en la ventana de información.

La instrucción SQL que se ejecuta internamente es al seleccionar la opción «Índices» es CREATE INDEX.

La descripción de cada uno de los campos que aparecen en la ventana es la siguiente:

«Tabla»	Nombre de la tabla en curso.
«Nombre Índice»	Nombre de un índice perteneciente a la tabla activa. Dicho nombre deberá ser un identificador SQL único que no haya sido utilizado anteriormente para la definición de otro índice.
«Propietario»	Este campo es rellenado automáticamente.
«Duplicado»	La clasificación de las columnas componentes del índice se establece en función de que aquéllas admitan valores duplicados («Y») o no («N») para formar respectivamente un índice duplicado o único.

«Columna»	Nombres de las columnas componentes del índice (máximo 8). Pulsando la tecla correspondiente a la opción <Selecc> podremos seleccionar el nombre del índice entre las diferentes columnas de la tabla en curso.
«Orden <A/D>»	Tipo de ordenación («A»scendente o «D»escendente) para las columnas del índice.

«Permisos»

Esta opción permite conceder y revocar permisos de acceso a la tabla en curso o a cualquiera de sus columnas, así como consultar los permisos ya definidos.

Para ejecutar esta opción se deberá seleccionar previamente una tabla. De no ser así se mostrará una ventana para seleccionarla. Una vez realizada esta operación aparecerá un menú de tipo «Pop-up» con las siguientes opciones:

«Conceder»

Esta opción permite la concesión de permisos de acceso sobre la tabla en curso o sobre cualquiera de sus columnas a diferentes usuarios. La instrucción de SQL que se ejecuta internamente es GRANT.

Al ejecutar esta instrucción aparecerá la ventana que se muestra en la Figura 20 junto con un menú («Lotus» en UNIX/Linux y de iconos en Windows) que, bajo el título «Definir», incluye las siguientes opciones:

«Lista de usuarios»	Permite indicar los usuarios a los que se va a conceder permisos.
«Máscara de permisos»	Permite introducir la máscara de permisos a otorgar.
«Columnas»	Permite introducir las columnas afectadas.
«Ejecutar»	Lleva a cabo los cambios especificados ejecutando la correspondiente instrucción SQL.

Ventana para concesión de permisos.

La ventana que se muestra está dividida en tres partes, una para indicar los usuarios a los que se van a conceder permisos, otra las columnas afectadas (que por defecto serán las de la tabla activa) y la tercera para establecer la máscara de permisos.

Los campos de esta ventana son los siguientes:

«Usuarios»	Nombre de un usuario de la base de datos. Si el nombre indicado no estuviese definido como tal se le asignará automáticamente el permiso de «CONNECT», que se mostrará en el campo «Tipo».
«Columnas»	
«Nombre»	Nombre de una columna de la tabla en curso. Para seleccionar una columna se podrá emplear la tecla asignada a la opción

<Selecc> que se muestra en la ventana de teclas de ayuda en la parte inferior de la pantalla.

«Select»/«Update» Deberá indicarse «Y» o «N» para establecer o no respectivamente permisos de SELECT y UPDATE sobre la columna.

«Esquema de permisos»:

«ALL» Sus posibles valores son «Y», que indicará que se van a conceder permisos para todas las instrucciones que figuran en los campos siguientes, o «N», que permite definir permisos individuales para cada una de las instrucciones (éste es el valor por defecto). Si se indica «Y» no será necesario cumplimentar los restantes campos.

En los restantes campos («Select», «Update», «Insert», «Delete», «Index», «Alter» y «GRANT») se podrá indicar «Y», para conceder permisos sobre las instrucciones SELECT, UPDATE, INSERT, DELETE, CREATE INDEX, ALTER TABLE y GRANT, o bien «N» para no modificarlos, siendo este último el valor por defecto.

«Retirar»

Mediante esta opción se podrán cancelar los permisos de acceso sobre la tabla en curso o sobre cualquiera de sus columnas a diferentes usuarios. La instrucción de SQL que se ejecuta internamente es REVOKE.

Al ejecutar esta instrucción aparecerá una ventana con el mismo formato que la mostrada en la figura anterior (que en este caso llevará por título «Retirar permisos») junto con un menú que, bajo el título «Definir», incluye las siguientes opciones:

«Lista de usuarios» Permite indicar la lista de usuarios a los que se va a revocar sus permisos.

«Máscara de permisos» Permite introducir la máscara de permisos a retirar.

«Columnas» Permite introducir las columnas afectadas.

«Ejecutar» Lleva a cabo los cambios especificados ejecutando la correspondiente instrucción SQL.

La descripción de los campos de la ventana es similar a la de la opción «Conceder», si bien en este caso los valores que se indiquen estarán referidos a la cancelación de los permisos.

«Información»

Esta opción permite consultar por pantalla información sobre los permisos definidos a cada usuario dentro de la tabla en curso. El aspecto de la ventana que aparece al ejecutar esta instrucción es el que se muestra en la figura.

Información sobre Privilegios								
Tabla	albaranes							
Concedido por	Usuario	Select	Update	Insert	Delete	Index	Alter	Conceder
system	public	Y	Y	Y	Y	Y	N	N

Aspecto de la ventana que aparece al ejecutar la opción «Información».

Los valores posibles que puede tener cada instrucción son:

- Y Indica que el usuario tiene permiso para ejecutar la instrucción. N El usuario no tiene permiso de ejecución.
- C En las instrucciones SELECT y UPDATE hace referencia a algunas columnas de la tabla y no a ésta en su conjunto. Pulsando la tecla que se indica al pie de la pantalla podremos ver a qué columnas se refiere.

«Chequear/Reparar»

Mediante esta opción es posible comprobar, y en su caso reparar, la consistencia de la información contenida en los índices de una tabla. Las operaciones de chequeo y reparación de índices se efectúan mediante los comandos del CTL tchkidx y trepidx, respectivamente, cuya ejecución se encuentra implícita en esta opción.

Una vez seleccionada esta opción aparecerá la ventana de selección múltiple (FORM) en la que se podrá marcar una varias tablas mediante la tecla [RETURN]. En esta ventana figurarán todas las tablas existentes en la base de datos. Para definir un conjunto determinado de ellas podremos emplear la tecla asociada a la acción <Seleccionar> que se indica al pie de la ventana y utilizar los metacaracteres del «query». Si se desea seleccionar todas las tablas se podrá hacer directamente pulsando la tecla asociada a la acción <Marcar Todos>.

Una vez seleccionada la tabla o tablas deseadas, pulsando la tecla correspondiente a «aceptar» accederemos a un menú «Pop-up» con las siguientes opciones:

- «Chequear tabla(s)» Esta opción permite revisar las tablas marcadas en la ventana anterior. El resultado de la comprobación podrá ser consultado a través de la opción «Listar monitorización» de este mismo menú. Una vez finalizado el proceso de comprobación aparecerá un mensaje indicando que se pulse una tecla para volver al menú anterior.
- «Reparar índices» Mediante esta opción se podrán reparar los índices de las tablas marcadas en la ventana anterior. El resultado de esta operación podrá ser consultado a través de la opción «Listar monitorización» de este mismo menú.

Una vez finalizado el proceso de reparación aparecerá un mensaje indicando que se pulse una tecla para volver al menú anterior.
- «Listar monitorización» MultiBase lleva un control de las incidencias que se producen sobre la base de datos, con el fin de que el administrador de la misma pueda consultarlas cuando así lo desee. Las incidencias que se pueden producir sobre una base de datos vienen determinadas por los siguientes aspectos:
 - Los intentos de acceso a la base de datos cuando ésta se encuentra en estado defectuoso.
 - Las tablas que han sido detectadas con índices defectuosos.
 - Los chequeos y reparaciones realizados sobre las tablas.

Toda esta información podrá ser consultada a través de esta opción. Para ello, una vez seleccionada accederemos a un menú (de tipo «Lotus» en UNIX/Linux y de iconos en Windows), con las siguientes opciones:

- «Accesos» Esta opción facilita información en una ventana sobre los intentos de acceso a la base de datos estando ésta defectuosa. Cualquier módulo CTL que trabaje con tablas de la base de datos y detecte que alguna de ellas tiene índices defectuosos, avisará al operador sobre esta anomalía al objeto de que no

prosiga en su trabajo y proceda al chequeo y, en su caso, a la reparación de la misma. Siempre que se produce una incidencia de este tipo, el sistema deja constancia indicando el nombre del usuario, la fecha y la hora en que se produjo, con el fin de que el administrador de la base de datos pueda consultarla posteriormente.

«Errores» Esta opción facilita información sobre los errores detectados por el comando de comprobación tchkidx, de forma que después de reparar una tabla que contenga errores, éstos serán suprimidos, quedando únicamente aquellas tablas sin de errores. Siempre que se produce una incidencia de este tipo, el sistema deja constancia indicando el nombre del usuario, la fecha, la hora y el programa en que se produjo, con el fin de que el administrador de la base de datos pueda consultarla posteriormente.

«Regeneraciones» Cada vez que se efectúa una comprobación o reparación sobre el estado de una tabla, el sistema deja constancia de la fecha, la hora, el programa y el resultado con el fin de que el administrador de la base de datos pueda consultar posteriormente esta información.

«Todo» Esta opción permite consultar cualquiera de las tres incidencias anteriores.

«Borrar» Esta opción permite borrar la tabla activa, incluyendo todas las definiciones relacionadas con ella. Una vez seleccionada aparecerá un menú «Pop-up» que, bajo el título «Elegir», incluye las siguientes opciones:

«Tabla» Borra la tabla en curso. La instrucción de SQL que se ejecuta internamente es DROP TABLE. Una vez seleccionada aparecerá un cuadro de diálogo para pedir confirmación antes de proceder al borrado de la tabla.

«Índice» Borra un índice de la tabla en curso. La instrucción de SQL que se ejecuta es DROP INDEX. Una vez seleccionada aparecerá una ventana de selección para elegir el índice que se desea borrar.

«Clave Primaria» Permite borrar la clave primaria de la tabla en curso. Una vez seleccionada se ejecuta la instrucción de SQL encargada de borrar la clave primaria.

«Clave Referencial» Permite borrar una clave referencial de la tabla en curso. Una vez seleccionada aparecerá en pantalla una ventana de selección para elegir la clave referencial que se desea borrar.

«Documentar» Esta opción permite editar el texto correspondiente a la documentación de la tabla en curso. Este texto será utilizado por el documentador automático de MultiBase (TDOCU) a la hora de confeccionar el Manual del programador, y aparecerá en el fichero «idtabla.dta» como parte de la información de la tabla.

Una vez seleccionada esta opción aparecerán dos ventanas, una de información, que indicará el nombre de la tabla activa junto con el número de líneas de

texto ya documentadas, y otra de edición, que nos permitirá ver dicho texto y, en su caso, corregirlo o ampliarlo.

Opción «Views»

Esta opción contempla todas las operaciones relativas al tratamiento de las «views» (definición, borrado, documentación, etc.). Al ejecutarla se mostrará un menú de tipo «Pop-up» con las siguientes opciones: «Definir», «Ver», «Probar», «Borrar» y «Documentar».

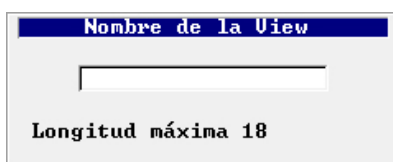
A continuación se explica en detalle cada una de ellas.

«Definir»

Esta opción permite la definición y creación de una «view». La definición de una «view» lleva implícita la de la instrucción SELECT asociada a ella. Si se desea modificar el contenido de una «view» habrá que borrar su definición y volver a crearla. La instrucción de SQL que se ejecuta internamente al crear una «view» es CREATE VIEW.

Una vez seleccionada esta opción se deberá proceder de acuerdo a los siguientes pasos:

1. Indicar el nombre que se desea asignar a la «view» en la ventana de edición que aparece en la pantalla.

Una ventana de diálogo con un título azul que dice "Nombre de la View". En el centro hay un campo de texto rectangular. Debajo del campo, el texto "Longitud máxima 18" indica el límite de caracteres.

Ventana para indicar el nombre de la «view» a crear.

2. Una vez confirmado el nombre asignado a la «view» (con la tecla [F1] generalmente), aparecerá un cuadro de diálogo con dos opciones relativas al empleo de la utilidad «EasySQL»:

- | | |
|----|--|
| SI | La instrucción SELECT será compuesta a través de la utilidad «EasySQL» mediante diferentes menús de selección. |
| NO | En caso de elegir esta opción aparecerá en pantalla el editor ASCII que se hubiese definido para escribir la instrucción SELECT. (Téngase en cuenta que en la definición no deberá incluirse la instrucción CREATE VIEW, sino únicamente la SELECT). |

3. A continuación deberá ejecutarse la instrucción de creación de la «view». En caso de producirse algún error se podrá rectificar la instrucción SELECT y repetir de nuevo el proceso

4. Una vez generada la «view», ésta pasa a ser automáticamente la «view» en curso, mostrándose su nombre en un cartel en la parte superior derecha de la pantalla.

«Ver»

Esta opción muestra en pantalla una ventana de información con la instrucción SELECT asociada a la definición de la «view» en curso. Si la «view» no hubiese sido seleccionada, al ejecutar esta opción aparecerá una ventana de selección para elegirla.

«Probar»

Esta opción permite ejecutar la instrucción SELECT asociada a la definición de la «view» en curso y obtener su resultado por pantalla. Si la «view» no hubiese sido seleccionada, al ejecutar esta opción aparecería una ventana de selección para elegirla.

«Borrar»

Esta opción permite borrar la «view» en curso. La instrucción de SQL que se ejecuta internamente es DROP VIEW. Una vez seleccionada la opción aparecerá en pantalla un cuadro de diálogo para confirmar el borrado de la «view».

«Documentar»

Esta opción permite documentar la «view» en curso. El texto incluido en la documentación de esta opción aparecerá en el fichero «idtabla.dta» como parte de la información de la tabla a la hora de generar el Manual del Programador.

Una vez seleccionada esta opción aparecerán dos ventanas, una de información, que indicará el número de líneas de texto ya documentadas, y otra de edición para documentar la «view» en curso.

Opción «Usuarios»

Esta opción contempla todos los procesos relativos a los permisos de usuarios de la base de datos. Al ejecutarla se mostrará un menú «Pop-up» con las siguientes opciones: «Conceder», «Retirar» e «Información».

A continuación se comenta en detalle cada una de ellas.

«Conceder»

Mediante esta opción se podrán establecer los diferentes niveles de permiso de acceso a la base de datos para uno o varios usuarios. La instrucción de SQL asociada a la concesión de permisos es GRANT. Una vez seleccionada esta opción aparecerá en pantalla un nuevo menú «Pop-up» con las siguientes opciones:

«Connect»

Mediante esta opción se podrá conceder permiso de «CONNECT» (conexión a la base de datos) a uno o varios usuarios. Aquellos usuarios que tengan definido permiso de «CONNECT» podrán realizar consultas y actualizaciones sobre las tablas de la base de datos, pero no ejecutar instrucciones de definición, como por ejemplo CREATE, ALTER o DROP. Una vez seleccionada esta opción aparecerá en pantalla la ventana que se muestra en la figura.

Usuarios	Permiso vigente
▶	<ninguno>

Ventana para definir los usuarios a los que se asignará permiso de «CONNECT».

Junto con esta ventana se mostrará un menú que, bajo el título «Lista de Usuarios», incluye las siguientes opciones:

- | | |
|-------------|---|
| «Agregar» | Añade un usuario a la lista de usuarios afectados por la concesión del permiso de «CONNECT». |
| «Modificar» | Permite modificar el nombre de un usuario existente en la lista de usuarios o su permiso de «CONNECT». |
| «Borrar» | Permite eliminar cualquiera de los usuarios existentes en la lista de usuarios. Una vez seleccionada aparecerá un cuadro de diálogo para pedir conformidad al operador antes de proceder a la operación de borrado. |

«Conforme»	Concede el permiso de «CONNECT» a los usuarios indicados en la lista mediante la ejecución de la instrucción GRANT. Si no se especifica ningún usuario concreto se aplicará a «PUBLIC» (todos los usuarios).
------------	--

La descripción de los campos que aparecen en esta ventana es la siguiente:

«Usuarios»	En este campo se deberá especificar el nombre de un usuario de la base de datos. Este identificador va ligado al valor de la variable de entorno DBUSER, que incluye en su definición el nombre con el que se identifica al usuario al acceder a la base de datos. Bajo sistema operativo UNIX/Linux su valor por defecto es «logname».
«Permiso vigente»	En este campo se mostrará el permiso vigente en ese momento para el usuario indicado en el campo anterior.

«Resource»

Mediante esta opción se podrá conceder permiso de «RESOURCE» (conexión y manejo de la base de datos) a uno o varios usuarios. Aquellos usuarios que tengan asignado permiso RESOURCE podrán realizar consultas y actualizaciones sobre las tablas de la base de datos, así como crear tablas, índices, etc. y conceder permisos a otros usuarios sobre cualquier elemento creado por ellos, o sobre aquellos elementos no creados por ellos pero sobre los cuales les haya sido concedido permiso mediante la cláusula «WITH GRANT OPTION».

La ventana de introducción de datos y las opciones del menú asociado («Lista de Usuarios») son iguales a las comentadas anteriormente para el permiso «CONNECT».

«DBA»

Con esta opción se podrá conceder permiso de «DBA» (funciones de administrador de la base de datos) a uno o varios usuarios. Los usuarios que tengan asignado permiso «DBA» sobre la base de datos podrán realizar cualquier operación sobre la misma.

La ventana de introducción de datos y las opciones del menú asociado («Lista de Usuarios») son iguales a las comentadas anteriormente para el permiso «CONNECT».

«Todos»

Esta opción se encuentra inhabilitada en lo referente a la concesión de permisos.

«Retirar»

Mediante esta opción se podrán cancelar los diferentes niveles de permisos de acceso previamente concedidos para uno o varios usuarios de la base de datos. La instrucción de SQL asociada a la cancelación de permisos es REVOKE. La ejecución de esta opción presenta en pantalla un menú «Pop-up» cuyas opciones son las mismas que las de la opción «Conceder» explicada anteriormente:

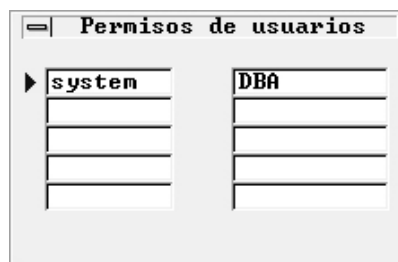
«Connect»	Retira el nivel de acceso «CONNECT» a uno o varios usuarios. Los usuarios indicados deberán existir en la base de datos (pulsando la tecla asociada a la opción <Selecc> se podrán consultar todos los usuarios definidos).
«Resource»	Retira el nivel de acceso «RESOURCE».
«DBA»	Retira el permiso «DBA».
«Todos»	Retira todos los permisos concedidos sobre la base de datos a uno o varios usuarios.

Al ejecutar cada una de estas opciones aparecerá en pantalla la ventana «Lista de Usuarios» mostrada anteriormente junto con un menú con las siguientes opciones:

«Agregar»	Añade un usuario a la lista de usuarios afectados por la retirada del permiso correspondiente («CONNECT», «RESOURCE» o «DBA»).
«Modificar»	Permite modificar el nombre de un usuario existente en la lista de usuarios o su correspondiente permiso.
«Borrar»	Permite eliminar cualquiera de los usuarios existentes en la lista de usuarios. Una vez seleccionada aparecerá un cuadro de diálogo para pedir conformidad al operador antes de proceder a la operación de borrado.
«Conforme»	Esta opción permite hacer efectiva la retirada de permisos a usuarios mediante la ejecución de la instrucción REVOKE

«Información»

Esta opción permite consultar por pantalla los distintos niveles de permisos asignados a los diferentes usuarios de la base de datos. Una vez seleccionada aparecerá en pantalla la ventana de información que se muestra en la figura . Esta ventana consta de dos columnas: en la de la izquierda se mostrará el identificador del usuario y en la de la derecha el nivel de permiso concedido sobre la base de datos. La primera línea de esta ventana corresponde al usuario «DBA» que creó la base de datos, es decir, al administrador de la misma.



Ventana de información sobre permisos de usuarios.

Opción «Información»

A través de esta opción se podrá obtener información sobre los distintos componentes de una tabla de la base de datos (columnas, índices, claves primaria y referenciales, etc.). Al ejecutarla se presentará en pantalla una ventana en la que el usuario deberá elegir la tabla o tablas sobre las que desea obtener información.



Ventana de selección de tablas para obtención de información.

Una vez seleccionadas las tablas, pulsando la tecla correspondiente a «aceptar» (normalmente [F1]) se mostrará un menú «Pop-up» con las siguientes opciones:

«Columnas»

Mediante esta opción se podrá consultar información sobre todas las columnas de las tablas seleccionadas en la ventana anterior: su nombre, tipos de datos, longitud, si admite o no valores nulos, etc. Una vez seleccionada aparecerá una ventana que mostrará las principales características de cada columna.

tabname	num	colname	type	collength	nulls
albaranes	1	albaran	integer	4	N
albaranes	2	cliente	integer	4	N
albaranes	3	fecha_albaran	date	4	
albaranes	4	fecha_envio	date	4	
albaranes	5	fecha_pago	date	4	
albaranes	6	formpago	char	2	
albaranes	7	estado	char	1	
articulos	1	articulo	smallint	2	N
articulos	2	proveedor	integer	4	N
articulos	3	descripcion	char	15	
articulos	4	pr_vent	money	9	
articulos	5	pr_cost	money	9	
articulos	6	existencias	smallint	2	
articulos	7	sobre_maximo	smallint	2	

Ventana de información sobre las columnas de las tablas seleccionadas.

Pulsando [RETURN] o haciendo clic con el ratón (Windows) sobre cualquiera de las columnas que aparecen en la lista el programa mostrará una información más detallada sobre los diferentes atributos de la columna elegida.

tabname	albaranes
colname	albaran
helptxt	
helpnum	
a_autonext	N
a_video	N
a_verify	N
a_queryclear	N
lk_tab	
lk_key	
lk_disp	
l_video	N

Información sobre los atributos de una columna.

«Índices»

Esta opción muestra información de los índices definidos para cada una de las tablas seleccionadas: su nombre, columnas que lo componen, si es único o admite duplicados y el orden (ascendente o descendente). Una vez seleccionada aparece una ventana de información con los datos del índice en curso. Utilizando [FLECHA ARRIBA] y [FLECHA ABAJO] podremos consultar la información de los demás índices.

«Primarias»

Esta opción muestra la clave primaria definida para cada una de las tablas seleccionadas, incluyendo las columnas que la componen. Una vez seleccionada aparece una ventana de información con los datos de la clave primaria en curso. Mediante las teclas [FLECHA ARRIBA] y [FLECHA ABAJO] podremos consultar las demás claves primarias de las restantes tablas.

«Claves»

Esta opción muestra información sobre las claves referenciales definidas para cada una de las tablas seleccionadas. La información que se muestra incluye las columnas que componen la clave referencial, el nombre de la relación y las acciones definidas al borrar y modificar. Una vez seleccionada aparece una ventana de información con los datos de la clave referencial en curso. Mediante las teclas [FLECHA ARRIBA] y [FLECHA ABAJO] podremos consultar la información correspondiente a las demás claves referenciales.

«Permisos»

Esta opción permite consultar la información relativa a la máscara de permisos definida para cada usuario. Una vez ejecutada aparece en pantalla una ventana en la que se muestran los permisos definidos para cada usuario sobre la tabla en curso. Pulsando [FLECHA ABAJO] y [FLECHA ABAJO] accederemos a las restantes tablas (si se hubiese seleccionado más de una).

«General»

Esta opción muestra una ventana con información de carácter general sobre la tabla o tablas seleccionadas (nombre del propietario, «path», número de columnas componentes, etc.). Esta información se encuentra grabada en la tabla «systables» del catálogo de la base de datos. Pulsando [FLECHA ABAJO] y [FLECHA ABAJO] accederemos a las restantes tablas (si se hubiese seleccionado más de una).

Estado de la Tabla	
Tabla	albaranes
Propietario	system
Path	albar160
Long. Fila	23
Nº Columnas	7
Nº Filas	50
Creada el	19/01/2015
Tipo	I

Ventana de información general sobre el estado de una tabla.

Opción «DBSQL»

Esta opción contempla todos los procesos relativos al traspaso de aplicaciones y a la carga y descarga de tablas. Su ejecución presenta un menú «Pop-up» que incluye las opciones que se comentan a continuación.

Acción sobre BD
Generar procedimiento de creación de BD
Generar esquema de una tabla
Importar esquema desde procedimiento SQL
Importar diccionario de programación
Cargar tablas
Descargar tablas

Menú «Pop-up» correspondiente a la opción «DBSQL» de la persiana «Base de Datos».

«Generar procedimiento de creación de BD»

Esta opción genera un fichero SQL con la estructura de la base de datos en el directorio que se indique en la ventana que aparece al ejecutarla (esta ventana mostrará por defecto el nombre del directorio en curso). Dicho fichero recogerá todas las instrucciones del SQL referidas a la definición de tablas, «views» e índices, ejecutadas sobre la base de datos. El nombre del fichero que se genera es igual al de la base de datos en curso, pero con la extensión «.sql».

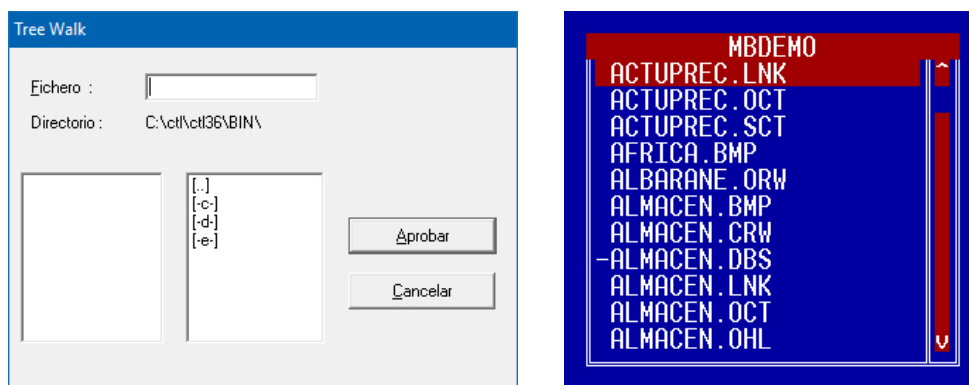
«Generar esquema de una tabla»

Esta opción permite recoger en un fichero SQL todas las instrucciones del SQL referidas a la definición de una tabla. El nombre de dicho fichero será igual al de la tabla, pero con la extensión «.sql».

Al ejecutar esta opción aparecerá en pantalla una ventana para indicar el nombre de la tabla, pudiendo emplear también los metacaracteres del operador «MATCHES» para seleccionar un grupo de tablas que cumplan ciertas condiciones indicadas por dicho operador (en este caso las tablas se mostrarán en una nueva ventana para elegir una de ellas). Una vez seleccionada la tabla habrá que indicar el directorio en el que se desea almacenar el fichero SQL a generar (la ventana que aparece muestra por defecto el directorio en curso).

«Importar esquema desde procedimiento SQL»

Esta opción permite seleccionar un fichero de SQL y ejecutarlo, pudiendo asimismo editarlo y modificar su contenido. Una vez seleccionada esta opción aparecerá una ventana de selección de ficheros cuyo aspecto variará en función del entorno operativo en que nos encontremos (ver figura). Dentro de esta ventana aparecerán tanto los nombres de ficheros SQL (ficheros con extensión «.sql», si bien ésta no se muestra en las versiones para UNIX/Linux), como nombres de directorios (identificados en dichos entornos por ir precedidos por un guión). Una vez localizado el fichero que se desea insertar, habrá que seleccionarlo mediante [RETURN] o la tecla correspondiente a la acción «aceptar» (en Windows se podrá emplear también el ratón).



Ventana para selección de ficheros en Windows (izquierda) y UNIX/Linux (derecha).

Tras seleccionar el fichero se mostrará un elemento de diálogo para confirmar la modificación de su contenido, ofreciéndonos la posibilidad de editarlo y ejecutarlo posteriormente.

«Importar diccionario de programación»

Esta opción se utiliza fundamentalmente para traspasar el contenido de una base de datos entre distintos entornos operativos, dado que no es posible traspasar el contenido del directorio de extensión «.dbf».

El procedimiento para descargar el contenido de las tablas «ep*» en ficheros «.unl» se realiza a través de la secuencia de opciones «Base de Datos» — «DBSQL» — «Generar procedimiento de creación de BD».

Una vez seleccionada esta opción aparecerá una ventana de edición para indicar el directorio donde se encuentran los ficheros «ep*.unl», que por defecto será el directorio en curso.

«Cargar tablas»

Mediante esta opción se podrán cargar las tablas de usuario a partir de la información contenida en ficheros ASCII. Asimismo, se ofrece la posibilidad de generar las instrucciones SQL de carga con el fin de alterar o modificar su ejecución.

Al ejecutar esta opción accederemos a un menú que, bajo el título «Carga/Descarga», incluye las opciones:

- | | |
|---------------|--|
| «Ejecutar» | Ejecuta la instrucción de carga (LOAD) sobre la tabla seleccionada a partir del fichero indicado. |
| «Generar SQL» | Genera un fichero SQL con las instrucciones LOAD en el directorio indicado. Una vez finalizado el proceso de generación muestra el fichero temporal generado. Mediante el editor de textos podremos cambiarle el nombre si así se desea. |

«Descargar tablas»

Mediante esta opción se podrán descargar las tablas de usuario a partir de la información contenida en ficheros ASCII. Asimismo, se ofrece la posibilidad de generar las instrucciones SQL de descarga con el fin de alterar o modificar su ejecución.

Al ejecutar esta opción accederemos a un menú que, bajo el título «Carga/Descarga», incluye las siguientes opciones:

«Ejecutar»	Ejecuta la instrucción de descarga (UNLOAD) sobre el fichero indicado en la tabla seleccionada.
«Generar SQL»	Genera un fichero SQL con las instrucciones UNLOAD en el directorio indicado. Una vez finalizado el proceso de generación muestra el fichero temporal generado. Mediante el editor de textos podremos cambiarle el nombre si se desea.

Opción «Esquema»

Esta opción permite obtener un esquema de las tablas de la base de datos a través de la impresora o de un fichero. Para ello, al seleccionarla presenta un menú «Pop-up» con las opciones que se comentan a continuación.

«Impresora»

Mediante esta opción se podrá obtener por impresora información sobre una o varias tablas de la base de datos. La información facilitada hace referencia a columnas, tipos de datos, longitud, si son requeridas o no, etiquetas, claves primarias y referenciales e índices. Al ejecutarla se mostrará en pantalla una ventana para seleccionar la tabla o tablas sobre las que se desea obtener información.

Una vez seleccionada la tabla habrá que definir el formato de la página que se obtendrá por impresora en el elemento de diálogo que se muestra en la figura.

Cuadro de diálogo para establecer el formato de las páginas impresas.

«Fichero»

Esta opción permite obtener en un fichero información sobre una o varias tablas de la base de datos. La información facilitada hace referencia a columnas, tipos de datos, longitud, si son requeridas o no, etiquetas, claves primarias y referenciales e índices. Al ejecutarla se mostrará en pantalla un elemento de diálogo para indicar el nombre del fichero en el que se guardará el esquema de la tabla. Una vez aceptado dicho nombre habrá que elegir la tabla o tablas sobre las que se desea obtener información. A continuación se presentará el mismo elemento de diálogo que en la opción «Impresora» (ver figura anterior) para establecer el formato de las páginas del fichero. El fichero indicado se generará en el directorio en curso, que será el mismo que el de la base de datos activa.

Opción «Borrar»

Esta opción permite eliminar la base de datos en curso. Al ejecutarla se mostrará un cuadro de diálogo para pedir conformidad al operador antes de proceder a la operación de borrado.

El borrado de una base de datos implica la eliminación de todos sus componentes (tablas, índices, claves, etc.). Esta operación eliminará asimismo el directorio de extensión «.dbs» correspondiente a la base de datos borrada.

Opción «Verif. Diccionario»

Esta opción incluye todos los procesos relativos a la comprobación y actualización de los distintos elementos que conforman el diccionario de la base de datos. Al seleccionarla aparecerá en pantalla un menú «Pop-up» con las opciones que se comentan a continuación.

«Verificación»

Esta opción permite comprobar y actualizar el contenido de las tablas del entorno de desarrollo (tablas «ep*») a partir de las tablas del catálogo de la base de datos (tablas «sys*»). Cada vez que se selecciona una base de datos se ejecuta un proceso de verificación del diccionario de tablas, tras el cual puede aparecer un mensaje indicando la conveniencia de ejecutar esta opción.

El «catálogo de la base de datos» se encuentra siempre asociado a la propia base de datos, y está compuesto por un conjunto de tablas («sys*») que guardan toda la información referente a las instrucciones del SQL. Por su parte, el «catálogo del entorno de desarrollo» está compuesto por las tablas «ep*», las cuales guardan información asociada a la base de datos no relacionada con el SQL. Esta información hace referencia a nombres de módulos, atributos de columnas para generación de código, etc. Las tablas «ep*» se actualizan automáticamente después de cualquier operación realizada a través de las distintas opciones del entorno de desarrollo de MultiBase (TRANS). No obstante, si dichas operaciones se ejecutan directamente a través de la persiana «SQL» de dicho entorno no se produciría la actualización.

«Chequear BD»

Esta opción ejecuta internamente el comando tchkidx para comprobar los índices de todas las tablas de la base de datos. Si lo que se desea es comprobar únicamente los índices de una determinada tabla habrá que ejecutar la secuencia de opciones «Base de Datos» — «Tablas» — «Chequear/Reparar». Este proceso de comprobación afecta también a las tablas «ep*», no así a las «sys*», las cuales no deben comprobarse ni repararse nunca.

Una vez finalizado el proceso de comprobación, el programa indica si dicho proceso ha sido satisfactorio o si, por el contrario, se han encontrado índices defectuosos. Esta información puede ser consultada a través de la opción «Listar monitorización» de este mismo menú.

«Listar monitorización»

Mediante esta opción el administrador de la base de datos podrá consultar todas las incidencias habidas sobre la misma, tales como intentos de acceso a la base de datos encontrándose ésta defectuosa, tablas detectadas con índices defectuosos, comprobaciones y reparaciones llevadas a cabo sobre las tablas, etc.

Una vez seleccionada esta opción accederemos a un menú que, bajo el título «Tipo», incluye las siguientes opciones:

«Accesos»

Cualquier módulo CTL que trabaje con tablas de la base de datos y detecte que alguna de ellas tiene índices defectuosos avisará al operador de esta anomalía, al objeto de que no prosiga en su trabajo y proceda a la comprobación, y en su caso a la reparación, de la tabla. El sistema deja constancia de esta incidencia anotando el usuario, la fecha, la hora y el programa en que se produjo con el fin

de que esta información pueda ser consultada por el administrador de la base de datos a través de esta opción.

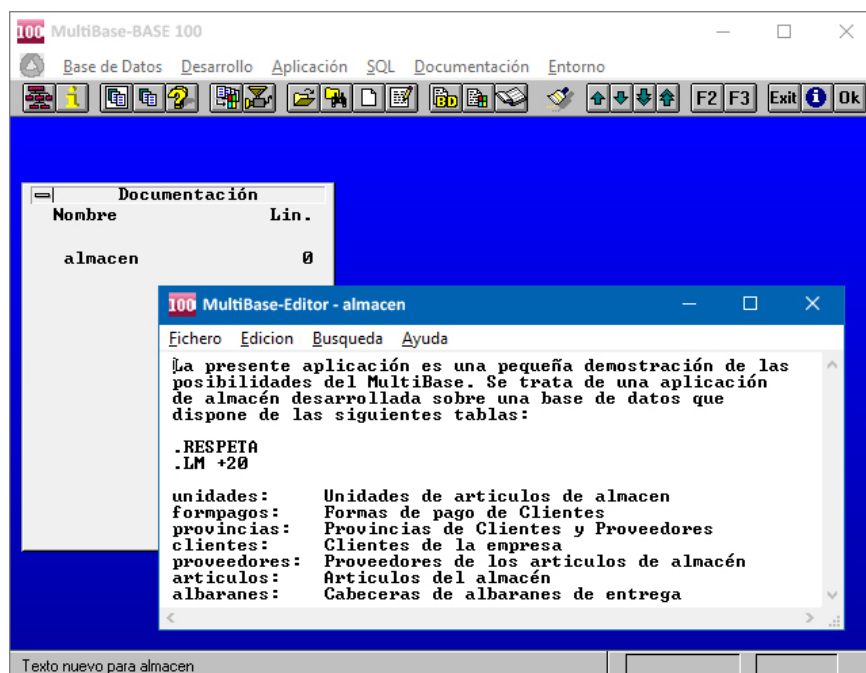
«Errores»	Esta opción permite consultar los errores detectados por el comando tchkidx durante el proceso de comprobación de los índices de las tablas de la base de datos. Estos errores serán eliminados automáticamente al reparar la tabla. El sistema deja constancia de esta incidencia anotando el usuario, la fecha, la hora y el programa en que se produjo el error con el fin de que esta información pueda ser consultada por el administrador de la base de datos.
«Regeneraciones»	Cada vez que se efectúa la comprobación o reparación de una tabla de la base de datos, el sistema deja constancia de esta incidencia anotando la fecha, la hora, el programa y el resultado de la comprobación o reparación con el fin de que el administrador de la base de datos pueda consultar posteriormente esta información a través de esta opción.
«Todo»	Mediante esta opción se podrá consultar de una sola vez toda la información facilitada por las tres opciones anteriores.

«Borrar monitorización»

Mediante esta opción podremos eliminar toda la información relativa a las incidencias proporcionada por la opción «Listar monitorización». Al ejecutarla aparecerá un cuadro de diálogo para pedir conformidad al operador antes de proceder a la operación de borrado.

Opción «Documentar»

Mediante esta opción podremos editar el texto correspondiente a la documentación de la base de datos activa. Al ejecutarla el programa presentará un elemento de diálogo (que indicará el nombre de la base de datos y el número de líneas de texto de documentación existentes), junto a una ventana, provocada por la ejecución del editor de textos de MultiBase, en la que se mostrará el texto de la documentación, pudiendo entonces modificarlo o añadir más texto.



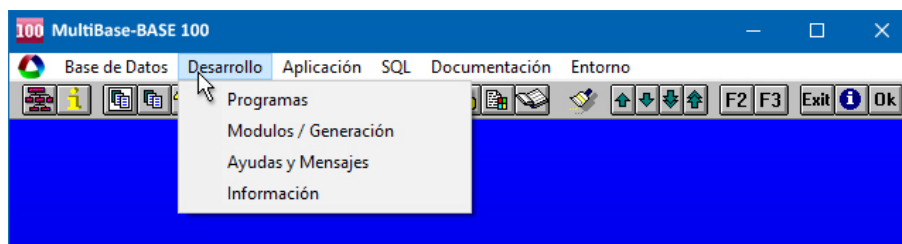
Ejecución de la opción «Documentar» de la persiana «Base de Datos».

Persiana «Desarrollo»

Esta persiana incluye las opciones relativas al tratamiento de los diferentes tipos de programas y módulos manejados por MultiBase. Estas opciones son las siguientes:

- «Programas».
- «Módulos/Generación».
- «Ayudas y Mensajes».
- «Información».

En los siguientes apartados se comenta en detalle cada una de ellas.



Persiana «Desarrollo» del menú principal de MultiBase.

Opción «Programas»

Esta opción contempla todos los procesos relativos a la definición, edición, compilación, enlace y ejecución de los distintos programas que componen una aplicación.

La definición de un programa se corresponde con un conjunto de módulos en los que obligatoriamente uno de ellos, y sólo uno, debe contener la sección MAIN. El comando utilizado para enlazar («linkar») un conjunto de módulos y obtener como resultado un programa es ctlink. Los programas se identifican por la extensión «.lnk».

Esta opción se utilizará para definir cuántos y cuáles son los módulos que componen un programa. Todos estos módulos deberán haber sido definidos previamente a través de la opción «Módulos» de esta misma persiana.

Tras ejecutar la opción «Programas» accederemos a un menú que, bajo el título «Tipo», incluye cinco opciones para elegir el tipo de programa: «Entrada Datos», «Listados», «Menús», «Librerías» y «Varios».

NOTA: Esta clasificación en cinco tipos es arbitraria y no implica ningún tratamiento especial, únicamente servirá para su localización posterior.

A continuación se comenta en detalle cada una de estas opciones.

«Entrada Datos»

Al ejecutar esta opción se presentará un menú «Pop-up» (ver figura) con las opciones que se comentan a continuación.

«Definir»

Mediante esta opción se podrán llevar a cabo las siguientes operaciones:

- Definir nuevos programas.
- Modificar características del programa: «debugger», parámetros, etc.
- Agregar, modificar o borrar componentes (módulos) de un programa.
- Editar y compilar los componentes (módulos) de un programa.



Menú «Pop-up» para tratamiento de programas de entrada de datos.

Al ejecutar esta opción aparecerá una ventana de edición, con el título «Mantenimiento de Programas» (ver figura), que mostrará algunos datos sobre el programa en curso. En el caso de que no hubiese ningún programa activo, dicha ventana aparecería con sus campos en blanco, pudiendo entonces elegir el programa deseado a

través de la tecla correspondiente a la opción <Selec>. El nombre del programa en curso se mostrará en una ventana en la parte superior derecha de la pantalla.

Mantenimiento de Programas	
Programa	: articulos Depuración ☒
Path	: c:\mbdemo
Parámetros llamada	:

Ventana de «Mantenimiento de Programas».

La descripción de los campos que aparecen en esta ventana es la siguiente:

- «Programa» Nombre del programa a crear o modificar. Para seleccionar un programa se podrá utilizar la tecla correspondiente a la opción <Selec> que figura en la línea de teclas de ayuda en la parte inferior de la pantalla.
- «Depuración» En este campo indicaremos si se va a utilizar o no el depurador («debugger») a la hora de compilar y ejecutar el programa. El contenido de este campo no se guarda como información en las tablas del entorno, por lo que su valor se perderá al abandonar el menú de programas, siendo necesario por tanto cambiar su valor cuando se vuelva a utilizar el programa. En este caso, se utilizará el valor por defecto indicado para su compilación y ejecución a través de la secuencia de opciones «Aplicación» — «Definir». Sus posibles valores son «Y» o «N», que indican respectivamente la utilización o no del «debugger» a la hora de la compilación y ejecución, siendo este último el valor por defecto.
- «Path» Al crear un módulo se podrá indicar el directorio donde se desea ubicarlo (por defecto será el mismo que definimos a través de la secuencia de opciones «Aplicación» — «Definir» — «Entrada de Datos»). Pulsando la tecla correspondiente a la opción <Selec> que aparece en la línea de teclas de ayuda en la parte inferior de la pantalla podremos cambiar de directorio, eligiéndolo en la ventana que muestra los diferentes directorios definidos en las variables de entorno DBPROC y DBLIB. El valor indicado en este campo ha de corresponder necesariamente con un directorio existente.
- «Parámetros llamada» Este campo permite definir determinados parámetros necesarios para la ejecución de algunos programas. La ejecución del programa se realizará siempre con el último valor indicado. La separación entre un parámetro y otro se entiende que es uno o varios espacios en blanco. No obstante, se podrán emplear comillas para considerar como valor un espacio en blanco. El orden de los valores en esta lista ha de coincidir con el orden de definición de los parámetros en el programa.

Una vez cumplimentada la ventana anterior, pulsando la tecla correspondiente a «aceptar» (normalmente [F1]) accederemos a una nueva ventana (ver figura siguiente) para tratamiento de módulos, junto con un menú que, bajo el título «Componentes del Programa», incluye las siguientes opciones:

- «Agregar» Esta opción permite agregar módulos a la definición del programa en curso. Al activarla comienza el proceso de edición de campos. Una vez cumplimentado el nombre del módulo se valida mediante la tecla correspondiente a «aceptar».
- «Borrar» Permite eliminar un módulo como componente del programa en curso. Este borrado no afecta sin embargo a su definición como módulo. En caso de que existiesen varios módulos podremos acceder a ellos con las teclas [FLECHA ARRIBA] y [FLECHA ABAJO], siendo el módulo en curso aquel en el que el cursor aparece en la parte derecha de la ventana. Asimismo, si el número de columnas existen-

tes es superior a ocho, podremos emplear también las teclas [AV-PÁG] y [RE-PÁG]. Antes de proceder al borrado de un módulo el programa pide conformidad al operador.

- «Encontrar» Esta opción permite consultar todos los componentes del programa en curso.
- «Editar» Esta opción permite editar los módulos componentes del programa en curso.
- «Compilar» Permite compilar los módulos componentes del programa en curso. Una vez seleccionada se mostrará un mensaje en la línea inferior de la pantalla indicando la compilación del módulo en curso. Si el módulo tuviese errores aparecerá una ventana de información indicando el número de errores, pudiendo corregirlos tras pulsar «cancelar». Las líneas que comienzan con el carácter «#» en el fichero de errores nos informan del error producido. Estas líneas se eliminarán automáticamente una vez finalizada la corrección, por lo que no será necesario borrarlas. Una vez corregidos los errores podremos elegir entre las opciones:
 - «Compilar» Efectúa la compilación teniendo en cuenta los cambios realizados.
 - «Salvar y salir» Hace efectivos los cambios realizados y abandona la compilación.
 - «Descartar y salir» Descarta los cambios y abandona la compilación.

Módulos del Programa					
Módulo	Tipo	Comp.	Último acceso		Usuario
articulos	Entrada de Datos	N	04/01/1995	12:29:29	system

Ventana de módulos correspondientes al programa en curso.

Los campos que aparecen en la ventana de módulos son los siguientes:

- «Módulo» Nombre de un módulo definido que formará parte de la definición del programa. De todos los módulos componentes de un programa sólo uno de ellos deberá contener la sección MAIN. Se puede seleccionar el módulo pulsando la tecla asociada a la opción <Selec> que aparece en la línea de teclas de ayuda en la parte inferior de la pantalla. Al seleccionar el módulo se completa la información relativa a los siguientes campos de la ventana.
- «Tipo» Tipo de programa («Entrada de datos», «Listados», «Menús», «Librerías» o «Varios»).
- «Comp» En este campo se mostrará uno de los siguientes valores:
 - «Y» Módulo compilado con «debugger».
 - «N» Módulo compilado sin «debugger».
 - «E» Módulo compilado con errores.
 - «?» Módulo editado y no compilado posteriormente.
- «Último acceso» Fecha y hora de la última compilación del módulo.
- «Usuario» Identificador del usuario que creó o modificó el módulo.

«Elegir»

Mediante esta opción podremos seleccionar cuál será el programa en curso. Si ya hubiese algún programa activo, esta opción nos permitirá también cambiarlo.

Al ejecutar esta opción aparecerá una ventana de edición en la que se podrán indicar los siguientes valores:

- Nombre del programa (que pasará a ser el programa en curso).
- Pulsando [RETURN] o «aceptar» sin introducir ningún valor se mostrará una ventana de selección con todos los programas definidos para poder elegir uno de ellos.
- Asimismo, se podrá indicar un «patrón» de búsqueda de programas utilizando los metacaracteres «?» y «*» del operador MATCHES (por ejemplo, «a*» buscará todos los programas que comiencen por «a»).

Una vez elegido un programa su nombre se mostrará en una ventana en la parte superior derecha de la pantalla.

«Compilar»

Esta opción realiza la compilación del programa en curso, es decir, compilará cada uno de los módulos que componen dicho programa. El comando encargado de esta operación es `ctlcomp`. Además de la compilación se efectúa también el «linkado» (enlace) de los distintos módulos componentes del programa. La utilización o no del «debugger» en compilación dependerá del valor indicado en la opción «Definir» de este mismo menú.

«Borrar»

Esta opción borra la definición como programa del programa en curso dentro de las tablas «ep*», así como su fichero asociado (fichero con extensión «.lnk»), pero no afecta sin embargo a la definición de los módulos que lo componen ni a sus ficheros asociados.

«Renombrar»

Esta opción permite cambiar de nombre la definición como programa del programa en curso dentro de las tablas «ep*». El cambio de nombre del programa afecta también al fichero asociado (fichero de extensión «.lnk»). Una vez seleccionada esta opción aparecerá una ventana de edición para indicar el nuevo nombre del programa en curso, teniendo en cuenta que dicho nombre no deberá coincidir con el de ningún otro programa existente ni superar los ocho caracteres de longitud como máximo.

«Ejecutar»

Mediante esta opción podremos ejecutar el programa en curso. La utilización o no del «debugger» en ejecución dependerá del valor indicado en la opción «Definir» de este mismo menú. En las versiones de MultiBase para Windows la ejecución se realizará utilizando el fichero de inicialización (fichero con extensión «.ini») indicado a través de la secuencia de opciones «Entorno» — «Varios» — «Fichero WININI».

«Enlazar»

Mediante esta opción podremos enlazar los módulos componentes del programa en curso. El comando encargado de llevar a cabo el enlace entre módulos es `ctllink`. El resultado de esta operación es la generación de un fichero cuyo nombre será igual al del programa en curso con la extensión «.lnk». Una vez finalizada la ejecución de esta opción podremos consultar un informe sobre el proceso de «linkado» con información de las funciones definidas en cada uno de los módulos. Si se produjesen errores aparecería una ventana indicando la descripción del error.

«Documentar»

Esta opción permite editar el texto correspondiente a la documentación del programa en curso. El texto incluido a través de esta opción aparecerá a la hora de general el Manual del Programador en el fichero «ep_dict.dta». Al ejecutar la opción «Documentar» el programa presentará un elemento de diálogo (que indicará el nombre del programa en curso y el número de líneas de texto de documentación existentes), junto a

una ventana, provocada por la ejecución del editor de textos de MultiBase, en la que se mostrará el texto de la documentación, pudiendo entonces modificarlo o añadir más texto (ver la opción «Documentar» comentada anteriormente).

«Clasificar»

Mediante esta opción es posible cambiar la clasificación del programa en curso de «Entrada de Datos» a cualquiera de los otros tipos existentes («Listados», «Menús», «Librerías» y «Varios»). El cambio de clasificación de un programa no afecta al directorio de los ficheros asociados. Una vez seleccionada esta opción aparecerá un menú «Pop-up» («Nuevo Tipo») con las opciones: «Entrada Datos», «Listados», «Menús», «Librerías» y «Varios». Las opciones disponibles en este menú serán todas excepto aquella que corresponda al tipo del programa en curso cuya clasificación se desea modificar (en este caso «Entrada Datos» estaría inactiva).

«Listados»

Al ejecutar esta opción se presentará un menú «Pop-up» idéntico al de la opción anterior («Entrada Datos»). El manejo y la función de cada una de las opciones de este menú son asimismo idénticos, si bien en este caso harán referencia a los programas definidos del tipo «Listados». La única variación es la relativa a la opción «Clasificar», ya que en este caso la opción inactiva será «Listados».

«Menús»

Al ejecutar esta opción se presentará un menú «Pop-up» idéntico al de la opción «Entrada Datos». El manejo y la función de cada una de las opciones de este menú son asimismo idénticos, si bien en este caso harán referencia a los programas definidos del tipo «Menú». La única variación es la relativa a la opción «Clasificar», ya que en este caso la opción inactiva será «Menús».

«Librerías»

Al ejecutar esta opción se presentará un menú «Pop-up» idéntico al de la opción «Entrada Datos». El manejo y la función de cada una de las opciones de este menú son asimismo idénticos, si bien en este caso harán referencia a los programas definidos del tipo «Librerías». La única variación es la relativa a la opción «Clasificar», ya que en este caso la opción inactiva será «Librerías».

«Varios»

Al ejecutar esta opción se presentará un menú «Pop-up» idéntico al de la opción «Entrada Datos». El manejo y la función de cada una de las opciones de este menú son asimismo idénticos, si bien en este caso harán referencia a los programas definidos del tipo «Varios». La única variación es la relativa a la opción «Clasificar», ya que en este caso la opción inactiva será «Varios».

Opción «Módulos/Generación»

Esta opción contempla todos los procesos relativos a la definición, compilación, generación y ejecución de los distintos módulos CTL de la aplicación (un módulo CTL es un fichero que contiene instrucciones del lenguaje de cuarta generación). Los módulos podrán ser de dos tipos, módulos fuente, identificados por la extensión «.sct», y módulos compilados (generados tras la ejecución del comando `ctlcomp`), que se identifican por la extensión «.oct».

La clasificación de los módulos depende también de que incluyan o no la función de entrada al programa (MAIN). Esto hace que distingamos entre «módulos con MAIN» y «módulos sin MAIN».

Tras ejecutar la opción «Módulos/Generación» accederemos a un menú que, bajo el título «Tipo», incluye las siguientes opciones: «Entrada Datos», «Listados», «Menús», «Librerías» y «Varios».

A continuación se comenta en detalle cada una de ellas.

«Entrada Datos»

Al ejecutar esta opción se presentará un menú «Pop-up» (ver figura) con las opciones que se explican a continuación.

«Definir»

Mediante esta opción se podrán llevar a cabo las siguientes operaciones:

- Definir nuevos módulos.
- Modificar características del módulo: «debugger», función de entrada, argumentos y directorio.

Todos los módulos fuente se identifican por tener la extensión «.sct». Una vez compilados se generarán ficheros homónimos con extensión «.oct».

Al ejecutar esta opción aparecerá una ventana de edición con el título «Mantenimiento de Módulos» (similar a la expuesta para el Mantenimiento de Programas), cuyos campos son:

«Módulo»	Nombre del módulo a crear o modificar. Para seleccionar un módulo se podrá utilizar la tecla correspondiente a la opción <Selecc> que figura la línea de teclas de ayuda en la parte inferior de la pantalla.
Depuración»	En este campo indicaremos si se va a utilizar o no el depurador («debugger») a la hora de compilar y ejecutar el módulo. Sus posibles valores son «Y» o «N», que indican respectivamente la utilización o no del «debugger» a la hora de la compilación y ejecución, siendo este último el valor por defecto.
«Path»	Directorio donde se encuentra el módulo en curso. Su valor deberá coincidir con el nombre de algún directorio existente en el sistema que se encuentre definido en las variables de entorno DBPROC o DBLIB.
«Función de entrada»	En este campo deberá indicarse el nombre de una función existente en el módulo en curso que servirá de entrada al mismo (aquellos módulos que no tienen MAIN necesitan una función que sirva de entrada para su prueba). De esta forma podremos probar todas las funciones que contiene un módulo sin MAIN.
«Argumentos función»	Si la función de entrada especificada en el campo anterior para probar el módulo precisa de argumentos, éstos se indicarán en este campo. Los valores que se incluyan han de ir separados por comas y, según el tipo de dato, necesitarán comillas o no.

«Elegir»

Mediante esta opción podremos seleccionar cuál será el módulo en curso. Si ya hubiese algún módulo activo, esta opción nos permitirá también cambiarlo.

Al ejecutar la opción «Elegir» aparecerá una ventana de edición en la que se podrán indicar los siguientes valores:

- Nombre del módulo (que pasará a ser el módulo en curso).
- Pulsando [RETURN] o «aceptar» sin introducir ningún valor se mostrará una ventana de selección con todos los módulos definidos para poder elegir uno de ellos.
- Asimismo, se podrá indicar un «patrón» de búsqueda utilizando los metacaracteres «?» y «*» del operador MATCHES (por ejemplo, «a*» buscará todos los módulos que comiencen por «a»).

Una vez elegido un módulo su nombre se mostrará en una ventana en la parte superior derecha de la pantalla.

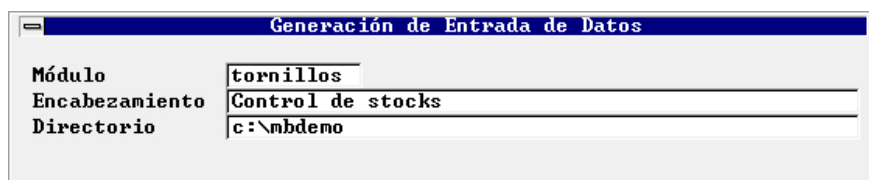


Menú «Pop-up» para tratamiento de módulos de entrada de datos..

«Generar Fuente»

Mediante esta opción podremos acceder al generador de módulos fuente. En todos los casos se realiza primero la generación del módulo fuente y posteriormente su compilación, quedando el módulo compilado como módulo en curso.

Al ejecutar «Generar Fuente» el programa presentará un menú «Pop-up» con las opciones «Entrada Datos» y «Cabecera-Líneas». Cualquiera de estas dos opciones presentará la ventana que se muestra en la figura, donde deberá indicarse el nombre del módulo fuente que se desea generar.

A screenshot of a Windows-style dialog box titled "Generación de Entrada de Datos". It contains three labeled text input fields: "Módulo" with the text "tornillos", "Encabezamiento" with the text "Control de stocks", and "Directorio" with the text "c:\mbdemo".

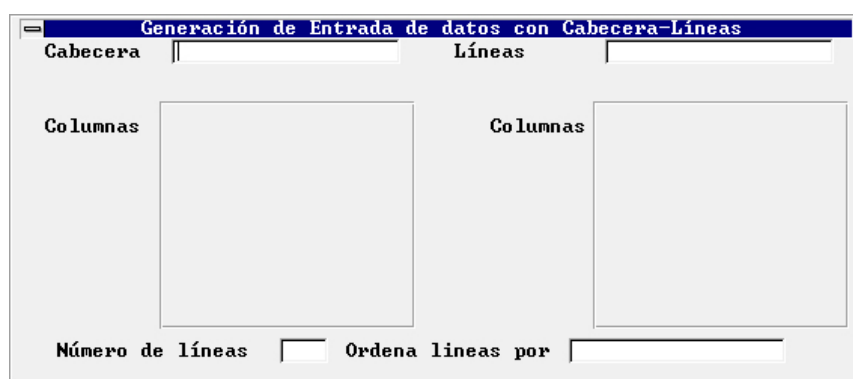
Ventana para generación de un módulo de entrada de datos.

Los campos que aparecen en esta ventana son los siguientes:

- | | |
|------------------|--|
| «Módulo» | Nombre del fichero fuente de entrada de datos que se desea crear, no siendo necesario indicar la extensión «.sct». Este nombre no deberá coincidir con el de ningún módulo o programa existente en la base de datos. |
| «Encabezamiento» | Texto que será utilizado en la generación como título de la ventana de entrada de datos. |
| «Directorio» | Este campo no es modificable por el usuario, y en él se muestra el «path» donde se generará el módulo, que por defecto será el directorio indicado mediante la secuencia de opciones «Aplicación» — «Definir» — «Entrada Datos». |

En el caso de haber seleccionado la opción «Entrada Datos», al aceptar los valores introducidos (normalmente con la tecla [F1]), aparecerá una nueva ventana para elegir la tabla a partir de la cual se va a generar el módulo de entrada de datos.

Por su parte, la opción «Cabecera-Líneas» presentará la ventana que se muestra en la siguiente figura para establecer la relación cabecera-líneas.

A screenshot of a Windows-style dialog box titled "Generación de Entrada de datos con Cabecera-Líneas". It has two main sections. The left section is labeled "Cabecera" and contains a text input field and a "Columnas" label above a large empty box. The right section is labeled "Líneas" and contains a text input field and a "Columnas" label above a large empty box. At the bottom, there are two input fields labeled "Número de líneas" and "Ordena líneas por".

Ventana para establecer la relación cabecera-líneas.

La descripción de cada uno de los campos que aparecen en la ventana anterior es la siguiente:

- | | |
|------------|---|
| «Cabecera» | Nombre de la tabla de cabecera en la relación cabecera-líneas. Para seleccionar una tabla entre las existentes en la base de datos podremos utilizar la tecla que se indica en la línea de mensajes en la parte inferior de la pantalla. Una vez seleccionada la tabla aparecerán por defecto las columnas que forman la clave primaria de la tabla de cabecera, pudiendo éstas ser modificadas por el usuario. |
|------------|---|

«Columnas»	Nombre de una columna perteneciente a la tabla indicada en el campo anterior (para seleccionar una columna podremos utilizar igualmente la tecla indicada en la parte inferior de la pantalla). Este campo es obligatorio, debiendo indicar, al menos, una columna (máximo 8).
«Líneas»	Nombre de la tabla de líneas que formará la relación cabecera-líneas con la tabla indicada en el campo «Cabecera».
«Columnas»	Nombre de una columna perteneciente a la tabla indicada en el campo anterior. Ha de tenerse en cuenta que el número de columnas indicadas en este campo (8 como máximo) ha de coincidir necesariamente con el especificado en el campo homónimo de la tabla de cabecera. Asimismo, cada pareja de columnas de ambas tablas (la primera con la primera, la segunda con la segunda, etc.) deberán tener el mismo tipo de dato SQL. Asimismo, las columnas de la tabla de líneas deberían formar parte de un índice duplicado que tuviera la misma estructura de la clave primaria de la tabla de cabecera (para seleccionar un índice podremos emplear la tecla indicada en la parte inferior de la pantalla). No obstante, y aunque es recomendable que el enlace entre las tablas se realice a través de la clave primaria y de un índice duplicado, esto no es obligatorio, por lo que podremos establecer un enlace sólo entre columnas de una y otra tabla.
«Número de líneas»	Número de filas de la tabla de líneas que se mostrarán simultáneamente en pantalla.
«Ordenar líneas por»	Nombre de una columna de la tabla de líneas por la que se ordenarán las líneas al presentarse en pantalla. Este campo no es obligatorio.

Una vez aceptados los valores indicados, tanto en la opción «Entrada Datos» como en «Cabecera-Líneas», accederemos a una ventana (ver siguiente figura) para poder alterar las características de la generación por defecto de acuerdo a las columnas de la tabla que tomarán parte en la generación del módulo (en la opción «Cabecera-Líneas» se mostrarán primero las columnas de la tabla de cabecera y a continuación las correspondientes a la tabla de líneas).

En esta ventana las teclas de movimiento del cursor nos permitirán situarnos sobre cada uno de sus campos. El campo activo será aquel que se encuentre marcado con una flecha (en UNIX/Linux) o con el símbolo «>» (en Windows).

Tabla : albaranes 1/7		
Ord.	Etiqueta	Sel.
1	»Num. Albaran	> *
2	Cod. Cliente	*
3	Fecha Albaran	*
4	Fecha Envio	*
5	Fecha Pago	*
6	Cod. F. Pago	*
7	Facturado	*

Ventana para modificar la generación por defecto.

La forma de modificar cualquiera de los valores que se presentan en la ventana será mediante [RETURN], aceptándose con la tecla correspondiente a «aceptar» (normalmente [F1]). La descripción de los campos es la siguiente:

«Ord.»	Permite alterar el orden de edición y presentación de las columnas en el módulo a generar. Al pulsar [RETURN] aparecerá una nueva ventana para seleccionar la columna que deberá ocupar la posición en que se encuentra el cursor.
«Etiqueta»	Permite modificar la etiqueta que aparece por defecto para la columna en curso.
«Sel.»	Permite seleccionar o no la columna en curso para la generación.

Una vez realizados los pasos anteriores se procederá a generar el módulo fuente. El proceso de generación lleva implícita la compilación del módulo, así como su definición como programa.

En el caso de que estuviésemos en una base de datos en la que no se hubiese generado aún ningún módulo fuente, al llevar a cabo este proceso se generará también una librería de funciones, denominada «mblib» (para más información consulte la opción «Generar Fuente» de «Librerías» de este mismo menú «Tipo».

«Editar»

Esta opción permite editar el módulo en curso mediante el editor de textos correspondiente, por defecto «Tword» en UNIX/Linux y «Wedit» en Windows, o bien el indicado a través de la secuencia de opciones «Entorno» — «Varios» — «Editor».

«Compilar»

Esta opción permite compilar el módulo en curso. Una vez seleccionada se mostrará un mensaje en la línea inferior de la pantalla indicando el proceso de compilación. En caso de existir errores aparecerá una ventana de información indicando su número, pudiendo corregirlos tras pulsar «cancelar». Las líneas que comienzan con el carácter «#» en el fichero de errores nos informan del error producido. Estas líneas se eliminarán automáticamente una vez finalizada la corrección, por lo que no será necesario borrarlas. Una vez corregidos los errores podremos elegir entre las siguientes opciones:

«Compilar»	Efectúa la compilación teniendo en cuenta los cambios realizados.
«Salvar y salir»	Hace efectivos los cambios realizados y abandona la compilación.
«Descartar y salir»	Descarta los cambios realizados y abandona la compilación.

«Borrar»

Esta opción borra el módulo en curso dentro de las tablas «ep*», así como su definición en los programas en los que estuviera definido como componente. El borrado de un módulo afecta también a sus respectivos ficheros fuente («.sct») y compilado («.oct»). Antes de proceder al borrado del módulo el programa pide conformidad al operador.

«Renombrar»

Esta opción permite cambiar de nombre al módulo en curso dentro de las tablas «ep*». Esta operación afecta igualmente a sus respectivos ficheros fuente («.sct») y compilado («.oct»). Una vez seleccionada la opción «Renombrar» aparecerá una ventana de edición para indicar el nuevo nombre del módulo, teniendo en cuenta que éste no debe coincidir con el de ningún otro módulo existente ni superar los ocho caracteres de longitud como máximo.

«Probar»

Esta opción permite ejecutar el módulo en curso. La utilización o no del «debugger» en ejecución dependerá del valor indicado en la opción «Definir» de este mismo menú. En las versiones de MultiBase para Windows la ejecución se realizará utilizando el fichero de inicialización (fichero con extensión «.ini») indicado a través de la secuencia de opciones «Entorno» — «Varios» — «Fichero WININI».

«Documentar»

Esta opción permite editar el texto correspondiente a la documentación del módulo en curso. El texto incluido a través de esta opción aparecerá a la hora de general el Manual del Programador en el fichero «ep_dict.dta». Al ejecutar la opción «Documentar» el programa presentará un elemento de diálogo (que indicará el nombre del módulo en curso y el número de líneas de texto de documentación existentes), junto a una ventana, provocada por la ejecución del editor de textos de MultiBase, en la que se mostrará el texto de la documentación, pudiendo entonces modificarlo o añadir más texto.

«Clasificar»

Mediante esta opción es posible cambiar la clasificación del módulo en curso de «Entrada de Datos» a cualquiera de los otros tipos existentes («Listados», «Menús», «Librerías» y «Varios»). El cambio de clasificación de un módulo no afecta al directorio de los ficheros asociados. Una vez seleccionada esta opción aparecerá un menú «Pop-up» («Nuevo Tipo») con las opciones: «Entrada Datos», «Listados», «Menús», «Librerías» y «Varios». Las opciones disponibles en este menú serán todas excepto aquella que corresponda al tipo del módulo en curso cuya clasificación se desea modificar (en este caso «Entrada Datos» estaría inactiva).

«Listados»

Al ejecutar esta opción se presentará un menú «Pop-up» idéntico al de la opción «Entrada Datos» comentada anteriormente. El manejo y la función de cada una de sus opciones son prácticamente iguales, con dos pequeñas excepciones que se comentan a continuación.

- En «Clasificar», la opción inactiva será «Listados».
- Al seleccionar la opción «Generar Fuente» aparecerá una ventana para indicar el nombre del módulo de tipo listado a generar. Una vez aceptado dicho nombre accederemos a un menú que, bajo el título «Elija salida para el listado», incluye las siguientes opciones:

«Terminal»	La generación del fuente para el listado contempla que la salida sea por pantalla.
«Impresora»	La generación del fuente para el listado contempla que la salida sea por impresora.
«Fichero»	La generación del fuente para el listado contempla que la salida sea a un fichero, cuyo nombre podrá indicarse a la hora de probar el módulo.

Al seleccionar cualquiera de estas opciones aparecerá un nuevo menú compuesto a su vez por otras tres opciones:

«Usando EasySQL»	Permite generar una instrucción SELECT a través de la utilidad EasySQL. El listado se realiza sobre la instrucción SELECT generada, que podrá incluir cruces entre tablas, ordenaciones, filtros, etc. Una vez seleccionada esta opción arranca la utilidad EasySQL (para más información sobre el manejo de esta herramienta consulte el capítulo 2 del Manual de Aplicaciones de Ofimática).
«De una tabla»	Genera un listado del contenido de una tabla. Una vez ejecutada aparece en pantalla una ventana de selección para elegir la tabla de la que se desea obtener el listado a través de la salida indicada en el menú anterior.
«De una View»	El listado que se genera hace referencia a todas las filas seleccionadas en la instrucción SELECT de definición de la «view». Al ejecutar esta opción aparecerá una ventana para elegir la «view» de la cual se desea obtener el listado a través de la salida indicada en el menú anterior.

«Menús»

Al ejecutar esta opción se presentará un menú «Pop-up» idéntico al de la opción «Entrada Datos» comentada anteriormente. El manejo y la función de cada una de sus opciones son prácticamente iguales, con dos pequeñas excepciones que se comentan a continuación.

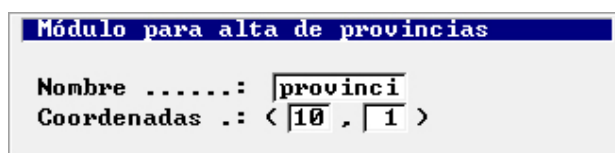
- En «Clasificar», la opción inactiva será «Menús».
- Al seleccionar la opción «Generar Fuente» el programa generará un módulo fuente en el que se agruparán, en un menú de tipo «pulldown», las llamadas a todos los programas definidos en los tipos entrada de datos, listados, librerías y varios. El módulo generado se denominará por defecto to-

mando como raíz el nombre de la base de datos en curso. Este nombre podrá ser modificado por el operador mediante la opción «Renombrar» del menú «Pop-up».

«Librerías»

Al ejecutar esta opción se presentará un menú «Pop-up» idéntico al de la opción «Entrada Datos» comentada anteriormente. El manejo y la función de cada una de sus opciones son prácticamente iguales, con dos pequeñas excepciones que se comentan a continuación.

- En «Clasificar», la opción inactiva será «Librerías».
- Al ejecutar «Generar Fuente» aparecerá una ventana para indicar el nombre del módulo a generar. Una vez aceptado dicho nombre comenzará el proceso de generación, el cual se divide en dos fases: la primera consiste en la generación de funciones independientes de la base de datos (por ejemplo, para emitir mensajes por pantalla: «disp_msg()»), mientras que la segunda tiene por objeto generar funciones de integridad de datos que serán llamadas en la edición de módulos de entrada de datos. Estas funciones de integridad vienen determinadas por la integridad referencial definida en la base de datos. Por cada relación de integridad definida se mostrará una ventana para modificar, si se desea, los parámetros establecidos por defecto.



La imagen muestra una ventana de diálogo con un título azul que dice "Módulo para alta de provincias". Dentro de la ventana, hay dos líneas de texto. La primera línea es "Nombre" con un campo de entrada que contiene el texto "provinci". La segunda línea es "Coordenadas .:" con un campo de entrada que contiene el texto "< 10 , 1 >".

Ventana de parámetros para generación de fuente de librería.

Los campos de esta ventana son los siguientes:

- | | |
|---------------|--|
| «Nombre» | Nombre del módulo que se utilizará en la llamada CTL para dar de alta las filas de la tabla cuyo nombre figura en la cabecera de la ventana. Con este nombre se genera la llamada al módulo CTL indicado. Dicha llamada se genera dentro de la librería en la función «exist_add_nombre_tabla». Pulsando la tecla que se indica en la parte inferior de la pantalla se podrá seleccionar cualquiera de los módulos existentes. |
| «Coordenadas» | Números de línea y columna donde comenzará a representarse el módulo CTL llamado. |

Una vez finalizado este proceso de generación se procederá a la compilación del módulo fuente de librería.

«Varios»

Al ejecutar esta opción se presentará un menú «Pop-up» idéntico al de la opción «Entrada Datos» comentada anteriormente. El manejo y la función de cada una de sus opciones son prácticamente iguales, con dos pequeñas excepciones que se comentan a continuación.

- En «Clasificar», la opción inactiva será «Varios».
- Al seleccionar la opción «Generar Fuente» se mostrará una ventana para indicar el nombre del módulo a generar. La finalidad de este módulo será la realización de consultas por pantalla o impresora sobre las columnas de la tabla seleccionada. Una vez aceptado el nombre del módulo, el programa solicitará la tabla sobre la cual se realizará la consulta. Tras elegir ésta accederemos a una nueva ventana para poder alterar las características de la generación por defecto. La descripción de los campos de esta ventana es la siguiente:

- | | |
|------------|---|
| «Ord.» | Permite establecer el orden en que se mostrarán las columnas al realizar la consulta. |
| «Etiqueta» | Permite modificar la etiqueta que aparece por defecto para la columna en curso. |
| «Sel.» | Permite seleccionar o no la columna en curso para la generación. |

Una vez realizados los pasos anteriores se procederá a generar el módulo de consulta. El proceso de generación lleva implícita la compilación del módulo, así como su definición como programa.

Opción «Ayudas y Mensajes»

Mediante esta opción se podrán definir, editar y compilar los módulos fuentes de mensajes y ayudas que serán utilizados por los módulos fuentes CTL de la aplicación. Las características de los ficheros de mensajes y ayudas son las siguientes:

- Ficheros de Mensajes: Aquellas instrucciones del CTL que lleven asociado un texto constante podrán incluirlo en un fichero de mensajes, que se identificará por su nombre, definido por el usuario, más un número único. Esto permite la traducción de una aplicación construida con MultiBase sin necesidad de modificar los módulos fuente, sino únicamente traduciendo los ficheros de mensajes.
- Ficheros de Ayuda: Todas las instrucciones del CTL que tienen interface con el usuario a través de la pantalla pueden llevar asociado un texto que se mostrará automáticamente cada vez que se pulse la tecla de ayuda sobre el elemento en curso. Asimismo, esta información será utilizada para la generación del Manual del Usuario, quedando de este modo como ayuda «on line» e incluida en el manual.

Una vez seleccionada la opción «Ayudas y Mensajes» aparecerá menú «Pop-up» que, bajo el título «Acciones», incluye las opciones que se comentan a continuación.

«Elegir»

Mediante esta opción se podrá elegir un módulo de mensajes. Los módulos de mensajes son ficheros ASCII con la extensión «.shl». Al ejecutarla aparecerá una ventana para indicar el nombre del módulo que deseamos establecer como módulo de mensajes en curso. En esta ventana de edición se podrán utilizar los metacaracteres del operador

«MATCHES» para determinar un grupo de módulos. Si no se especifica ningún nombre, al pulsar la tecla correspondiente a «aceptar» se mostrará una nueva ventana para elegir un módulo entre los existentes. El nombre del módulo elegido se presentará en una ventana en la parte superior derecha de la pantalla.

«Nuevo»

Esta opción permite generar un módulo de mensajes nuevo. Al ejecutarla aparecerá una ventana de edición para indicar el nombre del módulo que se desea crear. Una vez confirmado este nombre y elegido el directorio en el que se ubicará accederemos al editor de MultiBase para introducir el texto correspondiente a dicho módulo (la estructura de los ficheros de ayuda se comenta en el capítulo «Ficheros de Ayuda y Mensajes» de este mismo volumen).

«Modificar»

Mediante esta opción podremos modificar el texto correspondiente al módulo de mensajes en curso. Una vez seleccionada esta opción arrancará el editor de textos de MultiBase mostrando el contenido del fichero elegido para su modificación.

«Compilar»

Esta opción permite compilar el módulo de mensajes en curso. El compilador de módulos de mensajes es thelpcomp. Por cada módulo fuente («.shl») se genera un módulo objeto («.ohl»). Si se producen errores en la compilación, el programa avisará al operador al objeto de que pueda editar el módulo y corregirlos (las líneas que contengan errores se identificarán por el símbolo «#», no siendo necesario eliminarlas, ya que el programa lo hace de manera automática). Una vez corregidos los errores accederemos a un menú que, bajo el título «Compilación» incluye las siguientes opciones:

«Compilar»

Efectúa la compilación teniendo en cuenta los cambios realizados.

«Salvar y salir»	Hace efectivos los cambios realizados y abandona la compilación.
«Descartar y salir»	Desecha los cambios realizados y abandona la compilación.

Los errores más comunes son debidos generalmente a incorrecciones en la estructura del fichero (para más información consulte el capítulo sobre [«Ficheros de Ayuda y Mensajes»](#) en este mismo volumen).

«Borrar»

Mediante esta opción podremos borrar de la base de datos tanto la definición del módulo de mensajes en curso como sus ficheros asociados («.shl» y «.ohl»). Una vez seleccionada esta opción aparecerá un menú para pedir confirmación antes de proceder a la operación de borrado.

Opción «Información»

Mediante esta opción podremos obtener información sobre las definiciones realizadas en módulos, programas y mensajes a través de diferentes criterios, tales como usuarios, directorios, etc. Una vez ejecutada esta opción se presentará un menú «Pop-up» con las opciones que se comentan a continuación.

«Programas por Módulo»

Esta opción permite consultar por pantalla en qué programas se encuentra definido cada módulo de la aplicación. Para ello, al ejecutarla se mostrará una ventana en la que aparecerán los nombres de los módulos y los programas a los que pertenecen, así como su tipo (entrada de datos, listados, etc.).

«Programas/Módulos por Usuario»

Mediante esta opción podremos consultar por pantalla todos los módulos y programas definidos agrupados por usuarios. La información facilitada incluye la fecha y la hora en que se generó el módulo, así como el tipo de programa al que pertenece.

«Programas sin módulos»

Esta opción proporciona información sobre aquellos programas que no tienen asignado ningún módulo, incluyendo la fecha y hora de creación y el usuario al que pertenecen.

«Módulos sin programas»

Mediante esta opción se podrán consultar por pantalla todos aquellos módulos que no han sido asignados a ningún programa.

«Lista de programas/módulos»

Esta opción permite consultar por pantalla todos los módulos y programas definidos.

«Ficheros de Mensajes»

Permite consultar los módulos de ayuda y mensajes definidos.

«Directorios de la Aplicación»

Muestra una ventana con los directorios utilizados en la definición de módulos y programas, indicando por cada uno de ellos los módulos y programas existentes junto con su tipo (entrada de datos, listados, etc.), si se trata de un módulo o programa, de un fichero de ayuda o un procedimiento SQL.

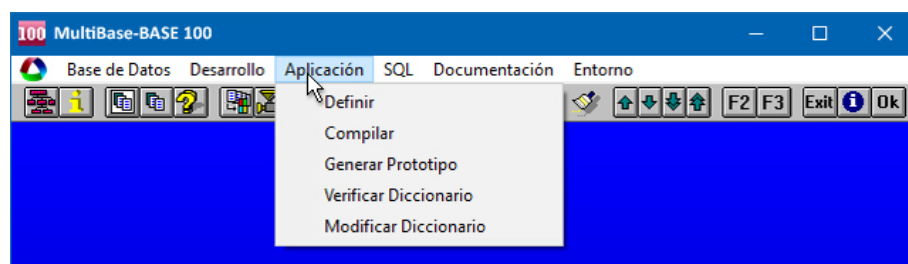
Persiana «Aplicación»

Esta persiana contempla todos los procesos relativos a la definición de directorios de trabajo de la base de datos, compilación de los módulos y programas que la componen, generación de módulos fuentes, etc.

Las opciones incluidas dentro de esta persiana son las siguientes:

- «Definir».
- «Compilar».
- «Generar Prototipo».
- «Verificar Diccionario».
- «Modificar Diccionario».

En los siguientes apartados se comenta en detalle cada una de ellas.



Persiana «Aplicación» del entorno de desarrollo.

Opción «Definir»

Mediante esta opción se podrán definir los directorios de trabajo para la base de datos. Cada uno de los tipos de módulos que se pueden definir desde el entorno de desarrollo (entrada de datos, listados, menús, librerías, etc.) lleva asociado por defecto un directorio, que será el establecido para la base de datos en el momento de generarla.

Los directorios que se especifiquen a través de esta opción deberán estar necesariamente incluidos en la definición de las variables de entorno que aparecen en la persiana «Entorno» del menú principal. Los directorios indicados para los tipos de módulos correspondientes a entrada de datos, listados, librerías, menús y varios deberán definirse en las variables de entorno DBPROC o DBLIB, mientras que el correspondiente a mensajes deberá especificarse en la variable MSGDIR.

Una vez seleccionada esta opción aparecerá una ventana (ver Figura 43) para indicar los directorios correspondientes a entrada de datos, listados, menús, varios, librerías, procedimientos SQL y mensajes.

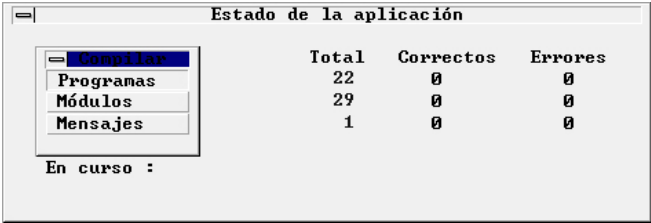
Directorios por defecto	
Entrada de Datos	c:\mbdemo
Listados	c:\mbdemo
Menús	c:\mbdemo
Varios	c:\mbdemo
Librerías	c:\mbdemo
Proced.SQL	c:\mbdemo
Mensaje	c:\mbdemo

Ventana para definición de directorios.

En esta ventana se deberá indicar el directorio de trabajo por cada tipo de módulo (para seleccionar un directorio se podrá emplear la tecla correspondiente a la opción <Selec> que figura al pie de la pantalla), así como si se va a emplear o no el depurador a la hora de la compilación y ejecución de los módulos.

Opción «Compilar»

Mediante esta opción se podrán compilar los distintos componentes de la base de datos (módulos y programas CTL y módulos de ayuda). Al ejecutarla se mostrará un menú «Pop-up» superpuesto sobre la ventana que posteriormente facilitará el resultado de la compilación. Este menú incluye las opciones que se comentan a continuación.



	Total	Correctos	Errores
Programas	22	0	0
Módulos	29	0	0
Mensajes	1	0	0

En curso :

Ejecución de la opción «Compilar» de la persiana «Aplicación».

«Programas»

Esta opción permite una compilación global de todos los programas que componen la aplicación. Se entiende por compilación de un programa la compilación de todos los módulos que lo componen y el enlace de todos ellos (comando **ctl**ink). Si se produce algún error durante el proceso de compilación o en el enlace de alguno de los programas o módulos, al finalizar dicho proceso aparecerá una ventana indicando el nombre del módulo o programa donde se produjo el error.

Una vez seleccionada esta opción accederemos a un nuevo menú «Pop-up» con las dos opciones siguientes:

«Con información para depuración»

Realiza la compilación para la utilización posterior del depurador («debugger»), es decir, todos los módulos o programas serán compilados con la opción «-g» del comando **ctlcomp**. Dicha opción permitirá utilizar el «debugger» a la hora de ejecutar los módulos o programas. Una vez seleccionada esta opción aparecerá en pantalla una ventana de información indicando el nombre del módulo o programa en curso junto con un nuevo menú «Pop-up» con las siguientes opciones:

«Actual»	Realiza la compilación sobre el módulo o programa en curso.
«Siguiente»	Muestra el siguiente módulo o programa de la lista.
«Selecciona»	Permite seleccionar el módulo o programa a compilar.
«Todos»	Compila todos los módulos o programas de la lista.
«Fin»	Cancela el proceso.

«Sin información para depuración»

Realiza la compilación sin la opción «-g» del comando **ctlcomp**, por lo que no se podrá utilizar el «debugger» a la hora de ejecutar los módulos o programas. Tanto el «Pop-up» que se muestra como las opciones que en él se incluyen son idénticas a las expuestas para la compilación con empleo del depurador.

«Módulos»

Esta opción permite una compilación general de todos los módulos que componen la aplicación. El proceso y las posibles operaciones que se pueden llevar a cabo a través de esta opción son idénticos a los comentados anteriormente para la compilación de programas.

«Mensajes»

Esta opción permite una compilación general de todos los módulos de mensajes y ayudas que componen la aplicación. El proceso y las posibles operaciones que se pueden llevar a cabo a través de esta opción son bási-

camente iguales a los comentados anteriormente para la compilación de programas, con la salvedad de que en este caso no se muestra el «Pop-up» para elegir o no la utilización del depurador.

Opción «Generar Prototipo»

Esta opción permite generar los módulos fuentes bien para toda la aplicación en su conjunto, integrando en un menú general la generación de los diferentes tipos de módulos, o bien individualmente para cada uno de ellos (entrada de datos, librerías, menús, consultas, etc.).

Una vez seleccionada la opción «Generar Prototipo» accederemos a un menú que, bajo el título «Elija», incluye las opciones que se comentan a continuación.

«Aplicación»

Esta opción permite generar el prototipo de una aplicación en su conjunto, incluyendo el proceso de generación de los módulos de entrada de datos, cabecera-líneas, listados, consultas y menús. En definitiva, se trata de una opción que engloba a las demás de este mismo menú. Al ejecutarla se mostrará en la pantalla una ventana indicando el módulo a generar junto con un menú «Pop-up» con las siguientes opciones:

«Actual»	Ejecuta la operación que se indica en la ventana sobre el elemento activo. Una vez finalizado dicho proceso retorna nuevamente al menú.
«Siguiente»	Muestra el siguiente elemento en orden alfabético, pudiendo entonces ejecutar cualquiera de las opciones del «Pop-up».
«Selecciona»	Permite seleccionar un elemento (módulo o programa) entre todos los existentes para llevar a cabo la generación.
«Todos»	Realiza la generación sobre todos los elementos de la lista a partir del elemento activo.
«Fin»	Finaliza el proceso y retorna al menú anterior.

«E. Datos»

Esta opción permite la generación de un módulo fuente CTL de entrada de datos sobre una tabla. Al ejecutarla se mostrará una ventana indicando el módulo de entrada de datos en curso junto un menú «Pop-up» idéntico al de la opción «Aplicación».

Al generar el módulo de entrada de datos se genera también la librería «mblib», si es que ésta no fue definida previamente. Una vez compilado el módulo generado, éste se enlaza (comando **ctlink**) con la librería «mblib» para permitir la utilización de funciones comunes. Este proceso de generación implica asimismo la generación de una entrada en el diccionario como módulo y como programa del tipo entrada de datos.

Las operaciones que se podrán realizar con el módulo fuente generado son: alta, baja, consulta y modificación de las filas de la tabla.

«Cab/Líneas»

Genera un módulo fuente CTL por cada pareja de tablas en las que se haya definido integridad referencial. A la hora de generar el módulo se genera también la librería «mblib» si es que ésta no fue definida previamente. Una vez compilado el módulo generado, éste se enlaza (comando **ctlink**) con la librería «mblib» para permitir la utilización de funciones comunes. La generación del módulo implica asimismo una entrada en el diccionario como módulo y como programa. Las operaciones que se podrán realizar con el módulo fuente generado son: alta, baja, consulta y modificación de las filas de las dos tablas relacionadas.

Al seleccionar esta opción aparecerá en pantalla una ventana de información indicando los nombres de dos tablas relacionadas (cabecera-líneas) junto con un «Pop-up» idéntico al de la opción «Aplicación».

«Librería»

Esta opción permite la generación de un módulo de librería cuyo nombre por defecto será «mblib» al ejecutarla por primera vez. Si la librería «mblib» ya estuviese definida se asignaría otro nombre distinto al del cualquier otra librería definida en el sistema.

El contenido de la librería es un conjunto de funciones generales, unas independientes de la base de datos y otras que dependen de la integridad referencial definida sobre las tablas de la base de datos. Mediante esta opción es posible actualizar el contenido de la librería según el estado de la base de datos.

«Listado»

Esta opción genera un módulo fuente de listado sobre una tabla previamente elegida. Dicho módulo se identifica por un nombre, cuya longitud máxima no podrá exceder de ocho caracteres ni coincidir con el de otro ya existente en la base de datos.

La generación de este módulo implica una entrada en el diccionario como módulo y como programa de tipo listado. Una vez seleccionada esta opción aparece en pantalla una ventana indicando el nombre de la tabla en curso junto con un «Pop-up» idéntico al de la opción «Aplicación».

«Menú»

Esta opción genera un menú a partir de los programas definidos para cada tipo (entrada de datos, listados y varios). Asimismo, genera una entrada en el diccionario como módulo y como programa de tipo menú. Las operaciones que se podrán realizar con el módulo fuente generado son: dar de alta, dar de baja, consultar y modificar las filas de la tabla.

«Consulta»

Esta opción permite la generación, sobre una tabla previamente elegida, de un módulo fuente CTL del tipo varios. Al generar el módulo se genera también la librería «mblib» si ésta no hubiese sido definida previamente. Una vez compilado el módulo, éste se enlaza (comando **ctlink**) con la librería «mblib» para la utilización de funciones comunes. Además del módulo en sí se genera también una entrada en el diccionario como módulo y como programa del tipo varios. Las operaciones que se podrán realizar con el módulo fuente generado son las siguientes:

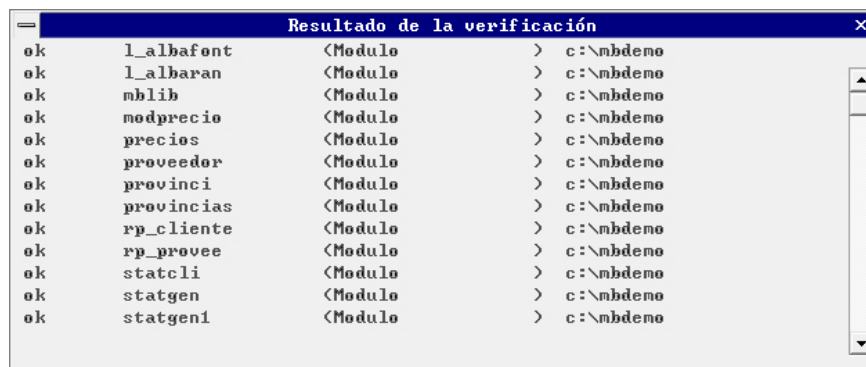
- Editar las condiciones que deberán cumplir las filas a consultar.
- Enviar el resultado de la consulta a pantalla o a impresora.
- formatear la salida en el caso de consulta por impresora

Una vez seleccionada esta opción aparecerá una ventana indicando el nombre de la tabla en curso junto con un «Pop-up» idéntico al de la opción «Aplicación».

Opción «Verificar Diccionario»

Esta opción comprueba que cada uno de los módulos y programas definidos en el entorno se encuentren efectivamente en el «path» que se les hubiese asignado.

Al finalizar el proceso aparecerá una ventana de información (ver figura) en la que se indicará el estado («ok» o «error») de cada módulo según que éste se encuentre o no respectivamente en el «path» indicado.



Resultado de la verificación			
ok	l_albafont	<Modulo	> c:\mbdemo
ok	l_albaran	<Modulo	> c:\mbdemo
ok	mblib	<Modulo	> c:\mbdemo
ok	modprecio	<Modulo	> c:\mbdemo
ok	precios	<Modulo	> c:\mbdemo
ok	proveedor	<Modulo	> c:\mbdemo
ok	provinci	<Modulo	> c:\mbdemo
ok	provincias	<Modulo	> c:\mbdemo
ok	rp_cliente	<Modulo	> c:\mbdemo
ok	rp_provee	<Modulo	> c:\mbdemo
ok	statcli	<Modulo	> c:\mbdemo
ok	statgen	<Modulo	> c:\mbdemo
ok	statgen1	<Modulo	> c:\mbdemo

Resultado de la ejecución de la opción «Verificar Diccionario».

Opción «Modificar Diccionario»

Mediante esta opción se podrán modificar los directorios donde se encuentran los distintos componentes de una aplicación (módulos, programas y ficheros SQL). Se trata de una opción especialmente útil cuando se desea traspasar una aplicación entre diferentes entornos (por ejemplo entre distintos sistemas operativos o distintos «paths»). Al ejecutarla aparecerá en pantalla un menú «Pop-up» con las opciones que se comentan a continuación.

«Entrada de Datos»

Mediante esta opción se podrá modificar el «path» de los módulos y programas del tipo entrada de datos. Una vez seleccionada aparecerá en pantalla una ventana de edición para indicar el «path» que se desea modificar, pudiendo emplear para ello los metacaracteres de búsqueda del operador «MATCHES». Tras aceptar los valores introducidos el programa pedirá conformidad al operador.

«Listados»

Mediante esta opción se podrá modificar el «path» de los módulos y programas del tipo listados. Una vez seleccionada aparecerá en pantalla una ventana de edición para indicar el «path» que se desea modificar, pudiendo emplear para ello los metacaracteres de búsqueda del operador «MATCHES». Tras aceptar los valores introducidos el programa pedirá conformidad al operador.

«Menús»

Permite modificar el «path» de los módulos y programas del tipo menú. Una vez seleccionada aparecerá en pantalla una ventana de edición para indicar el «path» que se desea modificar, pudiendo emplear para ello los metacaracteres de búsqueda del operador «MATCHES». Tras aceptar los valores introducidos el programa pedirá conformidad al operador.

«Varios»

Permite modificar el «path» de los módulos y programas del tipo varios. Una vez seleccionada aparecerá en pantalla una ventana de edición para indicar el «path» que se desea modificar, pudiendo emplear para ello los metacaracteres de búsqueda del operador «MATCHES». Tras aceptar los valores introducidos el programa pedirá conformidad al operador.

«Librerías»

Permite modificar el «path» de los módulos y programas del tipo librería. Una vez seleccionada aparecerá en pantalla una ventana de edición para indicar el «path» que se desea modificar, pudiendo emplear para ello los metacaracteres de búsqueda del operador «MATCHES». Tras aceptar los valores introducidos el programa pedirá conformidad al operador.

«Proced. SQL»

Permite modificar el «path» de los módulos y programas del tipo procedimiento SQL. Una vez seleccionada aparecerá en pantalla una ventana de edición para indicar el «path» que se desea modificar, pudiendo emplear para ello los metacaracteres de búsqueda del operador «MATCHES». Tras aceptar los valores introducidos el programa pedirá conformidad al operador.

«Mensaje»

Permite modificar el «path» de los módulos y programas de mensajes. Una vez seleccionada aparecerá en pantalla una ventana de edición para indicar el «path» que se desea modificar, pudiendo emplear para ello los metacaracteres de búsqueda del operador «MATCHES». Tras aceptar los valores introducidos el programa pedirá conformidad al operador.

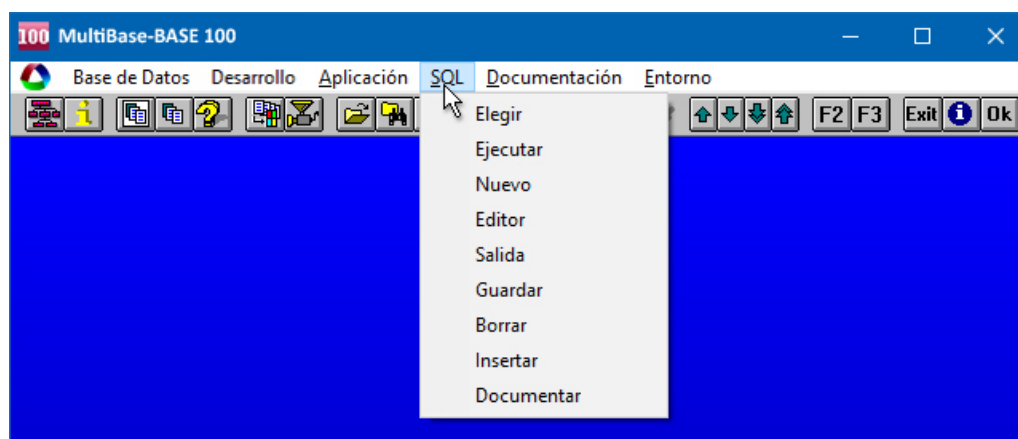
«Todos»

Permite modificar el «path» de los módulos y programas de todos los tipos anteriores. Una vez seleccionada aparecerá en pantalla una ventana de edición para indicar el «path» que se desea modificar, pudiendo emplear para ello los metacaracteres de búsqueda del operador «MATCHES». Tras aceptar los valores introducidos el programa pedirá conformidad al operador.

Persiana «SQL»

A través de esta persiana se podrán llevar a cabo todas las operaciones relativas al tratamiento de ficheros SQL. Las opciones que en ella se incluyen son las siguientes:

- «Elegir».
- «Ejecutar».
- «Nuevo».
- «Editor».
- «Salida».
- «Guardar».
- «Borrar».
- «Insertar».
- «Documentar».



Persiana «SQL» del entorno de desarrollo.

A continuación se comenta en detalle cada una de las opciones de esta persiana.

Opción «Elegir»

Mediante esta opción podremos seleccionar como fichero SQL de trabajo cualquiera de los definidos previamente en la base de datos. Un fichero SQL se define a través de las opciones «Guardar», «Insertar» o «Nuevo» de esta misma persiana.

Las opciones relativas a ejecutar, editar, guardar, borrar y documentar se referirán siempre al fichero SQL en curso. Todas las operaciones que se realicen sobre el fichero SQL en curso a través de la opción «Editor» de esta misma persiana no tendrán reflejo hasta que se ejecute la opción «Guardar».

Al ejecutar la opción «Elegir» aparecerá una ventana para indicar el nombre del fichero SQL que se desea seleccionar como fichero de trabajo, pudiendo emplear los metacaracteres del operador «MATCHES» para establecer un criterio de selección. No obstante, si no se indica nada, pulsando la tecla correspondiente a «aceptar» se presentará una nueva ventana con los ficheros de SQL existentes para poder seleccionar uno de ellos.

Opción «Ejecutar»

Mediante esta opción se podrán ejecutar las instrucciones del SQL contenidas en el fichero de trabajo SQL en curso. La forma de acceder a dichas instrucciones es a través de la opción «Editor» de esta misma persiana.

Opción «Nuevo»

Mediante esta opción se podrá generar un nuevo fichero de trabajo SQL. En dicho fichero se podrán incluir aquellas instrucciones del SQL que se deseen ejecutar a través del editor. Todas las instrucciones del SQL que se incluyan tienen que finalizar necesariamente con un punto y coma («;»).

Una vez seleccionada esta opción accederemos al correspondiente editor de MultiBase (que por defecto será el «Tword» en UNIX/Linux y el «Wedit» en Windows) para escribir las instrucciones del SQL que se deseen ejecutar.

Opción «Editor»

A través de esta opción accederemos al editor de textos de MultiBase (que por defecto será el «Tword» en UNIX/Linux y el «Wedit» en Windows) para modificar el fichero de trabajo SQL en curso, pudiendo entonces escribir las instrucciones del SQL a ejecutar.

Opción «Salida»

Esta opción permite enviar el resultado de la ejecución del fichero de trabajo SQL en curso a una salida distinta de la utilizada por defecto (la pantalla). Una vez seleccionada se presentará un menú «Pop-up» con las opciones que se comentan a continuación.

«Impresora»

Mediante esta opción es posible redirigir el resultado de la ejecución de una instrucción SELECT a la impresora definida en la variable de entorno DBPRINT.

«Fichero (Nuevo)»

Permite redirigir el resultado de la ejecución de una instrucción SELECT a un fichero nuevo. Una vez seleccionada aparece una ventana de edición para indicar el nombre de un fichero nuevo al que se enviará el resultado de la ejecución de la instrucción SELECT correspondiente.

«Fichero (Agregar)»

Mediante esta opción es posible redirigir el resultado de la ejecución de una instrucción SELECT hacia un fichero ya existente, agregando dicho resultado al contenido del fichero. Una vez ejecutada aparecerá una ventana para seleccionar el nombre del fichero al que se desea agregar el resultado de la ejecución de la instrucción SELECT correspondiente (igual que sucede al ejecutar la opción «Insertar» de la persiana «SQL»).

«Proceso»

Esta opción es válida únicamente en las versiones de MultiBase para sistema operativo UNIX/Linux. Gracias a ella es posible redirigir el resultado de la ejecución de una instrucción SELECT hacia un proceso. Una vez seleccionada aparece una ventana de edición para indicar el nombre del proceso a lanzar.

Opción «Guardar»

Mediante esta opción se definen los ficheros SQL dentro de la base de datos, bien asignando un nombre nuevo al fichero SQL en curso o bien guardando su contenido actual en sustitución del existente hasta ese momento (en este último caso tenga especial cuidado para evitar posibles pérdidas de información).

El «path» asociado al fichero será el definido para los «Procedimientos SQL» a través de la opción «Definir» de la persiana «Aplicación».

Una vez ejecutada la opción «Guardar» aparecerá una ventana para especificar el nombre del fichero SQL.

Opción «Borrar»

Mediante esta opción se podrán borrar los ficheros SQL previamente definidos a través de las opciones «Guardar», «Nuevo» e «Insertar» de esta misma persiana.

Al ejecutar esta opción el programa pedirá conformidad al operador antes de llevar a cabo la operación de borrado.

Opción «Insertar»

A través de esta opción se podrán añadir ficheros de SQL a la base de datos. Una vez insertado un fichero, éste queda definido como tal en la base de datos, pudiendo entonces realizar con él cualquiera de las operaciones contempladas en esta misma persiana.

Al seleccionar esta opción se mostrará una ventana para seleccionar un fichero. El aspecto de esta ventana variará en función del entorno operativo en que nos encontremos (ver Figura 30 en este mismo capítulo). Tras elegir el fichero el programa pedirá conformidad al operador para insertarlo en la base de datos.

Opción «Documentar»

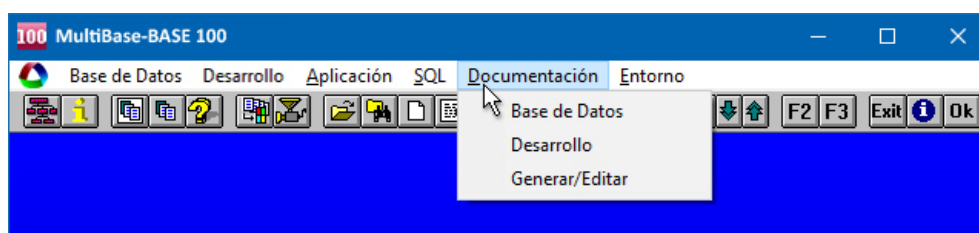
Mediante esta opción se podrá documentar el fichero SQL en curso para la posterior elaboración del correspondiente manual del programador. El texto incluido en la documentación de esta opción aparecerá en el capítulo cuarto del manual del programador al explicar la estructura de los programas. Dicho texto se incluye en un fichero cuyo nombre es igual al de la base de datos en curso más la extensión «.dpm».

Una vez seleccionada esta opción el programa presentará un elemento de diálogo (que indicará el fichero SQL en curso y el número de líneas de documentación existentes), junto a una ventana, provocada por la ejecución del editor de textos de MultiBase, en la que se mostrará el texto de la documentación, pudiendo entonces modificarla o añadir más texto.

Persiana «Documentación»

A través de las opciones incluidas en esta persiana se podrá elaborar la documentación correspondiente a una aplicación, bien de forma global o individualizada por cada uno de los elementos que la componen. Dichas opciones son las siguientes:

- «Base de Datos».
- «Desarrollo».
- «Generar/Editar».



Persiana «Documentación» del entorno de desarrollo.

A continuación se explica en detalle cada una de las opciones de esta persiana.

Opción «Base de Datos»

Esta opción permite documentar los diferentes elementos que conforman la base de datos (tablas, índices, «views», etc.). La documentación de dichos elementos podrá llevarse a cabo de manera global a través de esta opción o, de forma individual para cada uno de ellos a través de las correspondientes opciones de documentación de las distintas persianas del entorno de desarrollo.

Una vez seleccionada esta opción accederemos a un menú «Pop-up» que incluye las opciones que se comentan a continuación.

«Base de Datos»

Mediante esta opción podremos introducir los textos necesarios para escribir la documentación asociada a la base de datos. Este proceso podrá llevarse a cabo igualmente a través de la secuencia de opciones «Base de Datos» — «Documentar».

El texto introducido se almacenará en un fichero (cuyo nombre será igual al de la base de datos más la extensión «.dpm») al generar la documentación mediante la secuencia de opciones «Documentar» — «Generar/Editar» — «Programador» — «Generar manual».

El texto de la documentación podrá obtenerse por impresora utilizando el procesador de textos «Tprocess» incluido en MultiBase, pudiendo emplear los comandos de éste para mejorar el resultado final (márgenes, tipos de letra, etc.).

Una vez ejecutada esta opción aparecerá un elemento de diálogo (que indicará el nombre de la base de datos y el número de líneas de texto ya documentadas), junto con una ventana, provocada por la ejecución del editor de textos de MultiBase, en la que se mostrará el texto de la documentación, pudiendo entonces modificarlo o añadir más texto.

«Tablas»

Mediante esta opción podremos escribir el texto relativo a la documentación asociada a cada una de las tablas que componen la base de datos. Este proceso podrá llevarse a cabo igualmente a través de la secuencia de opciones «Base de Datos» — «Tablas» — «Documentar».

El texto escrito aparecerá en un fichero, cuyo nombre será igual al de la tabla más la extensión «.dta», al generar la documentación mediante la secuencia de opciones «Documentar» — «Generar/Editar» — «Programador» — «Generar manual».

Al igual que en la opción anterior se podrá emplear el procesador de textos «Tprocess» para mejorar el resultado final.

Una vez ejecutada esta opción aparecerá un elemento de diálogo con las tablas existentes en la base de datos (y su número de líneas de texto documentadas) para poder elegir una de ellas, junto con un menú que incluye las siguientes opciones:

«Encontrar»	Actualiza la información de las tablas y su número de líneas.
«Modificar»	Edita el texto asociado a la tabla en curso para modificarlo.
«Borrar texto»	Borra el texto asociado a la tabla en curso.

«Índices»

Mediante esta opción podremos escribir el texto relativo a la documentación asociada a cada uno de los índices que componen la base de datos. Este proceso podrá llevarse a cabo igualmente a través de la secuencia de opciones «Base de Datos» — «Tablas» — «Índices» — «Documentar».

El texto escrito aparecerá en un fichero, cuyo nombre será igual al de la tabla al que pertenece el índice más la extensión «.dta» (dentro del capítulo dedicado a los índices de la tabla) una vez generada la documentación mediante la secuencia de opciones «Documentar» — «Generar/Editar» — «Programador» — «Generar manual».

Al igual que en la opción anterior se podrá emplear el procesador de textos «Tprocess» para mejorar el resultado final.

La ejecución de esta opción es igual a la de «Tablas», incluyendo las mismas opciones en el menú que aparece, pero en este caso referidas a los índices.

«Vistas»

Mediante esta opción podremos escribir el texto relativo a la documentación asociada a cada una de las «views» que componen la base de datos. Este proceso podrá llevarse a cabo igualmente a través de la secuencia de opciones «Base de Datos» — «Views» — «Documentar».

El texto escrito aparecerá en un fichero, cuyo nombre será igual al de la tabla a la que pertenece la «view» más la extensión «.dta» (dentro del capítulo dedicado a las «views de la base de datos») una vez generada la documentación mediante la secuencia de opciones «Documentar» — «Generar/Editar» — «Programador» — «Generar manual».

Al igual que en la opción anterior se podrá emplear el procesador de textos «Tprocess» para mejorar el resultado final.

La ejecución de esta opción es igual a la de «Tablas», incluyendo las mismas opciones en el menú que aparece, pero en este caso referidas a las «views».

Opción «Desarrollo»

Esta opción permite realizar la documentación de manera individualizada para los distintos programas y módulos que conforman una aplicación. Una vez ejecutada se presentará en pantalla un menú «Pop-up» compuesto por dos opciones, «Programa» y «Módulo».

«Programa»

Mediante esta opción podremos escribir el texto relativo a la documentación asociada a cada uno de los tipos de programas definidos sobre la base de datos. Este proceso podrá llevarse a cabo igualmente a través de la secuencia de opciones «Base de Datos» — «Programas» — «Entrada Datos/Listados/Menús/Librerías/Varios» — «Documentar».

El texto escrito aparecerá en el fichero «ep_dict.dta» (en el capítulo dedicado a la estructura de programas y según el tipo de cada uno de ellos) una vez generada la documentación mediante la secuencia de opciones «Documentar» — «Generar/Editar» — «Programador» — «Generar manual».

El texto de la documentación podrá obtenerse por impresora utilizando el procesador de textos «Tprocess» incluido en MultiBase, pudiendo emplear los comandos de éste para mejorar el resultado final (márgenes, tipos de letra, etc.).

Una vez ejecutada aparecerá un nuevo «Pop-up», compuesto por las opciones que se comentan a continuación, para elegir el tipo de programa que se desea documentar:

«Entrada de Datos»

Mediante esta opción podremos escribir el texto relativo a la documentación asociada a cada uno de los programas definidos del tipo entrada de datos. Este proceso podrá llevarse a cabo igualmente a través de la secuencia de opciones «Base de Datos» — «Módulos/Programas» — «Entrada Datos» — «Documentar». El texto escrito aparecerá en el fichero «ep_dict.dta» (en el capítulo dedicado a la estructura de programas) una vez generada la documentación mediante la secuencia de opciones «Documentar» — «Generar/Editar» — «Programador» — «Generar manual».

Una vez seleccionada aparecerá un elemento de diálogo en el que se mostrarán los programas de entrada de datos existentes y el número de líneas de documentación existentes en cada uno de ellos. Una vez elegido un programa accederemos a un nuevo menú compuesto por las siguientes opciones:

«Encontrar»	Actualiza la información de los programas y su número de líneas.
«Modificar»	Edita el texto asociado al programa en curso para permitir su modificación.
«Borrar texto»	Borra el texto asociado al programa en curso.

«Listado»

Mediante esta opción podremos escribir el texto relativo a la documentación de los programas definidos del tipo listado. Este proceso podrá llevarse a cabo igualmente a través de la secuencia de opciones «Base de Datos» — «Módulos/Programas» — «Listado» — «Documentar». Su funcionamiento es idéntico al de la opción anterior.

«Menú»

Mediante esta opción podremos escribir el texto relativo a la documentación de los programas definidos del tipo menú. Este proceso podrá llevarse a cabo igualmente a través de la secuencia de opciones «Base de Datos» — «Módulos/Programas» — «Menú» — «Documentar». Su funcionamiento es idéntico al de la opción anterior.

«Varios»

Mediante esta opción podremos escribir el texto relativo a la documentación de los programas definidos del tipo varios. Este proceso podrá llevarse a cabo igualmente a través de la secuencia de opciones «Base de Datos» — «Módulos/Programas» — «Varios» — «Documentar». Su funcionamiento es idéntico al de la opción anterior.

«SQL»

Mediante esta opción podremos escribir el texto relativo a la documentación de los programas definidos del tipo SQL. Este proceso podrá llevarse a cabo igualmente a través de la secuencia de opciones «Base de Datos» — «SQL» — «Documentar». Su funcionamiento es idéntico al de la opción anterior.

«Librerías»

Mediante esta opción podremos escribir el texto relativo a la documentación de los programas definidos del tipo librerías. Este proceso podrá llevarse a cabo igualmente a través de la secuencia de opciones «Base de Datos» — «Módulos/Programas» — «Librerías» — «Documentar». Su funcionamiento es idéntico al de la opción anterior.

«Mensajes»

Mediante esta opción podremos escribir el texto relativo a la documentación de los programas definidos del tipo mensajes. Este proceso podrá llevarse a cabo igualmente a través de la secuencia de opciones «Base de Datos» — «Ayudas y Mensajes» — «Documentar». Su funcionamiento es idéntico al de la opción anterior.

«Módulo»

Esta opción tiene el mismo objeto y funcionalidad que «Programa», aplicándose en este caso a los distintos tipos de módulos definidos (entrada de datos, menús, librerías, etc.).

Opción «Generar/Editar»

Mediante esta opción se podrá generar la documentación correspondiente al manual del usuario y al manual del programador de la aplicación.

- El texto que se incluirá en el manual del usuario será el correspondiente a los textos de ayuda («help nn») y a la directiva DOCTEXT. La generación de esta documentación es realizada de manera automática por el documentador «Tdocu» incluido en MultiBase, siendo necesario únicamente incluir como parámetro el nombre del programa que se desea documentar.
- Por su parte, los textos correspondientes a la documentación del manual del programador serán los incluidos a través de las opciones «Documentar» de las diferentes persianas del entorno de desarrollo de MultiBase.

Una vez seleccionada esta opción accederemos a un menú «Pop-up» compuesto por dos opciones, «Usuario» y «Programador», que se comentan a continuación.

«Usuario»

Esta opción contempla todas las operaciones relativas a la generación del manual del usuario de la aplicación. Esta documentación es generada automáticamente por el documentador «Tdocu» incluido en MultiBase a partir de un programa de entrada.

Para la generación del manual es imprescindible que todos los programas que intervengan en dicho proceso se encuentren compilados y libres de errores.

Una vez seleccionada esta opción aparecerá un nuevo «Pop-up» compuesto por las siguientes opciones:

- | | |
|------------------|--|
| «Generar manual» | Genera la documentación sobre cualquiera de los programas definidos. Todos los ficheros de documentación generados se almacenan en un subdirectorío dentro del directorio donde se encuentra la base de datos (el nombre del subdirectorío será igual al de la base de datos más la extensión «.doc»). Los ficheros de documentación que se generan son ficheros ASCII con el formato necesario para ser interpretados por el «Tprocess». Al seleccionar esta opción habrá que |
|------------------|--|

	indicar el nombre del programa que se desea documentar, bien directamente o eligiéndolo en una ventana de selección.
«Doc. Programa»	Permite documentar cualquiera de los programas definidos. Su funcionamiento es idéntico al de la opción anterior.
«Editar»	Mediante esta opción se podrá editar un fichero de documentación para modificarlo.
«Borrar»	Esta opción permite borrar cualquiera de los ficheros de documentación previamente generados.
«Imprimir»	Esta opción permite imprimir los documentos generados en el proceso de documentación. Los comandos incluidos en dichos documentos son interpretados por el «Tprocess». Tras elegir el fichero o ficheros que se desean imprimir aparecerá una ventana de edición para establecer los diferentes parámetros que se pueden utilizar:
ninguna	Salida por pantalla.
Wait	Espera al acabar la página.
Print	Salida por impresora.
File <fichero>	Salida a <fichero>.
Type	Salida por pantalla.
Model <name>	Modelo de pantalla o impresora.
Repeat n	Repite «n» veces.
p [u]	Desde la página «p» y, opcionalmente, hasta «u».
Odd/Even	Imprime sólo páginas impares o pares.

«Programador»

Esta opción contempla todas las acciones relacionadas con la generación del manual del programador de la aplicación. La información que se incluye a la hora de generar dicho manual hace referencia a tablas, columnas, índices, integridad referencial y atributos, además de a los módulos y programas definidos.

Una vez seleccionada aparecerá un menú «Pop-up» con las siguientes opciones:

«Generar manual»	Genera la documentación teniendo en cuenta todas las definiciones existentes en la base de datos. Todos los ficheros de documentación generados se almacenan en un subdirectorio cuyo nombre será igual al de la base de datos más la extensión «.dpm». Dicho subdirectorio se encuentra dentro del directorio correspondiente a la base de datos. Los ficheros de documentación son ficheros ASCII con el formato necesario para ser interpretados por «Tprocess». Una vez seleccionada esta opción aparecerá una ventana para establecer el formato de las páginas.
«Editar»	Mediante esta opción se podrá editar un fichero de documentación para modificarlo.
«Borrar»	Permite borrar cualquiera de los ficheros de documentación previamente generados.
«Imprimir»	Esta opción permite imprimir los documentos generados en el proceso de documentación. Los comandos incluidos en dichos documentos son interpretados por «Tprocess». Tras elegir el fichero o ficheros que se desean imprimir habrá

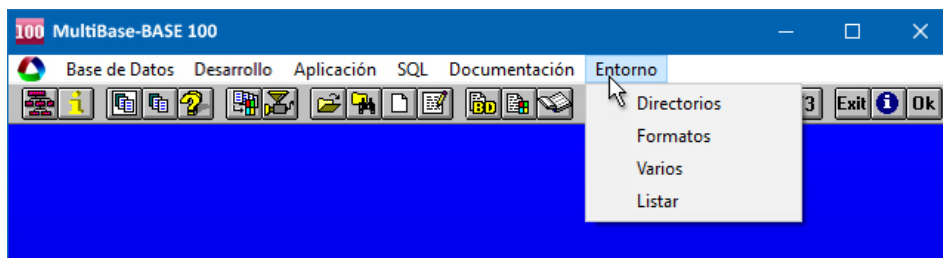
que especificar los diferentes parámetros que se deseen utilizar al igual que sucedía al elaborar la documentación correspondiente al manual del usuario.

Persiana «Entorno»

A través de esta persiana se podrán establecer diferentes aspectos generales relativos al entorno de desarrollo de MultiBase, tales como definición de variables de entorno, formatos que se aplicarán en la representación de las fechas, horas, etc.

Las opciones incluidas dentro de esta persiana son las siguientes:

- «Directorios».
- «Formatos».
- «Varios».
- «Listar».



Persiana «Entorno» del menú principal de MultiBase.

A continuación se comenta en detalle cada una de las opciones de esta persiana.

Opción «Directorios»

Esta opción contempla todos los procesos relativos a la definición de variables de entorno referidas a directorios. Al seleccionarla aparecerá un «Pop-up» con las siguientes opciones:

«Base de Datos»

Los diferentes directorios en los que se encuentran las distintas bases de datos que pueden ser seleccionadas se especifican en la variable de entorno DBPATH. El valor de dicha variable sólo podrá ser modificado en el caso de no tener seleccionada ninguna base de datos. En caso contrario esta opción se estará inactiva.

«Programas»

Los directorios donde se ubican los módulos de la aplicación que serán utilizados por el compilador y el intérprete para su localización se definen en la variable de entorno DBPROC. Una vez seleccionada esta opción aparecerá en pantalla una ventana para indicar el «path» correspondiente a los módulos y programas.

«Mensajes»

Los directorios donde se ubican los ficheros de mensajes y ayuda que serán utilizados por el compilador de mensajes y el intérprete en ejecución se definen en la variable de entorno MSGDIR. Una vez seleccionada esta opción aparecerá en pantalla una ventana para indicar el «path» correspondiente a dichos ficheros.

«Ficheros temporales»

Durante su ejecución, MultiBase genera una serie de ficheros temporales que se almacenarán en el directorio especificado en la variable de entorno DBTEMP (por defecto «tmp»). Una vez seleccionada esta opción aparecerá en pantalla una ventana para indicar el «path» correspondiente a los ficheros temporales.

«Librerías»

Los directorios donde se ubican los módulos de librería (módulos sin MAIN) que serán utilizados por el compilador y el intérprete en ejecución se definen en la variable de entorno DBLIB. Una vez seleccionada esta opción aparecerá en pantalla una ventana para indicar el «path» correspondiente a dichos módulos.

«Includes»

Los directorios donde se ubican los ficheros «include» utilizados por el compilador se definen en la variable de entorno DBINCLUDE. Una vez seleccionada esta opción aparecerá en pantalla una ventana para indicar el «path» correspondiente a dichos ficheros.

Opción «Formatos»

Esta opción contempla todos los procesos relativos a la definición de las variables de entorno asociadas a formatos de representación. Al seleccionarla aparecerá un «Pop-up» compuesto por las siguientes opciones:

«Moneda»

Esta opción permite definir la máscara de representación para todas las columnas cuyo tipo de dato sea MONEY (valor monetario). La variable de entorno asociada es DBMONEY. Una vez seleccionada aparecerá en pantalla una ventana para indicar el formato a utilizar en las columnas con valores monetarios.

«Fecha»

Esta opción permite definir la máscara de representación para todas las columnas cuyo tipo de dato sea DATE (fecha). La variable de entorno asociada es DBDATE. Una vez seleccionada aparecerá en pantalla una ventana para indicar el formato a utilizar en las fechas.

«Hora»

Esta opción permite definir la máscara de representación para todas las columnas cuyo tipo de dato sea TIME (hora). La variable de entorno asociada es DBTIME. Una vez seleccionada aparecerá en pantalla una ventana para indicar el formato a utilizar en las columnas de formato hora.

Opción «Varios»

Esta opción permite definir las variables de entorno de carácter general no contempladas en las dos opciones anteriores. Una vez seleccionada se presentará un menú «Pop-up» compuesto por las siguientes opciones:

«Editor»

Mediante esta opción se podrá especificar el editor de textos que se empleará en el entorno de desarrollo cuando se seleccione la acción de editar («Tword», «Wedit» — en Windows—, etc.). La variable de entorno ligada a esta opción es DBEDIT. Si ésta no estuviese definida podremos elegir entre utilizar el «Tword» o una ventana simple de edición. Los posibles valores que se podrán indicar en la variable DBEDIT son, por ejemplo, «tword -e», «vi», «edit», «wedit», etc., dependiendo del entorno operativo.

«Metacaracteres de Screen»

Mediante esta opción se podrá establecer el juego de caracteres que se utilizará en compilación. La variable de entorno ligada a esta opción es DBMETACH. A la hora de compilar existe un conjunto de caracteres que son interpretados por el compilador como caracteres especiales. En el caso de que deseásemos utilizar algún carácter distinto de los definidos en dicha variable, tendríamos que incluirlo en la definición de ésta en sustitución del carácter que no fuésemos a utilizar. Una vez seleccionada esta opción aparecerá en pantalla una ventana para indicar la lista de caracteres que compondrán el conjunto de símbolos que deberán ser interpretados por el compilador como caracteres especiales.

«Separador de Carga»

Mediante esta opción se podrá definir el carácter que se utilizará como separador a la hora de importar o exportar datos. El separador de carga que se emplea a la hora de importar o exportar datos se define en la variable de entorno DBDELIM, y las instrucciones que se utilizan son LOAD y UNLOAD. El carácter empleado por defecto como separador es la barra vertical («|», carácter ASCII 124). Una vez seleccionada esta opción aparecerá una ventana para indicar el carácter que se utilizará como separador de columnas en las operaciones relativas a las instrucciones LOAD y UNLOAD.

«Programa de impresión»

Mediante esta opción es posible definir el dispositivo que se utilizará para la obtención de los listados por impresora. La variable de entorno ligada a esta opción es DBPRINT, cuyo valor por defecto es «lp» en UNIX/Linux y «PRN» en Windows. La diferencia entre estos entornos radica en que mientras en UNIX/Linux el valor interpretado es un programa, en Windows dicho valor hace referencia a ficheros. No obstante, en el caso concreto de Windows, dicho valor podrá referirse también al programa «printman» (o Administrador de Impresión de dicho entorno).

«Filtro de Impresora»

Esta opción define un conjunto de secuencias de escape asociadas a funciones de cada impresora. Al utilizar secuencias de escape sobre la impresora desde los módulos de una aplicación se hace necesario emplear un fichero en el que se encuentren definidos los diferentes tipos de impresoras que se pueden utilizar: La variable de entorno ligada a esta opción es RPRINTER. El fichero de impresoras estará compuesto por un conjunto de definiciones identificadas en su cabecera por una cadena de caracteres, cuyo valor será el indicado en la variable de entorno RPRINTER como filtro de impresora (por ejemplo: «f3150», «epsonfx», etc.).

«Ventana de BD»

Mediante esta opción podremos establecer que la ventana con información sobre la base de datos en curso se muestre o no en pantalla. Por defecto, cada vez que se selecciona una base de datos aparece en pantalla una ventana indicando su nombre y su «path» correspondiente. Gracias a esta opción podremos omitir dicha información.

«Compilación para Depuración»

Esta opción permite establecer si se utilizará o no el depurador («debugger») a la hora de compilar un módulo. Al ejecutarla el programa pedirá conformidad para activar o desactivar la compilación con «debugger».

«Fichero WININI»

Esta opción, disponible únicamente en las versiones de MultiBase para Windows, permite indicar cuál será el fichero de inicialización que se deberá utilizar al ejecutar las opciones «Probar» (módulos) o «Ejecutar» (programas). Si no se indica el «path» completo de dicho fichero, éste se buscará en el directorio «\$TRANSDIRetc».

Opción «Listar»

Esta opción muestra en una ventana los valores especificados para todas las variables de entorno definidas sobre la base de datos en curso (ver figura). En el caso de que no se hubiese seleccionado ninguna base de datos los valores que aparecerán en la ventana serán los correspondientes a las variables definidas para el entorno que se estuviese ejecutando.



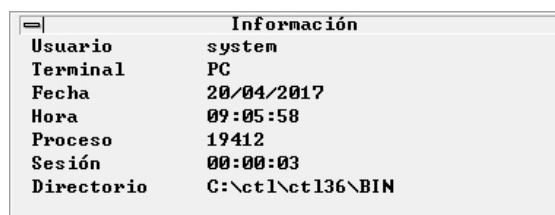
varname	value
DBDATE	DMY4/
DBDELIMITER	
DBEDIT	wedit.exe
DBINCLUDE	c:\mbdemo
DBLIB	c:\mbdemo
DBMETACH	
DBMONEY	
DBPRINT	PRN
DBPROC	c:\ct13609\mbdemo
DBTEMP	\tmp
DBTIME	\$
DEBUG	c:\mbdemo
EP_FRM	c:\mbdemo
EP_HLP	c:\mbdemo

Ejecución de la opción «Listar» de la persiana «Entorno» teniendo activa una base de datos.

Información del Entorno TRANS

Siempre que esté utilizando MultiBase podrá consultar información relativa al entorno de trabajo. Para ello habrá que pulsar la tecla correspondiente a la opción <Info> que se muestra en la línea de teclas de ayuda en la parte inferior de la pantalla (normalmente [F2]).

El resultado de pulsar la tecla indicada variará dependiendo de si tenemos activa una base de datos o no. En este segundo caso, al pulsar [F2] se mostrará una ventana con la siguiente información:



Información	
Usuario	system
Terminal	PC
Fecha	20/04/2017
Hora	09:05:58
Proceso	19412
Sesión	00:00:03
Directorio	C:\ct1\ct136\BIN

Ejecución de la opción <Info>.

La información que aparece en esta ventana es la siguiente:

- «Usuario»: Identificación del usuario de acceso, que corresponderá con el valor de la variable DBUSER o el «LOGNAME» de UNIX/Linux.
- «Terminal»: Identificación del terminal que se está utilizando (en UNIX/Linux). En Windows su valor será «PC».
- «Fecha»: Fecha de comienzo de la sesión.
- «Hora»: Hora de comienzo de la sesión.
- «Proceso»: Número de proceso que se está ejecutando (en UNIX/Linux).
- «Sesión»: Tiempo de duración de la sesión.
- «Directorio»: Nombre del directorio en curso.

Si tuviésemos seleccionada una base de datos, al pulsar [F2] aparecerá en pantalla un menú «Pop-up» compuesto por las siguientes opciones:

- «Sesión» La ejecución de esta opción produce el mismo efecto que si se pulsa [F2] sin tener seleccionada una base de datos (ver figura anterior).
- «Tablas» Con esta opción se podrá consultar información relativa a la estructura de las tablas de la base de datos (columnas, índices, claves, relaciones, etc.). Al ejecutarla se mostrará una ventana para seleccionar la tabla o tablas (mediante [RETURN] o la tecla correspondiente a la opción <Selec> o <Todos>, según el caso)

de las que se desea obtener información. A continuación, pulsando la tecla de «aceptar» (normalmente [F1]) se mostrará un nuevo «Pop-up» para elegir qué tipo de información se desea consultar:

- Información de cada una de las columnas.
- Información de los índices.
- Información de las claves primarias.
- Información de las claves referenciales.
- Información sobre permisos.
- Información general.

«Views»	Mediante esta opción se podrán consultar las instrucciones SELECT que definen cualquiera de las «views» que forman parte de la base de datos.
«Usuarios»	Mediante esta opción podremos consultar por pantalla todos los usuarios que tienen permiso de acceso a la base de datos, así como el nivel de acceso de cada uno de ellos. Una vez seleccionada aparece en pantalla una ventana compuesta por dos columnas, una en la que se muestra el nombre del usuario y otra que indica su nivel de acceso a la base de datos. Si el número de usuarios existentes es mayor del que pueden representarse en la ventana, podremos consultar los restantes mediante las teclas [FLECHA ARRIBA] y [FLECHA ABAJO].
«Documentación»	Con esta opción podremos consultar el número de líneas documentadas de cada uno de los elementos de la base de datos: tablas, índices, «views», base de datos, etc. Una vez seleccionada aparece en la pantalla una ventana de consulta con todos los elementos, indicando por cada uno de ellos el número de líneas documentadas. Si el número de elementos fuese mayor de los que pueden representarse en la ventana podremos consultar los restantes mediante las teclas [FLECHA ARRIBA] y [FLECHA ABAJO].
«Programas»	Mediante esta opción es posible consultar la información referida a los módulos y programas definidos. Una vez seleccionada aparece un menú «Pop-up» para elegir el nivel de consulta deseado. Las opciones que comprende dicho menú son las siguientes: • «Programas por módulo». • «Programas/módulos por Usuario». • «Programas sin módulos». • «Módulos sin programas». • «Lista de programas/módulos». • «Ficheros de mensajes». • «Directorios de la aplicación».

TRANS Interactivo

El TRANS interactivo es una herramienta que permite ejecutar cualquier instrucción del SQL. Para ello, emplea un menú que nos proporciona el acceso a un editor en el que se podrán incluir aquellas instrucciones que se deseen ejecutar.

La forma de acceder al TRANS interactivo es la siguiente:

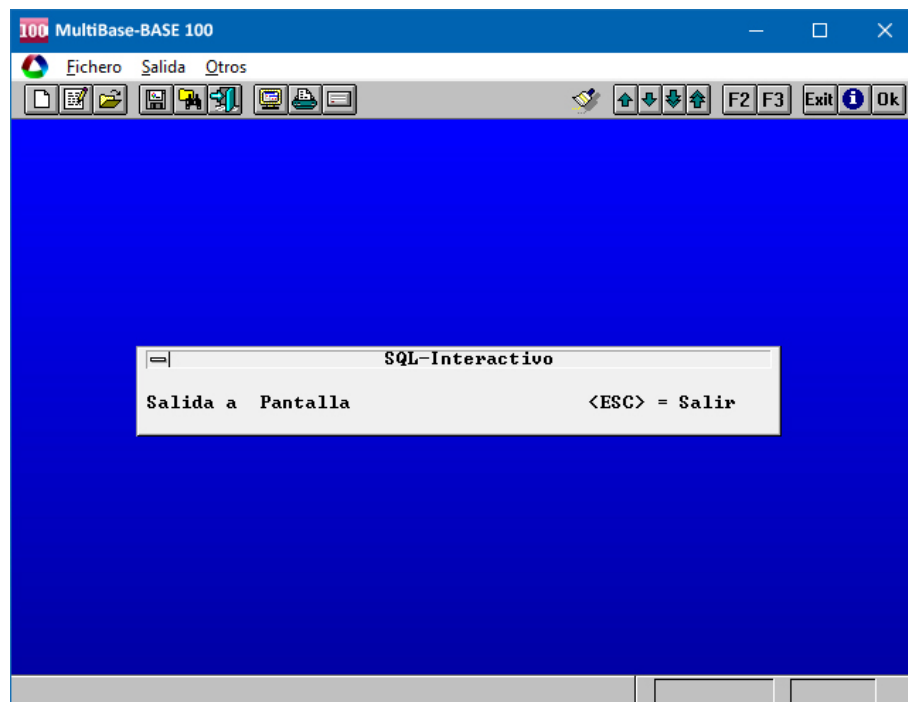
a) En UNIX/Linux: Ejecutando el comando trans con el siguiente formato:

trans parámetro1 parámetro2

Donde:

parámetro1	Sus posibles valores son:	
	base_datos	Nombre de la base de datos que se desea seleccionar como base de datos en curso.
	– (guión)	No selecciona base de datos.
parámetro2	Sus posibles valores son:	
	fichero	Nombre de un fichero SQL existente en el directorio en curso o en el «path» indicado en «fichero» (la extensión «.sql» no hay que indicarla). Dicho «fichero» se ejecutará directamente sin acceder al menú del TRANS interactivo.
	– (guión)	No selecciona fichero.

b) En Windows: Ejecutando el icono «Sql Interactivo» del grupo «MultiBase». El aspecto que presentará la pantalla es el que se muestra en la figura.



Ejecución del TRANS interactivo en Windows.

Persiana «Fichero»

Esta persiana incluye las siguientes opciones:

- «Nuevo».
- «Editar».
- «Abrir».
- «Guardar».
- «Ejecutar».
- «Salir».

Cada una de estas opciones realiza las siguientes tareas:

«Nuevo»

Ejecuta el editor indicado a través de la opción «Editor» de la persiana «Otros» del menú principal para generar un nuevo fichero de trabajo SQL. En dicho fichero se podrán incluir aquellas instrucciones del SQL que se deseen ejecutar. Todas las instrucciones que se incluyan tienen que finalizar obligatoriamente con un punto y coma («;»). En caso de no tener definido ningún editor, el programa presentará por defecto una ventana de edición que incluirá las opciones básicas de un editor de textos.

«Editar»

Esta opción permite editar el fichero de trabajo SQL en curso para su modificación.

«Abrir»

Permite seleccionar un fichero SQL entre los existentes en el sistema. El fichero elegido pasará a ser el fichero de trabajo. Las modificaciones realizadas en este fichero podrán incorporarse al fichero SQL previamente seleccionado a través de la opción «Guardar» de esta misma persiana.

«Guardar»

Permite guardar el contenido del fichero de trabajo en curso con otro nombre. Para ello, el programa presentará una ventana en la que deberá indicarse el nombre (sin extensión «.sql») que se desea asignar al nuevo fichero SQL.

«Ejecutar»

Permite ejecutar las instrucciones de SQL contenidas en el fichero de trabajo en curso.

«Salir»

Abandona el TRANS interactivo.

Persiana «Salida»

Las opciones incluidas en esta persiana permitirán indicar la salida a la cual se dirigirá el resultado de ejecutar una instrucción SELECT del SQL. Dichas opciones son:

- «Pantalla».
- «Impresora».
- «Fichero».

Cada una de estas opciones realiza las siguientes tareas:

«Pantalla»

Mediante esta opción se enviará a la pantalla el resultado de ejecutar las instrucciones SELECT que se encuentren en el fichero de trabajo en curso. Ésta es la opción por defecto.

«Impresora»

Mediante esta opción se enviará a la impresora el resultado de ejecutar las instrucciones SELECT que se encuentren en el fichero de trabajo en curso.

«Fichero»

Mediante esta opción se enviará a un fichero el resultado de ejecutar las instrucciones SELECT que se encuentren en el fichero de trabajo en curso. Para ello, el programa presentará una ventana de edición para indicar el nombre del fichero donde se guardará el resultado de dicha ejecución. Este fichero contendrá únicamente el resultado de la última ejecución.

Persiana «Otros»

Esta persiana incluye las siguientes opciones:

- «Editor».
- «Delimitador».

Cada una de estas opciones realiza las siguientes tareas:

«Editor»

Mediante esta opción podremos indicar el nombre del editor que se utilizará al ejecutar las opciones «Nuevo» y «Editar» de la persiana «Fichero» (por ejemplo «vi», «wedit», etc.). Si no se indica ningún editor el programa utilizará por defecto una ventana con las funciones básicas de un editor de textos.

«Delimitador»

Esta opción permite especificar el carácter que se utilizará como separador de campos en las operaciones de carga y descarga de datos desde/hacia ficheros ASCII (instrucciones LOAD y UNLOAD, respectivamente). El carácter utilizado por defecto es la barra vertical (carácter ASCII 124).

Catálogo del entorno de desarrollo (Tablas «ep_*»)

El catálogo del entorno de desarrollo de MultiBase está compuesto por un conjunto de tablas, cuyos nombres comienzan siempre por «ep_», y cuya función es la de guardar la información referida a los distintos elementos que componen el desarrollo de una aplicación (por ejemplo, módulos, programas, atributos, variables de entorno, etc.).

Tabla «ep_col_descr»

Guarda el texto descriptivo asociado a la documentación de las columnas de una tabla. Sus columnas son las siguientes:

Nombre	tipo	null	longitud	etiqueta
tablename	char		18	
colname	char		18	
description	char		255	

tablename Nombre de la tabla.

colname Nombre de la columna.

description Línea de texto perteneciente a la documentación de las columnas de una tabla.

Los índices de esta tabla son los siguientes:

Índice	Tipo	Columnas
i_descr	Único	tablename, colname

Ejemplo:

```
select tablename, colname, description from ep_col_descr
```

Resultado:

tablename	colname	description
albaranes	albaran	El número de albarán se corresponde con la numeración de la empresa en curso.
albaranes	cliente	Código del cliente al cual se asigna el albarán.
albaranes	fecha_albaran	Fecha de salida del albarán.
albaranes	fecha_envio	Fecha de envío de los artículos del albarán.
albaranes	fecha_pago	Fecha de pago del total del albarán.
albaranes	formpago	Forma de pago en que se contrató el albarán.
albaranes	estado	Estado del albarán.

Tabla «ep_d_comm»

Guarda el texto de documentación asociado a cada uno de los elementos que intervienen en la aplicación, excepto la documentación asociada a las columnas, que se guarda en la tabla «ep_col_descr».

Las columnas de la tabla «ep_d_comn» son las siguientes:

Nombre	tipo	null	longitud	etiqueta
c_key	integer		4	
c_line	smallint		2	
commen_t	char		60	

c_key Identificador del objeto. Tiene relación con la columna «d_key» de la tabla «ep_dict».

c_line Columna de ordenación de las líneas asociadas a un mismo número de identificación del elemento.

commen_t Comentario que forma parte de la descripción de la documentación asociada al elemento.

Los índices de esta tabla son los siguientes:

Índice	Tipo	Columnas
i1_dcomm	Único	c_key, c_line

Ejemplo:

```
select c_key, c_line, commen_t from ep_d_comm
```

Resultado:

c_key	c_line	commen_t
1	1	La presente aplicación es una pequeña demostración
1	2	de las posibilidades del MultiBase. Se trata de una aplicación
1	3	de almacén desarrollada sobre una base de datos que
1	4	dispone de las siguientes tablas:
1	5	
1	6	.RESPETA
1	7	.LM +20
1	8	
1	9	unidades: Unidades de artículos de almacén
1	10	formpagos: Formas de pago de Clientes
1	11	provincias: Provincias de Clientes y Proveedores

c_key	c_line	commen_t
1	12	clientes: Clientes de la empresa
1	13	proveedores: Proveedores de los artículos de almacén
1	14	articulos: Artículos del almacén
1	15	albaranes: Cabeceras de albaranes de entrega

Tabla «ep_dict»

Su función es guardar todas las definiciones que puedan realizarse desde el entorno de desarrollo: módulos, atributos de tablas, documentación, etc. Sus columnas son las siguientes:

Nombre	tipo	null	longitud	etiqueta
d_key	serial	N	4	
d_name	char		18	
d_type	char		1	
object	char		1	
d_program	char		1	
dirpath	char		64	
d_la_date	date		4	
d_la_hour	time		4	
d_la_user	char		8	
d_params	char		75	
d_entry	char		20	

d_key	Número secuencial que identifica el elemento.
d_name	Nombre del elemento.
d_type	Tipo de elemento: «B»: Base de datos; «T»: Tabla; «V»: View; «I»: Índice; «M»: Módulo; «P»: Programa; «H»: Módulo de ayuda; «S»: Módulo SQL;
object	Estado de compilación del módulo: «N»: Definido y no compilado; «O»: Compilado sin errores; «E»: Compilado con errores; «U»: Editado y no compilado.
d_program	Tipo de módulo o programa: «D»: Entrada de datos; «R»: Report; «M»: Menú; «O»: Varios; «L»: Librería.
dirpath	Directorio donde está situado el elemento. d_la_date Última fecha de actualización del elemento. d_la_hour Última hora de actualización del elemento.
d_la_user	Usuario que creó/actualizó por última vez el elemento. d_params Parámetros con que se probará el módulo o el programa en que se definen.
d_entry	Nombre de la función que servirá de prueba para el módulo de librería.

Los índices de esta tabla son los siguientes:

Índice	Tipo	Columnas
i1_ep_dict	Único	d_name,d_type

Ejemplo:

```
select d_key, d_name, d_type, object, d_program, dirpath, d_la_date, d_la_hour, d_la_user from ep_dict
```

Resultado:

d_key	d_name	d_type	object	d_program	dirpath	d_la_date	d_la_hour	d_la_user
1	almacen	B	B			24/05/1995	10:10:14	rai
2	almacen	A	A			24/05/1995	10:10:14	rai
3	articulos	M	O	D	/usr/rai/demo	24/05/1995	10:10:30	rai
4	unidades	M	O	D	/usr/rai/demo	24/05/1995	10:10:43	rai
5	formpago	M	O	D	/usr/rai/demo	24/05/1995	10:10:37	rai
6	clientes	M	O	D	/usr/rai/demo	24/05/1995	10:10:33	rai
7	provincias	M	O	D	/usr/rai/demo	24/05/1995	10:10:41	rai
8	proveedor	M	O	D	/usr/rai/demo	24/05/1995	10:10:40	rai
9	editabar	M	O	D	/usr/rai/demo	24/05/1995	10:10:36	rai
10	triple	M	O	O	/usr/rai/demo	24/05/1995	10:11:05	rai
11	rp_cliente	M	O	R	/usr/rai/demo	24/05/1995	10:11:14	rai
12	rp_provee	M	O	R	/usr/rai/demo	24/05/1995	10:11:16	rai

Tabla «ep_environ»

Su función es almacenar el entorno de variables asociado a cada usuario. De esta forma, al abrir la base de datos se actualizará automáticamente el entorno de variables con sus definiciones para el usuario en curso. Las columnas de esta tabla son las siguientes:

Nombre	tipo	null	longitud	etiqueta
varname	char		18	
username	char		8	
varvalue	char		64	

varname Identificador de la variable de entorno.

username Nombre del usuario.

varvalue Valor de la variable de entorno para el usuario.

Los índices de esta tabla son los siguientes:

Índice	Tipo	Columnas
i_env	Único	varname, username

Ejemplo:

```
select * from ep_environ
```

Resultado:

varname	username	varvalue
DBINCLUDE	rai	/usr/rai/demo
DBPROC	rai	/usr/rai/demo
DBLIB	rai	/usr/rai/demo
MSGDIR	rai	/usr/rai/demo
DBTEMP	rai	/tmp
DBDATE	rai	DMY4/
DBTIME	rai	S
DBMONEY	rai	

varname	usrname	varvalue
DBDELIM	rai	
DBMETACH	rai	
DBEDIT	rai	
DBPRINT	rai	PRN
RPRINTER	rai	
EP_FRM	rai	/usr/rai/demo
EP_RPT	rai	/usr/rai/demo
EP_MNU	rai	/usr/rai/demo
EP_OTH	rai	/usr/rai/demo
EP_HLP	rai	/usr/rai/demo
DDEBUG	rai	Y
EP_SQL	rai	/usr/rai/demo

Tabla «ep_magic»

Su función es guardar información que permita conocer si se ha realizado alguna modificación sobre el diccionario que necesite efectuar la verificación de éste con la opción correspondiente del entorno de desarrollo. Las columnas de esta tabla son las siguientes:

Nombre	tipo	null	longitud	etiqueta
chk_magic1	integer		4	
chk_magic2	integer		4	

Tabla «ep_modules»

Su función es guardar la relación existente entre un programa y los módulos que lo componen. Sus columnas son las siguientes:

Nombre	tipo	null	longitud	etiqueta
p_key	Integer	N	4	
m_key	Integer	N	4	
m_link	char		1	

p_key Identificador del elemento de tipo «P» (programa).

m_key Identificador del elemento de tipo «M» (módulo).

m_link Columna sin uso actualmente.

Los índices de esta tabla son los siguientes:

Índice	Tipo	Columnas
i2_ep_modules	Único	p_key, m_key
i1_ep_modules	Duplicados	p_key

Ejemplo:

```
select * from ep_modules
```

Resultado:

p_key	m_key	m_link
14	3	
16	8	
17	6	
18	5	
19	4	
20	7	
21	9	
22	10	
23	13	
24	12	
25	11	
32	29	
26	27	
30	45	
26	35	
33	35	
33	42	
31	40	
44	43	
44	35	
28	37	
36	39	
36	41	
34	38	
34	41	

Tabla «ep_prog_attr»

Su función es guardar la información de todos los atributos referidos a columnas que no pertenecen al SQL y que serán utilizados por los generadores de fuentes de entrada de datos para la confección de éstos. Sus columnas de la tabla son las siguientes:

Nombre	tipo	null	longitud	etiqueta
tabname	char		18	
colname	char		18	
helptxt	char		64	
helpnum	smallint		2	
a_autonext	char		1	
a_video	char		1	
a_verify	char		1	
a_queryclear	char		1	
lk_tab	char		18	
lk_key	char		18	
lk_disp	char		36	
l_video	char		1	

tablename	Nombre de la tabla.
colname	Nombre de la columna.
helptxt	Texto de ayuda en la edición de la columna. Se corresponde con el atributo: COMMENTS.
helpnum	Número de ayuda del fichero de mensajes. Se corresponde con el atributo «help nro».
a_autonext	Indica si la columna se editará con retorno automático o no. Se corresponde con el atributo AUTONEXT.
a_video	Indica el atributo de visualización de la columna. Los valores posibles son: «U»: («underline» —subrayado— y «R»: «reverse» —vídeo inverso—). Se corresponde con el atributo REVERSE. Por defecto es «underline».
a_verify	Indica si se quiere verificar la introducción de datos en la columna. Esta verificación consiste en editar dos veces la columna y comprobar que los valores introducidos coinciden. Se corresponde con el atributo VERIFY.
a_queryclear	Indica si se ha de limpiar el campo de cruce en un módulo de entrada de datos con varias tablas relacionadas. Se corresponde con el atributo QUERYCLEAR.
lk_tab	Nombre de la tabla de enlace.
lk_key	Nombre de la columna de la tabla de enlace por la que se realiza éste.
lk_disp	Columnas que se mostrarán pertenecientes a la tabla de enlace. Se corresponde con el atributo LOOKUP.
l_video	Indica el atributo de vídeo a utilizar sobre la etiqueta que acompaña en la edición a la columna.

Los índices de esta tabla son los siguientes:

Índice	Tipo	Columnas
i_attr	Único	tablename, colname

Ejemplo:

```
select tablename,colname,helpnum,a_autonext,a_video,a_verify,a_queryclear,l_video from ep_prog_attr
```

Resultado:

tablename	colname	helpnum	a_autonext	a_video	a_verify	a_queryclear	l_video
provincias	provincia	0	N	N	N	N	N
provincias	descripcion	0	N	N	N	N	N
provincias	prefijo	0	N	N	N	N	N
formpagos	formpago	0	N	N	N	N	N
formpagos	descripcion	0	N	N	N	N	N
unidades	unidad	0	N	N	N	N	N
unidades	descripcion	0	N	N	N	N	N
clientes	cliente	0	N	N	N	N	N
clientes	empresa	0	N	N	N	N	N
clientes	apellidos	0	N	N	N	N	N
clientes	nombre	0	N	N	N	N	N
clientes	direccion1	0	N	N	N	N	N

tabname	colname	helpnum	a_autonext	a_video	a_verify	a_queryclear	l_video
clientes	direccion2	0	N	N	N	N	N
clientes	poblacion	0	N	N	N	N	N

Tabla «ep_prtypes»

Guarda las descripciones de los elementos mediante su asociación con un código. Sus columnas son:

Nombre	tipo	null	longitud	etiqueta
pt	char		1	
ptt	char		18	

pt Código que identifica al literal.

ptt Descripción del literal.

Los índices de esta tabla son los siguientes:

Índice	Tipo	Columnas
i1_prtypes	Único	pt

Ejemplo:

```
select * from ep_prtypes
```

Resultado:

pt	ptxt
P	Programa
M	Módulo
D	Entrada de datos
R	Listado
m	Menú
L	Librería
O	Varios
H	Mensaje
B	Base de datos
T	Tabla
I	Índice
V	View
K	Clave Primaria
C	Clave Referencial
S	Procedimiento SQL

Tabla «ep_tynames»

Guarda la codificación de los diferentes tipos de datos que se manejan en el SQL, así como un texto descriptivo de éstos. Sus columnas de la tabla son las siguientes:

Nombre	tipo	null	longitud	etiqueta
c_type	smallint		2	
t_name	char		15	
t_length	smallint		2	

c_type Identificador de los tipos de datos permitidos en la definición de una columna.

t_name Descripción del tipo de dato que se mostrará en pantalla.

t_length Longitud en bytes del tipo de dato.

Los índices de esta tabla son los siguientes:

Índice	Tipo	Columnas
idx_typename	Único	c_type

Ejemplo:

```
select * from ep_tynames
```

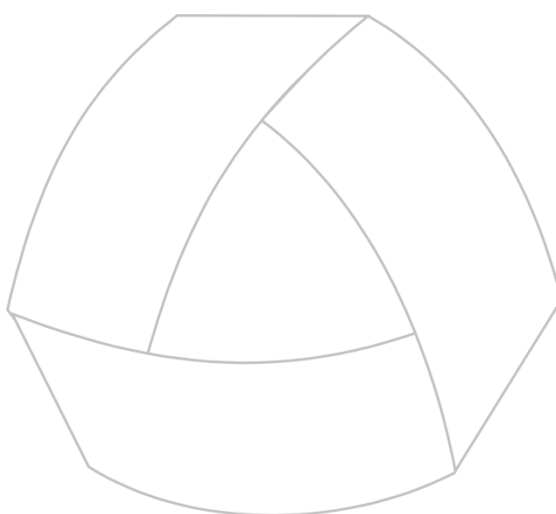
Resultado:

c_type	t_name	t_length
0	char	8
1	smallint	2
2	integer	4
3	time	4
4	(reserved)	0
5	decimal	16
6	serial	4
7	date	4
8	money	16

Columnas de las tablas del entorno de desarrollo

Columna	Tabla	Tipo	Lon	Nul	Ins	Upd	Cas	Alg	Zr	Fm	Df	Ck
a_autonext	ep_prog_attr	char	1	-	-	-	-	-	-	-	-	-
a_queryclear	ep_prog_attr	char	1	-	-	-	-	-	-	-	-	-
a_verify	ep_prog_attr	char	1	-	-	-	-	-	-	-	-	-
a_video	ep_prog_attr	char	1	-	-	-	-	-	-	-	-	-
c_key	ep_d_comm	integer	4	-	-	-	-	-	-	-	-	-
c_line	ep_d_comm	smallint	2	-	-	-	-	-	-	-	-	-
c_type	ep_tynames	smallint	2	-	-	-	-	-	-	-	-	-
chk_magic1	ep_magic	integer	4	-	-	-	-	-	-	-	-	-
chk_magic2	ep_magic	integer	4	-	-	-	-	-	-	-	-	-
colname	ep_col_descr	char	18	-	-	-	-	-	-	-	-	-
colname	ep_prog_attr	char	18	-	-	-	-	-	-	-	-	-
commen_t	ep_d_comm	char	60	-	-	-	-	-	-	-	-	-
d_entry	ep_dict	char	20	-	-	-	-	-	-	-	-	-
d_key	ep_dict	serial	4	N	-	-	-	-	-	-	-	-
d_la_date	ep_dict	date	4	-	-	-	-	-	-	-	-	-
d_la_hour	ep_dict	time	4	-	-	-	-	-	-	-	-	-
d_la_user	ep_dict	char	8	-	-	-	-	-	-	-	-	-
d_name	ep_dict	char	18	-	-	-	-	-	-	-	-	-
d_params	ep_dict	char	75	-	-	-	-	-	-	-	-	-
d_program	ep_dict	char	1	-	-	-	-	-	-	-	-	-
d_type	ep_dict	char	1	-	-	-	-	-	-	-	-	-
description	ep_col_descr	char	255	-	-	-	-	-	-	-	-	-

Columna	Tabla	Tipo	Lon	Nul	Ins	Upd	Cas	Alg	Zr	Fm	Df	Ck
dirpath	ep_dict	char	64	-	-	-	-	-	-	-	-	-
helpnum	ep_prog_attr	smallint	2	-	-	-	-	-	-	-	-	-
helptxt	ep_prog_attr	char	64	-	-	-	-	-	-	-	-	-
l_video	ep_prog_attr	char	1	-	-	-	-	-	-	-	-	-
lk_disp	ep_prog_attr	char	36	-	-	-	-	-	-	-	-	-
lk_key	ep_prog_attr	char	18	-	-	-	-	-	-	-	-	-
lk_tab	ep_prog_attr	char	18	-	-	-	-	-	-	-	-	-
m_key	ep_modules	integer	4	N	-	-	-	-	-	-	-	-
m_link	ep_modules	char	1	-	-	-	-	-	-	-	-	-
object	ep_dict	char	1	-	-	-	-	-	-	-	-	-
p_key	ep_modules	integer	4	N	-	-	-	-	-	-	-	-
pt	ep_prtypes	char	1	-	-	-	-	-	-	-	-	-
ptt	ep_prtypes	char	18	-	-	-	-	-	-	-	-	-
t_length	ep_tnames	smallint	2	-	-	-	-	-	-	-	-	-
t_name	ep_tnames	char	15	-	-	-	-	-	-	-	-	-
tablename	ep_col_descr	char	18	-	-	-	-	-	-	-	-	-
tablename	ep_prog_attr	char	18	-	-	-	-	-	-	-	-	-
username	ep_environ	char	8	-	-	-	-	-	-	-	-	-
varname	ep_environ	char	18	-	-	-	-	-	-	-	-	-
varvalue	ep_environ	char	64	-	-	-	-	-	-	-	-	-



MultiBase
MANUAL DEL PROGRAMADOR

CAPÍTULO 3. Cómo empezar

Introducción

En este capítulo explicaremos la forma de comenzar a desarrollar una aplicación con MultiBase. Para ello, supondremos que la herramienta ha sido instalada de manera satisfactoria y que el entorno operativo ha sido configurado correctamente en lo que respecta a las diferentes variables de entorno.

En el Manual del Administrador se explica cómo instalar MultiBase en los distintos sistemas operativos, así como la configuración del entorno del usuario encargado del desarrollo de una aplicación.

No obstante, a continuación indicamos las variables de entorno básicas para comenzar a desarrollar una aplicación con MultiBase:

TRANSDIR	Indica el directorio donde se encuentra instalado MultiBase.
PATH	Esta variable de entorno tiene que incluir el subdirectorio «bin» de MultiBase para poder ejecutar sus comandos.
TERM o MBTERM	Nombre del terminal que se está utilizando. En Windows este nombre es «PC».
DBTEMP	Esta variable de entorno es obligatoria en MultiBase Windows. Su finalidad es indicar el directorio en el que se generarán los ficheros temporales o de trabajo de la herramienta.

Dependiendo del tipo de instalación, es posible que requiera alguna variable de entorno además de las aquí mencionadas. En caso de duda, consulte el capítulo sobre «Ficheros de Personalización» en el Manual del Administrador.

Los pasos que hay que seguir para comenzar el desarrollo de una aplicación son los siguientes:

- Ejecutar el entorno de desarrollo.
- Crear una base de datos.
- Establecer permisos sobre la base de datos.
- Crear las tablas de la base de datos.
- Definir la integridad referencial entre todas las tablas mediante la creación de claves primarias y referenciales sobre las tablas.
- Crear los índices (opcional).
- Generar módulos.

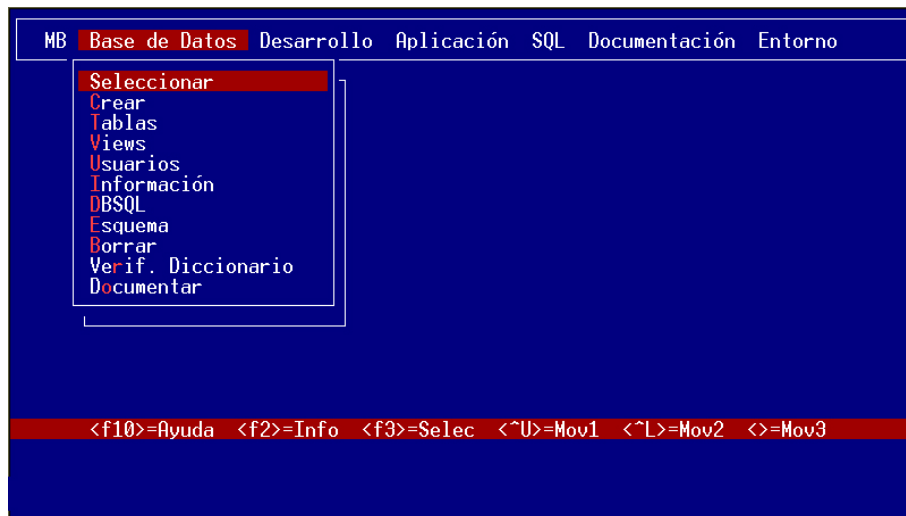
En los siguientes apartados se explica en detalle cada uno de estos pasos.

Ejecución del entorno de desarrollo

Una vez configuradas las variables de entorno necesarias para trabajar con MultiBase, la forma de acceder al entorno de desarrollo dependerá del sistema operativo que estemos utilizando:

a) UNIX/Linux:

La forma de invocar al entorno de desarrollo de MultiBase será mediante la ejecución del comando `trans` («\$trans»). Una vez ejecutado dicho comando, la pantalla presentará el aspecto que se muestra en la figura.



Ejecución del entorno de desarrollo de MultiBase bajo UNIX/Linux.

**b) Windows:**

Si bien es posible arrancar el entorno de desarrollo pulsando directamente el icono «Trans» del grupo «Multi-Base» que se crea al instalar la herramienta, la forma más lógica de hacerlo cuando se trata de desarrollar una aplicación es definiéndole a ésta un icono propio.

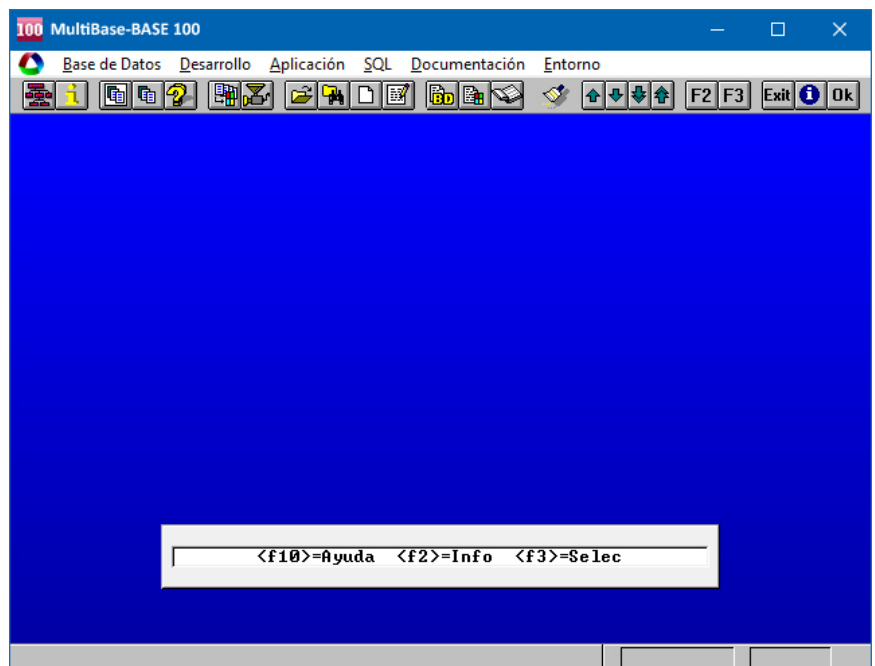
Por ejemplo, si va a desarrollar una aplicación de «facturación», genere un icono denominado «Facturación» a través de la opción «Nuevo» de la persiana «Archivo» del Administrador de Programas de Windows. Para ello, en el cuadro de diálogo «Propiedades del elemento de programa», además de la descripción del icono («Facturación» en nuestro caso) habrá que indicar la siguiente línea de comando:

```
c:\ctl\bin\ctl -env -cd c:\mbdemo ep_trans ""
```

Este ejemplo indica que el entorno de desarrollo se ejecutará sin seleccionar ninguna base de datos y que el directorio de trabajo será «c:\mbdemo».

No obstante, en la línea de comando indicada se pueden incluir más opciones. En el capítulo denominado «Comandos MultiBase» del Manual del Administrador se indican todas las posibles opciones que se pueden incluir en una línea de comando CTL.

Podrá especificar también el directorio de trabajo en el campo correspondiente. El contenido de este campo debe coincidir necesariamente con el «path» o directorio donde residirá la base de datos.



Entorno de desarrollo de MultiBase en Windows.

Una vez definido el icono deberá ejecutarlo para acceder al entorno de desarrollo de MultiBase y comenzar a desarrollar su aplicación. El aspecto que presentará la pantalla es el que se muestra en la figura.

NOTA: Para más información sobre cómo iconizar una aplicación de usuario, consulte el capítulo denominado «Traspaso de Aplicaciones» en el Manual del Administrador.

Como puede observarse en las figuras anteriores, el menú principal que aparece es de tipo «persiana» o «pull-down». La forma de desplazarse a través de este menú es mediante las teclas de flechas (a derecha e izquierda para seleccionar una persiana; arriba y abajo para seleccionar una opción de una persiana). Para ejecutar una determinada opción bastará con posicionarnos sobre ella y pulsar la tecla [RETURN] o, alternativamente, la tecla asignada a la acción <facept> en el fichero «Tactions» (por defecto [F1]). Recuerde que en Windows podrá utilizar también el ratón tanto para seleccionar una persiana como una opción.

Otra forma de ejecutar una opción es pulsando directamente la tecla correspondiente a la letra de dicha opción que figura en distinta intensidad de vídeo (en UNIX/Linux), o subrayada (en Windows). Tenga en cuenta que sólo podrán ejecutarse aquellas opciones que aparezcan con una intensidad de vídeo normal. Las que no puedan ser ejecutadas en un determinado momento aparecerán con una intensidad más atenuada.

Creación de una base de datos

El primer paso para comenzar a desarrollar una aplicación consiste en crear una base de datos. Para ello, habrá que establecer previamente:

- Si la base de datos será transaccional o no.
- Si la base de datos (transaccional o no transaccional) tendrá asignado un fichero de ordenación.

La opción «Crear» de la persiana «Base de Datos» del entorno de desarrollo crea una base de datos NO transaccional con o sin fichero de ordenación («collating sequence»). Dicha opción implica indirectamente la ejecución de la instrucción CREATE DATABASE del SQL. Si se ha definido la variable de entorno DBPATH, el entorno de desarrollo preguntará el directorio en el que se desea ubicar dicha base de datos. En caso contrario, es decir, si no se definió la variable, la base de datos se creará en el directorio en curso.

Sin embargo, si quiere que la base de datos que va a crear sea de tipo transaccional, habrá que hacerlo indicando directamente la instrucción CREATE DATABASE con la cláusula «WITH LOG IN» a través de la opción «Nuevo» o «Editor» de la persiana «SQL» del entorno de desarrollo. La sintaxis que habrá que emplear es la siguiente:

```
CREATE DATABASE factu WITH LOG IN "/usr/usuario1/fichlog"  
[COLLATING "/usr/ctl/msg/spanish"]
```

A continuación, para ejecutarla, seleccione la opción «Ejecutar» de la misma persiana «SQL». Esta instrucción creará la base de datos en el primer directorio indicado en la variable de entorno DBPATH, si es que ésta estuviese definida o, en su defecto, en el directorio en curso. La base de datos será un subdirectorio (con extensión «.dbs») dentro del directorio elegido (en nuestro ejemplo: «factu.dbs»).

Dentro de este subdirectorio será donde se generen las tablas del catálogo de la base de datos, el cual estará compuesto por las tablas del entorno de desarrollo (tablas «ep_*» y las correspondientes al CTSQL (tablas «sys*»). Para una explicación más detallada de las primeras [consulte el capítulo anterior](#) de este mismo volumen. Por lo que respecta a las tablas del CTSQL, éstas se explican en el Manual del SQL.

Asimismo, las tablas del usuario se generarán por defecto dentro de este subdirectorio. No obstante, éstas podrán ubicarse en cualquier directorio de cualquier unidad lógica de la máquina.

Una vez creada la base de datos, ésta se convertirá automáticamente en la base de datos activa o en curso.

NOTA: Si nuestra base de datos es NO transaccional y queremos convertirla a transaccional habrá que emplear la instrucción `START DATABASE` del SQL.



Nota a la versión para Windows:

Para crear una base de datos en un directorio concreto bastará con especificarlo en el campo «Directorio de trabajo» dentro del cuadro de diálogo de «Propiedades del elemento de programa». No obstante, esta operación puede realizarse igualmente indicando el directorio en la variable de entorno `DBPATH` antes de arrancar Windows, o bien en la cláusula «-cd» del comando `ctl`.

Nota para instalaciones con «gateways»:

Dependiendo del gestor de base de datos utilizado, el proceso a seguir es el siguiente:

a) Oracle

Crear la base de datos desde el gestor de Oracle y ejecutar a continuación la instrucción `CREATE DATABASE` para generar el catálogo de dicha base de datos (tablas «mb*»).

b) Informix

Si la base de datos ya está creada habrá que ejecutar la instrucción `CREATE MULTIBASE CATALOGS` para crear el catálogo de la base de datos. Por el contrario, si la base de datos no estuviese creada, la instrucción `CREATE DATABASE` la creará en Informix con su respectivo catálogo (tablas «sys*» y «mb*»).

Permisos sobre la base de datos

El propietario de la base de datos dependerá del entorno operativo en que nos encontremos. Así, bajo sistema operativo UNIX/Linux, el nombre del propietario será el del usuario con el que accedamos al sistema, es decir, el nombre del «login». Por su parte, en las versiones monopuesto de Windows el propietario será siempre «system», mientras que en las versiones para red local o en instalaciones con arquitectura cliente-servidor habrá que definir el nombre del usuario del «nodo» a través de la variable de entorno `DBUSER`.

En caso de no realizar ninguna otra operación, sólo el usuario propietario de la base de datos será quien tenga permiso de acceso a la misma. Además, este usuario, en principio, será el único que podrá conceder permisos al resto de usuarios de la máquina.

Una vez hecha esta salvedad, habrá que decidir si se va a permitir o no el acceso a la base de datos a otros usuarios del sistema y qué tipo de permisos se van a conceder. En caso afirmativo, la forma de conceder estos permisos es mediante la ejecución de la opción

«Usuarios» de la persiana «Base de Datos» del entorno de desarrollo. El permiso más restrictivo sobre una base de datos es `CONNECT`, mientras que el más amplio es `DBA`. La instrucción que se ejecuta internamente para la concesión de permisos a otros usuarios es `GRANT`.

Para más información acerca de los permisos sobre bases de datos y tablas consulte el capítulo referente al «Lenguaje de Control de Acceso a Datos» del Manual del SQL.

Creación de tablas

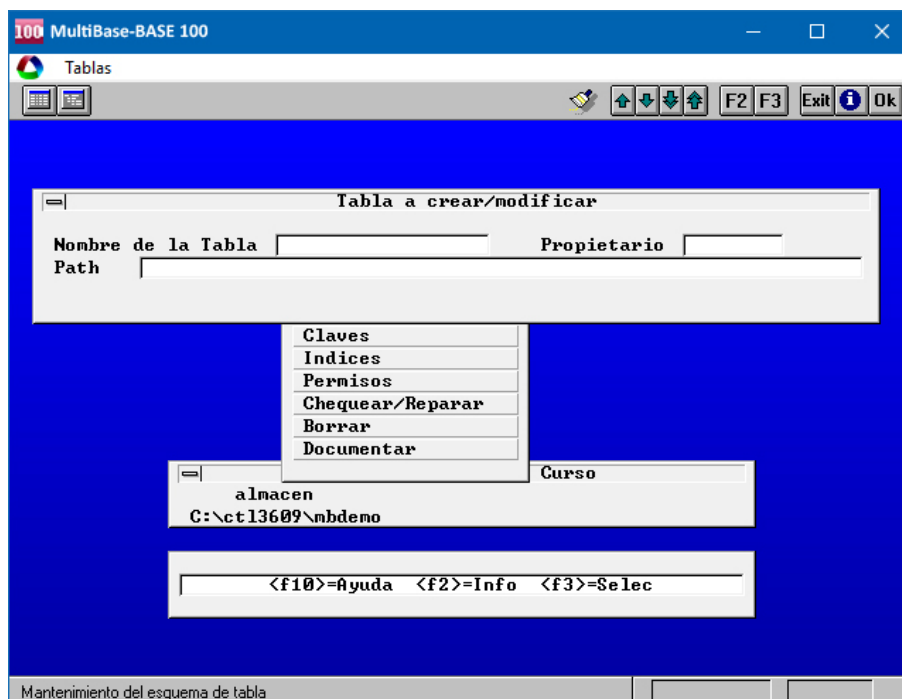
A continuación, una vez creada la base de datos y, opcionalmente, establecidos los correspondientes permisos, habrá que proceder a definir las tablas pertenecientes a la base de datos en cuestión. Esta labor se realiza a través de la opción «Tablas» de la persiana «Base de Datos» del entorno de desarrollo.

La ejecución de dicha opción presenta un menú en el que se muestran todas las operaciones que podrán realizarse sobre las tablas de la base de datos activa. La primera opción de este menú («Definición») nos permitirá crear nuevas tablas o alterar la estructura de las ya existentes. Alterar la estructura de una tabla puede consistir en las siguientes operaciones:

- Añadir columnas nuevas a la tabla.
- Borrar columnas existentes.
- Modificar atributos de columnas existentes.
- Borrar atributos de columnas existentes.
- Crear o borrar claves primarias.
- Crear o borrar claves referenciales.

En nuestro caso, y dado que acabamos de crear una base de datos nueva por vez primera, utilizaremos la opción «Definición» para crear las tablas, ya que se supone que no existirá ninguna en nuestro sistema.

Esta opción dispone de dos formatos para la definición (o alteración) de las tablas: «Sólo Esquema» y «Esquema y Atributos», siendo este último el más completo al llevar implícito el primero. Estos formatos aparecerán representados en un menú de tipo «Lotus» en UNIX/Linux, o en un menú de iconos en Windows (ver figura).



Aspecto de la pantalla al ejecutar la opción «Definición» de tablas.

En nuestro caso, seleccionaremos «Esquema y Atributos». Acto seguido, el programa solicitará el nombre de la tabla a crear y el «path» donde se desea ubicar. Por defecto, el «path» que se utiliza siempre será el subdirectorio «.dbs» correspondiente a la base de datos activa, es decir aquella a la que pertenecerá la tabla.

Una vez aceptado (tecla asignada a la acción <faccept>, normalmente [F1]) el nombre de la tabla a crear junto con su «path» y propietario («owner»), aparecerá una nueva pantalla («Diccionario de columnas», ver figura) en la que habrá que indicar las características de cada una de las columnas que pertenecerán a la tabla que se desea crear.

Aspecto de la pantalla de definición de características de las columnas pertenecientes a la tabla que se está creando.

A continuación se explica el significado de cada uno de los campos que se presentan en esta pantalla, así como su correspondencia con la definición CTL o CTSQL (para más información sobre atributos CTL y CTSQL consulte los Apéndices del Manual de Referencia).

Como puede observarse, el aspecto de la pantalla está dividido en tres áreas con distinto significado: la primera contiene datos de identificación de la columna que se define, la segunda engloba los atributos que utilizará el CTSQL (si se activan) y, por último, la tercera está referida a los atributos de programación o CTL que serán utilizados en la fase de generación de programas. La descripción de cada uno de estos campos es la siguiente:

Columna	Nombre de la columna a definir o modificar. Este identificador debe comenzar necesariamente por una letra, pudiendo especificar a continuación cualquier secuencia de caracteres, incluyendo números y signos de subrayado. Los únicos caracteres que no se podrán incluir son los correspondientes a las cuatro operaciones aritméticas básicas («+», «-», «*» o «/»).
Trae atr.	Permite asignar a una columna los atributos de otra ya definida, incluso para la misma tabla. Los posibles valores son «N» (por defecto) o «Y».
Documentar	Su valor por defecto es siempre «N». Indicando «Y» pedirá un texto descriptivo de la columna que servirá para el documentador de manuales.
Tipo	Especifica el tipo de dato correspondiente a la columna que se está definiendo: CHAR, SMALLINT, INTEGER, TIME, DECIMAL, SERIAL, DATE o MONEY.
Longitud	Permite establecer la longitud en bytes que ocupará en disco la columna que se está definiendo para los tipos de datos CHAR, MONEY y DECIMAL. Para los restantes tipos de datos este campo muestra su longitud por defecto.
Precisión	Expresa el número de dígitos a la derecha del punto decimal (por defecto 2) para los tipos de datos DECIMAL y MONEY, careciendo de significado en los demás tipos.

Etiqueta	Permite definir el texto que la columna utilizará como título en cualquier instrucción SELECT. Se corresponde con el atributo LABEL.
Requerido	Indicando «Y» significa que el campo se creará como NOT NULL. Es decir, a la hora de grabar una fila sobre la tabla, el valor asignado a la columna que se está definiendo no podrá ser nunca «NULL».
No agregar	Esta columna no se podrá incluir en una operación de inserción de filas. Se corresponde con el atributo NOENTRY.
No actualizar	Esta columna no se podrá incluir en una operación de actualización de filas. Se corresponde con el atributo NOUPDATE.
May./Min.	En columnas con tipos de datos CHAR admite dos valores: «U», para indicar conversión a mayúsculas, o «D», para conversión a minúsculas. Se corresponde con los atributos UPSHIFT y DOWNSHIFT, respectivamente.
Alineación	Provoca alineación a la derecha o a la izquierda con RIGHT o LEFT. Por defecto, los tipos de datos numéricos se alínean a la derecha, mientras los de tipo CHAR se alínean a la izquierda.
Llenar a ceros	Rellena el campo con ceros. Se corresponde con el atributo ZEROFILL. formato: Establece una plantilla de formato para columnas numéricas, de fecha y de hora. Se corresponde con el atributo FORMAT.
Máscara	Establece una plantilla de formato para columnas de tipo CHAR. Se corresponde con el atributo PICTURE.
V. defec.	Establece un valor por defecto al insertar una fila, siempre y cuando a esta columna se le asigne un valor «NULL». Se corresponde con el atributo DEFAULT.
Condición	Permite establecer una validación del contenido del campo mediante una condición booleana. Se corresponde con el atributo CHECK.
Ret. auto.	Activa el salto al siguiente campo de una entrada de datos una vez terminada la edición del campo actual sin necesidad de pulsar [RETURN]. Se corresponde con el atributo AUTONEXT.
Atr. Vídeo	Permite definir la representación del área de edición de la columna como subrayado («U») o vídeo inverso («R»). Se corresponde con los atributos UNDERLINE y REVERSE, respectivamente.
Atr. Título	Similar al anterior, pero referido a la etiqueta de la columna (LABEL). Este campo no se corresponde con ningún atributo específico.
Borr. Auto.	Limpia el área de representación de los cruces existentes en la tabla activa antes de una búsqueda. Se corresponde con el atributo QUERYCLEAR.
Verificar	Obliga a realizar la entrada del valor correspondiente dos veces para aceptarla como válida. Se corresponde con el atributo VERIFY.
Núm. mensaje	Indica el número de mensaje de ayuda en el «fichero de ayuda». Se corresponde con el atributo HELP.
Ayuda	Permite incluir un texto que aparezca en la línea de mensajes durante la edición de la columna. Se corresponde con el atributo COMMENTS.
CRUCE	Los siguientes atributos se tratan como un bloque, puesto que no se definen independientemente, sino estrechamente ligados entre sí. Un cruce de una co-

columna con otra de otra tabla sirve para mostrar y/o validar el valor introducido en la primera respecto a los valores existentes en la segunda.

Tabla Tabla contra la que se realiza el cruce.

Columna Columna contra la que se valida.

Columnas a Mostrar Lista de columnas de la tabla de cruce, separadas por comas, que se mostrarán al introducir/mostrar el valor de la columna que definimos.

Estas tres últimas definiciones se corresponden con el atributo LOOKUP.

Realmente, de todas estas cláusulas las únicas obligatorias son: Nombre de la columna, tipo de dato SQL y longitud de la columna.

NOTAS:

- 1.— Una tabla puede crear o alterar su estructura a nivel de columnas, atributos, índices, claves primarias y referenciales en cualquier momento del desarrollo o explotación de una aplicación.
- 2.— En instalaciones con «gateways», si las tablas ya han sido creadas en el gestor correspondiente habrá que volver a crearlas con la instrucción CREATE TABLE pero en modo mantenimiento. Esto se consigue mediante las instrucciones «BEGIN MAINTENANCE MODE» y «END MAINTENANCE MODE», que implican la carga del catálogo de la base de datos en el «gateway» (tablas «mb*»). Si las tablas no hubiesen sido creadas la instrucción CREATE TABLE se encargará de hacerlo. Esta misma operación habrá que repetirla en la ejecución de las distintas instrucciones de definición de datos del DDL.

Establecimiento de la integridad referencial

La integridad referencial de una base de datos relacional consiste en la consistencia entre las filas («tuplas») de distintas tablas normalizadas, en base a sus columnas de enlace y a la relación de interdependencia de las mismas.

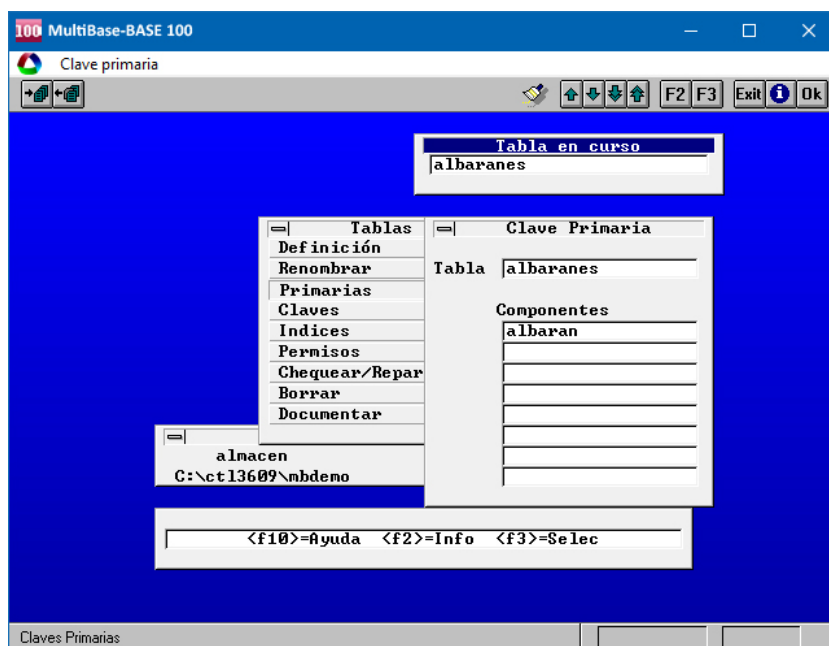
Establecer integridad referencial consiste en la creación de claves primarias («primary keys») y claves referenciales («foreign keys»). La creación de una integridad referencial correcta supone un beneficio a la hora de emplear los generadores de programas del entorno de desarrollo.

NOTA: Para más información acerca de la integridad referencial consulte el capítulo correspondiente al «Lenguaje de Definición de Datos» del Manual del SQL.

Claves primarias («Primary Keys»)

La creación de una clave primaria equivale a la creación de un índice único para la columna o columnas que la componen, con la diferencia de que dicha clave va a servir, además de para facilitar el acceso rápido a los datos, para controlar la integridad referencial contra otra u otras tabla. Cada tabla de la base de datos sólo puede tener una clave primaria.

Para crear las claves primarias de cada tabla de la base de datos hay que ejecutar la opción «Primarias» del submenú «Tablas» de la persiana «Base de Datos» del entorno de desarrollo. Una vez ejecutada aparecerá una ventana de petición de datos con el aspecto que se muestra en la figura.



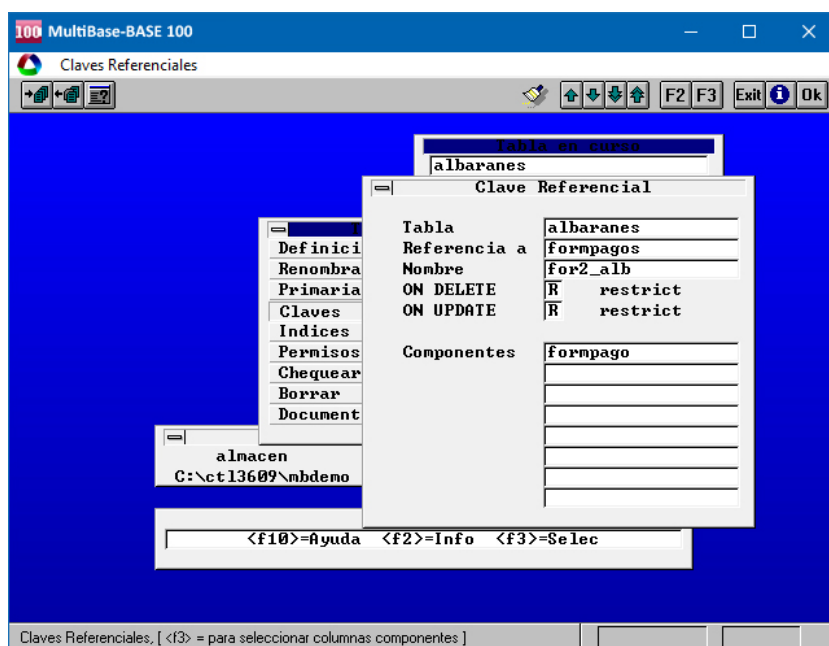
Ejecución de la opción «Primarias» del submenú «Tablas».

En esta pantalla sólo habrá que indicar los nombres de las columnas que van a componer la clave primaria. Todas estas columnas, obligatoriamente, deben tener asignado el atributo NOT NULL en su definición.

Claves referenciales («Foreign Keys»)

Una clave referencial establece la relación entre la tabla para la que se crea y la tabla que referencia. Mediante esta relación el operador nunca podrá actualizar la columna o columnas que compone la clave primaria en la tabla referenciada o borrar la fila si existe información de ella en la tabla referencial.

Para crear las claves referenciales sobre las tablas hay que ejecutar la opción «Claves» del submenú «Tablas» de la persiana «Base de Datos» del entorno de desarrollo. Una vez ejecutada aparecerá una pantalla de petición de datos cuyo aspecto es el que se muestra en la figura.



Pantalla para definición de claves referenciales.

En esta pantalla habrá que indicar el nombre de la tabla referenciada, el de la clave referencial (identificador SQL), el modo de integridad en las operaciones de UPDATE y DELETE (RESTRICT o SET NULL) y la columna o columnas que componen dicha clave.

Una clave referencial no se puede crear si no se ha creado previamente la clave primaria en la tabla referenciada.

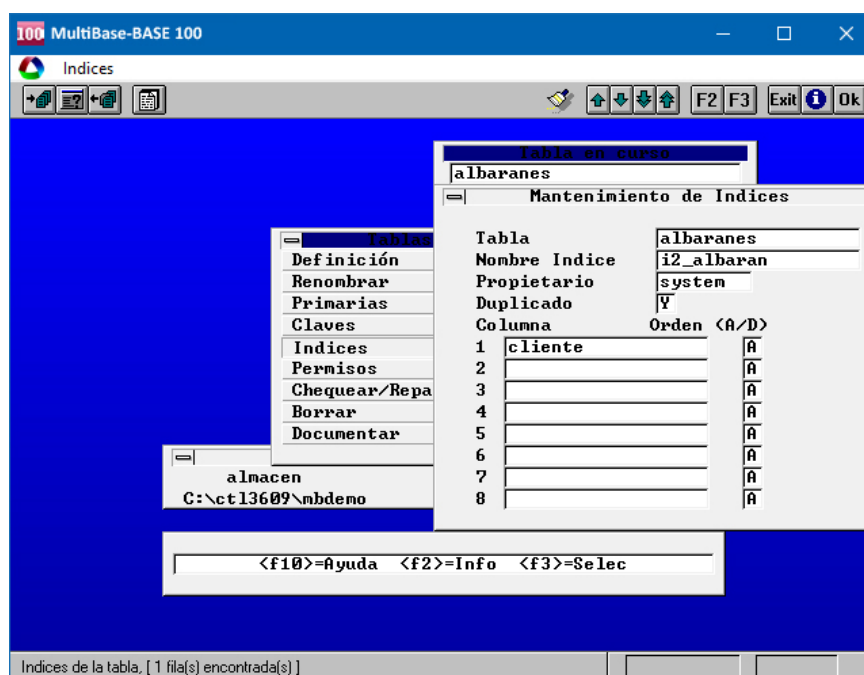
Una tabla puede tener más de una clave referencial apuntando a varias tablas de la misma base de datos.

NOTA: Una clave referencial NO es un índice.

Creación de índices

Un índice es una utilidad manejada por el SQL para el acceso a los datos de las tablas. Opcionalmente, y si se conoce la forma de operar del SQL respecto a los accesos a tablas, se pueden crear los índices de aquellas tablas previamente creadas. No obstante, en caso de duda, es preferible no crearlos hasta concluir el desarrollo de la aplicación, o bien hasta que verdaderamente estemos seguros de su función y utilidad. Tal y como se explica en el Manual del SQL, éste sabe acceder a los datos de las tablas sin necesidad de tener creado un índice para ellas.

Si se opta por crear los índices habrá que ejecutar la opción «Índices» del submenú «Tablas» de la persiana «Base de Datos» del entorno de desarrollo. Internamente, esta opción ejecuta la instrucción CREATE INDEX del SQL. Una vez ejecutada esta opción aparecerá una ventana de petición de datos con el aspecto que se muestra en la figura.



Ejecución de la opción «Índices» del submenú «Tablas».

En esta pantalla habrá que indicar el nombre del índice (identificador SQL), si éste admitirá o no valores duplicados, y la columna o columnas que lo componen.

NOTAS:

1.— La creación de índices incorrectos puede ralentizar la explotación de la aplicación cuando ésta tenga un número elevado de datos.

2.— La creación y borrado de índices sobre las tablas de la base de datos puede realizarse en cualquier momento del desarrollo de la aplicación.

Generación de módulos

Una vez realizados todos los pasos anteriores, la siguiente operación consistirá en comenzar el desarrollo de la aplicación. Para ello habrá que utilizar los diversos generadores de programas del CTL. Estos generadores se encuentran representados por diferentes opciones del entorno de desarrollo.

Los generadores que actúan en el entorno de desarrollo de MultiBase son los siguientes:

- Generador de mantenimientos de tablas (altas, bajas, modificaciones y consultas).
- Generador de mantenimientos de tablas con relación «cabecera-líneas» (altas, bajas, modificaciones y consultas) de dos tablas unidas, al menos, por una columna común entre ellas.
- Generador de listados.
- Generador del menú final de la aplicación.
- Generador de librerías.
- Generador de consultas individuales por tabla.

Para acceder a estos generadores podremos utilizar dos vías diferentes:

1.— Generar Prototipo: Esta opción se encuentra en la persiana «Aplicación» del entorno de desarrollo. Mediante su ejecución el programador podrá decidir qué tipos de programas desea generar. No obstante, dentro del menú que aparece al ejecutar esta opción (de tipo «Lotus» o de iconos, dependiendo del entorno operativo), existe una opción, también denominada «Aplicación», que engloba a todos los generadores, es decir, que activará todos ellos para generar cada uno los programas específicos sobre cada tabla de la base de datos.

2.— Generar Fuente: Esta opción se encuentra al activar cada uno de los submenús que aparecen al ejecutar la opción «Módulos/Generación» de la persiana «Desarrollo» del entorno de desarrollo de MultiBase. Dependiendo del tipo de módulo elegido, actuará el correspondiente generador:

- Los generadores de mantenimientos y cabeceras-líneas se activan desde la clasificación «Entrada Datos».
- El generador de listados se activa desde la clasificación de módulos denominada «Listados».
- El generador de menús se activa desde la clasificación de módulos denominada «Menús».
- El generador de librerías se activa desde la clasificación de módulos denominada «Librerías».
- El generador de consultas se activa desde la clasificación de módulos denominada «Varios».

NOTA: Para el desarrollo de una aplicación lo normal será elegir la generación de sus «módulos» de forma individual y no de todos a la vez. Es decir, será más lógico utilizar los generadores según se ha expuesto en el segundo punto («Generar Fuente»).

Cómo probar la aplicación

En caso de haber construido la aplicación mediante la opción «Generar Prototipo», el siguiente paso consistiría en probar la programación generada. Para ello podremos utilizar dos vías distintas:

1.— Ejecución de la aplicación desde el entorno de desarrollo. Para ello habrá que seleccionar la secuencia de opciones: «Desarrollo» — «Módulos/Generación» — «Menús» — «Probar». Una vez ejecutada esta secuencia, el programa pedirá el nombre del módulo que se desea ejecutar. En nuestro caso, en principio, sólo existirá un módulo en dicha clasificación, que será el que forzosamente habrá que ejecutar. Este módulo, cuyo nombre coincidirá con el de la base de datos activa, contendrá el menú final de la aplicación.

2.— Ejecución del menú final de la aplicación desde el sistema operativo. Este supuesto es válido únicamente para las versiones de MultiBase bajo UNIX/Linux. En este caso, la forma de activar el menú final de la aplicación será ejecutando el comando `ctl` desde la línea de comandos del sistema operativo. La sintaxis de este comando es la siguiente:

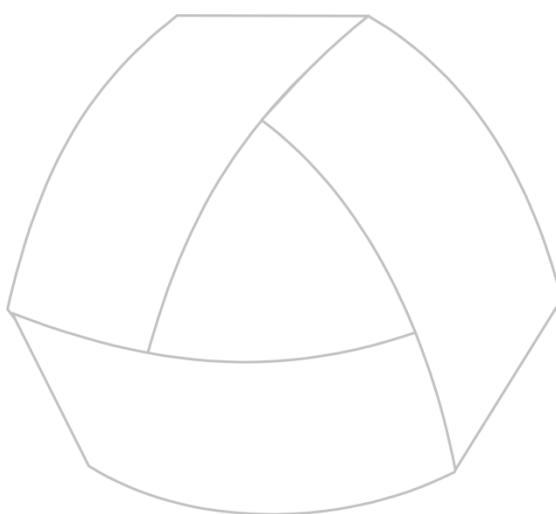
```
ctl nombre_menu
```

Donde:

<code>nombre_menu</code>	Es el nombre del menú final de la aplicación, que coincidirá con el nombre de la base de datos.
--------------------------	---



Por lo que respecta a las versiones de MultiBase para Windows, la forma de ejecutar el menú final de una aplicación será mediante el correspondiente icono previamente creado para la misma (para más información consulte el capítulo «Traspaso de Aplicaciones» en el Manual del Administrador).



MultiBase
MANUAL DEL PROGRAMADOR

CAPÍTULO 4. Sintaxis empleada en CTL

Convenciones utilizadas

Las convenciones empleadas en los diferentes manuales de MultiBase para presentar la sintaxis de cualquier elemento son las siguientes:

[]	Aquellos elementos que se encuentren entre corchetes son opcionales.
{ }	Indica la obligatoriedad de incluir uno de los elementos encerrados entre llaves. Dichos elementos estarán separados por el carácter de «barra vertical» (carácter ASCII 124).
	Este carácter se utiliza para separar los diferentes elementos opcionales u obligatorios de la sintaxis, dependiendo de si éstos van entre corchetes o entre llaves, respectivamente.
...	El elemento anterior a los puntos puede aparecer más veces en la sintaxis.
,...	Como el caso anterior, pero cada nueva aparición del dicho elemento debe ir precedida por una coma.
palabra	Las palabras en minúsculas corresponden con identificadores CTL o CTSQL, expresiones, etc.
PALABRA	Las palabras en mayúsculas y en negrita son reservadas del lenguaje CTL o CTSQL, y por tanto invariables.
<u>subrayado</u>	Si no se especifica otra opción, la palabra o palabras subrayadas se toman por defecto.
negrita	Cualquier signo de las convenciones que se encuentre en negrita es obligatorio en la sintaxis.

El resto de signos se utilizan con su valor. Así, en aquellos lugares en los que se pueden especificar comillas, éstas podrán ser simples o dobles (" , '), con la obligatoriedad de cerrar el texto con el mismo tipo con el que se abrió. Asimismo, se pueden incluir comillas como parte de un texto, en cuyo caso, dichas comillas deberán ser necesariamente del tipo contrario a las que sirven de delimitadores del texto. Es decir, en el caso de definir un literal entre comillas que incluya una parte también entrecomillada, habrá que combinar ambos tipos. Por ejemplo:

```
tsql "load from 'fichero.unl' insert into tabla"
```



El símbolo del ratón indica que el texto que figura a continuación se refiere exclusivamente a la versión de MultiBase para Windows.

Módulos y programas

Una aplicación es un conjunto heterogéneo de tareas que permiten el tratamiento y la solución de problemas «reales», así como de sus datos. El CTL —lenguaje de cuarta generación de MultiBase— ofrece como elementos para la resolución de esta problemática un sistema de base de datos, programas, módulos, funciones y librerías o bibliotecas de funciones.

Un módulo fuente de CTL es un fichero ASCII (con extensión «.sct») que contiene instrucciones CTL (ver Manual de Referencia). Este módulo fuente tiene que ser compilado mediante el compilador de CTL (ctlcomp) para que pueda intervenir en la explotación de la aplicación. El fichero objeto emanado de esta compilación se identifica por la extensión «.oct». El desarrollo de una aplicación se realiza por medio de módulos CTL.

La creación de módulos desde el entorno de desarrollo puede realizarse por dos caminos diferentes:

a) Mediante la opción «Generar». Como ya se ha indicado anteriormente, el entorno de desarrollo dispone de generadores para distintos tipos de programas. En caso de utilizar dichos generadores, el módulo generado se define por sí solo, por lo que no será necesario utilizar la opción «Definir». Para acceder a la opción «Generar» de cada tipo de módulo hay que seleccionar la siguiente secuencia de opciones:

«Desarrollo» — «Módulos/Generación» — «Entrada Datos/ Listados/Menús/Librerías/Varios» — «Generar»

b) Mediante la opción «Definir». En caso de no utilizar los generadores de módulos, será necesario definir un módulo en el momento de comenzar el desarrollo. Esta definición se realiza desde la opción «Definir» de cada clasificación de módulos. Para llegar a esta opción hay que seleccionar la siguiente secuencia de opciones del entorno de desarrollo:

«Desarrollo» — «Módulos/Generación» — «Entrada Datos/ Listados/Menús/Librerías/Varios» — «Definir»

En el desarrollo de una aplicación intervienen dos tipos de módulos:

1.— Módulos principales (o con sección MAIN en su sintaxis).

Este tipo de módulos se identifican porque dentro de su sintaxis se encuentra definida una sección denominada «MAIN». Cuando un módulo de estas características no necesita de ningún elemento de programación externo a él (funciones, menús, etc., es decir, que se encuentra en otro módulo), se trata de un módulo ejecutable directamente por el comando ctl, sin que sea necesario convertirlo en programa (ver la definición de «programa» más adelante en esta misma sección). En este caso, la ejecución del módulo comienza en la primera instrucción incluida en dicha sección MAIN.

2.— Módulos de librerías (o sin sección MAIN en su sintaxis).

Este tipo de módulos se identifican porque no incluyen la sección MAIN en su sintaxis. Un módulo librería puede estar compuesto por distintos objetos no procedurales del CTL: funciones (librerías), menús y «pulldowns». Estos objetos, y siempre mediante la ejecución de funciones, pueden ser compartidos por uno o varios programas de la aplicación. Es decir, para ejecutar un menú o un «pulldown» de una librería, éstos siempre tienen que ejecutarse desde una función local a dicho módulo.

Los módulos de librerías no pueden ejecutarse por sí solos. La única forma de intervenir en ejecución es invocando cualquier función definida en ellos desde otro módulo principal o librería. No obstante, esta ejecución es individual de dicho objeto y no del módulo completo.

IMPORTANTE

Un módulo librería se carga en memoria en el momento que desde otro módulo (principal o librería) se invoca a cualquiera de sus objetos. Este módulo no se descargará de memoria hasta que no se abandone la aplicación.

La conjunción de los dos tipos de módulos citados anteriormente es lo que conforma los diferentes programas de una aplicación.

Un programa es un conjunto de módulos enlazados entre sí mediante el comando ctlink, generando un fichero de extensión «.lnk». En el conjunto de módulos que componen un programa, sólo uno de ellos obligatoriamente ha de tener la sección MAIN. El resto de los módulos componentes del programa han de ser librerías, es decir, módulos sin sección MAIN. Entre todos los módulos que componen un programa, los nombres de las funciones (librerías) deben ser unívocos.

Como ya hemos indicado anteriormente, no es necesario convertir un módulo principal en programa para poder ejecutarlo. Esto significa que, a la hora de ejecutar un programa, el comando ctl buscará el fichero de extensión «.lnk» y, en caso de no encontrarlo, ejecutará el fichero de extensión «.oct».

Será necesario volver a enlazar módulos para la construcción de un programa cuando se dé cualquiera de las siguientes circunstancias:

- Cuando aumente o disminuya el número de módulos que componen el programa.
- Cuando cambie de nombre cualquiera de las funciones de un módulo componente del programa o bien cuando aumente o disminuya el número de funciones.

Es aconsejable que los programas generados tengan el mismo nombre que el de su correspondiente módulo principal.

Para crear un programa desde el entorno de desarrollo habrá que ejecutar la opción «Definir» («Programas»), e indicar los nombres de los módulos que lo componen. Una vez realizada esta operación, la opción «Enlazar» será la encargada de unir todos los módulos, generando el programa «.lnk».

Para llegar a la opción «Definir» hay que seleccionar la siguiente secuencia de opciones:

«Desarrollo» — «Programas» — «Entrada Datos/Listados/Menús/Librerías/Varios» — «Definir»

La forma de ejecutar un programa CTL desde la línea de comando del sistema operativo es mediante el comando `ctl` seguido del nombre del programa:

`ctl nombre_programa`



En las versiones de MultiBase para Windows, la forma de llevar a cabo esta operación sería generando un icono desde el Administrador de Programas de dicho entorno, incluyendo los parámetros correspondientes en la línea de comando. Por ejemplo:

`c:\ctl\bin\ctl.exe -ef fichero.env -cd c:\mbdemo nombre_programa`

Por su parte, la ejecución de un programa desde otro se puede realizar de dos formas distintas mediante la utilización de instrucciones CTL:

1.— Mediante la instrucción CTL. Con esta instrucción, el segundo programa compartirá los mismos procesos del sistema que el primero.

2.— Mediante la instrucción RUN. Con esta instrucción, el segundo programa dispone de procesos del sistema nuevos, independientemente de los del primer programa.

NOTA: Es aconsejable que el nombre del programa sea igual al del módulo principal (módulo que contenga la sección MAIN). De esta forma, cuando estemos situados en el menú de manejo de módulos en el entorno de desarrollo, al «Probar» el módulo principal estaremos ejecutando indirectamente el programa, ya que el CTL buscará el fichero «.lnk» antes que el «.oct».

Secciones de un módulo fuente CTL

Un módulo fuente CTL está dividido en secciones, cada una de las cuales es un apartado de un módulo con una función particular. A continuación se indican las características y funciones de cada una de estas secciones y dónde pueden ubicarse dentro de los dos tipos de módulos CTL.

1.— Sección DATABASE. Sección opcional común a los módulos principales y librerías. Su función es determinar qué base de datos se utilizará en compilación. Su definición es obligatoria en el momento que se define cualquier objeto, procedural o no procedural, mediante la cláusula «LIKE». Asimismo, será necesaria su definición en el caso del objeto no procedural «FORM», ya que éste está relacionado directamente con la base de datos. Si no se indica lo contrario dentro del flujo del programa, la base de datos indicada en esta sección será la que se activará en la explotación de la aplicación.

2.— Sección DEFINE. Sección opcional común a módulos principales y librerías. Esta sección define los objetos a emplear en el módulo. Dichos objetos se clasifican en producedurales y no procedurales y son los siguientes:

- Procedurales: VARIABLES, PARÁMETROS y ARRAYS.
- No Procedurales: FORM, FRAME y STREAM.

Las particularidades de cada uno de estos módulos se indican en sus respectivos capítulos dentro de este mismo manual.

3.— Sección GLOBAL. Sección opcional que únicamente puede incluirse en el módulo principal. Su función consiste en la definición de características y acciones globales de un programa. Esto significa que son acciones sobre el módulo principal y los módulos de librerías. Esta sección está compuesta a su vez por otras dos: CONTROL y OPTIONS.

- La sección CONTROL de la GLOBAL de un módulo principal contiene diversas instrucciones no procedurales, cuya ejecución se activa cuando comienza el programa («ON BEGINNING»), cuando finaliza la ejecución de la última instrucción de la sección MAIN («ON ENDING») y cuando se pulsa una tecla concreta en cualquier punto de ejecución del programa («ON KEY» y «ON ACTION») o programas sucesivos llamados desde el que contiene la sección GLOBAL.
- Por su parte, la sección OPTIONS define ciertas características comunes a todos los módulos del programa. Realmente, esta sección es idéntica y admite todas las cláusulas de la instrucción OPTIONS. Esta instrucción, junto con todas sus cláusulas, se describe en el Manual de Referencia.

4.— Sección LOCAL. Sección opcional que sólo puede definirse en los módulos de librerías. Esta sección permite redefinir características locales a un módulo (librería). Está compuesta a su vez por una sección OPTIONS, cuyo objeto es idéntico a la de la OPTIONS de la GLOBAL, explicada anteriormente. Esta sección se corresponde con la instrucción OPTIONS del CTL (ver Manual de Referencia).

5.— Sección MAIN. Esta sección sólo puede incluirse en un módulo principal, siendo además la única obligatoria. Esta sección sirve de punto de entrada a la ejecución de un programa. Al ejecutar la última instrucción de esta sección, siempre y cuando no tenga repercusión sobre algún objeto no procedural de CTL, se terminará la ejecución del programa en curso.

6.— FUNCTION. Palabra reservada que indica la definición de una de las funciones locales al módulo. Estas funciones son objetos que pueden incluirse tanto en el módulo principal como en los módulos de librerías, y contienen un conjunto de instrucciones CTL cuyo código es reutilizable en cualquier punto del módulo (incluido el módulo principal) o del programa (si se incluye en cualquier módulo librería).

7.— MENU/PULLDOWN. Palabras reservadas que indican respectivamente la definición de menús de tipo «Lotus» (o «de iconos» en Windows) y «pulldown». La definición de estos objetos (menús y «pulldowns») puede incluirse tanto en el módulo principal como en los módulos de librerías, y siempre deberá llevarse a cabo en el módulo que lo vaya a utilizar.

Estructura de un módulo principal CTL

La estructura o «esqueleto» de un módulo principal es la siguiente:

```
[DATABASE base_datos] [
    DEFINE
        [objetos_procedurales] [objetos_no_procedurales]
    END DEFINE]

[GLOBAL
    [CONTROL
        lista_bloques
    END CONTROL]
    [OPTIONS
        opciones_ctl
    END OPTIONS]
```

```
END GLOBAL]

MAIN BEGIN
    instrucciones
...
END MAIN

[FUNCTION nombre_función([parámetros])
    instrucciones
END FUNCTION]

[MENU nombre_menú
    opciones_menú
END MENU]

[PULLDOWN nombre_pulldown
    menús
END PULLDOWN]
```

Donde:

base_datos	Nombre de una base de datos existente en el directorio en curso o en alguno de los definidos en la variable de entorno DBPATH.
objetos_procedurales	Definición de un objeto de DEFINE válido. Estos objetos procedurales son: VARIABLES, PARÁMETROS y ARRAYS.
objeto_no_procedurales	Definición de un objeto de DEFINE no procedural válido. Estos objetos no procedurales son: FORM, FRAME y STREAM.
lista_bloques	Instrucciones no procedurales válidas. Estas instrucciones pueden ser las siguientes: «ON BEGINNING», «ON ENDING», «ON KEY» y «ON ACTION».
opciones_ctl	Opciones permitidas por la instrucción OPTIONS (ver Manual de Referencia).
instrucciones	Cualquier instrucción CTL válida o conjunto de instrucciones entre BEGIN y END.
opciones_menú	Definición de las opciones de un menú.
menús	Definición de menús pertenecientes a un menú de tipo «pulldown».

NOTA: El orden de definición de las funciones (FUNCTION), menús de tipo «Lotus» —de «iconos» en Windows— (MENU), y los «pulldown» (PULLDOWN) es indiferente, siempre y cuando vayan después de la sección MAIN.

Estructura de un módulo librería CTL

La estructura de un módulo librería es la siguiente:

```
[DATABASE base_datos]

[DEFINE
    [objetos_procedurales]
    [objetos_no_procedurales]
END DEFINE]

[LOCAL
    [OPTIONS
        opciones_ctl
    END OPTIONS]
END LOCAL]
```



```
[FUNCTION nombre_función([parámetros])
    instrucciones
END FUNCTION]
```

```
[MENU nombre_menú
    opciones_menú
END MENU]
```

```
[PULLDOWN nombre_pulldown
    menús
END PULLDOWN]
```

Donde:

base_datos	Nombre de una base de datos existente en el directorio en curso o en alguno de los definidos en la variable de entorno DBPATH.
objetos_procedurales	Definición de un objeto de DEFINE válido. Estos objetos procedurales son: VARIABLES, PARÁMETROS y ARRAYS.
objetos_no_procedurales	Definición de un objeto de DEFINE no procedural válido. Estos objetos no procedurales son: FORM, FRAME y STREAM.
opciones_ctl	Opciones permitidas por la instrucción OPTIONS (ver Manual de Referencia).
instrucciones	Cualquier instrucción CTL válida o conjunto de instrucciones entre BEGIN y END.
opciones_menú	Definición de las opciones de un menú.
menús	Definición de menús pertenecientes a un menú de tipo «pulldown».

NOTA: El orden de definición de las funciones (FUNCTION), menús de tipo «Lotus» —de «iconos» en Windows— (MENU), y los «pulldown» (PULLDOWN) es indiferente, siempre y cuando vayan después de la sección LOCAL.

Componentes de un módulo

Un módulo fuente CTL está compuesto por ciertos elementos de programación que hacen posible el desarrollo de una aplicación. Entre otros, un módulo puede estar compuesto por los siguientes elementos:

1.— Secciones: Como ya hemos indicado en los apartados anteriores, referidos a la estructura de un módulo principal y de librería, un módulo está compuesto por secciones. Estas secciones son: DATABASE, DEFINE, GLOBAL (módulo principal), LOCAL (módulo librería), MAIN (módulo principal), objetos FUNCTION, objetos MENU y objetos PULLDOWN.

2.— Objetos: Un objeto CTL es un elemento de programación. En un módulo CTL se pueden definir (dentro de la sección DEFINE) objetos procedurales y no procedurales. Los objetos denominados procedurales son: VARIABLES, PARAMETROS y ARRAYS. Por su parte, los no procedurales son: FORM, FRAME y STREAM.

Asimismo, existe otro tipo de objetos de programación, que son los siguientes:

- **CURSOR:** Objeto que maneja información de la base de datos. Su definición puede realizarse en cualquier punto de un módulo donde pueda incluirse una instrucción de control de flujo.
- **FUNCTION:** Objeto función que se define, como hemos visto en la estructura de cada tipo de módulo, debajo de la sección MAIN en un módulo principal y después de la sección LOCAL de un módulo librería.
- **MENU:** Objeto menú de tipo «Lotus» (de «iconos» en Windows). Su definición se realiza debajo de la sección MAIN en un módulo principal y después de sección LOCAL de un módulo librería.

- **PULLDOWN:** Objeto «pulldown» que se define debajo de la sección MAIN en un módulo principal y después de la sección LOCAL de un módulo librería.

3.— Identificadores: A cada uno de los elementos de programación CTL se le asigna un identificador. Este identificador será el medio de referenciar a uno de estos elementos.

4.— Atributos CTL/CTSQL: A la hora de definir un objeto o elemento de programación CTL es posible asignarle ciertas características para su funcionamiento.

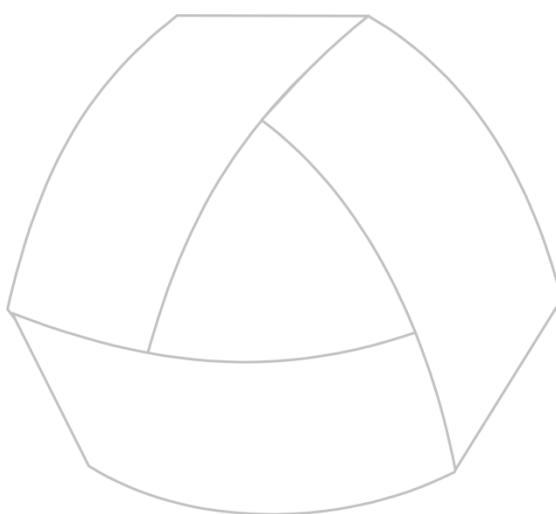
5.— Expresiones: En programación se utilizan constantemente expresiones CTL. Las expresiones válidas en CTL se explican en el siguiente capítulo de este mismo manual.

6.— Instrucciones procedurales: Utilización de instrucciones 3GL.

7.— Instrucciones no procedurales: Una instrucción no procedural es aquella cuya ejecución no es secuencial, sino que depende de las acciones y del transcurso del programa en cada momento.

8.— Instrucciones SQL embebidas: CTL dispone de todas las instrucciones de tipo SQL embebidas en su sintaxis, pudiendo ejecutarlas como si fuesen instrucciones propias del lenguaje de cuarta generación.

NOTA: Las características de cada uno de estos conceptos se indican en el siguiente capítulo.



MultiBase
MANUAL DEL PROGRAMADOR

CAPÍTULO 5. Conceptos básicos del CTL

Identificador CTL

Un identificador CTL es el nombre asignado a un elemento CTL. Este nombre puede estar formado por letras, números y signos de subrayado («_») —no emplear nunca signos aritméticos: «-», «+», «*», «/»—. Obligatoriamente, el primer carácter deberá ser siempre una letra. A menos que se especifique lo contrario, un identificador CTL puede tener de uno a cincuenta caracteres. Su estructura es la siguiente:

letra [subrayado | letra | número]

Donde:

letra	Puede ser mayúsculas o minúsculas. Rango «a-z».
subrayado	Carácter de subrayado («_»).
número	Carácter numérico. Rango «0-9».

En un mismo módulo CTL los identificadores de un mismo tipo de elemento CTL deben ser unívocos.

Tipos de datos CTL

Un tipo de datos es un conjunto de valores representables. Un valor es atómico y, por tanto, no permite subdivisiones. Un valor puede ser una cadena de caracteres alfanuméricos o un número. El tipo de datos determina los valores que acepta una columna de una tabla, así como su modo de almacenamiento. Los tipos de datos posibles son los siguientes: CHAR, SMALLINT, INTEGER, TIME, DECIMAL, DATE, MONEY.

CHAR(n)	Cadena de caracteres alfanuméricos de longitud «n». «n» tiene que ser mayor o igual que uno «1» y menor o igual que «32.767» («1<=n» y «<=32.767»). El número de bytes que reserva en disco este tipo de dato es el indicado en «n». El parámetro «n» es necesario para la definición del elemento en concreto.
SMALLINT	Este tipo de dato permite almacenar números enteros. El rango de valores numéricos que admite está entre «-32.767» y «+32.767». Este tipo de datos ocupa siempre 2 bytes, independientemente del número de caracteres numéricos a grabar. Es decir, si se introduce el valor «1», éste ocupará lo mismo que el valor «32.767».
INTEGER	Este tipo de dato permite almacenar números enteros. El rango de valores numéricos que admite es desde «-2.147.483.647» hasta «+2.147.483.647», ambos inclusive. Este tipo de datos ocupa siempre 4 bytes, independientemente del número de caracteres numéricos que se incluyan como dato.
TIME	Este tipo de dato admite horas con el formato por defecto «HH:MM:SS» y almacenada como el número de segundos desde las «00:00:01». Este tipo de datos es equivalente a un INTEGER, pero el rango de valores admitidos está entre «1» y «86.400», que se corresponden respectivamente con los valores «00:00:01» y «24:00:00». Al igual que el tipo de dato INTEGER, ocupa 4 bytes. Los tipos TIME se introducen como una secuencia de «hora, minutos y segundos», sin separador entre ellos (tampoco espacios en blanco). Su representación depende de la variable de entorno DBTIME (para más información consulte el capítulo relativo a las variables de entorno en el Manual del Administrador). Asimismo, permite ordenaciones, operaciones y comparaciones horarias entre dos elementos CTL de tipo TIME.

- DECIMAL [(m[,n])]** Tipo de dato que almacena números decimales de coma flotante con un total de «m» dígitos significativos (precisión) y «n» dígitos a la derecha de la coma decimal (escala). «m» puede ser como máximo menor o igual a 32 («m <= 32») y «n» menor o igual que «m».
- Cuando se asignan valores a «m» y «n», la variable decimal tendrá punto aritmético fijo. El segundo parámetro «n» es opcional, y si se omite se tratará como un decimal de coma flotante. Un elemento «decimal(m)» tiene una precisión «m» y un rango de valor absoluto entre «10-130» y «10125». En caso de no especificar parámetros a un elemento DECIMAL, éste será tratado como «decimal(16)».
- SERIAL[(n)]** Este tipo de datos de CTSQL no puede utilizarse en CTL para definir ningún elemento. No obstante, equivale a un INTEGER.
- DATE** Tipo de datos que admite fechas con el formato por defecto «DD/MM/YYYY» y las almacena como el número de días partiendo desde el «01/01/1900». Dicho número será positivo o negativo dependiendo de si la fecha es posterior o anterior, respectivamente, a dicho valor inicial. Este tipo de datos es equivalente a un INTEGER, ocupando 4 bytes.
- Los tipos DATE son introducidos como una secuencia «día, mes y año», con caracteres numéricos. El día puede representarse como el día del mes (1 ó 01, 2 ó 02, etc.). El mes se representa como un número (Enero: 1, Febrero: 2, etc.). El año se representa como un número de cuatro dígitos (0001 a 9999). En caso de mostrar dos dígitos para el año, CTSQL asume que el año es «19yy».
- Se puede ordenar por una columna de tipo DATE y hacer una comparación cronológica entre dos columnas de tipo DATE.
- El orden en que se introduzcan el año, mes y día dependerá del valor de la variable de entorno DBDATE (consulte el capítulo sobre «Variables de Entorno» en el Manual del Administrador).
- MONEY [(m[,n])]** Tipo de dato que almacena números decimales de coma flotante con un total de «m» dígitos significativos (precisión) y «n» dígitos a la derecha de la coma decimal (escala) —igual al tipo de dato DECIMAL—. Este tipo de datos se utiliza para representar cantidades referentes a unidades monetarias. La variable de entorno que utiliza para identificar la unidad monetaria a la que se refiere el importe definido como valor es DBMONEY (consulte el capítulo sobre «Variables de Entorno» en el Manual del Administrador).
- El tipo de dato «money(m)» se define como el tipo de dato «decimal(m,2)». Si no se especifican parámetros, MONEY se trata como una variable de tipo «decimal(16,2)».

El siguiente cuadro recoge de manera esquemática las principales características de los distintos tipos de datos citados anteriormente.

Tipo de dato	Valor mínimo	Valor máximo	Ocupación (bytes)
CHAR(n)	1 carácter	32.767 caracteres	n
SMALLINT	-32.767	+32.767	2
INTEGER	-2.147.483.647	+2.147.483.647	4
TIME	00:00:01	24:00:00	4
DECIMAL(m,n)	10-130	10125	1+m/2

Tipo de dato	Valor mínimo	Valor máximo	Ocupación (bytes)
DATE	01/01/0001	31/12/9999	4
MONEY(m,n)	10-130	10125	1+m/2

La conversión de estos tipos de datos entre sí es válida en todos los casos siempre y cuando los valores a convertir cumplan las reglas de rango indicadas anteriormente para cada tipo en particular.

NOTA: Los tipos de datos vistos en este apartado, además del SERIAL, también serán válidos para la definición de las columnas en las tablas de la base de datos.

Atributos

Un atributo es una característica asignada a ciertos elementos CTL, y sólo podrá utilizarse en la definición de éstos. Los atributos aquí comentados corresponden exclusivamente al CTL, si bien algunos de ellos afectan igualmente al gestor CTSQL de MultiBase. Por su parte, el atributo REQUIRED del CTL equivale al NOT NULL del CTSQL.

Los atributos del CTL son los siguientes (entre paréntesis se indican también aquellos que afectan al CTSQL):

AUTONEXT	NOUPDATE (CTSQL)
CHECK (CTSQL)	PICTURE (CTSQL)
COMMENTS	QUERYCLEAR
DEFAULT (CTSQL)	REQUIRED
DOWNSHIFT (CTSQL)	REVERSE
FORMAT (CTSQL)	RIGHT (CTSQL)
HELP	SCROLL
INCLUDE	UNDERLINE
LABEL (CTSQL)	UPSHIFT (CTSQL)
LEFT (CTSQL)	VERIFY
LOOKUP	ZEROFILL (CTSQL)
NOENTRY (CTSQL)	

A continuación se comenta en detalle cada uno de estos atributos en lo que respecta exclusivamente a su utilización en CTL.

AUTONEXT (CTL)

Este atributo activa el siguiente campo editable cuando se alcanza o sobrepasa la longitud del campo que tiene asignada. Es decir, en caso de llegar al último carácter editable de un campo con este atributo, no es necesario pulsar [RETURN] para pasar al siguiente campo editable de la pantalla. Su sintaxis es la siguiente:

AUTONEXT

Este atributo puede asignarse a cualquier variable CTL que se utilice para entrada de datos.

CHECK (CTL y CTSQL)

Este atributo se utiliza para definir una expresión de tipo booleana (condición). Todos los valores de la variable que hagan que dicha expresión evalúe a FALSE serán rechazados. Su sintaxis es la siguiente:

CHECK (condición)

Donde:

condición

Lista de una o más subcondiciones separadas por operadores AND, OR y NOT. Estas condiciones pueden incluir el signo dólar («\$») para sustituir el nombre de la variable a la que se aplica este atributo. Las condiciones válidas en CTL son las siguientes:

expresion operador_relación expresion

Donde:

operador_relación Puede ser uno de los siguientes:

=	Igual.
!= o <>	Distinto.
>	Mayor que.
>=	Mayor o igual que.
<	Menor que.
<=	Menor o igual que.

[NOT] expresión BETWEEN expr1 AND expr2

Devuelve verdadero o falso en caso de que el resultado de una expresión «\$» esté o no comprendido en el rango dado por otras dos (ambas inclusive). La partícula «NOT» invierte el resultado.

[NOT] expresión IN (lista_valores)

Devuelve verdadero o falso en caso de que el resultado de una expresión «\$» sea igual o no a uno de los valores incluidos en «lista_valores». La partícula «NOT» invierte el resultado.

Donde:

lista_valores Lista de valores separados por comas.

[NOT] expresión LIKE "literal"

Devuelve verdadero o falso en caso de que el resultado de una expresión se ajuste o no al patrón definido en «literal». La partícula «NOT» invierte el resultado.

Donde:

literal	Expresión alfanumérica que puede contener dos metacaracteres que tienen un significado especial:
%	Indica cero o más caracteres en la posición donde aparezca.
_ (subrayado)	Indica cualquier carácter (uno solo) en la posición indicada.

[NOT] expresión MATCHES "literal"

Devuelve verdadero o falso en caso de que el resultado de una expresión se ajuste o no al patrón definido en «literal». La partícula «NOT» invierte el resultado.

Donde:

«literal»	Expresión alfanumérica que puede contener varios metacaracteres que tienen un significado especial:
*	Indica cero o más caracteres en la posición indicada.
?	Indica cualquier carácter (uno solo) en la posición donde aparezca.
[lista]	Indica que cualquiera de los caracteres incluidos en la «lista» puede aparecer en esa posición. Se puede indicar un rango de caracteres separando el inicial y el final por medio de un guión. Si el primer carácter de «lista» es «^», indica que los caracteres o rangos no pueden aparecer en esa posición.
\	Indica que el carácter al que precede no debe ser considerado como especial (metacarácter). Siempre precederá a cualquiera de los metacaracteres indicados anteriormente.

expresión **IS [NOT] NULL**

Devuelve verdadero o falso en caso de que el resultado de una expresión sea o no un valor nulo. La partícula «NOT» invierte el resultado.

Todas las condiciones indicadas pueden enlazarse mediante los siguientes operadores lógicos:

AND	Devolverá TRUE si ambas condiciones evalúan a TRUE. condición AND condición
OR	Devolverá TRUE si al menos una de las condiciones evalúa a TRUE. condición OR condición

NOTAS:

- 1.— Los paréntesis agrupan operandos y operaciones alterando la precedencia de evaluación de los operadores.
- 2.— En todos estos puntos, «expresión» puede sustituirse por el signo «\$», lo que indicará que se trata del elemento CTL (variable) que se está definiendo.
- 3.— Si una expresión devuelve «NULL» no hará cierta ninguna condición, excepto la condición «expresión IS NULL».

COMMENTS (CTL)

Este atributo muestra un mensaje de ayuda en la línea de mensajes cuando el cursor de la pantalla se posiciona en el campo al que se ha asignado dicho atributo. Su sintaxis es la siguiente:

COMMENTS {"literal" | número_ayuda}

Donde:

literal	Cadena entrecomillada de caracteres alfanuméricos.
número_ayuda	Número de mensaje de un fichero de ayuda. Este comentario desaparecerá de la pantalla cuando se salte al siguiente campo del que se ha asignado dicho atributo.

En caso de indicar el «número_ayuda» habrá que especificar también el fichero de ayuda que lo contendrá. La forma de seleccionar un fichero de ayuda se comenta en el capítulo relativo a «Ficheros de Ayuda» en este mismo volumen.

DEFAULT (CTL y CTSQL)

Este atributo se utiliza para asignar un valor por defecto a una variable en el momento de editarla en modo agregar para provocar inserción de valores nuevos. Su sintaxis es la siguiente:

DEFAULT valor

Donde:

valor	Constante numérica o alfanumérica que se tomará por defecto. Si la variable es de tipo DATE, TIME o CHAR, este valor tiene que ir entre comillas.
-------	---

En caso de no utilizar este atributo, todas las variables definidas en un programa tomarán por defecto el valor nulo («NULL»). Hay que tener en cuenta que si cualquier variable con tipo de dato que permita realizar operaciones aritméticas (SMALLINT, INTEGER, TIME, DECIMAL, DATE o MONEY), si dicha variable tiene asignado el valor «NULL», el resultado de la operación será igualmente nulo. Por ejemplo:

variable = variable + 1

En este ejemplo, si «variable» tiene asignado el valor «NULL», el resultado de la operación será también nulo.

En las variables de tipo DATE y TIME pueden utilizarse las variables internas de CTSQL para asignar valores por defecto («TODAY» y «NOW», respectivamente).

DOWNSHIFT (CTL y CTSQL)

Este atributo convierte las letras mayúsculas de los valores de una variable de tipo CHAR en minúsculas en el momento de la asignación de valores. Su sintaxis es la siguiente:

DOWNSHIFT

En caso de coincidencia entre los atributos DOWNSHIFT y UPSHIFT, CTL tendrá en cuenta únicamente el último que se encuentre en la definición de la variable.

FORMAT (CTL y CTSQL)

Este atributo permite establecer el formato de presentación de los valores de las variables a las que se asigne. Este atributo puede asignarse a todos los tipos de datos, excepto a CHAR (éste emplea el atributo PICTURE). Su formato es el siguiente:

FORMAT "formato"

Donde:

formato	Cadena de caracteres entre comillas que indica el formato. Dependiendo del tipo de dato, los posibles formatos son:
	a) Numéricos (SMALLINT, INTEGER, MONEY y DECIMAL). Los caracteres a emplear son los siguientes:
#	Representa cada dígito. No cambia los espacios en blanco por ningún otro carácter.
.	El punto indica la posición de la coma decimal. Aparecerá una coma o un punto dependiendo de la definición de la variable de entorno DBMONEY.

- , Permite indicar la separación de miles, millones, etc. Se imprimirá una coma si DBMONEY tiene como valor un punto, y un punto si dicho valor es una coma (para más información consulte el capítulo sobre «Variables de Entorno» en el Manual del Administrador).
- * Rellena espacios en blanco con asteriscos.
- & Rellena los espacios en blanco con ceros.
- Imprime un signo menos cuando la expresión sobre la que se aplica es menor que cero. Cuando se especifican varios signos «-», sólo aparecerá uno lo más a la derecha posible.
- + Imprime el signo «+» o «-» dependiendo de que la expresión sobre la que se aplica sea mayor o igual a cero («+») o menor que cero («-»).
- (Imprime un signo «abrir paréntesis» antes de un número negativo. Cuando se especifican varios signos «(», sólo aparecerá uno lo más a la derecha posible.
-) Imprime este mismo signo al final de un formato en el que se ha especificado «)».
- \$ Literal que se imprime como un signo de moneda. Cuando se indican varios en la plantilla, aparecerá uno solo lo más a la derecha posible. Si se ha definido la parte «front» de la variable de entorno DBMONEY, será esto último lo que se incluirá en el resultado, y no el signo «\$» (consulte la variable DBMONEY en el capítulo sobre «Variables de Entorno» en el Manual del Administrador).
- % Este literal se imprimirá como tal en la posición donde se especifique. En caso de emplear más de un «%», aparecerán en las posiciones asignadas. Estos signos no tienen ningún efecto sobre la expresión a evaluar.

El siguiente cuadro recoge de forma resumida algunos ejemplos sobre la utilización de diferentes tipos de formatos:

Números	formato	Resultado
-13423,41	"###,###.##"	13.423,41
13423,41	"###,###.##"	13.423,41
124713,12	"###,###.##"	*****
30	"###,###.##"	30,00
2478	"&&, &&&"	02.478
2076	"** , ***"	*2.076
20	"** , ***"	****20
-23453	"+++ ,+++"	-23.453
-1	" +##,###"	- 1
23453	"+++ ,+++"	+23.453
1	" +##,###"	+ 1
-6498	"—, —"	-6.498
20	"—, —"	20
-25	"-##,###"	- 25
-3712	"(++,+++)"	(-3.712)
-50	"—, —%"	- 50%

Números	formato	Resultado
-24	"(-,-%)"	(- 24%)
24	"(-,-%)"	24%

b) Fechas (DATE). En este caso los caracteres que se evalúan con un significado para la plantilla de formato son «d», «m» e «y», en las combinaciones que se indican a continuación, representándose cualquier otro carácter como tal.

dd	Dos dígitos para el día del mes (01 a 31).
ddd	Tres letras significativas del nombre del día de la semana (Lun a Dom).
mm	Dos dígitos para el mes (01 a 12).
mmm	Tres letras significativas del nombre del mes (Ene a Dic).
yy	Dos dígitos para el año desde 1900 (00 a 99).
yyyy	Cuatro dígitos para el año (0000 a 9999).

Casos prácticos:

Fecha	formato	Resultado
01/07/2018	dd-mm-yyyy	01-07-2018
01/07/2018	dd ddd / mm mmm / yyyy	01 Vie / 07 Jul / 2018
01/07/2018	dd / mmm / yyyy	01 / Jul / 2018
01/07/2018	dd-mm-yy	01-07-18

c) Horas (TIME). En este caso los caracteres que se evalúan con un significado para la plantilla de formato son «h», «m» y «s», en las combinaciones que se indican a continuación:

hh:mm:ss	Dos dígitos para la hora, dos para los minutos y dos para los segundos.
hh:mm	Dos dígitos para la hora y dos dígitos para los minutos.
hh	Dos dígitos para la hora.

Casos prácticos:

15:10:20	hh:mm:ss	15:10:20
15:10:20	hh:mm	15:10
15:10:20	hh	15

HELP (CTL)

Este atributo referencia a un mensaje de ayuda para el campo al que se asigne. Dicho mensaje aparecerá en una ventana cuando se active la tecla asignada a la acción <fhhelp> en el fichero «tactions» ([F10] en UNIX). Su sintaxis es la siguiente:

HELP número_ayuda

Donde:

número_ayuda Número de mensaje de un fichero de ayuda.

En caso de indicar el «número_ayuda» habrá que especificar el fichero de ayuda que lo contendrá. La forma de seleccionar un fichero de ayuda se indica en el capítulo relativo a «Ficheros de Ayuda» en este mismo volumen.

La ventana de ayuda desaparecerá de la pantalla cuando se pulse la tecla asignada a la acción <fquit> en el fichero de acciones ([F2] en UNIX y [ESC] en Windows).

INCLUDE (CTL)

Este atributo indica la lista o rango de valores admitidos por la variable a la que se le asigna. Su sintaxis es la siguiente:

INCLUDE (lista_valores)

Donde:

lista_valores	Lista de valores separados por comas («valor1, valor2, valor3, ...») o un rango de valores («valor1 to valorx»), o bien una combinación de ambos («valor1, valor2, valor3 to valorn»).
---------------	--

En el caso de indicar rangos en las variables de tipo CHAR, TIME y DATE, los valores indicados entre paréntesis deben ir entre comillas. Sin embargo, cuando se indica una lista de valores el entrecomillado es opcional.

Cuando se especifica un rango de valores, el menor de ellos debe aparecer el primero.

LABEL (CTL y CTSQL)

Este atributo define una etiqueta (título) para la variable que se está definiendo. Dicha etiqueta será mostrada con las instrucciones de entrada/salida del lenguaje de cuarta generación CTL. Asimismo, en caso de asignarse a una variable de un FORM, dicho literal definirá un «alias» para dicha variable.

La sintaxis de este atributo es la siguiente:

LABEL {"literal" | número_ayuda}

Donde:

literal	Cadena de caracteres alfanumérica entre comillas que corresponderá a la etiqueta de la variable o al alias de la variable del FORM.
número_ayuda	Número de mensaje de un fichero de ayuda.

En caso de indicar el «número_ayuda» habrá que especificar el fichero de ayuda que lo contendrá. La forma de seleccionar un fichero de ayuda se explica en el capítulo relativo a «Ficheros de Ayuda» en este mismo volumen.

LEFT (CTL y CTSQL)

Alínea a la izquierda el valor asignado a una variable. Su sintaxis es la siguiente:

LEFT

Si no se especifica ningún atributo de alineación, los datos numéricos se alinean por defecto a la derecha y los del tipo de dato CHAR a la izquierda.

NOTA: En caso de coincidir el atributo LEFT con RIGHT, CTL obedecerá al último que se encuentre en la definición de la variable.

LOOKUP (CTL)

Este atributo muestra el contenido de una o varias columnas de una tabla que tenga(n) relación con una columna de la tabla activa que se mantiene en el FORM. También actúa sobre las variables de un FRAME. Su sintaxis es la siguiente:

LOOKUP [identificador = columna [,]]
 [identificador = columna [, ...]]
JOINING [*] tabla.columna_join

Donde:

identificador	Nombre asignado al espacio reservado para la representación de una variable de FORM/FRAME en su respectiva sección SCREEN.
columna	Nombre de la columna perteneciente a la tabla «tabla» que se mostrará en el lugar reservado por «identificador».
* (asterisco)	Este carácter hace que el cruce sea obligatorio, es decir, el valor introducido en la variable del mantenimiento (FORM/FRAME) deberá forzosamente existir en la segunda tabla.
tabla	Nombre de la tabla de la que se extraerá la información de cruce.
columna_join	Nombre de la columna que servirá de enlace para la extracción de los valores de la columna «columna».

NOTA: Para más información acerca de la actuación de este atributo consulte los capítulos relativos al [FORM](#) y al [FRAME](#) en este mismo volumen.

NOENTRY (CTL y CTSQL)

Este atributo no permite insertar un valor a la variable (FORM/FRAME) mediante las instrucciones de edición de valores (ADD, ADD ONE, INTERCALATE, INPUT FRAME... FOR ADD). Sin embargo, no impedirá la actualización o modificación de un valor asignado previamente (MODIFY, INPUT FRAME... FOR UPDATE). Su sintaxis es la siguiente:

NOENTRY

NOUPDATE (CTL y CTSQL)

Este atributo no permite actualizar un valor a la variable (FORM/FRAME) mediante las instrucciones de modificación de valores (MODIFY, INPUT FRAME... FOR UPDATE). Sin embargo, no impedirá la acción de insertar un valor nuevo sobre dichas variables (ADD, ADD ONE, INTERCALATE, INPUT FRAME... FOR ADD). Su sintaxis es la siguiente:

NOUPDATE

PICTURE (CTL y CTSQL)

Este atributo especifica la máscara de edición de una variable de tipo CHAR. Su sintaxis es la siguiente:

PICTURE "máscara"

Donde:

máscara	Cadena de caracteres que especifica el formato deseado. Esta cadena tiene que ir entre comillas y puede estar compuesta por la combinación de los siguientes símbolos:
---------	--

<u>Símbolo</u>	<u>Representación</u>
A	Cualquier carácter alfabético.
#	Cualquier carácter numérico.
X	Cualquier carácter alfanumérico.

Cualquier otro carácter se tratará como un literal y aparecerá en el campo en la misma posición que se especifique en la máscara.

QUERYS CLEAR (CTL)

Este atributo «limpia» los valores presentados por el atributo LOOKUP al efectuar una consulta. Su sintaxis es la siguiente:

QUERYS CLEAR

Este atributo sólo tiene sentido cuando existe el correspondiente LOOKUP sobre el que actúa.

REQUIRED (CTL)

Este atributo obliga a que la variable (FORM/FRAME) tenga un valor distinto de nulo («NULL»). Su sintaxis es la siguiente:

REQUIRED

CTL utiliza «NULL» como valor por defecto en la inicialización de variables. Este atributo no tiene efecto en actualización o modificación de variables. En caso de asignar los atributos DEFAULT y REQUIRED a una variable, este último no tiene efecto.

REVERSE (CTL)

Este atributo muestra el valor de la variable en vídeo inverso cuando se utilicen instrucciones de entrada de valores por teclado (PROMPT FOR, ADD, MODIFY, INTERCALATE, INPUT FRAME, etc.). Su sintaxis es la siguiente:

REVERSE

RIGHT (CTL y CTSQL)

Alínea a la derecha el valor asignado a una variable. Su sintaxis es la siguiente:

RIGHT

Si no se especifica ningún atributo de alineación, los datos numéricos se alinean por defecto a la derecha y los de tipo CHAR a la izquierda.

NOTA: En caso de coincidir este atributo con LEFT, CTL obedecerá al último que se encuentre en la definición de la variable.

SCROLL (CTL)

Este atributo realiza «scroll» lateral de izquierda a derecha y viceversa en variables de tipo CHAR cuyo mantenimiento se realiza en los objetos FORM y FRAME. Su sintaxis es la siguiente:

SCROLL

UNDERLINE (CTL)

Este atributo muestra subrayado el valor de la variable cuando se utilicen instrucciones de entrada de valores por teclado (PROMPT FOR, ADD, MODIFY, INTERCALATE, INPUT FRAME, etc.). Su sintaxis es la siguiente:

UNDERLINE

UPSHIFT (CTL y CTSQL)

Este atributo convierte las letras minúsculas de los valores de una variable de tipo CHAR en mayúsculas en el momento de la asignación de valores. Su sintaxis es la siguiente:

UPSHIFT

NOTA: En caso de coincidir este atributo con DOWNSHIFT, CTL obedecerá al último que se encuentre en la definición de la variable.

VERIFY (CTL)

Obliga a introducir un valor dos veces, verificando que sea el mismo en ambas. Su sintaxis es la siguiente:

VERIFY

ZEROFILL (CTL y CTSQL)

Este atributo alinea a la derecha el valor sin tener en cuenta el tipo de dato (numérico o alfanumérico), y rellena con ceros por la izquierda. Su sintaxis es la siguiente:

ZEROFILL

NOTA: Este atributo produce automáticamente RIGHT.

Expresiones CTL

Una expresión CTL es cualquier dato, constante o variable, o la combinación de varios de ellos mediante operadores, que es evaluado y devuelve un resultado. Las expresiones válidas en CTL son las siguientes:

- Expresiones simples.
- Expresiones compuestas.
- Expresiones de STREAMS.
- Expresiones de FORM.
- Expresiones de CURSOR.

En los siguientes apartados se explica en detalle cada una de ellas.

Expresiones simples

Las expresiones clasificadas como simples en CTL son las siguientes:

número	Número expresado como una constante en el programa fuente CTL.
«literal»	Cadena compuesta por caracteres numéricos y/o alfanuméricos entre comillas. Este «literal» se expresa como constante en instrucciones CTL y puede representar una fecha o un tipo de dato hora. Dichos tipos de datos se representan como «dd/mm/yyyy» y «hh:mm:ss», respectivamente.

Un literal no puede contener un salto de párrafo, es decir, no se puede emplear [RETURN]. Para indicar un literal de más de una línea habrá que utilizar la concatenación de dos literales. Por ejemplo:

```
let a= "1234.....          INCORRECTO
      5678....."
let a= "123....."&&      CORRECTO
      "5678....."
```

nombre_variable	Indica el valor que tenga en dicho momento la variable especificada en «nombre_variable», pudiendo ser cualquier tipo de variable definible en un módulo CTL.
TODAY	Palabra reservada que devuelve la fecha del sistema.
NOW	Palabra reservada que devuelve la hora del sistema.
TRUE	Palabra reservada que evalúa siempre a uno («1»), siendo éste su único valor en CTL.
FALSE	Palabra reservada que evalúa siempre a cero («0»), siendo éste su único valor en CTL.
NULL	Palabra reservada que evalúa siempre a nulo (distinto de cualquier cadena, incluso de espacios en blanco o « »).
ROWID	El «rowid» es el número de orden que ocupa físicamente una fila en una tabla. Mediante este valor se produce el acceso más rápido a los datos de la tabla, por lo que su uso es prioritario.

Expresiones compuestas

Las expresiones clasificadas como compuestas en CTL son las siguientes:

-expresión	En expresiones numéricas invierte el signo. Es decir, las cantidades positivas las convierte en negativas y viceversa.
(expresión)	Evalúa en primer lugar la expresión entre paréntesis (precedencia explícita).
expresión operador expresión	Cualquier combinación de expresiones mediante los operadores aritméticos y de formato. Estos operadores se indican en la sección 5.5 de este mismo capítulo.
función(expresión)	Cualquier nombre de función, bien sea interna CTL o definida por el usuario.
expresión[ini[, fin]]	Esta expresión sólo se puede aplicar a variables de tipo CHAR. Devuelve los caracteres que se encuentran desde la posición «ini» hasta la posición «fin», ambas inclusive. En caso de omitir esta última devolverá el carácter que se encuentra en la posición «ini».

Expresiones de STREAMS

Las expresiones relacionadas con el objeto STREAM son las siguientes:

CURRENT {PAGE | LINE} OF {STANDARD | nombre_stream}

Esta expresión devuelve el número de página («PAGE») o línea («LINE») asociado al objeto STREAM cuyo nombre es «nombre_stream». En caso de utilizar el STREAM por defecto en CTL, éste se expresará en la expresión con la palabra reservada «STANDARD». Este «STREAM STANDARD» no es necesario definirlo, ya que se encuentra siempre activo.

Los valores en el caso de «PAGE» serán dependientes de los asignados en la instrucción de CTL «FORMAT STREAM».

EOF OF {STANDARD | nombre_stream}

Esta expresión devuelve verdadero («TRUE») o falso («FALSE») si se ha alcanzado el fin del fichero en la entrada asociada a «nombre_stream». En caso de utilizar el STREAM por defecto en CTL, éste se expresará en la expresión con la palabra reservada «STANDARD». Este «STREAM STANDARD» no hay que definirlo, ya que se encuentra siempre activo.

Expresiones de FORM

Las expresiones relacionadas con el objeto FORM son las siguientes:

CURRENT {COUNT | TOTAL | AVG | MAX | MIN} OF variable_form

Esta expresión devuelve los valores agregados del CTSQL sobre las filas consultadas en un FORM. «variable_form» debe ser una variable definida en el objeto FORM, la cual se corresponde con una columna de la tabla que se mantiene en él.

Expresiones de CURSOR

Las expresiones relacionadas con el objeto CURSOR son las siguientes:

[GROUP] {COUNT | PERCENT | {TOTAL | AVG | MAX | MIN} OF variable} [WHERE condición]

Esta expresión devuelve los valores agregados de un objeto CURSOR. La cláusula «GROUP» sólo puede utilizarse en las instrucciones no procedurales «AFTER GROUP» de la declaración del CURSOR. Sin embargo, el resto de expresiones sólo se pueden utilizar en la instrucción no procedural «ON LAST ROW» de la declaración del CURSOR. «variable» corresponde a una columna leída en el CURSOR. «condición» es una condición booleana que delimita a su vez el ámbito de aplicación del valor agregado.

Operadores CTL

Los operadores que pueden emplearse en expresiones de tipo CTL se clasifican en los siguientes tipos:

- Operadores aritméticos.
- Operadores de formato.
- Operadores unarios.
- Operadores de comparación.
- Operadores lógicos.

A continuación se indican los operadores incluidos en cada una de estas clasificaciones:

1.— Operadores aritméticos

+	Suma.
-	Resta.
*	Multiplicación.
/	División.
%	Módulo.
**	Exponenciación.

2.— Operadores de formato

&&&	Concatenación de dos expresiones respetando la longitud de cada una de ellas en caso de que una fuese nula («NULL»).														
&&	Concatenación de dos expresiones respetando los blancos de la derecha de cada expresión. Sin embargo, si alguna de las expresiones fuese nula («NULL») no respetaría su longitud. En conclusión, es igual que «&&&», pero sin respetar la longitud en caso de valores nulos.														
&	Concatenación de dos expresiones eliminando los blancos de la derecha de cada expresión. Asimismo, en caso de que alguna de las expresiones fuese nula no respetaría su longitud. En conclusión, es igual que «&&», pero realizando «CLIPPED» de ambas expresiones.														
, (coma)	Separador de parámetros en ciertas instrucciones CTL (PUT, RUN, CTL, etc.).														
CLIPPED	Palabra reservada que, pospuesta a una variable de tipo CHAR, elimina los blancos de la derecha.														
LEFT	Palabra reservada que, pospuesta a una variable, ajusta su valor a la izquierda. Este ajuste se utiliza por defecto en el tipo de dato CHAR.														
RIGHT	Palabra reservada que, pospuesta a una variable, ajusta su valor a la derecha. Este ajuste es el que utilizan por defecto los tipos de datos numéricos (SMALLINT, INTEGER, DECIMAL y MONEY).														
USING	Palabra reservada utilizada para formatear expresiones numéricas, independientemente del tipo de dato asignado. Los metacaracteres que pueden utilizarse con esta palabra reservada son idénticos a los manejados por el atributo FORMAT. Dependiendo del tipo de dato a formatear, estos metacaracteres son los siguientes: a) Numéricos (SMALLINT, INTEGER, DECIMAL y MONEY). Los caracteres a emplear son los siguientes: <table> <tr> <td>#</td><td>Representa cada dígito. No cambia los espacios en blanco por ningún otro carácter.</td></tr> <tr> <td>. (punto)</td><td>El punto indica la posición de la coma decimal. Aparecerá una coma o un punto, dependiendo de la definición de la variable de entorno DBMONEY.</td></tr> <tr> <td>, (coma)</td><td>Permite indicar la separación de miles, millones, etc. Se imprimirá una coma si DBMONEY tiene como valor un punto y un punto si aquél es una coma (consulte el capítulo sobre «Variables de Entorno» en el Manual del Administrador).</td></tr> <tr> <td>*</td><td>Rellena los espacios en blanco con asteriscos.</td></tr> <tr> <td>&</td><td>Rellena los espacios en blanco con ceros.</td></tr> <tr> <td>-</td><td>Imprime un signo menos cuando la expresión sobre la que se aplica es menor que cero. Cuando se especifican varios signos «-», sólo aparecerá uno lo más a la derecha posible.</td></tr> <tr> <td>+</td><td>Imprime el signo «+» o «-» en dependencia de que la expresión sobre la que se aplica sea mayor o igual a cero («+») o menor que cero («-»).</td></tr> </table>	#	Representa cada dígito. No cambia los espacios en blanco por ningún otro carácter.	. (punto)	El punto indica la posición de la coma decimal. Aparecerá una coma o un punto, dependiendo de la definición de la variable de entorno DBMONEY.	, (coma)	Permite indicar la separación de miles, millones, etc. Se imprimirá una coma si DBMONEY tiene como valor un punto y un punto si aquél es una coma (consulte el capítulo sobre «Variables de Entorno» en el Manual del Administrador).	*	Rellena los espacios en blanco con asteriscos.	&	Rellena los espacios en blanco con ceros.	-	Imprime un signo menos cuando la expresión sobre la que se aplica es menor que cero. Cuando se especifican varios signos «-», sólo aparecerá uno lo más a la derecha posible.	+	Imprime el signo «+» o «-» en dependencia de que la expresión sobre la que se aplica sea mayor o igual a cero («+») o menor que cero («-»).
#	Representa cada dígito. No cambia los espacios en blanco por ningún otro carácter.														
. (punto)	El punto indica la posición de la coma decimal. Aparecerá una coma o un punto, dependiendo de la definición de la variable de entorno DBMONEY.														
, (coma)	Permite indicar la separación de miles, millones, etc. Se imprimirá una coma si DBMONEY tiene como valor un punto y un punto si aquél es una coma (consulte el capítulo sobre «Variables de Entorno» en el Manual del Administrador).														
*	Rellena los espacios en blanco con asteriscos.														
&	Rellena los espacios en blanco con ceros.														
-	Imprime un signo menos cuando la expresión sobre la que se aplica es menor que cero. Cuando se especifican varios signos «-», sólo aparecerá uno lo más a la derecha posible.														
+	Imprime el signo «+» o «-» en dependencia de que la expresión sobre la que se aplica sea mayor o igual a cero («+») o menor que cero («-»).														

(	Imprime un signo «abrir paréntesis» antes de un número negativo. Cuando se especifican varios signos «(», sólo aparecerá uno lo más a la derecha posible.
)	Imprime este signo al final de un formato en el que se ha especificado «)».
\$	Literal que se imprime como un signo de moneda. Cuando se indican varios en la plantilla, aparecerá uno solo lo más a la derecha posible. Si se ha definido la parte «front» de la variable de entorno DBMONEY, será esto último lo que se incluirá en el resultado y no el signo «\$». (Ver «Variables de Entorno» en el Manual del Administrador).
%	Este literal se imprimirá como tal en la posición donde se especifique. En caso de emplear más de un «%», aparecerán en las posiciones asignadas. Estos signos no tienen ningún efecto sobre la expresión a evaluar.

b) Fechas (DATE). En este caso los caracteres que se evalúan con un significado para la plantilla de formato son «d», «m» e «y», en las combinaciones que se indican a continuación, representándose cualquier otro carácter como tal:

dd	Dos dígitos para el día del mes (01 a 31).
ddd	Tres letras significativas del nombre del día de la semana (Lun a Dom).
mm	Dos dígitos para el mes (01 a 12).
mmm	Tres letras significativas del nombre del mes (Ene a Dic).
yy	Dos dígitos para el año desde 1900 (00 a 99).
yyyy	Cuatro dígitos para el año (0000 a 9999).

c) Horas (TIME). En este caso los caracteres que se evalúan con un significado para la plantilla de formato son «h», «m» y «s», en las combinaciones que se indican a continuación:

hh:mm:ss	Dos dígitos para la hora, dos para los minutos y dos para los segundos.
hh:mm	Dos dígitos para la hora y dos para los minutos.
hh:	Dos dígitos para la hora.

3.— Operadores unarios

- (guión) Inversión de signo. Las cantidades positivas las convierte en negativas y viceversa.

4.— Operadores de comparación

Estos operadores se utilizan para comparar dos expresiones, obteniendo como resultado TRUE o FALSE.

expresion operador_relación expresión

Donde:

operador_relación Puede ser uno de los siguientes:

=	Igual.
!= o <>	Distinto.

>	Mayor que.
>=	Mayor o igual que.
<	Menor que.
<=	Menor o igual que.

[NOT] expresión **BETWEEN** expr1 **AND** expr2

Devuelve verdadero o falso en caso de que el resultado de una «expresión» esté o no comprendido en el rango dado por otras dos («expr1» y «expr2», ambas inclusive). La partícula «NOT» invierte el resultado.

[NOT] expresión **IN** (lista_valores)

Devuelve verdadero o falso en caso de que el resultado de una «expresión» sea igual o no a uno de los valores incluidos en «lista_valores». En caso de que la «expresión» sea de los tipos de datos CHAR, TIME o DATE, cada uno de los valores de la «lista_valores» ha de ir entre comillas. La partícula «NOT» invierte el resultado.

Donde:

lista_valores Lista de valores separados por comas.

[NOT] expresión **LIKE** "literal"

Devuelve verdadero o falso en caso de que el resultado de una «expresión» se ajuste o no al patrón definido en «literal». La partícula «NOT» invierte el resultado.

Donde:

«literal» Expresión alfanumérica entrecomillada que puede emplear dos metacaracteres que tienen un significado especial, así como cualquier otro carácter de la tabla de caracteres ASCII:

%	Indica cero o más caracteres en la posición donde aparezca.
_(subrayado)	Indica cualquier carácter (uno solo) en la posición indicada.

[NOT] expresión **MATCHES** "literal"

Devuelve verdadero o falso en caso de que el resultado de una «expresión» se ajuste o no al patrón definido en «literal». Este operador de comparación es similar al «LIKE», pero con más metacaracteres. La partícula «NOT» invierte el resultado.

Donde:

«literal» Expresión alfanumérica entrecomillada que puede contener varios metacaracteres que tienen un significado especial:

*	Indica cero o más caracteres en la posición indicada.
?	Indica cualquier carácter (uno solo) en la posición donde aparezca.
[lista]	Indica que cualquiera de los caracteres incluidos en la «lista» puede aparecer en esa posición. Se puede indicar un rango de caracteres separando el inicial y el final por medio de un guión. Si el primer carácter de «lista» es el «^», indica que los caracteres o rangos no pueden aparecer en esa posición.
\ (backslash)	Indica que el carácter al que precede no debe considerarse como especial (metacarácter), sino como un carácter más de-

ntro del «literal». Siempre precederá a cualquiera de los metacaracteres indicados anteriormente para eliminar su acción como tal.

expresión **IS [NOT] NULL**

Devuelve verdadero o falso en caso de que el resultado de una «expresión» sea o no un valor nulo («NULL»). La partícula «NOT» invierte el resultado.

5.— Operadores lógicos

Todas las condiciones indicadas anteriormente pueden enlazarse mediante los siguientes operadores lógicos:

AND Devolverá TRUE si ambas condiciones evalúan a TRUE.

condición **AND** condición

OR Devolverá TRUE si al menos una de las condiciones evalúa a TRUE.

condición **OR** condición

NOTAS:

- 1.— Los paréntesis agrupan operandos y operaciones alterando la precedencia de evaluación de los operadores.
- 2.— Si una expresión devuelve «NULL» no hará cierta ninguna condición, excepto la condición «expresión IS NULL».

Reglas de precedencia de operadores

En el apartado anterior se ha mencionado el uso del paréntesis para agrupar expresiones y condiciones, variando así el orden de evaluación de éstas y los operadores que las relacionan. Este orden alterado por los paréntesis se denomina explícito. Cuando no se utilizan los paréntesis, el orden de evaluación se denomina implícito. La siguiente tabla presenta los operadores CTL por orden descendente de prioridad en la evaluación de expresiones y condiciones. Los operadores en la misma línea tienen la misma precedencia:

()	Paréntesis.
**	Aritmético.
*, /, %	Aritméticos.
+, -	Aritméticos
>, >=, <, <=, LIKE, MATCHES	Comparación.
=, !=, <>, IS NULL	Comparación (igualdad).
AND	Y lógico.
NOT	NO lógico.
OR	O lógico.
=	Asignación. Separa listas de argumentos (en funciones y ciertas instrucciones del CTL).

Variables internas CTL

CTL dispone de unas variables internas cuyo mantenimiento es automático. Estas variables son las siguientes:

today	Devuelve la fecha del sistema.				
now	Variable interna que devuelve la hora del sistema.				
ambiguous	Esta variable admite dos valores, TRUE y FALSE, dependiendo de que una instrucción SELECT devuelva como resultado más de una fila o una sola, respectivamente.				
cancelled	Esta variable contendrá el valor TRUE si se cancela la entrada de las siguientes instrucciones: INPUT FRAME, FORM (ADD, ADD ONE, MODIFY, QUERY), PROMPT FOR y WINDOW. En caso de no cancelar la entrada de valores en dichas instrucciones, la variable toma el valor FALSE. Esto significa que sólo tendrá valor al ejecutar alguna de las instrucciones anteriores o las utilizadas por dichos objetos. Por lo tanto, su utilización sólo tendrá sentido después de cada una de las instrucciones indicadas anteriormente.				
curfield	Nombre de la variable que está activa en un objeto FORM o FRAME. Es decir, nombre de la variable que se está editando.				
currdb	Nombre de la base de datos en curso.				
currec	Número de orden que ocupa una fila dentro de la lista de filas que cumplen las condiciones de una consulta (QUERY). Dicha lista de filas se considera la «lista en curso» del FORM.				
curtable	Nombre de la tabla en curso dentro del objeto FORM.				
errno	Número de error que se ha producido al ejecutar una instrucción CTL.				
found	Esta variable admite dos valores, TRUE y FALSE. Estos valores se asignan dependiendo de si se ha procesado una única fila o ninguna, respectivamente.				
lastkey	Esta variable contiene el código de la última tecla pulsada.				
linenum	Número de línea en la que se encuentra el cursor de pantalla sobre la lista de filas que cumplen las condiciones impuestas por el operador (QUERY) dentro de un objeto FORM declarado como «LINES n», siendo «n» el número máximo de líneas de esta tabla que se mostrarán simultáneamente en pantalla, así como el valor máximo valor admitido por LINENUM.				
locked	Esta variable admite el valor TRUE si la fila en curso de un CURSOR está bloqueada por otro usuario. En caso contrario su valor será FALSE. Dicho CURSOR tiene que declararse con la cláusula «NOWAIT».				
newvalue	Admite TRUE y FALSE, dependiendo de si se ha modificado o no el contenido del último campo editado en un objeto FORM o FRAME.				
numrecs	Número de filas que cumplen las condiciones impuestas por el operador en la operación de consulta dentro del objeto FORM.				
operation	Esta variable contiene el nombre de la última operación que se está realizando en un objeto FORM o FRAME. Los valores que admite son los siguientes: <table data-bbox="683 1977 1082 2042"> <tr> <td>«add»</td><td>Altas.</td></tr> <tr> <td>«update»</td><td>Modificaciones.</td></tr> </table>	«add»	Altas.	«update»	Modificaciones.
«add»	Altas.				
«update»	Modificaciones.				

	«query»	Consultas.
	«delete»	Bajas.
status	Código devuelto por un programa ejecutado con la instrucción RUN. Asimismo, recoge el valor devuelto por un programa que finaliza con la instrucción RETURN.	
username	Variable que evalúa el nombre del usuario con el que se accede a la base de datos. Este nombre de usuario equivale al «LOGNAME» de UNIX/Linux.	
sqlrows	Esta variable interna contiene el número de filas procesadas en la última operación de INSERT, DELETE, UPDATE, SELECT o LOAD.	

El ámbito de las variables internas CTL es por cada programa.

Funciones internas CTL

CTL dispone de diversas funciones internas que se clasifican en los siguientes grupos:

- Funciones de fecha.
- Funciones de hora.
- Funciones decimales.
- Funciones de caracteres.
- Funciones de mensajes.
- Funciones de teclado.
- Funciones de sistema.
- Funciones aritméticas.
- Funciones de manejo de directorios.
- Funciones de manejo de CURSOR.
- Funciones de manejo de «QUERIES» en un FORM.
- Funciones de manejo de cursores SQL
- Funciones de evaluación.
- Funciones de manejo de ficheros.
- Funciones trigonométricas y de conversión.
- Funciones financieras.
- Funciones de control de páginas de FORM/FRAME.
- Funciones de asignación dinámica de variables.
- Funciones diversas.
- Funciones específicas de la versión para Windows.

En los siguientes apartados se proporciona una breve descripción de cada una de las funciones disponibles en los apartados anteriores.

Funciones de fecha

date (expresión)	Devuelve el valor de «expresión» convertido a tipo DATE.
day (expresión)	Devuelve el día de la fecha que resulte de la conversión de «expresión».
month (expresión)	Devuelve el mes de la fecha que resulte de la conversión de «expresión».
year (expresión)	Devuelve el año de la fecha que resulte de la conversión de «expresión».
weekday (expresión)	Devuelve el día de la semana de la fecha que resulte de la conversión de «expresión» («0=Domingo, 1=Lunes», etc.).

mdy(mes, día, año) Devuelve un valor de tipo DATE. «mes», «día» y «año» son expresiones que representan los componentes de la fecha.

Funciones de hora

time(expresión) Devuelve el valor de «expresión» convertido a tipo TIME.

hour(expresión) Devuelve las horas de la hora que resulten de la conversión de «expresión».

minute(expresión) Devuelve los minutos de la hora que resulten de la conversión de «expresión».

second(expresión) Devuelve los segundos de la hora que resulten de la conversión de «expresión».

hms(hora, minuto, segundo) Devuelve un valor de tipo TIME. «hora», «minuto» y «segundo» son expresiones numéricas que representan los componentes de la hora.

mt(expresión) Devuelve «AM» o «PM», dependiendo del valor de la hora que devuelva «expresión» (Meridian Time).

tomt(expresión) Devuelve el valor de tipo TIME correspondiente a la hora del meridiano.

Funciones decimales

round(expresión, precisión) Redondea el valor devuelto por «expresión» con la «precisión» indicada.

truncate(expresión, precisión) Redondea por defecto el valor devuelto por «expresión» con la «precisión» indicada truncando el resultado.

Funciones de caracteres

length(expresión) Devuelve la longitud de una expresión de tipo carácter.

varlength(expresión) Devuelve el espacio en bytes definido para la variable de tipo CHAR indicada por «expresión».

substring(expresión, ini, longi) Devuelve la cadena de caracteres de una variable tipo CHAR («expresión»), indicada por «ini» (comienzo) y «longi» (longitud).

upcase(expresión) Devuelve el valor de la variable CHAR («expresión») convertido a mayúsculas.

lowcase(expresión) Devuelve el valor de «expresión» convertido a minúsculas.

ascii(expresión) Devuelve en base 10 el valor ASCII de un carácter (su posición en la tabla ASCII). «expresión» sólo podrá evaluar un carácter.

character(expresión) Devuelve el carácter correspondiente a la posición en la tabla ASCII que evalúe «expresión».

inwords(expresión [,género]) Devuelve una cadena que expresa en letra el valor numérico que evalúe «expresión», incluso en femenino («f») o masculino («m») en caso de indicarse en «género». Esta cadena es dependiente de la variable de entorno DBLANG, en cuyo directorio se encuentra el programa «inwords».

strrtrim(expresión) Devuelve «expresión» eliminando blancos por la derecha.

strltrim(expresión) Devuelve «expresión» eliminando blancos por la izquierda.

strtrim(expresión) Realiza un «strrtrim + strltrim» y reduce cada grupo de blancos entre palabras a uno solo.

strlocate(expr1,expr2[, número]) Busca un «literal» en otro.

Donde:

expr1	Expresión alfanumérica donde buscar.
expr2	Expresión alfanumérica a buscar.
número	Valor numérico opcional (por defecto 1). número < 0 Busca desde el final. número = -1 Indica el último. número = -2 Indica el penúltimo.

strcount(expr1, expr2) Devuelve el número de veces que «expr2» se encuentra en «expr1».

strreplace(expr1, expr2, expr3[, número]) Sustituye un valor por otro dentro de un «string» (literal), siendo «número» el número de sustituciones a efectuar (por defecto «todas»), y devuelve el resultado de la sustitución siempre que la longitud de «expr2» sea igual a la de «expr3». Si no es así, devuelve un máximo de 512 caracteres. Si «expr3» es nulo indica que se desea eliminar «expr2».

Donde:

expr1	Expresión alfanumérica donde se realizará la sustitución de valores.
expr2	Expresión alfanumérica a buscar dentro de «expr1».
expr3	Expresión alfanumérica que representa el valor que sustituirá a «expr2».

strgetword(expr1, expr2 [, número [, expr3]]) Devuelve la enésima palabra en un «string».

Donde:

expr1	Expresión alfanumérica a examinar.
expr2	Expresión alfanumérica que representa el carácter a utilizar como separador.
número	Número de palabra que se desea. Si no se especifica, se asume la primera por defecto.
expr3	Si su valor es FALSE indicará que varios separadores «expr2» consecutivos equivalen a un solo separador, TRUE en caso contrario. Su valor por defecto es FALSE si el separador es un espacio en blanco (« »), TRUE si no lo es. Por tanto, este parámetro sólo admitirá los valores TRUE y FALSE.

strnumwords(expr1, expr2 [, expr3]) Cuenta las palabras en un «string».

Donde:

expr1	Expresión alfanumérica a examinar.
expr2	Expresión alfanumérica que representa el carácter a utilizar como separador.
expr3	Si su valor es FALSE indicará que varios separadores «expr2» consecutivos equivalen a un solo separador, TRUE en caso contrario. Su valor por defecto es FALSE si el separador es un espacio en blanco (« »), TRUE si no lo es. Por tanto, este parámetro sólo admitirá los valores TRUE y FALSE.

strrepeat(expresión, número) Repite «expresión» tantas veces como indique «número». La longitud máxima del resultado es de 512 caracteres.

Funciones de mensajes

msgtext(expresión [, expresión1]) Devuelve el texto del mensaje del fichero de ayuda cuyo número es «expresión». Opcionalmente, esta función admite un segundo parámetro, cuyo valor será sustituido por la secuencia «%s» incluida dentro del texto del mensaje del fichero de ayuda.

sysmsg(expresión) Devuelve el texto del mensaje de sistema cuyo número sea el devuelto por «expresión».

Funciones de teclado

actionlabel(tecla | código) Devuelve una cadena de caracteres que representa la función asociada a la «tecla» o a su «código». Si la tecla pulsada no tiene una función asociada devolverá un nulo.

keylabel(función | código) Devuelve una cadena de caracteres que representa el nombre de la tecla asociada a la función o código de tecla devueltos respectivamente por «función» o «código».

keycode(tecla) Devuelve el código de la tecla cuyo nombre es devuelto por «tecla».

dokey(expresión) Simula la pulsación de una tecla, cargándola en el «buffer» del teclado. En caso de que «expresión» sea numérica:

>= 2000 «tecla» correspondiente a una acción.

1 a 256 Carácter ASCII a introducir.

En caso de que «expresión» sea CHAR:

— Nombre de una acción (<fcancel>, <fquit>, etc.).

— Si no es ninguna acción, se introduce carácter a carácter todo el «literal».

keypressed() Devuelve TRUE si hay un carácter disponible en el «buffer» del teclado, FALSE en caso contrario. Un vez que devuelve TRUE, la tecla se puede «leer» con READ KEY o PAUSE, o bien será tratada por cualquier elemento del CTL.

Funciones de sistema

getenv(expresión) Devuelve el valor de la variable de entorno cuyo nombre devuelva la «expresión».

putenv(expresión, valor [, modo]) Añade o modifica el «valor» de una variable de entorno indicada en «expresión». «modo» es un parámetro que indica si esa variable «expresión» ha de pasarse al CTSQL o no. Los valores admitidos son los siguientes:

— Si «modo = 0», la variable sólo será considerada por el CTL (y por CTSQL si es una variable de éste). Éste es el valor por defecto para dicho parámetro.

— Si «modo = 1», la variable será considerada sólo por CTSQL.

— Si «modo = 2», la variable será considerada por el CTL y por el CTSQL.

procid()	Devuelve el número del proceso que se está ejecutando en UNIX/Linux. En las versiones de y Windows, este número de proceso interno es simulado, devolviendo un número aleatorio y único en máquina.
basename(expresión)	Devuelve la parte correspondiente al último elemento del «path» que evalúe «expresión».
dirname(expresión)	Devuelve «expresión» eliminando el último elemento del «path» que aquél evalúe.

Funciones aritméticas

abs(expresión)	Devuelve un número equivalente al valor absoluto de «expresión».
ceil(expresión)	Devuelve el menor número entero mayor o igual que el valor de «expresión».
exp(expresión)	Devuelve el número real equivalente a elevar el número «e» a la potencia equivalente a «expresión».
floor(expresión)	Devuelve el mayor número entero menor o igual que el valor de «expresión».
ln(expresión)	Devuelve el número real equivalente al logaritmo natural (neperiano) del valor de «expresión».
log(expresión)	Devuelve el número real equivalente al logaritmo decimal (en base 10) del valor de «expresión».
mod(expresión1, expresión2)	Devuelve el número real equivalente al resto de dividir el valor de «expresión1» entre el valor de «expresión2».
pow(expresión1, expresión2)	Devuelve el número real equivalente a tomar como base el valor de «expresión1» y elevarlo a la potencia equivalente al valor de «expresión2».
sqrt(expresión)	Devuelve un número real equivalente a la raíz cuadrada del valor de «expresión».

Funciones de manejo de directorios

getcwd()	Devuelve el directorio en curso.
chdir(expresión)	Cambia el directorio en curso, siendo «expresión» el nombre de aquel que se desea establecer como nuevo directorio en curso.
opendir(expresión)	Abre el directorio cuyo «path» se indica en «expresión». Devuelve TRUE si el directorio existe y lo puede abrir, FALSE en caso contrario. No podrá existir más de un directorio abierto de forma simultánea. En el caso de ejecutar dos «opendir» seguidas, la segunda ejecutará automáticamente un «closedir» del primero.
readdir()	Lee fichero a fichero el contenido de un directorio previamente abierto con la función «opendir()». Devuelve el siguiente fichero o subdirectorio del directorio abierto, o NULL en el caso de estar vacío o después del último.
closedir()	Cierra el directorio abierto previamente por la función «opendir()».

Función de manejo del cursor de la pantalla

cursorpos(expresión, línea, columna)	Permite encender o apagar el cursor (si lo permite el terminal), así como situarlo en una posición determinada. La «expresión» indica
---	---

«TRUE» o «FALSE» si el cursor debe aparecer o desaparecer, respectivamente. Los parámetros «línea» y «columna» indican las coordenadas donde se situará el cursor.

Funciones de manejo de «Queries» en un FORM

fquery() Equivale a la instrucción QUERY, pero sólo devuelve las condiciones SQL que el usuario haya seleccionado.

fquery(expresión) Si «expresión» evalúa a TRUE, devolverá la instrucción SELECT del FORM completa, incluyendo su cláusula «ORDER BY», en caso de que la hubiera. De lo contrario (FALSE), su funcionamiento es idéntico a «fquery()».

fgetquery() Limpia la lista en curso del FORM.

fgetquery([condiciones [, orden]]) Equivale a la instrucción «GET QUERY WHERE...». Devuelve el número de filas seleccionadas en la consulta.

Donde:

condiciones Expresión alfanumérica que indica la(s) condición(es) «WHERE» y/u «ORDER BY» a usar como «QUERY».

orden Expresión alfanumérica que indica la lista de columnas, separadas por comas, que se añadirán a la SELECT que ejecuta el «QUERY». Si se usa la cláusula «ORDER BY», se deben indicar las columnas de ordenación, ya que en caso contrario el SQL devolverá el error: «La lista del SELECT debe incluir la columna (...) del ORDER BY».

fgetquery(expresión_select) Admite como primer parámetro una instrucción SELECT completa, cuya primera columna debe ser el «rowid».

Donde:

expresión_select Deberá evaluar una expresión SELECT completa. Por ejemplo:

1. fgetquery("select rowid, descripcion from provincias " && "where provincia >10 order by descripcion desc")
2. fgetquery(fquery(true))

addtoclist(rowid) Añade la fila con el «rowid» indicado a la lista de la tabla en curso del FORM.

endquery() Muestra el número de filas de la lista de la tabla en curso del FORM y se posiciona en la primera.

gotoclist(expresión) Activa la fila indicada por «expresión» dentro de la lista en curso de la tabla activa. Los valores que pueden emplearse en «expresión» son los siguientes:

expresión = 0 Se posiciona en la fila en curso.

expresión = n Se posiciona en la «n-ésima» fila de la lista en curso.

expresión = -1 Se posiciona en la última fila de la lista en curso.

expresión = -2 Se posiciona en la penúltima fila de la lista en curso.

...

El valor devuelto por la función es el número de registro de la lista en curso donde se posiciona («currec»).

Funciones de manejo de «Cursores» SQL

sqlreset ([flag])	Cierra «cursores». Si «flag = 0», cierra todos los «cursores» del módulo, tanto del FORM y Windows como los de usuario (DECLARE CURSOR). Éste es el valor por defecto. Si «flag = 1», sólo se cierran los «cursores» del FORM. Si «flag = 2», se cierran todos los «cursores» abiertos en el módulo, excepto los del FORM.
sqlselect (select)	Dada una instrucción SELECT, abre un CURSOR para ella y devuelve un número (de «0 a n») que lo identifica, o NULL si se produce algún error.
sqlfetch (número)	Devuelve la siguiente fila, o NULL si se ha llegado al final del CURSOR «número» asociado a una SELECT en la función «sqlselect()». La fila es devuelta en formato UNLOAD.
sqlclose (número)	Cierra el CURSOR «número» asociado a la instrucción SELECT en la función «sqlselect()».

Funciones de evaluación

evalfun (nombre_función [, param1, ...paramn])	Ejecuta la función de nombre «nombre_función» y devuelve su resultado, si lo hubiere.
Donde:	
nombre_función	Expresión alfanumérica que indica el nombre de una función de usuario o interna.
evalcond (expr1, oper, expr2, exprtrue, exprfalse)	Compara «expr1» con «expr2» según el operador «oper», pudiendo devolver los siguientes valores:
exprtrue	Si el resultado es cierto.
exprfalse	Si el resultado es falso.
NULL	Si hay error de parámetros.
evalnull (expr1, exprtrue, exprfalse)	Evalúa si «expr1» es o no nulo («NULL»). Devuelve los siguientes valores:
exprtrue	Si «expr1» es nulo.
exprfalse	Si «expr1» no es nulo.
evalvar (variable)	Evalúa el nombre de una «variable» de programa (cualquiera incluida en la sección DEFINE del módulo, incluidas las de FORM y FRAME).
evalarray (array, número)	Igual que «evalvar», pero aplicada al elemento «número» del ARRAY indicado en «array».

Funciones de manejo de ficheros

nocomments (expr1, expr2)	Copia el contenido del fichero «expr1» al fichero «expr2», eliminando las líneas de comentarios (estas líneas se identifican porque comienzan con el símbolo «#»).
Donde:	
expr1	Expresión alfanumérica que indica el nombre del fichero origen.

	expr2	Expresión alfanumérica que indica el nombre del fichero destino.
	Esta función puede devolver los siguientes valores:	
	NULL	Error de parámetros o imposibilidad de abrir un fichero.
	TRUE/FALSE	Si ha copiado o no algo al fichero de salida «expr2».
testfile(expr1, expr2)	Comprueba si un fichero «expr1» es de un determinado tipo o bien si tiene determinados permisos.	
	Donde:	
	expr1	Expresión alfanumérica que indica el nombre del fichero a comprobar.
	expr2	Expresión alfabética que contiene los permisos que se desean comprobar. Cada carácter indica un tipo de permiso:
	r	Lectura.
	w	Escritura.
	x	Ejecución (en ficheros) o búsqueda (en directorios).
	f	Si es un fichero.
	d	Si es un directorio.
	Se puede especificar cualquier combinación de los anteriores. Los valores que puede devolver son los siguientes:	
	TRUE	Si cumple todos los permisos.
	FALSE	Si no cumple alguno de ellos o no existe el fichero.
	NULL	Error de parámetros.
filesize(expresión)	Devuelve el tamaño, en bytes, del fichero indicado en «expresión», o NULL si aquél no existe o hay error de parámetros.	

Funciones trigonométricas y de conversión

acos(expresión)	Devuelve un número equivalente al arco, expresado en radianes, cuyo coseno es equivalente a «expresión».
asin(expresión)	Devuelve un número equivalente al arco, expresado en radianes, cuyo seno es equivalente a «expresión».
atan(expresión)	Devuelve un número equivalente al arco, expresado en radianes y en el intervalo («-pi/2», «+ pi/2»), cuya tangente es equivalente a «expresión».
atan2(expresión1, expresión2)	Devuelve un número equivalente al arco, expresado en radianes y en el intervalo («-pi», «+pi»), cuya tangente es equivalente al cociente «expresión1/expresión2». El signo de ambas expresiones se utiliza para determinar el cuadrante correspondiente al valor devuelto por la función.
cos(expresión)	Devuelve un número equivalente al coseno del arco dado en radianes por «expresión».
deg(expresión)	Devuelve un número equivalente a la medida en grados sexagesimales (y fracción decimal de grado, si procede) del arco dado en radianes por «expresión».
pi()	Devuelve el número constante «3.141592653589793».

rad (expresión)	Devuelve un número equivalente a la medida en radianes del arco dado en grados sexagesimales (y fracción decimal de grado, si procede) por «expresión».
sin (expresión)	Devuelve un número equivalente al seno del arco dado en radianes por «expresión».
tan (expresión)	Devuelve un número equivalente a la tangente del arco dado en radianes por «expresión».

Funciones financieras

Las funciones que siguen permiten resolver cualquier variable, conocidas las restantes, de la ecuación financiera:

$$0 = PV + (1+i)^n \text{PMT} \left(\frac{1-(1+i)^{-n}}{i} \right) + FV (1+i)^{-n}$$

Donde, supuestos unos ingresos (signo positivo) o pagos (signo negativo) iguales realizados periódicamente, las variables representan:

PV	Valor actual neto.
PMT	Valor de los ingresos o pagos periódicos.
FV	Valor futuro, o saldo, tras el último período.
n	Número de períodos.
S	Modo de pago:
0	Indica que PMT se realiza al inicio de cada período.
1	Indica que PMT se realiza al final de cada período.
i	Tasa de interés en el período (como fracción decimal).

Las funciones disponibles son:

fv (i, n, PMT, PV, S)	Devuelve un número equivalente al valor futuro (variable FV), que satisface la ecuación financiera antes citada, cuando se sustituyen en ella las variables «i», «n», «PMT», «PV» y «S» por los valores de las expresiones numéricas correspondientes («i», «n», «PMT», «PV», «S») en la llamada a la función. El valor devuelto por FV puede ser positivo (correspondiente a un ingreso) o negativo (a un pago).
npv (i, expression [, expression....])	Calcula la cantidad de dinero requerida ahora para producir un flujo de caja específico en un futuro (dado un interés).
pmt (i, n, PV, FV, S)	Devuelve un número equivalente al importe periódico (variable PMT), que satisface la ecuación financiera antes citada, cuando se sustituyen en ella las variables «i», «n», «PV», «FV» y «S» por los valores de las expresiones numéricas correspondientes («i», «n», «PV», «FV», «S») en la llamada a la función. El valor devuelto por PMT puede ser positivo (correspondiente a un ingreso) o negativo (a un pago).
pv (i, n, PMT, FV, S)	Devuelve un número equivalente al valor presente (variable PV), que satisface la ecuación financiera antes citada, cuando se sustituyen en ella las variables «i», «n», «PMT», «FV» y «S» por los valores de las expresiones numéricas correspondientes («i», «n», «PMT», «FV», «S») en la llamada a la función. El valor

devuelto por PV puede ser positivo (correspondiente a un ingreso) o negativo (a un pago).

irr(n, PMT, PV, FV, S) Devuelve un número equivalente al interés, expresado como fracción decimal (variable i), que satisface la ecuación financiera antes citada, cuando se sustituyen en ella las variables «n», «PV», «PMT», «FV» y «S» por los valores de las expresiones numéricas correspondientes («i», «n», «PMT», «FV», «S») en la llamada a la función. El valor devuelto por IRR (denominado «tasa interna de retorno», o «internal return rate») sólo tiene interpretación económica válida si es mayor o igual a cero.

Función de control de página en un FORM o FRAME

gotoscreen(expresión) Permite ir a la página («SCREEN») indicada en «expresión» de la lista en curso de un FORM o un FRAME. Los valores posibles de «expresión» son:

expresión = 0 Se posiciona en la página en curso.
expresión = n Se posiciona en la página «n» de la lista en curso.
expresión = -1 Se posiciona en la última página de la lista en curso.
expresión = -2 Se posiciona en la penúltima página de la lista en curso.

...

El valor devuelto por la función es el número de página donde se posiciona.

Funciones de asignación dinámica de variables

letvar(variable, valor) Asigna el valor contenido en «valor» a la variable de programación indicada en «variable».

letarray(array, número, valor) Igual que «letvar» pero para el elemento «número» del ARRAY indicado en «array».

Ejemplo:

```
define
  variable var1 integer
  array arr1[20] char(10)
end define

main begin
  call letvar("var1", 5)
  pause var1
  ...
  call letarray ("arr1", 15, "hola")
  pause arr1[15]
  ...
end main
```

Funciones diversas

labelname(etiqueta) Devuelve el nombre de la variable del FORM para la cual se ha definido como atributo: LABEL «etiqueta».

sleep(expresión) Detiene la ejecución del programa tantos segundos como indique «expresión».

yes(expresión [, def]) Devuelve TRUE si se selecciona «Sí» del menú que presenta, mostrando «expresión» como pregunta al usuario. FALSE en caso contrario.

Donde:

expresión	Literal que aparecerá al operador junto las opciones «Sí/No» del menú.
def	Expresión alfabética que admite los valores «Y» y «N» para indicar la opción por defecto del menú («Sí/No», respectivamente).

getctlvalue(expresión) Devuelve un valor dependiendo del que tenga «expresión». Los valores posibles de «expresión» son los siguientes:

«version»	Versión del CTL (por ejemplo, «2.0.05»).
«O.S.»	Sistema o entorno operativo (por ejemplo, «Windows»).
«porting»	Indica variación dentro de un mismo sistema operativo.
«atcol»	Columna en la que arrancó el programa en curso.
«atrow»	Fila en la que arrancó el programa en curso.
«ctlname»	Nombre del CTL en curso.

getipaddress() Devuelve la dirección IP de la máquina cliente en la que se está ejecutando el programa CTL.

licence() Devuelve un número entero correspondiente al número de la Licencia de Uso de MultiBase.

setprinteropt(name) Esta función permite indicar el tipo de impresora que se empleará en los listados para la utilización de los diferentes atributos de impresión, es decir, su función en un programa de MultiBase es la misma que la de la opción PRINTER de la sección Options.

sqldescribe(select[, flag][, expr1]) Dada una instrucción SELECT, devuelve un «literal» con la lista de columnas, sus tipos y, opcionalmente, sus atributos, tal y como están definidos en la base de datos.

Donde:

select	Instrucción SELECT.
flag	Expresión numérica. Si «flag = 0», no incluye atributos (valor por defecto). Si «flag = 1», incluye los atributos.
«expr1»	Expresión alfanumérica que representa el separador de la lista. Por defecto, este separador es la coma.

Sqldisconnect() Desconexiones dinámicas. Mediante esta función se logra la desconexión de la base de datos en uso en el programa CTL, consiguiendo así un mecanismo de conexión/desconexión dinámica en un programa CTL. En el momento que desconectamos de una base de datos se pueden cambiar los valores de las variables relacionadas con las conexiones a nuevas bases de datos. Por ejemplo, cambiar las variables DBHOST, DBPATH, DBUSER, DBPASSWD y DBSERVICE para poder acceder a otras bases de datos residentes en otros ordenadores (cliente-servidor).



Funciones específicas de la versión para Windows

Las funciones que a continuación se detallan sólo tienen efecto en las versiones de MultiBase para Windows.

Funciones diversas

getwinparm(expresión) Dependiendo del valor de «expresión», devolverá las coordenadas (en «pixels») de la posición de la ventana de ejecución del CTL, o el ancho o el alto de la misma. El valor devuelto por «expresión» deberá ser uno de los siguientes:

- x Si se desea conocer la línea de posicionamiento de la ventana CTL.
- y Si se desea conocer la columna de posicionamiento de la ventana CTL.
- w Si se desea conocer el ancho de la ventana CTL.
- h Si se desea conocer el alto de la ventana CTL.

movewin(expy, expx [, expy, expw]) Permite cambiar el tamaño y el posicionamiento de la ventana de ejecución del CTL. Los valores de estas expresiones deben estar en «pixels».

iconizewin() Iconiza la ventana de ejecución del CTL.

maximizewin() Maximiza la ventana de ejecución del CTL.

restorewin() Restaura la ventana a su tamaño y posición anteriores a la maximización o iconización.

setwintitle(expresión) Modifica el título de la ventana de ejecución de CTL. El nuevo título será el devuelto por «expresión».

messagebox(exptit, exptext [, expstyle]) Llamada a la función estándar de mensajes de Windows.

Donde:

- exptit Será el título de la ventana del mensaje.
- exptext Será el texto del mensaje.
- expstyle Expresión que podrá contener hasta tres cadenas separadas por blancos. Estas cadenas serán:
[ICO] [BOT] [DEF]

Donde:

- [ICO] Cualquiera de los siguientes:
 - ICO-I Icono de información de Windows.
 - ICO-STOP Icono de parar del Windows.
 - ICO-? Icono de interrogación del Windows.
- [BOT] Dependiendo de qué botones se deseen en el diálogo podrá ser:
 - OK Sólo boton OK.
 - OKCANCEL Botones OK y CANCEL.
 - RETRYCANCEL Botones RETRY y CANCEL.
 - ABORTRETRYIGNORE Botones ABORT, RETRY y CANCEL.
 - YESNO Botones YES y NO.
 - YESNOCANCEL Botones YES, NO y CANCEL.

[DEF] Indica cuál de los botones será el empleado por defecto. Si no se incluye, el botón por defecto será el primero. Puede ser uno de los valores siguientes:

DEF2 El botón por defecto será el segundo.

DEF3 El botón por defecto será el tercero.

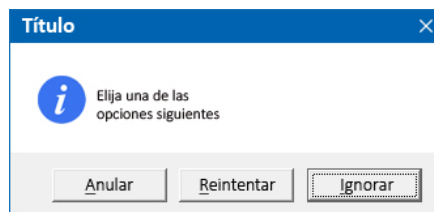
Los valores devueltos por la función dependerán del botón pulsado por el usuario. Los posibles valores devueltos serán:

«OK»	«NO»
«ABORT»	«YES»
«CANCEL»	«RETRY»
«IGNORE»	

Por ejemplo, el programa:

```
main begin
pause messagebox ("Título", "Elija una de las" && character(10) && "opciones siguientes",
"ICO-I ABORTRETRYIGNORE DEF3")
end main
```

producirá la salida que se muestra en la figura:



Resultado en pantalla de la ejecución del ejemplo anterior.

Dependiendo del botón que pulse el usuario aparecerá uno de los siguientes mensajes en pausa: ABORT, RETRY o IGNORE.

- gethinstance()** Devuelve la instancia Windows del CTL que se está ejecutando.
- gethwnd()** Devuelve un entero (INTEGER o DWORD), que se corresponde con el manejador («handle») para Windows de la ventana principal del CTL. Sólo es útil para su uso con DLLs.
- xtpixels([expresión])** Devuelve el tamaño en «pixels» que ocupa el número de columnas CTL indicado por «expresión». Si no se especifica ningún parámetro se asume «1» como valor por defecto.
- ytpixels([expresión [, flag]])** Devuelve el tamaño en «pixels» que ocupa el número de filas CTL indicado por «expresión». Si no se especifica ningún parámetro se asume «1» por defecto. Por su parte, «flag» podrá tener dos valores: TRUE o FALSE. Si se indica TRUE se suma al resultado el tamaño en «pixels» de la ventana de botones que aparece en la parte superior. Por defecto se asume FALSE.
- dllfun(función, param1, ...paramn)** Ejecuta la función «función» con los parámetros indicados («param1 ...paramn») y retorna, en su caso, el valor devuelto por «función».
- ansitooem(expresión) y oemtoansi(expresión)** Convierten el string «expresión» del «set» de caracteres ANSI (Windows) al set OEM () y viceversa. Por ejemplo:
- ```
let expr1 = ansitooem (expr2)
```



### Funciones de manejo de protocolo DDE

Estas funciones permiten al CTL comunicarse como cliente con cualquier aplicación Windows que soporte protocolo DDE.

**DDEinitiate(apli, topic)** Abre dinámicamente un canal de comunicación para intercambio de datos y retorna el número del canal abierto.

Donde:

**apli** Nombre DDE de la aplicación con la que se quiere comenzar la sesión DDE. Dicho nombre depende de la aplicación con la que se desea conectar (por ejemplo: «Excel»).

**topic** Describe algo como el documento o el tipo de comunicación que se desea. Este valor depende de la aplicación con la que se desea conectar. Puede ser nulo.

Devuelve:

**NULL** Error de parámetros.

**0** No se ha establecido comunicación.

**número** Número de canal.

Ejemplo:

```
variable canal smallint
...
run "WINWORD c:\test.doc&"
let canal = DDEinitiate("WINWORD", "c:\text.doc")
```

**DDEexecute(canal, comando)** Envía la orden de ejecución al servidor.

Donde:

**canal** El valor devuelto por «DDEinitiate».

**comando** Texto que especifica el comando.

Devuelve:

**-1** Canal incorrecto.

**0** Permiso denegado.

**1** Ejecución correcta.

**2** El servidor está ocupado.

**3** «TIMEOUT» después de un bucle de espera. El servidor no responde.

**DDEpoke(canal, item, data)** Envía datos al servidor.

Donde:

**canal** El valor devuelto por «DDEinitiate».

**item** Es el texto que identifica lo que se desea mandar. Este valor depende del servidor.

**data** Es el valor que se desea enviar.

Devuelve los mismos valores que «DDEexecute».

**DDEterminate(canal)** Cierra el canal de comunicación DDE. Devuelve TRUE o FALSE si el canal es o no válido respectivamente.



### Funciones CTL asociadas al manejo de impresión

**setpmattr**(atributo, valor [, unidad]) Permite establecer un atributo de impresión.

Donde:

|             |                                                                                              |
|-------------|----------------------------------------------------------------------------------------------|
| atributo    | Puede ser cualquiera de los siguientes:                                                      |
| topmargin   | Tamaño del margen superior.                                                                  |
| botmargin   | Tamaño del margen inferior.                                                                  |
| leftmargin  | Tamaño del margen izquierdo.                                                                 |
| fixprtcolum | Si se desea que la cláusula del CTL «COLUMN» proporcione posicionamiento fijo.               |
| lines       | Número de líneas de la página (sin incluir márgenes).                                        |
| columns     | Número de columnas de la página (sin incluir márgenes).                                      |
| curcol      | Establece columna en curso.                                                                  |
| rightalign  | Establece columna en curso con ajuste a la derecha.                                          |
| graphsize   | Tamaño de la línea utilizada para imprimir semi-gráficos con «fonts» ANSI.                   |
| valor       | El valor que se quiere establecer. En el caso de «fixprtcolum», éste podrá ser TRUE o FALSE. |
| unidad      | Puede ser:                                                                                   |
| 1           | Indica que el valor es en «pixels».                                                          |
| 0           | Indica que el valor es en columnas.                                                          |
|             | Por defecto el valor es 0.                                                                   |
|             | No se usa para «lines» y «columns».                                                          |

Devuelve:

|       |                                   |
|-------|-----------------------------------|
| NULL  | Error de parámetros.              |
| TRUE  | Se ha activado dicho atributo.    |
| FALSE | No se ha activado dicho atributo. |

Los atributos «curcol» y «rightalign» sólo podrán usarse después de abrir el STREAM de impresión con la sentencia: «START OUTPUT STREAM... THROUGH PRINTER». Los restantes se deberán usar con el STREAM cerrado.

**getpmattr**(atributo [, unidad]) Retorna el valor en curso. Los valores de «atributo» y «unidad» son los mismos que los de la función «setpmattr».

El resultado de esta función será el valor actual de dicho parámetro expresado en «pixels» si «unidad» vale «1» (para «columns», «lines», «fixprtcolum» y «graphsize» no tiene sentido usar este parámetro).

Devuelve:

|        |                        |
|--------|------------------------|
| NULL   | Error en parámetros.   |
| Número | Valor antes comentado. |

Aparte de los diferentes «fonts» que pueden utilizarse dependiendo de cada impresora, existen algunos predeterminados en Windows con los siguientes nombres:

|                |   |     |
|----------------|---|-----|
| ANSI_FIXED     | o | A_F |
| ANSI_VAR       | o | A_V |
| DEVICE_DEFAULT | o | D_D |
| OEM_FIXED      | u | O_F |
| SYSTEM_FONT    | o | S_F |

Para facilitar el uso del gestor de impresión, algunos de los valores anteriormente citados se pueden definir en la sección «.CONFIGURATION» del fichero «MB.INI». Estos valores son: PRTSET, DBPRINT, RPRINTER, PRINTMANFONT, FIXPRTCOLUMN y GRAPHSIZE.

**setprtsetup(attribute, value)** El acceso a la modificación de la configuración de las impresoras definidas en el Administrador de Impresión puede realizarse mediante el programa «wprintsetup», al cual puede accederse desde la persiana de Accesorios en la opción «Ajuste de Impresora».

Mediante esta función podremos también realizar la modificación de los siguientes atributos ligados a la definición de una impresora. Los cambios realizados están disponibles para todos los módulos hasta que se abandona el primer módulo CTL llamado.

Donde:

|               |                                                                                                                                                                                             |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| attribute     | Puede ser cualquiera de los siguientes:                                                                                                                                                     |
| «name»        | Nombre de una impresora definidas en el Administrador de Impresión. La impresora asociada al nombre indicado quedará como impresora en curso para las próximas impresiones desde MultiBase. |
| «port»        | Identificador del puerto. El puerto indicado quedará como puerto en curso.                                                                                                                  |
| «orientation» | Tipo de impresión: vertical y horizontal.                                                                                                                                                   |
| «copies»      | Número de copias.                                                                                                                                                                           |

Ejemplo:

```
...
call SetPrtSetup("name","HP LaserJet III")
call SetPrtSetup("port","LPT1:")
call SetPrtSetup("orientation","landscape")
call SetPrtSetup("copies","3")
...
```

**getprtsetup(attribute)** Devuelve el valor asociado al atributo indicado sobre la impresora en curso.

Donde:

|               |                                           |
|---------------|-------------------------------------------|
| attribute     | Puede ser cualquiera de los siguientes:   |
| «name»        | Nombre de la impresora en curso.          |
| «port»        | Identificador del puerto en curso.        |
| «orientation» | Tipo de impresión: vertical y horizontal. |
| «copies»      | Número de copias.                         |

Ejemplo:

```
...
let v_orientation = GetPrtSetup("orientation")
```

**spoolfile(file, text[,prtname,portname])** La utilización del Administrador de Impresión desde Multi-Base no permite enviar secuencias de escape directamente a la impresora. Mediante esta función podremos enviar a impresión ficheros que sí contengan secuencias de escape, no pudiendo en estos casos utilizar características del Administrador de Impresión (por ejemplo fuentes). Esta funcionalidad permite mantener la compatibilidad con los listados que ya utilizaban las secuencias de escape para la impresión.

Donde:

|            |                                                                                               |
|------------|-----------------------------------------------------------------------------------------------|
| «file»     | Nombre del fichero a imprimir.                                                                |
| «text»     | Identificativo que aparece en el Administrador de Impresión.                                  |
| «prtname»  | Nombre de la impresora de Windows a utilizar. Por defecto utiliza la impresora en curso.      |
| «portname» | Identificador del puerto a utilizar. Por defecto utiliza el asignado a la impresora en curso. |

#### IMPORTANTE

El fichero indicado en «file» se eliminará una vez finalizada la impresión.

## Definición de objeto

Un objeto CTL es un elemento de programación del lenguaje de cuarta generación de MultiBase. En un módulo CTL se pueden definir objetos procedurales y no procedurales. El lugar de definición de estos objetos es la sección DEFINE. Los objetos de la sección DEFINE denominados procedurales son: VARIABLES, PARÁMETROS y ARRAYS. Por su parte, los no procedurales son: FORM, FRAME y STREAM.

Asimismo, existe otro tipo de objetos de programación que afectan al CTL y que son los siguientes:

|                 |                                                                                                                                                                                                  |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CURSOR</b>   | Objeto que maneja información de la base de datos. Su definición puede realizarse en cualquier punto de un módulo donde pueda incluirse una instrucción de control de flujo.                     |
| <b>FUNCTION</b> | Objeto función que se define, como ya se ha visto en la estructura de cada tipo de módulo, debajo de la sección MAIN en un módulo principal y después de la sección LOCAL de un módulo librería. |
| <b>MENU</b>     | Objeto menú de tipo «Lotus» (de «iconos» en Windows). Su definición se realiza debajo de la sección MAIN en un módulo principal y después de la sección LOCAL de un módulo librería.             |
| <b>PULLDOWN</b> | Objeto «pulldown» que se define debajo de la sección MAIN en un módulo principal y después de la sección LOCAL de un módulo librería.                                                            |

## Componentes de un objeto

Dependiendo del tipo de objeto que se defina, procedural o no procedural, éste estará constituido por más o menos componentes. Los componentes que puede utilizar un objeto son los siguientes:

### **a) Secciones**

Este componente sólo puede definirse en el caso de los objetos no procedurales. Las secciones son subdivisiones especializadas en la ejecución de ciertas instrucciones no procedurales dependientes del momento o acción actual del programa.

### **b) Instrucciones relacionadas**

En CTL existen ciertas instrucciones, procedurales y no procedurales, propias de cada uno de los objetos. Algunas de estas instrucciones son comunes para varios objetos.

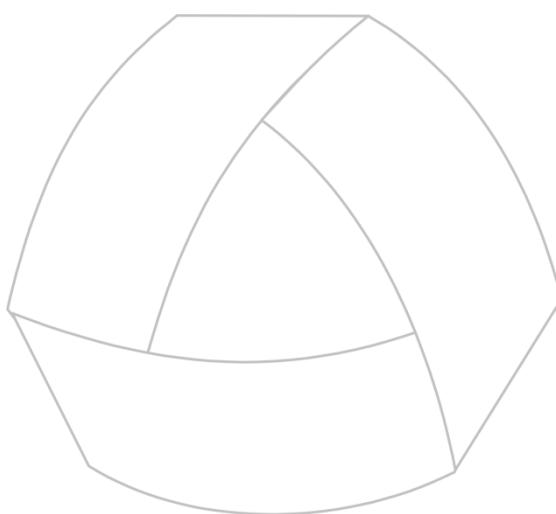
### **c) Variables internas**

CTL dispone de variables que se asignan valores de forma automática, dependiendo de la actividad del objeto en concreto al que se refieren. Las variables internas de cada objeto se han descrito anteriormente en este mismo capítulo.

### **d) Funciones internas**

Al igual que ocurre con las variables internas CTL para cada objeto, sucede con las funciones internas. Cada objeto dispone de ciertas funciones útiles para su programación y manejo. Todas las funciones internas de cada objeto se han descrito anteriormente en este mismo capítulo.





MultiBase  
MANUAL DEL PROGRAMADOR

**CAPÍTULO 6. Objetos procedurales  
de la sección DEFINE**

# Introducción

Los objetos considerados como procedurales cuya definición se realiza en la sección DEFINE de un módulo CTL son los siguientes:

- VARIABLES (compartidas).
- PARÁMETROS.
- ARRAYS.

En los siguientes apartados de este capítulo se explican las características y particularidades de cada uno de ellos.

# Variables

La definición de una variable CTL en un módulo tiene la siguiente sintaxis:

**[[NEW] SHARED] VARIABLE** nombre\_var {**LIKE** tabla.columna | tipo\_sql} [lista\_atributos]

Donde:

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NEW SHARED</b> | Palabras reservadas opcionales que indican la definición de una variable que será compartida por varios programas o módulos.                                                                                                                                                                                                                                                                                                                                                |
| <b>SHARED</b>     | Palabra reservada opcional que declara una variable compartida por varios programas o módulos. No obstante, para poder definir una variable como SHARED, tiene que haberse definido previamente en otro programa o módulo una variable «NEW SHARED» con el mismo nombre. Un programa en cuyo módulo principal se haya declarado una variable SHARED no puede ejecutarse sin que previamente se ejecute un programa en el que se defina la misma variable como «NEW SHARED». |
| <b>VARIABLE</b>   | Palabra reservada obligatoria que indica la definición de un objeto variable.                                                                                                                                                                                                                                                                                                                                                                                               |
| nombre_var        | Identificador que indica el nombre de la variable a definir.                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>LIKE</b>       | Palabra reservada opcional que indica que el tipo de datos SQL, la longitud y los atributos de la variable a definir serán como los de una columna existente en una tabla de la base de datos. En caso de utilizar esta cláusula, habrá que indicar obligatoriamente al principio del módulo la base de datos de la que se extrae la información mediante la sección DATABASE.                                                                                              |
| tabla.columna     | Nombres de la tabla y columna cuyos tipos de datos, longitud y atributos se adquieren para la definición de la variable.                                                                                                                                                                                                                                                                                                                                                    |
| tipo_sql          | Nombre del tipo de dato asignado a la variable. Estos tipos de datos son los siguientes: CHAR, SMALLINT, INTEGER, TIME, DECIMAL, DATE y MONEY (para más información acerca de estos tipos de datos consulte el apartado <a href="#">«Tipos de datos CTL»</a> en el capítulo anterior de este mismo volumen).                                                                                                                                                                |
| lista_atributos   | Lista de atributos CTL/CTSQL que serán asignados a una variable (para más información sobre estos atributos consulte el apartado <a href="#">«Atributos»</a> en el capítulo anterior de este mismo volumen).                                                                                                                                                                                                                                                                |

La definición de variables no compartidas es local a cada módulo (principal o librería). Cuando se utiliza una variable compartida en cualquiera de los módulos de un programa, dicha variable deberá estar definida como

NEW al menos en uno de ellos, teniendo que ser anterior en la secuencia de llamada. Si estuviese definida en más de uno, el CTL utilizará la última definición que encuentre en la cadena.

Al incluir una variable, compartida o no, en programación dentro de una instrucción de tipo SQL, dicha variable toma el nombre de variable receptora o «host», dependiendo de la cláusula de la instrucción SQL donde inter venga. En caso de ser una variable «host» tendrá que incluirse un dólar «\$» delante de su nombre.

A continuación, se indican varios ejemplos de definición de variables y cómo funcionan:

**Ejemplo 1 [DEFINE1]. Definición de una variable CHAR asignándole un valor inicial mediante el atributo DEFAULT:**

```
define
 variable i char(10) default "HOLA"
end define

main begin
 pause i
end main
```

La instrucción PAUSE de la sección MAIN muestra el valor de la variable «i» y detiene la ejecución del programa hasta que el operador pulse una tecla.

**Ejemplo 2 [DEFINE2]. Definición de una variable con la cláusula «LIKE»:**

```
database almacén

define
 variable i like clientes.empresa
end define

main begin
 let i = "TransTools"
 pause i
end main
```

La columna «empresa» de la tabla «clientes» en la base de datos «almacén» está definida con tipo de dato CHAR y longitud 25 bytes. Asimismo, tiene asignado el atributo UPSHIFT del CTL/CTSQL. En la ejecución de este programa, al asignar el literal «TransTools» en minúsculas a una variable definida con el atributo UPSHIFT, automáticamente CTL cambiará dicho literal a mayúsculas, presentándolo mediante la instrucción PAUSE.

**Ejemplo 3 [DEFINE3]. Variables compartidas por distintos programas:**

```
define
 new shared variable letras char(50)
 variable numero smallint default 1530
end define

main begin
 let letras = inwords (numero)
 ctl "define31"
 pause letras
end main
```

Este programa define dos variables: «letras» y «numero». A «numero» se le asigna el valor «1530» mediante el atributo DEFAULT. La primera instrucción del programa asigna a la variable «letras» el literal correspondiente a la traducción de «numero» en letras. Esta operación se realiza con la función interna de CTL «inwords()». Por lo tanto, en este primer programa la variable «letras» tiene el siguiente valor asignado: «mil quinientas treinta». Posteriormente, se ejecuta el segundo programa, «define31», que comprueba el valor de la variable SHARED «letras», que tendrá asignado el mismo valor que el indicado anteriormente.

Después de comprobar el valor, el programa asigna uno nuevo, finalizando a continuación, con lo que el control vuelve al primero de ellos («define3»). Por último, en este primer programa se comprueba el valor de la variable «letras» después de regresar del programa «define31». El valor que tiene en estos momentos la variable «letras» es: «siete mil ochocientas treinta y cinco».

### Ejemplo 3.1 [DEFINE31]. Segundo programa con variable SHARED:

```
define
 shared variable letras char(50)
 variable numero smallint default 7835
end define

main begin
 pause letras
 let letras = inwords(numero)
end main
```

### Ejemplo 4 [DEFINE4]. Variables compartidas por distintos módulos de un programa:

```
define
 new shared variable i smallint default 10
end define

main begin
 pause i
 pause comprueba()
end main
```

El módulo librería se llama «define41» y tiene el siguiente aspecto:

```
define
 shared variable i smallint
end define

function comprueba() begin
 if i = 10 then return "Prueba Correcta"
 else return "Prueba Incorrecta"
end function
```

Para poder ejecutar este programa, han tenido que enlazarse previamente ambos módulos mediante el comando **ctlink** generando el programa «define4.lnk».

El módulo principal define una variable «i» numérica y le asigna el valor «10». Después de mostrar dicho valor mediante la instrucción PAUSE y pulsar una tecla para que continúe la ejecución, se hace la llamada a una función que devolverá un valor. Este valor será mostrado en pantalla por medio de la instrucción PAUSE. Dicha función «comprueba» si se encuentra en otro módulo (librería) y realiza una condición. Dependiendo de si la evaluación de dicha condición es verdadera o falsa, la función devuelve al módulo principal un valor u otro.

## Parámetros

Un parámetro es una variable definida en el programa que se encuentra inicializada con un valor que procede de otro programa CTL o bien desde la línea de comando del sistema operativo en el momento de arrancar el programa. La sintaxis de definición de un parámetro es la siguiente:

**PARAMETER**[n] nombre\_para {LIKE tabla.columna | tipo\_sql} [lista\_atributos]

Donde:

|                  |                                                                                |
|------------------|--------------------------------------------------------------------------------|
| <b>PARAMETER</b> | Palabra reservada obligatoria que indica la definición de un objeto parámetro. |
|------------------|--------------------------------------------------------------------------------|

|                 |                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [n]             | Número entero que indica el orden del parámetro en la llamada (al primer parámetro hay que asignarle el número «1», y así sucesivamente).                                                                                                                                                                                                                                    |
| nombre_para     | Identificador que indica el nombre del parámetro a definir.                                                                                                                                                                                                                                                                                                                  |
| LIKE            | Palabra reservada opcional que indica que el tipo de datos SQL, la longitud y los atributos del parámetro a definir serán como los de una columna existente en una tabla de la base de datos. En caso de utilizar esta cláusula habrá que indicar obligatoriamente al principio del módulo la base de datos de la que se extrae la información mediante la sección DATABASE. |
| tabla.columna   | Nombre de la tabla y columna cuyo tipo de dato, longitud y atributos se adquieren para la definición del parámetro.                                                                                                                                                                                                                                                          |
| tipo_sql        | Nombre del tipo de dato asignado al parámetro. Estos tipos de datos son los siguientes: CHAR, SMALLINT, INTEGER, TIME, DECIMAL, DATE y MONEY (para más información acerca de estos tipos de datos consulte el apartado <a href="#">«Tipos de datos CTL»</a> en el capítulo anterior de este mismo volumen).                                                                  |
| lista_atributos | Lista de atributos CTL/CTSQL que serán asignados al parámetro (para más información sobre estos atributos consulte el apartado <a href="#">«Atributos»</a> en el capítulo anterior de este mismo volumen).                                                                                                                                                                   |

La ejecución de un programa CTL desde la línea de comando se consigue mediante el comando **ctl**. La forma de enviar un parámetro a un programa desde la línea de comando es separándolos mediante espacios en blanco. Por ejemplo:

```
ctl programa valor1 valor2 valor3
```

En el caso de las versiones de MultiBase para Windows, este comando tiene que indicarse en el campo «Línea de Comando» del correspondiente cuadro de diálogo de «Propiedades del Icono».

Por su parte, la forma de ejecutar un programa CTL desde otro programa puede ser mediante las instrucciones CTL o RUN. En ambos casos, la forma de enviar parámetros al segundo programa es separándolos por una coma. Por ejemplo:

```
ctl "programa", var1, var2, var3
```

Siendo:

|                |                                                                              |
|----------------|------------------------------------------------------------------------------|
| programa       | Nombre del programa donde se encuentran definidos los parámetros.            |
| var1,var2,var3 | Variables del programa inicial que pasan sus valores actuales al «programa». |

La definición de parámetros sólo tiene sentido en el módulo principal de un programa, ya que los módulos librerías no se ejecutan mediante las instrucciones indicadas anteriormente sino mediante la ejecución de alguno de los objetos incluidos en él. En el momento que comienza la ejecución de un programa que recibe parámetros, el tratamiento de éstos es idéntico al de las variables.

#### IMPORTANTE

CTL emitirá tantos mensajes de error como parámetros no recibidos en la llamada a un programa.

A continuación se indica un ejemplo de definición de parámetros para comprobar su funcionamiento y sintaxis:

**Ejemplo 1 [DEFINES]. Programa que asigna unos valores a dos variables que serán pasadas a un segundo programa como parámetros en su llamada:**

```
define
 variable numero smallint
 variable letra char(10)
end define

main begin
 let numero = 55
 let letra = "MultiBase"
 ctl "define51", numero, letra, 0
 pause "Fin de programa"
end main
```

Este programa define dos variables, «numero» y «letra», a las que se cargan dos valores, «55» y «MultiBase», respectivamente. Posteriormente, estas variables se pasan como parámetros al programa «define51» junto con un tercer parámetro constante, que es un número «0». El programa «define51» define tres parámetros, ya que son tres valores los que se le envían: «55», «MultiBase» y «0». Este segundo programa muestra los valores que ha recibido en cada parámetro. Pulsando una tecla en cada parámetro mostrado se termina el programa, devolviendo el control al primero, que mostrará un mensaje indicando el final de la ejecución.

**Ejemplo 2 [DEFINE51]. Programa llamado desde «define5»:**

```
database almacén

define
 parameter[1] a smallint
 parameter[2] b char(10)
 parameter[3] c like provincias.provincia
end define

main begin
 pause "Valor del primer parámetro: "&& a
 pause "Valor del segundo parámetro: "&& b
 pause "Valor del tercer parámetro: "&& c
end main
```

Los signos «&&» de las instrucciones PAUSE sirven para concatenar dos expresiones. En caso de ejecutar este segundo programa sin enviar tres valores en la línea de comando o en la instrucción CTL se producirían tres mensajes de error.

## Arrays

Un ARRAY define un número determinado de variables, siendo todas ellas del mismo tipo de dato SQL. Los ARRAYS que CTL maneja son de una dimensión. La sintaxis de definición de los ARRAYS es la siguiente:

**ARRAY** nombre\_array[n] tipo\_sql

Donde:

|              |                                                                                                                                                            |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ARRAY</b> | Palabra reservada obligatoria que indica la definición de un objeto ARRAY.                                                                                 |
| nombre_array | Identificador que indica el nombre del ARRAY a definir.                                                                                                    |
| [n]          | Número entero que indica los elementos de que se compone el ARRAY. Es decir, número de variables en que se subdivide un ARRAY. El primer elemento es el 1. |
| tipo_sql     | Nombre del tipo de dato asignado a todos los elementos del ARRAY. Estos tipos de datos son los siguientes: CHAR, SMALLINT, INTEGER, TIME, DECIMAL, DATE y  |

MONEY. Para más información acerca de estos tipos de datos consulte el apartado [«Tipos de datos CTL»](#) en el capítulo anterior de este mismo manual.

La forma de identificar uno de los elementos de un ARRAY es mediante el «nombre\_array» y el número de elemento entre corchetes. Por ejemplo, para asignar un valor al elemento 2 de un ARRAY habría que indicar:

```
define
 array meses[12] char(10)
end define

main begin
 let meses[2] = "Febrero"
end main
```

Asimismo, para hacer referencia a un carácter en concreto o rango de caracteres de un elemento de un ARRAY habría que indicarlo de la siguiente manera en cualquier instrucción de control de flujo:

```
if meses[2][1,3] = "Feb" then ...
```

Esta expresión tomaría los tres primeros caracteres asignados al elemento 2 del ARRAY «meses».

#### IMPORTANTE

Un elemento de un ARRAY no puede ser variable receptora de datos SQL ni variable «host», es decir, no puede incluirse dentro de una instrucción de tipo SQL.

A continuación se indica un ejemplo de manejo de ARRAYS:

**[DEFINE6]. Mostrar la fecha del día con un formato similar al siguiente: «8 de Septiembre de 2016». Este formato de fecha no puede conseguirse mediante los atributos FORMAT o USING. Los formatos de fecha de CTL podrían mostrar como máximo los tres primeros caracteres de un mes.**

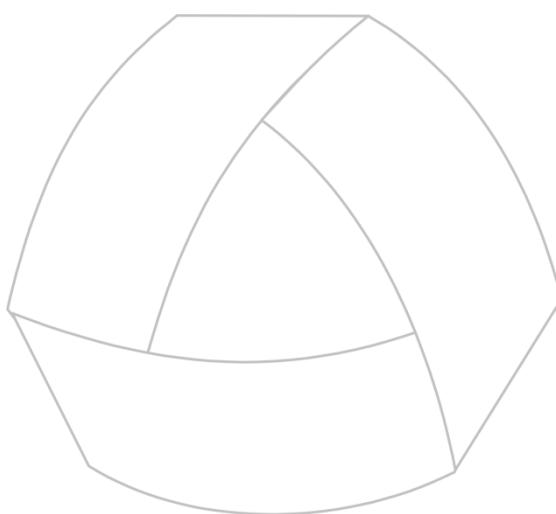
```
define
 array meses[12] char(10)
end define

main begin
 let meses[1] = "Enero"
 let meses[2] = "Febrero"
 let meses[3] = "Marzo"
 let meses[4] = "Abril"
 let meses[5] = "Mayo"
 let meses[6] = "Junio"
 let meses[7] = "Julio"
 let meses[8] = "Agosto"
 let meses[9] = "Septiembre"
 let meses[10] = "Octubre"
 let meses[11] = "Noviembre"
 let meses[12] = "Diciembre"
 pause day(today) using "##" && " de " && meses[month(today)] && " de " && year(today)
 using "&&&&"
end main
```

Este programa define un ARRAY de 12 posiciones, todas ellas «char(10)». En cada posición se asigna un mes para la construcción de la fecha completa. Este formato se consigue mediante la utilización de las funciones internas de fecha: «day()», «month()» y «year()».







MultiBase  
MANUAL DEL PROGRAMADOR

## **CAPÍTULO 7. Control de flujo en CTL**

## Instrucciones de control de flujo

---

CTL es un lenguaje de cuarta generación estructurado. Una de las características de un lenguaje de cuarta generación es que incluye instrucciones propias de un lenguaje de tercera generación. Este tipo de instrucciones son comunes en la mayoría de los lenguajes de programación, encargándose del control de flujo del programa.

Una de las características fundamentales e indispensables en todo módulo CTL es la repetición en la ejecución de instrucciones. La ejecución repetitiva de instrucciones se denomina «bucle» o «iteración». Un bucle se caracteriza fundamentalmente por dos aspectos:

- La(s) condición(es) que se evaluará(n) antes de cada repetición de la(s) instrucción(es).
- Conjunto de instrucciones que se ejecutarán un número indeterminado de veces, dependiendo de la evaluación de la(s) condición(es).

El algoritmo empleado por una estructura de control de flujo que implica la ejecución repetitiva de cierto número de instrucciones CTL es el siguiente:

- 1.— Evaluación de la(s) condición(es). En caso de evaluar a falso (FALSE) se continúa en el punto 4.
- 2.— Ejecución del conjunto de instrucciones afectadas por la instrucción de control de flujo.
- 3.— Retornar al punto 1.
- 4.— Continuación del programa.

Asimismo, existen ciertas instrucciones de control de flujo que permiten la ejecución selectiva o condicional de sentencias CTL. Estas instrucciones necesitan los siguientes componentes:

- 1.— Evaluación de la(s) condición(es).
- 2.— Conjunto de instrucciones que se ejecutarán cuando la evaluación de las condiciones sea verdad (TRUE).
- 3.— Conjunto de instrucciones que se ejecutarán cuando la evaluación de las condiciones sea falsa (FALSE).

La secuencia de operaciones o algoritmo empleado para la ejecución de este tipo de estructuras es la siguiente:

- 1.— Evaluación de la(s) condición(es). En caso de evaluar a verdadero (TRUE), la ejecución continuará en el punto 2. Si evalúa a falso (FALSE) continúa en el punto 4.
- 2.— Ejecución del conjunto de instrucciones afectadas por la instrucción de control de flujo al evaluar las condiciones a verdadero (TRUE).
- 3.— Continuar la ejecución en el punto 5.
- 4.— Ejecución del conjunto de instrucciones afectadas por la instrucción de control de flujo al evaluar las condiciones a falso (FALSE).
- 5.— Continuación del programa.

Las instrucciones que emplean este tipo de estructuras de control de flujo son: FOR, FOREVER, IF, SWITCH y WHILE.

NOTA: En los ejemplos que siguen, el conjunto de instrucciones a ejecutar, dependiendo de la evaluación de la(s) condición(es), puede estar compuesto por una sola instrucción CTL o por varias. En este último caso, todas las instrucciones deberán definirse entre las palabras BEGIN y END. Por ejemplo:

```
BEGIN
 instrucción 1
 instrucción 2
 ...
 instrucción x
END
```

## Instrucción FOR

La instrucción FOR inicializa una variable con un valor y ejecuta un conjunto de instrucciones repetidas veces hasta que dicha variable adquiera el valor indicado como límite en la instrucción. Su sintaxis es la siguiente:

```
FOR variable = expresión1 [DOWN] TO expresión2 [STEP expresión3]
 instrucciones
```

### Ejemplo [PFOR1]

```
define
 variable i smallint default 0
end define

main begin
 for i = 1 to 10
 pause inwords(i)
 end main
```

Este módulo cuenta desde el 1 al 10 y muestra por pantalla el resultado expresado en letras.

## Instrucción FOREVER

La instrucción FOREVER construye un bucle infinito. Su sintaxis es la siguiente:

```
FOREVER instrucciones
```

La forma de romper un bucle infinito de estas características es mediante la ejecución de cualquiera de las siguientes instrucciones: BREAK, EXIT PROGRAM y RETURN. Estas instrucciones se explican en la siguiente sección de este mismo capítulo.

En caso de que la instrucción FOREVER afecte a más de una instrucción, hay que establecer tanto el principio como el final con las palabras reservadas BEGIN y END, respectivamente.

### Ejemplo [PFOREVE1]:

```
main begin
 forever begin
 display now at 5,5 no list
 view
 if keypressed() = true then
 if yes("Desea finalizar","N") = true then break
 end
 end
end main
```

Este módulo construye un bucle infinito hasta que el operador pulse una tecla y responda que «Sí» desea finalizar en el menú de tipo «Lotus» (de iconos en Windows). Mientras no se pulse ninguna tecla, el módulo irá presentando en pantalla la hora actual. La instrucción que se encarga de mostrar la hora por pantalla es DISPLAY, siendo VIEW la encargada de llevar a cabo el refresco de la misma.

## Instrucción IF

La instrucción IF se encarga de realizar las bifurcaciones condicionales dentro de un módulo CTL, dependiendo de la evaluación de la condición o condiciones empleadas. La sintaxis de esta instrucción es la siguiente:

```
IF condición THEN instrucciones [ELSE instrucciones]
```

### Ejemplo [PIF1]:

```
main begin
 if yes("Elija una opción","Y") = true
 then pause "opción SI. Pulse Return para continuar"
 else pause "opción NO. Pulse Return para continuar"
```

```
 pause "Fin del programa"
 end main
```

Este módulo principal presenta un menú tipo «Lotus» (de «iconos» en el caso del Windows) con dos opciones («SÍ» y «NO»). Este menú es generado por la ejecución de la función interna de CTL «yes()». Dependiendo de que el operador elija la opción «SI» o «NO» se ejecutará la instrucción de la cláusula «THEN» o «ELSE», respectivamente. La instrucción PAUSE muestra por pantalla un valor y detiene la ejecución del programa hasta que el operador pulse una tecla.

### Instrucción SWITCH

Así como la instrucción IF permite tomar dos caminos alternativos, dependiendo de la evaluación de las condiciones, la instrucción SWITCH ofrece también varias posibilidades. Esta instrucción tiene dos tipos de sintaxis:

(A)

```
SWITCH expresión1
 CASE expresión2
 instrucciones
 [CASE expresión3
 instrucciones]
 ...
 [DEFAULT
 instrucciones]
END
```

(B)

```
SWITCH
 CASE condición1
 instrucciones
 [CASE condición2
 instrucciones]
 ...
 [DEFAULT
 instrucciones]
END
```

Una instrucción SWITCH equivale a la anidación de varias «IF THEN ELSE».

#### Ejemplo [PSWITCH1]:

```
main begin
 switch weekday(today)
 case 0
 pause "DOMINGO"
 case 1
 pause "LUNES"
 case 2
 pause "MARTES"
 case 3
 pause "MIERCOLES"
 case 4
 pause "JUEVES"
 case 5
 pause "VIERNES"
 case 6
 pause "SABADO"
 end
end main
```

Este módulo muestra el día de la semana de la fecha actual. La función interna de CTL «weekday( )» devuelve un número, entre 0 y 6, que corresponde al día de la semana. En caso de haber utilizado el segundo formato de la instrucción SWITCH, el ejemplo quedaría de la siguiente forma:

```
switch
 case weekday(today) = 1
 pause "LUNES"
 case weekday(today) = 2
 pause "MARTES"
 case ...
end
```

NOTA: En caso de que se cumpla más de un «CASE» en la misma instrucción SWITCH, sólo se ejecutará el primero encontrado en dicha instrucción.

## Instrucción WHILE

Esta instrucción evalúa una o varias condiciones y, si el resultado es verdadero (TRUE), ejecuta repetidas veces un conjunto de instrucciones hasta que la evaluación de dichas condiciones sea falsa (FALSE). Su sintaxis es la siguiente:

**WHILE** condición instrucciones

La variable o variables que se condicionan en esta instrucción tendrán que variar su valor dentro del conjunto de instrucciones afectadas por la estructura. Si no fuese así la ejecución de dicho bucle no tendría fin, es decir, sería infinita.

### Ejemplo [PWHILE1]:

```
define
 variable i smallint default 0
end define

main begin
 while i <= 10 begin
 let i = i + 1
 end
 pause "El valor de i es " && i using "<<<<<<"
end main
```

Este módulo incrementará la variable «i» con valores consecutivos desde el «0». Cuando el valor que adquiere la variable «i» es mayor que 10 finaliza la instrucción. En este caso, cuando «i» adquiera el valor 11 se acabará el bucle. Para comprobar el valor de dicha variable una vez se rompe el bucle se utiliza la instrucción PAUSE. Esta instrucción mostrará la concatenación de un literal y el valor de la variable «i» ajustada a la izquierda («using «<<<<<<»»).

En caso de no inicializar la variable «i» a cero con el atributo DEFAULT, la condición del bucle WHILE no se cumpliría nunca, ya que el valor por defecto que toma una variable es nulo («NULL»). Es decir, la evaluación de una condición siempre será falsa cuando una de sus expresiones a evaluar sea nula («NULL»).

NOTA: Para más información acerca de la sintaxis y características particulares de cada una de estas instrucciones consulte el Manual de Referencia.

## Anidación de estructuras

En el apartado anterior se han indicado las instrucciones de control de flujo que construyen bucles de repetición de un conjunto de instrucciones CTL e instrucciones que solucionan problemas de selección. Ahora bien, tanto unas como otras pueden contener a su vez otras instrucciones. Por ejemplo, una instrucción WHILE puede contener como conjunto de instrucciones a ejecutar una IF, y ésta a su vez otra instrucción FOR, y así sucesivamente. El siguiente ejemplo indica que cada instrucción puede estar anidada (contenida) dentro de otra. La estructura de estos anidamientos sería como sigue:

```
main begin
 forever begin
 while condición begin
 if condición then ...
 else ...
 ...
 end
 ...
 end
end main
```

En este ejemplo, la instrucción IF es la que más veces se ejecutará, ya que se encuentra anidada (contenida) en la WHILE, y ésta a su vez en la FOREVER.

## Instrucciones de ruptura de bucles

---

En el apartado [«Instrucciones de control de flujo»](#) de este capítulo se han indicado las instrucciones que construyen bucles o iteraciones para la repetición de un conjunto de instrucciones CTL. Dentro de esta sección vamos a tratar aquellas instrucciones encargadas de la ruptura de bucles. El uso de estas instrucciones es especialmente útil en el caso de la FOREVER, que como veíamos anteriormente era la encargada de construir un bucle infinito.

Las instrucciones de ruptura de bucles son: BREAK, CONTINUE, EXIT PROGRAM y RETURN.

A continuación se indican las características de cada una de ellas:

### Instrucción BREAK

Esta instrucción rompe cualquier bucle, continuando la ejecución del programa en la instrucción siguiente a la que construye el bucle (FOR, WHILE, FOREVER). Su sintaxis es la siguiente:

#### BREAK

##### Ejemplo [PBREAK1]:

```
define
 variable i smallint default 0
end define

main begin
 for i = 1 to 10 begin
 pause i
 if actionlabel(lastkey) = "fquit" then break
 end
 pause "Fin del programa"
end main
```

Este módulo muestra los números del 1 al 10. Para pasar de un número a otro, el operador tiene que pulsar una tecla para continuar. En caso de que pulse la tecla asignada a la acción <fquit> ([F2] en UNIX y [ESC] en Windows) en el fichero «tactions» de MultiBase se producirá la ruptura del bucle FOR. Por último, presenta un literal indicando la finalización del programa. Habrá que pulsar una tecla para que desaparezca dicho literal y finalice el módulo.

### Instrucción CONTINUE

Esta instrucción salta una iteración o vuelta del bucle en la que está insertada. No obstante, y salvo que se trate de la última vuelta del bucle, éste sigue activo. Su sintaxis es la siguiente:

#### CONTINUE

##### Ejemplo [PCONTI1]:

```
define
 variable i smallint default 0
end define

main begin
 for i = 1 to 10 begin
 if i = 5 then continue
 pause i
 end
end main
```

```

end
 pause "Fin del programa"
end main

```

Este módulo cuenta desde el número 1 hasta el 10. Estos números son presentados por medio de la instrucción PAUSE. Al llegar al «5», salta esa vuelta y no lo presenta, continuando la ejecución.

## Instrucción EXIT PROGRAM

Esta instrucción finaliza la ejecución de un programa, incluso dentro de un bucle. Su sintaxis es la siguiente:

### EXIT PROGRAM

**Ejemplo [PEXPRO1]:**

```

main begin
 forever begin
 display "Pulsa una tecla " && keylabel("fquit") &&
 " para terminar" at 5,10 no list
 pause keylabel(lastkey)
 if actionlabel(lastkey) = "fquit" then exit program
 end
 pause "Fin de programa"
end main

```

Este módulo muestra un literal indicando que el bucle terminará cuando el operador pulse la tecla asignada a la acción <fquit> en el fichero de acciones «tactions». Por cada tecla pulsada aparecerá a continuación su etiqueta. Cuando se pulse la tecla indicada en el cartel inicial, el módulo ejecuta la instrucción EXIT PROGRAM, terminando su ejecución.

## Instrucción RETURN

Esta instrucción suele situarse dentro de las funciones locales al módulo para que devuelva un valor. No obstante, también puede emplearse dentro de cualquier estructura de instrucciones CTL situada en la sección MAIN para romper su ejecución. Esta ruptura consiste en abandonar la ejecución de dicho programa. Asimismo, como utilidad en esta ruptura se puede incluso devolver un valor al programa que realizó la llamada al programa cancelado. Dicho valor se puede obtener mediante la variable interna de CTL «status», pudiendo comprobar su valor después de la llamada a dicho programa con la instrucción «CTL».

Su sintaxis es la siguiente:

### RETURN [valor]

Ejemplos:

**[PRETURN1]:**

```

main begin
 ctl "preturn2"
 switch status
 case 10
 pause "El operador pulsó " && keylabel("fquit")
 case 20
 pause "El operador pulsó " && keylabel("faccept")
 end
 pause "Fin del programa"
end main

```

**[PRETURN2]:**

```

main begin
 forever begin
 pause "La última tecla pulsada es: " && actionlabel (lastkey)
 end
end main

```

```
 if actionlabel(lastkey) = "fquit" then return 10
 else if actionlabel(lastkey) = "facept" then return 20
 end
 pause "Fin del programa"
end main
```

El primer programa, «preturn1», llama al segundo, y, dependiendo de la tecla con la que el operador salió del bucle de dicho programa, pinta un mensaje con su etiqueta. La comunicación entre ambos programas se realiza mediante la instrucción RETURN y la variable interna «STATUS».

El segundo programa presentará la acción asignada en el fichero de acciones «tactions» de MultiBase a la última tecla pulsada. La función interna de CTL «actionlabel», al pasarle el código de la última tecla pulsada, devuelve dicha etiqueta o acción. Al no haber pulsado ninguna tecla en la primera vuelta del bucle FOREVER, su acción saldrá en blanco. En caso de que la tecla que se pulse no tenga acción en el fichero «tactions», no aparecerá ninguna etiqueta. El bucle FOREVER se rompe cuando la última tecla pulsada es la que corresponde con la acción <fquit> o <facept> en dicho fichero. Cuando se ejecuta la instrucción «RETURN 10» o «RETURN 20» el módulo termina en dicho punto sin permitir que el control siga por la siguiente instrucción del bucle.

## Asignación de valores a variables CTL

Cualquier variable o elemento de un ARRAY toma siempre por defecto nulo («NULL») como valor inicial, con independencia del tipo de dato con el que se defina. No obstante, en el caso concreto de las variables, puede especificarse el atributo DEFAULT para inicializarlas con un valor distinto de nulo. Esta forma de inicializar las variables es especialmente recomendable en las de tipo numérico (SMALLINT, INTEGER, DECIMAL y MONEY). Las instrucciones de CTL que se encargan de asignar valores de variables de usuario son LET y PROMPT FOR.

No obstante, en la sintaxis de CTL existen otras instrucciones que permiten asimismo asignar valores a variables CTL de un objeto FORM o FRAME (ADD, ADD ONE, MODIFY, QUERY, INPUT FRAME, etc.). Estas últimas se comentan en los capítulos correspondiente a dichos objetos dentro de este mismo volumen.

## Instrucción LET

Esta instrucción asigna un valor a una variable, a un parámetro o a un elemento de un ARRAY. Su sintaxis es la siguiente:

**LET** variable[[subíndice1 [,subíndice2]]] = expresión

Esta instrucción permite todo tipo de conversiones de datos, siempre y cuando no se superen los límites de cada uno.

### Ejemplo [PLET1]:

```
define
 variable horaini time
 variable horafin time
end define
main begin
 let horaini = now
 display horaini at 3,5 no list
 view
 while keypressed() = false begin
 let horafin = now
 display horafin at 5,5 no list
 view
 end
 display "Tiempo ejecución " && horafin - horaini at 7,5 no list
 view
end main
```



Este módulo presenta su tiempo de ejecución. Al principio se recoge la hora actual («NOW»), que se muestra mediante la instrucción `DISPLAY`. A continuación, y mientras no se pulse ninguna tecla [`«keypressed()»`], se presentará cada segundo que transcurra en la ejecución. Cuando el bucle `WHILE` termina con la pulsación de una tecla, el módulo presenta el tiempo de ejecución, restando la hora final de la inicial.

NOTA: Esta instrucción también puede asignar valores a variables de un `FORM` o un `FRAME`. Este tipo de asignaciones se comentan en el capítulo correspondiente a dichos objetos CTL.

## Instrucción `PROMPT FOR`

La instrucción `PROMPT FOR` solicita por pantalla al operador un dato que será asignado a una variable o a un parámetro. Aunque realmente se trata de una instrucción de asignación de valores a variables, la consideramos como una instrucción de entrada de datos, y como tal se comenta en el siguiente capítulo de este mismo volumen relativo a la entrada y salida de información por pantalla.

## Llamadas a funciones

En programación es muy frecuente que un determinado procedimiento de cálculo, definido por un grupo de instrucciones, tenga que repetirse varias veces en distintos puntos de un módulo o programa, lo que implica que se tengan que escribir tantos grupos de dichas instrucciones como veces aparezca dicho proceso. La utilización de una función soluciona este problema de repetición, pudiendo hacer referencia a ella en cualquier punto del módulo principal, si es local a éste, o bien en cualquier módulo del programa (principal o librería) si se encuentra en un módulo librería.

La característica fundamental de una función es el cálculo de un único valor, pudiendo devolverlo al punto de llamada de la misma. La instrucción que se utiliza para que una función devuelva un valor es `RETURN`.

Como ya se ha indicado en el capítulo 5 de este mismo volumen, CTL dispone de ciertas funciones internas clasificadas en distintos apartados. Asimismo, un módulo (principal o librería) puede definir sus propias funciones. La estructura de una función de usuario es la siguiente:

```
FUNCTION identificador([parámetros])
 variable_local {tipo_sql | LIKE tabla.columna}
BEGIN
 instrucciones
END FUNCTION
```

La llamada a una función se realiza mediante la instrucción `CALL` del CTL. Esta instrucción no recoge el valor devuelto por la función en caso de que ésta incluya la instrucción `RETURN` en su sintaxis. La sintaxis de la instrucción `CALL` es la siguiente:

```
CALL identificador([parámetros])
```

Asimismo, una función que devuelva un valor mediante la instrucción `RETURN` puede incluirse en cualquier instrucción CTL que admita una expresión (`IF`, `WHILE`, `LET`, `SWITCH`, etc.). En ambos casos, dicha llamada está compuesta por el identificador de la función y por la lista de parámetros separados por comas.

De la misma manera —ya sea utilizando la instrucción `CALL` o incluyendo la llamada a la función como una expresión dentro de una instrucción CTL—, es independiente si la función está definida en el módulo local o bien en cualquier módulo (librería) componente del programa actual. En este último caso, es requisito imprescindible que todos los módulos donde se encuentran las librerías que utiliza un programa estén previamente enlazados («linkados»), es decir, mediante la utilización del comando `ctlink`.

### Ejemplo [PCALL1]:

```
define
 variable cantidad integer default 0
end define

main begin
 prompt for cantidad label "Teclear cantidad"
 end prompt
 call calculo()
end main

function calculo() begin
 if cantidad is null then let cantidad = 0
 if cantidad % 2 = 0 then
 pause "La cantidad " && cantidad && " es par"
 else pause "La cantidad " && cantidad && " es impar"
 end function
```

Este módulo solicita al usuario una cantidad para comprobar si es par o impar. Esta labor la realiza una función local al módulo, denominada «calculo()», que muestra un cartel indicando si la cantidad introducida es par o impar. Si la cantidad que se pasa a la función es nula («NULL»), la función la convierte en cero («0»). Para determinar si una cantidad es par o impar, la función calcula el «módulo» de la división entre dos. Este cálculo se realiza con el operador «%». Si el resultado de esta operación es cero significa que la cantidad dividida es par, e impar en caso contrario.

### Ejemplo [PCALL2]:

```
define
 variable cantidad integer default 0
 variable cambio char(10)
end define

main begin
 prompt for cantidad label "Teclear cantidad"
 end prompt
 let cambio = cantidad left
 if calculo() = true then
 pause "La cantidad " && cambio clipped && " es par"
 else pause "La cantidad " && cambio clipped && " es impar"
 end main

function calculo() begin
 if cantidad is null then let cantidad = 0
 if cantidad % 2 = 0 then return true
 else return false
 end function
```

Este módulo realiza la misma operación que el ejemplo anterior, pero la llamada a la función se realiza dentro de la instrucción IF del CTL. Esta instrucción se encuentra dentro de la sección MAIN y, dependiendo del valor devuelto por la función «calculo()» (TRUE o FALSE), mostrará un cartel indicando que la cantidad es par o impar, respectivamente. Este cartel en este módulo emplea una variable auxiliar («cambio») de distinto tipo a la empleada en el ejemplo anterior. Esta variable obtiene la conversión de la cantidad tecleada a tipo de dato CHAR.

### Ejemplo [PCALL4]

```
define
 variable numero smallint default 0
end define

main begin
```

```

 forever begin
 prompt for numero label "Teclear un número"
 end prompt
 if cancelled = true then break
 pause factorial(numero)
 end
 pause "Fin del programa"
end main

function factorial(n) begin
 if n = 0 then return 1
 else return n * factorial (n - 1)
end function

```

Este programa calcula el factorial de un número. En primer lugar, mediante la instrucción PROMPT FOR, pide al operador el número sobre el que se va a calcular su factorial. En caso de que el operador no pulse la tecla asociada a la acción <fquit> se mostrará el cálculo realizado por la función «factorial». Esta función es recursiva, ya que en caso de ser un «número» mayor que «0» realiza un número indeterminado de llamadas a sí misma.

Para más información sobre funciones consulte el [capítulo 5](#) de este mismo volumen.

## Llamadas a módulos/programas CTL

Como ya vimos en el [capítulo 4](#) de este mismo volumen, un programa está compuesto por la unión de varios módulos, siendo sólo uno de ellos el que incluye la sección de entrada al programa, es decir, la sección «MAIN» (módulo principal), mientras que el resto de módulos se denominan librerías.

Dentro de este apartado vamos a considerar la posibilidad de ejecución de un programa desde la línea de comando del sistema operativo y desde otro programa CTL. Asimismo, se verá cómo se ejecutan los objetos incluidos en un módulo librería.

## Ejecución desde la línea de comando del sistema operativo

La ejecución de un programa CTL desde la línea de comando del sistema operativo se efectúa mediante el comando **ctl** seguido del nombre del programa a ejecutar. En el caso del Windows, habrá que crear un icono especificando la línea de comando correspondiente en el cuadro de diálogo «Propiedades del Elemento de Programa». La forma de iconizar una aplicación se explica en el capítulo «Traspaso de Aplicaciones» en el Manual del Administrador. Otra forma de ejecutar un programa desde Windows sería a través del icono «CTL» del grupo «MultiBase» (generado automáticamente en la instalación del producto).

En caso de que el programa que se ejecuta desde la línea de comando tenga que recibir algún parámetro, los valores que adquirirán deben incluirse en dicha línea de comando separados por un espacio en blanco. Si se necesita especificar un literal que incluya espacios en blanco, se deberá encerrar todo el literal entre comillas. En caso de pasar menos valores que parámetros tenga definidos el programa, se producirán tantos mensajes de error como valores falten. Estos mensajes de error no impiden que se ejecute el programa, sin embargo, los parámetros que definidos en él adquirirán el valor nulo como valor inicial.

En caso de enviar más valores que parámetros estén definidos en el programa, no se producirá ningún error. No obstante, aquellos valores que no tengan su correspondiente parámetro definido no serán recogidos, y por lo tanto no intervendrán en el programa.

Esta forma de ejecutar un programa CTL siempre habrá que utilizarla, aunque sólo se emplee para lanzar el primer programa de la aplicación. Este primer programa normalmente será el menú que agrupe todas las opciones de la aplicación.

NOTA: Para más información sobre las opciones admitidas por el comando **ctl**, consulte el capítulo sobre comandos en el Manual del Administrador.

### Llamada desde otro programa CTL

La ejecución de un programa CTL desde otro se realiza mediante la instrucción CTL del Lenguaje de Cuarta Generación de MultiBase. La sintaxis de esta instrucción es la siguiente:

```
CTL nombre_programa [lista_parámetros] [[NO] CLEAN]
[AT línea, columna] [DEBUG]
```

Si el programa llamado («nombre\_programa») necesita información utilizada en el programa que lo llama, éste puede enviarla como parámetros en la instrucción CTL, o bien, mediante la creación de variables compartidas (SHARED y NEW SHARED) entre ambos programas.

Al lanzar un programa mediante esta instrucción, aquél comparte los mismos recursos (procesos) del sistema operativo que el programa que lo llama. Un programa CTL que esté ejecutándose genera dos procesos UNIX/Linux: CTL y CTSQL.

Al ejecutar un programa con esta instrucción, el programa que realiza la llamada quedará activo en segundo plano, continuando su ejecución cuando finalice aquél. La ejecución continúa por la instrucción siguiente a la sentencia «CTL».

En caso de que algún programa esté compuesto de módulos librerías, éstos se cargarán en memoria y no se descargarán hasta que se abandone la aplicación.

Ejemplos:

#### Programa [PCTL1]:

```
define
 new shared variable contador integer default 0
end define

main begin
 forever begin
 let contador = contador + 1
 pause "Contador en el primer programa " && contador
 ctl "pctl2"
 if yes("Desea continuar","Y") = false then break
 end
end main
```

#### Programa [PCTL2]:

```
define
 shared variable contador integer
end define

main begin
 let contador = contador + 1
 pause "Contador en el segundo programa " && contador
end main
```

Entre ambos programas se mantiene una variable compartida que contendrá el número de ejecuciones total de ambos programas. En cada uno de ellos se muestra el valor asignado. Por último, se emplea la función «yes()» para determinar si se desea continuar con la ejecución o abandonarla.

NOTA: Indirectamente, también puede realizarse la llamada mediante la instrucción «RUN 'CTL ...'», pudiendo emplear en este caso todas las opciones del comando **ctl**. Para más información sobre este comando consultar el capítulo sobre «Comandos del CTL» en el Manual del Administrador.

## Llamada a objetos de un módulo librería

Un módulo librería no se puede ejecutar por sí solo. En caso de intentarlo no realizará ninguna operación. La ejecución de este tipo de módulos no es como la que entendemos por ejecución de un programa. La ejecución de un módulo librería consiste en la ejecución de alguno(s) de sus objetos (funciones, menús, «pulldowns»). No obstante, la ejecución de los objetos (menús y «pulldowns») se tiene que realizar por medio de funciones. Es decir, un objeto menú o «pulldown» incluido en un módulo librería, sólo podrá ejecutarse mediante su invocación en una función de dicho módulo local. Por lo tanto, el menú o «pulldown» podrá ejecutarse desde cualquier programa de la aplicación al realizar la llamada a dicha función incluida en el módulo librería.

Como ya se ha indicado anteriormente, la ejecución de una función, local al módulo o incluida en un módulo librería, se consigue mediante la instrucción CALL, o bien incluyéndola dentro de una instrucción CTL que admita una expresión como parámetro.

Ejemplos:

### Programa [PCALL3] (módulo principal):

```
define
 variable i integer default 0
end define

main begin
 forever begin
 if sino() = false then break
 let i = i + 1
 end
 pause "Número de vueltas " && i
end main
```

### Programa [PLIBRE1] (módulo librería):

```
define
 variable a smallint
end define

function sino() begin
 menu m1
 if a = true then return true
 else return false
end function

menu m1 "E l e g i r" down skip 2 no wait line 5 column 15
 option "C o n t i n u a r"
 let a = true
 option "S a l i r"
 let a = false
end menu
```

Este programa ejecuta un bucle en el que en cada vuelta pregunta si se desea seguir o cancelar. Esta pregunta se realiza mediante un menú de tipo «Pop-Up» con dos opciones que se encuentran en una librería. La llamada a este menú se realiza mediante la ejecución de una función de dicho módulo librería, siendo ésta la que invoque el menú. La comunicación entre el menú y esta función es mediante la asignación de valores a una variable local («a») del módulo. En caso de querer emplear esta forma de salir de cualquier programa de la aplicación, simplemente habría que enlazar el módulo principal con esta librería y realizar la llamada a la función «sino()».

## Variables de entorno relacionadas

Las variables de entorno relacionadas con la ejecución de programas o utilización de librerías son DBPROC y DBLIB.

- La variable de entorno DBPROC incluye una lista de directorios en los que pueden distribuirse los programas de la aplicación.
- Por su parte, la variable DBLIB incluye una lista de directorios en los que pueden distribuirse los módulos librerías de la aplicación.

Dependiendo del sistema operativo, el separador de directorios será el punto y coma, como ocurre Windows, o los dos puntos, como sucede en UNIX/Linux.

En caso de no tener definidas estas variables de entorno, CTL tomará por defecto el directorio en curso. Estas variables se examinan de izquierda a derecha.

Para más información acerca de estas variables consulte el capítulo sobre «Variables de Entorno» en el Manual del Administrador.

## Llamadas a programas del sistema operativo

Desde un módulo, principal o librería, se puede ejecutar cualquier comando del sistema operativo mediante la utilización de la instrucción RUN del CTL. La sintaxis de esta instrucción es la siguiente:

**RUN** comando parámetros [WAIT | SILENT] [[NO] CLEAN]

En UNIX/Linux, la ejecución de esta instrucción supone la parada momentánea del programa que está activo mientras se ejecuta el comando indicado en ella. Una vez que finaliza la ejecución de dicho comando, el programa que contiene la instrucción RUN continuará ejecutándose.



Por lo que respecta al caso del Windows, si se ejecuta un comando del sistema operativo se produce un «scroll» de la pantalla para abrir la sesión indicada. Una vez abierta, el comando se ejecutará, pero en «background», es decir, la ejecución del programa CTL que ha realizado la llamada continúa.

### Ejemplo [PRUN1]:

```
main begin
 switch getctlvalue("O.S.")
 case "WINDOWS"
 run "excel"
 case "UNIX"
 run "vi fichero"
 end main
```

Dependiendo del sistema operativo, este programa ejecuta un comando distinto en cada uno. En Windows ejecutará la hoja de cálculo «Excel» y en UNIX editará un fichero con el editor «vi».

La instrucción RUN es igualmente válida para la ejecución de un programa CTL, ya que en lugar de utilizar la instrucción CTL, se puede emplear el comando **ctl** de la línea de comando. El inconveniente de este método es que para el segundo programa se lanzan dos procesos nuevos en el sistema operativo (UNIX). En Windows se cargaría de nuevo en memoria el comando **ctl**, lo que implicaría consumir más recursos de la máquina. Su utilización sólo es recomendable cuando desde un programa CTL que maneja una base de datos se desea ejecutar otro programa que abre otra base de datos distinta, siempre y cuando no exista intercambio de información entre ellas.

Algunos de los comandos del sistema operativo se encuentran implementados en la sintaxis del CTL, por lo que no es necesario emplear la instrucción RUN para ellos (para más información [consulte el siguiente apartado](#) de este mismo capítulo).

Para ejecutar un comando del sistema operativo en «background» se tiene que emplear la siguiente sintaxis:

run "comando &"

En caso de que el «comando» utilice la salida estándar (por pantalla), esta salida se tiene que redireccionar a un fichero o al «/dev/null» (UNIX). Por ejemplo:

```
main begin
 run "ls *.sct | wc -l > numero &"
end main
```

En este ejemplo se cuentan cuántos ficheros con extensión «.sct» existen en el directorio en curso. El número calculado se grabará en un fichero, denominado «numero». Esta operación se realizará en «background». El tratamiento es idéntico en Windows.

## Comandos del sistema operativo embebidos

Ciertas instrucciones del lenguaje de cuarta generación de MultiBase tienen una ejecución idéntica a algunos comandos básicos del sistema operativo. El siguiente cuadro refleja las equivalencias entre instrucciones del CTL y los comandos de cada sistema operativo:

| CTL     | UNIX    | Windows |
|---------|---------|---------|
| cp      | cp      | copy    |
| mv      | mv      | rename  |
| rm      | rm      | delete  |
| test    | test    |         |
| ctlcomp | ctlcomp | ctlcomp |
| mkdir   | mkdir   | mkdir   |

La sintaxis de estas instrucciones es idéntica a la del comando en el sistema operativo:

```
cp expresión1 expresión2
mv expresión1 expresión2
rm expresión
test -permiso fichero {-o fichero | -a fichero}
ctlcomp expresión
mkdir expresión
```

### Ejemplo [PCOMAN1]. Compilación de un módulo cuyo nombre se pasa como parámetro:

```
define
 parameter[1] modulo char(18)
end define

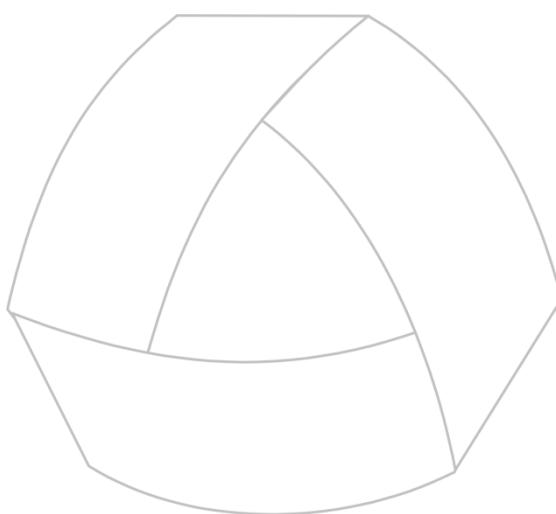
main begin
 ctlcomp modulo clipped
 test "-f", modulo & ".err"
 if status = 0 then begin
 clear
 switch getctlvalue("O.S.")
 case "WINDOWS"
 run "wedit ", modulo & ".err"
 case ""
 run "tword -e ", modulo & ".err"
 case "UNIX"
 run "tword -e ", modulo & ".err"
 end
 if yes("Borra el fichero de errores","Y") = true then
 rm modulo & ".err"
 end
 end
end main
```

Este programa compila el nombre del módulo pasado como parámetro en su llamada. En caso de existir algún error o aviso («warning») de compilación, comprueba si existe el fichero de errores «.err». Si existe, se presen-

ta mediante la ejecución del editor de ficheros de cada sistema operativo. El sistema operativo lo devuelve la función interna de CTL «getctlvalue(«O.S.»)». Por último, el programa pregunta mediante la función «yes()» si se desea borrar dicho fichero. En caso afirmativo, se utiliza la instrucción «rm» para eliminar el fichero de errores.

NOTA: Para más información sobre las instrucciones CTL indicadas en este capítulo consulte el Manual de Referencia.





MultiBase  
MANUAL DEL PROGRAMADOR

## **CAPÍTULO 8. Entrada y salida de información por pantalla**

## La lista de «display»

---

CTL maneja una lista de «display» que está formada por todos los elementos mostrados en pantalla por las diferentes instrucciones que se comentan dentro de este apartado, además de las relacionadas con los objetos FORM y FRAME, que son las que provocan su presentación en pantalla. Cada nueva instrucción DISPLAY aumenta esta lista de «display», identificando al nuevo elemento por las coordenadas (cláusula «AT») de las instrucciones DISPLAY y su tipo (FORM, FRAME) o dimensiones (cláusula «WITH»). Es muy importante tener en cuenta que según se presentan más elementos en pantalla, si no se elimina ninguno de ellos la lista incrementa, lo que podría ralentizar la presentación de nuevos elementos.

Del mismo modo que existen instrucciones que incrementan la lista de «display», existen también otras que eliminan elementos de dicha lista. Estas instrucciones son las «CLEAR...», excepto CLEAR por sí sola.

Por su parte, la instrucción VIEW vuelca los elementos de la lista de «display» no presentados aún en pantalla. Asimismo, las instrucciones que esperan una entrada del usuario, como PAUSE, READ KEY, MENU, etc., provocan igualmente este volcado de la lista de «display». Por otro lado, la instrucción REDRAW recupera el contenido de la pantalla, es decir, representa todos los elementos que se encuentren activos en la lista de «display». Cuando un programa CTL termina su ejecución, se vuelcan todos los elementos que se encuentren activos en la lista de «display».

Cada programa CTL mantiene separada su propia lista de «display», pudiendo inhibir la lista anterior con la cláusula «CLEAN» de la instrucción CTL al llamar a un nuevo programa (ver instrucción CTL en el Manual de Referencia).

## Instrucciones de salida

---

Dentro de este apartado se comentan las instrucciones del CTL encargadas de la entrada y salida de información por pantalla. No obstante, los objetos FORM, FRAME, MENU, PULLDOWN y STREAM emplean también la opción de entrada/salida por pantalla, pero su forma de trabajar se explica en sus respectivos capítulos dentro de este mismo manual.

Las instrucciones CTL que se encargan de la entrada/salida de información por pantalla, excluyendo las propias de los objetos anteriormente citados, son las siguientes:

|                        |                          |
|------------------------|--------------------------|
| <b>CLEAR</b>           | <b>DISPLAY BOX</b>       |
| <b>CLEAR AT</b>        | <b>DISPLAY FRAME BOX</b> |
| <b>CLEAR BOX</b>       | <b>DISPLAY LINE</b>      |
| <b>CLEAR FRAME BOX</b> | <b>MESSAGE</b>           |
| <b>CLEAR FROM</b>      | <b>PAUSE</b>             |
| <b>CLEAR LINE</b>      | <b>REDRAW</b>            |
| <b>CLEAR LIST</b>      | <b>VIEW</b>              |
| <b>DISPLAY</b>         |                          |

Cada una de las instrucciones de «display» tiene su respectiva CLEAR, lo que significa que cuando se represente algún elemento en pantalla, habrá que emplear su correspondiente instrucción CLEAR para eliminarlo siempre que se utilice la lista de «display».

En todas las instrucciones de salida que se comentan a continuación se puede incluir la opción «NO LIST». Esta opción afecta a todas las instrucciones de DISPLAY, salvo las relacionadas con los objetos FORM y FRAME, per-

mitiendo no incluir el elemento afectado (gráfico, expresión, etc.) en la lista de «display». Esto supone que CTL no mantendrá información sobre él, siendo aconsejable su utilización cuando se vaya a hacer un uso intensivo de estas instrucciones que sobrecargarían dicha lista, por ejemplo, varios «displays» en un bucle.

Ha de tenerse en cuenta que una instrucción CLEAR AT no puede borrar un elemento mostrado con la opción «NO LIST», por lo que se emitirá el correspondiente error. La única forma de borrar dicho elemento será efectuando otro «display» de una expresión que devuelva espacios en blanco, o bien ejecutando una instrucción CLEAR sin argumentos. La única forma de mantenerlo en pantalla será repintarlo con la misma opción o instrucción.

A continuación se indican las principales características de cada una de las instrucciones antes citadas (para una información más exhaustiva consulte el Manual de Referencia):

## Instrucción CLEAR

Esta instrucción borra de la pantalla todo lo que se encuentre representado en ella. Sin embargo, su ejecución no limpia los elementos incluidos en la lista de «display». Su sintaxis es la siguiente:

### CLEAR

En caso de borrar la pantalla con esta instrucción, se puede reconstruir mediante la ejecución de la instrucción REDRAW.

#### Ejemplo [PCLE1]:

```
define
 variable i smallint default 0
 variable j smallint default 0
end define

main begin
 for i = 1 to 20 begin
 for j = 1 to 80 begin
 display "+" at i, j
 end
 view
 end
 pause "Pulse Return para continuar"
 clear
 pause "Después del borrado. Pulse Return para continuar"
 redraw
 pause "Fin del programa"
end main
```

Este programa llena toda la pantalla con signos «+». La presentación de estos caracteres se realizará línea a línea. Esto se consigue al extraer la instrucción VIEW del bucle de presentación de dicho carácter («display «+»»). En caso de incluirse la VIEW junto a la respectiva DISPLAY, la representación se efectuaría carácter a carácter, lo que evidentemente ralentizaría la ejecución del programa. Una vez completadas las 20 primeras líneas con el signo indicado se detiene la ejecución del programa. A continuación, se limpia la pantalla mediante la instrucción CLEAR. Por último, se reconstruye su aspecto inicial con la instrucción REDRAW.

## Instrucción CLEAR AT

Esta instrucción borra de la lista de «display» (y consecuentemente también de la pantalla) el último elemento, de cualquier tipo, que se haya representado en las coordenadas indicadas en la cláusula «AT». Su sintaxis es la siguiente:

**CLEAR AT** línea, columna

En caso de intentar borrar un elemento que realmente no existe, CTL producirá un aviso («warning») indicando que no hay nada mostrado en dicha posición. El borrado en pantalla no se producirá hasta que ésta no se refresque con cualquiera de las instrucciones que realizan dicha operación (VIEW, READ KEY, PAUSE, etc.).

### Ejemplo [PCLEAT1]:

```
define
 variable i smallint default 0
 variable j smallint default 0
end define

main begin
 display double box at 1,1 with 20,80
 view
 for i = 2 to 19 begin
 for j = 2 to 79 begin
 display "+" at i, j
 view
 clear at i,j
 end
 end
 clear box at 1,1
 pause "Fin del programa"
end main
```

Este programa muestra una caja, confeccionada con caracteres semigráficos dobles, que ocupa toda la pantalla. En el interior de esta caja se representará un signo «+» que irá recorriendo desde la esquina superior izquierda hacia la derecha, y línea a línea hasta alcanzar el final de la caja. Según se va representando el signo, éste se va eliminado, provocando el desplazamiento de un solo «+». Al final del programa se borra la caja mediante la instrucción CLEAR BOX (en este caso concreto se podría haber empleado igualmente la instrucción CLEAR AT, ya que no existe ningún otro elemento en dicha posición).

## Instrucción CLEAR BOX

Esta instrucción borra una caja mostrada en una determinada posición por la instrucción DISPLAY BOX (siempre que no incluya la cláusula «NO LIST»), sin tener en cuenta el tipo de caracteres semigráficos empleados (simples o dobles). Su sintaxis es la siguiente:

**CLEAR [DOUBLE] BOX AT** línea, columna

En caso de intentar borrar cualquier otro elemento que no sea una caja con semigráficos simples o dobles, CTL provocará un aviso («warning») indicando que no existe ninguna caja mostrada en dicha posición. Al igual que ocurre en la instrucción DISPLAY, el cambio en pantalla no se reflejará hasta que se ejecute una instrucción de refresco o petición de datos al operador (VIEW, PAUSE, MENU, etc.).

### Ejemplo [PCLEBOX1]

```
define
 variable i smallint default 0
end define

main begin
 for i = 1 to 9 begin
 display double box at i,i with 20 - 2 * i, 80 - 2 * i
 view
 clear box at i,i
 end
 pause "Fin del Programa. Pulse <Return> para continuar"
end main
```

Este programa pinta nueve cajas, cada una más pequeña que la anterior, que se van borrando según se representan gracias al empleo de la instrucción CLEAR BOX. Al final del programa la pantalla se queda vacía.

## Instrucción CLEAR FRAME BOX

Esta instrucción borra de la lista de «display» (y por tanto de la pantalla) una caja estándar de CTL pintada previamente con la instrucción DISPLAY FRAME BOX sin la cláusula «NO LIST». Su sintaxis es la siguiente:

**CLEAR FRAME BOX AT** línea, columna

El borrado físico de la pantalla no se producirá hasta que se ejecute una instrucción que provoque su refresco (VIEW, PAUSE, MENU, READ KEY, etc.).

### Ejemplo [PCLEFRB1]:

```
main begin
 display frame box at 1,1 with 19,79
 label "P r e s e n t a c i ó n"
 display box at 3,5 with 14,30
 display double box at 3,45 with 14,30
 display line at 10,7 with 12
 display double line at 10,47 with 12
 display down line at 5,19 with 10
 display down double line at 5,58 with 10
 display "S e m i g r á f i c o s" at 17,8
 display "S e m i g r á f i c o s" at 17,48
 display "S i m p l e s" at 18,13
 display "D o b l e s" at 18,55
 pause "Pintados los elementos. Pulse Return para continuar"
 clear at 18,55
 clear at 18,13
 clear at 17,48
 clear at 17,8
 pause "Borrados los literales. Pulse Return para continuar"
 clear down double line at 5,58
 clear double line at 10,47
 clear down line at 5,19
 clear line at 10,7
 pause "Borradas las líneas. Pulse Return para continuar"
 clear box at 3,5
 clear box at 3,45
 pause "Borradas las cajas. Pulse Return para continuar"
 clear frame box at 1,1
 pause "Fin del programa. Pulse Return para continuar"
end main
```

Este programa muestra una serie de elementos CTL: una caja estándar, cajas con semigráficos simples y dobles, líneas verticales y horizontales con semigráficos simples y dobles y literales. Una vez representados todos los elementos en la pantalla se eliminan individualmente con sus respectivas instrucciones CLEAR. Cada cierto número de borrados el programa detiene su ejecución con la instrucción PAUSE, prosiguiendo a continuación.

## Instrucción CLEAR FROM

Esta instrucción permite borrar de la lista de «display» y de la pantalla todos los elementos activos en dicha con posterioridad al elemento indicado. Éste puede ser cualquier elemento gráfico (LINE, BOX, FRAME BOX, etc.) como un objeto nominado (MENU, FRAME, FORM). Se borran todos los elementos excepto el referenciado en la instrucción. Su sintaxis es la siguiente:

**CLEAR FROM** elemento **AT** línea, columna

### Ejemplo [PCLEFRO1]:

```
main begin
 display frame box at 1,1 with 19,79
 label "P r e s e n t a c i ó n"
 display box at 3,5 with 14,30
 display double box at 3,45 with 14,30
 display line at 10,7 with 12
 display double line at 10,47 with 12
 display down line at 5,19 with 10
 display down double line at 5,58 with 10
 display "S e m i g r á f i c o s" at 17,8
 display "S e m i g r á f i c o s" at 17,48
 display "S i m p l e s" at 18,13
 display "D o b l e s" at 18,55
 pause "Pintados los elementos. Pulse Return para continuar"
 clear from frame box at 1,1
 pause "Fin del programa. Pulse Return para continuar"
end main
```

Este programa es equivalente al expuesto para la instrucción CLEAR FRAME BOX, salvo que ahora se eliminan todos los elementos pintados en pantalla, excepto la caja estándar, con una sola instrucción CLEAR.

## Instrucción CLEAR LINE

Esta instrucción borra una línea de la lista de «display» (y en consecuencia también de la pantalla), sin tener en cuenta el tipo de caracteres semigráficos empleados para su representación. No obstante, esta instrucción permite borrar también una línea que se haya representado con la instrucción DISPLAY LINE sin la cláusula «NO LIST». Su sintaxis es la siguiente:

**CLEAR [DOWN] [DOUBLE] LINE AT** línea, columna

Al igual que en casos anteriores, la instrucción CLEAR LINE no elimina realmente la línea de la pantalla hasta que no se ejecute una instrucción que provoque el refresco de la misma o que solicite la entrada de datos al operador (VIEW, PAUSE, MENU, READ KEY, etc.).

### Ejemplo [PCLELIN1]:

```
main begin
 display double line at 1,1 with 79
 display down double line at 1,1 with 20
 display down double line at 1,79 with 20
 display double line at 20,1 with 79
 pause "Pintado de CAJA. Pulse Return para continuar"
 clear double line at 1,1
 pause "Primera línea. Pulse Return para continuar"
 clear down double line at 1,1
 pause "Segunda línea. Pulse Return para continuar"
 clear down double line at 1,79
 pause "Tercera línea. Pulse Return para continuar"
 clear double line at 20,1
 pause "Fin del programa. Pulse Return para continuar"
end main
```

Este programa simula la representación de una caja doble en la pantalla, empleando para ello cuatro líneas, dos verticales y dos horizontales, con semigráficos dobles. Las cuatro primeras instrucciones se encargan de representar la caja. A continuación, y línea a línea, se van borrando hasta que vuelve a desaparecer la caja.

## Instrucción CLEAR LIST

Esta instrucción borra de la lista de «display» y de la pantalla todos aquellos elementos representados con instrucciones DISPLAY, sin cláusula «NO LIST», cuyo origen se encuentre en el área especificada por los parámetros definidos en ella. Es decir, simula una caja ficticia (no representada realmente) en la pantalla, eliminando todos aquellos elementos activos en la lista de «display» cuyas coordenadas de representación se encuentren dentro de la misma. Su sintaxis es la siguiente:

**CLEAR LIST AT** línea, columna **WITH** líneas, columnas

Los elementos pintados en pantalla con la cláusula «NO LIST» no intervendrán en el borrado.

### Ejemplo [PCLEIS1]:

```
main begin
 display frame box at 1,1 with 19,79
 label "P r e s e n t a c i ó n"
 display box at 3,5 with 14,30
 display double box at 3,45 with 14,30
 display line at 10,7 with 12
 display double line at 10,47 with 12
 display down line at 5,19 with 10
 display down double line at 5,58 with 10
 display "S e m i g r á f i c o s " at 17,8
 display "S e m i g r á f i c o s " at 17,48
 display "S i m p l e s " at 18,13
 display "D o b l e s " at 18,55
 pause "Pintados los elementos. Pulse Return para continuar"
 clear list at 2,2 with 18,75
 pause "Fin del programa. Pulse Return para continuar"
end main
```

La ejecución de este programa es idéntica a la del anterior, sólo que en este caso se ha empleado otra instrucción de borrado. En este ejemplo, todos aquellos elementos mostrados dentro de la caja estándar se borrarán, ya que sus coordenadas de representación se encuentran dentro de la caja ficticia creada por la instrucción CLEAR LIST.

## Instrucción DISPLAY

Esta instrucción muestra por pantalla, en las coordenadas que se indiquen, una expresión válida de CTL. Su sintaxis es la siguiente:

**DISPLAY** expresión [atributos] [**AT** línea, columna]  
[**WITH** líneas, columnas] [, expresión ....][**NO LIST**]

La «expresión» no se presentará en pantalla hasta que se ejecute una instrucción que refresque la lista de «display» (VIEW, PAUSE, MENU, READ KEY, etc.). En caso de incluir la cláusula «NO LIST» la «expresión» no se incluirá en la lista de «display». Para poder eliminar la expresión de la pantalla se deberá sobreescribirla, por ejemplo con espacios en blanco, ya que en este caso no pueden emplearse las instrucciones de la familia «CLEAR...».

### Ejemplo [PDISPLA1]:

```
define
 variable i smallint default 0
end define

main begin
 for i = 1 to 100 begin
 display i using "###" at 5,10 no list
 view
 display " " at 5,10 no list
```

```
 view
 end
 pause "Pulse Return para continuar"
 end main
```

Este programa cuenta y muestra por pantalla desde el número 1 hasta el 100. La presentación en pantalla no utiliza la lista de «display», ya que se encuentra activada la cláusula «NO LIST» de la instrucción DISPLAY. Por lo tanto, la única forma de borrar dichos números de la pantalla es mostrando blancos encima.

### Instrucción DISPLAY BOX

Esta instrucción muestra en pantalla una caja según las coordenadas y el tamaño especificados en su sintaxis. La caja puede representarse con caracteres semigráficos simples o dobles («DOUBLE»), siempre que éstos sean soportados por la pantalla que se esté utilizando (si se intenta representar una caja con semigráficos dobles en una pantalla que no admita este tipo de caracteres, éstos aparecerán por defecto como caracteres semigráficos simples). Si no se emplea la cláusula «NO LIST», la caja que se representa se incluye como un elemento más en la lista de «display».

La sintaxis de la instrucción DISPLAY BOX es la siguiente:

**DISPLAY [DOUBLE] BOX AT** línea, columna **WITH** líneas, columnas  
**[NO LIST]**

Para eliminar una caja mostrada en pantalla que emplee la lista de «display» habrá que utilizar la instrucción CLEAR BOX, o bien cualquiera de la familia «CLEAR...» que elimine más de un elemento de la lista de «display». Para repintar dicha caja en el momento de ejecutarse la instrucción DISPLAY BOX, ésta tendrá que ir seguida de una instrucción que refresque la lista de «display» en pantalla (PAUSE, VIEW, READ KEY, etc.).

#### Ejemplo [PDISBOX1]:

```
main begin
 display box at 5,10 with 4, 20
 display "Literal" at 7,15
 pause "Fin del programa"
end main
```

Este programa muestra un «Literal» dentro de una caja construida con semigráficos simples. Las instrucciones de «display» empleadas utilizan la lista de «display», cuyo refresco se provoca por la ejecución de la instrucción PAUSE.

### Instrucción DISPLAY FRAME BOX

Muestra en pantalla una caja estándar CTL. Este tipo de cajas se caracteriza por tener la parte superior en vídeo inverso, pudiendo opcionalmente incluir un literal como cabecera. El resto de la caja se construye con caracteres semigráficos dobles. Su sintaxis es:

**DISPLAY FRAME BOX AT** línea, columna **WITH** líneas, columnas  
**[LABEL {"literal" | número}] [NO LIST]**

En caso de emplear la lista de «display», es decir, si no se incluye la cláusula «NO LIST», para borrar una caja estándar habrá que utilizar su correspondiente instrucción «CLEAR...» (en este caso CLEAR FRAME BOX), o bien cualquier otra instrucción de la familia «CLEAR...» que limpie más de un elemento de la lista de «display».

#### Ejemplo [PDISFRB1]:

```
main begin
 display frame box at 1,1 with 19,79
 label "P r e s e n t a c i ó n"
 display box at 3,5 with 14,30
 display double box at 3,45 with 14,30
 display line at 10,7 with 12
```



```

display double line at 10,47 with 12
display down line at 5,19 with 10
display down double line at 5,58 with 10
display "S e m i g r á f i c o s " at 17,8
display "S e m i g r á f i c o s " at 17,48
display "S i m p l e s " at 18,13
display "D o b l e s " at 18,55
view
end main

```

Este programa muestra unos ejemplos de semigráficos simples y dobles con cajas y líneas, todos ellos encerrados en una caja estándar representada por la instrucción DISPLAY FRAME BOX. Todas estas instrucciones de «display» utilizan la lista de «display», lo que significa que todas las cajas y líneas deberían ser borradas por sus respectivas instrucciones «CLEAR...».

## Instrucción DISPLAY LINE

Esta instrucción muestra en pantalla una línea vertical u horizontal empleando caracteres semigráficos simples o dobles (al igual que ocurría con la instrucción DISPLAY BOX, si se intenta representar una línea con caracteres semigráficos dobles en una pantalla que no los admite, dichos caracteres serán sustituidos por semigráficos simples).

La sintaxis de la instrucción DISPLAY LINE es la siguiente:

**DISPLAY [DOWN] [DOUBLE] LINE AT** línea, columna **WITH** longitud  
**[NO LIST]**

La línea que se representa por defecto con la instrucción DISPLAY LINE es horizontal con semigráficos simples.

### Ejemplo [PDISLIN1]:

```

main begin
display line at 1,10 with 10
display double line at 4,10 with 10
display down line at 6,10 with 10
display down double line at 6,20 with 10
pause "Fin del Programa"
end main

```

Este programa muestra cuatro líneas, dos horizontales y dos verticales, empleando en ambos casos semigráficos simples y dobles.

## Instrucción MESSAGE

Esta instrucción del CTL muestra un mensaje en la línea de mensajes reservada en la pantalla, que por defecto es la número 23, si bien existe la posibilidad de cambiarla mediante la cláusula «MESSAGE LINE» de la instrucción OPTIONS.

Su sintaxis es la siguiente:

**MESSAGE** [expresión] **[REVERSE | UNDERLINE] [BELL]**

La instrucción MESSAGE maneja la lista de «display», por lo que el mensaje no se mostrará en pantalla hasta que no se fuerce su salida mediante la instrucción VIEW o bien mediante cualquiera de las que solicitan una entrada por parte del usuario (PAUSE, READ KEY, etc.).

### Ejemplo [PMESSAGE]:

```

define
variable numero smallint
end define

```

```
main begin
 message "Introduce un valor numérico" reverse
 prompt for numero label "Codigo"
end prompt
options
 message line 10
end options
message "El valor introducido es " && numero &&
 " .Pulse Return para continuar"
pause
end main
```

Este programa muestra un literal avisando al operador de que el valor que tiene que introducir para la instrucción PROMPT FOR es numérico. Posteriormente, se cambia la línea de mensajes a la 10 y se muestra la concatenación de dos literales con el valor introducido en pantalla por el operador.

### Instrucción PAUSE

Esta instrucción detiene la ejecución del programa hasta que el usuario pulse una tecla. Además, representa todos los elementos activos en la lista de «display» que no estuviesen pintados previamente. Asimismo, muestra una expresión en la línea de mensajes de la pantalla (por defecto la 23), que se elimina automáticamente cuando el usuario pulsa una tecla. Su sintaxis es la siguiente:

**PAUSE** [expresión] [**REVERSE** | **UNDERLINE**] [**BELL**]

En caso de haber empleado la instrucción MESSAGE antes de la ejecución de la PAUSE, el mensaje se borrará si ésta muestra una expresión. Esto se debe a que ambas instrucciones utilizan la misma línea de la pantalla.

#### Ejemplo [PPAUSE1]:

```
main begin
 display "Pulse una tecla. " && keylabel("fquit") clipped &&
 " para terminar" at 5,1
 view
 forever begin
 pause
 if actionlabel(lastkey) = "fquit" then break
 display "Tecla: " && keylabel(lastkey) clipped
 && " Acción: " && actionlabel(lastkey) at 15,1
 pause "Pulse una tecla para continuar"
 clear at 15,1
 end
 pause "Fin del programa. Pulse Return para continuar"
end main
```

Este programa construye un bucle para, después de solicitar al operador la pulsación de una tecla, y mediante las funciones internas de CTL «keylabel()» y «actionlabel()», mostrar la etiqueta de la tecla pulsada así como la acción asignada en el fichero «tactions» de MultiBase. El bucle terminará cuando el operador pulse la tecla asignada a la acción <fquit> (normalmente [F2] en UNIX y [ESC] en y Windows) en el fichero de acciones indicado. La instrucción PAUSE se emplea para obligar al operador a pulsar una tecla y posteriormente para representar un mensaje y permitir ver la tecla y la acción durante el tiempo que el operador estime oportuno.

### Instrucción REDRAW

Esta instrucción muestra en la pantalla todos los elementos que se encuentren activos en la lista de «display». Su sintaxis es la siguiente:

**REDRAW**

Esta instrucción anula el efecto de la instrucción CLEAR.

**Ejemplo:**

```

define
 parameter[1] modulo char(18)
end define

main begin
 display frame box at 1,1 with 15, 79 label "C a j a"
 display box at 5,10 with 4,50
 display "Compilar el módulo " && modulo clipped at 7,12
 pause "Pulse return para continuar"
 clear
 ctlcomp modulo clipped
 pause "Después de la compilación. Pulse Return para continuar"
 clear
 redraw
 clear at 7,12
 if status = 0 then
 display "No existen errores de compilacion" at 7,12
 else display "Existen errores de compilación" at 7,12
 view
 pause "Fin del programa. Pulse Return para continuar"
end main

```

Este programa muestra en primer lugar una caja estándar, dentro de la cual se incluye a su vez otra construida con semigráficos simples y dentro de ésta un comentario. Este comentario indica que se va a compilar el módulo recogido como parámetro. A continuación se limpia la pantalla, pero no la lista de «display», y se compila el módulo recogido como parámetro. Después de la compilación se dibuja nuevamente la pantalla anterior con las cajas y se cambia el comentario que se incluía dentro de la caja simple. El comentario que lo reemplaza indicará si la compilación del módulo ha resultado con errores o no.

## Instrucción VIEW

Esta instrucción se encarga de presentar por pantalla todos aquellos elementos que se encuentren en la lista de «display» y que no hayan sido mostrados todavía. Su sintaxis es la siguiente:

### VIEW

No obstante, existen otras instrucciones del CTL que, además de otras operaciones adicionales, provocan igualmente el refresco de la pantalla, como por ejemplo READ KEY, MENU, PAUSE, etc. En caso de no utilizar ninguna de éstas para refrescar la pantalla, CTL mostrará toda la lista de «display» que no esté activada en pantalla al finalizar el programa.

**Ejemplo [PVIEW1]:**

```

define
 variable i smallint default 0
 variable j smallint default 0
end define

main begin
 for i = 1 to 20 begin
 for j = 1 to 79 begin
 display "+" at i, j
 end
 view
 end
 pause "Fin del programa"
end main

```

Este programa rellena todas las columnas y filas de la pantalla con el signo «+». La representación se realizará línea a línea y no carácter a carácter. Este efecto se consigue al incluir la instrucción VIEW en el bucle que cuen-

ta las líneas. Sin embargo, si hubiésemos especificado la VIEW dentro del bucle «FOR» (que cuenta las columnas), al encontrarse éste afectado también por el bucle que cuenta las líneas, la representación se habría realizado carácter a carácter.

## Instrucciones de entrada

---

Al igual que existen instrucciones cuya característica fundamental es la presentación de información en pantalla (instrucciones de salida), existen otras que ofrecen la posibilidad de que sea el propio operador o usuario quien introduzca dicha información en la pantalla (instrucciones de entrada).

Dentro de este apartado vamos a tratar exclusivamente aquellas instrucciones de entrada de información por pantalla que manejan variables comunes de un programa. Otras instrucciones de entrada, orientadas al mantenimiento de los objetos FORM y FRAME del CTL se comentan en posteriores capítulos dentro de este mismo volumen.

Las instrucciones de entrada que vamos a tratar en este epígrafe son: EDIT VARIABLE, PAUSE, PROMPT FOR y READ KEY.

Para una información más exhaustiva acerca de estas instrucciones consulte el Manual de Referencia.

### Instrucción EDIT

Esta instrucción permite introducir o presentar información sobre una variable definida en el módulo o bien sobre un fichero ASCII del sistema operativo. En ambos casos, la edición de la información se realiza sobre una ventana cuyo tamaño se define en esta instrucción. Su sintaxis es la siguiente:

**EDIT** {**FILE** expresión | variable [**NO LINES**]} **AT** línea, columna  
**WITH** líneas, columnas [**LABEL** expresión1]

El límite máximo de edición de un fichero o variable con esta instrucción es de 32 Kbytes. Durante la edición, habrá que pulsar [RETURN] cada vez que se alcance el margen derecho de cada línea. No obstante, en el caso de la edición de variables, estos saltos de línea no serán grabados como tales si se especifica la cláusula «NO LINES». En definitiva, se trata de un editor de ficheros o variables de programa que contempla las opciones básicas de cualquier programa de tratamiento de textos convencional (movimiento por el texto, inserción y reemplazo, etc.). Para grabar la información editada en esta ventana sobre el fichero o sobre la variable especificada habrá que pulsar la tecla asignada a la acción <faccept> en el fichero de acciones «tactions» (que por defecto es la tecla [F1]). Por su parte, para abandonar la edición de una variable o un fichero con esta instrucción hay que pulsar la tecla asignada a la acción <fquit> (por defecto [F2] en UNIX y [ESC] en Windows).

#### Ejemplo [PEDIT1]:

```
define
 variable observa char(1560)
 variable i smallint default 0
 variable j smallint default 1
end define

main begin
 edit observa no lines at 1,1 with 20,78
 clear
 for i = 1 to 1560 step 78 begin
 let j = j + 1
 display observa[i,i + 77] at j,1
 end
 pause "Contenido de la variable editada. Pulse Return para continuar"
end main
```

Este programa edita una variable cuya longitud es de 1.560 bytes. Los saltos de línea introducidos en su edición no serán almacenados como tales. Una vez finalizada la edición de la variable, ésta se borrará la pantalla, presentándose a continuación su contenido. Esta presentación se realiza mediante subíndices de la variable.

#### Ejemplo [PEDIT2]:

```
define
 parameter[1] modulo char(18)
 variable moduloerror char(18)
 variable modulofuente char(18)
end define

main begin
 ctlcomp modulo clipped
 let moduloerror = modulo & ".err"
 let modulofuente = modulo & ".sct"
 while status <> 0 begin
 if yes("Desea editar el fichero de errores", "Y") = true then begin
 clear
 edit file moduloerror clipped at 1,1 with 20,78
 if cancelled = false then begin
 call nocomment(moduloerror clipped, modulofuente clipped)
 rm moduloerror
 ctlcomp modulo clipped
 end
 end
 else break
 end
end main
```

Este programa compila el módulo que recibe como parámetro y, en caso de existir algún error, pregunta al usuario si desea editar el fichero de errores resultante de la compilación. En caso afirmativo, el programa utiliza la instrucción EDIT FILE para corregir dicho módulo fuente de errores. Si el operador graba las modificaciones, se renombra dicho fichero, pero eliminando todos aquellos mensajes de error insertados en la compilación anterior. Después de esta sustitución de ficheros fuente (de «.err» a «.sct») se borra el fichero de errores y vuelve a compilar el módulo corregido. Si existen errores otra vez, se repetirá el mismo proceso anterior un número indeterminado de veces.

## Instrucción PAUSE

Esta instrucción suspende momentáneamente la ejecución de un programa hasta que el operador pulsa una tecla, permitiendo a su vez mostrar una expresión en pantalla. Su sintaxis es la siguiente:

**PAUSE** [expresión] [**REVERSE** | **UNDERLINE**] [**BELL**]

Esta instrucción se ha comentado en el apartado anterior al tratar las instrucciones de salida.

## Instrucción PROMPT FOR

La instrucción PROMPT FOR es la encargada de solicitar por pantalla al operador un valor que será asignado a una variable o a un parámetro definido en el módulo. La sintaxis de esta instrucción para la función anteriormente indicada es la siguiente:

**PROMPT FOR** identificador [[**NO**] **CLEAN**] lista\_atributos  
 [AT línea, columna]  
 [**ON ACTION** acciones  
 instrucciones]  
 [**ON KEY** teclas  
 instrucciones]  
**END PROMPT**

Por defecto, esta instrucción realiza siempre la petición del valor en la línea 22 de la pantalla, siempre y cuando no se haya indicado la cláusula «AT». No obstante, esta línea puede cambiarse mediante la cláusula «PROMPT LINE» de la instrucción OPTIONS. La línea donde se realiza la petición del dato se limpia de forma automática a no ser que se indique la cláusula «NO CLEAN». Una vez introducido el dato basta con pulsar la tecla asignada a la acción <faccept> en el fichero de acciones o [RETURN] para que la variable indicada en «identificador» tome dicho valor. Sin embargo, si se pulsa la tecla asignada a la acción <fquit> se cancelará la edición. Esta instrucción sólo asigna valores a variables y a los parámetros de la sección DEFINE. Nunca permitirá realizar una entrada sobre variables locales a las funciones ni sobre elementos de un ARRAY.

### Ejemplo [PPROMPT1]:

```
define
 variable nombre char(10)
end define

main begin
 display "Pulsa `control-t` o `" && keylabel("fleft-page")
 && "" para ejecutar acciones" at 19,1
 view
 prompt for nombre label "Teclea un nombre" upshift
 on key "control-t"
 pause "Se ha pulsado " && keylabel(lastkey)
 on action "fleft-page"
 pause "Se ha pulsado " && keylabel(lastkey)
 end prompt
 pause nombre
end main
```

Este módulo muestra un literal en el que indica que pulsando las teclas correspondientes se activarán las instrucciones no procedurales de la instrucción PROMPT FOR. La ejecución de la PROMPT FOR muestra un literal en la línea 22 para que el usuario teclee un nombre. Este nombre aparecerá en mayúsculas al teclearlo, ya que se ha incluido el atributo UPSHIFT. En caso de pulsar una de las teclas indicadas en el primer literal, se mostrará un cartel indicando qué tecla se ha pulsado. Por último, una vez aceptado el nombre introducido mediante [RETURN], o bien mediante la tecla asignada a la acción <faccept>, se muestra el valor asignado a la variable «nombre».

NOTA: Esta instrucción permite también la lectura de un STREAM abierto para lectura (INPUT).

## Instrucción READ KEY

Esta instrucción permite leer desde el teclado una tecla o carácter. Su sintaxis es la siguiente:

### READ KEY

La instrucción READ KEY se emplea también para el manejo de los objetos FORM y FRAME. Su ejecución es similar a la de la instrucción PAUSE, salvo que en este caso no se muestra ninguna expresión por pantalla. Su función consiste únicamente en paralizar la ejecución del programa hasta que el operador pulse una tecla. Esta instrucción actualiza la variable interna de CTL «lastkey».

### Ejemplo [PREADKE1]:

```
main begin
 forever begin
 message "Pulsar ["&&keylabel("fquit") & "]" para finalizar"
 read key
 display "" at 5,1 with 1,50
 if actionlabel(lastkey) = "fquit" then break
 display "La tecla pulsada es " &&
 keylabel (lastkey) at 5,1 no list
 end forever
 view
```

```

end
pause "Fin del programa"
end main

```

Este programa muestra un mensaje que indica la tecla que hay que pulsar para finalizar. A continuación, el programa solicita que se pulse esa tecla. Si no se hace así, aparecerá un literal indicando la etiqueta de la tecla pulsada.

## Alteración de las características estándares del CTL

CTL dispone de ciertas características que utiliza por defecto a la hora de ejecutar cualquier programa. No obstante, estas características pueden cambiarse a criterio del programador. La instrucción encargada de modificar las características estándares del CTL es **OPTIONS**, la cual se comenta a continuación.

### Instrucción **OPTIONS**

Esta instrucción se utiliza para definir opciones de funcionamiento de programas/módulos, según se incluya en las secciones «GLOBAL» (programa) o «LOCAL» (módulo librería). La instrucción **OPTIONS** puede incluirse también en cualquier parte del programa, alterando así las características CTL. Su sintaxis es la siguiente:

```

OPTIONS
 {[HELP FILE "literal"] |
 [HELP BOX AT línea, columna WITH líneas, columnas] |
 [MESSAGE LINE número] |
 [PROMPT LINE número] |
 [MENU LINE número] |
 [HELP MESSAGE número] |
 [PRINTER expresión] |
 [NO SHELL] |
 [NO MESSAGE] |
 [MESSAGE ON]}
END OPTIONS

```

Las líneas reservadas por defecto para la acción de ciertas instrucciones CTL son las siguientes (\*):

| <u>Línea</u> | <u>Instrucción</u> |
|--------------|--------------------|
| 21           | PROMPT FOR         |
| 22           | MESSAGE            |
| 23           | MENU               |

(\*) Estos números pueden variar en función del número de líneas que pueda manejar el terminal que se esté utilizando. En nuestro caso hemos considerado un terminal que maneja 24 líneas.

Estos valores por defecto pueden cambiarse mediante la ejecución de la instrucción **OPTIONS** y las cláusulas «**PROMPT LINE**», «**MESSAGE LINE**» y «**MENU LINE**».

#### Ejemplo [**POPTION1**]:

```

define
 variable i smallint default 0
 variable numero smallint
end define

main begin
 for i = 1 to 25 begin
 display i at i , 1
 view
 end
 for i = 1 to 2 begin
 prompt for numero label "Teclea un número"
 end
end

```

```
 end prompt
 message "Línea de mensajes. Pulse [Return] para continuar"
 pause
 options
 prompt line 1
 message line 2
 end options
 end
end main
```

Este programa enumera las líneas de la pantalla (de la 1 a la 25). Después de esta operación, solicita un valor numérico mediante la ejecución de la instrucción PROMPT FOR y muestra un mensaje. Estas dos operaciones se realizan con los valores por defecto de CTL. A continuación, estos valores se cambian a las líneas 1 y 2, respectivamente. Una vez cambiados estos valores vuelve a repetirse el mismo ciclo: Petición de un valor y muestra de un mensaje. En esta segunda ejecución, ambas operaciones aparecerán en las líneas especificadas por la instrucción OPTIONS.

## Otras instrucciones

---

Con independencia de las instrucciones relacionadas con los objetos FORM, FRAME, MENU, PULLDOWN y STREAM del CTL, cuya explicación se incluye más adelante en este mismo manual, dentro de este apartado trataremos ciertas instrucciones del Lenguaje de Cuarta Generación de MultiBase que, además de presentar cierta información por pantalla, permiten la recepción de valores sobre variables. Estas instrucciones son: PATH WALK, TREE WALK, WINDOW y TSQL.

Asimismo, como ya se indicó en el capítulo relativo a «Conceptos Básicos», existe un gran número de funciones internas del CTL que también manejan la pantalla e, incluso, algunas de ellas permiten manejar el teclado y realizar peticiones de valores al operador. Por ejemplo, la función «yes()» permite mostrar un comentario en pantalla y realizar un menú compuesto por dos opciones («SI» y «NO»). De estas dos opciones, el operador tendrá que elegir una de ellas para que el programa continúe.

### Ejemplo [PYES1]:

```
define
 variable i smallint default 0
end define

main begin
 display i at 5,1 no list
 view
 while yes("Desea continuar","Y") = true begin
 let i = i + 1
 display i at 5,1 no list
 view
 end
 pause "Fin del programa"
end main
```

Este programa define una variable numérica para contabilizar el número de vueltas que realizará un bucle a petición del operador. Dicha variable se inicializa con el valor cero. En caso de no utilizar el atributo «DEFAULT» y no haber inicializado dicha variable con cualquier instrucción CTL de asignación, el resultado del incremento realizado dentro del bucle sería siempre nulo. Esto se debe a que una expresión aritmética en la que intervenga un valor nulo siempre dará como resultado un nulo. El programa finaliza cuando así lo decide el operador respondiendo a la pregunta formulada por la función interna del CTL «yes()».

A continuación se comentan las características de las instrucciones indicadas anteriormente.



## Instrucción PATH WALK

Esta instrucción muestra en sucesivas ventanas el contenido de la lista de directorios especificada en su sintaxis, permitiendo seleccionar y obtener un valor entre los presentados. Dicho valor se recogerá en una variable definida en el módulo. En la última línea de la ventana se presenta el directorio en curso de la lista.

La sintaxis de esta instrucción es la siguiente:

```
PATH WALK FROM expresión [EXTENT extensión]
 líneas LINES columnas COLUMNS
 [LABEL {"literal" | número}]
 AT línea, columna
 SET variable
```

En caso de indicar más de un directorio en la «expresión» habrá que pulsar las teclas [AV-PÁG] y [RE-PÁG] para ver los ficheros o directorios incluidos en ellos.

### Ejemplo [PPATHWA1]:

```
define
 variable base char(18)
end define

main begin
 path walk from getenv("DBPATH") extent ".dbs"
 10 lines 2 columns
 label "Elegir base de datos"
 at 5,15
 set base
 pause "La base de datos elegida es "&& base
end main
```

Como ya se verá en el siguiente capítulo, una base de datos creada con el gestor CTSQL de MultiBase se identifica físicamente por ser un directorio con extensión «.dbs». Pues bien, este programa muestra los nombres de las bases de datos existentes en cada uno de los directorios incluidos en la variable de entorno DBPATH. En caso de que ésta contenga más de un directorio, las teclas [AV-PÁG] y [RE-PÁG] nos permitirán movernos por la lista. Cuando el operador pulse [RETURN], o bien la tecla asignada a la acción <facept> (por defecto [F1]) en una de estas ventanas, el valor sobre el que esté situado será el elegido, y por lo tanto el que se asignará a la variable «base».

## Instrucción TREE WALK

Esta instrucción muestra una serie de ventanas superpuestas (una por directorio) en forma de escalera. Con esta instrucción se puede cambiar de directorio para seleccionar uno de los ficheros que aparecen en ellas. El valor seleccionado (mediante [RETURN] o la tecla asignada a la acción <facept> —normalmente [F1]—, será el que se asigne a la variable del módulo. Dicho valor seleccionado incluye el «path» completo donde se encuentra el fichero. La sintaxis de la instrucción TREE WALK es la siguiente:

```
TREE WALK FROM expresión [EXTENT extensión]
 líneas LINES WIDTH columnas
 AT línea, columna
 SET variable
```

Esta instrucción facilita un posicionamiento directo sobre cualquiera de los directorios anteriores que componen el «path» del actual. Para realizar esta operación hay que pulsar la tecla [RETURN] o la asignada a la acción <facept> en la cabecera de la última persiana. En ese momento aparecerá una ventana con la lista de directorios que componen el «path» actual. Mediante las teclas de movimiento del cursor se puede seleccionar uno, apareciendo seguidamente su ventana con la información que contiene dicho directorio.

### Ejemplo [PTREEWA1]:

```
define
 variable modulo char(100)
end define

main begin
 forever begin
 tree walk from "." extent ".sct"
 10 lines width 20
 at 5,15
 set modulo
 if cancelled = true then break
 run "ctlcomp " && modulo clipped && " > mensajes"
 if status = 0 then pause modulo & ". COMPILACION CORRECTA"
 else pause modulo & ". COMPILACION ERRONEA"
 end
 pause "Fin del programa"
 end main
```

Este programa muestra los módulos fuente contenidos en el directorio en curso. En caso de elegir uno de ellos, el programa lo compila mediante el comando **ctlcomp**. Dependiendo de que se produzcan o no errores de compilación, el programa mostrará un mensaje indicando dicha situación. Este programa utiliza el comando **ctlcomp** y no la instrucción CTLCOMP del CTL porque mediante la instrucción RUN se pueden redireccionar los mensajes que muestra dicho comando. Dichos mensajes se ubicarán en un fichero, creado en el directorio en curso, denominado «mensajes».

## Instrucción TSQL

Esta instrucción ejecuta cualquier instrucción de SQL. Esta instrucción o grupo de instrucciones puede estar almacenada en un fichero o bien ser el resultado de una expresión. Su sintaxis es la siguiente:

```
TSQL [FILE] expresión [{TO | THROUGH} salida
[APPEND]] [[[líneas LINES] [LABEL {"literal" | número}]
[AT línea, columna]] [LOG]
```

En caso de que la instrucción SQL a ejecutar sea una SELECT, se mostrarán por defecto todas las filas de su tabla derivada sobre una ventana en la pantalla, siempre y cuando no se redirija su salida mediante las cláusulas «TO» o «THROUGH».

Dado que hasta estos momentos no hemos considerado ningún elemento del SQL, esta instrucción se comentará en el siguiente capítulo de este mismo volumen al tratar el manejo del SQL de forma dinámica.

## Instrucción WINDOW

Esta instrucción muestra una ventana cuyo contenido puede ser uno de los siguientes:

- Fichero ASCII.
- Tabla derivada resultante de una instrucción SELECT.
- Tabla derivada contemplada por un CURSOR.
- Resultado de la ejecución de un comando del sistema operativo (sólo en UNIX).

Asimismo, la instrucción WINDOW permite extraer valores de dicha ventana sobre una o varias variables del programa. No obstante, dentro de este apartado trataremos exclusivamente la primera opción (presentación de un fichero ASCII) y la última (ejecución de un programa del sistema operativo), ya que el resto de opciones están ligadas al manejo del SQL.

La sintaxis de la instrucción WINDOW es la siguiente:

```
WINDOW [SCROLL | CLIPPED] {FROM fichero | THROUGH programa}
 [AS FILE | BY COLUMNS]
 líneas LINES [columnas COLUMNS] [WIDTH tamaño]
 [STEP salto] [LABEL {"literal" | número}]
 AT línea, columna
 [SET lista_variables [FROM lista_columnas]]
```

Las cláusulas «SCROLL» y «CLIPPED» de esta instrucción están implementadas a partir de la versión 2.0, «release» 0.0 de MultiBase.

La cláusula «AS FILE» invalida la utilización de las opciones: «WIDTH», «STEP» y «SET».

#### Ejemplo [PWINDOW1]:

```
define
 variable fichero char(100)
end define

main begin
 while yes("Desea listar un fichero","Y") = true begin
 tree walk from "."
 10 lines width 20
 at 5,10
 set fichero
 if cancelled = false then
 window scroll from fichero clipped
 as file
 18 lines 80 columns
 label "L i s t a d o d e u n F i c h e r o"
 at 1,1
 end
 end
end main
```

Este programa pregunta al operador si desea listar algún fichero creado previamente. En caso afirmativo, mediante la instrucción TREE WALK se muestran todos los ficheros del directorio en curso. Cuando el operador pulse [RETURN], o bien la tecla asociada a la acción <faccept>, se asignará el valor sobre el que estaba situado a la variable «fichero». A continuación, este fichero se muestra en la pantalla mediante la instrucción WINDOW y la cláusula «AS FILE», pudiendo utilizar entonces todas las teclas de movimiento del cursor en la ventana que aparece.

Aunque en Windows la cláusula THROUGH compilará perfectamente, sólo ejecutará de manera correcta en las versiones de MultiBase sobre sistema operativo UNIX/Linux.

#### Ejemplo [PWINDOW2]:

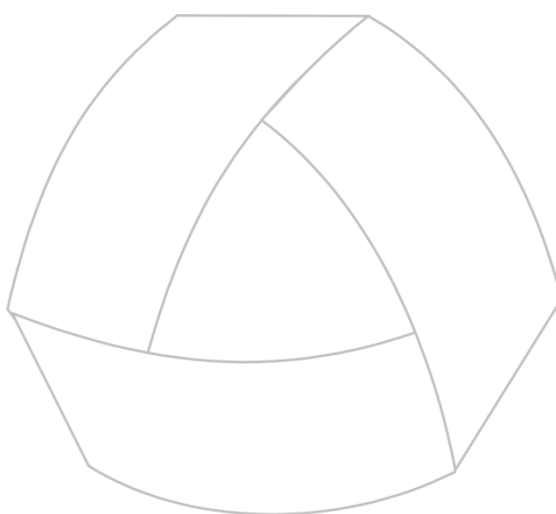
```
define
 variable fichero char(18)
end define

main begin
 while yes("Desea listar un fichero","Y") = true begin
 window through "ls *.sct"
 by columns
 10 lines 3 columns width 20
 label "E l e g i r"
 at 5,10
 set fichero
 if cancelled = false then
 window scroll from fichero clipped
 as file
 18 lines 80 columns
 end
 end
end main
```

```
 label "Listado de un Fichero"
 at 1,1
 end
 end main
```

Este programa es similar al anterior («pwindow1»), con la diferencia de que para la elección del fichero a editar hemos utilizado la instrucción WINDOW en lugar de la TREE WALK. En dicha instrucción WINDOW se emplea la cláusula «THROUGH», la cual extrae la información de la ejecución del comando **ls** del UNIX/Linux. Dicha información se presentará en la ventana en tres columnas. En este caso, cuando el operador decida pasar de página ya no podrá volver a la anterior, como ocurría en el supuesto de la presentación de ficheros en el ejemplo anterior.

Para más información sobre el funcionamiento de estas instrucciones consulte el Manual de Referencia.



MultiBase  
MANUAL DEL PROGRAMADOR

## **CAPÍTULO 9. Uso del SQL embebido en CTL**

### Introducción

---

El lenguaje SQL no es propiamente un lenguaje de programación, sino que se trata de un lenguaje de acceso y manipulación de datos. Esta manipulación se realiza sobre las tablas de una base de datos. Una característica importante de un lenguaje de programación es la definición y manejo de variables, característica ésta no contemplada por el SQL.

Por su parte, el CTL es un lenguaje de programación de cuarta generación que puede manejar perfectamente variables de programación, pero no datos sobre ficheros.

En consecuencia, el SQL, que maneja perfectamente los datos sobre tablas, y el CTL, que maneja perfectamente variables de programación, constituyen un binomio ideal para el desarrollo de aplicaciones.

Todas las instrucciones del SQL, cuya explicación se recoge en el Manual del SQL, se pueden incluir tal cual dentro de la gramática de un módulo CTL. CTL no hará distinción entre una instrucción propia y otra del CTSQL (SQL de MultiBase).

### Variables «hosts»

---

Una variable «host» es una variable (incluso «SHARED» o «NEW SHARED»), parámetro, variable de un FORM («variable», «displaytable», «pseudo», etc.), variable de un FRAME, etc., definida en un módulo que interviene en la sintaxis de una instrucción de SQL embebida en un módulo CTL. Cuando intervienen en una instrucción SQL, estas variables se identifican por ir precedidas del signo dólar («\$»).

En el Manual del SQL se indican todas las instrucciones disponibles en dicho lenguaje. No obstante, en los ejemplos expuestos en el citado manual no se han empleado variables «hosts», sino constantes. Pues bien, estas constantes deberán ser normalmente datos variables cuando las instrucciones SQL se encuentren embebidas en un módulo CTL. Dichos datos variables serán los posibles valores que podrán tomar las variables «hosts».

Las variables «hosts» están asociadas a ciertas cláusulas de algunas de las instrucciones propias del SQL. Estas cláusulas y las instrucciones a las que pertenecen son las siguientes:

a) Cláusula «WHERE». Esta cláusula se emplea en las instrucciones SELECT, UPDATE y DELETE del SQL. Su función consiste en establecer condiciones sobre ciertas columnas de una(s) tabla(s) para obtener su tabla derivada implicada en la operación de lectura, actualización y borrado, respectivamente.

#### Ejemplo [PWHERE1]:

```
database almacén

define
 variable provincia like provincias.provincia
 variable descripcion like provincias.descripcion
end define

main begin
 forever begin
 prompt for provincia label "Teclea código provincia"
 end prompt
 if cancelled = true then break
 select descripcion by name from provincias
 where provincias.provincia = $provincia
 if found = false then pause "PROVINCIA INEXISTENTE"
 else pause "Descripción " && descripcion clipped
 end
end main
```

Este programa define dos variables cuyos nombres coinciden con los de las columnas de la tabla «provincias». En primer lugar, el programa solicita al operador el código de la provincia que desea buscar en la tabla «provincias» de la base de datos «almacen». Una vez introducido el código y aceptado su valor, se busca su correspondiente «descripcion» mediante la instrucción SELECT. En caso de encontrar la fila aparecerá un mensaje indicando la descripción del código de la provincia solicitada por el operador. En caso contrario el mensaje indicará que esa provincia no existe. Este proceso se repetirá indefinidamente hasta que el operador cancele la introducción de un código de provincia.

b) Cláusula «HAVING». Esta cláusula se emplea en la instrucción SELECT para establecer condiciones sobre grupos de SQL. Por lo general irá acompañada de la cláusula «GROUP BY».

#### Ejemplo [PHAVING1]:

```
database almacen

define
 variable limite integer
end define
main begin
 prompt for limite label "Límite Inferior"
 end prompt
 window from select descripcion, count(*) cantidad
 from clientes, provincias
 where clientes.provincia = provincias.provincia
 group by descripcion
 having count(*) > $limite
 10 lines label "C l i e n t e s"
 at 5,10
 if found = false then pause "No existen filas que cumplan las condiciones"
end main
```

Este programa solicita al operador un dato numérico correspondiente al límite inferior de la condición empleada en la cláusula «HAVING». Dicho valor condicionará la selección de las filas de la tabla de «clientes» y «provincias» agrupadas por la columna «descripcion». En resumen, esta instrucción SELECT mostrará la cantidad de clientes que existen en cada una de las provincias. La cláusula «HAVING» indica que aquellas provincias en las que existan menos clientes de los especificados por el dato introducido por el operador no aparecerán en la ventana mostrada por la instrucción WINDOW. En caso de que la SELECT no devuelva ninguna fila aparecerá un mensaje indicándolo.

c) Cláusula «USING». Esta cláusula no existe en la sintaxis de las instrucciones SELECT, UPDATE y DELETE del SQL. Su función es establecer condiciones de igualdad sobre las columnas indicadas con las variables «hosts» cuyo nombre sea coincidente. En caso de que exista ambigüedad en los nombres de las columnas se podrá incluir la cláusula «ON» acompañada del nombre de la tabla a la que pertenece la columna especificada.

#### Ejemplo 1:

```
database almacen

define
 ...
 variable provincia like provincias.provincia
 ...
end define

main begin
 ...
 select descripcion from provincias
 where provincias.provincia = $provincia
 ...
end main
```

### Ejemplo 2:

```
database almacen

define
...
variable provincia like provincias.provincia
...
end define

main begin
...
select descripcion from provincias
using provincia
...
end main
```

En ambos ejemplos se lee de la tabla «provincias», pero con cláusulas diferentes. Las cláusulas «WHERE» y «USING» indican la misma condición sobre la tabla.

d) Valor. Algunas instrucciones del SQL admiten valores variables en su sintaxis. Por ejemplo, las instrucciones UPDATE, INSERT, DATABASE, etc. admiten variables «hosts» en su sintaxis para establecer respectivamente los valores a actualizar en una tabla, los valores a insertar y la base de datos a seleccionar.

### Ejemplo [PVALOR1]:

```
define
variable base char(18)
end define

main begin
path walk from getenv("DBPATH") extent ".dbs"
10 lines 2 columns
label "Elija Base de datos"
at 5,20
set base
if cancelled = false then begin
let base = basename(base)
database $base
pause "La base de datos seleccionada es "&& currd
end
end main
```

Este programa muestra en una ventana los nombres de todas las bases de datos (CTSQL) que se encuentren en el directorio en curso o bien en los directorios indicados en la variable de entorno DBPATH. Si el operador elige uno de los nombres mostrados en dicha ventana se procederá a la selección de dicha base de datos. A partir de ese momento la base de datos elegida será la base de datos en curso.

### Ejemplo [PUPDATE1]:

```
database almacen

define
variable formpago like formpagos.formpago
variable descripcion like formpagos.descripcion
end define

main begin
message "Elegir la forma de pago a actualizar"
window from select formpago, descripcion from formpagos
10 lines label "F o r m a s d e P a g o"
at 5,10
set formpago, descripcion from 1,2
```



```

if cancelled = true then exit program
prompt for descripcion label "Nueva descripcion"
end prompt
update formpagos set descripcion = $descripcion
using formpago
message "forma de pago después de su actualización"
window from select formpago, descripcion from formpagos
where formpagos.formpago = $formpago
10 lines label "F o r m a s d e P a g o"
at 5,10
end main

```

Este programa muestra una ventana con todas las filas de la tabla «formpagos» de la base de datos de demostración «almacen». Si el operador elige una de las filas, a continuación se pedirá la nueva descripción para la fila elegida. Cuando el operador acepte el nuevo valor, éste se grabará como nueva descripción en dicha tabla. Por último, y para comprobar que la actualización se ha realizado con éxito, se muestra la información de la fila recién actualizada.

### IMPORTANTE

Un elemento de un ARRAY ***nunca*** podrá ser una variable «host».

## Variables receptoras

Una variable receptora es aquella variable de programa (incluso «SHARED» y «NEW SHARED»), parámetro, variable de un FORM («vartable», «displaytable», «pseudo», etc.) o variable de un FRAME que recibe un valor devuelto por una instrucción SELECT.

Una instrucción SELECT devuelve una tabla derivada compuesta por aquellas filas que cumplen ciertas condiciones «WHERE». La forma de recoger los valores de las expresiones («columnas») leídas en la «selección» es empleando variables receptoras. Existen varias cláusulas para recoger valores de una instrucción SELECT en las que se pueden emplear variables receptoras. Estas cláusulas son las siguientes:

a) «INTO variables». Esta cláusula se incluye antes de la «FROM» de la instrucción SELECT. En ella hay que especificar, separados por comas, los nombres de aquellas variables que recogerán los valores de las columnas o expresiones leídas en dicha SELECT («INTO variable1, variable2, ..., variablen»). Estas variables pueden pertenecer a un FRAME.

### Ejemplo [PINTO1]:

```

database almacen

define
 variable albaran like albaranes.albaran
 variable total_albaran decimal(16,2)
end define

main begin
 window from select albaran, fecha_albaran, empresa
 from albaranes, clientes
 where albaranes.cliente = clientes.cliente
 10 lines label "A l b a r a n e s"
 at 5,5
 set albaran from 1
 if cancelled = false then begin
 select sum (cantidad * precio * (1 - descuento /100))
 into total_albaran
 from lineas
 where lineas.albaran = $albaran
 end
end

```

```
 pause "Total del albaran: " && total_albaran using "<<<<, <<<<, <<<<, <<<<, <<<<."
 end
end main
```

Este programa muestra una ventana que incluye una tabla derivada devuelta por una instrucción SELECT que lee de las tablas «albaranes» y «clientes» de la base de datos de demostración «almacen». Cuando el operador elige uno de los albaranes mostrados en dicha ventana se ejecuta una SELECT que devuelve una sola fila y un solo valor. Este valor es el total del albarán elegido por el operador. Por último, el valor recogido en una variable receptora se muestra mediante una instrucción PAUSE.

b) «BY NAME». Esta cláusula tiene la misma función que la «INTO», con la ventaja respecto a ella de que si los nombres de las variables receptoras coinciden con los de las columnas o expresiones leídas en la instrucción SELECT no es necesario especificar la lista de variables.

### Ejemplo [PBYNAME1]:

```
database almacen

define
 variable albaran like albaranes.albaran
 variable total_albaran decimal(16,2)
end define

main begin
 window from select albaran, fecha_albaran, empresa
 from albaranes, clientes
 where albaranes.cliente = clientes.cliente
 10 lines label "A l b a r a n e s"
 at 5,5
 set albaran from 1
 if cancelled = false then begin
 select sum (cantidad * precio * (1 - descuento /100)) total_albaran
 by name
 from lineas
 where lineas.albaran = $albaran
 pause "Total del albaran: " && total_albaran using "<<<<, <<<<, <<<<, <<<<, <<<<."
 end
end main
```

Este ejemplo es similar al expuesto para la cláusula «INTO», con la diferencia de que en este caso se ha utilizado la cláusula «BY NAME» en su lugar. Para ello ha sido necesario definir un «alias», denominado «total\_albaran», cuyo nombre es el mismo mismo que el de la variable decimal definida en el módulo. Esta coincidencia en el nombre hace que la instrucción SELECT asigne el valor calculado de la expresión empleada en la «selección» a la variable de igual nombre al alias definido para ella.

c) «BY NAME ON nombre\_frame». Si bien aún no hemos tratado las características del objeto FRAME, existe también la posibilidad de emplear sus variables como receptoras. En esta sintaxis se cargarán los valores de las columnas, expresiones o alias seleccionados en la «selección» en aquellas variables del FRAME que se denominen igual.

Asimismo, existe la posibilidad de emplear todas las variables de un FRAME como receptoras, independientemente del nombre con el que se hubiesen definido. La sintaxis para esta recepción de valores es la siguiente:

**INTO** frame.\*

La forma de trabajar con un FRAME se más adelante en este mismo volumen.

### NOTAS:

1.— En muchas ocasiones, es conveniente crear las variables que se definen en programación con el mismo nombre que las columnas de la tabla a tratar.

2.— Una instrucción que devuelve más de una fila como tabla derivada se tendrá que tratar como un CURSOR para la recogida de los valores de cada una de sus filas y columnas.

3.— Un elemento de un ARRAY no puede ser variable receptora.

## Apertura y cierre de una base de datos

La apertura de una base de datos sólo será necesaria cuando en el módulo se utilice alguna instrucción propia del SQL, no así en cualquier otro caso. La apertura de una base de datos se realiza mediante la ejecución de la instrucción DATABASE, cuya sintaxis es la siguiente:

**DATABASE** base\_datos [EXCLUSIVE]

Esta instrucción constituye una de las secciones de un módulo (principal o librería) de MultiBase. Dicha sección opcional será la primera de todas. No obstante, su inclusión será obligatoria cuando en la sección DEFINE exista algún objeto que utilice la cláusula «LIKE» en su definición. Por ejemplo:

```
database almacen

define
 variable cliente like clientes.cliente
end define

main begin
 ...
end main
```

En este ejemplo, la base de datos «almacen» se selecciona para la compilación del módulo, convirtiéndose además en la base de datos en curso durante su ejecución. Es decir, todas las operaciones que se realicen sobre tablas en el transcurso del programa estarán referidas a dicha base de datos, siempre y cuando no se indique lo contrario mediante la apertura de otra base de datos en cualquier otra sección.

La apertura de una base de datos implica tener unos «cursores» internos con los datos de las tablas del catálogo de la base de datos (compuesto por las tablas «sys\*», que se encuentran en el subdirectorio generado para la base de datos —directorio con extensión «.dbs»—).

La base de datos en curso no se cierra al finalizar la ejecución de un programa. Esto significa que la apertura de la base de datos deberá realizarse en el primer programa que haga referencia a alguna de sus tablas, no siendo necesaria su reapertura en cada programa que componga la aplicación. A partir de dicho programa, el resto de programas tendrán como base de datos en curso la abierta en aquél.

La ejecución de la instrucción DATABASE implica automáticamente el cierre de la base de datos que estuviese activa en ese momento. A su vez, el cierre de una base de datos conlleva el cierre de todos los «cursores» internos que necesita CTL, más los «cursores» utilizados en todos los módulos por parte del programador. Esto significa que deberán ser abiertos nuevamente todos aquellos «cursores» que se necesiten y ejecutar otra vez las frases «preparadas» (instrucción PREPARE) que interesen para la base de datos recién abierta. Esta operación se complica aún más cuando los «cursores» internos, los del usuario, las instrucciones WINDOW (que internamente generan un CURSOR) y las instrucciones PREPARE pertenecen a un módulo librería ya cargado en memoria, es decir, que alguna de sus funciones ya ha sido ejecutada en cualquier programa.

En SQL existe una instrucción para el cierre de la base de datos en curso sin necesidad de abrir otra. Dicha instrucción es CLOSE DATABASE.

Un CURSOR es la asignación de un nombre a una tabla derivada devuelta por la ejecución de una instrucción SELECT. Esta tabla derivada podrá leerse mediante instrucciones propias de los «cursores». La lectura de dicha tabla derivada se realizará de forma individual por cada una de sus filas. Para más información acerca del objeto CURSOR [consulte el siguiente capítulo](#) de este mismo volumen.

### Ejemplo [PDATA1]:

```
database almacen

define
 variable cliente like clientes.cliente
 variable empresa like clientes.empresa
end define

main begin
 prompt for cliente label "Código de cliente"
 end prompt
 select empresa by name from clientes
 using cliente
 pause "Base de datos en curso " && currdb
 pause "Empresa seleccionada " && empresa
end main
```

En este programa la directiva DATABASE del principio del módulo selecciona la base de datos «almacen». Esta base de datos será la que se utilice en compilación del módulo para obtener el tipo de dato SQL, longitud y atributos a asignar a las variables «cliente» y «empresa», ya que en su definición se emplea la cláusula «LIKE». Asimismo, dicha base de datos será la que se tomará como base de datos en curso durante la ejecución del programa, ya que en éste no se ha especificado ninguna otra instrucción de apertura de base de datos. Para comprobar que dicha base de datos es la que se encuentra activa durante la ejecución del programa, éste muestra un mensaje donde interviene la variable interna «currdb», que es la que contiene el nombre de la base de datos activa en ese momento.

Por el contrario, si no se hubiesen definido las variables «cliente» y «empresa» con la cláusula «LIKE», la directiva DATABASE no sería necesaria. En este caso, si bien la compilación sería correcta, se produciría un error en ejecución al intentar interpretar la instrucción SELECT, ya que la base de datos no estaría abierta.

### Ejemplo [PDATA2]:

```
database almacen

main begin
 pause "Base de datos en curso " && currdb
 ctl "pdata3"
 pause "Base de datos en curso " && currdb
end main
```

### Ejemplo [PDATA3]:

```
database almacen

define
 variable cliente like clientes.cliente
 variable empresa like clientes.empresa
 variable base char(18)
end define

main begin
 path walk from getenv("DBPATH") extent ".dbs"
 10 lines 2 columns
 label "Elija Base de datos"
 at 5,20
 set base
 if cancelled = false then begin
 let base = basename(base)
 database $base
 pause "La base de datos seleccionada es " && currdb
 prompt for cliente label "Código de cliente"
 end
end main
```

```

end prompt
select empresa by name from clientes
 using cliente
 pause "Empresa seleccionada " && empresa
end
end main

```

Para comprobar el funcionamiento de estos dos programas tendrían que existir, por lo menos, dos bases de datos con la misma estructura de la tabla «clientes».

Estos dos programas utilizan la base de datos «almacen» para la compilación. El primero de ellos, «pdata2», podría incluir la instrucción DATABASE dentro de la sección MAIN en lugar de especificarla como directiva, ya que no define ningún objeto de la sección DEFINE con la cláusula «LIKE». En dicho programa se comprueba cuál es la base de datos en curso antes y después de llamar al segundo programa.

El segundo programa expuesto utiliza la base de datos «almacen» como directiva a la hora de la compilación, pero en ejecución podrá utilizar cualquiera de las bases de datos existentes en cualquiera de los directorios de la variable de entorno DBPATH o bien en el directorio en curso («.»). La base de datos seleccionada en este segundo programa será la base de datos en curso, incluso para el primer programa cuando éste acabe.

Este cambio de base de datos puede ser erróneo en determinadas situaciones. Por ejemplo, supongamos que el primer programa utilizase «cursores» e instrucciones preparadas antes de la ejecución del segundo. Cuando este segundo programa finalizase, la base de datos habría cambiado, y por lo tanto todos los «cursores» del primer programa se habrían cerrado. Dichos «cursores» deberían ser abiertos nuevamente, y lo mismo sucedería con las instrucciones PREPARE empleadas anteriormente. Esta operación tendría que realizarse siempre y cuando dichos elementos volviesen a intervenir en la ejecución.

#### IMPORTANTE

SQL sólo permite tener activa una base de datos.

## Manipulación de datos con instrucciones SQL

La manipulación de datos en CTL sobre las tablas de la base de datos se realiza mediante la ejecución de las instrucciones que componen los sublenguajes de Manipulación de Datos (DML) y de Consulta (QL) del SQL (INSERT, UPDATE y DELETE por un lado y SELECT por otro, respectivamente). La integración de estas instrucciones dentro del ámbito del CTL ya se ha indicado en apartados anteriores de este mismo capítulo. La única forma de manipular los datos de las tablas de la base de datos es mediante las cuatro instrucciones indicadas anteriormente.

Todas las instrucciones citadas anteriormente manejan los datos de las tablas en bloque. Por su parte, la instrucción SELECT cuando lee de una(s) tabla(s) puede devolver más de una fila en su tabla derivada. En las restantes instrucciones el tratamiento es similar.

En los siguientes apartados se indican las características de cada una de estas instrucciones embebidas en el CTL.

### Instrucción SELECT

La instrucción SELECT es la única que permite la lectura de datos de las tablas de una base de datos. Por lo tanto, en el momento que un programa CTL necesite leer algún dato de las tablas de la base de datos habrá que emplear esta instrucción. La sintaxis de la instrucción SELECT embebida en CTL puede ser de dos formas:

(A)

```
SELECT [ALL | DISTINCT | UNIQUE] selección
[INTO {lista_variables | frame.*} | BY NAME [ON nombre_frame]]
FROM lista_tablas
[USING columnas [ON tabla]]
[WHERE {condición_comparación | condición_subselect}]
[GROUP BY lista_grupos]
[HAVING condición_grupos]
[ORDER BY columna [ASC | DESC] [,columna [ASC | DESC] [, ...]]
[INTO TEMP tabla_temporal]
```

(B)

```
SELECT [ALL | DISTINCT | UNIQUE] selección
[INTO {lista_variables | frame.*} | BY NAME [ON nombre_frame]]
FROM lista_tablas
[WHERE {condición_comparación | condición_subselect}]
[GROUP BY lista_grupos]
[HAVING condición_grupos]
[UNION [ALL]]
SELECT [ALL | DISTINCT | UNIQUE] selección
FROM lista_tablas
[WHERE {condición_comparación | condición_subselect}]
[GROUP BY lista_grupos]
[HAVING condición_grupos]
[UNION [ALL]]
[SELECT ...]
[ORDER BY columna [ASC | DESC] [,columna [ASC | DESC] [, ...]]
[INTO TEMP tabla_temporal]
```

Si una instrucción SELECT devuelve más de una fila en la tabla derivada y sus valores se desean recoger en variables receptoras, los valores que se asignarán serán nulos («NULL»). Ante esta situación, habría que definir la SELECT como un CURSOR y tratar de forma individual cada una de sus filas. Para más información sobre el manejo y tratamiento de un CURSOR [consulte el siguiente capítulo](#) de este mismo volumen.

La ejecución de una instrucción SELECT activa las variables internas del CTL «errno», «found» y «ambiguous».

Para una información más exhaustiva sobre la instrucción SELECT consulte el Manual de Referencia o bien el capítulo 7 del Manual del SQL.

## Instrucción INSERT

Esta instrucción agrega una o varias filas en una tabla de la base de datos en curso. La inserción física de estas filas en la tabla se realiza en el primer hueco que encuentre libre. Los posibles huecos o espacios vacíos existentes en la tabla serán debidos a un previo borrado de filas. Si no existiese ningún hueco donde se pudiese incluir la fila a agregar, ésta se añadiría al final del fichero. Esto significa que NUNCA podremos fiarnos del orden de inserción de las filas respecto a su lectura secuencial.

Dependiendo de que se vaya a insertar una sola fila o un número indeterminado de ellas, la instrucción INSERT admite dos tipos de sintaxis:

(A) Inserción de una sola fila:

```
INSERT INTO tabla [(columnas)] VALUES (valores)
```

(B) Inserción de un bloque indeterminado de filas (tabla derivada):

```
INSERT INTO tabla [(columnas)] instrucción_select
```

En el primer caso, los «valores» a insertar en la «tabla» serán probablemente variables «hosts» o constantes, mientras que en el segundo, la «instrucción\_select» puede ser cualquier instrucción SELECT válida que no in-

cluya las cláusulas «ORDER BY» e «INTO TEMP». Por supuesto, dicha SELECT puede incluir variables «hosts» dentro de sus cláusulas «WHERE» y «HAVING».

En el momento que se inserta una o varias filas en una tabla los índices de ésta se actualizan automáticamente con los nuevos valores.

## Instrucción DELETE

Esta instrucción permite borrar una o varias filas de una tabla de la base de datos en curso. Su sintaxis es la siguiente:

```
DELETE FROM tabla
 [WHERE {condicion_comparación | condición_subselect}]
```

Si no se especifica la cláusula «WHERE» se borrarán todas las filas de la tabla, siempre y cuando no se controle la integridad referencial, lo que demuestra el tratamiento en bloque de la información por parte de la instrucción DELETE.

El borrado de filas provocado por esta instrucción no es físico, sino lógico. Es decir, cuando una fila ha sido borrada empleando la instrucción DELETE, dicha fila no ha desaparecido «físicamente» del fichero, sino que únicamente ha sido marcada como «fila borrada». El borrado físico de esa fila sólo se producirá cuando se dé alguna de las siguientes circunstancias:

- En el momento en que se ejecute la instrucción INSERT, es decir, cuando se inserte otra fila en el hueco dejado por la fila marcada como «borrada».
- Cuando se provoque una alteración de la estructura de la tabla (instrucción ALTER TABLE).
- Cuando se repare la tabla (ejecución del comando **trepidix** de MultiBase).

En caso de intentar borrar una fila que se encuentre bloqueada por otro usuario, la instrucción DELETE esperará hasta que dicha fila quede liberada.

Si existiesen índices, éstos serán mantenidos automáticamente según se produzca el borrado de filas.

En caso de tener establecida integridad referencial «restrictiva» con otra u otras tablas, la instrucción DELETE no podrá borrar aquellas filas referenciadas en la tabla o tablas referenciantes. En caso de no encontrarse en una base de datos transaccional, la instrucción DELETE borrará todas aquellas filas previas a dicho error. Sin embargo, en caso de establecer integridad referencial («SET NULL») esta instrucción permitirá el borrado de todas las filas que cumplan las condiciones de la cláusula «WHERE», actualizando a su vez todas aquellas filas referenciadas en la tabla o tablas referenciantes.

### Ejemplo [PDELETE1]:

```
database almacén

define
 variable cliente like clientes.cliente
end define

main begin
 message "Elegir el cliente a borrar"
 forever begin
 window from select cliente, empresa
 from clientes
 order by empresa
 15 lines label "C l i e n t e s"
 at 2,20
 set cliente from 1
 if cancelled = true then break
 delete from clientes
 where clientes.cliente = $cliente
```

```
 if errno <> 0 then pause "ERROR EN BORRADO. Pulse [Return] para continuar"
 else pause "FILA BORRADA. Pulse [Return] para continuar"
 end
end main
```

Este programa presenta una ventana en la que se muestran todos los códigos de cliente y sus respectivas empresas de la tabla «clientes» de la base de datos de demostración «almacen». En caso de seleccionar algún código (mediante [RETURN] o la tecla asignada a la acción <faccept> —normalmente [F1]—), dicha fila se borrará al ejecutarse la instrucción DELETE. Si no se han producidos errores (integridad referencial con otras tablas), aparecerá un mensaje indicando que la fila ha sido borrada. En caso contrario el mensaje indicará la existencia de un error. Esto se detecta mediante la variable interna de CTL «errno». Este proceso se repetirá un número indeterminado de veces hasta que el operador no elija ningún código de cliente en la ventana (WINDOW).

En caso de que la instrucción DELETE vaya a borrar las filas de un CURSOR definido con la cláusula «FOR UPDATE», la sintaxis de la instrucción sería la siguiente:

```
DELETE FROM tabla
WHERE CURRENT OF nombre_cursor
```

Para más información sobre «cursores» [consulte el siguiente capítulo](#) de este mismo manual.

## Instrucción UPDATE

Esta instrucción permite cambiar y actualizar los valores de una o de todas las columnas componentes de una o varias filas existentes en una tabla. Su sintaxis es la siguiente:

```
UPDATE tabla SET columna = expresión
[, columna = expresión] [...]
[WHERE {condición_comparación | condición_subselect}]
```

Si no se especifica la cláusula «WHERE» se actualizarán todas las filas de la tabla, lo que demuestra el tratamiento en bloque de la información por parte de esta instrucción.

En caso de intentar actualizar una columna de una fila bloqueada por otro usuario, la instrucción UPDATE esperará hasta que dicha fila quede liberada.

Si hay establecida integridad referencial «restrictiva» con otra u otras tablas, la instrucción UPDATE no podrá actualizar aquellas columnas componentes de la clave primaria en la tabla referenciada cuyas filas tengan referencia en la tabla o tablas referenciantes. En caso de no encontrarse en una base de datos transaccional la instrucción UPDATE actualizará todas aquellas filas previas a dicho error. Sin embargo, en caso de establecer integridad referencial («SET NULL»), esta instrucción permitirá actualizar las columnas componentes de la clave primaria en aquellas filas que cumplan las condiciones de la cláusula «WHERE». En este caso se actualizarán también las columnas componentes de la clave o claves referenciales con valores nulos («NULL») en la tabla o tablas referenciantes.

En caso de existir índices, éstos se mantienen automáticamente según se produzca la actualización de filas.

### Ejemplo [PUPDATE2]:

```
database almacen

define
 variable cliente like clientes.cliente
 variable direccion1 like clientes.direccion1
end define

main begin
 message "Elegir el cliente a actualizar"
 forever begin
```



```

window from select cliente, empresa, direccion1
 from clientes
 15 lines label "C l i e n t e s"
 at 2,1
 set cliente, direccion1 from 1, 3
if cancelled = true then exit program
prompt for direccion1 label "Nueva dirección"
end prompt
if cancelled = false then begin
 update clientes set direccion1 = $direccion1
 where clientes.cliente = $cliente
 if erno <> 0 then pause "ERROR EN ACTUALIZACION. Pulse [Return] para continuar"
 else pause "FILA ACTUALIZADA. Pulse [Return] para continuar"
end
end
end main

```

Este programa muestra una ventana con el código del cliente, la empresa y la dirección de cada uno de los clientes grabados en la tabla «clientes». El operador tiene la oportunidad de elegir uno de los códigos mostrados en ella para actualizar la dirección de la empresa. La selección se produce cuando el operador pulsa [RETURN] o bien la tecla asignada a la acción <faccept>. En este momento aparecerá la petición de la nueva dirección manteniendo la antigua. Si el operador acepta la nueva, mediante [RETURN] o la tecla asignada a la acción <faccept>, dicha dirección será actualizada para el código de cliente elegido en la ventana inicial. En caso contrario aparecerá de nuevo dicha ventana para elegir otro cliente, y así sucesivamente hasta que el operador no elija ningún cliente.

En caso de que la instrucción UPDATE vaya a actualizar las filas de un CURSOR definido con la cláusula «FOR UPDATE» y que lea de una sola tabla, la sintaxis de dicha instrucción sería la siguiente:

```

UPDATE tabla SET columna = expresión
[, columna = expresión] [...]
WHERE CURRENT OF nombre_cursor

```

Para más información sobre «cursores» [consulte el siguiente capítulo](#) de este mismo manual.

## Ventanas de consulta y selección (instrucción WINDOW)

Las ventanas de consulta y selección de datos de la base de datos están representadas por la ejecución de la instrucción WINDOW del CTL. Esta instrucción, aparte de poder representar más información, permite mostrar en una ventana el resultado de una tabla derivada devuelta por la ejecución de una instrucción SELECT. Esta SELECT puede estar asociada a la declaración de un CURSOR. La sintaxis de la instrucción WINDOW relacionada con el SQL es la siguiente:

```

WINDOW [SCROLL | CLIPPED] FROM {instr_select | CURSOR nombre_cursor}
[BY COLUMNS]
líneas LINES [columnas COLUMNS]
[STEP salto] [LABEL {"literal" | número}]
AT línea, columna
[SET lista_variables [FROM lista_columnas]]

```

Esta instrucción mostrará una ventana en el caso de que la instrucción SELECT devuelva alguna fila en su tabla derivada. Para manejar la información sobre esta ventana se podrán emplear las teclas de movimiento del cursor (las flechas a izquierda y derecha sólo podrán emplearse si la ventana es «SCROLL»). Para abandonar la ventana sin seleccionar ningún dato habrá que emplear la tecla asignada a la acción <fquit>, mientras que si se desea seleccionar alguno se deberá pulsar [RETURN] o la tecla asignada a la acción <faccept>. Asimismo, las teclas [INS-CHAR] y [DEL-CHAR] permiten avanzar o eliminar caracteres para la «congelación»/«descongelación» de las columnas presentadas en esta ventana. Todas las teclas mencionadas son las que MultiBase utiliza por defecto en cualquier terminal; no obstante, éstas pueden ser modificadas por el ad-

ministrador a través del fichero «tactions» (para más información sobre la forma de llevar a cabo estas posibles modificaciones consulte el Manual del Administrador).

En caso de mostrar alguna columna numérica que tenga asignada algún formato («FORMAT») en diccionario, es decir, que dicho atributo afecte al CTSQL, la variable que recoja el valor de dicha columna en la cláusula «SET» ha de ser de tipo de dato CHAR.

Cada instrucción WINDOW genera internamente un CURSOR para el manejo de la tabla derivada presentada en ella. Hemos de tener en cuenta que el cambio de base de datos o la ejecución de la función interna «sqlreset()» cierra este tipo de «cursores» internos junto con los del usuario, debiendo prepararse de nuevo.

NOTA: En el capítulo anterior se explicó el funcionamiento de la instrucción WINDOW con la representación de un fichero o del resultado de la ejecución de un comando del sistema operativo (sólo en UNIX).

### Ejemplo [PWINDOW3]:

```
database almacen

define
 variable cliente integer
end define

main begin
 forever begin
 message "Pulsar " && keylabel("fuser1") clipped &&
 " para consultar"
 prompt for cliente label "Código de cliente"
 on action "fuser1"
 window from select cliente, empresa
 from clientes
 15 lines label "C l i e n t e s"
 at 2,15
 set cliente from 1
 end prompt
 if cancelled = true then break
 window scroll from select albaranes.albaran, fecha_albaran,
 descripcion, cantidad, precio, descuento,
 cantidad * precio * (1 - descuento / 100) T_O_T_A_L
 from albaranes, lineas, articulos
 where albaranes.cliente = $cliente
 and albaranes.albaran = lineas.albaran
 and lineas.articulo = articulos.articulo
 10 lines label "C o m p r a s —>"
 at 5,1
 if found = false then
 pause "NO HA REALIZADO COMPRAS. Pulse [Return] para continuar." reverse bell
 let cliente = null
 end
 end
end main
```

Este programa realiza una petición al operador, indicándole a su vez que si pulsa la tecla asignada a la acción <fuser1> aparecerá una ventana de consulta de códigos de clientes existentes en la base de datos. Una vez que el operador teclea un código o lo extrae de la ventana de consulta y lo acepta —mediante [RETURN] o la tecla asignada a la acción <faccept>—, aparecerá una ventana que contendrá la información referente a las compras realizadas por el cliente cuyo código fue indicado anteriormente. Esta información se extrae del enlace entre las tablas «albaranes», «lineas» y «articulos». En caso de que dicho cliente no haya realizado ninguna compra aparecerá un mensaje en vídeo inverso y sonará la campana del ordenador.

La columna «albaran» de la segunda instrucción WINDOW incluye delante el nombre de la tabla. Esto se debe a que dicha columna pertenece tanto a la tabla «albaranes» como a «lineas». Por lo tanto, se trata de una columna ambigua que hay que distinguirla indicando previamente el nombre de la tabla.

Asimismo, una de las columnas de esta tabla derivada es un cálculo («expresión») en el que intervienen tres columnas de la tabla «lineas». A esa expresión se le asigna un «alias» para que en la ventana aparezca dicho literal como encabezamiento de los datos.

## Importación y exportación de datos desde/hacia ficheros ASCII

CTL dispone de dos instrucciones, una para realizar la carga de datos desde un fichero ASCII hacia una tabla de la base de datos (instrucción LOAD), y otra para efectuar la descarga hacia ficheros ASCII de los datos contenidos en tablas de la base de datos (instrucción UNLOAD).

El formato que debe tener el fichero para la instrucción LOAD es la representación ASCII (en texto) de los valores a cargar, seguido cada uno de ellos de un carácter delimitador, de tal forma que por cada línea haya tantos caracteres de delimitación como columnas a cargar en la tabla. El carácter a emplear como delimitador se puede definir mediante la variable de entorno DBDELIM (consulte el capítulo sobre «Variables de Entorno» del Manual del Administrador). A su vez, cada registro viene separado por un carácter de nueva línea (este mismo formato es el que genera la instrucción UNLOAD). El aspecto de este fichero sería:

```
1|ALAVA|945|
2|ALBACETE|967|
3|ALICANTE|965|
4|ALMERIA|951|
33|ASTURIAS|985|
5|AVILA|918|
6|BADAJOZ|924|
7|BALEARES|971|
...
```

Si el carácter delimitador se encuentra como carácter dentro de una columna cuyo valor se va a descargar a un fichero ASCII, dicho carácter deberá ir precedido del signo «\», que indicará que el delimitador no deberá ser considerado como tal. Por ejemplo, si definimos la variable «DBDELIM=», (la coma será el carácter delimitador), el siguiente supuesto:

```
unload to "fichero.unl" select codigo, empresa, direccion1
from clientes

1,MOTOROLA,Duque de Medinacelli\, 13,
2,RTC S.A.,Profesor Waksman\, 49,
4,P.P.B.,Nueva direccion\, 12,
6,FCP S.A.,Orfila\, 85,
7,MATE,Juan Belmonte\, 53,
8,DRAPOSA,Gran Vía\, 57,
9,ALCOLEA,Narvaez\, 48,
...
```

considerará la coma de la columna «direccion» como un carácter más y no como delimitador, ya que se le ha antepuesto el signo «\».

NOTA: La importación y exportación de datos desde/hacia ficheros ASCII puede realizarse igualmente mediante el manejo de STREAMS de entrada y salida (INPUT para carga y OUTPUT para descarga). Para más información acerca del STREAM [consulte el capítulo dedicado a dicho objeto](#) en este mismo volumen.

A continuación se comentan las características de las instrucciones LOAD y UNLOAD.

### Instrucción LOAD

Esta instrucción carga datos provenientes de un fichero ASCII sobre una tabla de la base de datos. Su sintaxis es la siguiente:

**LOAD FROM** "fichero\_ascii" **INSERT INTO** tabla [(lista\_columnas)]

La «lista\_columnas» habrá que especificarla cuando el registro del fichero ASCII no contenga información para todas las columnas de la tabla, o bien cuando no se correspondan en el orden de definición.

Por lo que respecta a la ejecución de la instrucción LOAD, conviene advertir que ésta se abortará cuando se produzca cualquiera de las siguientes circunstancias:

- Si se está actualizando la variable interna de CTL «errno» y el fichero a utilizar en la operación de carga no existe.
- Cuando el primer registro (línea) no tenga el mismo número de campos que de columnas expresado en «lista\_columnas» de la instrucción INSERT.
- Si no se ha indicado la instrucción INSERT y el número de columnas que contiene la tabla no coinciden una a una en tipo y límites de rango según el tipo de dato SQL (CHAR, SMALLINT, INTEGER, etc.).
- Si la tabla tiene asociado un índice único, se cancelará la carga en el primer registro que contenga una clave equivalente a una ya existente en él.
- Cuando se intente cargar un dato no admitido por la columna, es decir, cualquier valor rechazado por los atributos «CHECK» o «NOT NULL».

La carga de datos sobre una base de datos transaccional tiene que realizarse controlando una transacción (BEGIN WORK y COMMIT WORK o ROLLBACK WORK), dependiendo respectivamente de que existan o no errores en la carga de datos. Esta operación en bases de datos transaccionales conlleva el bloqueo de cada una de las filas que intervienen en dicha carga, por lo que es recomendable bloquear la tabla antes de proceder a la carga de datos. El bloqueo de la tabla hay que realizarlo mediante la instrucción LOCK TABLE. Una vez se haya realizado la carga sin errores podremos desbloquear la tabla mediante la instrucción UNLOCK TABLE. Por ejemplo:

```
database almacen

main begin
 lock table provincias in exclusive mode
 begin work
 load from "fichero" insert into provincias
 if errno <> 0 then rollback work
 else commit work
 unlock table provincias
end main
```

#### Ejemplo [PLOAD1]:

```
database almacen

main begin
 whenever error ignore
 create temp table provincias1 (provincia smallint,
 descripcion char(20),
 prefijo smallint)
 test "-f", "provincias.unl"
 if status = 0 then begin
 load from "provincias.unl" insert into provincias1
 if errno <> 0 then pause "ERROR EN CARGA. Pulse [Return] para continuar." reverse bell
 else pause "CARGA CORRECTA. Pulse [Return] para continuar." reverse bell
 end
 else pause "FICHERO ASCII INEXISTENTE. Pulse [Return] para continuar." reverse bell
end main
```

Este programa crea una tabla temporal para la inserción de los registros contenidos en un fichero ASCII. En primer lugar, se comprueba, mediante la instrucción TEST y la variable interna de CTL «status», si existe el fichero ASCII de carga (en este caso «provincias.unl»). Si dicho fichero no existiese, aparecería un mensaje indicando el error. Por contra, si el fichero existe, se intenta realizar la carga sobre la tabla «provincias» de la base de datos «almacen». En el caso de que se produjese algún error aparecería un mensaje advirtiendo la interrupción de este proceso, con lo que el programa finalizaría por defecto su ejecución. Para evitar esta circunstancia (la finalización del programa), se emplea la instrucción «WHENEVER ERROR IGNORE», que permite proseguir con el proceso de carga hasta su finalización con independencia de que existan errores. Si no se hubiese producido ningún error el programa mostrará un mensaje indicando que la carga se ha realizado de manera satisfactoria.

NOTAS:

- 1.— Es recomendable que la tabla sobre la que se va a realizar la carga masiva de datos no tenga índices definidos (éstos podrán crearse una vez efectuada la carga).
- 2.— La carga masiva de datos desde un fichero ASCII puede realizarse también mediante un programa CTL que maneje STREAMS de entrada («INPUT»).

## Instrucción UNLOAD

Esta instrucción permite la descarga de información recogida en las tablas de la base de datos. Dicha información se almacena en un fichero ASCII (de texto). Su sintaxis es la siguiente:

**UNLOAD TO** "fichero\_ascii" instrucción\_select

La «instrucción\_select» ha de ser una instrucción SELECT válida, pudiendo incorporar variables «hosts», pero no receptoras.

Respecto a la ejecución de esta instrucción, conviene señalar que no es necesario que el fichero sobre el que se va a realizar la descarga haya sido creado previamente. Por el contrario, si dicho fichero ya existiese, éste se sobrescribiría al ejecutar la instrucción. Los únicos casos en los que podría producirse un error durante la ejecución de una instrucción UNLOAD serían:

- Utilización de una SELECT incorrecta.
- Falta de permisos de acceso al fichero (a nivel del sistema operativo).
- Falta de espacio en el sistema de archivos.

En caso de que la «instrucción\_select» no devuelva ninguna fila como tabla derivada, el fichero ASCII se creará con 0 bytes.

Aunque por antagonismo con el término carga se utiliza la palabra descarga para UNLOAD, esta instrucción NO supone el borrado de las filas que cumplan las condiciones de la instrucción SELECT, es decir, genera un fichero con esos datos pero las filas permanecen en la base de datos, mientras que la instrucción LOAD sí altera el contenido de la tabla (realmente «inserta» filas).

### Ejemplo [PUNLOAD1]:

```
database almacen

define
 variable tabla char(18)
end define

main begin
 forever begin
 window from select tablename from systables
 where tabid > 149
 15 lines label "T a b l a s"
```

```
 at 5,20
 set tabla from 1
 if cancelled = true then break
 tsql "unload to "" & tabla && ".unl" select * from " && tabla clipped
 if errno <> 0 then pause "ERROR de descarga. Pulse [RETURN] para continuar" reverse
 else pause "PROCESO CORRECTO. Pulse [RETURN] para continuar" reverse
end
end main
```

Este programa permite descargar toda la información de cualquiera de las tablas de la base de datos de demostración «almacen» en un fichero ASCII. El nombre de este fichero coincidirá con el de la tabla, pero con extensión «.unl». En primer lugar, se muestra una ventana en la que el operador podrá seleccionar uno de los nombres que aparecen en ella. La instrucción UNLOAD de descarga se construye de forma dinámica mediante la instrucción TSQL, ya que el nombre de la tabla y el del fichero ASCII pueden ser diferentes en cada iteración del bucle «FOREVER». En caso de producirse algún error de descarga aparecerá un mensaje indicándolo. En lugar de utilizar la instrucción TSQL para la construcción de la UNLOAD podría haberse empleado alternativamente las instrucciones PREPARE y EXECUTE.

NOTA: La descarga masiva de datos hacia un fichero ASCII puede efectuarse también mediante un programa CTL que maneje STREAMS de salida («OUTPUT»).

## SQL dinámico

---

Hasta ahora, todas las instrucciones del SQL que se han tratado desde CTL no han variado su sintaxis por el hecho de estar embebidas, aunque alguna de ellas se ha visto ampliada por necesidades de comunicación entre CTL y SQL. Pues bien, estas instrucciones del SQL pueden construirse de forma dinámica mediante el empleo de expresiones. Es decir, mediante concatenaciones de variables de programación CTL y constantes alfanuméricas se podrán construir estas instrucciones SQL para su posterior ejecución.

Por ejemplo, supongamos que un usuario desea un listado de clientes ordenado por el nombre de la empresa, o por el nombre de la persona de contacto, etc. En este supuesto, dichos listados se consiguen mediante una instrucción SELECT cuya cláusula «ORDER BY» deberá ser necesariamente distinta en cada caso. Pues bien, gracias a la construcción dinámica de instrucciones de SQL, todos estos listados con distintas ordenaciones se consiguen con la ejecución de un solo programa CTL en el que la instrucción de lectura SELECT será dinámica, dependiendo el orden elegido por el operador. Por ejemplo:

```
"select cliente, empresa, nombre, direccion1 from clientes order by " && variable_orden
```

Esta instrucción SELECT se ha construido mediante la concatenación («&&») de un literal («...») y una variable de programación CTL («variable\_orden»). Pues bien, siguiendo con nuestro supuesto, si el operador elige el orden del listado, la variable «variable\_orden» tomará como valor el nombre de la columna por la cual desea ordenar. Es decir, en un caso el valor de esta variable será «empresa» y en otro «nombre».

Las instrucciones CTL que manejan el SQL dinámico son TSQL, PREPARE y EXECUTE. La razón por la cual estas instrucciones permiten el dinamismo del SQL es porque admiten como parámetro una expresión.

En los siguientes apartados se comentan las características principales de cada una de estas instrucciones.

## Instrucción TSQL

La instrucción TSQL permite ejecutar cualquier instrucción, o grupo de instrucciones, del SQL, previamente almacenada(s) en un fichero cuya extensión tiene que ser «.sql», o bien una única instrucción cuya sintaxis sea el resultado de evaluar una expresión. Por lo general, esta expresión estará compuesta por la concatenación de constantes alfanuméricas y variables de programación CTL. La sintaxis de la instrucción TSQL es la siguiente:

**TSQL** [**FILE**] expresión [{**TO** | **THROUGH**} salida  
 [**APPEND**]] [[[líneas **LINES**] [**LABEL** {"literal" | número}]]  
 [**AT** línea, columna]] [**LOG**]

En caso de que la instrucción SQL a ejecutar sea una SELECT, las filas de la tabla derivada se mostrarán en pantalla sobre una ventana estándar CTL, siempre y cuando no se redireccione su salida («TO» o «THROUGH»). En instalaciones sobre Windows, y pese a compilar correctamente, la cláusula «THROUGH» no funcionará por las carencias de dichos entornos en lo referente a multiproceso. Sin embargo, las restantes instrucciones del SQL no producirán ninguna salida, realizando la operación oportuna en cada caso.

### Ejemplo1 [PTSQL1]:

```
database almacén

define
 variable fichero char(64)
 variable tabla char(18)
end define

main begin
 whenever error silent ignore
 forever begin
 path walk from "." extent ".unl"
 10 lines 2 columns
 label "F i c h e r o s A S C I I"
 at 5,7
 set fichero
 if cancelled = true then break
 let fichero = fichero & ".unl"
 window from select tablename tablas from systables
 where tabid > 149
 15 lines
 label "T a b l a s"
 at 2,30
 set tabla from 1
 if cancelled = true then break
 tsq "load from "" & fichero && "" insert into "" &&
 tabla clipped
 if errno <> 0 then pause "ERROR EN CARGA. Pulse [Return] para continuar." reverse bell
 else pause "CARGA CORRECTA. Pulse [Return] para continuar." reverse bell
 end
end main
```

Este programa muestra una ventana con todos los ficheros existentes en el directorio en curso cuya extensión sea «.unl». En caso de seleccionar uno de ellos, aparecerá a continuación otra ventana con los nombres de todas las tablas de la base de datos «almacén». Si se elige una de ellas se realizará la carga masiva de datos desde el fichero ASCII seleccionado hacia dicha tabla. En caso de que la carga produzca algún error aparecerá un mensaje indicándolo. El programa realizará constantemente estas operaciones hasta que el operador cancele la selección del fichero ASCII o de la tabla a cargar.

### Ejemplo2 [PTSQL2]:

```
define
 variable base char(18)
end define

main begin
 prompt for base label "Nombre de la base de datos a crear"
 end prompt
 tsq1 "create database " && base clipped
 if errno <> 0 then begin
 pause "ERROR. Pulse [Return] para continuar." bell
 exit program
 end
 tsq1 file "almacen"
end main
```

Para que este programa funcione es necesario tener un fichero («almacen.sql» de la base de datos de demostración) en el directorio en curso. Su ejecución consiste en crear una base de datos y ejecutar las instrucciones SQL incluidas en el fichero «almacen.sql». Este fichero se supone que contiene instrucciones de creación de tablas, integridad referencial e índices.

Para crear la base de datos podría haberse indicado igualmente la siguiente instrucción:

```
create database $base
```

## Instrucción PREPARE

Esta instrucción permite construir una instrucción de tipo SQL mediante la concatenación de expresiones para la comprobación de su sintaxis. Dicha instrucción preparada se podrá ejecutar mediante la instrucción EXECUTE, o bien, en el caso de tratarse de una SELECT, se podrá declarar un CURSOR. La sintaxis de la instrucción PREPARE es la siguiente:

**PREPARE** nombre\_prepare **FROM** expresión

CTSQL analiza la sintaxis de las instrucciones de tipo SQL embebidas en un programa CTL cada vez que se ejecutan. Gracias al empleo de la instrucción PREPARE, y dependiendo del proceso a realizar, dicha comprobación se efectuará una sola vez, siempre y cuando la instrucción no se encuentre dentro de un bucle.

Si la instrucción a preparar es una SELECT, ésta no podrá incluir cualquiera de las cláusulas que permiten la recepción de valores, es decir, «INTO» o «BY NAME».

En la sintaxis de la instrucción SQL preparada se tendrán que incluir signos de «cerrar interrogación» («?») en aquellos lugares donde irían las variables «hosts». Dichas variables serán pasadas posteriormente en la ejecución de la instrucción mediante la cláusula «USING» de la EXECUTE o de la «OPEN» o «FOREACH» si la instrucción preparada es una SELECT. Ahora bien, si una variable de programación CTL interviene en la construcción de la sintaxis de la instrucción SQL preparada, la instrucción PREPARE tendrá que situarse en un lugar del módulo donde dicha variable haya adquirido un valor. En caso contrario, la PREPARE podría situarse en un lugar del módulo donde no estuviese afectada por ningún bucle o proceso anterior.

En caso de preparar una instrucción SELECT se pueden realizar con ella las siguientes operaciones:

a) Ejecutarla mediante la instrucción EXECUTE y posteriormente comprobar las variables internas de CTL «found» y «ambiguous»:

```
prepare inst1 from "select cliente,empresa from clientes where " && condiciones
execute inst1
if found = true then ...
if ambiguous = true then ...
```



En este caso, la variable de programación «condiciones» tendrá que incluir aquellas condiciones válidas SQL sobre las columnas de la tabla «clientes».

b) Declararla como un CURSOR y tratar dicha tabla derivada mediante las instrucciones propias de los cursores (OPEN, FETCH, CLOSE y FOREACH). Asimismo, para mostrar la tabla derivada devuelta por la ejecución de la SELECT en una ventana se puede emplear la instrucción «WINDOW FROM CURSOR». El resultado de este procedimiento será similar a la ejecución de la SELECT mediante la instrucción TSQL, pero con la posibilidad de poder elegir valores en la ventana gracias al empleo de la WINDOW, cosa que no ocurre cuando se ejecuta únicamente con la instrucción TSQL. Para más información acerca de los «cursores» [consulte el siguiente capítulo](#) de este mismo volumen.

```
prepare inst1 from "select cliente,empresa from clientes where " && condiciones
declare cursor consulta for inst1
window from cursor consulta
 10 lines label "C o n s u l t a"
 at 5, 10
 set empre from 2
```

En este caso, la variable «condiciones» tendría que incluir como valor cualquier condición válida SQL sobre las columnas de la tabla «clientes». Asimismo, en la variable de programación «empre» se obtendrá el valor elegido de la ventana producida por la instrucción WINDOW con las filas de la tabla derivada de la instrucción SELECT dinámica.

#### Ejemplo1 [PPREPAR1]:

```
database almacén

define
 variable provincia like provincias.provincia
 variable descripcion like provincias.descripcion upshift
 variable maximo like provincias.provincia
end define

main begin
 prepare insercion from "insert into provincias (provincia, " &&
 " descripcion) values (?,?)"
 forever begin
 prompt for descripcion label "Descripcion Provincia"
 end prompt
 if cancelled = true then break
 select max(provincia) maximo by name from provincias
 if maximo is null then let maximo = 0
 let provincia = maximo + 1
 execute insercion using provincia, descripcion
 if errno <> 0 then pause "ERROR en inserción. Pulse [Return] para continuar" reverse bell
 let descripcion = NULL
 end
 window from select provincia, descripcion, prefijo
 from provincias
 15 lines label "P r o v i n c i a s"
 at 3,20
end main
```

Este programa inserta filas en la tabla «provincias». Las columnas que toman valores son «provincia» y «descripcion», quedando en este caso el «prefijo» a «NULL» en cada fila insertada. La instrucción INSERT se prepara incluyendo interrogaciones en el lugar donde posteriormente en ejecución irán las variables «hosts». El código de la provincia se calcula en el módulo simulando el tipo de dato SERIAL, es decir, se lee el último y a este valor se suma uno. Sin embargo, la descripción de la provincia se le pide al operador mediante la instrucción PROMPT FOR. En caso de producirse algún error en la inserción de estas filas aparecerá un mensaje avisándolo.

Cuando el operador cancele la inserción de valores aparecerá una ventana con todas las filas de la tabla «provincias».

Si bien en este programa no sería necesario emplear las instrucciones PREPARE y EXECUTE, ya que se podría incluir directamente la instrucción INSERT con las variables «hosts» a insertar, este procedimiento implicaría que cada vez que se insertase una nueva fila sobre la tabla se comprobase la sintaxis de la instrucción. Para evitar esto es por lo que se utilizan las instrucciones PREPARE y EXECUTE, ya que de esta forma, si estuviésemos en un bucle de inserción de filas en el que no existiese interacción con el operador, resultaría el procedimiento más rápido.

### Ejemplo2 [PPREPAR2]:

```
database almacen

define
 variable condiciones char(200)
 frame provin like provincias.*
end frame

end define

main begin
 forever begin
 input frame provin for query by name to condiciones
 end input
 if cancelled = true then break
 if condiciones is null then let condiciones = "provincia > 0"
 prepare ins1 from "select provincia, descripcion " &&
 "from provincias where " && condiciones clipped
 declare cursor consulta for ins1
 window from cursor consulta
 10 lines label "CONSULTA"
 at 5,10
 end
 end
end main
```

Aunque la estructura FRAME no se ha comentado hasta este momento, se expone este ejemplo porque su utilización es muy habitual. Este programa pide condiciones sobre las variables de un FRAME. Un FRAME es una estructura compuesta de un conjunto de variables, pudiendo ser de distinto tipo de dato SQL. En este programa, el FRAME definido adquiere la misma estructura que la tabla «provincias» en la base de datos:

```
provincia smallint
descripcion char(20)
prefijo smallint
```

El resultado de la consulta sobre las variables del FRAME se recoge en una variable de programación CTL («condiciones»). Esta variable puede adquirir el valor «NULL» si el operador no introduce ninguna condición válida («query») sobre las variables. Sin embargo, si el operador introduce alguna condición sobre cualquiera de las variables de dicho FRAME, el resultado será una concatenación de condiciones mediante los operadores «AND» y «OR». Estas condiciones se recogen en la variable de programación «condiciones». Una vez que esta variable recoge los posibles valores se prepara la instrucción de lectura cuyas condiciones son totalmente dinámicas. Mediante la declaración de un CURSOR con dicha instrucción SELECT preparada y la ejecución de una WINDOW se muestran las filas de la tabla «provincias» que cumplen las condiciones indicadas.

## Instrucción EXECUTE

Esta instrucción ejecuta otra instrucción previamente preparada en una PREPARE. Su sintaxis es la siguiente:

**EXECUTE** nombre\_instrucción\_prepare [**USING** lista\_variables]

La cláusula «USING» será obligatoria en el caso de haber indicado algún signo de interrogación («?») identificando las futuras variables «hosts» en la sintaxis de la PREPARE.

En caso de ejecutar una SELECT preparada previamente, al ejecutar mediante EXECUTE se pueden comprobar los valores de las variables internas de CTL «found» y «ambiguous». Ahora bien, si lo que se desea es recoger sobre variables de programación la información devuelta por dicha SELECT preparada, obligatoriamente habrá que definir un CURSOR y recoger dichos valores en la instrucción FETCH o en la FOREACH (propias del CURSOR). En este caso no sería necesario ejecutar la instrucción EXECUTE.

#### Ejemplo [PEXECUT1]:

```
database almacén

define
 variable sino char(2)
 variable tabla char(18)
end define

main begin
 forever begin
 window from select tabname from systables
 where tabid > 149
 and tabtype = "T"
 10 lines label "T a b l a s"
 at 5,25
 set tabla from 1
 if cancelled = true then break
 prepare lectura from "select 5 from " && tabla clipped
 prepare borrado from "drop table " && tabla clipped
 execute lectura
 if found = true then let sino = "SI"
 else let sino = "NO"
 if yes(sino && " contiene filas. Desea borrarla", "N") = true then execute borrado
 end
end main
```

Este programa muestra todas las tablas de la base de datos de demostración «almacén» para su posible borrado. No obstante, antes de llevar a cabo esta operación se realiza una lectura sobre la tabla elegida para comprobar si existen o no filas en ella. Tanto en un caso como en otro se informa al operador antes de ejecutar el borrado. El programa realizará constantemente esta operación hasta que el operador cancele la elección de tablas. El borrado de la tabla elegida también se realiza mediante las instrucciones dinámicas, ya que en cada vuelta del bucle «FOREVER» el nombre de la tabla a borrar puede ser diferente.

## Tablas temporales

Una tabla temporal es una tabla que no pertenece a ninguna base de datos y que es eliminada automáticamente al finalizar el programa que la creó. Este tipo de tablas pueden emplearse cuando el programa requiera el manejo de ARRAYS de dos dimensiones. Sin embargo, el manejo de los datos sobre estas tablas es idéntico al de cualquier otra tabla de una base de datos, es decir, tanto la inserción, el borrado, la actualización o la lectura se tienen que realizar mediante instrucciones propias del SQL (INSERT, DELETE, UPDATE y SELECT, respectivamente).

CTSQL dispone de dos tipos de tablas temporales:

- Tabla temporal creada con la cláusula «INTO TEMP» de la instrucción SELECT.
- Tabla temporal creada con la instrucción CREATE TEMP TABLE.

En ambos casos, estas tablas temporales, al igual que las tablas de una base de datos, están representadas por dos ficheros, cuyas extensiones son «.dat» e «.idx», siendo su nombre dinámico asignado automáticamente por el SQL. Estos dos ficheros se encuentran ubicados en el directorio indicado por la variable de entorno DBTEMP.

Una tabla temporal es accesible a todos los programas llamados por el que la creó. Una de sus principales utilidades radica en el traspaso de información entre distintas bases de datos, ya que no pertenecen a ninguna en concreto. Es decir:

```
database primera
select ... from tabla
into temp uno
database segunda
insert into tabla (columna1, columna2) select ... from uno
```

o bien,

```
database primera
create temp table uno (columna1 smallint, columna2 char(10))
insert into uno (columna1, columna2) select ... from tabla
database segunda
insert into tabla (columna1, columna2) select ... from uno
```

Las particularidades de cada uno de estos tipo de tablas temporales se comentan en los siguientes epígrafes de este capítulo.

### Tabla temporal creada con la cláusula «INTO TEMP»

Las características principales de una tabla temporal creada con la cláusula «INTO TEMP» son las siguientes:

- 1.— La información con la que se crea una tabla temporal mediante la cláusula «INTO TEMP» es la tabla derivada devuelta por la instrucción SELECT. Para insertar más filas, borrar, actualizar o leer dichas filas hay que emplear respectivamente las siguientes instrucciones del SQL: INSERT, DELETE, UPDATE y SELECT.
- 2.— Este tipo de tabla temporal no puede mantenerse en un objeto FORM.
- 3.— Los nombres de las columnas de la tabla temporal son los que aparecen en la «lista\_selección» de la instrucción SELECT. En caso de utilizar «alias» para una columna o expresión, el nombre de columna de la tabla temporal será el nombre de dicho alias.
- 4.— Una tabla temporal creada con «INTO TEMP» no permite la creación de índices (CREATE INDEX) ni de claves primarias ni referenciales.
- 5.— La cláusula «INTO TEMP» es incompatible con la «ORDER BY».
- 6.— El nombre de la tabla temporal debe ser único en un proceso. En caso de existir otra tabla con el mismo nombre, tanto si es temporal como si no, se producirá un error a la hora de crearla. En caso de incluir la instrucción SELECT con la cláusula «INTO TEMP» en un bucle habrá que producir el borrado físico de la tabla temporal en cada iteración mediante la instrucción DROP TABLE.

#### Ejemplo [PINTOTE1]:

```
database almacen

define
 variable tanto decimal(4,2)
end define

main begin
 select articulo, descripcion, pr_vent from articulos
 into temp incremento
```

```

prompt for tanto label "Tanto por ciento de incremento a todos los articulos"
end prompt
update incremento set pr_vent = pr_vent * (1 + $tanto / 100)
window from select * from incremento
 15 lines label "R e s u l t a d o I n c r e m e n t o"
 at 3,20
end main

```

Este programa realiza una actualización de todas las filas de la tabla «articulos» sobre una tabla temporal que contiene todas las filas de aquella. En primer lugar, el programa crea la tabla temporal «incremento» mediante la cláusula «INTO TEMP» de la instrucción SELECT. La tabla temporal creada tiene tres columnas, «articulo», «descripcion» y «pr\_vent», cuyos tipos coinciden con los de las columnas en la tabla original («articulos»). A continuación, el programa pide al operador el tanto por ciento en que desea incrementar los precios de todos los artículos (este porcentaje podrá indicarse con decimales). Una vez aceptado el porcentaje se actualizan todas las filas de la tabla temporal aplicando la subida de dicho tanto por ciento. Por último, se muestra una ventana con la información de toda la tabla temporal recién actualizada. Dicha actualización se ha realizado sobre una tabla que se eliminará cuando finalice la ejecución del programa. Por tanto, el único objetivo de esta actualización es comprobar cómo pueden quedar los precios de los artículos en caso de aplicar un determinado tanto por ciento.

## Tabla temporal creada con la instrucción CREATE TEMP TABLE

Las características principales de una tabla temporal creada con la instrucción CREATE TEMP TABLE son las siguientes:

- 1.— La tabla temporal creada con esta instrucción se genera con cero filas. Para insertar filas en ella hay que emplear la instrucción INSERT del SQL.
- 2.— Este tipo de tabla temporal Sí se puede mantener en un objeto FORM. No obstante, la operación que hay que realizar es la que se expone a continuación, ya que el compilador **ctlcomp** produce un error al no pertenecer dicha tabla a la base de datos:
  - a) Crear una tabla con la misma estructura que la tabla temporal, pero en diccionario. Con esta operación, las tablas del catálogo de la base de datos (tablas «sys\*») se han actualizado con dicha información.
  - b) Compilar el módulo CTL en el que se realizará el mantenimiento de esta tabla temporal mediante el objeto FORM del CTL.
  - c) En caso de no producirse ningún error de compilación, borrar la tabla creada en el paso «a)», con lo que, a partir de estos momentos, el objeto FORM mantendrá datos sobre una tabla temporal creada, posiblemente, en su mismo módulo.
- 3.— A la tabla temporal creada mediante CREATE TEMP TABLE se le pueden asignar atributos del CTL y del CTSQL, así como definirle índices, claves primarias y claves referenciales.
- 4.— Un programa CTL lanzado desde otro que crea una tabla temporal puede crear a su vez una tabla homónima que no afectará ni a su esquema ni a sus datos, recuperándose la primera tabla del mismo nombre al regresar al programa que origina la llamada (anidamiento).

### Ejemplo [PTEMP1]:

```

database almacén

define
 variable tanto decimal(4,2) required
end define

main begin
 forever begin

```

```
create temp table incremento (articulo smallint,
 descripcion char(15),
 pr_vent decimal(9,2))
insert into incremento (articulo, descripcion, pr_vent)
 select articulo, descripcion, pr_vent from articulos
prompt for tanto label "Tanto por ciento de incremento a todos los articulos"
end prompt
if cancelled = true then break
update incremento set pr_vent = pr_vent * (1 + $tanto / 100)
call sqlreset()
window from select * from incremento
 15 lines label "R e s u l t a d o I n c r e m e n t o"
 at 3,20
let tanto = null
end
end main
```

La ejecución de este programa es similar al del ejemplo anterior («pintote1»), pero utilizando la instrucción CREATE TEMP. La diferencia fundamental se encuentra en la creación de la tabla temporal y en su carga de datos. En el ejemplo anterior, cláusula «INTO TEMP», la creación y la carga de datos se realizaba con la ejecución de una instrucción SELECT. En este programa se tiene que crear la tabla temporal con la instrucción CREATE TEMP TABLE y posteriormente cargarla mediante la instrucción INSERT. La actualización del precio de los artículos se realiza sobre esta tabla temporal. En este caso, la operación de actualización se ha incluido dentro de un bucle «FOREVER» para realizarla un número indeterminado de veces. Por esta razón hay que emplear la función interna de CTL «sqlreset()» para que se cierre el CURSOR interno generado por la instrucción WINDOW y que se vuelva a generar en cada iteración. En caso de no ejecutar esta función, la instrucción WINDOW siempre mostrará las filas de la primera de las tablas temporales.

## «Views» actualizables

Una VIEW (vista) es un elemento CTSQL que nos permite acceder a datos de la base de datos presentándolos con estructura de tabla. En realidad, una VIEW es una instrucción SELECT que queda almacenada en el catálogo de la base de datos (tablas «sys\*») y a la que se puede acceder mediante otras instrucciones SELECT, lo que le confiere el tratamiento de tabla (con la salvedad de que, al no existir como tal, no tiene índices ni permisos de acceso, y su actualización es posible bajo determinadas circunstancias). Su sintaxis es la siguiente:

```
CREATE VIEW nombre_view [(lista_columnas_view)]
AS instrucción_select [WITH CHECK OPTION]
```

Por lo tanto, la VIEW maneja la tabla derivada devuelta por la instrucción SELECT que la crea. Las cláusulas «ORDER BY» y «UNION» de la SELECT son incompatibles con la creación de la VIEW. Esto también sucedía en el caso de la creación de tablas temporales con la cláusula «INTO TEMP». La cláusula «WITH CHECK OPTION» indica que todas las modificaciones realizadas a través de la VIEW deberán cumplir la condición «WHERE» de la instrucción SELECT.

Una VIEW permanece en la base de datos hasta que no se produce su borrado mediante la ejecución de la instrucción DROP VIEW del SQL. Asimismo, una VIEW no puede alterar su estructura, ya que no tiene una estructura física en la base de datos y su esquema viene definido por la «lista\_selección» de la instrucción SELECT asociada. La única forma de alterar su estructura es borrándola y volviéndola a crear con las modificaciones necesarias.

Una VIEW puede estar compuesta por una o varias tablas (o VIEWS) e incluir todas o sólo algunas de las columnas de éstas. Los nombres de estas columnas se pueden indicar en la creación de la VIEW. Ahora bien, en caso de no especificarse dichos nombres, las columnas de la VIEW tomarán los mismos nombres que los de las columnas indicadas en la «lista\_selección» de la instrucción SELECT. Si alguna de estas columnas de la «lis-

ta\_selección» es una expresión, o existen dos columnas con el mismo nombre, se deberá incluir la lista de columnas en la definición («lista\_columnas\_view»). En este caso el uso de alias de columnas no soluciona el problema.

En conclusión, una VIEW es una tabla más de la base de datos, pero sin ficheros físicos que la representen («.dat» e «.idx»).

Las VIEWS tienen fundamentalmente las siguientes utilidades:

- 1.— Presentar los datos de las tablas en una estructura más próxima al concepto que posee el usuario de una aplicación sobre la forma en que se estructura dichos datos. Esto es debido a que, por motivos de normalización de tablas, se llegue a una fragmentación que suponga constantes operaciones de «JOIN» a la hora de recuperar dichas tablas.
- 2.— Una VIEW puede ser un medio de nominar instrucciones SELECT cuya estructura sea de uso muy frecuente.
- 3.— Delimitar el acceso a los datos en conjuntos preestablecidos, con lo que se puede establecer una privacidad lógica sobre los mismos. Hay que considerar aquí que como una tabla (la resultante de la VIEW) consta de columnas y de filas, esta delimitación de acceso se puede aplicar en uno o ambos sentidos. Sobre columnas significa no acceder a determinados datos de cada tabla (por ejemplo, ver el precio de venta de un artículo, pero no el de coste). Esto mismo, aplicado a las filas, significa acceder a un número de filas que cumplan cierta condición (por ejemplo, cada departamento sólo puede consultar sus filas).

#### IMPORTANTE

Una VIEW carece de «ROWID» y por tanto no puede mantenerse en el objeto FORM.

Como se ha indicado anteriormente, las VIEWS pueden ser actualizables. Esto significa que mediante el manejo de esta VIEW, indirectamente en la(s) tabla(s) que intervinieron en su creación se pueden insertar nuevas filas, actualizar las ya existentes o borrarlas. Esto supone que cualquiera de estas operaciones se descompondrá en las correspondientes sobre las tablas base y no sobre la VIEW, que es el resultado de la SELECT asociada a ella.

Para que una VIEW sea actualizable tiene que cumplir las siguientes condiciones:

- La «lista\_selección» de la instrucción SELECT que la crea no debe incluir «expresiones» ni valores agregados [«count()», «max()», «min()», «avg()», «sum()»].
- La instrucción SELECT asociada a la VIEW no debe incluir las cláusulas «DISTINCT» o «UNIQUE» ni la «GROUP BY», ni por consiguiente tampoco la «HAVING».
- Aquellas columnas de las tablas sobre las que se realiza la lectura para la creación de la VIEW que tengan asignado el atributo de CTSQL «NOT NULL» tienen que incluirse en la «lista\_selección» de la instrucción SELECT.
- Asimismo, la «lista\_selección» de la instrucción SELECT debe incluir las columnas que componen la clave primaria de cada tabla o las columnas que identifiquen unívocamente la fila en cada tabla.
- Se debe tener permiso de SELECT y UPDATE de todas aquellas columnas que se incluyan en la «lista\_selección» de la instrucción SELECT asociada.

#### Ejemplo1 [PVIEW2]:

```
database almacén

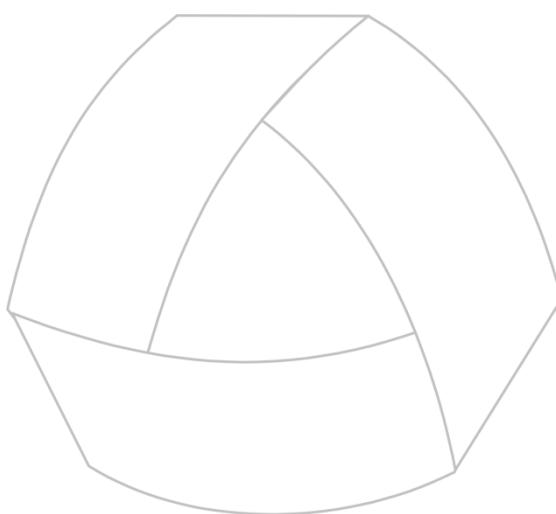
define
 variable cliente like clientes.cliente
 variable direccion1 like clientes.direccion1
end define

main begin
 select "" from systables
 where tablename = "clien_prov"
```

```
if found = false then
 create view clien_prov as select cliente, empresa, direccion1,
 provincias.provincia, descripcion
 from clientes, provincias
 where clientes.provincia = provincias.provincia
 forever begin
 window scroll from select * from clien_prov
 15 lines label "F i l a s d e l a V i e w"
 at 3,1
 set cliente, direccion1 from 1,3
 if cancelled = true then break
 prompt for direccion1 label "Nueva dirección"
 end prompt
 update clien_prov set direccion1 = $direccion1
 where clien_prov.cliente = $cliente
 window from select cliente, empresa, direccion1
 from clientes
 where clientes.cliente = $cliente
 5 lines label "T a b l a O r i g i n a l"
 at 10,4
 end
 end main
```

Este programa emplea una VIEW actualizable para presentar los clientes de la base de datos junto con la provincia a la que pertenecen. En primer lugar, lee de la tabla del catálogo de la base de datos «systables» para comprobar si la VIEW «clien\_prov» ha sido creada previamente o no. En caso negativo la crea y directamente muestra todas sus filas en una ventana (WINDOW). En caso de que el operador elija algún cliente de esta ventana será para modificar su dirección. La dirección nueva se solicita mediante la ejecución de la instrucción PROMPT FOR. Posteriormente, con este dato se actualiza la fila seleccionada sobre la VIEW, aunque dicha actualización indirectamente se realiza en la tabla base «clientes». Por último, para comprobar que la actualización se ha realizado en esta tabla base («clientes») se muestra la fila recién afectada.





MultiBase  
MANUAL DEL PROGRAMADOR

## CAPÍTULO 10. El objeto CURSOR

## EL CURSOR

Un CURSOR es una tabla virtual asociada al módulo que lo define, cuyo contenido será el resultado de la instrucción SELECT utilizada en su definición. La definición de un CURSOR será necesaria cuando una instrucción SELECT de la que se desea extraer información devuelva como tabla derivada más de una fila. Sin embargo, en el caso de que se desee conocer únicamente si la SELECT va a devolver o no una o varias filas no será necesario declarar el CURSOR (en este caso habrá que emplear las variables internas de CTL «found» y «ambiguous»).

Dependiendo de su definición, el CURSOR se considerará como CURSOR de consulta o de actualización:

- Los «cursores» de consulta son aquellos que se utilizarán para mostrar las filas que componen su tabla derivada en pantalla (WINDOW), o bien para realizar un listado hacia un determinado dispositivo (pantalla, impresora, fichero, etc.), o también para realizar una simple consulta, recogiendo sus datos en variables para procesar dicha información. La declaración de estos «cursores» se compone únicamente de la SELECT asociada. Ya que todos los listados implican la declaración de un CURSOR y el redireccionamiento de datos mediante un STREAM, [consulte también el capítulo correspondiente a este objeto](#) dentro de este mismo volumen.
- Por su parte, los «cursores» de actualización se emplean para actualizar o borrar las filas de la tabla sobre la que se realiza la lectura en la instrucción SELECT asociada a dicho CURSOR.

Un CURSOR se declara mediante la directiva DECLARE. Esta directiva permite declarar «cursores» de una instrucción SELECT estática o dinámica (preparada mediante la instrucción PREPARE). La forma de tratar las filas de la tabla derivada de un CURSOR es mediante las siguientes instrucciones específicas: OPEN, FETCH, CLOSE y FOREACH. Todas estas instrucciones se comentan en este mismo capítulo.

## 10.2. Declaración de un CURSOR: Directiva DECLARE

La sintaxis de la directiva que se encarga de declarar un CURSOR de forma dinámica o estática es la siguiente:

```
DECLARE CURSOR nombre_cursor FOR
 {instrucción_select [FOR UPDATE [OF lista_columnas] [NOWAIT]] |
 nombre_prepare}
[CONTROL
 [{BEFORE | AFTER} GROUP OF {columna_order | número}
 ...
 ...]
 [ON EVERY ROW
 ...
 ...]
 [ON LAST ROW
 ...
 ...]
END CONTROL]
```

Esta directiva puede ubicarse en cualquier punto de un módulo donde se permita la programación de procesos. Esta instrucción no ejecuta absolutamente nada, simplemente define la instrucción SELECT declarada como CURSOR.

### Ejemplo1 [PCURSOR1]:

```
database almacen

main begin
 declare cursor consulta for select empresa, descripcion
 from clientes, provincias
 where clientes.provincia = provincias.provincia
 order by empresa
 window from cursor consulta
```

```

 15 lines label "C l i e n t e s"
 at 3,20
 end main

```

En este programa se declara un CURSOR compuesto por una tabla derivada que relaciona los clientes con su respectiva provincia. Dicha tabla derivada se ordenará por el nombre de la empresa. Posteriormente, todas las filas de la tabla derivada del CURSOR se muestran en una ventana producida por la instrucción WINDOW. El resultado de este programa sería igual si la SELECT de la declaración del CURSOR se incluyese en la instrucción WINDOW:

```

window from select empresa, descripcion
 from clientes, provincias
 where clientes.provincia=provincias.provincia
 order by empresa
 15 lines label "C l i e n t e s"
 at 3,20

```

Como se puede comprobar en la sintaxis de la directiva DECLARE, es posible declarar «cursores» de una SELECT preparada en una instrucción PREPARE. Este tipo de declaración se denomina dinámica, ya que la sintaxis de la SELECT se ha podido construir de forma dinámica mediante una instrucción PREPARE.

#### Ejemplo2 [PCURSOR2]:

```

database almacén

define
 variable columna_orden char(18)
end define

main begin
 forever begin
 window from select colname from syscolumns, systables
 where systables.tabname = "clientes"
 and systables.tabid = syscolumns.tabid
 10 lines label "C o l u m n a s"
 at 2,20
 set columna_orden from 1
 if cancelled = true then break
 prepare inst1 from "select * from clientes order by " && columna_orden clipped
 declare cursor consulta for inst1
 window scroll from cursor consulta
 15 lines label "C l i e n t e s"
 at 3,1
 end
 end
end main

```

Este programa muestra una ventana con todas las columnas de la tabla «clientes». Cuando el operador pulse [RETURN] o la tecla asignada a la acción <faccept> se asignará el nombre de la columna en la que estuviese situado el CURSOR de la ventana a la variable de programación «columna\_orden». Esta columna será la que fije el orden de selección de la instrucción SELECT declarada como CURSOR. En este caso, la SELECT se prepara sobre una instrucción PREPARE, ya que la cláusula «ORDER BY», como se ha indicado anteriormente, puede ser aleatoria en cada iteración del bucle (dinámica). Posteriormente, la SELECT preparada se declara como un CURSOR y, por último, éste se muestra en una ventana (WINDOW) con «scroll» lateral. Este proceso se repetirá un número indeterminado de veces hasta que el operador pulse la tecla asignada a la acción <fquit> en la ventana de selección de la columna de ordenación.

La recepción de los valores devueltos por la ejecución de la instrucción SELECT se puede realizar mediante las cláusulas «INTO», «BY NAME» o «BY NAME ON frame» sobre la propia instrucción SELECT, o bien en la lectura del CURSOR mediante las instrucciones FETCH o FOREACH (cláusula «INTO»). En caso de definir la recepción de los valores en la declaración del CURSOR y en su lectura, predomina la de la lectura (FETCH o FOREACH). Asi-

mismo, existe la posibilidad de declarar «cursores», estáticos o dinámicos, en los que las variables «hosts» que intervengan en él no se indicarán hasta el momento de su apertura. Este momento será cuando se ejecute la instrucción SELECT asociada al CURSOR (instrucciones OPEN o FOREACH con cláusula «USING»). La forma de identificar estas variables «hosts» en la sintaxis de la instrucción DECLARE es mediante el signo de interrogación («?») en el lugar donde deberán ubicarse en su apertura.

### Ejemplo 3 [PCURSOR3]:

```
database almacen

define
 variable cliente like clientes.cliente
 variable empresa like clientes.empresa
 variable i smallint default 3
end define

main begin
 declare cursor consulta for select cliente, empresa by name
 from clientes
 where cliente >= ?
 message "Cliente a partir del que se desea consultar"
 prompt for cliente label "Código de cliente"
 end prompt
 display frame box at 1,1 with 19,79 label "C l i e n t e s"
 display "Código" at 2,5, "E m p r e s a" at 2,40
 display "_____" at 3,5, "_____" at 3,40
 foreach consulta using cliente begin
 let i = i + 1
 display cliente + 0 using "####" at i,5,
 empresa & null clipped at i,30
 view
 if i = 18 then begin
 pause "Pulsar [Return] para continuar."
 clear list at 4,3 with 18,70
 let i = 3
 end
 end
 pause "Fin de Listado. Pulse [Return] para continuar"
 clear
 end main
```

Este programa realiza una presentación en pantalla de la información devuelta por el CURSOR. Si bien podría considerarse como un listado, ésta no es la forma más idónea de realizar listados hacia pantalla (éstos por lo general se realizan mediante la utilización de STREAMS).

En primer lugar, se declara el CURSOR con la instrucción SELECT, en la cual no se ha indicado la variable «host» que condicionará el código del cliente. A continuación, el programa solicita el código del cliente a partir del cual se desea realizar la consulta. Dicha variable intervendrá en la apertura del CURSOR cuando se ejecute la instrucción FOREACH y será la que sustituya al signo de interrogación empleado en la declaración del CURSOR. La instrucción FOREACH leerá cada una de las filas del CURSOR, presentándose éstas mediante la instrucción DISPLAY, en la que el parámetro de la línea es dinámico («i»).

En este ejemplo, como las variables receptoras «cliente» y «empresa» se han definido con la cláusula «LIKE», al mostrarlas con la instrucción DISPLAY aparecerá también su etiqueta, ya que la columna empleada en su definición («LIKE») la tiene asignada. Para evitar esta salida del atributo «LABEL» hay que convertir dichas variables en expresiones. En nuestro caso, a la columna numérica se le suma «0» y a la columna «CHAR» se le concatena un «NULL». Mediante el contador de líneas («i») se controla que al mostrar la línea 18 se limpie toda la pantalla, inicializándose la variable para que el listado vuelva a mostrarse desde la línea 4 de la pantalla.

Un CURSOR de actualización debe declararse con una instrucción SELECT que solamente lea de una tabla de la base de datos, debiendo ir acompañada, si es posible, de la cláusula «FOR UPDATE», que se encarga del bloqueo instantáneo y automático de cada una de las filas del CURSOR en el momento de su lectura (instrucciones FETCH y FOREACH), desbloqueándola cuando se lea la siguiente fila. Esta forma de bloquear filas se produce cuando la base de datos no es transaccional. Si la base de datos es transaccional, al controlar una transacción con «BEGIN WORK», «COMMIT WORK» o «ROLLBACK WORK», todas las filas devueltas por la instrucción SELECT del CURSOR quedarán automáticamente bloqueadas. Ésta es la única forma de bloquear más de una fila de una tabla.

La cláusula «FOR UPDATE» es incompatible con la «ORDER BY» de la instrucción SELECT. En caso de emplear «FOR UPDATE» la actualización (UPDATE o DELETE) de las filas devueltas por el CURSOR podrá realizarse con la cláusula «WHERE CURRENT OF nombre\_cursor», que garantiza la actualización de una sola fila (la fila en curso) y su acceso sin optimización. Sin embargo, si la cláusula «FOR UPDATE» no puede especificarse en la declaración del CURSOR, es aconsejable la lectura del «ROWID» de cada fila, debiendo emplearse la condición por «ROWID» cuando se realice la actualización o borrado de las filas. Por ejemplo:

**DELETE FROM TABLA WHERE ROWID = variable**

En este caso, «variable» es una variable que contendrá el «ROWID» leído como una columna más en la instrucción SELECT del CURSOR.

#### Ejemplo 4 [PCURSOR4]:

```
database almacén

define
 frame arti like articulos.*
end frame
end define

main begin
 declare cursor actualiza for select *
 by name on arti
 from articulos
 for update
 foreach actualiza begin
 input frame arti for update variables pr_vent
 end input
 if cancelled = false then
 update articulos set pr_vent = $arti.pr_vent
 where current of actualiza
 end
 end
end main
```

Aunque todavía no hemos tratado el objeto FRAME, en este ejemplo hemos utilizado una de sus características. Se trata de un conjunto de variables similar a la composición de una fila en una tabla de la base de datos. En este programa, su función es recoger la información devuelta por el CURSOR y permitir posteriormente modificar el precio de venta («pr\_vent») de cada artículo, siendo ésta la única variable a actualizar. Esta operación se realiza mediante la instrucción INPUT FRAME con la cláusula «FOR UPDATE» sobre la variable «pr\_vent». En caso de aceptar la modificación de cada precio de artículo (mediante la tecla asignada a la acción <facept>) se realizará la modificación de dicho precio en la tabla «articulos». Hay que tener en cuenta que el CURSOR ha sido definido con la cláusula «FOR UPDATE», por lo cual, cuando se lee un artículo en la instrucción FOREACH, éste queda automáticamente bloqueado hasta que se vuelva a leer el siguiente artículo. Asimismo, a la hora de actualizar cada fila se emplea la instrucción UPDATE, pero con la cláusula «WHERE CURRENT OF actualiza», asegurándonos de esta forma la modificación de una sola fila. Esta forma de acceso es más rápida que si se tuviese que localizar la fila mediante optimizador. En nuestro ejemplo, esta instrucción quedaría de la siguiente forma:

```
update articulos set pr_vent = $arti.pr_vent
where articulos.articulo = $arti.articulo
```

### Ejemplo 5 [PCURSORS5]:

```
database almacen

define
 variable num_fila integer
 variable pr_vent like articulos.pr_vent
 variable articulo like articulos.articulo
 variable descripcion like articulos.descripcion
end define

main begin
 declare cursor actualiza for select rowid num_fila, articulo,
 descripcion, pr_vent
 by name
 from articulos
 order by descripcion
 display "Artículo" at 12, 2, "Descripcion" at 12,20,
 "Precio Venta actual" at 12, 60
 display "—————" at 13, 2, "—————" at 13,20,
 "—————" at 13, 60
 foreach actualiza begin
 display articulo + 0 using "#####" at 15, 2,
 descripcion & null at 15,20,
 pr_vent + 0 using "###,###,###.##" at 15, 60
 prompt for pr_vent label "Nuevo precio de venta"
 end prompt
 if cancelled = false then
 update articulos set pr_vent = $pr_vent
 where rowid = $num_fila
 clear list at 15,1 with 1,65
 end
end main
```

Al igual que el ejemplo anterior («pcursor4») este programa actualiza el precio de cada uno de los artículos de la tabla «articulos». La diferencia entre ambos programas es la actualización ordenada de este último. Como ya se ha indicado anteriormente, cuando la instrucción SELECT incluye la cláusula «ORDER BY» la «FOR UPDATE» es incompatible. Por lo tanto, en este caso se selecciona el «ROWID» de cada fila elegida en la instrucción SELECT para condicionarlo posteriormente en la actualización (UPDATE). La recepción de valores en este programa se realiza sobre variables y no sobre un objeto FRAME. Por lo tanto, estas variables se muestran mediante la instrucción DISPLAY, y para pedir el nuevo precio de venta se emplea la instrucción PROMPT FOR. Si se cambia el precio de venta se actualiza la tabla «articulos» empleando como condición el «ROWID».

## Sección CONTROL del CURSOR

La sección CONTROL es opcional y en ella se especifican instrucciones no procedurales cuya ejecución se producirá cuando suceda la acción que representan. Estas instrucciones son las que se muestran en la siguiente sintaxis:

```
DECLARE CURSOR ...
[CONTROL
 [{BEFORE | AFTER} GROUP OF {columna_order | número}
 ...]
 [ON EVERY ROW
 ...]
 [ON LAST ROW
 ...]
END CONTROL]
```

Normalmente, estas instrucciones no procedurales se emplean para la realización de listados, aunque también pueden utilizarse para los «cursores» de actualización. Por ejemplo, a partir del ejemplo 3 de la sección anterior, en la directiva DECLARE podrían resolverse introduciendo todas las instrucciones afectadas por el bucle FOREACH en la cláusula «CONTROL» con la instrucción no procedural «ON EVERY ROW».

La situación de estas instrucciones no procedurales dentro de la sección CONTROL es indiferente, es decir, no tienen por qué seguir un orden concreto entre ellas. Cuando suceda la acción o se produzca la situación en la ejecución del programa indicada por una instrucción no procedural entonces se ejecutarán las instrucciones que la siguen («...»). En nuestro caso, el proceso afectado por una instrucción no procedural comienza en dicha instrucción no procedural y acaba en la siguiente instrucción no procedural, o bien en el final de la sección CONTROL.

Las instrucciones no procedurales incluidas en la sintaxis de la sección CONTROL son las siguientes:

#### ON EVERY ROW

La lista de instrucciones que sigue a esta instrucción no procedural se ejecutará después de leer cada fila devuelta por la instrucción SELECT del CURSOR.

#### ON LAST ROW

La lista de instrucciones que sigue a esta instrucción no procedural se ejecutará después de leer la última fila devuelta por la instrucción SELECT del CURSOR.

#### Ejemplo 6 [PCURSOR6]:

```
database almacén

define
 variable num_fila integer
 variable pr_vent like articulos.pr_vent
 variable articulo like articulos.articulo
 variable descripcion like articulos.descripcion
 variable actualizados smallint default 0
end define

main begin
 declare cursor actualiza for select rowid num_fila, articulo,
 descripcion, pr_vent
 by name
 from articulos
 order by descripcion
 control
 on every row
 display articulo + 0 using "#####" at 15, 2,
 descripcion & null at 15,20,
 pr_vent + 0 using "###,###,###.##" at 15, 60
 prompt for pr_vent label "Nuevo precio de venta"
 end prompt
 if cancelled = false then begin
 update articulos set pr_vent = $pr_vent
 where rowid = $num_fila
 let actualizados = actualizados + 1
 end
 clear list at 15,1 with 1,65
 on last row
 clear
 pause "Artículos actualizados " && actualizados using "<<<<<<<<"
 end control
 display "Artículo" at 12, 2, "Descripcion" at 12,20,
 "Precio Venta actual" at 12, 60
 display "—————" at 13, 2, "—————" at 13,20,
```

```
"_____ " at 13, 60
foreach actualiza begin end
end main
```

Este programa es muy parecido al PCURSOR5, pero empleando las instrucciones no procedurales ON EVERY ROW y ON LAST ROW. Como puede comprobarse, la instrucción FOREACH no afecta a ninguna instrucción de forma directa. Ahora bien, indirectamente cuando se produzca la lectura de cada fila implicada en el CURSOR, el control del programa pasará a la sección CONTROL del CURSOR y allí será donde para cada una de ellas se realice la actualización de la tabla de «artículos». Como puede comprobarse, las instrucciones incluidas en la instrucción no procedural ON EVERY ROW en este programa son idénticas a las afectadas por la instrucción FOREACH en el programa «PCURSOR5». La única diferencia es el incremento del contador «actualizados» para contabilizar cuantas filas se actualizan al final del proceso. Este contador se mostrará en pantalla mediante la instrucción PAUSE después de actualizar las filas necesarias implicadas en el CURSOR.

**[{BEFORE | AFTER} GROUP OF {columna\_order | número}]**

Para poder emplear esta instrucción no procedural la SELECT del CURSOR debe incluir la cláusula «ORDER BY columna\_order». En ese caso, las instrucciones que le siguen serán ejecutadas antes/después de que la columna indicada cambie de valor entre una fila recogida y la anterior. Al ejecutar la cláusula «AFTER GROUP» se considerará que la columna tiene el valor correspondiente al de la fila anterior.

En caso de indicar varias instrucciones no procedurales de este tipo sobre columnas diferentes, dichas instrucciones se ejecutarán según el orden de las columnas en la cláusula «ORDER BY» de la SELECT.

En caso de declarar un CURSOR con una instrucción SELECT dinámica, mediante una instrucción PREPARE, estas instrucciones no procedurales han de indicar el «número» de la columna en la cláusula «ORDER BY». En este caso, no se puede especificar el nombre de la columna de ordenación.

El orden de ejecución de estas instrucciones no procedurales, dependiendo de una cláusula «ORDER BY» de tres columnas («a,b,c») es el siguiente:

```
BEFORE GROUP OF a
 BEFORE GROUP OF b
 BEFORE GROUP OF c
 ON EVERY ROW
 AFTER GROUP OF c
 AFTER GROUP OF b
 AFTER GROUP OF a
 ON LAST ROW
```

### Ejemplo 7 [PCURSOR7]:

```
database almacen

define
 variable i smallint default 0
 variable cliente like clientes.cliente
 variable empresa like clientes.empresa
 variable provincia like provincias.provincia
 variable descripcion like provincias.descripcion
end define

main begin
 declare cursor listado for
 select cliente, empresa, provincias.provincia, descripcion
 by name
 from clientes, provincias
 where clientes.provincia = provincias.provincia
 order by provincias.provincia, empresa
 control
```



```

on every row
 if i = 18 then begin
 pause "Pulse [Return] para cambiar de página"
 let i = 4
 clear list at 4,1 with 15,79
 end
 let i = i + 1
 display cliente + 0 using "####" at i, 10,
 empresa & null at i, 30
before group of provincia
 clear
 display frame box at 1,1 with 20, 79
 label "Clientes agrupados por Provincias"
 display descripcion at 2,3
 let i = 4
after group of provincia
 pause "TOTAL CLIENTES " && (group count) using "###,###"
on last row
 pause "T O T A L D E G R U P O S" &&
 count using "###,###"
end control
foreach listado begin end
clear
pause "FIN DEL PROGRAMA. Pulse [Return] para continuar"
end main

```

Este programa muestra un listado de clientes agrupados por provincias. Si bien lo lógico en CTL es realizar los listados mediante el empleo de STREAMS, dado que no hemos tratado aún este objeto dentro del presente volumen es por lo que, en el ejemplo anterior, la información se muestra en pantalla a través de las instrucciones de entrada/salida DISPLAY y CLEAR. La sección CONTROL incluye todas las instrucciones que producirán la salida del listado repartidas entre las instrucciones no procedurales. Cuando la instrucción FOREACH comienza la lectura de las filas, el control del programa se dirige hacia la instrucción no procedural correspondiente, dependiendo de la información leída. En este ejemplo se han utilizado dos valores agregados propios de «cursos» («group count» y «count») en las instrucciones no procedurales «AFTER GROUP ...» y «ON LAST ROW», respectivamente. Todos los valores agregados posibles en los «cursos» se explican en la sección 10.7 de este capítulo.

En nuestro ejemplo, los dos valores agregados citados devuelven respectivamente el número de filas pertenecientes a cada grupo y el número total de filas leídas en el CURSOR. Al igual que ocurría en ejemplos anteriores, al mostrar una variable definida con la cláusula «LIKE» aparece el atributo LABEL. Este atributo se elimina convirtiendo el dato a mostrar en una expresión. En nuestro caso, la columna numérica «cliente» se incrementa con «0» y a la columna CHAR «empresa» se le concatena un «NULL».

#### Ejemplo 8 [PCURSOR8] (dinámico):

```

database almacén

define
 frame condicion like provincias.*
end frame

variable i smallint default 0
variable cliente like clientes.cliente
variable empresa like clientes.empresa
variable prov like provincias.provincia
variable descrip like provincias.descripcion
variable optimiza char(200)
end define

main begin
 input frame condicion for query on provincias.provincia provincias.descripcion provincias.prefijo

```

```
to optimiza
end input
if optimiza is not null then let optimiza = "and " && optimiza clipped
prepare inst1 from
"select cliente, empresa, provincias.provincia, descripcion "&&
"from clientes, provincias " &&
"where provincias.provincia = clientes.provincia " &&
optimiza clipped &&
" order by 3, empresa"
declare cursor listado for inst1
control
 on every row
 if i = 18 then begin
 pause "Pulse [Return] para cambiar de página"
 let i = 4
 clear list at 4,1 with 15,79
 end
 let i = i + 1
 display cliente + 0 using "#####" at i, 10,
 empresa & null at i, 30
 before group of 1
 clear
 display frame box at 1,1 with 20, 79
 label "Clientes agrupados por Provincias"
 display descrip at 2,3
 let i = 4
 after group of 1
 pause "TOTAL CLIENTES " && (group count) using "###,###"
 on last row
 pause "T O T A L D E G R U P O S" &&
 count using "###,###"
end control
foreach listado into cliente, empresa, prov, descrip begin end
clear
pause "FIN DEL PROGRAMA. Pulse [Return] para continuar"
end main
```

El programa anterior realiza un listado de clientes agrupados por provincia en el que se emplea un CURSOR con una instrucción SELECT dinámica. La parte dinámica de dicha instrucción SELECT son las posibles condiciones que el operador puede introducir en un objeto FRAME mediante su operación de QUERY. La petición de condiciones mediante un objeto FRAME es muy común, por lo que la hemos incluido en este ejemplo pese a no haber tratado aún dicho objeto.

No obstante, un FRAME es un grupo de variables cuya estructura es similar a la composición de una fila en una tabla de la base de datos. El resultado de la entrada de datos sobre este FRAME es un literal sobre la variable de programación «optimiza». Este literal está compuesto por condiciones sobre las columnas especificadas en la cláusula «ON» de la instrucción INPUT FRAME. En este caso, la cláusula «BY NAME» de esta instrucción no puede emplearse, ya que el resultado de las condiciones introducidas por el operador sólo incluirán el nombre de la columna y no el de la tabla, pudiendo producir el error «Columna ambigua».

Una vez introducidas las condiciones sobre el FRAME, el resultado se emplea para la construcción de una instrucción SELECT que posteriormente se declarará como un CURSOR. Este CURSOR incluye la sección CONTROL, en la que emplea todas las instrucciones no procedurales para la realización del listado. Estas instrucciones no procedurales ejecutarán el proceso que les sigue cuando la instrucción FOREACH comience a leer filas y éstas cumplan o no las acciones de dichas instrucciones no procedurales. La recepción de valores del CURSOR sobre variables de programación se realiza en la instrucción FOREACH mediante la cláusula «INTO». El listado a pantalla se realiza mediante las instrucciones de entrada/salida de CTL, ya que hasta ahora no hemos explicado nada sobre el STREAM.

## Apertura del CURSOR: Instrucción OPEN

La apertura del CURSOR consiste en ejecutar la instrucción SELECT declarada como CURSOR para obtener su tabla derivada y poder realizar un proceso concreto de fila en fila mediante las instrucciones de lectura del CURSOR. Dependiendo del optimizador, la apertura de un CURSOR leerá por completo todas las filas que cumplan las condiciones establecidas en la cláusula «WHERE» o bien sólo una parte de ellas. Por ejemplo, si la instrucción SELECT asociada al CURSOR incluye la cláusula «ORDER BY», y ésta realiza la ordenación mediante ficheros temporales y no mediante índices, la tabla derivada se generará con todas las filas que cumplan las condiciones. Sin embargo, si no se incluye la «ORDER BY», el CURSOR se leerá en bloques cargando en memoria sólo una parte de aquellas filas que cumplan la condición, realizando la lectura de las siguientes filas cuando sea necesario. La instrucción que abre un CURSOR es la siguiente:

**OPEN** nombre\_cursor [**USING** lista\_variables\_host]

La ejecución de esta instrucción a nivel de sistema operativo implica la apertura de los ficheros que representan la tabla o tablas especificadas en la cláusula «FROM» de la instrucción SELECT. Indirectamente, esta característica puede desembocar en una saturación de límites por parte del sistema operativo (UNIX/Linux, o Windows) a nivel del número máximo de ficheros abiertos.

La cláusula «USING» de esta instrucción será obligatoria cuando en la declaración del CURSOR las variables «hosts» que intervienen en la SELECT se sustituyen por signos de interrogación («?»). La «lista\_variables\_host» sustituirá, por orden de aparición, a dichos signos de interrogación en la declaración del CURSOR. Esta posibilidad es interesante cuando se va a condicionar a un CURSOR con diferentes variables desde distintos puntos del programa. Por ejemplo:

```
declare cursor consulta for select empresa by name from clientes
 where cliente between ? and ?
```

```
open consulta using desde, hasta
fetch ...
...
close consulta
```

```
open consulta using desdecli, hastacli
fetch ...
...
close consulta
```

La utilización de esta instrucción OPEN sólo es necesaria cuando las filas de la tabla derivada del CURSOR se van a tratar con una instrucción FETCH. En caso de emplear, otras instrucciones asociadas con el CURSOR (por ejemplo, la instrucción WINDOW), ellas se encargarán de abrir y leer el CURSOR cuando sea necesario. Asimismo, la instrucción FOREACH tiene implícita la apertura del CURSOR.

### Ejemplo 1: POPEN1

```
database almacén

define
 variable empresa like clientes.empresa
end define

main begin
 declare cursor consulta for select empresa by name from clientes
 open consulta
 fetch consulta
 while found = true begin
 pause empresa
 end while
end main
```

```
 fetch consulta
 end
 close consulta
 end main
```

Este programa declara un CURSOR que se encarga de leer todas las empresas existentes en la tabla «clientes». La recepción de los valores se hace sobre la variable cuyo nombre sea idéntico a la columna leída («empresa»). La apertura «OPEN» es la que ejecuta la instrucción SELECT asociada, mientras que la instrucción FETCH siguiente leerá la primera de las filas de la tabla derivada. Dependiendo de que esta instrucción lea o no la primera fila («FOUND»), se muestra la empresa correspondiente, leyendo a continuación la siguiente fila consecutiva, y así sucesivamente hasta la última. Una vez mostradas todas las empresas se cierra el CURSOR, liberando los ficheros abiertos en la apertura.

### Ejemplo 2 [POPEN2]:

```
database almacen

define
 variable empresa like clientes.empresa
 variable desde like clientes.cliente default 0
 variable hasta like clientes.cliente default 9999
 check ($ is not null and $ >= desde)
end define

main begin
 declare cursor consulta for select empresa by name from clientes
 where cliente between ? and ?
 prompt for desde label "Desde cliente"
 end prompt
 if cancelled = true then let desde = 0
 prompt for hasta label "Hasta cliente"
 end prompt
 open consulta using desde, hasta
 fetch consulta
 while found = true begin
 pause empresa
 fetch consulta
 end
 close consulta
end main
```

Este ejemplo es similar al anterior, salvo que en este caso se emplea un rango de valores para condicionar la instrucción SELECT asociada al CURSOR. Estas variables se pueden situar en la declaración del CURSOR o bien se pueden pasar en la cláusula «USING» de la instrucción OPEN, como en este caso. En nuestro ejemplo, la declaración podría haberse realizado de la siguiente forma:

```
declare cursor consulta for select empresa by name from clientes
 where cliente between $desde and $hasta
```

de modo que la apertura fuese:

```
open consulta
fetch consulta
...
close consulta
```

## Lectura de filas del CURSOR

Hasta estos momentos hemos explicado la forma de declarar un CURSOR y cómo abrirlo. Esta apertura consistía en ejecutar la instrucción SELECT asociada al CURSOR, obteniendo su tabla derivada. La lectura de esta tabla

derivada se realiza de fila en fila para su tratamiento particular dentro de un proceso concreto. Las instrucciones de lectura de las filas de un CURSOR son FETCH y FOREACH.

La ejecución de cualquiera de las instrucciones mencionadas implica activar las instrucciones no procedurales correspondientes definidas en la sección CONTROL de la declaración del CURSOR: BEFORE/AFTER GROUP, ON EVERY ROW y ON LAST ROW.

Asimismo, al ejecutar cualquiera de las instrucciones FETCH o FOREACH de un CURSOR declarado con la cláusula «FOR UPDATE» (CURSOR de actualización), la fila leída queda bloqueada automáticamente para el resto de usuarios. Dicha fila se desbloqueará en la lectura de la siguiente fila del CURSOR, siempre y cuando no se controle una transacción en una base de datos transaccional. Es decir:

```
begin work
foreach nombre_cursor begin
...
update tabla ...
...
end
commit work
```

bloqueará todas las filas devueltas por el CURSOR y tratadas en transacción para el resto de usuarios, desbloqueándose cuando finalice la transacción (COMMIT WORK o ROLLBACK WORK). Para más información consulte el capítulo sobre [«Bloqueos»](#) en este mismo volumen.

Los cursores que maneja CTL son bidireccionales, es decir, la lectura de las filas de la tabla derivada se pueden leer de la primera a la última o viceversa, es decir, en ambas direcciones. En consecuencia, desde cualquier punto en el proceso de la lectura del CURSOR se puede leer la primera fila, la anterior, la siguiente o la última.

A continuación se indican las características más significativas de cada una de las instrucciones de lectura de un CURSOR.

## Instrucción FETCH

La instrucción FETCH se encarga de leer una fila de la tabla derivada devuelta por la instrucción SELECT asociada al CURSOR. Su sintaxis es la siguiente:

```
FETCH [FIRST | PREVIOUS | LAST] nombre_cursor
[INTO list_variables_receptoras]
```

En caso de que el CURSOR haya sido abierto recientemente, la ejecución de esta instrucción provocará la lectura de la primera fila de dicha tabla derivada, cargando las variables receptoras indicadas en las cláusulas «INTO», «BY NAME» o «BY NAME ON frame», de la instrucción SELECT, o bien en la cláusula «INTO» de la FETCH, si es que existe. En caso de indicar las variables receptoras en la instrucción SELECT y también en la cláusula «INTO» de la FETCH, predominará esta última antes que la propia declaración. Para poder leer las sucesivas filas de la tabla derivada del CURSOR habrá que ejecutar tantas instrucciones FETCH como filas se deseen.

### Ejemplo 1 [PFETCH1]:

```
database almacén

define
 variable empresa like clientes.empresa
end define

main begin
 declare cursor consulta for select empresa by name from clientes
 open consulta
 fetch consulta
 display frame box at 2,1 with 17,60 label "C u r s o r e s B i d i r e c c i o n a l e s"
 display box at 4,5 with 3,50
```

```
display box at 9,5 with 8,50
display keylabel("fup") at 10,10, " — Anterior" at 10,20,
 keylabel("fdown") at 11,10, " — Posterior" at 11,20,
 keylabel("ftop") at 12,10, " — Primero" at 12,20,
 keylabel("fbottom") at 13,10, " — Ultimo" at 13,20,
 keylabel("fquit") at 15,10, " — F I N" at 15,20
while found = true begin
 display empresa at 5,10
 read key
 clear at 5,10
 switch actionlabel(lastkey)
 case "fup"
 fetch previous consulta
 case "fdown"
 fetch consulta
 case "ftop"
 fetch first consulta
 case "fbottom"
 fetch last consulta
 case "fquit"
 break
 end
end
close consulta
clear
end main
```

El programa anterior demuestra el funcionamiento de los «cursores» bidireccionales. El CURSOR que se declara lee toda la tabla «clientes», mostrando, mediante las instrucciones de salida DISPLAY, el valor obtenido en la variable receptora «empresa». A continuación, si la instrucción SELECT del CURSOR devuelve alguna fila como tabla derivada, se muestra la primera. Posteriormente, se pide al operador que pulse una tecla para provocar el movimiento bidireccional de la lectura del CURSOR. La recepción de valores sobre la variable «empresa» se podría incluir en la propia instrucción FETCH. El programa finalizará cuando se dé alguna de las siguientes circunstancias:

- Cuando el operador pulse la tecla asignada a la acción <fquit> en el fichero de acciones.
- Cuando al estar seleccionada la primera fila de la tabla derivada se intente leer la fila anterior.
- Cuando al estar seleccionada la última fila de la tabla derivada se intente leer la fila siguiente.

## Instrucción FOREACH

La instrucción FOREACH se encarga de abrir un CURSOR declarado previamente y de leer todas las filas de su tabla derivada de forma secuencial. Una vez leída la última fila lo cierra automáticamente. Su sintaxis es la siguiente:

```
FOREACH nombre_cursor [INTO lista_variables_receptoras]
 [USING {lista_variables | nombre_frame.* }]
```

Al emplear esta instrucción FOREACH para la lectura de un CURSOR no tendremos que emplear ni la OPEN para abrirlo ni la CLOSE para cerrarlo. Por esta razón, la instrucción FOREACH incluye la cláusula «USING» para poder abrir «cursores» en los que las variables «hosts» se indican en la apertura mediante signos de interrogación («?»). Asimismo, la cláusula «INTO», al igual que en la instrucción FETCH, sirve para recepción de los valores leídos en el CURSOR. Esta cláusula «INTO» es especialmente útil cuando se declara un CURSOR cuya instrucción SELECT es dinámica (definida mediante una PREPARE) y se desea extraer información sobre variables de programación.

Al igual que la instrucción FETCH, en los «cursores» de actualización (declarados con la cláusula «FOR UPDATE») al provocar la lectura de una fila ésta quedará bloqueada para el resto de usuarios que tengan acceso a la

base de datos. En caso de no trabajar con transacciones, esta fila quedará desbloqueada cuando se lea la siguiente fila del CURSOR.

La instrucción FOREACH construye un bucle para la lectura de todas las filas de la tabla derivada. La forma de romper la ejecución de un bucle FOREACH, al igual que en el resto de bucles de CTL, es mediante las instrucciones: BREAK, EXIT PROGRAM o RETURN. Por la lectura de cada fila de la tabla derivada del CURSOR el control del programa pasará, en caso de que exista, hacia aquellas instrucciones no procedurales indicadas en la cláusula «CONTROL» de la directiva DECLARE al definir el CURSOR.

#### Ejemplo [PFOREAC1]:

```

database almacén

define
 frame condicion like provincias.*
end frame
variable cliente like clientes.cliente
variable empresa like clientes.empresa
variable prov like provincias.provincia
variable descrip like provincias.descripcion
variable optimiza char(200)
variable salida char(1)
variable sz smallint
variable tp smallint
variable bt smallint
variable rt smallint
variable lt smallint
end define

main begin
 forever begin
 input frame condicion for query
 on provincias.provincia provincias.descripcion provincias.prefijo to optimiza
 end input
 if cancelled = true then break
 clear frame condicion
 if optimiza is not null then let optimiza = "and " && optimiza clipped
 prepare inst1 from "select cliente, empresa, provincias.provincia, descripcion "&&
 "from clientes, provincias " &&
 "where provincias.provincia = clientes.provincia " && optimiza clipped &&
 " order by 3, empresa"
 declare cursor listado for inst1
 control
 on every row
 put stream standard column 10,
 cliente + 0 using "####", column 30,
 empresa & null , skip 1
 before group of 1
 put stream standard column 3, descrip, skip 2
 after group of 1
 put stream standard skip 1, column 30,
 "TOTAL CLIENTES ",
 column 60, (group count) using "###,###", skip 2
 on last row
 put stream standard skip 2, column 20,
 "TOTAL DE GRUPOS",
 count using "###,###", skip 1
 end control
 menu destino
 format stream standard size sz margins top tp,
 bottom bt, left lt, right rt
 first page header size 6
 end forever
end main

```

```
 put stream standard column 30, "L i s t a d o",
 skip 1, column 34, "d e", skip 1,
 column 30, "C l i e n t e s", skip 2
 call cabecera()
 page header size 2
 call cabecera()
 page trailer size 2
 put stream standard column 40, "Pág.: ",
 current page of standard, skip 1
end format
switch salida
 case "P"
 start output stream standard to "/tmp/fic"&procid() using "&&&&&"
 case "I"
 start output stream standard through printer
 end
foreach listado into cliente, empresa, prov, descrip begin end
stop output stream standard
if salida = "P" then begin
 window from "/tmp/fic" & procid() using "&&&&&" as file
 20 lines 79 columns label "L i s t a d o"
 at 1,1
 rm "/tmp/fic" & procid() using "&&&&&"
end
end
pause "FIN DEL PROGRAMA. Pulse [Return] para continuar"
end main

menu destino "E l e g i r" down no wait skip 2 line 5 column 20
 option "Pantalla" key "P"
 let salida = "P"
 let sz = 18
 let tp = 0
 let bt = 0
 let rt = 79
 let lt = 0
 option "Impresora" key "I"
 let salida = "I"
 let sz = 66
 let tp = 3
 let bt = 3
 let rt = 80
 let lt = 1
end menu

function cabecera() begin
 put stream standard column 10, today, column 30,
 "Listado de clientes", column 60, now, skip 1
end function
```

Este programa realiza un listado con un CURSOR dinámico dependiendo de la información introducida en el FRAME. La salida hacia pantalla o impresora se consigue mediante el manejo del STREAM. Si bien estos objetos no se han explicado aún, su utilización para la realización de listados es muy frecuente.

En primer lugar el programa pide al operador que introduzca las condiciones sobre qué provincias desea consultar los clientes existentes. Dependiendo de esta información se prepara una instrucción SELECT dinámica (cláusula «WHERE») que posteriormente se declarará como un CURSOR para la realización del listado. La declaración del listado emplea la sección CONTROL con todas las instrucciones no procedurales (ON EVERY ROW, ON LAST ROW y BEFORE/AFTER GROUP ...). Éstas contienen instrucciones que redirigen la información hacia el objeto STREAM (PUT). Dichas instrucciones se ejecutarán después de leer las filas del CURSOR con la instrucción FOREACH.



A continuación, aparecerá un menú «Pop-up» para elegir la salida del listado hacia impresora o pantalla. Dependiendo de esta elección se asignan unos valores a unas variables que representarán los márgenes y tamaño de página de cada salida (pantalla o impresora). Posteriormente, se abre el STREAM de salida hacia un fichero, cuyo nombre se construye con el número aleatorio devuelto por la función interna de CTL «procid()», o bien hacia la impresora. Previamente a esta apertura se ha formateado dicho STREAM, fijando los márgenes, la información que aparecerá en la primera página del listado «FIRST PAGE HEADER», así como la cabecera y el pie de cada página del listado. Este formateo debe realizarse antes de la apertura del STREAM. A continuación, mediante la instrucción FOREACH se abre el CURSOR, se leen todas sus filas y se extrae la información sobre las variables receptoras indicadas en la cláusula «INTO». Esta instrucción FOREACH fijará la ejecución de las instrucciones no procedurales incluidas en la sección CONTROL de la directiva DECLARE del CURSOR. Cuando la instrucción FOREACH lea la última fila se cerrarán el CURSOR y el STREAM. Por último, si la salida del listado es a pantalla, aparecerá una ventana con el fichero generado, pudiendo movernos sobre ella página a página y al principio y fin del fichero. Al abandonar esta ventana el fichero empleado para recoger el listado se elimina del disco mediante la instrucción «rm».

### Ejemplo 2 [PFOREAC2]:

```

database almacen

define
 variable cliente like clientes.cliente
 variable empresa like clientes.empresa
 variable total_factura like clientes.total_factura default 0
end define

main begin
 prepare actualiza from "update clientes set total_factura = ? where current of factura"
 declare cursor factura for select cliente, empresa by name from clientes
 for update
 foreach factura begin
 select sum(cantidad * precio * (1 - descuento / 100))
 into total_factura
 from albaranes, lineas
 where albaranes.cliente = $cliente and albaranes.albaran = lineas.albaran
 display empresa clipped at 15,1,
 total_factura using "###,###,##&.&&" at 15,40
 execute actualiza using total_factura
 pause "Pulse [Return] para continuar" reverse
 clear at 15,1
 clear at 15,40
 let total_factura = 0
 end
 window from select cliente, empresa, total_factura from clientes
 15 lines label "C l i e n t e s"
 at 1,10
end main

```

Este programa actualiza el «total\_facturado» de cada cliente existente en la tabla «clientes». Esta actualización se realiza mediante un CURSOR declarado «FOR UPDATE». La actualización de cada una de las filas leídas en este CURSOR se realiza mediante la instrucción UPDATE y la cláusula «WHERE CURRENT OF factura», que se preparó mediante una instrucción PREPARE. Para cada cliente implicado en el CURSOR se calcula su total facturado, leyendo de las tablas «lineas» y «albaranes». La información calculada se mostrará en pantalla mediante la instrucción DISPLAY. Cuando se hayan actualizado todos los clientes aparecerá una ventana (WINDOW) con todos los clientes existentes en la tabla «clientes».

## Cierre del CURSOR: Instrucción CLOSE

---

Esta instrucción cierra un CURSOR abierto previamente mediante la instrucción OPEN. Su sintaxis es:

**CLOSE** nombre\_cursor

Al ejecutar esta instrucción se abandona la tabla derivada generada previamente por la instrucción OPEN al abrir el CURSOR. Por lo tanto, una vez cerrado el CURSOR no se podrá ejecutar la instrucción FETCH para la lectura de una fila del CURSOR.

NOTA: Esta instrucción no se puede emplear cuando el CURSOR se haya abierto por medio de la instrucción FOREACH, ya que ésta lleva implícito el cierre del CURSOR.

### Ejemplo 1:

```
database almacen

define
 variable empresa like proveedores.empresa
end define

main begin
 declare cursor provee for select empresa by name from proveedores
 open provee
 fetch provee
 while found = true begin
 display empresa at 15,1
 pause "Pulse [Return] para continuar. " && keylabel("fquit") &&
 " para salir" reverse
 if actionlabel(lastkey) = "fquit" then break
 clear at 15,1
 fetch provee
 end
 clear
 close provee
end main
```

Este programa muestra cada uno de los proveedores existentes en la base de datos. Para cada proveedor que se muestra en pantalla el programa solicita al operador una tecla. Si la tecla pulsada tiene asignada la acción <fquit> en el fichero de acciones (por defecto [F2] en UNIX y [ESC] en y Windows), se rompe el bucle formado por la instrucción WHILE y se cierra el CURSOR, finalizando así el programa.

## Instrucción WINDOW asociada a un CURSOR

---

Como ya vimos en el capítulo anterior, la instrucción WINDOW permite mostrar en pantalla la tabla derivada asociada a un CURSOR. La sintaxis de esta WINDOW relacionada con SQL es la siguiente:

```
WINDOW [SCROLL | CLIPPED] FROM {instr_select | CURSOR nombre_cursor}
[BY COLUMNS]
líneas LINES [columnas COLUMNS]
[STEP salto] [LABEL {"literal" | número}]
AT línea, columna
[SET lista_variables [FROM lista_columnas]]
```

Para mostrar las filas de la tabla derivada de un CURSOR no es necesario abrirlo (instrucción OPEN), ni leerlo (FETCH) ni cerrarlo (CLOSE), ya que la propia instrucción WINDOW realiza todas estas operaciones. Asimismo, para mostrar dicha tabla derivada en una ventana (WINDOW), tampoco es necesario recoger los valores de las columnas indicadas en la «lista\_selección» de la instrucción SELECT asociada al CURSOR sobre variables receptoras.

NOTA: En el capítulo 8 de este mismo manual se explicó asimismo el funcionamiento de la instrucción WINDOW con la representación de un fichero o del resultado de ejecución de un comando del sistema operativo (esta última posibilidad sólo en UNIX).

#### Ejemplo [PWINDOW4]

```

database almacén

define
 frame condiciones like clientes.*
end frame
variable optimiza char(300)
end define

main begin
 forever begin
 input frame condiciones for query by name to optimiza
 end input
 if cancelled = true then break
 if optimiza is null then let optimiza = "cliente > 0"
 clear frame condiciones
 prepare inst1 from "select * from clientes where " && optimiza clipped
 declare cursor mostrar for inst1
 window scroll from cursor mostrar
 15 lines label "C l i e n t e s —>"
 at 1,1
 end
 clear
 end main
end main

```

Este ejemplo emplea un objeto FRAME para pedir condiciones sobre qué clientes se desean consultar. La operación QUERY de este FRAME devuelve una variable («optimiza») con las condiciones establecidas por el operador. Estas condiciones se emplean para formar una instrucción SELECT mediante PREPARE. Dependiendo de las condiciones establecidas, esta SELECT dinámica devolverá cada vez una tabla derivada distinta. Posteriormente, se declara esta instrucción SELECT como un CURSOR y, por último, todas las filas que cumplan las condiciones establecidas en el objeto FRAME se mostrarán en una ventana producida por la instrucción WINDOW.

## Expresiones del CURSOR

Una vez abierto un CURSOR es posible emplear unos valores agregados (estadísticas) referentes a las filas de dicho CURSOR. Las expresiones que pueden especificarse para calcular estos valores agregados tienen la siguiente sintaxis:

```

[GROUP] {COUNT | PERCENT | {[TOTAL | AVG | MAX | MIN]
 OF variable_receptora} [WHERE condiciones]

```

Las expresiones en las que intervenga la palabra reservada «GROUP» sólo podrán emplearse en las instrucciones no procedurales «AFTER GROUP OF columna» de la sección CONTROL de la directiva DECLARE del CURSOR.

#### Ejemplo [PCURSOR9]:

```

database almacén

define
 variable cliente like clientes.cliente
 variable empresa like clientes.empresa
 variable fact money(16,2)
end define

main begin

```

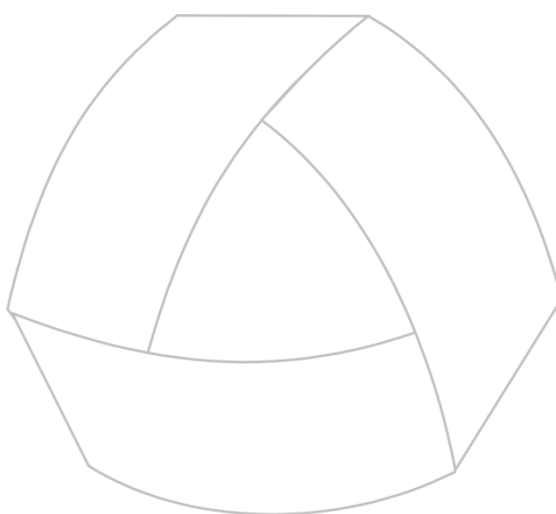
```
declare cursor agregados for select clientes.cliente, empresa,
cantidad * precio * (1 - descuento / 100) fact
by name
from clientes, albaranes, lineas
where clientes.cliente = albaranes.cliente
and albaranes.albaran = lineas.albaran
order by clientes.cliente
control
before group of cliente
display cliente using "####" at 5,1, empresa clipped at 5, 10
after group of cliente
display "Número: " at 7,1,
(group count) using "##,###" at 7,40
display "Porcentaje: " at 8,1,
(group percent) using "###" at 8,40
display "Suma: " at 9,1,
(group total of fact) using "###,###,##&.&&" at 9,40
display "Promedio: " at 10,1,
(group avg of fact) using "###,###,##&.&&" at 10,40
display "Máxima Compra: " at 11,1,
(group max of fact) using "###,###,##&.&&" at 11,40
display "Mínima Compra: " at 12,1,
(group min of fact) using "###,###,##&.&&" at 12,40
pause "Pulse [Return] para continuar" reverse
clear list at 5,1 with 10,50
on last row
display "T O T A L" reverse at 5,1
display "Número: " at 7,1,
(count) using "##,###" at 7,40
display "Porcentaje: " at 8,1,
(percent) using "###" at 8,40
display "Suma: " at 9,1,
(total of fact) using "###,###,##&.&&" at 9,40
display "Promedio: " at 10,1,
(avg of fact) using "###,###,##&.&&" at 10,40
display "Máxima Compra: " at 11,1,
(max of fact) using "###,###,##&.&&" at 11,40
display "Mínima Compra: " at 12,1,
(min of fact) using "###,###,##&.&&" at 12,40
end control
foreach agregados begin end
pause "F I N. Pulse [Return] para continuar" reverse
end main
```

Este programa emplea un CURSOR para calcular los valores agregados del grupo «clientes». En primer lugar, se declara el CURSOR con su sección CONTROL. Esta sección contiene tres instrucciones no procedurales: BEFORE GROUP, AFTER GROUP y ON LAST ROW:

- La instrucción BEFORE GROUP mostrará mediante DISPLAY el «cliente» del que se van a mostrar sus valores agregados.
- La instrucción AFTER GROUP mostrará todos los valores agregados de dicho grupo, es decir, del cliente mostrado anteriormente.
- La instrucción ON LAST ROW mostrará los valores agregados de todos los clientes leídos en el CURSOR. Ésta será la última instrucción no procedural que se ejecutará antes de que se cierre el CURSOR.

### IMPORTANTE

CTL dispone de diversas funciones internas para el manejo de un CURSOR. Para más información acerca de estas funciones internas consulte el capítulo [«Conceptos Básicos del CTL»](#) de este mismo manual.



MultiBase  
MANUAL DEL PROGRAMADOR

**CAPÍTULO 11. El FORM: Objeto  
no procedural de DEFINE**

## FORM: Definición y usos

---

El FORM es un objeto no procedural que permite manipular los datos de las tablas de la base de datos a través de una serie de variables con una cierta correspondencia con columnas de la base de datos y de operaciones que dependen de esas variables. Además, pueden utilizarse variables auxiliares que no tengan correspondencia con columnas de las tablas de la base de datos. Las funciones básicas de un FORM son las siguientes:

- 1.— Generar una lista de filas (registros), denominada «lista en curso» (internamente se trata de un CURSOR), ejecutando un «query by forms» donde se seleccionan las filas de una tabla de la base de datos cuyas columnas cumplan las condiciones dadas por las variables correspondientes. Dado que es posible moverse dentro de esta «lista en curso», será en este movimiento donde las variables del FORM adquieran los valores de las correspondientes columnas de la tabla de la base de datos. Los nombres de estas variables del FORM son iguales que los de las correspondientes columnas de la tabla que se está manteniendo de la base de datos activa. Una vez posicionados en una fila determinada es posible llevar a cabo cualquiera de las operaciones que se citan a continuación.
- 2.— Editar las variables que se corresponden con las columnas de esa fila y, posteriormente, «volcar» los nuevos valores a la tabla de la base de datos activa que se está manteniendo (operación de actualización — instrucción MODIFY—).
- 3.— Borrar la fila de la tabla que se está manteniendo de la base de datos activa (operación de borrado — instrucción REMOVE—).
- 4.— Editar para agregar una fila nueva a la lista en curso y a la tabla que se está manteniendo. Si la lista en curso estuviese vacía, esta acción la inicializará. Al iniciar la edición, las variables tendrán el valor inicial de la columna correspondiente, definido en el diccionario, es decir, cuando se creó la base de datos. Por defecto, todas las variables del FORM tienen inicialmente el valor «NULL», salvo especificación contraria a través del atributo DEFAULT en el diccionario de la base de datos.

Las propiedades del FORM hacen que este objeto se utilice fundamentalmente para entrada de datos (inserción de filas a la tabla que se está manteniendo) y consulta de las filas ya grabadas en dicha tabla.

Un objeto FORM se define en la sección DEFINE de un módulo principal o librería, pudiendo existir únicamente un FORM por módulo, es decir, en un programa podrán definirse tantos objetos FORM como módulos contenga dicho programa.

## Conceptos básicos empleados en el FORM

---

Además de todos los conceptos expuestos hasta ahora en capítulos precedentes, dentro de un FORM hemos de considerar también los siguientes:

- 1.— Tabla en curso. Como ya se verá más adelante dentro de este mismo capítulo, un FORM puede mantener más de una tabla, si bien sólo podrá tener activa una de ellas, debiendo desactivar una para activar otra. Esta operación es similar a la empleada para la selección de una base de datos. Cuando una tabla se encuentra activa, todas las operaciones que se realicen (inserción, consulta, actualización y borrado) sólo tendrán efecto sobre las filas de dicha tabla. Asimismo, aunque las columnas de todas las tablas que se estén manteniendo se encuentren en la pantalla, el operador sólo podrá llevar a cabo las operaciones antes mencionadas sobre aquellas columnas pertenecientes a la tabla activa.
- 2.— Lista en curso. Como ya hemos indicado en el apartado anterior, una de las operaciones básicas que se pueden realizar con un FORM es la consulta de filas de la tabla o tablas que se están manteniendo en dicho objeto. Dependiendo de las condiciones especificadas, esta consulta podrá afectar o no a más de una fila. Las filas afectadas por dicha consulta serán las que conformarán la lista en curso manejada por el FORM. Interna-

mente, esta lista en curso es manejada por un CURSOR bidireccional, pudiendo desplazarnos por ella mediante las teclas asociadas a las acciones <fup>, <fdwn>, <ftop> y <fbottom> en el fichero de acciones de MultiBase («tactions»).

La lista en curso generada por una consulta («query by form») se incrementará cada vez que el usuario inserte nuevas filas a la tabla, es decir, cuando se ejecute la instrucción correspondiente a agregar filas (ADD). En caso de que el usuario no haya realizado ninguna consulta, la lista en curso se inicializará con las filas nuevas que se inserten (ADD).

Cada vez que se provoque una consulta sobre la tabla en curso se inicializará la lista en curso.

3.— Fila en curso. La fila en curso es aquella de la pantalla sobre la que se encuentra posicionado el cursor (en el caso de «scroll» de líneas). Indirectamente, al activar una fila de la lista en curso las variables que maneja el FORM adquieren los valores representados en la pantalla para dicha fila en curso.

## Estructura de un FORM

El objeto FORM consta de diversas secciones con diferentes funciones. Cada una de ellas se abre con su nombre y se cierra con la palabra reservada «END» seguida de dicho nombre. El orden especificado en el caso de definir una sección es significativo. Estas secciones, en su orden de definición, son las siguientes:

|                  |                                                                                                                                                        |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SCREEN</b>    | Sección opcional que describe el formato de presentación de la pantalla.                                                                               |
| <b>TABLES</b>    | Ésta es la única sección obligatoria en un FORM. En ella se incluye la lista de tablas a utilizar.                                                     |
| <b>VARIABLES</b> | Sección opcional que describe la lista de variables del FORM junto con la lista de atributos.                                                          |
| <b>MENU</b>      | Sección opcional que describe el menú asociado al FORM.                                                                                                |
| <b>CONTROL</b>   | Sección opcional que indica instrucciones no procedurales que ejecutarán las acciones a tomar antes o después de comenzar con la edición.              |
| <b>EDITING</b>   | Sección opcional que contiene instrucciones no procedurales que ejecutarán las acciones precisas en las distintas fases de interacción con el usuario. |
| <b>LAYOUT</b>    | Sección opcional que permite la modificación de ciertos elementos de presentación de la pantalla.                                                      |
| <b>JOINS</b>     | Sección opcional que permite especificar enlaces entre tablas.                                                                                         |

La sintaxis de definición de un FORM es la siguiente:

```

DEFINE
 [FORM

 [SCREEN [MESSAGE número_ayuda]
 {
 Texto [identificador1]
 Texto [identificador2]
 }
 [SCREEN [MESSAGE número_ayuda]
 {
 Texto [identificador3]
 ...
 }
 END SCREEN]
```

```
TABLES
 tabla [LABEL etiqueta] [número LINES]
 tabla1 [LABEL etiqueta] [número LINES]
 ...
END TABLES

[VARIABLES
 {identificador = | UNTAGGED} variable_form [atributos]
 ...
END VARIABLES]

[MENU nombre_menú [opciones]
 OPTION [nombre_opción] ["Etiqueta"] [opciones]
 ...
 OPTION [nombre_opción] ["Etiqueta"] [opciones]
 ...
 OPTION ...
 ...
END MENU]

[CONTROL
 {{BEFORE | AFTER} {{ADD] [QUERY] [UPDATE] [DELETE] [DISPLAY]
 [LINE] OF] lista_tablas
 ...}
 [AFTER CANCEL OF lista_tablas
 ...]
 {{BEFORE | AFTER} lista_tablas
 ...}
 [ON KEY "tecla"
 ...]
 [ON ACTION "acción"
 ...]
END CONTROL]

[EDITING
 {{BEFORE | AFTER} {{EDITADD] [EDITUPDATE] [EDITQUERY] OF]
 {lista_tablas | lista_columnas}
 ...}
 [BEFORE CANCEL OF lista_tablas
 ...]
 [ON KEY "tecla"
 ...]
 [ON ACTION "acción"
 ...]
END EDITING]

[LAYOUT
 opciones
END LAYOUT]

[JOINS
 tabla_líneas LINES OF tabla_cabecera [ORDER BY lista_columnas]
 [COMPOSITES <lista_columnas_enlace_cabecera>
 <lista_columnas_enlace_líneas>]
 ...
END JOINS]

END FORM]
END DEFINE
...
...
```



**Ejemplo [PFORM1]. FORM básico:**

```

database almacen

define
 form
 tables
 provincias
 end tables
 end form
end define

main begin
 menu form
end main

```

Este ejemplo mantiene la tabla «provincias» de la base de datos «almacen». La instrucción «menu form» ejecutará el menú del FORM, que en este caso no se ha definido, por lo que el menú que aparecerá para el mantenimiento de esta tabla estará compuesto por las siguientes opciones:

- Añadir: Insertará nuevas filas sobre la tabla «provincias».
- Modificar: Modificará la fila en curso.
- Borrar: Borrará la fila en curso.
- Encontrar: Consultará las filas de la tabla «provincias», inicializando de esta forma la lista en curso.
- Tabla: Seleccionará otra tabla a mantener en el FORM. En este ejemplo, esta opción no tiene sentido, ya que la única tabla a mantener es «provincias».
- Salida: Permitirá listar el aspecto de la pantalla del FORM con la fila en curso o bien con todas las filas de la lista en en curso. El listado puede realizarse hacia un «fichero» o hacia un «programa».
- Insertar: Insertará nuevas filas sobre la tabla «provincias». Esta opción es similar a la de «Añadir». Sólo tiene sentido cuando se está manteniendo una tabla sobre un FORM con «scroll» de líneas.

## Aspecto de la pantalla: Sección SCREEN

El aspecto de la pantalla de mantenimiento de un FORM, es decir, el aspecto que le aparecerá al usuario, se define en la sección SCREEN. Una SCREEN representa una página de dicho mantenimiento. Como límite, un FORM admite hasta 20 pantallas, es decir, 20 secciones SCREEN. Según se vayan activando los campos durante la edición de una fila (añadir y modificar), o bien mientras se estén asignando condiciones (consulta), el FORM se encargará de activar una SCREEN u otra. No obstante, cuando el operador no se encuentre en cualquiera de las operaciones de edición mencionadas, la forma de pasar de una SCREEN a otra será mediante las teclas asignadas a las acciones «fnext-page» y «fprevious-page» en el fichero de acciones «tactions». Por su parte, en el caso de encontrarnos en un FORM con «scroll» de líneas (cláusula «LINES» de la sección TABLES), la forma de activar una SCREEN lateral será mediante las teclas asignadas a las acciones <left-page> y <right-page> (este último caso simula un «scroll» lateral de las pantallas).

La sintaxis de la sección SCREEN es la siguiente:

```

[SCREEN [MESSAGE número_ayuda]
{
 Texto [identificador1]
 ...
 Texto [identificador2]
}
[SCREEN [MESSAGE número_ayuda]
{
 Texto [identificador3]
 ...
}]
END SCREEN]

```

Las llaves de cada SCREEN («{ }») deben encontrarse obligatoriamente en la primera columna de nuestro módulo. Si no es así se producirá un error de compilación.

Como puede comprobarse en la sintaxis anterior, una página de un FORM (SCREEN) puede extraerse de un fichero de ayuda mediante la cláusula «MESSAGE número\_ayuda». En el «número\_ayuda» del fichero de ayuda se encontrarán todos los literales («Texto») de la sección SCREEN en el mismo orden, línea y columna que si se incluyesen en la sintaxis de la propia sección SCREEN. Esta utilidad nos permitirá aislar los literales de la programación, por si en algún momento deseamos traducir la aplicación a otro idioma, o bien para que el usuario pueda configurarse sus propios literales mediante la edición de un fichero ASCII (fichero de ayuda). Para más información acerca de estos ficheros consulte el capítulo sobre [«Ficheros de Ayuda»](#) en este mismo volumen.

El lugar donde se mostrará el valor de cada columna a mantener en el FORM se representa mediante un «identificador». Los nombres de estos identificadores deben comenzar necesariamente por una letra, y podrán variar siempre y cuando se cambie también su definición dentro de la sección VARIABLES. Este «identificador» estará asignado a una variable del FORM (sección VARIABLES), que será la que maneje los valores de una columna en el mantenimiento. El «identificador» se encuentra incluido entre corchetes («[ ]») o bien entre corchetes y separados por barras verticales («[... | ...]») si la siguiente variable está muy próxima. Con este procedimiento, a la hora de representar más de 27 campos de un solo carácter («char(1)») habrá que emplear sub-índices en las variables de tipo CHAR, siendo la longitud que éstos expresen la de edición del campo, sin tener en cuenta la longitud del «identificador» definido en la sección SCREEN. Esto permite definir más «identificadores» de un solo carácter («char(1)») de los que en principio se podría al utilizar el rango «a-z». Por ejemplo:

```
database xxx

define
 form
 screen
{
 Etiqueta :[a1]
}
 end screen

 tables
 xx
 end tables

 variables
 a1 = columna char(1)
 ...
 end variables

 ...
end form
end define

main begin
 ...
end main
```

La sección SCREEN admite unos caracteres especiales para construir cajas y emplear atributos de pantalla tales como vídeo inverso o subrayado. Estos metacaracteres son:

|   |                                       |
|---|---------------------------------------|
| [ | Principio de identificador de SCREEN. |
| ] | Fin de identificador de SCREEN.       |
| ! | Principio de caja estándar.           |
| @ | Fin de caja estándar.                 |

|    |                                                                                                                                                                                                                                             |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ^  | Principio y fin de línea doble.                                                                                                                                                                                                             |
| ~  | Principio y fin de subrayado.                                                                                                                                                                                                               |
| '  | Principio y fin de vídeo inverso.                                                                                                                                                                                                           |
| %  | Principio de caja simple (semigráficos simples).                                                                                                                                                                                            |
| *  | Fin de caja simple.                                                                                                                                                                                                                         |
| ?  | Principio de caja doble (semigráficos dobles).                                                                                                                                                                                              |
| \$ | Fin de caja doble.                                                                                                                                                                                                                          |
|    | Equivale a cerrar y abrir corchetes («[]»). Se emplea para aproximar al máximo dos campos en SCREEN (separación de una columna), ya que el cierre de corchete y la apertura del siguiente identificador ocupan dos columnas de la pantalla. |

Estos metacaracteres pueden cambiarse indicando otros distintos en la variable de entorno DBMETACH (para más información consulte el capítulo sobre «Variables de Entorno» en el Manual del Administrador).

#### IMPORTANTE

En caso de estar utilizando un terminal que no disponga de semigráficos dobles en su «set» de caracteres, en lugar de éstos se emplearán semigráficos simples.

La sección SCREEN no establece el orden de petición de los campos para el mantenimiento de la tabla activa (éste se fija en la sección VARIABLES, que se explica más adelante en este mismo capítulo).

#### Ejemplo [PFORM2]: Empleo de los metacaracteres de la SCREEN de un FORM:

```

database almacén

define
 form
 screen
{
 ! Mantenimiento de Provincias

 ?
 'Código Provincia:[a]'
 $
 ^
 % %
 Descripción:[b] Prefijo.....: [c]
 * *

 @
}
end screen

tables
 provincias
end tables

variables
 a = provincia required reverse
 b = descripcion
 c = prefijo
end variables
end form
end define

main begin

```

```
 menu form
 end main
```

Este programa mantiene la tabla «provincias» empleando el menú por defecto del FORM. La sección SCREEN utiliza los metacaracteres de construcción de todos los tipos de cajas existentes: Estándar, caja doble y simple. Asimismo, se emplean los metacaracteres de inicio y fin de vídeo inverso y principio y fin de línea doble.

## Mantenimiento de tablas: Sección TABLES

La lista de las tablas a mantener de la base de datos activa se indica en la sección TABLES del FORM. El número máximo de tablas que se pueden indicar en esta lista es 20. La sintaxis de la sección TABLES es la siguiente:

```
TABLES
 tabla [LABEL etiqueta] [número LINES]
 tabla1 [LABEL etiqueta] [número LINES]
 ...
END TABLES
```

En esta sección sólo deberán indicarse aquellas tablas cuyas columnas se vayan a mantener en el FORM, no siendo necesario tampoco indicar aquellas cuyas columnas se vayan a consultar mediante el atributo LOOKUP del CTL.

Si se define la sección TABLES pero no la VARIABLES, se entenderá que se tratarán todas las columnas de la tabla. Como ya veremos en el siguiente apartado de este capítulo, dentro de un FORM no es necesario mantener todas las columnas de una tabla.

La «etiqueta» asociada a una tabla será el nombre del «alias», al cual se podrá acceder mediante la cláusula «DISPLAY TABLE LABEL» de la sección LAYOUT del FORM.

Por su parte, «número LINES» indica el número de filas de la tabla a mantener que aparecerán simultáneamente en pantalla. Este modo de mantenimiento se denomina «scroll de líneas». Como ya hemos indicado anteriormente, este tipo de mantenimiento en caso de existir más de una sección SCREEN se denomina «scroll» lateral al pasar de una página a otra.

### Ejemplo [PFORM3]. Mantenimiento con «scroll» de líneas y lateral entre dos páginas:

```
database almacén

define
 form
 screen
{
! P r o v i n c i a s P a g . : 1

 C ó d i g o D e s c r i p c i ó n
 ^
 [a] [b]

 @
}
screen
{
! P r o v i n c i a s P a g . : 2

 D e s c r i p c i ó n P r e f i j o
 ^
```

```

 [b] [c]

 @
}
end screen

tables
 provincias 7 lines
end tables

variables
 a = provincia required
 b = descripcion
 c = prefijo
end variables
end form
end define

main begin
 menu form
end main

```

Este ejemplo define dos SCREEN para ver el funcionamiento del «scroll» lateral entre páginas. El mantenimiento de la tabla «provincias» se realiza con «scroll» de líneas, pudiendo ver en cada página un máximo de 7 filas. Los tres identificadores empleados en las secciones SCREEN se encuentran definidos como variables en la sección VARIABLES del FORM. Hay que tener en cuenta que el identificador «b», correspondiente a la descripción de la provincia, se repite en las dos páginas, aunque sólo se define una vez. Esto significa que un identificador puede repetirse varias veces a lo largo de la sección SCREEN. El programa comienza en la sección MAIN, siendo la instrucción «menu form» la que activa el FORM mostrando la primera página de la SCREEN y el menú por defecto del FORM.

Las tablas que se enumeran en la sección TABLES corresponden a nombres de tablas de la base de datos que están compuestas por filas. La tabla que maneja el FORM es una lista de variables cuyos nombres son iguales a los de las columnas de la tabla a mantener. Por la relación que el FORM mantiene con la base de datos, al trabajar con una variable de este objeto indirectamente se está trabajando con la información de la columna que tenga el mismo nombre en la tabla que se está manteniendo.

#### Ejemplo 4 [PFORM4]. Mantenimiento de dos tablas en el mismo FORM sin sección JOINS que las enlace:

```

database almacen

define
 form
 tables
 clientes
 provincias
 end tables
 end form
end define

main begin
 add one
 next table "provincias"
 add one
end main

```

Este programa mantiene las tablas «clientes» y «provincias». No obstante, el mantenimiento de cada una se realiza de forma individual, es decir, aplicando el concepto de tabla activa. En primer lugar se añadirá una fila (ADD ONE) en la primera tabla indicada en la sección TABLES («clientes»). Una vez aceptada la fila introducida (acción «faccept»), o cancelada su introducción (acción «fquit»), se cambiará de tabla activa mediante la instrucción NEXT TABLE del CTL. Posteriormente, se agrega una fila (ADD ONE) en la tabla recién seleccionada como activa (en este caso «provincias»). Una vez que el operador pulse la tecla asignada a la acción «faccept» para grabar o «fquit» para cancelar, acabará la ejecución del programa.

## Mantenimiento de columnas: Sección VARIABLES

El SQL tiene la facultad de poder leer, insertar y modificar parte de las columnas que componen una tabla en la base de datos. Pues bien, aplicando esta propiedad del SQL al FORM, éste podrá definir aquellas variables que se correspondan con las columnas que se vayan a mantener. En esta sección se pueden definir varios tipos de variables:

- Variables asociadas a columnas (enlace de columnas).
- Variables PSEUDO.
- Variables VARTABLE.
- Variables DISPLAYTABLE.

Todos los identificadores definidos en la sección SCREEN tienen que estar asignados a la definición de cualquiera de los tipos de variables citados anteriormente. Si se definen mediante un identificador significará que los valores de la columna o variable se representarán en la pantalla en el momento que ésta adquiera un valor. No obstante, aquella variable cuyos valores no se desea que aparezcan en la pantalla se deberá definir con la palabra reservada «UNTAGGED».

Todos estos tipos de variables pueden definirse en un mismo objeto FORM. En los siguientes apartados se comentan las características de cada uno de estos tipos de variables.

### Variables asociadas a columnas

Estas variables pueden tener o no un identificador en pantalla (SCREEN) referenciado en su definición como «tabla.columna». Los nombres de estas variables coinciden con los de las columnas de la tabla a mantener. Indirectamente, cuando se habla de este tipo de variables se está haciendo referencia a la columna cuyo nombre sea igual al de la tabla a mantener. Respecto a los atributos de CTL asignados a cada columna, éstos son extraídos automáticamente del diccionario de la base de datos e incluidos en la definición de cada variable. Su sintaxis es la siguiente:

```
{identificador = | UNTAGGED} [tabla.] columna [LINES n] [lista_atributos]
```

La cláusula «LINES n» indica el número de líneas en las que se mostrará la variable, y sólo podrá emplearse con columnas de tipo CHAR. La «lista\_atributos» es una lista de atributos de CTL separados por un espacio en blanco. Para más información acerca de los atributos del CTL consulte el capítulo sobre [«Conceptos Básicos del CTL»](#) de este mismo volumen.

#### Ejemplo 5 [PFORM5]:

```
database almacen

define
 form
 screen
{
 Código.....: [a]
 Descripción.....: [b]
}
```

```

end screen

tables
 provincias
end tables

variables
 a = provincias.provincia required
 b = provincias.descripcion upshift
 untagged provincias.prefijo
end variables
end form

variable i smallint
end define

main begin
 get query
 if numrecs <> 0 then begin
 pause "Prefijo: " && prefijo using "###"
 && ". Pulse [Return] para continuar"
 for i = 2 to numrecs begin
 next row
 pause "Prefijo: " && prefijo using "###" &&
 ". Pulse [Return] para continuar"
 end
 end
end
end main

```

En el ejemplo anterior, en el objeto FORM se han definido las tres columnas de la tabla de «provincias», si bien el «prefijo» no se mostrará nunca en pantalla (SCREEN) al no existir un identificador para representarlo. La columna «prefijo» se ha definido con la cláusula «UNTAGGED». El programa realiza una consulta de todas las filas de la tabla «provincias». Dicha consulta se provoca mediante la instrucción GET QUERY del CTL, la cual, al no incluir ninguna condición (cláusula «WHERE») lee todas las filas de la tabla. Aunque los valores de la columna «prefijo» no aparecen junto a las otras dos («provincia» y «descripcion»), dicha variable adquiere valores según se van leyendo las filas de la lista en curso mediante la instrucción NEXT ROW. Estos valores se muestran mediante la instrucción PAUSE. La variable interna de CTL «numrecs» contiene el número de filas que componen la lista en curso, por lo cual, cuando se muestre la última fila finalizará el programa.

#### IMPORTANTE

El orden de definición de las variables en la sección VARIABLES determina la secuencia de movimiento del cursor en la pantalla de la fila en edición.

Los nombres de columna referenciados en la sección VARIABLES mediante el atributo LOOKUP del CTL no necesitan incluir la definición de la tabla enlazada en la sección TABLES. Como ya se ha indicado, en dicha sección TABLES sólo tendrán que aparecer los nombres de las tablas cuyas columnas se van a mantener en el FORM. Las columnas referenciadas en el atributo LOOKUP sólo se muestran por enlace simple con otra tabla.

#### Ejemplo 6 [PFORM6]. El atributo LOOKUP:

```

database almacén

define
 form
 screen
{
 Código: [a]
 Empresa: [b]
 Dirección.....: [c]
 Provincia: [d |e]
}

```

```

 }

 end screen

 tables
 clientes
 end tables

 variables
 a = clientes.cliente required
 b = clientes.empresa upshift
 c = clientes.direccion1
 d = clientes.provincia
 lookup e = descripcion
 joining provincias.provincia
 end variables
 end form
end define

main begin
 menu form
end main

```

Este programa define un objeto FORM para el mantenimiento de la tabla «clientes». En él se incluyen sólo aquellas columnas que se van a mantener: «cliente», «empresa», «direccion1» y «provincia». La descripción de la provincia («descripcion») se muestra por medio del atributo LOOKUP, que será el encargado del cruce entre las tablas «clientes» y «provincias» por medio de los códigos de las provincias existentes en ambas («provincia»). Este atributo realizará el cruce de la provincia en cualquiera de las operaciones básicas de un mantenimiento: altas, bajas (quedará vacío el cruce), modificaciones y consultas.

La definición de este tipo de variables conlleva una modificación cuando se trata de un enlace de columnas de dos tablas unidas mediante la sección JOINS del FORM. La sintaxis empleada para la definición de dos columnas enlazadas de dos tablas diferentes es la siguiente:

```

{identificador = | UNTAGGED} [*] [tabla.] columna
 [lista_atributos;]
 = [*] [tabla.] columna [lista_atributos][:]
 [= [*] [tabla.] columna [lista_atributos] [: ...]]

```

Como diferencia respecto a la definición de una columna sin enlace con otra tabla, en este caso no puede emplearse la cláusula «LINES n». Asimismo, si una columna incluye atributos («lista\_atributos») al final de su definición se tiene que incluir un punto y coma. Este tipo de definición implica la utilización de un único identificador para más de una columna de mantenimiento pertenecientes a diferentes tablas.

### Ejemplo 7 [PFORM7]. Enlace de columnas para el JOIN de dos tablas:

```

database almacen

define
 form
 screen
{
 Cod. Unidad.....: [a]
 Unidades.....: [f2]

 Cod. Art Descripcion
 [f3] [f4]

}

end screen

```



```

tables
 unidades
 articulos 5 lines
end tables

variables
 a = unidades.unidad = articulos.unidad required
 f2 = unidades.descripcion
 f3 = articulos.articulo required
 f4 = articulos.descripcion
end variables

joins
 articulos lines of unidades
end joins
end form
end define

main begin
 menu form
end main

```

Este programa enlaza dos tablas («unidades» y «articulos») de la base de datos «almacen» en un objeto FORM. El programa invoca al FORM mediante la instrucción MENU FORM, la cual, al no disponer de un menú propio, mostrará el menú por defecto. En este caso, la opción «Tabla» de dicho menú es significativa, ya que nos permitirá seleccionar una tabla u otra como tabla en curso. El enlace de ambas tablas se realiza en la sección VARIABLES al definir el identificador «a», que representa el código de unidades. Asimismo, dentro de este ejemplo hemos incluido una sección nueva, la JOINS, que se explicará más adelante dentro de este capítulo. Esta sección determina la relación entre ambas tablas. En este FORM tampoco se mantienen todas las columnas de la tabla «articulos».

Esta definición de variables de enlace entre tablas del FORM tendrá que repetirse tantas veces como columnas comunes haya de enlace entre ellas. Nunca podrán enlazarse dos columnas de tipo de dato SERIAL, éstas sólo podrán enlazar con otras de tipo INTEGER. Los asteriscos que se pueden definir antes de cada columna de enlace fijan la columna dominante. Por ejemplo, en el programa anterior, si se antepone el asterisco a la columna «unidad» de la tabla «unidades», significará que en caso de intentar borrar una fila de la tabla «unidades» que tenga referencia en la tabla «articulos» el programa producirá un error. Este error es anterior al provocado por la integridad referencial de ambas tablas, si es que la hubiese.

#### IMPORTANTE

En caso de emplear el atributo LOOKUP en la definición de una columna de enlace entre dos tablas unidas mediante la sección JOINS habrá que definirlo en la «lista\_atributos» referida a la columna de la tabla de cabecera. Si se define dicho atributo en «lista\_atributos» de la columna perteneciente a la tabla «líneas», provocará en ejecución un «scroll» del dato de cruce de tantas líneas como se mantenga dicha tabla («x lines»).

La referencia a una de estas variables se realiza mediante el nombre de la variable o bien precediendo el nombre de la tabla a la variable:

```

"variable"
"tabla.variable"

```

## Variables PSEUDO

Las variables PSEUDO son componentes de la tabla del FORM como una columna más, excepto que no están asociadas directamente a una columna de la base de datos, al igual que ocurría con el tipo de variables comentadas en el apartado anterior.

Estas variables son de entrada/salida, es decir, muestran en pantalla en el lugar reservado para su identificador el valor que se les asigne mediante cualquier instrucción CTL de asignación. Asimismo, permiten la entrada de valores a través del teclado como si se tratase de una columna más a mantener en la tabla activa. Respecto al «scroll», estas variables adquieren el mismo que la «tabla» del FORM con la que se define. Es decir, si la tabla del FORM tiene o no «scroll» de líneas, la variable PSEUDO tendrá el mismo tipo. Ésta es la característica más significativa de este tipo de variables, y la que las diferencia de las variables VARTABLE y DISPLAYTABLE, que se explican en apartados posteriores. Su sintaxis es la siguiente:

```
{identificador = | UNTAGGED} PSEUDO tabla.variable {tipo_sql | LIKE tabla.columna}
[LINES n] [lista_atributos]
```

La referencia a estas variables es mediante su nombre o bien anteponiéndole el nombre de la tabla y un punto («tabla.variable»).

### Ejemplo 8 [PFORM8]. Funcionamiento de las variables PSEUDO:

```
database almacen

define
 form
 screen
{
!
 Cliente Empresa Facturación
 ^ ^
 [a] [b] [c]

 @
}

end screen

tables
 clientes 6 lines
end tables

variables
 a = clientes.cliente required
 b = clientes.empresa
 c = pseudo clientes.facturado money(16,2)
 format "##,###,##&.&&"
end variables

control
 after display of clientes
 select sum (cantidad * precio * (1 - descuento / 100))
 into facturado
 from albaranes, lineas
 where albaranes.cliente = $clientes.cliente
 and albaranes.albaran = lineas.albaran
 if found = false then let facturado = 0
 end control
end form
end define
```

```
main begin
 get query
 menu form
end main
```

Este programa declara un FORM para el mantenimiento de la tabla «clientes» con «scroll» de líneas. Es decir, cada vez que se consulten «clientes» podrá aparecer un máximo de 6 filas. En este mantenimiento se define una variable PSEUDO para mostrar la facturación de cada uno de los clientes consultados. En primer lugar, el programa consulta todos los clientes existentes en la tabla «clientes», mostrando a continuación el menú por defecto del FORM, que nos permitirá desplazarnos por la lista en curso generada por dicha consulta. Por cada fila mostrada en pantalla (DISPLAY) se ha realizado una lectura sobre las tablas «albaranes» y «lineas» de albarán para calcular el total facturado por cada cliente. El resultado de esta lectura se carga sobre la variable PSEUDO definida, es decir, ésta es receptora en la instrucción SELECT. Aquellos clientes que no hayan realizado compras aparecerán con el valor cero («0,00»). En caso de que el operador decida realizar otra consulta se repetirá el ciclo anterior por cada fila seleccionada, finalizando el programa al abandonar el menú por defecto del FORM.

Como puede comprobarse en la ejecución de este programa, la variable PSEUDO «facturado» se muestra con «scroll» de líneas, igual que la tabla a la que ha sido asignada en la definición «clientes».

Para anular la característica de INPUT de este tipo de variables basta con asignarle en su definición los atributos NOENTRY y NOUPDATE. En el ejemplo anterior, esta característica se programaría de la siguiente forma:

```
c = pseudo clientes.facturado money(16,2) format "###,###,##&.&&"
noentry noupdate
```

En este caso, la variable «facturado» no podrá editarse ni en altas (opción «Añadir») ni en modificaciones (opción «Modificar») ni en consultas (opción «Encontrar») de filas sobre la tabla «clientes». Este mismo efecto puede conseguirse mediante la ejecución de la instrucción NEXT FIELD en una instrucción no procedural de la sección EDITING (se explica más adelante en este mismo capítulo).

El orden de definición de una variable de tipo PSEUDO dentro de la sección VARIABLES del FORM tiene sentido cuando se va a realizar una entrada de valores (INPUT) sobre ella. Dependiendo del lugar que ocupe dentro de esta sección, dicha variable se introducirá después de las que la preceden, si es que existe alguna.

## Variables VARIABLE

Estas variables son de «sólo DISPLAY» e internamente constituyen una tabla ficticia denominada VARIABLE. Esta tabla no pertenece a la lista de tablas del FORM, y sus variables sólo servirán para la representación de expresiones por pantalla. En el momento que una variable VARIABLE adquiere un valor, éste se muestra automáticamente en la pantalla en el lugar asignado para el identificador de la sección SCREEN que la representa. Su sintaxis es la siguiente:

```
{identificador = | UNTAGGED} VARIABLE.variable {tipo_sql | LIKE tabla.columna}
[LINES n] [lista_atributos]
```

### IMPORTANTE

Una variable VARIABLE no admite la presentación de valores con «scroll» de líneas. Este tipo de «scroll» en variables del FORM sólo es admitido por las PSEUDO y las propias variables de mantenimiento de la tabla.

En caso de definir una variable de tipo VARIABLE como «UNTAGGED» significará que se está definiendo una variable de programación que sólo podrá emplearse en el FORM. A fin y al cabo, una VARIABLE es una variable de programación que, en el momento que adquiere un valor, lo muestra en pantalla, si es que tiene identificador en la sección SCREEN.

La referencia a este tipo de variables se realiza mediante el nombre de la propia variable. No obstante, en caso de existir ambigüedad con otra variable del módulo se podrá emplear la palabra reservada VARTABLE seguida de un punto y precediendo al nombre de dicha variable («VARTABLE.variable»).

### Ejemplo 9 [PFORM9]. Funcionamiento de las variables de tipo VARTABLE:

```
database almacen

define
 form
 screen
{
! C l i e n t e s
 ?
 Código Cliente:[a]
 $
 Empresa: [b]
 Direccion1: [c]
 Provincia: [d] [e]

 %
 F a c t u r a d o: [f]
 *

 @
}
end screen

tables
 clientes
end tables

variables
 a = clientes.cliente required
 b = clientes.empresa
 c = clientes.direccion1
 d = clientes.provincia lookup e = descripcion
 joining provincias.provincia
 f = vartable.facturado money(16,2)
 format "##,###,##&.&&"
end variables
control
 after display of clientes
 select sum (cantidad * precio * (1 - descuento / 100))
 into facturado
 from albaranes, lineas
 where albaranes.cliente = $clientes.cliente
 and albaranes.albaran = lineas.albaran
 if found = false then let facturado = 0
 end control
end form
end define

main begin
 get query
 menu form
end main
```

Este programa emplea la variable de tipo VARTABLE «facturado» para mostrar el total facturado de cada uno de los clientes consultados. Este ejemplo es similar al anterior, pero sin «scroll» de líneas, ya que las variables de tipo VARTABLE no pueden utilizarlo. El «scroll» empleado por este programa también puede ser simulado por las variables PSEUDO. La diferencia entre ambos tipos de variables es que la VARTABLE sólo muestra la in-

formación de lo facturado por cada cliente, mientras que la PSEUDO permite su edición, salvo que se indiquen los atributos NOENTRY y NOUPDATE. Su definición sería de la siguiente forma:

```
f = pseudo clientes.facturado money(16,2)
format "##,###,##&.&&" noentry noupdate
```

El cálculo del total facturado por cada cliente se realiza cada vez que se presenta un nuevo cliente en pantalla, es decir, después de cada repintado de la misma (instrucción no procedural AFTER DISPLAY de la sección CONTROL —se explica más adelante en este capítulo—).

Como ya se ha indicado anteriormente, el atributo LOOKUP sólo permite los cruces entre tablas que tengan como enlace una sola columna. En caso de querer realizar un cruce con otra tabla en el que intervenga más de una columna, dicho atributo no será válido. No obstante, mediante la utilización de variables de tipo VARTABLE podremos simular su ejecución. Así, por ejemplo, el programa anterior sin atributo LOOKUP en la columna «provincia» quedaría tal y como se expone en el Ejemplo 10 siguiente.

#### Ejemplo 10 [PFORM10]:

```
database almacén

define
 form
 screen
{
! C l i e n t e s
 ?
 Código Cliente:[a]
 $
 Empresa: [b]
 Direccion1: [c]
 Provincia: [d][e]

 %
 F a c t u r a d o: [f]
 *

 @
}

end screen

tables
 clientes
end tables

variables
 a = clientes.cliente required
 b = clientes.empresa
 c = clientes.direccion1
 d = clientes.provincia
 e = vartable.descripcion char(20)
 f = vartable.facturado money(16,2)
 format "##,###,##&.&&"
end variables

control
 after display of clientes
 select sum (cantidad * precio * (1 - descuento / 100))
 into facturado
 from albaranes, lineas
 where albaranes.cliente = $clientes.cliente
 and albaranes.albaran = lineas.albaran
 if found = false then let facturado = 0
```

```
 call cruce()
 end control

 editing
 after editadd editupdate of clientes.provincia
 call cruce()
 end editing
 end form
end define

main begin
 get query
 menu form
end main

function cruce() begin
 select descripcion by name from provincias
 where provincias.provincia = $clientes.provincia
 end function
```

El funcionamiento de este programa es idéntico al del Ejemplo 9, salvo que ahora no se ha utilizado el cruce LOOKUP. Dicho cruce se ha simulado mediante una variable VARTABLE y la carga de ésta se realiza mediante una instrucción SELECT en la que se podrán indicar las condiciones necesarias, siendo ésta la limitación del atributo LOOKUP.

## Variables DISPLAYTABLE

Estas variables son tanto de DISPLAY como de INPUT, e internamente componen una tabla ficticia denominada «displaytable». Los campos de esta tabla ficticia no tienen ninguna relación con columnas de la base de datos. Esta tabla es añadida automáticamente a la lista de tablas del FORM, es decir, no es necesario indicar el nombre «displaytable» en la sección TABLES. La activación de esta tabla ficticia se realiza mediante la instrucción NEXT TABLE del CTL, y su manejo por parte del FORM es idéntico al de una tabla de la base de datos. Su sintaxis de definición es la siguiente:

```
{identificador = | UNTAGGED} DISPLAYTABLE.variable {tipo_sql | LIKE tabla.columna}
[LINES n] [lista_atributos]
```

La forma de referenciar este tipo de variables es mediante su nombre o bien anteponiéndole el nombre de la tabla ficticia «displaytable» y un punto («DISPLAYTABLE.variable»).

Este tipo de variables se asemeja a las PSEUDO. No obstante, el conjunto de variables DISPLAYTABLE conforma una fila de una tabla ficticia, mientras que una variable PSEUDO simula una columna más de la tabla de la base de datos con la que se ha definido. Asimismo, para activar las variables DISPLAYTABLE de entrada hay que cambiar de tabla activa. Por último, estas variables, al igual que sucedía con las VARTABLE, no permiten emplear el «scroll» de líneas manejado por el FORM para el mantenimiento de tablas. Las únicas variables que permiten dicho «scroll» de líneas son las propias variables del FORM asociadas a columnas de la tabla a mantener y las variables de tipo PSEUDO.

Por otra parte, y por lo que respecta a la presentación en pantalla, la única diferencia entre las variables de tipo DISPLAYTABLE y las VARTABLE es que las primeras presentan sus valores subrayados, mientras que las segundas no. Teniendo en cuenta esta característica, si en el Ejemplo 10 del apartado anterior hubiésemos empleado una variable DISPLAYTABLE en lugar de una VARTABLE para la representación del cruce, la única diferencia sería que el resultado aparecería subrayado. Para ello, habría que sustituir en el ejemplo citado la línea:

```
e = vartable.descripcion char(20)
```

por la siguiente:

```
e = displaytable.descripcion char(20)
```

El Ejemplo 11 muestra la forma de introducir valores sobre variables DISPLAYTABLE.

#### Ejemplo 11 [PFORM11].

```

database almacen

define
 variable optimiza char(300)

 form
 screen
 {
 ! C l i e n t e s Pag.: 1
 ?
 Código Cliente[a]
 $
 Empresa: [b]
 Direccion1: [c]
 Provincia: [d] [e]

 @
 }
 screen
 {
 ! C l i e n t e s Pag.: 2
 Desde: [f]
 Hasta: [g]

 @
 }
 end screen

 tables
 clientes
 end tables
 variables
 a = clientes.cliente required
 b = clientes.empresa
 c = clientes.direccion1
 d = clientes.provincia lookup e = descripcion
 joining provincias.provincia
 f = displaytable.desde integer
 g = displaytable.hasta integer
 check ($ >= desde)
 end variables
 menu m1 "Mantenimiento"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Borrado Masivo"
 next table "displaytable"
 add one
 next table "clientes"
 if cancelled = false then begin
 delete from clientes
 where clientes.cliente between $desde and $hasta
 call fgetquery(optimiza)
 end
 option "Consulta"
 let optimiza = fquery()

```

```
 call fgetquery(optimiza)
 end menu
end form
end define

main begin
 menu form
end main
```

Este programa realiza el mantenimiento de la tabla «clientes», para lo cual construye un menú con las opciones apropiadas. Este menú está representado por la sección MENU del FORM —que se explica más adelante en este capítulo—, e incluye las siguientes opciones:

- «Añadir»: Ejecuta la instrucción ADD del CTL, que internamente equivale a la INSERT del SQL, pero para un número indeterminado de filas.
- «Modificar»: Ejecuta la instrucción MODIFY, equivalente a la UPDATE del SQL.
- «Borrar»: Ejecuta la instrucción REMOVE, equivalente a la DELETE del SQL.
- «Consulta»: Ejecuta dos funciones internas de CTL. La primera, «fquery()», devuelve las condiciones establecidas por el operador que se recogen sobre la variable «optimiza», y la segunda, «fgetquery()», realiza la consulta para dichas condiciones. Este procedimiento se utiliza para poder realizar de nuevo la consulta sin tener que introducir otra vez las mismas condiciones después de un borrado masivo.
- «Borrado Masivo»: Esta opción se encarga de activar las dos columnas de la tabla ficticia «displaytable» («desde» y «hasta»), desactivando la tabla «clientes» y permitiendo introducir un valor sobre dichas columnas (instrucción ADD ONE). Al activar la tabla ficticia «displaytable» se cambia automáticamente la pantalla, pasando a la página 2. Una vez aceptados los valores introducidos en «desde» y «hasta» se vuelve a activar la tabla «clientes», borrándose las filas que se encuentren en el rango indicado mediante la instrucción DELETE del SQL. Si se borra alguna fila de la tabla «clientes» que previamente hubiese consultado el usuario, al moverse por la lista en curso se producirá un aviso («warning»), indicando que se ha producido el borrado de una fila componente de dicha lista. Para evitar esto se realiza de nuevo la consulta, ejecutando la función interna de CTL «fgetquery()» con las mismas condiciones establecidas en la consulta inicial. En caso de no haber realizado una consulta inicial, al finalizar el borrado masivo de filas se consultarán todas las filas de la tabla de «clientes».

## Resumen de la sección VARIABLES

Dependiendo del tipo de variables que se vayan a definir, la sección VARIABLES admite la siguiente sintaxis:

### VARIABLES

```
{identificador = | UNTAGGED} [tabla.] columna [LINES n]
[lista_atributos]
```

```
{identificador = | UNTAGGED} [*] [tabla.] columna
[lista_atributos;]
= [*] [tabla.] columna [lista_atributos][:]
[= [*] [tabla.] columna [lista_atributos]
[: ...]]
```

```
{identificador = | UNTAGGED} PSEUDO tabla.variable
{tipo_sql | LIKE tabla.columna} [LINES n]
[lista_atributos]
```

```
{identificador = | UNTAGGED} VARTABLE.variable
{tipo_sql | LIKE tabla.columna} [LINES n]
[lista_atributos]
```

```
{identificador = | UNTAGGED} DISPLAYTABLE.variable
{tipo_sql | LIKE tabla.columna} [LINES n]
[lista_atributos]
```

### END VARIABLES



## Operaciones a realizar con un FORM: Sección MENU

Siempre que no se defina la sección MENU en un FORM, éste dispondrá de un menú por defecto que incluirá las siguientes opciones:

- «Añadir»: Inserta nuevas filas sobre la tabla activa.
- «Modificar»: Permite modificar la fila en curso.
- «Borrar»: Borra la fila en curso.
- «Encontrar»: Consulta las filas de la tabla activa, inicializando de esta forma la lista en curso.
- «Tabla»: Selecciona otra tabla a mantener en el FORM. Las tablas que podrán seleccionarse deberán incluirse dentro de la sección TABLES.
- «Salida»: Permite listar el aspecto de la pantalla del FORM con la fila en curso o bien con todas las filas de la lista en curso. El listado puede realizarse hacia un «fichero» o hacia un «programa».
- «Insertar»: Permite insertar nuevas filas sobre la tabla en curso. Similar a «Añadir», su utilización sólo tiene sentido cuando se está manteniendo una tabla sobre un FORM con «scroll» de líneas.

Las instrucciones CTL ejecutadas por cada una de estas opciones son las siguientes:

| RELACION OPCIONES-INSTRUCCIONES |                          |
|---------------------------------|--------------------------|
| OPCIÓN                          | INSTRUCCIÓN CTL          |
| Añadir                          | ADD                      |
| Borrar                          | REMOVE                   |
| Modificar                       | MODIFY                   |
| Encontrar                       | QUERY                    |
| Tabla                           | NEXT TABLE, LINES o HEAD |
| Salida                          | OUTPUT                   |
| Insertar                        | INTERCALATE              |

Este menú por defecto cambiará en el momento que se defina la sección MENU del FORM. Dicha sección incluirá un menú CTL compuesto por todas las opciones que se podrán llevar a cabo con el FORM. La estructura y sintaxis de la sección MENU del FORM es idéntica a la que pueda tener cualquier otro menú estándar del CTL (para más información sobre los distintos tipos de menús que pueden definirse en CTL [consulte el capítulo 14](#) de este mismo manual).

La sintaxis de la sección MENU del FORM (válida también para cualquier menú del CTL) es la siguiente:

```
[MENU nombre_menú [opciones]
 OPTION [nombre_opción] ["Etiqueta"] [opciones]
 ...
 OPTION [nombre_opción] ["Etiqueta"] [opciones]
 ...
 [OPTION ...
 ...]
END MENU]
```

Dependiendo de las «opciones» indicadas a dicho menú, éste será de tipo «Lotus» (de iconos en Windows) o bien «Pop-up» (cláusula «DOWN»).

La referencia a un menú del objeto FORM se realiza con la instrucción MENU FORM del CTL. Esta instrucción es la encargada de mostrar en pantalla el aspecto de la SCREEN definida, así como todas las opciones de que se compone el menú especificado en la sección MENU. Por ejemplo:

```
main begin
 menu form
end main
```

No obstante, la ejecución de esta instrucción puede simularse también con las siguientes:

```
main begin
 display form
 menu nombre_menu
 clear form
end main
```

En este segundo caso, «nombre\_menu» es un menú que puede estar definido tanto en la sección MENU del FORM como en la sección MENU de un módulo CTL, es decir, después de la sección MAIN.

El menú de un FORM emplea una serie de acciones o teclas para manejar la lista en curso. Estas acciones no podrán detectarse con las instrucciones no procedurales «ON ACTION» y «ON KEY» en la sección CONTROL del FORM. Por ejemplo, para pasar a la siguiente fila de la lista en curso hay que pulsar la tecla asignada a la acción <fdown>. El cuadro de la página siguiente muestra algunas de las acciones más comunes utilizadas en un menú del FORM.

| ACCIONES EMPLEADAS EN EL MENU DEL FORM |                                     |
|----------------------------------------|-------------------------------------|
| ACCIÓN                                 | OPERACIÓN QUE REALIZA               |
| <fdown>                                | Siguiente fila                      |
| <fup>                                  | Fila Anterior                       |
| <ftop>                                 | Primera fila                        |
| <fbottom>                              | Última fila                         |
| <fright>                               | Opción derecha del menú             |
| <fleft>                                | Opción izquierda del menú           |
| <fbackspace>                           | Opción izquierda del menú           |
| <fnext-page>                           | Próxima SCREEN                      |
| <fprevious-page>                       | SCREEN anterior                     |
| <fright-page>                          | Página derecha («scroll» lateral)   |
| <fleft-page>                           | Página izquierda («scroll» lateral) |
| <freturn>                              | Selección de una opción del menú    |
| <faccept>                              | Selección de una opción del menú    |
| <fquit>                                | Abandonar el menú                   |
| <fredraw>                              | Repintado de la pantalla            |
| etc.                                   |                                     |

Un menú CTL puede configurarse mediante un mensaje de un fichero de ayuda. La forma de configurar este tipo de menús se explica en el capítulo 16 de este mismo manual.

### Ejemplo 12 [PFORM12]:

```
database almacén

define
 form
 tables
 clientes
 end tables
 menu m1 "Mantenimiento"
 option "Añadir" key "d" comments "Añade filas a la tabla de clientes"
 add
 option "Modificar" key "r" comments "Modifica la fila en curso"
 modify
 option "Borrar" comments "Borra la fila en curso"
 remove
 option "Consultar" key "t" comments "Petición de condiciones para la consulta"
 query
 end menu
```

```

 end form
 end define

 main begin
 menu form
 end main

```

Este programa define una sección MENU del FORM para el mantenimiento de la tabla «clientes». El menú dispone de las cuatro opciones básicas para el mantenimiento de una tabla: Altas, Bajas, Modificaciones y Consultas. La forma de activar dicho menú es mediante la instrucción MENU FORM del CTL. Hasta ahora, todos los ejemplos expuestos sobre el FORM se han activado mediante esta instrucción, apareciendo en consecuencia el menú por defecto.

En este caso, se ha configurado un menú de tipo «Lotus» (o de iconos en Windows). La cláusula «KEY» de cada opción sirve para indicar qué letra del nombre de la opción servirá para activarla (por ejemplo, la opción «Añadir» se podrá activar con la letra «d»). También podrá activarse una opción posicionándose sobre ella y pulsando [RETURN]. El comentario («comments») de cada opción muestra un breve cartel indicando la función que se ejecutará.

## Control de procesos: Secciones CONTROL y EDITING

El FORM dispone de dos secciones (CONTROL y EDITING) en las que, mediante instrucciones no procedurales, se sitúan los procesos (conjuntos de instrucciones CTL) a realizar en determinados momentos de la interacción del programa con el usuario. La forma de interactuar con el usuario dependerá directamente de la operación que se esté llevando a cabo en el proceso de mantenimiento (añadir, borrar, modificar, repintar, etc.).

La sección CONTROL contiene instrucciones no procedurales que se activan antes o después de realizar una de las operaciones indicadas con una fila de la lista en curso. La estructura de esta sección es la siguiente:

```

[CONTROL
 [{(BEFORE | AFTER) [{(ADD) [QUERY] [UPDATE] [DELETE] [DISPLAY]
 [LINE]}] OF] lista_tablas
 ...]
[AFTER CANCEL OF] lista_tablas
 ...]
[{(BEFORE | AFTER) lista_tablas
 ...]
[ON KEY "tecla"
 ...]
[ON ACTION "acción"
 ...]
END CONTROL]

```

Por su parte, la sección EDITING está compuesta por instrucciones no procedurales que se activan antes o después de editar (operaciones añadir, modificar y consultar) cada una de las variables del FORM pertenecientes a la tabla a mantener. Asimismo, existen algunas instrucciones no procedurales que se activan antes o después de realizar una operación (altas, modificaciones y consultas) con una fila. La sintaxis de la sección EDITING es la siguiente:

```

[EDITING
 [{(BEFORE | AFTER) [{(EDITADD) [EDITUPDATE] [EDITQUERY]}] OF]
 {lista_tablas | lista_columnas}
 ...]
[BEFORE CANCEL OF] lista_tablas
 ...]
[ON KEY "tecla"
 ...]
[ON ACTION "acción"
 ...]

```

...]  
**END EDITING]**

Tanto en la sección CONTROL como en la EDITING existen instrucciones no procedurales que se activan al pulsar una tecla («ON KEY») o una acción («ON ACTION») incluida en el fichero de acciones «tactions». La sección CONTROL podrá activar dichas instrucciones no procedurales al pulsar la tecla o acción en cualquiera de las opciones del menú del FORM. Sin embargo, en la sección EDITING dichas instrucciones se activarán cuando el operador las active en edición (operaciones de añadir, modificar y consultar). En ningún caso se podrán detectar teclas que se empleen por defecto en el manejo tanto del menú como en edición de campos.

En las páginas que siguen se explican las distintas acciones que podrán realizarse en estas dos secciones en función de las operaciones que esté llevando a cabo el usuario.

### Grabación de filas nuevas

La grabación de filas nuevas sobre la tabla activa en el FORM se produce cuando se ejecutan las instrucciones ADD o ADD ONE o INTERCALATE del CTL. La ejecución de ambas instrucciones supone la inicialización de todas las variables del FORM asociadas a columnas de la tabla activa con su valor por defecto (éste puede asignarse mediante el atributo DEFAULT del CTL y el CTSQL). La diferencia entre ADD y ADD ONE consiste en que la primera construye un bucle para la grabación de un número indeterminado de filas hasta que el operador cancele la edición (acción <fquit>), mientras que la segunda permite únicamente la grabación de una sola fila. En caso de no haber definido dicho atributo para una columna, independientemente del tipo de dato con el que se definió en el diccionario de la base de datos, su valor por defecto será «NULL». Al aceptar los valores introducidos en cada variable del FORM, éstos se graban internamente como una fila de la tabla que se está manteniendo mediante la ejecución de la instrucción INSERT del CTSQL. En consecuencia, la instrucción ADD es equivalente a un bucle INSERT del CTSQL, mientras que la ADD ONE equivale a una sola instrucción INSERT.

La secuencia de ejecución de las instrucciones no procedurales de las secciones CONTROL y EDITING es la siguiente:

- 1.— «AFTER DISPLAY OF tabla» (sección CONTROL). Al ejecutar cualquiera de las instrucciones ADD, ADD ONE o INTERCALATE se limpian los valores que tuviesen las variables del FORM.
- 2.— «BEFORE EDITADD OF tabla» (sección EDITING). Ejecución antes de entrar en edición de los campos.
- 3.— «BEFORE EDITADD OF columna» (EDITING). Ejecución antes de activar el campo del FORM asociado a «columna».
- 4.— «AFTER EDITADD OF columna» (EDITING). Esta instrucción se ejecutará al abandonar el campo del FORM asociado a «columna».
- 5.— «AFTER EDITADD OF tabla» (EDITING). Esta instrucción se ejecuta cuando el operador pulsa la tecla asignada a la acción <faccept> pero la fila aún no ha sido grabada.
- 6.— «BEFORE ADD OF tabla» (CONTROL). Todavía no se ha grabado la fila. Equivale a la instrucción anterior pero en la sección CONTROL.
- 7.— «AFTER DISPLAY OF tabla» (CONTROL). Limpia los valores introducidos sobre las variables del FORM asociadas a columnas de la base de datos (ADD).
- 8.— «AFTER ADD OF tabla» (CONTROL). Instrucción que se activa una vez que los valores introducidos sobre las variables del FORM se graban en la tabla de la base de datos que se está manteniendo.
- 9.— Comienza de nuevo por el punto 1, siempre y cuando se esté utilizando la instrucción ADD y no la ADD ONE.

Por su parte, al cancelar la edición se activan las siguientes instrucciones no procedurales:

1.— «BEFORE CANCEL OF tabla» (EDITING). Esta instrucción se ejecuta después de pulsar la tecla asignada a la acción <fquit>, existiendo la posibilidad, incluso, de cancelar dicha operación y continuar con la edición hasta que se cumpla una condición.

2.— «AFTER CANCEL OF tabla» (CONTROL). Se ejecuta después de abandonar la edición.

3.— «AFTER DISPLAY OF tabla» (CONTROL). Limpia los valores introducidos sobre las variables del FORM asociadas a columnas de la base de datos y que no han sido grabados en la tabla.

Dependiendo de la tecla pulsada por el operador, la activación de instrucciones se consigue mediante las instrucciones no procedurales «ON KEY» y «ON ACTION» de las secciones CONTROL y EDITING.

En la sección EDITING, dichas instrucciones se activarán, cuando se intenta realizar un alta, en el momento en que el operador pulse la tecla correspondiente a la acción indicada en la instrucción (cuando el cursor se encuentre situado sobre alguno de los campos de la SCREEN).

En el caso de las instrucciones de la sección CONTROL, éstas se activarán cuando no estemos en edición, es decir, cuando podamos movernos sobre las distintas opciones del menú del FORM o bien cuando se ejecute la instrucción «READ KEY THROUGH FORM».

Las instrucciones no procedurales «ON KEY» y «ON ACTION» sólo podrán detectar aquellas teclas que no tengan asignada ya una acción en el propio objeto sobre el que se detectan. Por ejemplo, nunca se podrá detectar la tecla correspondiente a la acción <freturn>, ya que dicha tecla se emplea tanto en el menú del FORM como en edición para seleccionar una opción del menú (sección CONTROL) y para aceptar un valor en un campo del FORM activando a su vez el siguiente campo (sección EDITING).

**Ejemplo 13 [PFORM13]. Instrucciones no procedurales que se ejecutan en la operación de «Añadir» nuevas filas sobre la tabla activa del FORM:**

```
database almacen

define
 variable contador integer default 0
 variable maximo integer

 form
 screen
{
 ! C l i e n t e s

 ?
 Código Cliente: [a]
 $

 Empresa: [b]
 Nombre: [c]
 Apellidos: [d]
 Direccion1: [e]
 Provincia: [f] [g]
 Telefono: [h] [i]

 @
}
end screen

tables
 clientes
end tables

variables
 a = clientes.cliente
```

## MultiBase. Manual del Programador

```
b = clientes.empresa
c = clientes.nombre
d = clientes.apellidos
e = clientes.direccion1
f = clientes.provincia lookup g = descripcion, h = prefijo joining provincias.provincia
i = clientes.telefono
end variables

control
 after add of clientes
 window from select cliente, empresa
 from clientes
 where clientes.cliente = $clientes.cliente
 10 lines label "Cliente grabado"
 at 5,25
 let contador = contador + 1
 after cancel of clientes
 pause "TOTAL Clientes grabados: " &
 contador using "##,####"
end control

editing
 after editadd of clientes
 select max(cliente) into maximo from clientes
 if maximo is null then let maximo = 0
 let clientes.cliente = maximo + 1
 before cancel of clientes
 if contador = 0 then begin
 pause "Debe grabarse al menos una fila." &&
 " Pulse [Return] para continuar" reverse bell
 retry
 end
 before editadd of cliente
 next field "empresa"
 next field down "empresa"
 next field up "telefono"
 after editadd of provincia
 select "literal" from provincias
 where provincias.provincia = $clientes.provincia
 if found = false then begin
 if yes("Provincia inexistente. Desea darla de alta", "Y") = true then ctl "provincias"
 next field "provincia"
 end
 before editadd of provincia
 message "Pulsar " && keylabel("fuser1") clipped &&
 " para consultar provincias" reverse
 view
 on action "fuser1"
 if curfield = "provincia" then
 window from select provincia, descripcion
 from provincias
 10 lines label "P r o v i n c i a s"
 at 5,30
 set clientes.provincia from 1
 end editing
 end form
end define

main begin
 add
end main
```

Este programa insertará nuevas filas en la tabla «clientes». El programa sólo tiene la instrucción ADD en la sección MAIN, por lo que el mantenimiento carecerá de menú. En primer lugar, al situarse el cursor sobre la columna «cliente» mediante la instrucción «BEFORE EDITADD OF CLIENTE», se redirecciona a la anterior («teléfono») o posterior («empresa») mediante la instrucción NEXT FIELD, dependiendo de la última tecla pulsada. Es decir, mediante esta instrucción se impide el posicionamiento del cursor en dicho campo.

A continuación, la siguiente instrucción a ejecutar cuando se llegue a dicho campo es «BEFORE EDITADD OF PROVINCIA». Esta instrucción no procedural mostrará un mensaje en pantalla indicando la tecla que debe emplear el usuario en caso de querer consultar y elegir algún código de provincias existente en la tabla maestra de «provincias». En caso de pulsar dicha tecla en el campo «provincia» se ejecutará la instrucción no procedural «ON ACTION», mostrando una ventana con las filas de la tabla «provincias» para poder elegir una de ellas. En caso de introducir el código de una provincia que no exista en la tabla «provincias» se activará la instrucción no procedural «AFTER EDITADD OF PROVINCIA», permitiendo darla de alta en dicha tabla mediante la llamada al programa «ctl provincias». Cuando el operador pulse la tecla asignada a la acción <faccept> se activará la instrucción «AFTER EDITADD OF CLIENTES». Esta instrucción calcula el último código de cliente existente en la tabla y le asigna el número siguiente. A continuación, después de grabar la fila en curso se activa la instrucción no procedural «AFTER ADD OF CLIENTES», presentando en una ventana la fila recién insertada. Sin embargo, en caso de cancelar la edición, si no se ha grabado ninguna fila se producirá un mensaje impidiendo dicha cancelación. Este mensaje aparecerá en pantalla por la activación de la instrucción no procedural «BEFORE CANCEL OF CLIENTES». Por el contrario, en caso de haber grabado alguna fila aparecerá un mensaje indicando el número de filas grabadas en el programa. Este contador se acumula en el momento que se graba cada fila, es decir, en la instrucción no procedural «AFTER ADD OF CLIENTES».

## Consulta de filas

La consulta de filas sobre la tabla activa en el FORM se consigue mediante la ejecución de cualquiera de las siguientes instrucciones y funciones: QUERY, GET QUERY, «fquery()» y «fgetquery()». Todas ellas construyen una lista en curso con aquellas filas de la tabla activa que cumplan ciertas condiciones. Estas condiciones se pueden introducir por pantalla (operación que deberá realizar el usuario) o bien mediante programa. En el caso de la instrucción QUERY y la función «fquery()», las condiciones de consulta se introducen en la pantalla (SCREEN) del FORM, mientras que la instrucción GET QUERY consulta aquellas filas que cumplan las condiciones indicadas en su cláusula «WHERE». Esto significa que la GET QUERY nunca activará las instrucciones no procedurales de la sección EDITING. En el caso de emplear la instrucción QUERY o la función «fquery()» habrá que pulsar la tecla asignada a la acción <faccept> o ejecutar la instrucción EXIT EDITING para validar las condiciones establecidas por el operador en la pantalla.

La instrucción QUERY equivale a ejecutar las siguientes funciones:

```
call fgetquery(fquery())
```

La ejecución de la instrucción QUERY activa la siguiente secuencia de instrucciones no procedurales de las secciones CONTROL y EDITING:

- 1.— «AFTER DISPLAY OF tabla» (CONTROL). Limpia los valores que tuviesen las variables del FORM referentes a la fila en curso.
- 2.— «BEFORE EDITQUERY OF tabla» (EDITING). Ejecución antes de entrar en edición de campos.
- 3.— «BEFORE EDITQUERY OF columna» (EDITING). Ejecución antes de activar el campo del FORM asociado a «columna».
- 4.— «AFTER EDITQUERY OF columna» (EDITING). Esta instrucción se ejecutará al abandonar el campo del FORM asociado a «columna».

5.— «AFTER EDITQUERY OF tabla» (EDITING). Esta instrucción se ejecutará una vez pulsada la tecla asignada a la acción <facept> o bien si se ha ejecutado una EXIT EDITING validando las condiciones introducidas por el usuario.

6.— «BEFORE QUERY OF tabla» (CONTROL). Equivalente a la anterior, pero en la sección «CONTROL».

7.— «AFTER DISPLAY OF tabla» (CONTROL): Presenta la primera fila encontrada como válida en la consulta.

8.— «AFTER QUERY OF tabla» (CONTROL). Finaliza la consulta mostrando el total de filas que cumplen las condiciones establecidas por el operador.

La secuencia de instrucciones no procedurales seguida por la ejecución de la instrucción GET QUERY es la siguiente:

- «BEFORE QUERY OF tabla» (CONTROL).
- «AFTER DISPLAY OF tabla» (CONTROL).
- «AFTER QUERY OF tabla» (CONTROL).

Como puede comprobarse, estas tres instrucciones no procedurales coinciden con las tres últimas ejecutadas por la instrucción QUERY. Como ya se ha indicado, la instrucción GET QUERY nunca ejecutará las instrucciones no procedurales de la sección EDITING.

Cuando nos movemos por la lista en curso generada por una instrucción QUERY, GET QUERY o la función «fquery()» se estarán ejecutando constantemente las instrucciones no procedurales «AFTER DISPLAY OF tabla» y «AFTER | BEFORE LINE OF tabla» de la sección «CONTROL».

Las instrucciones no procedurales que se activan al cancelar la edición son las siguientes:

1.— «BEFORE CANCEL OF tabla» (EDITING). Esta instrucción se ejecuta después de pulsar la tecla asignada a la acción <fquit> o después de ejecutar una CANCEL. No obstante, podrá cancelarse también esta operación y continuar con la edición hasta que se cumpla una condición determinada.

2.— «AFTER CANCEL OF tabla» (CONTROL). Se ejecuta al abandonar la edición. El programa acepta la cancelación provocada por <fquit> o por la instrucción CANCEL.

3.— «AFTER DISPLAY OF tabla» (CONTROL). Muestra los valores de la última fila activa previa a la consulta cancelada.

Dependiendo de la tecla pulsada por el operador, la activación de instrucciones se consigue mediante las instrucciones no procedurales «ON KEY» y «ON ACTION» de las secciones CONTROL y EDITING.

En la sección EDITING, dichas instrucciones se activarán, cuando se intenta realizar una consulta, en el momento en que el operador pulse la tecla correspondiente a la acción indicada en la instrucción (cuando el cursor se encuentre situado sobre alguno de los campos de la SCREEN).

En el caso de las instrucciones de la sección CONTROL, éstas se activarán cuando no estemos en edición, es decir, cuando podamos movernos sobre las distintas opciones del menú del FORM o bien al ejecutar la instrucción «READ KEY THROUGH FORM».

Las instrucciones no procedurales «ON KEY» y «ON ACTION» sólo podrán detectar aquellas teclas que no tengan asignada ya una acción en el propio objeto sobre el que se detectan. Por ejemplo, nunca se podrá detectar la tecla correspondiente a la acción <freturn>, ya que dicha tecla se emplea tanto en el menú del FORM como en edición para seleccionar una opción del menú (sección CONTROL) y para aceptar un valor en un campo del FORM activando a su vez el siguiente campo (sección EDITING).

Después de realizar una consulta sobre la tabla activa se pueden emplear los valores agregados del FORM, que son los siguientes:

**CURRENT {COUNT | TOTAL | AVG | MAX | MIN} OF columnna**



Estas expresiones deberán situarse en alguna instrucción no procedural que se active después de construir una lista en curso. Por ejemplo, en la instrucción «AFTER QUERY OF tabla».

**Ejemplo 14 [PFORM14]: Activación de instrucciones no procedurales en consulta. Cada opción del menú será una forma distinta de realizar la consulta.**

```

database almacén

define
 variable optimiza char(300)
 variable i smallint default 0

 form
 screen
 {
 ! C l i e n t e s

 ?
 Código Cliente [a]
 $

 Empresa: [b]
 Nombre: [c]
 Apellidos: [d]
 Direccion1: [e]
 Provincia: [f] [g]
 Telefono: [h |i]

 @
 }
 end screen

 tables
 clientes
 end tables

 variables
 a = clientes.cliente
 b = clientes.empresa
 c = clientes.nombre
 d = clientes.apellidos
 e = clientes.direccion1
 f = clientes.provincia lookup g = descripcion, h = prefijo joining provincias.provincia
 i = clientes.telefono
 end variables

 menu principal "Mantenimiento"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 menu consulta
 end menu

 control
 after query of clientes
 pause "CONSULTADOS: " && numrecs
 end control

 editing

```

```

after editquery of clientes
 if clientes.provincia is null then begin
 window from select provincia, descripcion from provincias
 order by 2
 10 lines label "P r o v i n c i a s"
 at 8,30
 set clientes.provincia from 1
 if cancelled = true then
 let clientes.provincia = 28
 end
 before editquery of cliente
 next field "empresa"
 next field down "empresa"
 next field up "telefono"
 before editquery of provincia
 message "Pulsar " && keylabel("fuser1") clipped &&
 " para consultar provincias" reverse
 view
 on action "fuser1"
 if curfield = "provincia" then
 window from select provincia, descripcion
 from provincias
 order by 2
 10 lines label "P r o v i n c i a s"
 at 5,30
 set clientes.provincia from 1
 end editing
 end form
end define

global
 options
 no message
end options
end global

main begin
 menu form
end main

menu consulta "Consultas" no wait
 option "1.- QUERY"
 query
 puse "Máximo código de CLIENTE: " &&
 current max of cliente using "##,###" &&
 ". Plse [Return] para continuar"
 option "2.- GET QUERY"
 get query
 option "3.- Funciones"
 let optimiza = fquery()
 call fgetquery(optimiza clipped)
 prepare inst1 from "select cliente, empresa from clientes " &&
 optimiza clipped
 declare cursor consulta for inst1
 window from cursor consulta
 15 lines 50 columns
 label "FILAS EN LISTA EN CURSO"
 at 3,25
end menu

```

Este programa mantiene algunas de las columnas de la tabla «clientes». En primer lugar, la sección GLOBAL indica que no aparecerán los mensajes producidos. El programa comienza con la llamada al menú del objeto FORM («MENU FORM»), que pintará la SCREEN y mostrará el primer nivel del menú (sección MENU). Si el ope-

rador elige la opción «Consulta» en dicho menú, aparecerá un submnú («menu consulta») en el que se indican diversas formas de realizar consultas sobre la tabla activa:

a) QUERY. Esta instrucción permite editar las condiciones que el operador estime oportunas y, una vez aceptadas (acción <faccept>), se realizará la consulta, mostrándose en la pantalla la primera de las filas que forman la tabla activa. Después de esta instrucción se calcula, mediante valores agregados del FORM, cuál es el cliente cuyo código es el mayor.

b) GET QUERY. En este caso esta instrucción no lleva la cláusula «WHERE», por lo que, al no especificar condiciones, seconsultarán todas las filas de la tabla «clientes».

c) Funciones «fquery()» y «fgetquery()». La primera permite la edición de condiciones sobre las variables del FORM, mientras que la segunda ejecuta la consulta mostrando en la pantalla la primera de las filas que forman la tabla activa. En esta opción, las condiciones introducidas por el operador («fquery()») son recogidas en una variable («optimiza»). Con dicha variable se prepara (PREPARE) una SELECT que posteriormente se declarará como CURSOR. Este CURSOR se muestra en una ventana con las mismas filas que forman la lista activa del FORM.

Cuando el operador elija una de las tres opciones de este submenú volverá al menú principal. Esto se realiza mediante la cláusula «NO WAIT». En las consultas producidas por la instrucción QUERY y las funciones «fquery()» y «fgetquery()» se realizan los siguientes controles de edición:

- En primer lugar, el operador no puede establecer condiciones sobre el código del cliente, ya que se redirecciona mediante las instrucciones NEXT FIELD antes de activar dicha variable.
- Asimismo, antes de activar la variable «provincia» aparecerá un mensaje indicando la tecla que puede emplearse para la consulta masiva de las filas que contiene las provincias. Dicha tecla se recoge mediante la instrucción «ON ACTION», mostrándose todas aquellas filas de la tabla «provincias» en una ventana para que el operador pueda elegir una.
- En ambos casos, si el operador no selecciona ninguna provincia, el programa obliga a condicionarlas, mostrando una ventana con todas las provincias antes de realizar la consulta. En caso de no elegir ningún código se asigna por defecto el «28» para realizar la consulta.

En los tres casos, después de mostrarse la primera de las filas que componen la lista en curso aparecerá un mensaje, que indicará el número de filas consultadas. Este número de filas que componen la lista en curso se encuentra en la variable interna de CTL «numrecs».

## Modificación de la fila en curso

La modificación de las filas que componen la lista en curso del FORM se realiza mediante la instrucción MODIFY. Para aceptar dicha modificación podremos emplear la tecla asignada a la acción <faccept> o bien ejecutar la instrucción EXIT EDITING del CTL. Por su parte, la forma de cancelar una modificación será mediante la tecla asignada a la acción <fquit> o bien ejecutando la instrucción CANCEL del CTL. Cuando se graba una modificación en la tabla de la base de datos se ejecuta una UPDATE del CTSQL.

En el momento que se ejecuta una MODIFY sobre la fila en curso, ésta se bloquea automáticamente para el resto de usuarios del sistema (este bloqueo se producirá únicamente en las versiones de MultiBase para UNIX/Linux, en las de red local en o Windows y en instalaciones cliente-servidor).

En el caso de que otro usuario hubiese modificado recientemente una fila en su lista en curso, al seleccionarla como fila en curso para realizar sobre ella alguna operación, sus valores se actualizarían en función de las modificaciones realizadas.

No obstante, si se modifica una variable asociada a una columna que forme parte de la clave primaria de la tabla, al aceptar la modificación se producirá un mensaje de error si existen referencias en otra tabla que tenga una clave referencial restrictiva contra ella. Por contra, si la clave referencial se ha definido con la cláusula «SET

NULL», aunque exista referencia en otra tabla, al aceptar la modificación se actualizarán todas las filas de las tablas referenciales, poniendo a «NULL» el valor de los códigos de las columnas componentes de la clave.

El tratamiento de la operación de modificar en las secciones CONTROL y EDITING del FORM es muy similar al de las operaciones de consulta y alta de filas. En consecuencia, el orden de las instrucciones no procedurales que se activan en esta operación es el siguiente:

- 1.— «AFTER DISPLAY OF tabla» (CONTROL). Esta instrucción se ejecuta al comenzar la modificación y activar la SCREEN, desactivando a su vez el menú del FORM.
- 2.— «BEFORE EDITUPDATE OF tabla» (EDITING). Ejecución antes de entrar en edición de campos.
- 3.— «BEFORE EDITUPDATE OF columna» (EDITING). Ejecución antes de actuar el campo del FORM asociado a «columna».
- 4.— «AFTER EDITUPDATE OF columna» (EDITING). Esta instrucción se ejecutará al abandonar el campo del FORM asociado a «columna».
- 5.— «AFTER EDITUPDATE OF tabla» (EDITING). Esta instrucción se ejecutará al aceptar las modificaciones realizadas, ya sea mediante la tecla asignada a la acción <faccept> o bien mediante la instrucción EXIT EDITING.
- 6.— «BEFORE UPDATE OF tabla» (CONTROL). Equivalente a la anterior, pero en la sección «CONTROL». Los cambios realizados no se reflejan en la tabla de la base de datos.
- 7.— «AFTER UPDATE OF tabla» (CONTROL). Finaliza la consulta y graba en la tabla de la base de datos las modificaciones realizadas.

Por su parte, las instrucciones que se activan al cancelar la edición son las siguientes:

- 1.— «BEFORE CANCEL OF tabla» (EDITING). Se ejecuta al cancelar mediante la tecla asignada a la acción <fquit> o mediante la instrucción CANCEL. No obstante, podrá obviarse dicha cancelación y proseguir con la edición hasta que se cumpla una condición determinada.
- 2.— «AFTER CANCEL OF tabla» (CONTROL). Se ejecuta al abandonar la edición. El programa acepta la cancelación provocada por <fquit> o CANCEL.
- 3.— «AFTER DISPLAY OF tabla» (CONTROL). Muestra los valores anteriores de la fila activa antes de su modificación.

Dependiendo de la tecla pulsada por el operador, la activación de instrucciones se consigue mediante las instrucciones no procedurales «ON KEY» y «ON ACTION» de las secciones CONTROL y EDITING.

En la sección EDITING, dichas instrucciones se activarán, mientras se realiza la modificación, en el momento en que el operador pulse la tecla correspondiente a la acción indicada en la instrucción (cuando el cursor se encuentre situado sobre alguno de los campos de la SCREEN).

En el caso de las instrucciones de la sección CONTROL, éstas se activarán cuando no estemos en edición, es decir, cuando podamos movernos sobre las distintas opciones del menú del FORM o bien al ejecutarse la instrucción «READ KEY THROUGH FORM».

Las instrucciones no procedurales «ON KEY» y «ON ACTION» sólo podrán detectar aquellas teclas que no engan asignada ya una acción en el propio objeto sobre el que se detectan. Por ejemplo, nunca se podrá detectar la tecla correspondiente a la acción <freturn>, ya que dicha tecla se emplea tanto en el menú del FORM como en edición para seleccionar una opción del menú (sección CONTROL) y para aceptar un valor en un campo del FORM activando a su vez el siguiente campo (sección EDITING).

NOTA: En las versiones de MultiBase para Windows, y gracias al empleo del ratón, el operador podrá seleccionar directamente la variable a la que desea modificar su valor. En UNIX será necesario pasar por todas las variables anteriores a la que se desea modificar.

**Ejemplo 15 [PFORM15]:**

```

database almacén

define
 variable i smallint default 0

 form
 screen
{
! C l i e n t e s

 ?
 Código Cliente: [a]
 $

 Empresa: [b]
 Nombre: [c]
 Apellidos: [d]
 Direccion1: [e]
 Provincia: [f] [g]
 Telefono: [h] [i]

 @
}

end screen

tables
 clientes
end tables

variables
 a = clientes.cliente
 b = clientes.empresa
 c = clientes.nombre
 d = clientes.apellidos
 e = clientes.direccion1
 f = clientes.provincia lookup g = descripcion, h = prefijo joining provincias.provincia
 i = clientes.telefono
 untagged clientes.formpago
end variables

control
 after update of clientes
 window scroll from select cliente,empresa, nombre, apellidos, direccion1, provincia,
 telefono, formpago
 from clientes
 where clientes.cliente = $clientes.cliente
 3 lines label "CLIENTE MODIFICADO"
 at 15,1
 end control

editing
 after editupdate of clientes
 let clientes.formpago = "C"
 before editupdate of cliente
 next field "empresa"
 next field down "empresa"
 next field up "telefono"
 after editupdate of provincia
 selct "" from provincias
 where provincias.provincia = $clientes.provincia
 if found = false then begin

```

```
 if yes("Provincia inexistente. Desea darla de alta", "Y") = true then ctl "provincias"
 next field "provincia"
 end
 before editupdate of provincia
 message "Pulsar " && keylabel("fuser1") clipped &&
 " para consultar provincias" reverse
 view
 on action "fuser1"
 if curfield = "provincia" then
 window from select provincia, descripcion
 from provincias
 10 lines label "P r o v i n c i a s"
 at 5,30
 set clientes.provincia from 1
 end
 on action "fuser2"
 clear
 exit program
 end editing
end form
end define

main begin
 get query
 display box at 17,1 with 5,45
 display keylabel("faccept") reverse at 18,2,
 " = Grabar modificación" at 18,15,
 keylabel("fqui") reverse at 19,2,
 " = Cancelar modificación" at 19,15,
 keylabel("fuser2") reverse at 20,2,
 " = Salir" at 20,15
 view
 modify
 for i = 2 to numrecs begin
 next row
 modify
 end
end main
```

Este programa consulta todas las filas de la tabla «clientes» para modificarlas. La lista en curso se recorre desde la primera fila hasta la última («numrecs»). En primer lugar, la variable del FORM «cliente» asociada a la columna «cliente» de la tabla activa «clientes» no puede modificarse, ya que en el momento que intentemos posicionarnos sobre ella la instrucción NEXT FIELD desviarán su activación. Este proceso equivale al atributo NOUPDATE. Posteriormente, antes de activar el campo «provincia», aparecerá un mensaje indicando la tecla que permitirá ver la totalidad de provincias grabadas en la tabla maestra de «provincias». En caso de pulsar dicha tecla se activará la instrucción no procedural «ON ACTION 'fuser1'». En caso de introducir un código de «provincia» que no exista en la tabla maestra de «provincias» se permitirá darla de alta mediante la ejecución del programa «ctl provincias». Una vez aceptada la modificación (acción <faccept>), se asigna el valor constante «C» a la variable del FORM definida como «UNTAGGED». Por último, una vez realizada la modificación sobre la tabla «clientes», aparece una ventana con los nuevos valores de dicha fila.

El programa transcurre en la edición (modificación), por lo que, al pulsar la tecla asignada a la acción <fuser2> se activará la instrucción no procedural «ON ACTION 'fuser2'» y finalizará el programa.

## Borrado de la fila en curso

El borrado de la fila en curso en el FORM se realiza mediante la ejecución de la instrucción de REMOVE del CTL. Para poder ejecutar esta instrucción habrá que construir previamente la lista en curso y posteriormente seleccionar como fila en curso aquella que se desea eliminar. Asimismo, la ejecución de esta instrucción conlleva la confirmación de dicho borrado. Esta confirmación consiste en la presentación de un menú compuesto por dos

opciones: «SI» y «NO». En caso de confirmar el borrado de la fila en curso esta decisión sólo podrá ser revocada por el programador mediante la ejecución de la instrucción CANCEL del CTL en la instrucción no procedural de la sección CONTROL: «BEFORE DELETE OF tabla».

En caso de intentar eliminar una fila de la tabla en curso que esté referenciada en cualquier otra tabla de la base de datos —teniendo ambas tablas definida integridad referencial— (clave primaria en la tabla activa del FORM y claves referenciales en cualquier tabla de la base de datos), dependiendo del tipo de integridad se producirá una de las dos circunstancias siguientes:

- Si la clave referencial de las tablas de la base de datos (con referencia en la tabla que contiene la fila que se desea eliminar) es restrictiva (cláusula «RESTRICT»), no se ejecutará el borrado, presentándose un mensaje en la pantalla avisando del error producido.
- Por el contrario, si la clave referencial está definida como «SET NULL» se producirá el borrado, actualizándose a «NULL» todos los valores de la columna o columnas de enlace en las filas de la tabla referencial afectadas por la operación de borrado.

La fila en curso que se desea borrar queda bloqueada automáticamente para el resto de usuarios del sistema hasta que se produce su borrado físico. Este bloqueo sólo se producirá en las versiones de MultiBase para red local bajo Windows y también en UNIX/Linux, incluso en instalaciones con arquitectura cliente-servidor. Asimismo, en caso de eliminar una fila que cualquier otro usuario tuviese incluida en su lista en curso del FORM, al movernos por dicha lista se produciría un error, indicando que la lista en curso ha variado.

La operación de borrado no activa ninguna de las instrucciones no procedurales de la sección EDITING. Por lo tanto, sólo se activarán ciertas instrucciones no procedurales de la sección CONTROL. Estas instrucciones junto con el orden de activación son las siguientes:

- 1.— «AFTER DISPLAY OF tabla» (CONTROL). Instrucción no procedural que se activa después de la confirmación del borrado («SI/NO»).
- 2.— «BEFORE DELETE OF tabla» (CONTROL). Una vez elegida la opción de borrado, esta instrucción se activa antes de eliminar la fila en curso de la tabla. En estos momentos, podría cancelarse todavía la operación de borrado de la fila en curso mediante la ejecución de la instrucción CANCEL.
- 3.— «AFTER DISPLAY OF tabla» (CONTROL). Una vez aceptado el borrado de la fila (si no se cancela la operación) se limpian sus valores y aparecen los de la siguiente fila de la lista en curso.
- 4.— «AFTER DELETE OF tabla» (CONTROL). Esta instrucción se ejecutará en el momento que la fila en curso eliminada se ha borrado en la tabla de la base de datos.

En esta operación de borrado no existe la posibilidad de detectar una tecla en edición, ya que, como se ha comentado anteriormente la sección EDITING nunca se activa.

Asimismo, la única posibilidad de detectar la cancelación de un borrado es cuando ésta se realiza mediante la instrucción CANCEL. Es decir, dicha cancelación únicamente podrá realizarse por programa y no mediante la interacción con el operador. En este caso, al ejecutarse la instrucción CANCEL del CTL se activará la instrucción no procedural «AFTER CANCEL OF tabla» de la sección CONTROL.

#### Ejemplo 16 [PFORM16]:

```
database almacen

define
 form
 screen
{
! P r o v i n c i a s
 Código DescripciónPrefijo
^ ^
 [a] [b] [c]
```

```
@
}

end screen

tables
 provincias 6 lines
end tables

variables
 a = provincias.provincia required
 b = provincias.descripcion upshift
 c = provincias.prefijo
end variables

menu manteni "Mantenimiento"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query
end menu

control
 before delete of provincias
 select "" from clientes
 where clientes.provincia = $provincias.provincia
 if found = true then call cancelar("clientes")
 select "" from proveedores
 where proveedores.provincia = $provincias.provincia
 if found = true then call cancelar("proveedores")
 end control
end form

end define

main begin
 menu form
end main

function cancelar(a) begin
 pause "Imposible borrar, existen " && a & ". Pulsar [Return] para continuar" reverse bell
 ancel
end function
```

Este programa antiene la tabla «provincias» con «scroll» de líneas («provincias 6 lines»). Asimismo, para este mantenimiento se crea un menú con las cuatro opciones básicas: «Añadir» (ADD), «Modificar» (MODIFY), «Borrar» (REMOVE) y «Consultar» (QUERY). Cuando el operador decida eliminar la fila en curso, antes al borrado se comprueba que no existan ni clientes ni proveedores sobre dicha provincia. De alguna forma, éste es el control que realiza la propia integridad referencial a la hora de realizar el borrado de una fila que tenga referencia en otra tabla. En caso de existir algún cliente o proveedor en dicha provincia a borrar se cancelará la operación del borrado.

## Bloques de control activados con el repintado

La instrucción que provoca el repintado de la pantalla en el FORM es DISPLAY FORM. Por el contrario, la instrucción que se encarga de borrar dicha pantalla es CLEAR FORM. Estas dos instrucciones se encuentran implícitas en la instrucción MENU FRM. Al ejecutarse ésta se provoca el pintado de la SCREEN junto con las opciones



del men principal del FORM. Asimismo, cuando finaliza su ejecución se encarga de borrar tanto la SCREEN como las opciones del menú. Por otro lado, todas aquellas instrucciones que manejan la ista en curso del FORM también emplean el repintado de la sección SCREEN. Estas instrucciones son: ADD, ADD ONE, QUERY y GET QUERY.

Por lo tanto, cuando se ejecuta cualquiera de las instucciones DISPLAY FORM, CLEAR FORM, MENU FORM, ADD, ADD ONE, QUERY o GET QUERY se activa la instrucción no procedural «AFTER DISPLAY OF tabla» de la sección CONTROL.

## Instrucciones no procedurales activdas con cambio de tabla

Al comienzo de este capítulo indicamos la posibilidad de mantener más de una tabla en un FORM. Esta característica se consigue incluyendo los nombres de las tablas en la sección TABLES. El número máximo de tablas a mantener en un FORM es de 20. En este tipode mantenimientos existen dos posibilidades:

1.— Tablas enlazadas entre sí por medio de alguna columna común entre ellas. Dichas tablas se elazan mediante la sección JOINS del FORM (que se explica más adelante dentro de este capítulo). En este tipo de mantenimientos la tabla activa se selecciona mediante la ejecución de las instrucciones HEAD LINES. La primera seleccionará la tabla maestra, mientras que LINES seleccionará la tabla enlazada a ella que se encuentra en segundo nivel.

**Ejemplo 17 [PFORM17]. Mantenimiento de dos tablas («proveedores» y «artículos») enlazadas por la columna código de proveedor. Este ejemplo emplea la sección JOINS del FORM:**

```

databas almacen

define
 variable contador smallint default 0

 form
 screen
{
! Proveedores Pág: 1

 ?
 Código Proveedor: [a]
 $

 Empresa: [b]
 Nombre: [c]
 Apellidos: [d]
 Direccion1: [e]
 Provincia: [f] [g]
 Telefono: [h] [i]

 @
}

screen
{
! Proveedores Pág: 2

 ?
 Código Proveedor: [a]
 $

 Empresa: [b]
 ^

```

```

Artículo Descripción Precio_coste Precio_venta
^ ^
[] [k] [l] [m]

@
}
end screen

tables
 proveedores
 articulos 6 lines
end tables

variables
 a = proveedores.proveedor = articulos.proveedor required
 b = proveedores.empresa upshift
 c = proveedores.nombre
 d = proveedores.apellidos
 e = proveedores.direccion1
 f = proveedores.provincia lookup g = descripcion, h = prefijo joining provincias.provincia
 i = proveedores.telefono
 j = articulos.articulo required
 k = articulos.descripcion upshift
 l = articulos.pr_cost format "##,###,##&.&&"
 m = articulos.pr_vent format "##,###,##&.&&"
end variables

menu principal "Proveedores"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query
 option "Artículos"
 lines
 menu articulos
 call dokey("fprevious-page")
 head
end menu

control
 after proveedores
 select count(*) into contador
 from articulos
 where articulos.proveedor = $proveedores.proveedor
 pause "Proveedor con : " && contador using "##,###" &&
 " articulos. Pulse [Return] para continuar" reverse
 end control
joins
 articulos lines of proveedores
end joins
end form
end define

main begin
 menu form
end main

```

```

menu articulos "Artículos"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Insertar"
 intercalate
end menu

```

Este ejemplo enlaza las tablas «proveedores» y «artículos» mediante la columna «proveedor», que es común a ambas. Dichas tablas se encuentran enlazadas en la sección JOINS del FORM. En primer lugar, la instrucción MENU FORM activa el menú, que se encuentra en la sección MENU del FORM. Con este menú se activa la primera tabla cuyas columnas se encuentran en primer lugar dentro de la sección VARIABLES. En este caso, la tabla activa es «proveedores». Este primer menú incluye cinco opciones: «Añadir» (ADD), «Modificar» (MODIFY), «Borrar» (REMOVE), «Consultar» (QUERY) y «Artículos» (Cambio de tabla activa). En caso de seleccionar esta última opción se activará la instrucción no procedural «AFTER proveedores», la cual mostrará un mensaje indicando el número de artículos que tiene dicho proveedor.

En esta última opción del menú principal se cambia de tabla activa (LINES). Según la instrucción especificada en la sección JOINS, la tabla considerada de líneas es «artículos». En este caso, la instrucción LINES, además de cambiar de tabla activa, muestra todos los artículos del proveedor seleccionado en la tabla «proveedores». Asimismo, se llama a un submenú para el mantenimiento de la nueva tabla activa («artículos»). Dicho submenú se encuentra debajo de la sección MAIN y está compuesto por cuatro opciones: «Añadir» (ADD), «Modificar» (MODIFY), «Borrar» (REMOVE) e «Insertar» (INTERCALATE). Una vez que el operador abandone dicho submenú se volverá a seleccionar como tabla activa la de «proveedores». Esta selección se consigue mediante la ejecución de la instrucción HEAD. La llamada a la función interna «dokey()» con el parámetro «fprevious-page» es para que al abandonar la tabla «líneas» aparezca de nuevo la primera página, que es donde se encuentran todas las columnas a mantener de la tabla «proveedores».

2.— Tablas que no tienen ninguna relación entre sí. Para seleccionar una tabla como activa habrá que ejecutar la instrucción NEXT TABLE del CTL.

En este caso es imprescindible conocer qué tabla, de todas las que se mantienen, es la activa en cada momento. La variable interna de CTL «curtable» contendrá en todo momento el nombre de la tabla activa.

En ambos tipos de mantenimiento, cuando se selecciona una tabla activa se ejecuta la instrucción no procedural «AFTER tabla», siendo «tabla» el nombre de la tabla activa hasta ese momento. Posteriormente, se ejecutará la instrucción no procedural «BEFORE tabla», donde «tabla» es el nombre de la tabla activa actual.

NOTA: Indirectamente, cuando se han explicado las variables del FORM de tipo DISPLAYTABLE ya se ha empleado esta característica de mantenimiento de varias tablas, ya que todas las variables DISPLAYTABLE forman una tabla ficticia. Para emplear dicha tabla ficticia como entrada de datos habría que seleccionarla previamente mediante la ejecución de la instrucción NEXT TABLE del CTL.

#### Ejemplo 18 [PFORM18]:

```

database almacen

define
 variable contador smallint default 0

 form
 screen
{
! Proveedores Pág: 1

```

```

?
Código Proveedor: [a]
$

Emprea: [b]
Nombre: [c]
Apellidos: [d]
Direccion1: [e]
Provincia: [f] [g]
Telefono: [h |i]

@
}
screen
{
! Proveedores Pág: 2

?
Código Proveedor: [a]
$

^ Empresa: [b]
^ ^
Artículo Descripción Precio_coste Precio_venta
^ ^
[j] [k] [l] [m]

@
}
end screen

tables
 proveedores
 articulos 6 lines
end tables

variables
 a = proveedores.proveedor required
 b = proveedores.empresa upshift
 c = proveedores.nombre
 d = proveedores.apellidos
 e = proveedores.direccion1
 f = proveedores.provincia lookup g = descripcion, h = prefijo joining provincias.provincia
 i = proveedores.telefono
 j = articulos.articulo required
 k = articulos.descripcion upshift
 l = articulos.pr_cost format "##,###,##&.&"
 m = articulos.pr_vent format "##,###,##&.&"
 untagged articulos.proveedor
end variables

menu principal "Proveedores"
 option "Añadir"
 dd
 option "Modificar"
 modify
 option "Borar"

```

```

 remove
 option "Consultar"
 query
 option "Artículos"
 next table "articulos"
 get query where proveedor = $proveedores.proveedor
 menu articulos
 call dokey("fprevious-page")
 next table "proveedores"
 end menu

 control
 after proveedores
 select count(*) into contador
 from articulos
 where articulos.proveedor = $proveedores.proveedor
 pause "Proveedor con : " && contador using "###,###" &&
 " articulos. Pulse [Return] para continuar" reverse
 end control

 editing
 after editadd of articulos
 let articulos.proveedor = proveedores.proveedor
 end editing
 end form
end define

main begin
 menu form
end main

menu articulos "Artículos"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Inertar"
 intercalate
end menu

```

Este ejemplo es similar al anterior («pform17»), pero con sin emplear el enlace entre las dos tablas. En este caso no se ha utilizado la sección JOINS, realizándose la selección de la tabla activa mediante la instrucción NEXT TABLE, donde se indica el nombre de la tabla a seleccionar. En primer lugar, como en el ejemplo anterior, la tabla activa es «proveedores», que cambia a «articulos» al seleccionar la opción «Artículos» del menú principal. Mediante la instrucción GET QUERY y la cláusula «WHERE» se muestran todos los artículos del proveedor previamente seleccionado en la tabla «proveedores». Posteriormente, se cambia de menú para el mantenimiento de la nueva tabla activa («articulos»). Al abandonar este menú se vuelve a cambiar de tabla activa con la instrucción NEXT TABLE, cuyo parámetro en este caso es el nombre de la tabla («proveedores»). Al igual que en el ejemplo 17, se ejecuta la función «dokey()» para cambiar de página cuando se vuelva a seleccionar la tabla «proveedores».

Las diferencias entre los ejemplos 17 y 18 son las siguientes:

- En el ejemplo 17 el enlace de las columnas «proveedor» de ambas tablas se ha realizado con un mismo identificador, mientras que en el ejemplo 18 se ha definido un identificador para la columna «proveedor» de la tabla «proveedores», en tanto que la de la tabla «articulos» se ha definido como «UNTAGGED».

- La selección de tabla activa en el ejemplo 17 se ha realizado mediante las instrucciones LINES y HEAD, incluyendo la primera de ellas la selección de las filas que cumplan el enlace entre las dos tablas. Por su parte, en el ejemplo 18 la selección de tabla activa se ha efectuado mediante la instrucción NEXT TABLE, empleando la GET QUERY para simular la LINES.
- En el ejemplo 17 es necesario especificar la sección JOINS para indicar la relación entre ambas tablas, mientras que en el ejemplo 18 no.

## Diseño de la pantalla: Sección LAYOUT

La sección LAYOUT del FORM permite modificar el aspecto de presentación de la pantalla. Hasta ahora, algunas de las opciones contempladas en esta sección se han simulado empleando otros procedimientos. La sintaxis de la sección LAYOUT es la siguiente:

```
[LAYOUT
 [BOX
 [[NO] UNDERLINE]
 [REVERSE]
 [LINES líneas]
 [COLUMNS columnas]
 [LINE línea]
 [COLUMN columna]
 [LABEL {"literal" | número}]
 [HELP número_ayuda]
 [DISPLAY TABLE LABEL AT línea,columna]
]
END LAYOUT]
```

Las opciones anteriores son todas las que se pueden incluir dentro de esta sección opcional del FORM. Como hemos podido observar en los ejemplos expuestos hasta este momento, todos ellos presentaban una serie de características comunes, como por ejemplo:

- En toda las representaciones de pantalla realizadas hasta ahora, y salvo que se hayan utilizado determinados metacaracteres, no se ha empleado ninguna caja enmarcando las distintas variables del FORM. Esta característica se consigue mediante la cláusula «BOX». Esta cláusula calcula las dimensiones necesarias para no sobrescribir encima de algún literal o identificador definido en la sección SCREEN. En caso de querer variar las dimensiones calculadas por esta opción habría que emplear las cláusulas «LINES» y «COLUMNS» de la sección «LAYOUT».
- La pantalla del FORM (sección SCREEN) se muestra en las coordenadas «1,1» de la pantalla (línea 1, columna 1). Esta característica se puede modificar mediante las cláusulas «LINE línea» y «COLUMN columna».
- Por defecto, la petición de los valores para cada variable del FORM aparece siempre subrayada. Esta característica puede cambiarse con las opciones: «NO UNDERLINE» (aparecerá subrayado el campo activo), «REVERSE» (todos los campos aparecerán en vídeo inverso) y «UNDERLINE» (que es el atributo empleado por defecto, es decir, los campos aparecerán subrayados).

La etiqueta que se mostrará en la parte superior de la caja estándar diseñada por la cláusula «BOX» de esta sección puede extraerse de un fichero de ayuda. La forma de emplear los ficheros de ayuda se explica en el capítulo 16 de este mismo volumen.

**Ejemplo 19 [PFORM19]. Mantenimiento de la tabla «provincias» empleando algunas opciones de la sección LAYOUT:**

```
database almacen

define
 form
 screen
{
 ?
```

```

 Provincia: [a] $

 Descripción: [b]
 Prefijo: [c]

 }

end screen

tables
 provincias
end tables

variables
 a = provincias.provincia required
 b = provincias.descripcion upshift
 c = provincias.prefijo
end variables

menu provin "Mantenimiento"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query
end menu

layout
 box
 label "Mantenimiento de Provincias"
 no underline
 line 5
 column 10
end layout
end form
end define

main begin
 menu form
end main

```

Este ejemplo varía algunas de las características estándares de presentación del FORM. En primer lugar, todas las variables del FORM se encierran dentro de una caja estándar. En la cabecera de esta tabla aparece una etiqueta, indicada en la cláusula «LABEL» de la sección LAYOUT. Asimismo, las coordenadas donde aparecerá la caja estándar junto con las variables especificadas en la sección SCREEN son las siguientes: línea 5 («LINE 5») y columna 10 («COLUMN 10»).

## Enlace de tablas: Sección JOINS

Excepto en algún caso aislado, todos los ejemplos expuestos hasta ahora para el FORM realizaban el mantenimiento una sola tabla de la base de datos. Mediante la sección JOINS se podrán enlazar al menos dos tablas para su mantenimiento. El único requisito fundamental para poder enlazar dos tablas es que ambas compartan, al menos, una de sus columnas. Estas columnas se denominan «columnas de enlace».

La sintaxis de la sección JOINS es la siguiente:

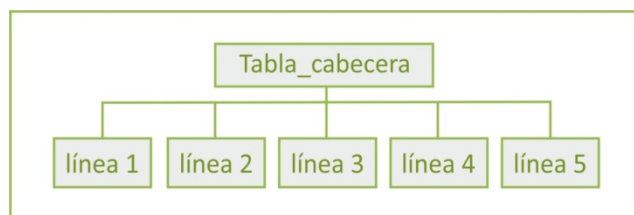
```
[JOINS
 tabla_líneas LINES OF tabla_cabecera [ORDER BY lista_columnas]
 [COMPOSITES <lista_columnas_enlace_cabecera>
 <lista_columnas_enlace_líneas> [< ... >]]
 ...
 ...
END JOINS]
```

A nivel de sintaxis, además de especificar la sección JOINS para el enlace entre tablas, es necesario enlazar también las columnas en la sección VARIABLES. Para ello emplearemos la sintaxis ya expuesta al tratar dicha sección:

```
{identificador = | UNTAGGED } [*] [tabla.] columna
 [lista_atributos;]
 = [*] [tabla.] columna [lista_atributos];]
 [= [*] [tabla.] columna [lista_atributos] [; ...]]
```

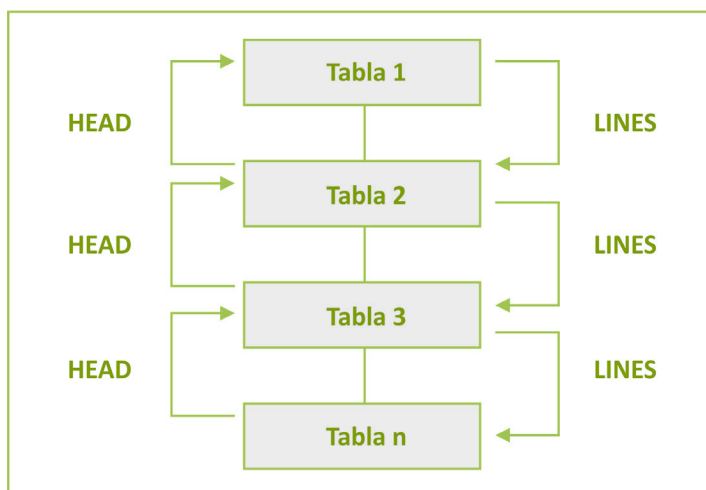
El enlace entre tablas puede realizarse de dos formas diferentes:

1.— Una tabla de cabecera y varias de líneas. El esquema de este tipo de mantenimiento es el que se muestra en la figura.



Esquema del mantenimiento con una tabla de cabecera y varias de líneas.

2.— Una tabla de cabecera enlazada con otra de líneas, siendo ésta a su vez cabecera de otra de líneas y así sucesivamente.



Enlace escalonado entre varias tablas cabecera-líneas.

Las tablas «Tabla 1», «Tabla 2» y «Tabla 3» se consideran tablas de líneas respecto a la inmediata anterior y de cabeceras respecto a la inmediata inferior.

Para seleccionar la tabla activa se pueden emplear las instrucciones HEAD (para seleccionar la tabla de cabecera) y LINES (para seleccionar la de líneas). En la Figura 1 anterior, al existir más de una tabla de líneas y sólo una de cabecera, para seleccionar como tabla activa una de las de líneas habrá que indicar «LINES nombre\_tabla\_líneas».



Como puede comprobarse en la sintaxis, la cláusula «ORDER BY» permite mostrar la lista en curso de la «tabla\_líneas», ordenada al menos por una de sus columnas. En caso de indicar más de una columna de ordenación, éstas tendrán que separarse mediante una coma. Si la columna de ordenación fuese numérica, ésta podría numerar automáticamente cada una de las filas insertadas para una cabecera. Para ello bastaría con asignarle los atributos NOENTRY y NOUPDATE en la definición de dicha variable del FORM, es decir, dentro del módulo CTL. No obstante, dicha columna no tiene por qué realizar la operación de numeración automática ni tampoco ser numérica.

En estas tablas deberían crearse las claves que definirán la integridad referencial entre ellas. Para ello habrá que definir como clave primaria todas las columnas de enlace pertenecientes a la tabla «cabecera», mientras que esas mismas columnas en la tabla de líneas compondrán la clave referencial.

En esta sección se enlazarán de dos en dos tablas, indicando cuál es la tabla principal («cabecera») y cual es la tabla de detalle («líneas»).

Al movernos sobre la lista en curso de la tabla de cabecera, es decir, siendo ésta la tabla de activa del FORM, aparecerá la primera fila de la tabla de líneas que compondrá la lista en curso cuando ésta sea seleccionada como tabla activa. Para presentar todas las filas enlazadas con la fila en curso de la tabla de cabecera habrá que seleccionar la tabla de líneas como «tabla activa», es decir, ejecutar la instrucción LINES.

#### Ejemplo 20 [PFORM20]. Enlace entre dos tablas:

```
database almacén

define
 form
 screen
 {
 %
 Núm. Albarán.....: [f1] *

 Cod. Cliente.....: [f2] [a]
 Fecha Albarán.....: [f6]
 Fecha Envío.....: [f7]
 Cod. F.Pago.....: [d] [z]
 ^
 Núm Artículo Proveedor Cantidad Descuento Precio
 ^
 [e] [f10] [f11] [f] [f13] [f14]

 }

end screen

tables
 albaranes
 lineas 6 lines
end tables

variables
 f1 = albaranes.albaran = lineas.albaran required
 f2 = albaranes.cliente required lookup a = empresa joining clientes.cliente
 f6 = albaranes.fecha_albaran
 f7 = albaranes.fecha_envio
 d = albaranes.formpago lookup z = descripcion joining formpagos.formpago
 e = lineas.linea noentry noupdate
 f10 = lineas.articulo
```

```
f11 = lineas.proveedor
f = lineas.cantidad
f13 = lineas.descuento
f14 = lineas.precio
end variables

menu cabecera 'Albaranes'
 option a1 'Consultar' comments 'Consulta de los registros.'
 query
 option a2 'Modificar' comments 'Modificacion del registro seleccionado.'
 if numrecs > 0 then modify
 else next option a1
 option a3 'Borrar' comments 'Borrado del registro seleccionado.'
 if numrecs > 0 then
 if yes ('Conforme con el borrado del registro' , "n") = true then remove
 else next option a1
 option a4 'Agregar' comments 'Introduccion de registros.'
 add one
 if cancelld = false then begin
 lines
 add
 head
 end
 option a5 'Lineas' comments 'Operaciones a realizar sobre la tabla Lineas'
 if numrecs > 0 then begin
 lines
 menu lineas
 head
 end
 else next option a1
end menu

editing
 before albaranes.cliente albaranes.formpago lineas.articulo
 message "Pulsar " &&
 keylabel("fuser1") clipped && " para consultar" reverse
 view
 on action 'fuser1'
 switch curtable & '.' & cufield
 case 'albaranes.cliente'
 window from select cliente,empresa
 from clientes
 10 lines label 'CLIENTES'
 at 5, 10
 set albaranes.cliente from 1
 case 'albaranes.formpago'
 window from select formpago, descripcion
 from formpagos
 where formpagos.formpago>""
 10 lines label 'FORMPAGOS'
 at 5,10
 set albarans.formpago from 1
 case 'linea.articulo'
 window from selet articulo, proveeor, descripcion
 from articulos
 10 lnes label 'ARTICULOS'
 at 5,10
 setlineas.articulo, lineas.proveedor from 1,2
 case 'lineas.proveedor'
 window from select articulo, proveedor, descripcion
 from articulos
 where articulos.articulo= $lineas.articulo
 10 lines label 'ARTICULOS'
```

```

 at 5,10
 set lineas.proveedor from 2
 end
end editing

layout
 box
 label 'Mantenimiento de Albaranes'
end layout

joins
 lineas lines of albaranes order by linea
end joins
end form
end define

main begin
 menu form
end main

menu lineas 'Lineas'
 option a1 'Añadir'
 add
 option a2 'Modificar'
 if numrecs > 0 then modify
 else next optio a1
 option a3 'Borrar'
 if numres > 0 then
 if yes ('Conforme con el borrado del registro' , "n") = true then remove
 else next option a1
 option a4 'Insertar'
 intercalate
 end menu

```

Este programa mantiene dos tablas, «albaranes» y «lineas», siendo esta última el detalle de la primera. En la sección SCREEN aparecen tanto las columnas de la tabla de cabecera («albaranes») como las de «lineas». Las columnas de esta tabla de líneas están en disposición de emplear el «scroll» de líneas. Ambas tablas se encuentran en la sección TABLES y, como se ha indicado anteriormente, la tabla «lineas» se mantendrá con «scroll» de 6 líneas. El enlace entre estas tablas se realiza mediante la unión de las variables del FORM «albaran», especificadas ambas en el identificador «f1» de la sección SCREEN. Asimismo, la columna elegida para ordenación en la tabla de líneas «linea» tiene asignada los atributos NOENTRY y NOUPDATE. Esta columna enumerará de forma automática todas las filas insertadas para un albarán.

Para el mantenimiento de estas dos tablas se emplean dos menús. El principal, «cabecera», se encarga del mantenimiento de la tabla «albaranes», y desde éste se llamará al submenú «lineas», que mantendrá la tabla «lineas» seleccionada como activa mediante la instrucción LINES. Cuando finalice el mantenimiento de la tabla «lineas», al abandonar el submenú se volverá a activar la tabla «albaranes» mediante la instrucción HEAD. En este ejemplo, la sección JOINS indica que la tabla «lineas» es líneas de la tabla «albaranes». Asimismo, la tabla de líneas mostrará sus filas de detalle de un albarán ordenadas por la columna «linea».

La instrucción COMPOSITES será necesaria cuando el enlace entre las tablas esté compuesto por más de una columna, es decir, cuando exista más de una columna de enlace entre ellas. En esta instrucción habrá que especificar todas las columnas de enlace de cada tabla, al igual que se especifican en la sección VARIABLES. Incluso, en caso de existir más de dos tablas, todas ellas enlazadas por más de una columna, habrá que indicar las columnas de enlace de todas las tablas.

Por ejemplo:

Tabla familia1:

```
cod_fam smallint
anno smallint
descripcion char(20)
```

Tabla subfamilia1:

```
cod_fam smallint
anno smallint
cod_subfam smallint
descripcion char(20)
```

Tabla articulos1:

```
cod_fam smallint
anno smallint
cod_subfam smallint
cod_art smallint
descripcion char(20)
precio money(16,2)
```

En este caso, el programa de mantenimiento de estas tres tablas tendría el aspecto que se muestra en el ejemplo siguiente.

### Ejemplo 21 [PFORM21.SCT] y [JOINS.SQL]. Mantenimiento de tres tablas enlazadas con múltiples columnas:

```
database almacen

define
 form
 screen
{
 Codigo familia:[a]
 Año: [b]
 Descripcion: [c]
^
 Subfamilia Descripción
^
 [d] [e]
^
^
^
 Artículo Descripción
^
 [f] [g]
^
}

end screen

tables
 familias1
 subfamilias1 3 lines
 articulos1 4 lines
end tables

variables
```

```

a = familias1.cod_fam = subfamilias1.cod_fam = articulos1.cod_fam required
lias1.anno = subfamilias1.anno = articulos1.anno required
c = familias1.descripcion
d = subfamilias1.cod_subfam = articulos1.cod_subfam required
e = subfamilias1.descripcion
f = articulos1.cod_art required
g = articulos1.descripcion
end variables

menu fam 'Familias'
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query
 option "Subfamilias"
 lines
 menu subfam
 head
end menu

layout
 box
 label "Familias/Subfamilias/Artículos"
end layout

joins
 subfamilias1 lines of familias1 order by descripcion
 articulos1 lines of subfamilias1 order by descripcion
 composites <familias1.cod_fam,familias1.anno>
 <subfamilias1.cod_fam,subfamilias1.anno>
 <articulos1.cod_fam,articulos1.anno>
 composites <subfamilias1.cod_fam,subfamilias1.anno, subfamilias1.cod_subfam>
 <articulos1.cod_fam,articulos1.anno,articulos1.cod_subfam>
end joins
end form
end define

main begin
 menu form
end main

menu subfam 'Subfamilias'
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Insertar"
 intercalate
 option "Artículos"
 lines
 menu arti
 head
end menu

menu arti 'Artículos'
 option "Añadir"
 add

```

```
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Insertar"
 intercalate
 end menu
```

Este programa enlaza las tablas «familias1», «subfamilias1» y «articulos». Al existir entre ellas más de una columna de enlace, habrá que emplear la instrucción COMPOSITES dentro de la sección JOINS. Como puede comprobarse, en la sección SCREEN se encuentran todas las columnas de las tres tablas. Los nombres de estas tablas a mantener se indican en la sección TABLES. En este caso, tanto la tabla «subfamilias1» como «articulos1» se mantienen con «scroll» de líneas, con 3 y 4 líneas respectivamente. Las columnas comunes entre las tres tablas se enlazan en la sección VARIABLES empleando para cada una de ellas un solo identificador de la sección SCREEN. Para cada tabla se construye un menú en el que la tabla activa se selecciona mediante la ejecución de las instrucciones LINES y HEAD. Por último, en la sección JOINS se incluyen dos líneas de enlace entre tablas para indicar las tablas de cabecera y sus respectivas tablas de líneas. Asimismo, como ya se ha indicado anteriormente, al tratarse de un enlace de tablas por más de una columna hay que especificar las instrucciones COMPOSITES para referenciar dichos enlaces.

## Mantenimiento de una tabla temporal

Las tablas temporales creadas mediante la cláusula «TEMP» de la instrucción CREATE TABLE pueden mantenerse en un objeto FORM. El requisito imprescindible para que el módulo compile perfectamente es que dicha tabla exista como una tabla base en el diccionario de la base de datos mientras dure la compilación, pudiendo borrarla en el momento que dicha compilación no detecte ningún error. Estas tablas se borrarán de forma automática cuando finalice la ejecución del programa. No obstante, mientras se encuentre activo el FORM su tratamiento es idéntico al de cualquier tabla base de la base de datos.

**Ejemplo 22 [PFORM22.SCT] y [TEMPORAL.SQL]. Mantenimiento de una tabla temporal («CREATETEMP TABLE»).**

```
database almacen

define
 form
 screen
{
 Código Descripción
 [a] [b]

end screen

tables
 temporal 5 lines
end tables

variables
 a = temporal.codigo required
 b = temporal.descrpcion
end variables

layout
 box
```

```

 label "Mantenimiento de una tabla temporal"
 end layout
end form
end define

main begin
 create temp table temporal (codigo smallint not null, descripcion char(20) upshift)
 primary key (codigo)
 menu form
end main

```

En primer lugar, este programa crea la tabla «temporal», que es la que se mantiene en el FORM definido anteriormente. Para poder compilar este programa correctamente habrá que ejecutar el fichero SQL «temporal.sql» de la base de datos «almacen». Una vez que la compilación ha sido satisfactoria, la tabla recién creada podrá eliminarse de la base de datos. Al crear la tabla temporal se activa el menú del FORM, que será el menú por defecto de este objeto si no se especifica la sección MENU.

## Empleo de más de un FORM por programa

Al principio de este capítulo indicábamos el hecho de que sólo se podía definir un FORM por módulo. En consecuencia, cuando un programa esté compuesto por varios módulos, uno principal y otros librerías, podrán definirse tantos objetos FORM como módulos compongan dicho programa. El manejo de estos FORMS tiene que realizarse desde el módulo en que se encuentra definido cada uno de ellos.

Cuando un ORM se encuentra definido en un módulo librería, su activación debe realizarse mediante tratamiento de funciones.

**Ejemplos 23 y 24 [PFORM23] y [PFORM24 (módulo librería)]. Mantenimiento de clientes (permitiendo dar de alta la provincia asignada en caso de que no exista) en un FORM definido en un módulo librería:**

**[PFORM23]:**

```

database almacen

define
 variable retorno char(1)

 form

 tables
 clientes
 end tables

 editing
 after edit add edit update of clientes.provincia
 select "" from provincias
 where provincias.provincia = $clientes.provincia
 if found = false then begin
 if yes("Provincia inexistente. Desea darla de alta (S/N)", "Y") = true then begin
 let retorno = alta(clientes.provincia)
 if retorno = false then begin
 let clientes.provincia = null
 pause "La provincia no ha sido dada de alta.
 Pulse [Return] para continuar" bell reverse
 end
 end
 end
 next field "provincia"
 end
 end editing
end define

```

```
 layout
 box
 label "Mantenimiento de Clientes"
 lines 14
 columns 50
 end layout
 end form
end define

main begin
 menu form
end main
```

### [PFORM24]:

```
database almacen

define
 variable provin like provincias.provincia
 form

 tables
 provincias
 end tabs

 editing
 before editadd of provincias
 let provincias.provincia = provin
 next field "descripcion"
 end editing

 layout
 ox
 label "Mantenimiento de Provincias"
 line 5
 column 20
 lines 5
 columns 40
 end layout
 end form
end define

function alta(a) begin
 let provin = a
 add one
 clear form
 if cancelled = true then return false
 else return true
end function
```

El módulo principal es el encargado del mantenimiento de la tabla «clientes». En el momento que se añade o modifica el código de la provincia a asignar al cliente en curso, se comprueba si dicho código ya existe en la tabla maestra («provincias»). Si no existe, el programa preguntará al operador si desea o no darlo de alta. En caso afirmativo, se recoge el valor devuelto por la función «alta()», que se encuentra definida en el módulo librería. Esta función contiene la instrucción ADD ONE para dar de alta una fila sobre la tabla activa («provincias»). Una vez finalizada la introducción de datos, se limpia el FORM activo, es decir, el de la tabla «provincias». Por último, si el operador ha cancelado la introducción de datos, la función devuelve al programa el valor «FALSE», y «TRUE» en caso de haber grabado una fila.

Dependiendo del valor devuelto por la función «alta()», el módulo principal mostrará un mensaje para indicar la cancelación, inicializando el código de la provincia con valor nulo («NULL»).



## Instrucciones relacionadas con el FORM

CTL dispone de diversas instrucciones que permiten el manejo de un FORM. Dichas instrucciones son las siguientes:

|                              |                                                                                                               |
|------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>ADD</b>                   | Genera un bucle que añade filas a la lista e curso del FORM.                                                  |
| <b>ADD ONE</b>               | Añade una fila a la lista en curso del FORM.                                                                  |
| <b>ANCEL</b>                 | Interrumpe una acción ejecutada por el operador (ADD, ADD ONE, MODIFY, QUERY, REMOVE).                        |
| <b>CLEAR FORM</b>            | Borra un FORM activo (sección SCREEN).                                                                        |
| <b>DISPLAY FORM</b>          | Muestra un FORM en pantalla.                                                                                  |
| <b>EXIT EDITING</b>          | Fuerza el fin de la edición de las variables del FORM. Esta finalización conlleva la grabación de la edición. |
| <b>GET QUERY</b>             | Genera una lista en curso del FORM.                                                                           |
| <b>HEAD</b>                  | Activa la tabla de cabecera como nueva tabla en curso.                                                        |
| <b>INTERCALATE</b>           | Inserta una fila en la tabla en curso.                                                                        |
| <b>LINES</b>                 | Activa la tabla de líneas como nueva tabla en curso.                                                          |
| <b>MENU FORM</b>             | Activa el menú de un FORM cediendo a éste el control de ejecución.                                            |
| <b>MODIFY</b>                | Edita la fila en curso de la tabla en curso para su modificación.                                             |
| <b>NEXT FIELD</b>            | En edición, altera la secuencia de entrada de los campos.                                                     |
| <b>NEXT ROW</b>              | Muestra en pantalla la siguiente fila de la lista en curso.                                                   |
| <b>NEXT TABLE</b>            | Cambia la tabla en curso.                                                                                     |
| <b>OUTPUT</b>                | «Vuelca» una pantalla FORM a un fichero.                                                                      |
| <b>PREVIOUS ROW</b>          | Muestra en pantalla la fila anterior de la lista en curso.                                                    |
| <b>QUERY</b>                 | Construye la lista en curso con las filas que cumplan las condiciones indicadas por el operador.              |
| <b>READ KEY THROUGH FORM</b> | Solicita una tecla y ejecuta su acción sobre el FORM.                                                         |
| <b>REMOVE</b>                | Borra la fila activa de la lista en curso.                                                                    |
| <b>RETRY</b>                 | Reinicia la edición en un FORM.                                                                               |

En los siguientes apartados se comentan las principales características de cada una de estas instrucciones. No obstante, para una información más detallada puede consultarse el Manual de Referencia de MultiBase.

### Instrucción ADD

Esta instrucción agrega un número indeterminado de filas sobre la tabla activa, inicializando las variables del FORM con su valor por defecto y permitiendo al operador introducir valores nuevos sobre ellas. Una vez aceptados estos valores (acción <facept> o ejecución de EXIT EDITING) se grabará una fila sobre la tabla activa del FORM, incrementando la lista en curso de este objeto.

La instrucción ADD se ejecuta hasta que el operador pulse la tecla asignada a la acción <fquit>, o bien hasta que e ejecute una instrucción CANCEL en las secciones CONTROL o EDITING del FORM. El valor por defecto empleado en CTL para variables que no tengan asignado el atributo DEFAULT es «NULL».

La sintaxis de esta instrucción es la siguiente:

### ADD

Eta instrucción puede situarse en cualquier punto de un módulo donde puedan incluirse cualquiera de las instrucciones comentadas en el Manual de Referencia. No obstante, el lugar más habitual donde suele situarse la instrucción ADD es en una de las opciones del menú del FORM.

#### Ejemplo 25 [PFORM25]. Mantenimiento de la tabla «clientes»:

```
database almacen

define
 form
 tables
 clientes
 end tables
 menu clien "Clientes"
 option "Añadir"
 add
 option "odificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 qury
 end menu
end form
end define

main begin
 menu form
end main
```

Este programa mantiene la tabla «clientes», pudiendo realizar las cuatro operaciones básicas: altas (ADD), modificaciones (MODIFY), bajas (REMOVE) y consultas (QUERY). La sección MENU del FORM se activa con la instrucción MENU FORM.

## Instrucción ADD ONE

Esta instrucción agrega una fila a la lista en curso del FORM. Su funcionamiento es idéntico al de la instrucción ADD. La diferencia entre ellas radica en que ADD ONE inserta solamente una fila sobre la tabla activa, mientras ue ADD generaba un bucle para añadir un número indeterminado de filas. Su sintaxis es la siguiente:

### ADD ONE

El lugar más habitual donde se sitúa esta instrucción es dentro de una opción de un menú.

#### Ejemplo 26 [PFORM26]. Mantenimiento de la tabla «clientes»:

```
database almacen

define
 form
 tables
 clientes
 end tables
 menu clien "Clientes"
 option "Añadir"
```

```

 add one
 optin "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query
 end menu
end form
end define

main begin
 menu form
end main

```

Este programa es similar al expuesto para la instrucción ADD, salvo que en este caso será necesario seleccionar la opción «Añadir» tantas veces como filas se desee agregar a la lista en curso.

## Instrucción CANCEL

Esta instrucción interrumpe cualquiera de las instrucciones que se emplean para realizar la edición sobre las variables del FORM o para el borrado de la fila activa (REMOVE). La instrucción CANCEL sólo puede situarse en las secciones CONTROL y EDITING del FORM. Su sintaxis es la siguiente:

### CANCEL

La ejecución de esta instrucción simula la pulsación de la tecla asignada a la acción <fquit>.

#### Ejemplo 27 [PFORM27]. Mantenimiento de la tabla «provincias» con «scroll» de líneas:

```

database almacén

define
 form
 screen
{
 ^
 Código Descripción Prefijo ^
 [a [b] [c]
}

end screen

tables
 provincias 6 lines
end tables

variables
 a = provincias.provincia required
 b = provincias.descripcion upshift
 c = provincias.prefijo
end variables

menu mm1 "provincias"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove

```

```
 option "Consultar"
 query
 end menu

 control
 before delete of provincias
 select "" from clientes
 where clientes.provincia = $provincias.provincia
 if found = true then begin
 pause "No puede borrarse, existen clientes. Pulse [Return] para continuar"
 bell reverse
 cancel
 end
 else begin
 select "" from proveedores
 where proveedores.provincia = $provincias.provincia
 if found = true then begin
 pause "No puede borrarse, existen proveedores.
 Pulse [Return] para continuar" bell reverse
 cancel
 end
 end
 end
end control

layout
 box
 label "P r o v i n c i a s"
 end layout
end form
end define

main begin
 menu form
end main
```

Este programa mantiene la tabla «provincias». Si el operador decide borrar una fila, se comprueba previamente que no existe ningún cliente ni proveedor en dicha provincia. En caso de existir alguno, la operación de borrado se cancela mediante la instrucción CANCEL.

## Instrucción CLEAR FORM

Esta instrucción borra de la pantalla el aspecto de un FORM, es decir, elimina la SCREEN representada previamente mediante la instrucción MENU FORM o DISPLAY FORM. Su sintaxis es la siguiente:

### CLEAR FORM

Esta instrucción se encuentra implícita en la MENU FORM. Cuando el menú empleado para el mantenimiento el FORM se canela (acción <fquit>), se borra de la pantalla el menú junto con el FORM.

### Ejemplo 28 [PFORM28]. Siulación de la instrucción MENU FORM:

```
database almacen
define
 form
 tables
 clientes
 end tables
 menu clien "Clientes"
 option "Aadir"
 add one
 otion "Modificar"
 modify
 option "Borrar"
```

```

 remove
 option "Consultar"
 query
 end menu
end form
end define

main begin
 display form
 menu clien
 clear form
end main

```

La instrucción DISPLAY FORM muestra en la pantalla el aspecto de la SCREEN diseñada en el FORM. En este caso, al no existir dicha sección SCREEN aparecerá la distribución por defecto empleada por este objeto para todas sus variables. A continuación, la instrucción «MENU clien» presenta el menú especificado en la sección MENU del FORM. Ésta es la forma de ejecutar cualquier menú diseñado en CTL. Acto seguido, al cancelar dicho menú (acción <quit>), éste se limpiará. Por último, mediante la instrucción CLEAR FORM se borra de la pantalla el aspecto de la sección SCREEN, es decir, se ejecuta la acción contraria a la realizada por la instrucción DISPLAY FORM.

## Instrucción DISPLAY FORM

Esta instrucción muestra el aspecto de la sección SCREEN y se encuentra implícita en la MENU FORM. Su sintaxis es la siguiente

**DISPLAY FORM [AT línea, columna]**

La cláusula «AT» puede simularse mediante las cláusulas «LINE» y «COLUMN» de la sección LAYOUT.

### Eemplo 29 [PFORM29]:

```

database almacen

defie
 form
 screen
{
 !
 ? P r o v i n c i a s ?
 'Código' 'Descripción' 'Prefijo'
 $ $ $
 % % %
 [a] [b] [c]

 * * *

 @
}
end screen

tables
 provincias 6 lines
end tables

variables
 a = provincias.provincia required
 b = provincias.descripcion upshift
 c = provincias.prefijo
end variables

menu m1 "Provincias"

```

```
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query
 end menu
end form
end define

main begin
 pause "Aspecto del form ... Pulsar [Return]"
 display form
 pause "Borrado del form ... Pulsar [Return]"
 clear form
 pause "Activación del menú ... Pulsar [Return]"
 menu form
end main
```

Este programa realiza el mantenimiento de la tabla «provincias», mostrando previamente el aspecto de la pantalla mediante la instrucción `DISPLAY FORM`. Posteriormente, se elimina dicha pantalla mediante la `CLEAR FORM` y, por último, se activa el menú del FORM mediante la instrucción `MENU FORM`.

### Instrucción EXIT EDITING

Esta instrucción fuerza el fin de la edición de la variables del FORM. Esta finalización conlleva la aceptación (grabación) de la operación que se esté realizando en esos momentos (`ADD`, `MODIFY` o `QUERY`). Su sintaxis es la siguiente:

#### EIT EDITING

Esta instrucción sólo podrá situarse en cualquiera de las instrucciones no procedurales de la sección `EDITING`.

#### Ejemplo 30 [PFORM30] y [MARCADO.SQL]. Marcado de filas sobre una tabla temporal:

```
database almacen

define
 form
 screen
{
 Marca Tabla

 [a] [b]

}

end screen

table
 marcado 7 lines
end tables

variables
 a = marcado.marca
 b = marcado.tabla
end variabls
```

```

editing
 before editupdate of marca
 if marca = "*" then let marca = null
 else let marca = "*"
 exit editing
 end editing

 layout
 box
 label "Marcado"
 end layout
 end form
end define

global
 options
 no message
 end options
end global

main begin
 create temp table marcado (marca char(1), tabla char(18))
 insert into marcado (tabla) select tabname from systables where tabid >= 150
 get query
 forever begin
 read key through form
 if actionlabel (lastkey) in ("freturn", "faccept") then modify
 else if actionlabel (lastkey) = "fquit" then break
 end
 window from select marca, tabla from marcado
 where marca is not null
 10 lines label "Resultado"
 at 5,30
end main

```

Para compilar este módulo es necesario crear previamente la tabla «marcado». Para ello ejecutaremos el fichero «marcado.sql» («trans almacen marcado»). Este programa inserta los nombres de las tablas existentes en la base de datos «almacen», y las consulta en el FORM mediante la instrucción GET QUERY. A continuación, mediante la instrucción «READ KEY THROUGH FORM» se permite al operador el manejo del teclado. En caso de pulsar as teclas asignadas a las acciones <freturn> o <faccept> se modificará la fila en curso. En estos omentos se activa la instrucción no procedural de la sección EDITING, asignando al campo «marca» un asterisco o un valor nulo («NULL»), dependiendo del valor que tuviese asignado previamente. La instrucción EXIT EDITING abandonará la edición grabando el valor asignado al campo «marca» sin necesidad de pulsar la tecla asignada a la acción <faccept>. Cuando el operador ejecuta la acción <fquit> se abandona el FORM, mostrándose en una ventana (mediante la instrucción WINDOW) las filas marcadas. No obstante, esta última operación podría realizarse también mediante la declaraión de un CURSOR y una WINDOW:

```

declare cursor marcados for select marca, tabla from marcado
 where marca is not null
window from cursor marcados
 10 lines label "Resultado"
 at 5,30

```

## Instrucción GET QUERY

Esta instrucción construye una lista en curso con aquellas filas que cumplan las condiciones especificadas en el programa mediante su cláusula «WHERE». Su sintaxis es la siguiente:

**GET QUERY [WHERE condiciones]**

Esta instrucción es equivalente a QUERY; la única diferencia radica en la forma de establecer las condiciones. En la instrucción GET QUERY las condiciones se indican en el programa (cláusula «WHERE»), mientras que en la QUERY son establecidas por el operador en edición.

### Ejemplo 31 [PFORM31]. Consulta con GET QUERY:

```
database almacen

define
 form
 tables
 formpagos
 end tables

 layout
 box
 lines 5
 columns 40
 label "Formas de Pago"
 end layout
 end form
 end define

main begin
 get query
 menu for
end main
```

Este programa consulta todas las filas de la tabla «formpagos» al no haberse incluido la cláusula «WHERE» en la instrucción «GET QUERY». A continuación, aparece el menú por defecto del FORM.

## Instrucción HEAD

Esta instrucción selecciona la tabla de cabecera como tabla activa del FORM. Su sintaxis es la siguiente:

### HEAD

Esta instrucción sólo podrá emplearse cuando se declare enlace entre tablas, es decir, al utilizar la sección JOINS del FORM.

### Ejemplo 32 [PFORM32]. Mantenimiento de cabeceras-líneas con «scroll» de líneas en ambas tablas.

```
database almacen

define
 form
 screen
 {
 Código Descripción Prefijo
 ^
 [a] [b]] [c]
 ^
 Código Empresa Dirección
 ^
 [d] [e]] [f]
 }
 end screen

 tables
```



```

 provincias 3 lines
 clientes 6 lines
 end tables

 variables
 a = provincias.provincia = clientes.provincia required
 b = provincias.descripcion upshift
 c = provincias.prefijo
 d = clientes.cliente required
 e = clientes.empresa
 f = clientes.direccion1
 end variables

 menu mm1 "provincias"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query
 option "Clientes"
 lines
 menu clien
 head
 end menu

 layout
 box
 label "P r o v i n c i a s / C l i e n t e s"
 end layout

 joins
 clientes lines of provincias
 end joins
 end form
end define

main begin
 menu form
end main

menu clien "Clientes"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Insertar"
 intercalate
end menu

```

Este programa mantiene dos tablas, «provincias» y «clientes», enlazadas por la columna «provincia». Ambas tablas se presentan con «scroll» de líneas. Como puede comprobarse, en la opción del menú principal «Clientes» se emplean las instrucciones HEAD y LINES para activar una tabla u otra (cabecera y líneas, respectivamente).

### Instrucción INTERCALATE

Esta instrucción agrega una fila a la lista en curso actual del FORM sobre la tabla que se está manteniendo. Su funcionamiento es similar a la ADD ONE. La diferencia estriba en que INTERCALATE puede insertar una fila entre otras dos ya existentes en la lista en curso siguiendo el orden establecido previamente. Por lo general, se emplea en mantenimientos con «scroll» de líneas. Su sintaxis es la siguiente:

#### INTERCALATE

##### Ejemplo 33 [PFORM33]. Mantenimiento con «scroll» de líneas:

```
database almacen

define
 form
 screen
{
 Código Descripción Prefijo
^
 [a] [b] [c]
^

}

end screen

tables
 provincias 6 lines
end tables

variables
 a = provincias.provincia required
 b = provincias.descripcion upshift
 c = provincias.prefijo
end variables

menu manteni "Mantenimiento"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Insertar"
 intercalate
 option "Consultar"
 query
end menu
end form
end define

main begin
 menu form
end main
```

El programa anterior mantiene la tabla «provincias» con «scroll» de líneas. En el menú existen dos opciones para agregar filas nuevas sobre la tabla: «Añadir» e «Insertar». La primera de ellas, al agregar una fila nueva se sitúa al final de la lista en curso, mientras que la segunda abre una línea anterior a la fila en curso para agregar en dicho lugar la nueva fila.

## Instrucción LINES

Esta instrucción selecciona la tabla de líneas como tabla activa del FORM, y sólo podrá emplearse cuando se mantenga una relación de tablas «cabecera-líneas», es decir, cuando se defina la sección JOINS. Su sintaxis es la siguiente:

**LINES** [tabla]

Esta instrucción genera una nueva lista en curso para la correspondiente tabla de líneas, haciendo un GET QUERY de forma automática con la columna o columnas de enlace.

### Ejemplo 34 [PFORM34]. Consulta sobre tres tablas enlazadas:

```

database almacén

define
 form
 screen
 {
 Codigo Cliente Empresa
 ^
 [f001] [f002]
 ^
 Alb Fec.Albaran Fec. Envío Fec.Pago Estado
 ^
 [f014][f016] [f017] [f018] [c]
 ^
 Lin Art. Prov. Descripción Unid. Desc Precio Total
 ^
 [f022][f023] [f024] [d] [f05] [f026] [f027] [e]

 }

end screen

tables
 clientes 2 lines
 albaranes 2 lines
 lineas 5 lines
end tables

variables
 f001 = clientes.cliente = albaranes.cliente
 f002 = clientes.empresa
 f014 = albaranes.albaran = lineas.albaran
 f016 = albaranes.fecha_albaran
 f017 = albaranes.fecha_envio
 f018 = albaranes.fecha_pago
 c = albaranes.estado
 f022 = lineas.linea noentry noupdate
 f023 = lineas.articulo
 f024 = lineas.proveedor
 d = pseudo lineas.descripcion like articulos.descripcion
 f025 = lineas.cantidad
 f026 = lineas.descuento
 f027 = lineas.precio
 e = pseudo lineas.total_linea like articulos.pr_vent
end variables

```

```
menu consulta "Consulta"
 option "Clientes"
 if curtable = "lineas" then begin
 head
 head
 query
 end
 else begin
 if curtable = "albaranes" then begin
 head
 query
 end
 else query
 end
 option "Albaranes"
 if curtable = "lineas" then head
 else if curtable = "clientes" then
 lines
 option "Lineas"
 if curtable = "clientes" then begin
 lines
 lines
 end
 else if curtable = "albaranes" then lines
 end menu

control
 after display of lineas
 select descripcion into $descripcion
 from articulos
 where articulo = $articulo
 and proveedor = $proveedor
 let total_linea = cantidad * precio * (1 - descuento / 100)
 end control

layout
 box
 label "Clientes/Albaranes/Líneas"
end layout

joins
 albaranes lines of clientes
 lineas lines of albaranes order by linea
end joins
end form
end define

main begin
 menu form
end main
```

Este programa consulta las filas relacionadas entre tres tablas: «clientes», «albaranes» y «lineas». El menú consulta constantemente la variable interna «curtable» para comprobar cuál de las tres se encuentra activa. Dependiendo de una u otra ejecutará las instrucciones LINES o HEAD necesarias para poder invocar a la instrucción QUERY de consulta.

## Instrucción MENU FORM

Esta instrucción ejecuta el menú definido en la sección MENU del FORM. Su sintaxis es la siguiente:

**MENU FORM** [DISABLE lista\_opciones] [AT línea, columna]

Al ejecutar esta instrucción se muestra automáticamente en pantalla tanto el aspecto de la sección SCREEN, si estuviese definida, como el menú especificado en la sección MENU. En caso de no existir la sección MENU se presentará un menú por defecto con las opciones: «Añadir», «Modificar», «Borrar», «Encontrar», «Tabla», «Salida» e «Insertar».

La instrucción MENU FORM se puede simular mediante la ejecución de las siguientes (siempre y cuando se define la sección MENU):

```
...
...
main begin
 display form
 menu nombre_menu
 clear form
end main
```

#### Ejemplo 35 [PFORM35]. SCREEN y menú por defecto en un mantenimiento:

```
database almacén

define
 form
 tables
 provincias
 end tables
 end form
end define

main begin
 menu form
end main
```

La instrucción MENU FORM presenta la pantalla definida en la sección SCREEN. Sin embargo, al no existir ésta presentará la pantalla por defecto y lo mismo ocurre con el menú.

## Instrucción MODIFY

Esta instrucción permite modificar los valores de la fila en curso, actualizándose en la tabla una vez aceptados. Para aceptar los valores habrá que pulsar la tecla asignada a la acción <faccept> o bien ejecutar la instrucción EXIT EDITING. Por su parte, para cancelar la modificación de valores habrá que pulsar la tecla asignada a la acción <fquit> o ejecutar la instrucción CANCEL. La sintaxis de la instrucción MODIFY es la siguiente:

### MODIFY

#### Ejemplo 36 [PFORM36]. Modificación de filas de la lista en curso:

```
database almacén

define
 form
 tables
 provincias
 end tables
 menu m1 "Provincias"
 option "Consultar"
 query
 option "Modificar"
 modify
 end menu
 end form
end define

main begin
```

```
 menu form
 end main
```

Este programa sólo permite consultar las filas de la tabla «provincias», pudiendo modificar sus valores. Esta modificación podrá realizarse sobre aquellas filas que no tengan integridad referencial con otras tablas. Sin embargo, en aquellas que sí exista integridad referencial sólo se podrá modificar la «descripcion» y el «prefijo», produciéndose error si se cambia el «codigo».

### Instrucción NEXT FIELD

Esta instrucción altera el orden de edición de las variables del FORM definido en la sección VARIABLES. Su sintaxis es la siguiente:

**NEXT FIELD [UP | DOWN] [campo]**

La instrucción NEXT FIELD sólo puede emplearse en cualquiera de las instrucciones de la sección EDITING.

**Ejemplo 37 [PFORM37]. Asignación automática del código de la provincia, no permitiendo al operador acceder a dichocampo:**

```
database almacen

define
 variable maximo like provincias.provincia
 form

 tables
 provincias
 end tables

 menu m1 "Provincias"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remve
 option "Consultar"
 query
 end menu

 editing
 before editadd editupdate of provincia
 next field "descripcion"
 next field down "descripcion"
 next field up "prefijo"
 after editadd of provincias
 select max(provincia) into maximo from provincias
 if maximo is null then let maximo = 0
 let provincia = maximo + 1
 end editing
 end form
end define

main begin
 menu form
end main
```

En este programa, el operador no podrá en ningún caso situarse sobre el código de la provincia. Esta restricción se consigue mediante la instrucción NEXT FIELD, que activa un campo u otro dependiendo de dónde provenga el cursor de la pantalla («UP» o «DOWN»). Asimismo, antes de grabar la fila introducida se calcula el valor máximo grabado en el código de la provincia para asignárselo a la fila actual.

## Instrucción NEXT ROW

Esta instrucción activa la siguiente fila a la actual de la lista en curso generada en el FORM. Su sintaxis es la siguiente:

### NEXT ROW

La ejecución de esta instrucción equivale a pulsar la tecla asignada a la acción <fdown> en el menú del FORM cuando existe una lista en curso.

**Ejemplo 38 [PFORM38]. Mantenimiento de clientes activando dos opciones en el menú para movernos por la lista en curso:**

```
database amacen

define
 form

 tables
 clientes
 end tables

 menu clien "Clientes"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar" remove
 option "Consultar"
 query
 option "Sigiente"
 next row
 option "Anterior"
 previous row
 end menu
 end form
end define

main begin
 menu form
end main
```

Mantenimiento por defecto de la tabla «clientes», donde se emplea un menú con las cuatro operaciones básicas (añadir, modificar, borrar y consultar) y dos más que sirven para moverse por la lista en curso, activando la fila anterior y posterior a la actual.

## Instrucción NEXT TABLE

Esta instrucción activa una de las tablas indicadas en la sección TABLES del FORM. Es decir, permite cambiar la tabla en curso. Su sintaxis es la siguiente:

### NEXT TABLE "tabla"

A diferencia de las sentencias HEAD y LINES, se puede cambiar de una tabla a otra sin que existan variables de enlace entre ellas.

Asimismo, como ya se ha indicado anteriormente, en el momento que se define una variable de tipo DISPLAY-TABLE, automáticamente se está definiendo una tabla ficticia denominada «displaytable», la cual puede seleccionarse mediante esta instrucción.

### Ejemplo 39 [PFORM39]. Mantenimiento de dos tablas sin enlazarlas mediante JOINS:

```
database almacen

define
 form

 tables
 provincias
 proveedores
 end tables

 menu provin "Provincias"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query
 option "Proveedores"
 next tble "proveedores"
 get query where provincia = $provincias.provincia
 menu provee
 next tble "provincias"
 end menu
 end menu

 layout
 box
 label "Provincias/Proveedores"
 end layout
 end form
end define

main begin
 menu form
end main

menu provee "Proveedores"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query
end menu
```

Este programa mantiene las tablas «provincias» y «proveedores» sin estar enlazadas por la sección JOINS. El cambio de tabla activa se realiza mediante la ejecución de la instrucción NEXT TABLE. Asimismo, al activar la tabla «proveedores» se genera una lista en curso con aquellos proveedores de la «provincia» seleccionada en la tabla anterior («provincias»).

## Instrucción OUTPUT

Esta instrucción «vuelca» el contenido de una pantalla de un FORM a la salida especificada en su sintaxis. Dicha sintaxis es la siguiente:

**OUTPUT** [ {**TO** | **THROUGH**} expresión] [**APPEND**] [**ROW**]



Esta instrucción se emplea en el menú utilizado por el FORM cuando no se declara la sección MENU. La opción que representa a esta instrucción en este menú por defecto es la denominada «Salida». La salida puede realizarse de la fila en curso, o bien de todas las filas que componen la lista en curso, pudiendo generar un fichero o bien concatenar en uno ya existente o emplear un programa para procesarla.

**Ejemplo 40 [PFORM40]. Mantenimiento de la tabla «formpagos» con posibilidad de realizar un «hard-copy» de la lista en curso o de las filas que la componen de forma individualizada:**

```
database almacén

define
 form

 tables
 formpagos
 end tables

 menu m1 "Formas de Pago"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query
 option "Hard-copy"
 output
 end menu
end form
end define

main begin
 menu form
end main
```

Al seleccionar la opción «Hard-copy» del menú del FORM aparecerá automáticamente un submenú en el que preguntará si la salida se desea realizar hacia un «fichero» o un «programa». Una vez elegido el destino aparecerá otro menú preguntando si se quiere realizar la salida de la fila en curso o de todas las filas que componen la lista en curso. En ambos casos solicitará si se desea concatenar a un fichero ya existente.

## Instrucción PREVIOUS ROW

Esta instrucción activa la fila anterior a la actual de la lista en curso generada en el FORM. Su sintaxis es la siguiente:

### PREVIOUS ROW

La ejecución de esta instrucción equivale a pulsar la tecla asignada a la acción <fup> en el menú del FORM cuando existe una lista en curso.

**Ejemplo 41 [PFORM41]. Consulta de unidades, activando dos opciones en el menú para movernos por la lista en curso:**

```
database almacén

define
 form

 tables
 unidades
 end tables
```

```
 menu uni "Unidades"
 option "Siguiente"
 next row
 option "Anterior"
 previous row
 option "Salir"
 exit menu
 end menu
 end form
end define

min begin
 get query
 menu form
end main
```

El menú generado para este FORM contiene tres opciones, dos de las cuales sirven para overnos por la lista en curso activando la fila anterior y posterior a la actual. La tercera opción del menú abandonará el FORM.

## Instrucción QUERY

Esta instrucción permite editar las condiciones que el operador precise para cada uno de los campos accesibles en la pantalla. Su sintaxis es la siguiente:

### QUERY

En caso de indicar una condición que supere la longitud reservada en la sección SCREEN, automáticamente se habilitará la línea 22 para seguir con ella.

Las posibles condiciones que puede emplear el operador sobre cada uno de los campos son las siguientes:

|               |                                                                                                              |
|---------------|--------------------------------------------------------------------------------------------------------------|
| < valor       | Los valores de la columna donde se especifica deben ser menores que el valor especificado.                   |
| > valor       | Los valores de la columna donde se especifica deben ser mayores que el valor especificado.                   |
| <= valor      | Los valores de la columna donde se especifica deben ser menores o iguales que el valor especificado.         |
| >= valor      | Los valores de la columna donde se especifica deben ser mayores o iguales que el valor especificado.         |
| <> valor      | Los valores de la columna donde se especifica deben ser distintos al valor especificado.                     |
| = valor       | Los valores de la columna donde se especifica deben ser iguales al valor especificado.                       |
| >>            | La lista en curso aparecerá ordenada de mayor a menor respecto a la columna donde se asignen estos símbolos. |
| <<            | La lista en curso aparecerá ordenada de menor a mayor respecto a la columna donde se asignen estos símbolos. |
| valor1:valor2 | Los valores de la columna donde se especifica deben estar en el rango indicado por «valor1» y «valor2».      |
| *             | En columnas de tipo CHAR representa cualquier carácter.                                                      |
| ?             | En columnas de tipo CHAR representa un carácter obligatorio.                                                 |
| =NULL         | Los valores de la columna donde se especifica deben ser nulos.                                               |

`v1|v2|v3|...` Los valores de la columna donde se indiquen pueden ser los especificados en «v1», «v2», «v3», etc. En este caso, las condiciones que genera cada valor están enlazadas mediante el operador «OR».

**Ejemplo 42 [PFORM42]. Mantenimiento de la tabla «proveedores»:**

```
database almacén

define
 form

 tables
 proveedores
 end tables

 menu m1 "Proveedores"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query
 end menu
end form
end define

main begin
 menu form
end main
```

En este FORM se ha definido un menú con las cuatro opciones básicas de un mantenimiento: «Añadir», «Modificar», «Borrar» y «Consultar». Esta última contiene la instrucción QUERY, la cual nos permite establecer condiciones por cada una de las variables del FORM. Estas variables asociadas a columnas de la ase de datos no tienen or qué ser índices o formar parte de uno.

NOTA: Existen dos funciones internas de CTL («fquery()» y «fgetquery()») que simulan la acción de la instrucción QUERY.

## Instrucción READ KEY THROUGH FORM

Esta instrucción solicita la pulsación de una tecla, ejecutando la correspondiente acción si dicha tecla la tiene asignada en el FORM Su sintaxis es la siguiente:

### READ KEY THROUGH FORM

Esta instrucción actualiza la variable interna de CTL «lastkey». Esta instrucción activa las instrucciones no procedurales «ON KEY» y «ON ACTION».

**Ejemplo 43 [PFORM43]. Mantenimiento de la tabla «formpagos» con un menú «Pop-up»:**

```
database almacén

define
 form

 tables
 formpagos
 end tables
 menu m1 "FORMPAOS" down skip 2 line 7 column 30
 option "Añadir"
```

```
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query
 display "Pulsar "&& keylabel("fquit") &&
 " para aandonar la consulta" at 20,1
 view
 forever begin
 red key through frm
 if actionlabel(lastkey) ="fquit" then break
 end
 clear at 20,1
 end menu
layout
 box
 label "Formas de Pago"
 lines 5
 columns 40
end layout
end form
end define

main begin
 menu form
end main
```

En este caso es necesario emplear la instrucción READ KEY porque el menú definido para el mantenimiento es de tipo «Pop-up», y tanto éste como el del FORM emplean las teclas asignadas a las acciones <fup> y <fdwn> para moverse por la lista en curso y por las opciones, respectivamente. Por ello, cuando se realiza la consulta se emplea dicha instrucción para que actúe el FORM y no el menú. Para activar el menú en ese instante habrá que pulsar la tecla asignada a la acción <fquit>.

## Instrucción REMOVE

Esta instrucción permite borrar la fila en curso, actualizando la lista en curso. Este borrado está condicionado por la definición de la integridad referencial entre las tablas de la base de datos. Su sintaxis es la siguiente:

### REMOVE

Antes de producirse el borrado de la fila en curso, éste deberá confirmarse en un submenú compuesto por dos opciones: «SI» y «NO».

#### Ejemplo 44 [PFORM44]. Mantenimiento de la tabla «clientes»:

```
database almacen

define
 form

 tables
 clientes
 end tables

 menu clien "Clientes"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
```

```

 remove
 option "Consultar"
 query
 end menu
 end form
end define

main begin
 menu form
end main

```

Este programa define un FORM con un menú compuesto por las cuatro operaciones básicas: «Añadir», «Modificar», «Borrar» y «Consultar». La opción «Borrar» es la que emplea la instrucción REMOVE.

## Instrucción RETRY

Esta instrucción fuerza el reinicio de edición de las variables del FORM. Su sintaxis es la siguiente:

### RETRY

Esta instrucción sólo puede especificarse en cualquiera de las instrucciones no procedurales de la sección EDITING.

**Ejemplo 45 [PFORM45]. Mantenimiento de la tabla «albaranes» comprobando que la fecha del albarán sea posterior a la del día actual:**

```

database almacen

define
 frm

 tables
 albaranes
 end tables

 menu alba "Albaranes"
 option "Añadir"
 add
 option "Modificar"
 modify
 option "Borrar"
 remove
 option "Consultar"
 query end menu

 editing
 after editadd editupdate of fecha_albaran
 if fecha_albaran < today then begin
 pause "La fecha debe ser superior a la e hoy. Pulse [Return] para continuar"
 bell reverse
 retry
 end
 end editing
end form
end define

main begin
 menu form
end main

```

Este programa mantiene la tabla «albaranes». En caso de modificar o agregar la «fecha\_albaran» se comprueba que dicha fecha sea igual o posterior a la del día actual («today»). De no ser así se reiniciará la edición, solicitando una nueva fila en la operación de «Añadir» cancelando la actualización en caso de «Modificar».

## Funciones y variables Internas CTL

El FORM tiene asociadas una serie de variables internas del CTL que automáticamente adquieren unos valores por el hecho de realizar determinadas operaciones. Las variables internas del CTL empleadas por el FORM son las siguientes:

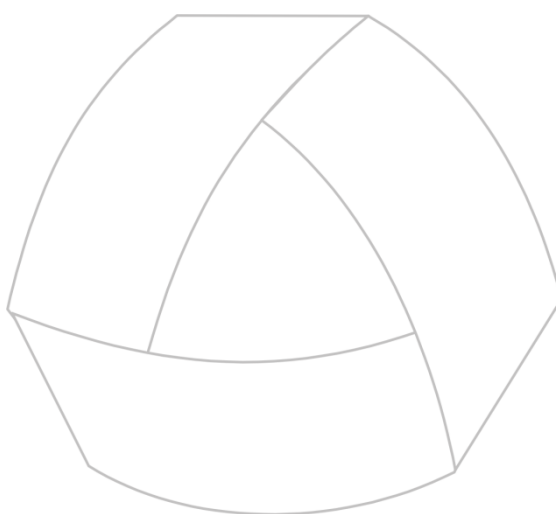
|             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| «curfield»  | Nombre del campo en edición del FORM.                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| «currec»    | Número de orden de la fila en curso dentro de la lista en curso del FORM.                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| «curtable»  | Nombre de la tabla activa del FORM.                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| «linenum»   | Número de línea en la que se encuentra el cursor de pantalla sobre una tabla del FORM definida como «LINES xx», siendo «xx» el número máximo de líneas de esta tabla que se mostrarán simultáneamente en pantalla. Por ejemplo, si se define una tabla como «LINE 10», «linenum» podrá valer «0» (si no existen filas en la lista en curso del FORM para esa tabla), o de «1» a «10», indicando así en cuál de la diez líneas estamos posicionados, independientemente de que el valor de «numrecs» sea mayor. |
| «newvalue»  | Variable interna que devolverá «TRUE» o «FALSE» dependiendo de si se ha modificado o no el contenido del último campo durante la edición.                                                                                                                                                                                                                                                                                                                                                                      |
| «numrecs»   | Número de filas que componen la lista en curso del FORM.                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| «operation» | Nombre de la operación que se está realizando con el FORM. Los valores que puede adquirir esta variable son los siguientes: ADD, UPDATE, DELETE y QUERY.                                                                                                                                                                                                                                                                                                                                                       |
| «cancelled» | Variable interna que devolverá «TRUE» o «FALSE» dependiendo de que el operador pulse la tecla asignada a la acción <fquit> en edición o de que se ejecute la instrucción CANCEL.                                                                                                                                                                                                                                                                                                                               |

NOTA: Todas estas variables contendrán el último valor asignado en cada caso. Por ejemplo, en la operación («operation») de añadir (ADD) sobre un FORM, si se invoca a un FRAME con una operación de modificar («FOR UPDATE»), al regresar al FORM la operación será UPDATE aunque se estuviesen agregando filas en el FORM.

Asimismo, existen ciertas funciones CTL que pueden emplearse para manejar el FORM. Estas funciones internas son las siguientes:

|                                           |                                                                                                                                                                                   |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>fquery( )</b>                          | Equivale a la instrucción QUERY, pero sólo devuelve las condiciones SQL que el usuario haya seleccionado.                                                                         |
| <b>fgetquery([condiciones [, orden]])</b> | Equivale a la instrucción «GET QUERY WHERE ». Devuelve el número de filas seleccionadas en la consulta. En caso de no indicar ningún parámetro limpia la lista en curso del FORM. |
| <b>addtoclist(rowid)</b>                  | Añade la fila con el «ROWID» indicado a la lista de la tabla en curso del FORM.                                                                                                   |
| <b>endquery()</b>                         | Muestra el número de filas de la lista de la tabla en curso del FORM y se posiciona en la primera.                                                                                |
| <b>gotoclist(expresión)</b>               | Activa la fila indicada por «expresión» dentro de la lista en curso de la tabla activa.                                                                                           |
| <b>gotoscreen(expresión)</b>              | Activa la pantalla indicada por «expresión».                                                                                                                                      |

NOTA: Para más información acerca de estas funciones y variables internas consulte el capítulo sobre [Conceptos Básicos del CTL](#) de este mismo volumen.



MultiBase  
MANUAL DEL PROGRAMADOR

## **CAPÍTULO 12. El FRAME: Objeto de entrada y salida de información**

## El FRAME: Definición y usos

El FRAME es un objeto no procedural que permite la edición de un conjunto de variables, estableciendo su estructura de representación, así como grupos de acciones sobre una o más variables. La principal diferencia entre un FORM y un FRAME es que el primero define variables asociadas con columnas de la base de datos, siendo considerado un medio habitual de mantenimiento interactivo de los datos de la base de datos, mientras que el FRAME, como hemos indicado, es un conjunto de variables asociadas (más convencionalmente, estructura), pero sin interacción implícita con la base de datos. Otra diferencia respecto al FORM es que un FRAME no puede manejar «scroll» de líneas.

Las funciones básicas de un FRAME son las siguientes:

- 1.— Editar todas las variables, o selectivamente algunas, para permitir al usuario:
  - Asignar valores a dichas variables.
  - Modificar sus valores.
  - Generar una condición compuesta (con operadores lógicos «AND» y «OR») al estilo de un FORM («Query by Forms»), reutilizable por las instrucciones dinámicas PREPARE y TSQL del CTSQL.
- 2.— Mostrar el contenido del conjunto de variables.
- 3.— Cargar y/o «volcar» estructuras de variables desde o hacia ficheros (ASCII o binarios). Esta faceta del FRAME está ligada con el manejo de STREAMS.

En este capítulo vamos a tratar únicamente las funcionalidades citadas en los puntos 1 y 2, mientras que el punto 3 se verá en detalle en el próximo capítulo, dedicado al STREAM.

## Estructura de un FRAME

A la hora de definir un FRAME hay que asignarle un nombre, pudiendo definirse varias estructuras FRAME dentro de un mismo módulo. Un FRAME está compuesto por diversas secciones que realizan funciones diferentes. Cada sección abre con su nombre, cerrándose con la palabra reservada «END» seguida de dicho nombre. El orden especificado en el caso de definir una sección es significativo. Las secciones de un FRAME, en su orden de definición, son las siguientes:

|                  |                                                                                                                                                        |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SCREEN</b>    | Sección opcional que describe el formato de presentación de la pantalla.                                                                               |
| <b>VARIABLES</b> | Sección opcional que describe la lista de variables del FRAME junto con la lista de atributos.                                                         |
| <b>MENU</b>      | Sección opcional que describe el menú asociado al FRAME.                                                                                               |
| <b>CONTROL</b>   | Sección opcional que indica instrucciones no procedurales que ejecutarán las acciones a tomar antes o después de comenzar la edición.                  |
| <b>EDITING</b>   | Sección opcional que contiene instrucciones no procedurales que ejecutarán las acciones precisas en las distintas fases de interacción con el usuario. |
| <b>LAYOUT</b>    | Sección opcional que permite la modificación de ciertos elementos de presentación de la pantalla del FRAME.                                            |

La sintaxis de la definición de un FRAME es la siguiente:

```
DEFINE
 [FRAME nombre_frame [LIKE tabla.*]
 {
 [SCREEN [MESSAGE número_ayuda]
```



```

 Texto [identificador1]
 Texto [identificador2]
 }
 [SCREEN [MESSAGE número_ayuda]
 {
 Texto [identificador3]
 ...
 }]
END SCREEN]

[VARIABLES
 [identificador = | UNTAGGED] variable_frame {tipo_sql | LIKE tabla.columna}
 [LINES n]
 [lista_atributos]
 ...
END VARIABLES]

[MENU nombre_menú [opciones]
 OPTION [nombre_opción] ["Etiqueta"] [opciones]
 ...
 OPTION [nombre_opción] ["Etiqueta"] [opciones]
 ...
 [OPTION ...
 ...]
END MENU]

[CONTROL
 [{(BEFORE | AFTER} {[ADD] [QUERY] [UPDATE] [DISPLAY] [LINE]}
 OF frame]
 ...
 [AFTER CANCEL OF frame
 ...]
 [ON KEY "tecla"
 ...]
 [ON ACTION "acción"
 ...]
END CONTROL]

[EDITING
 [{(BEFORE | AFTER} {[EDITADD] [EDITUPDATE] [EDITQUERY]} OF
 {frame | lista_variables}}
 ...
 [BEFORE CANCEL OF frame
 ...]
 [ON KEY "tecla"
 ...]
 [ON ACTION "acción"
 ...]
END EDITING]

[LAYOUT
 opciones
END LAYOUT]

END FRAME]
END DEFINE
...
...

```

En caso de definir un FRAME como «LIKE tabla.\*» habrá que especificar obligatoriamente la sección DATABASE al principio del módulo, no pudiendo incluirse en cambio ni la sección SCREEN ni la VARIABLES. Por ejemplo:

```
FRAME nombre_frame LIKE tabla.*
 [MENU ...]
 [CONTROL ...]
 [EDITING ...]
 [LAYOUT ...]
END FRAME
```

Si alguna variable del FRAME se define con la cláusula «LIKE» será obligatorio asimismo indicar de qué base de datos se extrae su definición.

### Ejemplo 1 [PFRAME1]. FRAME básico:

```
database almacen

define
 frame provin like provincias.*
end frame
end define

main begin
 input frame provin
end input
 pause "Pulse [Return] para continuar"
end main
```

Este programa define un FRAME con la misma estructura que la tabla «provincias» de la base de datos «almacen». Las variables del FRAME «provin» son las siguientes:

|                           |            |
|---------------------------|------------|
| provin.provincia required | (smallint) |
| provin.descripcion        | (char(20)) |
| provin.prefijo            | (smallint) |

El programa comienza con la petición de valores para las variables del FRAME. Esta petición se realiza mediante la instrucción INPUT FRAME. Una vez que el operador pulsa la tecla correspondiente a la acción <faccept> se abandona la entrada de datos, asignándose a las variables los valores introducidos por el operador. Por el contrario, si el operador pulsa la tecla asociada a la acción <fquit>, las variables del FRAME asumirán como valor «NULL», ya que éste es el valor por defecto empleado para cualquier tipo de variable definida en CTL.

En caso de no haber indicado la base de datos al principio del módulo, el programa produciría un error de compilación.

## Aspecto de la pantalla: Sección SCREEN

Al igual que sucedía en el FORM, el aspecto de la pantalla de mantenimiento en un FRAME se define en la sección SCREEN, representando cada SCREEN definida una página de dicho mantenimiento. Un FRAME admite hasta un total de 20 pantallas, es decir, 20 secciones SCREEN. El FRAME se encargará de ir activando una SCREEN u otra según se vayan activando los campos mientras se estén agregando o modificando valores de las variables o asignando condiciones de consulta (opciones añadir, modificar y consulta, respectivamente). No obstante, el operador podrá utilizar también las teclas asignadas a las acciones <fnext-page> y <fprevious-page> para pasar de una pantalla (SCREEN) a otra.

La sintaxis de la sección SCREEN es la siguiente:

```
[SCREEN [MESSAGE número_ayuda]
{
 Texto [identificador1]
 ...
 Texto [identificador2]
}
[SCREEN [MESSAGE número_ayuda]
{
 Texto [identificador3]
 ...
}]
END SCREEN]
```

Las llaves de cada SCREEN («{ }») deben encontrarse necesariamente en la primera columna de nuestro programa. De no ser así se producirá un error de compilación.

Como puede comprobarse en la sintaxis anterior, una página de un FRAME (SCREEN) puede extraerse de un fichero de ayuda mediante la cláusula «MESSAGE número\_ayuda». En el «número\_ayuda» del fichero de ayuda se encontrarán todos los literales («Texto») de la sección SCREEN en el mismo orden, línea y columna que si se incluyesen en la sintaxis de la propia sección SCREEN. Esta utilidad nos permitirá aislar los literales de la programación, por si en algún momento deseamos traducir la aplicación a otro idioma, o bien para que el usuario pueda configurarse sus propios literales mediante la edición de un fichero ASCII (fichero de ayuda). Para más información acerca de estos ficheros consulte el capítulo sobre [«Ficheros de Ayuda»](#) en este mismo volumen.

El lugar donde se mostrará el valor de cada variable del FRAME se representa mediante un «identificador». Los nombres de estos identificadores deben comenzar necesariamente por una letra, y podrán variar siempre y cuando se cambie también su definición dentro de la sección VARIABLES. Este «identificador» estará asignado a una variable del FRAME (sección VARIABLES). Los «identificadores» se encuentran incluidos entre corchetes («[ ]») o bien entre corchetes y separados por barras verticales («[... | ...]») si la siguiente variable está muy próxima. Con este procedimiento, a la hora de representar más de 27 campos de un solo carácter («char(1)») habrá que emplear subíndices en las variables de tipo CHAR, siendo la longitud que éstos expresen la de edición del campo, sin tener en cuenta la longitud del «identificador» definido en la sección SCREEN. Esto permite definir más «identificadores» de un solo carácter («char(1)») de los que en principio se podría al utilizar el rango «a-z». Por ejemplo:

```
database xxx

define
 frame
 screen
 {
 Literal :[a1]
 }
 end screen

 variables
 a1 = variable char(1)
 ...
 end variables

 ...
end frame
end define

main begin
 ...
end main
```

Al igual que sucedía en el FORM, la sección SCREEN del FRAME admite unos caracteres especiales para construir cajas y emplear determinados atributos de pantalla, tales como vídeo inverso o subrayados. Estos metacaracteres son los siguientes:

|    |                                                                                                                                                                                                                                                |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [  | Principio de identificador de SCREEN.                                                                                                                                                                                                          |
| ]  | Fin de identificador de SCREEN.                                                                                                                                                                                                                |
| !  | Principio de caja estándar.                                                                                                                                                                                                                    |
| @  | Fin de caja estándar.                                                                                                                                                                                                                          |
| ^  | Principio y fin de línea doble.                                                                                                                                                                                                                |
| ~  | Principio y fin de subrayado.                                                                                                                                                                                                                  |
| '  | Principio y fin de vídeo inverso.                                                                                                                                                                                                              |
| %  | Principio de caja simple (semigráficos simples).                                                                                                                                                                                               |
| *  | Fin de caja simple.                                                                                                                                                                                                                            |
| ?  | Principio de caja doble (semigráficos dobles).                                                                                                                                                                                                 |
| \$ | Fin de caja doble.                                                                                                                                                                                                                             |
|    | Equivalente a cerrar y abrir corchetes («[]»), se emplea para aproximar al máximo dos campos en SCREEN (separación de una columna), ya que el cierre de corchete y la apertura del siguiente identificador ocupan dos columnas de la pantalla. |

Estos metacaracteres pueden cambiarse indicando otros distintos en la variable de entorno DBMETACH (para más información consulte el capítulo sobre «Variables de Entorno» en el Manual del Administrador).

### IMPORTANTE

En caso de estar utilizando un terminal que no disponga de semigráficos dobles en su «set» de caracteres, en lugar de éstos se emplearán semigráficos simples.

La sección SCREEN no establece el orden de petición de los campos para el mantenimiento de la tabla activa (éste se fija en la sección VARIABLES, que se explica más adelante en este mismo capítulo).

### Ejemplo 2 [PFRAME2]. Empleo de los metacaracteres de la SCREEN en un FRAME:

```
define
 frame pantalla
 screen
 {
 ! P r e s e n t a c i ó n
 ? ? ? ? %
 ^ ^ 'Código'[a] *
 ^ ^
 $ $ $ $
 ?
 Nombre[b]
 ^
 Dirección[c]
 ^
 $
 @
 }
 end screen

 variables
 a = codigo smallint reverse
```

```

 b = nombre char upshift
 c = direccion char
 end variables
end frame
end define

main begin
 display frame pantalla at 5,10
 pause "Pulsar [Return] para continuar" reverse bell
 clear frame pantalla
end main

```

Este programa define un FRAME cuyo nombre es «pantalla». En él se especifica una sección SCREEN en la que se han incluido la mayoría de los metacaracteres que se pueden emplear para formar cajas, utilizar distintos atributos de vídeo, etc. Asimismo, dentro de la sección VARIABLES se han definido tres variables (identificadores «a», «b» y «c» de la SCREEN) indicando su tipo de dato y atributos.

La única operación que se realiza con este FRAME es presentar su aspecto mediante la instrucción DISPLAY FRAME, por lo que en este caso no se podrán asignar valores a las variables: «codigo», «nombre» y «direccion». A continuación, aparece un mensaje en vídeo inverso, a la par que suena el pitido de la pantalla, indicando que debe pulsarse [RETURN] para continuar, tras lo cual se limpia el FRAME mediante la instrucción CLEAR FRAME.

### Ejemplo 3 [PFRAME3]. FRAME con dos pantallas:

```

database almacén

define
 frame pantalla
 screen
 {
 !
 A s i g n a c i ó n Pag.: 1
 %
 'Código'[a] *

 ?
 Nombre: [b]
 Dirección: [c]
 $
 @
 }
 screen
 {
 !
 A s i g n a c i ó n Pag.: 2
 %
 'Código'[a] *

 ?
 Teléfono: [d]
 Provincia: [e |f]
 $
 @
 }
end screen

variables
 a = código smallint reverse
 b = nombre char upshift
 c = dirección char
 d = teléfono char
 e = provincia like provincias.provincia

```

```
 lookup f = descripcion
 joining provincias.provincia
 end variables
end frame
end define

main begin
 input frame pantalla at 5,10
end input
end main
```

Este módulo realiza la petición de valores sobre las variables definidas para el FRAME «pantalla». Estas variables se encuentran diseminadas entre dos pantallas (SCREEN). La activación de una u otra dependerá de la variable que se encuentre activa en cada momento. En este caso se ha definido la base de datos al principio porque la variable «provincia» emplea la cláusula «LIKE» para su definición y, además, utiliza el atributo LOOKUP. A su vez, este atributo, dependiendo del valor introducido en la variable «provincia» extrae información de una tabla de la base de datos. Una vez aceptados los valores (acción <faccept>), éstos quedan asignados a sus respectivas variables. Por el contrario, si se pulsa la tecla asignada a la acción <fquit>, no se asignarán valores. En cualquier caso, tanto si se aceptan los valores como si se cancela la operación, acto seguido finalizará el programa.

## Variables componentes del FRAME: Sección VARIABLES

---

La sección VARIABLES de un FRAME se tendrá que especificar cuando se dé cualquiera de las siguientes circunstancias:

- Cuando se incluya algún identificador en la sección SCREEN.
- Cuando no se haya definido el FRAME con la cláusula «LIKE tabla.\*».

Si se incluye la cláusula «LIKE tabla.\*», las variables del FRAME coincidirán tanto en tipo de dato y longitud como en atributos y en posicionamiento dentro de la estructura del FRAME con las columnas de la tabla indicada.

Por el contrario, si el FRAME se define sin la cláusula «LIKE tabla.\*», habrá que especificar siempre el tipo de dato y, opcionalmente, los atributos para cada variable componente del FRAME, o bien emplear la cláusula «LIKE tabla.columna» para definir específicamente cada una de las variables (en este segundo caso habrá que indicar el nombre de la tabla y de la columna). La sintaxis de la definición de una variable en un FRAME es la siguiente:

```
[identificador = | UNTAGGED] variable_frame {tipo_sql | LIKE tabla.columna}
[LINES n] [lista_atributos]
```

El orden de definición de las variables de un FRAME será el que determine el orden para la petición de datos al operador, pudiendo estar cada variable en una pantalla diferente. En este caso, cada pantalla se presentará en el momento que se active una de las variables incluidas en ella al emplear la instrucción INPUT FRAME.

Todos los identificadores definidos en la sección SCREEN tienen que estar asignados a la definición de una variable del FRAME. La asignación de un identificador a una variable implica que los valores de dicha variable se representarán en pantalla en el momento que aquélla adquiera un valor. Las variables que no se desea que aparezcan en pantalla deberán definirse con la palabra reservada «UNTAGGED».

NOTA: Cuando se define una variable con el atributo «LINES n», la «lista\_atributos» queda sin efecto. Estas variables sólo pueden ser de tipo CHAR.

La «lista\_atributos» es una lista de atributos del CTL separados por un espacio en blanco (para más información sobre estos atributos consulte el capítulo sobre [«Conceptos Básicos del CTL»](#) en este mismo volumen).

Las variables de un FRAME se referencian como «variable» o bien como «frame.variable» (esta referencia es idéntica a la empleada para las variables asociadas a columnas en el FORM). Las variables de un FRAME pueden intervenir como variables «hosts» dentro de una instrucción de SQL. Asimismo, pueden pasarse como parámetros a otros programas o a funciones locales o externas incluidas en otro módulo del programa.

**Ejemplo 4 [PFRAME4]. Petición de valores sobre una variable del FRAME con «scroll» de líneas:**

```
define
 frame lineas
 screen
 {
 !
 L í n e a s
 ?
 Código[a]
 $

 Empresa[b]
 'O b s e r v a c i o n e s : '

 [c]

 }

 @

 end screen
 variables
 a = codigo smallint reverse
 b = empresa char upshift
 c = observa char lines 6
 end variables
 end frame
end define

main begin
 input frame lineas
 end input
end main
```

Este programa realiza la petición de valores sobre las variables del FRAME «lineas». Una de estas variables («observa») se presenta en 6 líneas. En este caso, al introducir un valor que sobrepase la longitud de una línea habrá que pulsar [RETURN] para continuar con la siguiente hasta un total de seis. Hemos de tener en cuenta que los valores introducidos sobre estas variables no se graban en ningún sitio, sino que desaparecen en el momento que finaliza la ejecución del programa.

**Ejemplo 5 [PFRAME5]. Ejemplo de atributo LOOKUP:**

```
database almacen

define
 frame rangos
 screen
 {
 !
 R a n g o s

 %
 Desde Cliente [a |b]
 %
 Hasta Cliente [c |d]
 }
```

```

 *
 @
}
end screen
variables
 a = desde like clientes.cliente default 1
 check ($ is not null)
 lookup b = empresa joining clientes.cliente
 c = hasta like clientes.cliente
 check ($ is not null and $ >= desde)
 default 99 lookup d = empresa joining clientes.cliente
end variables
end frame
end define

main begin
 input frame rangos for add
end input
window from select cliente, empresa, direccion1
 from clientes
 where clientes.cliente between $rangos.desde and $rangos.hasta
 10 lines label "C l i e n t e s"
 at 7,3
clear frame rangos
end main

```

Este programa pide valores sobre las variables del FRAME «rangos». Este FRAME se ha declarado con dos variables: «desde» y «hasta». En la sección VARIABLES se definen los atributos DEFAULT, LOOKUP y CHECK. El atributo DEFAULT tiene efecto sólo cuando la entrada de valores sobre las variables incluye la cláusula «FOR ADD» en la instrucción INPUT FRAME, como sucede en nuestro ejemplo. Asimismo, el atributo CHECK en la definición de la variable «hasta» comprueba que ésta no pueda ser nula y, además, que el valor asignado sea igual o mayor que el introducido en la variable «desde». Por su parte, el atributo LOOKUP reflejará (si existen) los nombres de las empresas correspondientes a los códigos introducidos como rangos. Una vez que el operador acepte los valores introducidos se mostrará una ventana con aquellos clientes cuyo código se encuentre en el rango especificado. Esta ventana se construye mediante la instrucción WINDOW. Por último, al abandonar esta ventana (acción <fquit>) se eliminará de la pantalla tanto esta ventana como la SCREEN del FRAME.

## Menú del FRAME: Sección MENU

La sección MENU del FRAME incluirá un menú CTL que contendrá las distintas opciones que se podrán llevar a cabo con dicho objeto. La estructura y sintaxis de esta sección es idéntica a la que pueda tener cualquier otro menú estándar del CTL (para más información sobre los distintos tipos de menús que pueden definirse en CTL [consulte el capítulo 14](#) de este mismo volumen).

La sintaxis de la sección MENU del FRAME (válida también para cualquier menú CTL) es la siguiente:

```

[MENU nombre_menú [opciones]
 OPTION [nombre_opción] ["Etiqueta"] [opciones]
 ...
 OPTION [nombre_opción] ["Etiqueta"] [opciones]
 ...
 [OPTION ...
 ...]
END MENU]

```

Dependiendo de las «opciones» indicadas a dicho menú, éste será de tipo «Lotus» (de iconos en Windows) o bien «Pop-up» (cláusula «DOWN»).



La referencia a un menú del FRAME se realiza con la instrucción de CTL «MENU FRAME nombre\_frame». Esta opción se encarga de mostrar en pantalla el aspecto de la SCREEN definida, así como todas las opciones de que se compone el menú especificado en la sección MENU. No obstante, esta instrucción se puede simular mediante la combinación de la instrucción DISPLAY FRAME y «MENU nombre\_menu», siendo ésta la forma de llamar a cualquier menú en CTL. Por ejemplo:

```
...
...

main begin
 menu frame nombre_frame
end main
```

o también:

```
...
...

main begin
 display frame nombre_frame
 menu nombre_menu
 clear frame nombre_frame
end main
```

En este caso, «nombre\_menu» es un menú que puede estar definido tanto en la sección MENU del FRAME como en la sección MENU de un módulo CTL, es decir, después de la sección MAIN.

El menú de un FRAME emplea una serie de acciones para manejar la pantalla del FRAME. Estas acciones no podrán detectarse con la instrucción no procedural «ON ACTION» en la sección CONTROL del FRAME. El siguiente cuadro muestra algunas de las acciones más utilizadas en un menú del FRAME.

| ACCIONES EMPLEADAS EN EL MENU DEL FRAME |                                   |
|-----------------------------------------|-----------------------------------|
| ACCIÓN                                  | OPERACIÓN QUE REALIZA             |
| <fright>                                | Opción derecha del menú.          |
| <fleft>                                 | Opción izquierda del menú.        |
| <fbackspace>                            | Opción izquierda del menú.        |
| <fnext-page>                            | Próxima SCREEN.                   |
| <fprevious-page>                        | SCREEN anterior.                  |
| <freturn>                               | Selección de una opción del menú. |
| <faccept>                               | Selección de una opción del menú. |
| <fquit>                                 | Abandonar el menú.                |
| <fredraw>                               | Repintado de la pantalla.         |
| <fhelp>                                 | Ayuda de teclas.                  |
| etc.                                    |                                   |

Un menú CTL puede configurarse también mediante un mensaje de un fichero de ayuda. La forma de configurar este tipo de menús se explica en el capítulo 16 de este mismo manual.

**Ejemplo 6 [PFRAME6]. Actualización del precio de venta de los artículos incluidos en el rango establecido por el operador:**

```
database almacén

define
 frame rangos
 screen
{
 ! Rangos
```

```

 Desde artículo [a]

 Hasta artículo [b]

 @
 }

 end screen
 variables
 a = desde like articulos.articulo check ($ is not null)
 b = hasta like articulos.articulo
 check ($ is not null and $ >= desde)
 end variables
 end frame
 frame arti
 screen

{
! A r t í c u l o s
 %
 Código Artículo[a]
 *

 Descripción [b]

 Precio Venta [c]

 @
}

 end screen
 variables
 a = articulo like articulos.articulo
 b = descripcion like articulos.descripcion
 c = pr_vent like articulos.pr_vent reverse
 end variables

 menu actualiza "Actualización"
 option "Siguiente"
 fetch precio_venta
 if found = false then exit menu
 view
 option "Modificar precio"
 input frame arti for update variables pr_vent at 7,10
 end input
 if cancelled = false then update articulos set pr_vent = $arti.pr_vent
 where current of precio_venta
 option "Salir"
 exit menu
 end menu
 end frame
end define

main begin
 input frame rangos for add
 end input
 declare cursor precio_venta for select articulo, descripcion, pr_vent
 by name on arti
 from articulos
 where articulos.articulo between $rangos.desde and $rangos.hasta
 for update
 display frame arti at 7,10
 open precio_venta
 fetch precio_venta
 menu actualiza
 close precio_venta
 clear frame rangos

```

```
clear frame arti
end main
```

Este programa define dos FRAMES: «rangos» y «arti». El primero de ellos está compuesto por las variables «desde» y «hasta», las cuales contendrán el rango de aquellos artículos de los que se desea actualizar su precio de venta. Los valores a introducir en estas variables no deberán ser nulos, teniendo en cuenta, además, que el valor de «hasta» deberá ser necesariamente superior al de «desde».

Una vez aceptados los valores sobre estas variables del FRAME «rangos» (acción <faccept>), se declara un CURSOR que devolverá una tabla derivada compuesta por los artículos cuyo código se encuentre en el rango especificado por «desde» y «hasta». Cada una de las filas que componen la tabla derivada de este CURSOR se carga sobre las variables del segundo FRAME («arti»). A continuación, se activa el menú que controla dicho FRAME, el cual incluye las siguientes opciones:

- «Siguiente»: Esta opción lee la siguiente fila del CURSOR, mostrándola en pantalla. En caso de no encontrar más filas se abandona el menú.
- «Modificar precio»: Esta opción modifica el precio del artículo en curso mediante la instrucción INPUT FRAME y la cláusula «FOR UPDATE». La cláusula «VARIABLES» indica que la única variable a actualizar es «pr\_vent». En caso de aceptar la modificación, el nuevo precio asignado se actualiza en la tabla «articulos» de la base de datos.
- «Salir»: Abandona el menú.

Al salir del menú FRAME se cierra el CURSOR y se limpian los dos FRAMES de la pantalla.

## Formas de edición: Secciones CONTROL y EDITING

Como ya indicamos al principio de este capítulo, una de las funciones básicas de un FRAME es la edición de sus variables. Esto se consigue mediante la instrucción INPUT FRAME, cuya sintaxis es la siguiente:

```
INPUT FRAME nombre_frame [FOR {ADD | UPDATE}]
 [VARIABLES lista_variables] [AT línea, columna]
END INPUT

INPUT FRAME nombre_frame FOR QUERY
 [VARIABLES lista_variables]
 [{BY NAME | ON lista_columnas} TO variable]
 [AT línea, columna]
END INPUT
```

NOTA: Para información más detallada sobre la sintaxis y características de esta instrucción puede consultar la sección 12.9.5 de este capítulo o bien el Manual de Referencia.

Como se puede comprobar en la sintaxis anterior, la edición de variables se contempla en tres operaciones diferentes: «FOR ADD», «FOR UPDATE» y «FOR QUERY».

El objeto FRAME dispone de dos secciones (CONTROL y EDITING) en las que, mediante instrucciones no procedurales, se sitúan los procesos (conjuntos de instrucciones CTL) a realizar en cada situación de dicho objeto, es decir, en determinados momentos de la interacción del programa con el usuario. Esta interacción depende de forma directa de la operación que se esté realizando (añadir, modificar, consultar, repintado o cancelación de la edición).

La sección CONTROL contiene instrucciones no procedurales que se activan antes o después de concluir cualquiera de las siguientes operaciones con las variables del FRAME: añadir, modificar, consultar, repintar y cancelar. La estructura de esta sección es la siguiente:

```
[CONTROL
 [{BEFORE | AFTER} {[ADD] [QUERY] [UPDATE] [DISPLAY] [LINE]}
 OF frame]
 ...
 [AFTER CANCEL OF frame
 ...]
 [ON KEY "tecla"
 ...]
 [ON ACTION "acción"
 ...]
END CONTROL]
```

Por su parte, la sección EDITING está compuesta por instrucciones no procedurales que se activan antes o después de editar (operaciones de añadir, modificar y consultar) cada una de las variables del FRAME o bien de toda la estructura de variables, es decir, antes o después de finalizar la operación con el FRAME. La sintaxis de esta sección es la siguiente:

```
[EDITING
 [{BEFORE | AFTER} {[EDITADD] [EDITUPDATE] [EDITQUERY]} OF]
 {frame | lista_variables}}
 ...
 [BEFORE CANCEL OF frame
 ...]
 [ON KEY "tecla"
 ...]
 [ON ACTION "acción"
 ...]
END EDITING]
```

Asimismo, en ambas secciones existen instrucciones no procedurales que se activan al pulsar una tecla («ON KEY») o ejecutar una acción («ON ACTION») incluida en el fichero de acciones «tactions». La sección CONTROL podrá activar dichas instrucciones no procedurales al pulsar la tecla o ejecutar la acción en cualquiera de las opciones del menú del FRAME o bien al ejecutar la instrucción «READ KEY THROUGH FRAME». Sin embargo, en la sección EDITING dichas instrucciones se activarán cuando el operador las pulse en edición (operaciones añadir, modificar y consultar). En ningún caso se podrán detectar teclas que se empleen por defecto en el manejo tanto del menú como en edición de campos. Por ejemplo, estas instrucciones no procedurales comunes a ambas secciones no podrán detectar la tecla [RETURN] (acción <return>), ya que ésta se emplea tanto para la selección de una opción en un menú (sección CONTROL) como para activar la siguiente variable en edición (sección EDITING).

### IMPORTANTE

Las instrucciones no procedurales de la sección CONTROL actúan antes de comenzar la edición y justo después de finalizarla. En caso de existir sección MENU habría que indicar que estas instrucciones actúan antes o después de activar las opciones de dicho menú. Sin embargo, las instrucciones de la sección EDITING actúan cuando nos encontramos editando los valores de las variables del FRAME.

En los siguientes apartados se explican las posibles acciones que se podrán llevar a cabo en estas dos secciones, dependiendo de las operaciones que realice el operador (añadir, «FOR ADD»; modificar, «FOR UPDATE» y consultar, «FOR QUERY»). En caso de no especificar ninguno de estos operadores se asumirá por defecto «FOR UPDATE».

## Entrada de nuevos valores sobre variables

La asignación de nuevos valores sobre las variables de un FRAME se produce cuando se ejecuta la instrucción INPUT FRAME con la cláusula «FOR ADD». Este operador inicializa las variables del FRAME con el valor asignado en el atributo DEFAULT. En caso de no disponer de este atributo, las variables se inicializarán con el valor por

defecto en CTL, es decir, el valor nulo («NULL»). Al igual que sucedía en el FORM, para aceptar los valores sobre las variables habrá que pulsar la tecla asignada a la acción <faccept> o bien ejecutar la instrucción EXIT EDITING en la sección EDITING. Para cancelar la edición habrá que pulsar la tecla asignada a la acción <fquit>.

La secuencia de ejecución de las instrucciones no procedurales de las secciones CONTROL y EDITING es la siguiente:

- 1.— «AFTER DISPLAY OF nombre\_frame (CONTROL)». Al ejecutar la instrucción INPUT FRAME con la cláusula «FOR ADD» se inicializan las variables del FRAME.
- 2.— «BEFORE EDITADD OF nombre\_frame (EDITING)». Ejecución antes de entrar en edición de variables.
- 3.— «BEFORE EDITADD OF variable (EDITING)». Ejecución antes de activar la variable del FRAME.
- 4.— «AFTER EDITADD OF variable (EDITING)». Esta instrucción se ejecutará al abandonar la variable del FRAME.
- 5.— «AFTER EDITADD OF nombre\_frame (EDITING)». Se ejecutará una vez pulsada la tecla asignada a la acción <faccept>.
- 6.— «BEFORE ADD OF nombre\_frame (CONTROL)». Equivalente a la instrucción anterior pero en la sección CONTROL.
- 7.— «AFTER ADD OF nombre\_frame (CONTROL)». Última instrucción antes de activar el menú (en caso de especificar la sección MENU del FRAME).

Por su parte, al cancelar la edición se activan las siguientes instrucciones no procedurales de ambas secciones:

- 1.— «BEFORE CANCEL OF nombre\_frame (EDITING)». Esta instrucción se ejecutará después de pulsar la tecla asignada a la acción <fquit>, pudiendo incluso abortar la cancelación y continuar con la edición hasta que se cumpla una condición.
- 2.— «AFTER CANCEL OF nombre\_frame (CONTROL)». Ejecución después de abandonar la edición.
- 3.— «AFTER DISPLAY OF nombre\_frame (CONTROL)». Limpia los valores introducidos sobre las variables del FRAME.

Dependiendo de la tecla pulsada por el operador, la activación de instrucciones se consigue mediante las instrucciones no procedurales «ON KEY» y «ON ACTION» de las secciones CONTROL y EDITING.

En la sección EDITING, estas instrucciones se activarán, al intentar realizar un alta, cuando el operador pulse la tecla correspondiente a la acción indicada en la instrucción (cuando el cursor del terminal se encuentre dentro de alguna variable de la SCREEN).

Por lo que respecta a la sección CONTROL, las instrucciones no procedurales se activarán cuando no estemos en edición, es decir, cuando podamos movernos sobre las distintas opciones del menú del FRAME o bien cuando se ejecute la instrucción «READ KEY THROUGH FRAME», si es que no existe un menú definido.

Las instrucciones no procedurales «ON KEY» y «ON ACTION» sólo podrán detectar aquellas teclas que no tengan asignada ya una acción en el propio objeto sobre el que se detectan. Por ejemplo, nunca se podrá detectar la tecla correspondiente a la acción <freturn>, ya que dicha tecla se emplea tanto en el menú del FRAME como en edición para seleccionar una opción del menú (sección CONTROL) y para aceptar un valor en una variable del FRAME, activando a su vez el siguiente campo (sección EDITING).

**Ejemplo 7 [PFRAME7]. Instrucciones no procedurales que se ejecutan en la operación de añadir del FRAME. Simulación de la instrucción ADD del FORM para agregar filas sobre una tabla:**

```
database almacén
define
 frame provin
 screen
```

```

{
! FRAME Provincias

 Código: [a]
 Descripción.....: [b]
 Prefijo: [c]

 @
}

end screen

variables
 a = provincia like provincias.provincia
 b = descripcion like provincias.descripcion
 c = prefijo like provincias.prefijo
end variables

menu prov "Provincias"
 option "Añadir"
 forever begin
 input frame provin for add
 end input
 if cancelled = true then break
 end
 option "Salir"
 exit menu
end menu

control
 after add of provin
 insert into provincias (provincia, descripcion, prefijo)
 values ($provin.provincia, $provin.descripcion,$provin.prefijo)
end control

editing
 after editadd of provin.provincia
 select "" from provincias
 where provincias.provincia = $provin.provincia
 if found = true then begin
 pause "La provincia ya existe. Pulsar [Return] para continuar"
 reverse bell
 let provin.provincia = null
 next field "provincia"
 end
 before editadd of provin.provincia
 message "Pulsar " && keylabel ("fuser1") && " para ver provincias"
 reverse
 on action "fuser1"
 if curfield = "provincia" then
 window from select provincia, descripcion, prefijo from provincias
 10 lines label "P r o v i n c i a s"
 at 5,20
 end
end editing
end frame
end define

main begin
 menu frame provin
end main

```

Este programa simula la instrucción ADD del FORM mediante el manejo del FRAME. Como ya vimos en el capítulo anterior, la instrucción ADD se empleaba para agregar filas nuevas sobre la tabla activa del FORM. En este caso, la pantalla (SCREEN) del FRAME junto con las dos opciones del menu (MENU) se muestran en pantalla al

ejecutar la instrucción MENU FRAME. En dicho menú existe una opción «Añadir» que construye un bucle con la instrucción INPUT FRAME con la cláusula «FOR ADD». Este bucle finalizará cuando el operador pulse la tecla asignada a la acción <fquit> en alguna de las variables que componen el FRAME, activándose de nuevo las opciones del menú. En caso contrario, esta instrucción pedirá constantemente valores sobre las variables, grabando dichos valores en la tabla «provincias» cuando el operador pulse la tecla asignada a la acción <faccept>. El FRAME tiene la misma estructura que la tabla «provincias».

Mientras nos encontramos en edición sobre la variable «provincia» aparece un mensaje indicando la tecla que debe accionar el operador para consultar las filas existentes en la tabla «provincias» («BEFORE EDITADD OF variable»). En caso de pulsar dicha tecla aparecerá una ventana con todas las filas grabadas en dicha tabla («ON ACTION acción»). Después de asignar un valor a la variable «provincia» se comprueba que dicho código no exista ya en la tabla «provincias» («AFTER EDITADD OF variable»).

**Ejemplo 8 [PFRAME8]. Consulta sobre la tabla «proveedores» con petición del rango de los códigos a consultar:**

```
database almacén

define
 variable común like proveedores.proveedor
 frame rangos
 screen
{
 !
 C o n s u l t a

 Desde proveedor: [a] [b
 Hasta proveedor: [c] [d
]
 @
}

end screen

variables
 a = desde like proveedores.proveedor
 b = empresad like proveedores.empresa
 c = hasta like proveedores.proveedor
 d = empresah like proveedores.empresa
end variables

menu consulta "Consulta"
 option "Rangos"
 input frame rangos for add variables desde hasta
 end input
 option "Salir"
 exit menu
end menu

control
 after add of rangos
 window scroll from select * from proveedores
 where proveedores.proveedor between $rangos.desde and $rangos.hasta
 10 lines label "P r o v e e d o r e s ——>"
 at 7,1
 end control

editing
 after editadd of rangos.desde
 let empresad = empre(desde)
 after editadd of rangos.hasta
 let empresah = empre(hasta)
 before editadd of rangos.desde rangos.hasta
 message "Pulsar " && keylabel("fuser1") && " para consultar" reverse
```

```
on action "fuser1"
 window from select proveedor, empresa
 from proveedores
 10 lines label "P r o v e e d o r e s"
 at 5,15
 set comun from 1
 if curfield = "desde" then let rangos.desde = comun
 else let rangos.hasta = comun
end editing
end frame
end define

main begin
 menu frame rangos
end main

function empre (campo)
 empresa like proveedores.empresa
begin
 declare cursor empre for
 select empresa by name from proveedores
 where proveedores.proveedor = $campo
 open empre
 fetch empre
 close empre
 return empresa
end function
```

Este programa define un FRAME compuesto por cuatro variables, dos de las cuales equivalen al rango de los códigos de proveedores a consultar («desde» y «hasta»), mientras que las otras dos mostrarán, en caso de que existan, las empresas de los códigos asignados anteriormente.

En primer lugar, el programa activa la SCREEN del FRAME junto con su menú (MENU). Esta operación la realiza la instrucción MENU FRAME. El menú contiene dos opciones, una para realizar la petición de valores sobre las variables del FRAME («desde» y «hasta») para conseguir la consulta, y otra para salir del menú del FRAME (y por lo tanto del programa).

Antes de entrar en cada una de las variables que representan el rango «desde» y «hasta» aparecerá un mensaje indicando la tecla a emplear para mostrar una ventana con todos los códigos de los proveedores existentes en la base de datos, pudiendo elegir uno de ellos («BEFORE EDITADD OF variables» y «ON ACTION acción»). Asimismo, en el momento que se introduce un valor en cualquiera de los dos códigos («desde» o «hasta») se muestra, si existe, la empresa de cada uno de ellos. Esta empresa se lee en un CURSOR en el que la variable «host» es el parámetro recibido en la función. Esta empresa se devuelve a la sección EDITING mediante la instrucción RETURN. Por último, una vez aceptados los valores sobre las variables «desde» y «hasta», se muestra una ventana con todos los proveedores cuyo código se encuentra en el rango especificado.

## Modificación de valores sobre variables del FRAME

La modificación de los valores de las variables de un FRAME sólo se podrá realizar mediante la instrucción INPUT FRAME con la cláusula «FOR UPDATE». Para validar dicha modificación habrá que pulsar la tecla asignada a la acción <accept> o bien ejecutar la instrucción EXIT EDITING del CTL. Por su parte, para cancelar una modificación se empleará la tecla asignada a la acción <quit> o bien la instrucción CANCEL.

La operación de modificar en las secciones CONTROL y EDITING del FRAME es muy parecida a la de asignación de nuevos valores explicada en el apartado anterior. En consecuencia, el orden de las instrucciones no procedurales que se activan en la operación de modificación son las siguientes:

1.— «AFTER DISPLAY OF nombre\_frame (CONTROL)». La ejecución de la instrucción INPUT FRAME con la cláusula «FOR UPDATE» lleva implícita el repintado del FRAME.



- 2.— «BEFORE EDITUPDATE OF nombre\_frame (EDITING)». Ejecución antes de entrar en edición de variables.
- 3.— «BEFORE EDITUPDATE OF variable (EDITING)». Ejecución antes de activar la variable del FRAME.
- 4.— «AFTER EDITUPDATE OF variable (EDITING)». Esta instrucción se ejecutará al abandonar la variable del FRAME, activándose la siguiente especificada en la sección VARIABLES.
- 5.— «AFTER EDITUPDATE OF nombre\_frame (EDITING)». Se ejecutará una vez pulsada la tecla asignada a la acción <faccept>.
- 6.— «BEFORE UPDATE OF nombre\_frame (CONTROL)». Equivalente a la instrucción pero en la sección «CONTROL».
- 7.— «AFTER UPDATE OF nombre\_frame (CONTROL)». Última instrucción antes de activar el menú (si se especificó la sección MENU del FRAME).

Por su parte, las instrucciones no procedurales que se activan en ambas secciones al cancelar la edición son las siguientes:

- 1.— «BEFORE CANCEL OF nombre\_frame (EDITING)». Esta instrucción se ejecuta después de pulsar la tecla asignada a la acción <fquit>, pudiendo incluso interrumpir la cancelación hasta que se cumpla una condición.
- 2.— «AFTER CANCEL OF nombre\_frame (CONTROL)». Se ejecuta después de abandonar la edición.
- 3.— «AFTER DISPLAY OF nombre\_frame (CONTROL)». El control del programa vuelve al menú del FRAME (en caso de que exista), lo que conlleva el repintado.

Dependiendo de la tecla pulsada por el operador, la activación de instrucciones se consigue mediante las instrucciones no procedurales «ON KEY» y «ON ACTION» de las secciones CONTROL y EDITING.

En la sección EDITING, estas instrucciones se activarán, al intentar realizar una modificación de valores y un alta, cuando el operador pulse la tecla correspondiente a la acción indicada en la instrucción (cuando el cursor del terminal se encuentre dentro de alguna variable de la SCREEN).

Por lo que respecta a la sección CONTROL, las instrucciones no procedurales se activarán cuando no estemos en edición, es decir, cuando podamos movernos sobre las distintas opciones del menú del FRAME o bien cuando se ejecute la instrucción «READ KEY THROUGH FRAME», si es que no existe un menú definido.

Las instrucciones no procedurales «ON KEY» y «ON ACTION» sólo podrán detectar aquellas teclas que no tengan asignada ya una acción en el propio objeto sobre el que se detectan. Por ejemplo, nunca se podrá detectar la tecla correspondiente a la acción <freturn>, ya que dicha tecla se emplea tanto en el menú del FRAME como en edición para seleccionar una opción del menú (sección CONTROL) y para aceptar un valor en una variable del FRAME, activando a su vez el siguiente campo (sección EDITING).

NOTA: En las versiones de MultiBase para Windows, y gracias al empleo del ratón, el operador podrá seleccionar directamente la variable a la que desea modificar su valor. En y UNIX será necesario pasar por todas las variables anteriores a la que se desea modificar.

**Ejemplo 9 [PFRAME9] y [ACTU\_ARTI.SQL]. Marcado en un FORM sobre una tabla temporal de aquellos artículos de los que se desea actualizar su precio de venta mediante una estructura FRAME:**

```
database almacén
define
 form
 screen
{
 Marca Artículo
 [a] [b]]
```

```

 }

 end screen

 tables
 actu_arti 8 lines
 end tables

 variables
 a = marca
 b = descripcion
 end variables

 editing
 before editupdate of actu_arti
 if marca = "*" then let marca = " "
 else let marca = "*"
 exit editing
 end editing

 layout
 box
 label "M a r c a d o"
 line 3
 column 10
 end layout
 end form

 frame precios
 screen
{
 Artículo:[a]
 Proveedor: [b][c]
 Descripción: [d]
 Precio Venta: [e]
}

 end screen

 variables
 a = articulo like articulos.articulo
 b = proveedor like articulos.proveedor
 lookup c = empresa
 joining proveedores.proveedor
 d = descripcion like articulos.descripcion
 e = pr_vent like articulos.pr_vent
 format "##,###,###.##"
 end variables

 control
 after update of precios
 if newvalue = true then
 update articulos set pr_vent = $precios.pr_vent
 where articulos.articulo = $precios.articulo
 end control

 editing
 after editupdate of precios.pr_vent
 exit editing
 end editing

```

```

 layout
 box
 label "A c t u a l i z a c i ó n"
 line 7
 column 20
 end layout
 end frame
 end define

 global
 options
 no message
 end options
 end global

 main begin
 create temp table actu_arti (marca char(1) not null, articulo smallint not null,
 proveedor integer not null, descripcion char(20)) primary key (marca, articulo, proveedor)
 insert into actu_arti (marca, articulo, proveedor, descripcion)
 select " ", articulo, proveedor, descripcion
 from articulos
 get query
 forever begin
 read key through form
 switch
 case actionlabel(lastkey) = "fquit"
 break
 case actionlabel(lastkey) in ("freturn", "faccept")
 modify
 end
 end
 end
 declare cursor cambio_precio for select articulos.articulo, articulos.proveedor,
 articulos.descripcion, articulos.pr_vent
 by name on precios
 from articulos, actu_arti
 where articulos.articulo = actu_arti.articulo
 and articulos.proveedor = actu_arti.proveedor
 and actu_arti.marca = "*"
 display frame precios
 foreach cambio_precio begin
 input frame precios for update variables pr_vent
 end input
 end
 window from select articulos.articulo, articulos.proveedor, articulos.descripcion, articulos.pr_vent
 from articulos, actu_arti
 where actu_arti.marca = "*"
 and actu_arti.articulo = articulos.articulo
 and actu_arti.proveedor = articulos.proveedor
 10 lines label "Artículos actualizados"
 at 5,10
 end main

```

Para compilar este módulo es preciso ejecutar previamente el fichero «actu\_arti.sql» con el fin de crear la tabla que será considerada como temporal por el programa.

En primer lugar, se crea la tabla temporal «actu\_arti» y se carga con todos los artículos existentes en la tabla «articulos». A continuación, se presentan todos los artículos en un FORM con «scroll» de líneas para marcar aquellos de los que se desea actualizar su precio de venta («pr\_vent»). Este marcado se realiza automáticamente al pulsar cualquiera la tecla asignada a las acciones <freturn> o <faccept>. Los artículos marcados aparecerán con un asterisco al lado del nombre. Este proceso se lleva a cabo mediante un bucle en el que intervienen fundamentalmente las instrucciones «READ KEY THROUGH FORM» y EXIT EDITING de la sección EDITING

del FORM. Para finalizar el marcado habrá que pulsar la tecla asignada a la acción <fquit>. Posteriormente, se declara un CURSOR con los artículos marcados, presentándose éstos en un FRAME y permitiéndose únicamente la modificación de su precio de venta («pr\_vent»). Una vez aceptados los nuevos valores introducidos, éstos se actualizan en la tabla «articulos» de la base de datos. Por último, se muestra una ventana con los artículos marcados en el FORM y con la información actualizada en el FRAME.

### Consultas dinámicas con variables del FRAME

El FRAME permite la petición de datos de manera similar al FORM para la realización de un «Query By Forms», es decir, para consultar las filas de la base de datos. En el FRAME, esta petición de datos se realiza mediante la instrucción INPUT FRAME y la cláusula «FOR QUERY». En este caso, los valores de las variables del FRAME se inicializan a nulos («NULL»), pudiendo especificarse condiciones de selección en cada variable. El resultado de esta operación será un cadena de caracteres que representará las condiciones de una cláusula «WHERE» válida, pero sin la palabra reservada «WHERE», separando cada condición con los operadores lógicos «AND» y «OR» (esta última sólo en caso de emplear la condición «|» con varios valores).

Para validar las condiciones establecidas por el operador en la pantalla hay que pulsar la tecla asignada a la acción <faccept> o ejecutar la instrucción EXIT EDITING. La ejecución de esta instrucción en la operación de consulta activa la siguiente secuencia de instrucciones no procedurales de las secciones CONTROL y EDITING:

- 1.— «AFTER DISPLAY OF nombre\_frame (CONTROL)». Al ejecutar la instrucción «INPUT FRAME» con la cláusula FOR QUERY se inicializan las variables del FRAME. Indirectamente, la instrucción INPUT FRAME lleva implícita la operación de repintado de la sección SCREEN del FRAME.
- 2.— «BEFORE EDITQUERY OF nombre\_frame (EDITING)». Ejecución antes de entrar en edición de variables.
- 3.— «BEFORE EDITQUERY OF variable (EDITING)». Ejecución antes de activar la variable del FRAME.
- 4.— «AFTER EDITQUERY OF variable (EDITING)». Esta instrucción se ejecutará al abandonar la variable del FRAME, activándose la siguiente especificada en la sección VARIABLES.
- 5.— «AFTER EDITQUERY OF nombre\_frame (EDITING)». Esta instrucción se ejecutará na vez pulsada la tecla asignada a la acción <faccept> para aceptar las condiciones especificadas.
- 6.— «BEFORE QUERY OF nombre\_frame (CONTROL)». Equivalente a la instrucción pero en la sección CONTROL.
- 7.— «AFTER QUERY OF nombre\_frame (CONTROL)». Última instrucción antes de activar el menú, en caso de incluir la sección MENU en el FRAME.

Por su parte, al cancelar la edición se activan las siguientes instrucciones no procedurales de ambas secciones:

- 1.— «BEFORE CANCEL OF nombre\_frame (EDITING)». Se ejecuta después de pulsar la tecla asignada a la acción <fquit>, pudiendo incluso abortar la cancelación y proseguir con la edición hasta que se cumpla una condición.
- 2.— «AFTER CANCEL OF nombre\_frame (CONTROL)». Se ejecuta después de salir de edición.
- 3.— «AFTER DISPLAY OF nombre\_frame (CONTROL)». El control del programa vuelve al menú del FRAME, en caso de que exista, lo que conlleva el repintado.

Dependiendo de la tecla pulsada por el operador, la activación de instrucciones se consigue mediante las instrucciones no procedurales «ON KEY» y «ON ACTION» de las secciones CONTROL y EDITING.

En la sección EDITING, estas instrucciones se activarán, al intentar asignar condiciones y cuando el operador pulse la tecla correspondiente a la acción indicada en la instrucción (cuando el cursor del terminal se encuentre dentro de alguna variable de la SCREEN).

Por lo que respecta a la sección CONTROL, las instrucciones no procedurales se activarán cuando no estemos en edición, es decir, cuando podamos movernos sobre las distintas opciones del menú del FRAME o bien cuando se ejecute la instrucción «READ KEY THROUGH FRAME», si es que no existe un menú definido.

Las instrucciones no procedurales «ON KEY» y «ON ACTION» sólo podrán detectar aquellas teclas que no tengan asignada ya una acción en el propio objeto sobre el que se detectan. Por ejemplo, nunca se podrá detectar la tecla correspondiente a la acción <return>, ya que dicha tecla se emplea tanto en el menú del FRAME como en edición para seleccionar una opción del menú (sección CONTROL) y para aceptar un valor en una variable del FRAME, activando a su vez el siguiente campo (sección EDITING).

Las posibles condiciones que puede emplear el operador sobre cada uno de los campos son las siguientes:

|               |                                                                                                                                                                                                        |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| < valor       | Los valores de la columna donde se especifica deben ser menores que el valor especificado.                                                                                                             |
| > valor       | Los valores de la columna donde se especifica deben ser mayores que el valor especificado.                                                                                                             |
| <= valor      | Los valores de la columna donde se especifica deben ser menores o iguales que el valor especificado.                                                                                                   |
| >= valor      | Los valores de la columna donde se especifica deben ser mayores o iguales que el valor especificado.                                                                                                   |
| <> valor      | Los valores de la columna donde se especifica deben ser distintos al valor especificado.                                                                                                               |
| = valor       | Los valores de la columna donde se especifica deben ser iguales al valor especificado.                                                                                                                 |
| >>            | La lista en curso aparecerá ordenada de mayor a menor respecto a la columna donde se asignen estos símbolos.                                                                                           |
| <<            | La lista en curso aparecerá ordenada de menor a mayor respecto a la columna donde se asignen estos símbolos.                                                                                           |
| valor1:valor2 | Los valores de la columna donde se especifica deben estar en el rango indicado por «valor1» y «valor2».                                                                                                |
| *             | En columnas de tipo CHAR representa cualquier carácter.                                                                                                                                                |
| ?             | En columnas de tipo CHAR representa un carácter obligatorio.                                                                                                                                           |
| =NULL         | Los valores de la columna donde se especifica deben ser nulos.                                                                                                                                         |
| v1 v2 v3 ...  | Los valores de la columna donde se indiquen pueden ser los especificados en «v1», «v2», «v3», etc. En este caso, las condiciones que genera para cada valor están enlazadas mediante el operador «OR». |

**Ejemplo 10 [PFRAME10]. Consulta de clientes especificando condiciones sobre un FRAME:**

```
database almacen
define
 variable optimiza char(400)
 frame clientes
 screen
{
 Código: [a]
 Empresa: [b]
 Provincia: [c][d
}
}
```

```
end screen

variables
 a = cliente like clientes.cliente
 b = empresa like clientes.empresa
 c = provincia like clientes.provincia
 lookup d = descripcion
 joining provincias.provincia
end variables

control
 after query of clientes
 if optimiza is null then let optimiza = "cliente > 0"
 end control

editing
 before editquery of clientes.provincia
 message "Pulsar " &&
 keylabel("fuser1") &&
 " para consultar" reverse
 on action "fuser1"
 if curfield = "provincia" then
 window from select provincia, descripcion
 from provincias
 where provincia > 0
 10 lines label "P r o v i n c i a s"
 at 6,20
 set clientes.provincia from 1
 end editing
 end editing

layout
 box
 label "C o n d i c i o n e s"
 line 3
 column 10
 end layout
end frame
end define

global
 control
 on ending
 clear frame clientes
 end control
end global

main begin
 forever begin
 input frame clientes for query by name to optimiza
 end input
 if cancelled = true then break
 prepare inst1 from "select * from clientes where " &&
 optimiza clipped
 declare cursor consulta for inst1
 window scroll from cursor consulta
 10 lines label "C l i e n t e s ——>"
 at 5,1
 end
 end
end main
```

Este programa define un FRAME compuesto por tres variables sobre las que se podrán indicar condiciones para realizar la consulta sobre la tabla «clientes».

En primer lugar, se pedirán las condiciones mediante la instrucción `INPUT FRAME` con la cláusula «FOR QUERY». La cláusula «BY NAME» de esta instrucción indica que se establecerán condiciones con los mismos nombres que las variables del FRAME. El resultado de esta instrucción se recoge en la variable de programación «optimiza». En caso de que el operador no haya introducido ninguna condición se asignará una condición no restrictiva «cliente > 0». Esta asignación se realiza en la instrucción «AFTER QUERY OF frame». Asimismo, en la edición de las condiciones, al activar la variable «provincia» aparecerá un mensaje indicando que al pulsar cierta tecla se mostrará una ventana con los códigos de provincia válidos en la base de datos. Dichos códigos se extraen de la tabla «provincias».

Si el operador acepta las condiciones establecidas y no cancela la edición se construye una `SELECT` a base de constantes y variables `PREPARE` («optimiza»). Esta `SELECT` «preparada» se declara como un `CURSOR`. Por último, la tabla derivada de este `CURSOR` será la que se muestre en una ventana (`WINDOW`). Dicha tabla derivada será la que cumpla las condiciones establecidas en las variables del FRAME.

## Bloques de control activados con el repintado

El repintado de la pantalla en un FRAME se realiza específicamente con la instrucción `DISPLAY FRAME`. No obstante, la `INPUT FRAME` lleva implícita también esta operación. Por su parte, para borrar la pantalla del FRAME emplearemos la instrucción `CLEAR FRAME`.

Tanto la `DISPLAY FRAME` como la `CLEAR FRAME` se encuentran implícitas en la instrucción «MENU FRAME nombre\_frame». Al ejecutarse ésta se presentará la `SCREEN` junto con las opciones del menú del FRAME. Asimismo, dicha instrucción se encarga de borrar tanto la `SCREEN` como su menú cuando finaliza su ejecución.

En definitiva, cuando se ejecuta cualquiera de las instrucciones `DISPLAY FRAME`, `CLEAR FRAME`, `INPUT FRAME` y `MENU FRAME` se activa la instrucción no procedural «AFTER DISPLAY OF nombre\_frame» de la sección `CONTROL`.

## Diseño de la pantalla: Sección LAYOUT

Al igual que ocurría en el `FORM`, la sección `LAYOUT` del FRAME permite cambiar el aspecto de presentación de la pantalla. Hasta ahora, algunas de las opciones contempladas en esta sección se han simulado por otros medios. La sintaxis de la sección `LAYOUT` es la siguiente:

```
[LAYOUT
 [BOX]
 [[NO] UNDERLINE]
 [REVERSE]
 [LINES líneas]
 [COLUMNS columnas]
 [LINE línea]
 [COLUMN columna]
 [LABEL {"literal" | número}]
 [HELP número_ayuda]
END LAYOUT]
```

Las opciones especificadas en la sintaxis anterior son todas las que se pueden incluir dentro de esta sección opcional del FRAME. Como hemos podido comprobar en los ejemplos expuestos hasta ahora en este capítulo, todos ellos presentaban una serie de características comunes:

- La presentación de la pantalla, salvo en el caso de emplear metacaracteres, no incluía ninguna caja enmarcando a las variables del FRAME. Esta característica se consigue mediante la cláusula «BOX», la cual calcula las dimensiones necesarias para no pintar encima de algún literal o identificador definido en la sección `SCREEN`. En caso de querer variar las dimensiones calculadas por esta opción habría que emplear las cláusulas «LINES» y «COLUMNS» de la sección `LAYOUT`.

- La pantalla del FRAME (sección SCREEN) se muestra por defecto en las coordenadas 1,1 de la pantalla (línea 1, columna 1). Esta característica se puede modificar mediante las cláusulas «LINE línea» y «COLUMN columna».
- Por defecto, la petición de los valores para cada variable del FRAME aparece subrayada. Esta característica puede cambiarse con las opciones: «NO UNDERLINE» (aparecerá subrayado sólo el campo activo), «REVERSE» (todos los campos aparecerán en vídeo inverso) y «UNDERLINE» (que es el atributo empleado por defecto, es decir, todos los campos aparecerán subrayados).

La etiqueta que se mostrará en la parte superior de la caja estándar diseñada por la cláusula «BOX» de esta sección puede extraerse de un fichero de ayuda (la forma de emplear los ficheros de ayuda se explica en el capítulo 16 de este mismo manual).

### Ejemplo 11 [PFRAME11]. Empleo de algunas de las cláusulas de la sección LAYOUT:

```
database almacen

define
 frame estandar like proveedores.*
 end frame

 frame presenta like proveedores.*

 control
 after display of presenta
 pause "Frame con LAYOUT. Pulse [Return] para continuar" reverse bell
 end control

 layout
 box
 label "P r e s e n t a c i ó n"
 no underline
 line 5
 column 10
 lines 12
 columns 60
 end layout
end frame
end define

main begin
 display frame estandar
 pause "Frame presentado sin LAYOUT. Pulse [Return] para continuar" reverse bell
 clear frame estandar
 input frame presenta
 end input
end main
```

Este programa define dos FRAMES con la misma estructura de variables. En uno de ellos se han definido las secciones CONTROL y LAYOUT. En primer lugar, el programa muestra el primer FRAME. Éste se ha definido como una tabla de la base de datos, por lo que asume todas las características por defecto de un FRAME. Este FRAME se muestra en la línea 1 columna 1 de la pantalla. Además, las variables y sus respectivas etiquetas no se encuadran en una caja.

A continuación, se formula una petición de valores sobre el segundo FRAME. Éste tiene definida la sección LAYOUT con la mayoría de sus opciones. La cláusula «BOX» indica que todas las variables y sus etiquetas se enmarcarán en una caja estándar de CTL. Asimismo, la cláusula «LABEL» presenta un literal en la cabecera de dicha caja. Por su parte, la cláusula «NO UNDERLINE» utiliza el atributo subrayado el campo activo. Las cláusulas «LINE 5» y «COLUMN 10» fijan las coordenadas del comienzo de la caja pintada por «BOX». Por último, las cláusulas «LINES 12» y «COLUMNS 60» fijan la longitud en líneas y columnas de la caja representada por «BOX».



## Variables del FRAME en instrucciones SQL

Como se ha indicado anteriormente, un objeto FRAME está compuesto por una serie de variables de programación que forman una estructura. Pues bien, estas variables pueden ser tanto receptoras como variables «hosts» dentro de la sintaxis de las instrucciones de SQL.

Una variable «host» es una variable (incluso SHARED o NEW SHARED), parámetro, variable de un FORM («VARIABLE», «DISPLAYTABLE», «PSEUDO», etc.), variable de un FRAME, etc., definida en un módulo que interviene en la sintaxis de una instrucción de SQL embebida en un módulo CTL. Cuando intervienen en una instrucción SQL, estas variables se identifican por ir precedidas por el signo dólar («\$»).

Las variables «host» están asociadas a ciertas cláusulas de algunas de las instrucciones propias del SQL. Estas cláusulas son fundamentalmente «WHERE» y «HAVING». Las variables de un objeto FRAME se integran de la forma que se referencian, es decir:

```
variable
nombre_frame.variable
```

### Ejemplo 12 [PFRAME12]. Inserción de filas nuevas sobre la tabla «unidades»:

```
database almacen

define
 frame uni
 screen
{
 Unidad: [a]

 Descripción: [b]
}

end screen

variables
 a = unidad like unidades.unidad
 b = descripcion like unidades.descripcion
end variables

control
 after add of uni
 insert into unidades (unidad, descripcion)
 values ($uni.unidad, $uni.descripcion)
 end control

editing
 after editadd of uni.unidad
 select "" from unidades
 where unidades.unidad = $uni.unidad
 if found = true then begin
 pause "Unidad existente. Pulse [Return] para continuar" reverse bell
 let uni.unidad = ""
 next field "unidad"
 end
 end editing

layout
 box
 label "U n i d a d e s"
 end layout
end frame
```

```
end define

main begin
 forever begin
 input frame uni for add
 end input
 if cancelled = true then break
 end
 window from select unidad, descripcion from unidades
 10 lines label "U n i d a d e s"
 at 5,10
end main
```

Este programa inserta un número indeterminado de filas sobre la tabla «unidades». La petición de los valores a insertar se realiza mediante un objeto FRAME. Cuando se introduce el código de la «unidad» se comprueba de que ésta no se encuentre ya en la tabla. En este control, la variable «uni.unidad» realiza la función de variable «host». Asimismo, al grabar los valores sobre la tabla mediante la instrucción INSERT, tanto la variable «unidad» como «descripcion» se comportan como variables «hosts». Cuando el operador cancela la edición de valores se corta el bucle «FOREVER», mostrándose una ventana con todas las filas existentes en la tabla «unidades».

Una variable receptora es aquella variable de programa (incluso «SHARED» y «NEW SHARED»), parámetro, variable de un FORM («VARIABLE», «DISPLAYTABLE», «PSEUDO», etc) o variable de un FRAME que recibe un valor devuelto por una instrucción SELECT. Para recoger valores de una SELECT existen varias cláusulas donde se pueden emplear las variables receptoras. Todas estas cláusulas se incluyen antes de la cláusula «FROM» de la SELECT.

Las cláusulas en las que intervienen las variables de un FRAME son las siguientes:

a) «INTO variables». En ella hay que especificar los nombres de aquellas variables del FRAME, separadas por comas, que recogerán los valores de las columnas o expresiones leídas en la instrucción SELECT. Hay que tener en cuenta que una variable de un FRAME se referencia por su propio nombre, o bien anteponiéndole el nombre del FRAME seguido de un punto. Por ejemplo:

```
variable
frame.variable

INTO frame.variable1, frame.variable2, ..., frame.variabilen
```

Esta misma cláusula se puede incluir en la sintaxis de las instrucciones de manejo de «cursores» FETCH y FOREACH. En el caso de tratar instrucciones SELECT dinámicas (PREPARE), la única forma de recoger los valores en la cláusula «INTO» será mediante la declaración y el manejo de un CURSOR.

**Ejemplo 13 [PFRAME13]. Recepción de valores devueltos por una instrucción SELECT sobre las variables de un FRAME:**

```
database almacen

define
 variable optimiza char(500)

 frame clientes like clientes.*
 control
 after query of clientes
 if optimiza is null then let optimiza = "cliente > 0"
 end control

 editing
 before editquery of clientes
 message "Introducir condiciones. Pulsar " && keylabel("fhelp") &&
 " para ayuda" reverse
```

```

end editing

layout
 box
 label "C l i e n t e s"
 lines 13
 columns 60
end layout
end frame
end define

main begin
 forever begin
 input frame clientes for query by name to optimiza
 end input
 if cancelled = true then begin
 clear frame clientes
 break
 end
 clear frame clientes
 prepare inst1 from "select cliente, empresa, direccion1, " &&
 "telefono from clientes where " &&
 optimiza clipped
 declare cursor consulta for inst1
 display frame clientes at 5,10
 foreach consulta into clientes. cliente, clientes.empresa, clientes.direccion1,
 clientes.telefono begin
 view
 call sleep(1)
 end
 clear frame clientes
 end
end
end main

```

Este programa define un FRAME con la misma estructura que la tabla «clientes». Sobre este FRAME se pedirán las condiciones que definirán las consultas que el operador desea realizar. Esta operación se consigue mediante la instrucción INPUT FRAME con la cláusula «FOR QUERY». Con la información devuelta por esta instrucción se prepara una instrucción SELECT, seleccionando parte de las columnas de la tabla «clientes». Posteriormente, con la instrucción PREPARE se declara un CURSOR para el tratamiento individual de cada una de las filas que cumplen las condiciones establecidas por el operador. La recepción de valores de la SELECT «preparada» se lleva a cabo en la instrucción FOREACH con la cláusula «INTO». En este caso, las variables receptoras son las propias del FRAME, el cual, al estar repintado, refresca la información leída en cada vuelta del bucle FOREACH gracias al empleo de la VIEW. Este proceso se repite un número indeterminado de veces hasta que el operador pulse la tecla asignada a la acción <quit> en la petición de condiciones para la consulta.

b) «BY NAME ON nombre\_frame». En esta sintaxis se cargarán los valores de las columnas, expresiones o alias seleccionados en la «selección» en aquellas variables del FRAME que se llamen igual a ellas.

También existe la posibilidad de emplear todas las variables de un FRAME como receptoras, independientemente del nombre con el que se hubiesen definido. La sintaxis para esta recepción de valores es la siguiente:

```
.INTO frame.*
```

**Ejemplo 14 [PFRAME14]. Presentación de todos los proveedores existentes en la base de datos sobre una estructura FRAME:**

```

database almacen

define
 frame prov like proveedores.*
 layout

```

```
 box
 label "P r o v e e d o r e s"
 lines 13
 columns 60
 end layout
end frame
end define

main begin
 declare cursor provee for select * by name on prov from proveedores
 display frame prov at 5,10
 foreach provee begin
 view
 call sleep(1)
 end
 clear frame prov
end main
```

Este programa define un FRAME con la misma estructura que la tabla «proveedores». Este FRAME se utiliza para mostrar en pantalla la información devuelta por el CURSOR declarado en la sección MAIN. Este CURSOR leerá todos los proveedores existentes en la base de datos. En la declaración del CURSOR se emplea la cláusula «BY NAME ON frame» para la recepción de los valores devueltos por la instrucción SELECT sobre las variables del FRAME. El FRAME se repinta mediante la instrucción DISPLAY FRAME. A continuación, mediante la instrucción FOREACH se leen todas las filas implicadas en el CURSOR, mientras que la VIEW se encargará del refresco de la pantalla del FRAME. Entre fila y fila leída por el CURSOR se provoca una espera de aproximadamente un segundo («call sleep(1)»). Al finalizar la lectura del CURSOR el FRAME se elimina de la pantalla.

En el ejemplo anterior se podría haber empleado también la siguiente cláusula para la recepción de valores sobre las variables del FRAME:

```
 declare cursor provee for select * into prov.*
 from proveedores
```

### NOTAS:

- 1.— En muchas ocasiones, es conveniente crear las variables que se definen en programación con el mismo nombre que las columnas de la tabla a tratar.
- 2.— Una instrucción que devuelve más de una fila como tabla derivada debería tratarse como un CURSOR para la recogida de los valores de cada una de sus filas y columnas.
- 3.— Un elemento de un ARRAY no puede ser variable receptora.
- 4.— Para más información acerca de variables «hosts» y receptoras [consulte el capítulo 9](#) de este mismo volumen.

## Instrucciones relacionadas

CTL dispone de diversas instrucciones para manejar un FRAME. Dichas instrucciones son las siguientes:

|                      |                                                                  |
|----------------------|------------------------------------------------------------------|
| <b>CANCEL</b>        | Interrumpe la edición de las variables de un FRAME.              |
| <b>CLEAR FRAME</b>   | Borra un FRAME que se encuentre activo en pantalla.              |
| <b>DISPLAY FRAME</b> | Muestra un FRAME en pantalla.                                    |
| <b>EXIT EDITING</b>  | Fuerza el fin de la edición de las variables de un FRAME.        |
| <b>INPUT FRAME</b>   | Activa la edición de las variables del FRAME.                    |
| <b>MENU FRAME</b>    | Activa un menú definido en un FRAME, cediendo el control a éste. |

**NEXT FIELD** En edición, altera la secuencia de entrada de los campos.

**READ KEY THROUGH FRAME** Petición de una tecla y ejecución de su acción sobre el FRAME.

**RETRY** Reinicia la edición en un FRAME.

En los siguientes apartados se comentan las principales características de cada una de estas instrucciones. No obstante, para una información más detallada consulte el Manual de Referencia de MultiBase.

## Instrucción CANCEL

Esta instrucción interrumpe cualquiera de las instrucciones que se emplean para realizar la edición sobre las variables del FRAME. La instrucción CANCEL sólo puede situarse en las secciones CONTROL y EDITING del FRAME. Su sintaxis es la siguiente:

### CANCEL

La ejecución de esta instrucción simula la pulsación de la tecla asignada a la acción <fquit>.

**Ejemplo 15 [PFRAME15]. Consulta de proveedores mediante la petición de rangos sobre un FRAME:**

```
database almacén

define
 frame rangos
 screen
 {
 Desde Proveedor.....:[a]
 Hasta Proveedor.....:[b]
 }
end screen

variables
 a = desde like proveedores.proveedor
 b = hasta like proveedores.proveedor
end variables

editing
 after editadd of rangos.hasta
 if rangos.hasta < rangos.desde then cancel
end editing

layout
 box
 label "R a n g o s"
end layout
end frame
end define

main begin
 forever begin
 input frame rangos for add
 end input
 if cancelled = true then break
 window scroll from select proveedor, empresa, direccion1
 from proveedores
 where proveedores.proveedor between $rangos.desde and $rangos.hasta
 10 lines label "P r o v e e d o r e s ——>"
 at 5,10
 end
end main
```

Este programa define un FRAME con dos variables («desde» y «hasta») que servirán como rangos para la consulta de los proveedores. En caso de introducir un valor en la variable «hasta» que sea inferior al especificado en «desde» la edición se cancelará, activándose la variable interna «cancelled» y finalizando el programa. Dicha finalización también puede provocarse mediante la pulsación de la tecla asignada a la acción <fquit>. Si el rango introducido es válido aparecerá una ventana con los proveedores incluidos en dicho rango.

### Instrucción CLEAR FRAME

Esta instrucción borra de la pantalla el aspecto de un FRAME, es decir, elimina la SCREEN representada previamente mediante la instrucción MENU FRAME, INPUT FRAME o DISPLAY FRAME. Su sintaxis es la siguiente:

**CLEAR FRAME** nombre\_frame [AT línea, columna]

Esta instrucción se encuentra implícita en la MENU FRAME. Cuando el menú empleado para el mantenimiento del FRAME se cancela (acción <fquit>), se borra de la pantalla el menú junto con el FRAME.

#### Ejemplo 16 [PFRAME16]. Consulta de proveedores mediante la operación QUERY de un FRAME:

```
database almacén

define
 variable optimiza char(500)

 frame provee like proveedores.*

 control
 after query of provee
 if optimiza is null then let optimiza = "proveedor > 0"
 end control

 layout
 box
 label "Proveedores"
 column 10
 lines 12
 columns 60
 end layout
 end frame
end define

main begin
 forever begin
 input frame provee for query by name to optimiza
 end input
 if cancelled = true then begin
 clear frame provee
 break
 end
 prepare inst1 from "select * from proveedores where " &&
 optimiza clipped
 declare cursor consulta for inst1
 window scroll from cursor consulta
 10 lines label "P r o v e e d o r e s ———>"
 at 5,1
 end
 end
end main
```

Este programa realizará constantemente consultas sobre la tabla «proveedores». Las condiciones a emplear para dicha consulta se recogen en un FRAME mediante la operación QUERY. En caso de no cancelar la edición (acción <fquit>) se prepara una instrucción SELECT con el resultado devuelto por el FRAME, ya que las condiciones podrán ser diferentes (dinámicas) en cada vuelta del bucle FOREVER. Por último, aparecerá una ventana

(WINDOW) con las filas que cumplan las condiciones introducidas por el operador. En el momento que se cancele la edición de condiciones se limpiará el FRAME de la pantalla mostrado en un principio por la instrucción INPUT FRAME.

## Instrucción DISPLAY FRAME

Esta instrucción muestra el aspecto de la sección SCREEN de un FRAME. Esta instrucción se encuentra implícita en la MENU FRAME e INPUT FRAME. Su sintaxis es la siguiente:

**DISPLAY FRAME** nombre\_frame [AT línea, columna]

La cláusula «AT» puede simularse mediante las cláusulas «LINES» y «COLUMNS» de la sección LAYOUT.

**Ejemplo 17 [PFRAME17]. Listado de la tabla «provincias» mediante un FRAME:**

```
database almacen

define
 variable linea smallint default 3

 frame provin
 screen
 {
 Código: [a]
 Descripción: [b]
 Prefijo: [c]
 }
 end screen

 variables
 a = provincia like provincias.provincia
 b = descripcion like provincias.descripcion
 c = prefijo like provincias.prefijo
 end variables

 layout
 box
 label "P r o v i n c i a s"
 end layout
 end frame
end define

main begin
 declare cursor consulta for select * by name on provin
 from provincias
 order by descripcion
 display frame provin at 5, 3
 foreach consulta begin
 pause "Pulse [" && keylabel ("fup") &&
 "]" arriba y [" && keylabel ("fdown")
 && "]" abajo" reverse
 switch actionlabel(lastkey)
 case "fup"
 fetch previous consulta
 fetch previous consulta
 case "fquit"
 break
 end
 end
 clear frame provin
end main
```

Este programa declara un CURSOR con todas las filas de la tabla «provincias». La recepción de los valores de dicha tabla derivada se realiza sobre las variables del FRAME definido previamente. Por cada fila que se presente en el FRAME se pide una tecla. Dependiendo de la tecla pulsada podrá leerse la fila anterior, la siguiente, o cancelar la lectura. Por último, el FRAME se borra de la pantalla mediante la instrucción CLEAR FRAME.

### Instrucción EXIT EDITING

Esta instrucción fuerza el fin de la edición de las variables del FRAME. Esta finalización conlleva la aceptación (grabación) de la operación que se esté realizando en esos momentos (ADD, UPDATE o QUERY). Su sintaxis es la siguiente:

#### EXIT EDITING

Esta instrucción sólo podrá situarse en cualquiera de las instrucciones no procedurales de la sección EDITING.

**Ejemplo 18 [PFRAME18]. Consulta de proveedores introduciendo el rango sobre un FRAME. Para aceptar los valores especificados en el rango no será necesario pulsar la tecla asignada a la acción <faccept>:**

```
database almacén

define
 frame rangos
 screen
{
 Desde Proveedor.....: [a][a1]

 Hasta Proveedor.....: [b][b1]
}
end screen

variables
 a = desde like proveedores.proveedor
 check ($ is not null)
 lookup a1 = empresa
 joining proveedores.proveedor
 b = hasta like proveedores.proveedor
 check ($ is not null and $ >= desde)
 lookup b1 = empresa
 joining proveedores.proveedor
end variables

editing
 after editupdate of rangos.hasta
 exit editing
end editing

layout
 box
 label "R a n g o s"
 end layout
end frame
end define

main begin
 forever begin
 input frame rangos for update
 end input
 if cancelled = true then begin
 clear frame rangos
 break
 end
 end
end
```



```

 window scroll from select proveedor, empresa, direccion1
 from proveedores
 where proveedores.proveedor between $rangos.desde and $rangos.hasta
 10 lines label "P r o v e e d o r e s ——>"
 at 5,10
 end
end main

```

Este programa se encarga de pedir valores sobre las variables del FRAME. Esta petición se encuentra dentro de un bucle (FOREVER), por lo que, al emplear la cláusula «FOR UPDATE» de la instrucción INPUT FRAME se podrán modificar los valores introducidos previamente. En el momento que se acepte el valor sobre la variable «hasta» del FRAME, automáticamente se aceptarán dichos valores al ejecutarse la instrucción EXIT EDITING. A continuación, aparecerá una ventana con aquellos proveedores que se encuentren en el rango especificado en las variables del FRAME. El programa finalizará en el momento que el operador pulse la tecla asignada a la acción <fquit>.

## Instrucción INPUT FRAME

Esta instrucción se encarga de mostrar el FRAME activo en pantalla (SCREEN si existe) y pedir valores sobre las variables de dicho FRAME. La petición de valores puede realizarse en tres operaciones: «FOR ADD», «FOR UPDATE» y «FOR QUERY». La sintaxis de esta instrucción en cada una de estas operaciones es la siguiente:

```

INPUT FRAME nombre_frame [FOR {ADD | UPDATE}]
 [VARIABLES lista_variables] [AT línea, columna]
END INPUT

```

En la operación ADD se inicializarán todas las variables del FRAME con los valores por defecto. Dichos valores por defecto serán «NULL» si no se especifica el atributo DEFAULT del CTL. Sin embargo, la cláusula «FOR UPDATE» no inicializa los valores de las variables.

La cláusula «AT» puede simularse mediante las cláusulas «LINE línea» y «COLUMN columna» de la sección LAYOUT del FRAME.

```

INPUT FRAME nombre_frame FOR QUERY
 [VARIABLES lista_variables]
 [{BY NAME | ON lista_columnas} TO variable}
 [AT línea, columna]
END INPUT

```

La cláusula «FOR QUERY» permite introducir condiciones sobre las variables del FRAME, devolviendo sobre «variable» la composición de las condiciones introducidas por el operador.

Las posibles condiciones que puede emplear el operador sobre cada uno de los campos son las siguientes:

|          |                                                                                                      |
|----------|------------------------------------------------------------------------------------------------------|
| < valor  | Los valores de la columna donde se especifica deben ser menores que el valor especificado.           |
| > valor  | Los valores de la columna donde se especifica deben ser mayores que el valor especificado.           |
| <= valor | Los valores de la columna donde se especifica deben ser menores o iguales que el valor especificado. |
| >= valor | Los valores de la columna donde se especifica deben ser mayores o iguales que el valor especificado. |
| <> valor | Los valores de la columna donde se especifica deben ser distintos al valor especificado.             |
| = valor  | Los valores de la columna donde se especifica deben ser iguales al valor especificado.               |

|               |                                                                                                              |
|---------------|--------------------------------------------------------------------------------------------------------------|
| >>            | La lista en curso aparecerá ordenada de mayor a menor respecto a la columna donde se asignen estos símbolos. |
| <<            | La lista en curso aparecerá ordenada de menor a mayor respecto a la columna donde se asignen estos símbolos. |
| valor1:valor2 | Los valores de la columna donde se especifica deben estar en el rango indicado por «valor1» y «valor2».      |
| *             | En columnas de tipo CHAR representa cualquier carácter.                                                      |
| ?             | En columnas de tipo CHAR representa un carácter obligatorio.                                                 |
| =NULL         | Los valores de la columna donde se especifica deben ser nulos.                                               |
| v1 v2 v3 ...  | Los valores de la columna donde se indiquen pueden ser los especificados en «v1», «v2», «v3», etc.           |

**Ejemplo 19 [PFRAME19]. Demostración de las tres posibles operaciones a realizar con un FRAME (simulación de las operaciones de un mantenimiento de una tabla sobre un FORM):**

```
database almacen

define
 variable sw smallint default 0
 variable optimiza char(200)

 frame provin
 scren
{
 Código Provincia: [a]

 Descripción: [b]

 Prefijo: [c]
}
end screen

variables
 a = provincia like provincias.provincia nouupdate
 b = descripcion like provincias.descripcion
 c = prefijo like provincias.prefijo
end variables

menu m1 "Mantenimiento"
 option "Añadir"
 forever begin
 input frame provin for add
 end input
 if cancelled = true then break
 end
 option "Modificar"
 if provin.provincia is null then begin
 message "Debe consultar previamente" reverse bell
 next option a3
 end
 input frame provin for update
 end input
 option "Borrar"
 if provin.provincia is null then begin
 message "Debe consultar previamente" reverse bell
 next option a3
 end
end
```

```

 if yes("Conforme", "N") = true then
 delete from provincias
 where current of consulta
 option a3 "Consultar"
 input frame provin for query by name to optimiza
 end input
 if cancelled = false then begin
 if sw = 1 then begin
 close consulta
 let sw = 0
 end
 prepare inst1 from "select * from provincias where " && optimiza clipped &&
 " for update"
 declare cursor consulta for inst1
 open consulta
 fetch consulta into provin.provincia, provin.descripcion, provin.prefijo
 let sw = 1
 end
 option "Salir"
 exit menu
end menu

control
 after query of provin
 if optimiza is null then let optimiza = "provincia > 0"
 after update of provin
 update provincias set
 descripcion = $provin.descripcion, prefijo = $provin.prefijo
 where current of consulta
 after add of provin
 insert into provincias (provincia, descripcion, prefijo)
 values ($provin.*)
 on action "fdown"
 fetch consulta into provin.provincia, provin.descripcion, provin.prefijo
 on action "fup"
 fetch previous consulta into provin.provincia, provin.descripcion, provin.prefijo
 on action "ftop"
 fetch first consulta into provin.provincia, provin.descripcion, provin.prefijo
 on action "fbottom"
 fetch last consulta into provin.provincia, provin.descripcion, provin.prefijo
end control

layout
 box
 label "P r o v i n c i a s"
 end layout
end frame
end define

main begin
 menu frame provin
end main

```

Este programa simula el mantenimiento de una tabla («provincias») con un FRAME. En este ejemplo se han incluido las tres operaciones válidas en el manejo de este objeto («FOR ADD», «FOR UPDATE» y «FOR QUERY»). En primer lugar, el programa ejecuta el menú del FRAME, mostrándolo en pantalla. En este menú existe una opción «Añadir» que emplea la instrucción INPUT FRAME con la cláusula «FOR ADD». Después de aceptar los nuevos valores sobre las variables del FRAME se inserta una fila sobre la tabla «provincias» mediante la instrucción INSERT. En la opción «Modificar» se emplea la cláusula «FOR UPDATE», que implica la actualización de la fila en curso presentada sobre las variables del FRAME. Previamente a su ejecución debería haberse consultado alguna fila de la tabla «provincias», es decir, haber ejecutado la opción «Consultar».

Por último, en la opción «Consultar» se emplea la cláusula «FOR QUERY». Esta opción declara un CURSOR de una SELECT preparada de forma dinámica con la instrucción PREPARE. Este CURSOR se leerá de forma bidireccional dependiendo de las teclas tilizadas por el operador («up», «down», «top» y «bottom»). Por lo eneral, estas teclas se corresponderán con las cuatro flechas del teclado.

### Instrucción MENU FRAME

Esta instrucción ejecuta el menú definido en la sección MENU del FRAME. Su sintaxis es la siguiente:

**MENU FRAME** nombre\_frame [**DISABLE** lista\_opciones] [**AT** línea, columna]

Al ejecutar esta instrucción se muestra automáticamente en pantalla tanto el aspecto de la sección SCREEN, si está definida, como el menú definido en la sección MENU para dicho FRAME.

La instrucción MENU FRAME se puede simular mediante la ejecución de las siguientes:

```
...
...
main begin
 display frame nombre_frame
 menu nombre_menu
 clear frame nombre_frame
end main
```

**Ejemplo 20 [PFRAME20]. Menú compuesto por las opciones «Modificar» y «Agregar» valores sobre las variables del FRAME:**

```
database almacen

define
 frame peticion
 screen
{
 Desde Cliente.....: [a]

 Hasta Cliente.....: [b]

}

end screen

variables
 a = desde like proveedores.proveedor
 check ($ is not null)
 b = hasta like proveedores.proveedor
 check ($ is not null and $ >= desde)
end variables

menu m1 "Peticion" down skip 2 line 12 column 40
 option "Añadir"
 input frame peticion for add
 end input
 option "Modificar"
 input frame peticion for update
 end input
 option "Salir"
 exit menu
end menu

control
 after add update of peticion
 window scroll from select * from clientes
 where cliente between $peticion.desde and $peticion.hasta
```

```

 order by empresa
 10 lines label "C l i e n t e s ——>"
 at 5,1
 end control

 layout
 box
 label "P e t i c i ó n"
 end layout
 end frame
end define

main begin
 menu frame petition
end main

```

La instrucción «MENU FRAME petition» invoca al menú definido en la sección MENU del FRAME «petition». Este menú está compuesto por tres opciones: «Añadir», «Modificar» y «Salir». La primera de ellas permite agregar valores nuevos sobre las variables del FRAME, mientras que la segunda permite modificar los valores ya existentes. En ambas opciones, después de aceptar los valores asignados se comprueba que el rango «hasta» sea superior a «desde», mostrando a continuación una ventana con todos los clientes que se encuentren en el rango especificado. La última opción del menú, «Salir», ejecuta la instrucción EXIT MENU, provocando la finalización del programa.

## Instrucción NEXT FIELD

Esta instrucción altera el orden de edición de las variables del FRAME definido en la sección VARIABLES. Su sintaxis es la siguiente:

**NEXT FIELD [UP | DOWN] [variable\_frame]**

Esta instrucción sólo puede emplearse en cualquiera de las instrucciones de la sección EDITING.

**Ejemplo 21 [PFRAME21]. Consulta de filas de la base de datos dependiendo de la petición de rangos:**

```

database almacen

define
 frame petition
 screen
 {
 Desde Cliente.....: [a]

 Hasta Cliente.....: [b]
 }
end screen

variables
 a = desde like clientes.cliente
 b = hasta like clientes.cliente
end variables

editing
 after editadd of petition.hasta
 if petition.hasta < petition.desde then begin
 message "Rangos no válidos" reverse bell
 let petition.desde = null
 let petition.hasta = null
 next field "desde"
 end
end

```

```
end editing

layout
 box
 label "P e t i c i ó n"
 end layout
end frame
end define

main begin
 forever begin
 input frame peticion for add
 end input
 if cancelled = true then break
 window scroll from select * from clientes
 where cliente between $peticion.desde and $peticion.hasta
 order by empresa
 10 lines label "C l i e n t e s ——>"
 at 5,1
 end
 end
end main
```

Este programa es similar al expuesto en el ejemplo 20. No obstante, para emplear la instrucción NEXT FIELD se ha eliminado el atributo CHECK de la definición de las variables. En la sección EDITING se comprueba el rango. En caso de no ser válido, por ejemplo si «hasta» es inferior a «desde», se asigna el valor «NULL» a ambas variables y se vuelve a pedir el valor «desde». Una vez aceptados los valores válidos aparecerá una ventana con aquellos clientes cuyo códigos se encuentren en el rango especificado.

## Instrucción READ KEY THROUGH FRAME

Esta instrucción solicita al operador la pulsación de una tecla. Si dicha tecla estuviese referenciada en las instrucciones no procedurales «ON KEY» y «ON ACTION» de la sección CONTROL del FRAME se ejecutarán las instrucciones correspondientes. Su sintaxis es la siguiente:

**READ KEY THROUGH FRAME** nombre\_frame

Esta instrucción actualiza la variable interna de CTL «lastkey».

**Ejemplo 22 [PFRAME22]. Muestra de las filas de la tabla derivada de un CURSOR tratado de forma bidireccional en función de la tecla utilizada:**

```
database almacen

define
 frame teclas
 screen
 {
 Arriba: [a]
 Abajo: [b]
 Principio: [c]
 Fin: [d]
 }
end screen

variables
 a = arriba char
 b = abajo char
 c = principio char
 d = fin char
end variables

layout
```

```

 box
 label "T e c l a s"
 end layout
end frame

frame clien like clientes.*
control
 on action "ftop"
 fetch first lectura
 view
 on action "fbottom"
 fetch last lectura
 view
 on action "fup"
 fetch previous lectura
 view
 on action "fdown"
 fetch lectura
 view
 end control
end frame
end define

global
control
 on beginning
 let teclas.arriba = keylabel("fup")
 let teclas.abajo = keylabel("fdown")
 let teclas.principio = keylabel("ftop")
 let teclas.fin = keylabel("fbottom")
 end control
end global

main begin
 declare cursor lectura for select * by name on clien
 from clientes
 display frame clien at 1,1
 display frame teclas at 12,50
 open lectura
 fetch lectura
 if found = true then begin
 view
 forever begin
 read key through frame clien
 if actionlabel(lastkey) = "fquit" then break
 end
 end
 close lectura
 clear frame clien
 clear frame teclas
end main

```

Este programa define dos FRAMES que se emplearán para mostrar información por pantalla. Al FRAME «teclas» se le asignan al principio del programa («ON BEGINNING») las teclas que el operador puede emplear para manejar la tabla derivada del CURSOR presentada en el FRAME «clien». Este FRAME, que contendrá la información de la fila en curso del CURSOR, se muestra una sola vez, refrescando la información (instrucción VIEW) según se lee el CURSOR «FETCH ....».

Al comenzar la lectura del CURSOR se ejecuta la instrucción «READ KEY THROUGH FRAME clien» para que el operador pulse una de las teclas indicadas el FRAME «teclas». Dependiendo de la tecla pulsada se activará o no una instrucción no procedural «ON ACTION» de la sección CONTROL. Estas instrucciones leen del CURSOR de

forma bidireccional. Por último, en caso de que el operador pulse la tecla asignada a la acción <fquit> finalizará el programa, borrando de la pantalla ambos FRAMES.

### Instrucción RETRY

Esta instrucción fuerza el reinicio de edición de las variables del FRAME. Su sintaxis es la siguiente:

#### RETRY

Esta instrucción sólo puede especificarse en cualquiera de las instrucciones no procedurales de la sección EDITING.

#### Ejemplo 23 [PFRAME23]. Consulta de clientes con la petición de un rango sobre un FRAME:

```
database almacen

define
 frame peticion
 screen
 {
 Desde Cliente.....: [a]

 Hasta Cliente.....: [b]
 }
end screen

variables
 a = desde like clientes.cliente
 b = hasta like clientes.cliente
end variables

editing
 after editadd of peticion.hasta
 if peticion.hasta < peticion.desde then begin
 message "Rangos no válidos" reverse bell
 retry
 end
 else exit editing
end editing

layout
 box
 label "P e t i c i ó n"
end layout
end frame
end define

main begin
 forever begin
 input frame peticion for add
 end input
 if cancelled = true then break
 window scroll from select * from clientes
 where cliente between $peticion.desde and $peticion.hasta
 order by empresa
 10 lines label "C l i e n t e s ———>"
 at 5,1
 end
 clear frame peticion
 end
end main
```



Este programa es similar al expuesto en el ejemplo 21. En este caso, si el rango de valores para la consulta de «clientes» no es válido se reinicia la edición sin tener que asignarles valores nulos a cada variable como sucedía en aquel ejemplo.

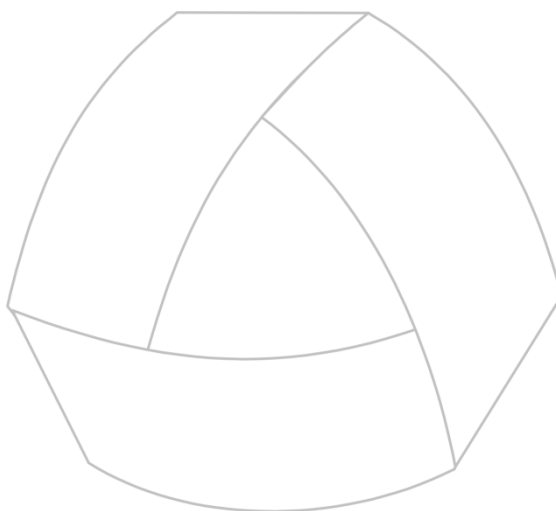
### Funciones y variables internas CTL

El FRAME tiene asociadas unas variables internas de CTL que adquieren un valor de forma automática por el hecho de realizar una determinada operación. Las variables internas empleadas por el FRAME son las siguientes:

|             |                                                                                                                                                                                  |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| «curfield»  | Nombre del campo en edición del FRAME.                                                                                                                                           |
| «newvalue»  | Variable interna que devolverá «TRUE» o «FALSE» dependiendo de si se ha modificado o no el contenido del último campo durante la edición.                                        |
| «operation» | Nombre de la operación que se está realizando en el FRAME. Los valores que puede adquirir esta variable son : ADD, UPDATE y QUERY.                                               |
| «cancelled» | Variable interna que devolverá «TRUE» o «FALSE» dependiendo de que el operador pulse la tecla asignada a la acción <fquit> en edición o de que se ejecute la instrucción CANCEL. |

NOTA: Todas estas variables contendrán el último valor asignado en cada caso. Por ejemplo, en la operación («operation») de añadir (ADD) sobre un FORM, si se invoca a un FRAME con una operación de modificar («FOR UPDATE»), al regresar al FORM la operación será UPDATE aunque se estén agregando filas.





MultiBase  
MANUAL DEL PROGRAMADOR

## **CAPÍTULO 13. EL STREAM: Objeto para comunicar con el sistema operativo**

# Introducción

Un STREAM es un canal de comunicación entre un programa de CTL y el sistema operativo (o UNIX). Es decir, es la vía de que dispone el CTL para leer o escribir ficheros propios del sistema operativo y para comunicarnos con programas (sólo en el caso del UNIX) y dispositivos (unidad de discos flexibles, cinta magnética, etc.). La comunicación con el sistema operativo puede ser bidireccional, es decir, de entrada/salida (INPUT/OUTPUT) y puede establecerse contra cualquier fichero o programa (exclusivamente en UNIX a través de «pipelines») del sistema operativo. En el capítulo 8 («Entrada/Salida de Información por Pantalla») se indicaron las instrucciones que manejan la lista en «display». Si bien mediante dichas instrucciones es posible realizar un listado por pantalla, sin embargo, en el caso de querer redireccionarlo hacia la impresora no hay más remedio que emplear STREAMS.

### IMPORTANTE

MultiBase permite enviar información a una base de datos remota (residente en una máquina servidor UNIX) desde una máquina «cliente» (Windows o UNIX), pudiendo utilizar cada una de ellas un gestor de base de datos diferente.

# Definición de STREAM

Cada programa CTL tiene asignado un STREAM por defecto, denominado «STANDARD», cuya salida es la pantalla y cuya entrada es el teclado. En cualquier instrucción del CTL donde pueda aparecer la cláusula STREAM, su omisión indicará que nos estamos refiriendo al STREAM por defecto, es decir, al STREAM STANDARD. Este STREAM STANDARD no es necesario definirlo en la sección DEFINE.

En un módulo CTL se puede definir hasta un máximo de cuarenta STREAMS. La definición de un STREAM se realiza dentro de la sección DEFINE mediante la palabra reservada STREAM y un identificador, es decir, un nombre de STREAM. Su sintaxis es la siguiente:

```
DEFINE
 STREAM nombre_stream
END DEFINE
```

Una vez definido el STREAM, éste se deberá asociar con el fichero, programa o dispositivo que se deseará utilizar.

# Tipos de STREAMS

Dependiendo de la operación a realizar con un STREAM, éstos se dividen en tres tipos: STREAMS de Salida, STREAMS de Entrada y STREAMS de Entrada/Salida. Esta clasificación viene dada por la apertura del STREAM, que es la que redirecciona la entrada, la salida, o ambas. La instrucción que determina el tipo de un STREAM es START, cuya sintaxis, según el tipo de STREAM, es la siguiente:

### a) STREAMS de Salida:

```
START OUTPUT [STREAM {nombre_stream | STANDARD}]
 {TO | THROUGH} destino [UNBUFFERED]
 [BINARY] [APPEND]
```

### b) STREAMS de Entrada:

```
START INPUT [STREAM {nombre_stream | STANDARD}]
 {FROM | THROUGH} fuente [[NO] ECHO] [UNBUFFERED]
 [BINARY]
```

## c) STREAMS de Entrada/Salida:

```
START INPUT-OUTPUT [STREAM {nombre_stream | STANDARD}]
 THROUGH programa [[NO] ECHO] [UNBUFFERED]
 [BINARY]
```

En este mismo capítulo se proporciona más información sobre el manejo de la instrucción START. No obstante, en el Manual de Referencia podrá encontrar una explicación más detallada sobre sus características.

En los siguientes apartados trataremos la forma de manejar cada uno de los tipos de STREAMS expuestos anteriormente.

## STREAMS de salida

Un STREAM de salida se emplea a la hora de realizar un listado. El manejo de este tipo de STREAMS conlleva las siguientes operaciones:

- Formateo de la página de salida (opcional).
- Apertura del STREAM.
- Redireccionamiento de la información hacia el canal recién abierto en el paso anterior.
- Cierre del STREAM.

A continuación se indican las particularidades de cada uno de estos pasos.

### Formateo de la página de salida

En el caso de los STREAMS de salida es posible especificar el formato que tendrá la página del listado. Este formateo de página es opcional, pero en caso de utilizarse tendrá que definirse antes de la apertura del STREAM (paso siguiente).

El formateo de una página de salida se realiza mediante la instrucción FORMAT, cuya sintaxis es la siguiente:

```
FORMAT [STREAM {nombre_stream | STANDARD}] [SIZE tamaño_página]
 [MARGINS { TOP expr | BOTTOM expr | RIGHT expr | LEFT expr }
 [FIRST PAGE HEADER SIZE tamaño
 ...]
 [PAGE HEADER SIZE tamaño
 ...]
 [PAGE TRAILER SIZE tamaño
 ...]
END FORMAT
```

Los valores por defecto que se emplean en un STREAM de salida sin formatear son los siguientes:

| VALORES POR DEFECTO |        |     |
|---------------------|--------|-----|
| Tamaño de la página | SIZE   | 66  |
| Margen superior     | TOP    | 3   |
| Margen inferior     | BOTTOM | 3   |
| Margen izquierdo    | LEFT   | 5   |
| Margen derecho      | RIGHT  | 132 |

En el caso de que no se deseen utilizar estos valores por defecto es cuando habrá que emplear la instrucción FORMAT, especificando las cláusulas «SIZE», para determinar el tamaño de la página, y «MARGINS» para establecer los márgenes.

Asimismo, como puede comprobarse en la sintaxis de la instrucción FORMAT, ésta contiene instrucciones no procedurales que se activarán cuando suceda su acción. Estas instrucciones no procedurales son las siguientes:

1.— «FIRST PAGE HEADER». Esta instrucción no procedural se activará, ejecutando las instrucciones que representa, en el momento en que se inicie el listado. Su finalidad es representar la cabecera de la primera página del listado.

2.— «PAGE HEADER». Esta instrucción no procedural se activará en todas las páginas del listado, representando su cabecera. Si se especifica FIRST PAGE HEADER, esta instrucción no afectará a la primera página del listado.

3.— «PAGE TRAILER». Esta instrucción no procedural se activará en todas las páginas del listado, representando el pie de cada una de ellas.

Para más información acerca de la instrucción FORMAT consulte el Manual de Referencia.

### Ejemplo 1 [PSTREA1]. Listado por pantalla de los 100 primeros números:

```
define
 variable i smallint
end define

main begin
 format stream standard size 24
 margins top 0, bottom 0, left 5, right 80
 first page header size 3
 put stream standard "Listado de 100 primeros números",
 column 50, "Fecha: ", today
 page header size 3
 put stream standard "Listado de 100 primeros números"
 page trailer size 2
 put stream standard skip 1, column 35, "Página: ",
 current page of standard
 end format

 start output stream standard to terminal
 for i = 1 to 100 begin
 put stream standard i , skip 1
 end
 stop output stream standard
end main
```

Este programa formatea el STREAM STANDARD (por lo que no hay que definirlo en la sección DEFINE) especificando todas las instrucciones no procedurales posibles: «FIRST PAGE HEADER», «PAGE HEADER» y «PAGE TRAILER». En la primera de ellas muestra un mensaje indicando el temario del listado junto con la fecha del día («TODAY»). Para dicha cabecera reserva tres líneas de la pantalla. Por su parte, la instrucción «PAGE HEADER» muestra el mismo mensaje que la instrucción anterior pero sin la fecha. Por último, la instrucción «PAGE TRAILER» reserva dos líneas a pie de página para incluir el número de la página en curso. Este número se obtiene mediante la expresión «CURRENT PAGE OF STANDARD».

Una vez formateada la página de salida del listado, éste se abre mediante la instrucción START, que utiliza la pantalla como salida («TO TERMINAL»). A continuación, como el canal ya se encuentra abierto, todas las instrucciones PUT que se empleen hacia dicho STREAM será la información que aparecerá en el listado. En nuestro caso, se construye un bucle «FOR» que cuenta del 1 al 100. Estos números serán los que aparezcan en nuestro listado, estando cada uno en una línea diferente. Al finalizar el bucle se cierra el STREAM mediante la instrucción STOP, apareciendo entonces el listado en pantalla.

## Apertura del STREAM

Una vez formateada la página de salida del listado, el siguiente paso será la apertura del STREAM hacia el fichero o dispositivo donde queremos que resida dicho listado. La apertura del STREAM se realiza mediante la instrucción START OUTPUT, cuya sintaxis es la siguiente:

**START OUTPUT [STREAM {nombre\_stream | STANDARD}]**  
**{TO | THROUGH} destino [UNBUFFERED][BINARY]**  
**[APPEND]**

Como puede apreciarse en esta sintaxis, la salida de información hacia un STREAM de salida puede ser «UNBUFFERED», lo que significa que la salida de la información se realizará carácter a carácter. Asimismo, se puede emplear la cláusula «BINARY» para escritura en binario. Por defecto, la escritura se interpreta como una cadena de caracteres resultado de la conversión según los tipos de datos de la variable o variables a volcar. Si se especifica «BINARY» se escribirán los datos en formato binario con las longitudes correspondientes a cada tipo de dato de SQL, según se muestra en el siguiente cuadro:

| VALORES      |                |                   |          |
|--------------|----------------|-------------------|----------|
| Tipo         | Mínimo         | Máximo            | Nº bytes |
| CHAR(n)      | 1 carácter     | 32.767 caracteres | n        |
| SMALLINT     | -32.767        | +32.767           | 2        |
| INTEGER      | -2.147.483.647 | +2.147.483.647    | 4        |
| TIME         | 00:00:01       | 24:00:00          | 4        |
| DECIMAL(m,n) | $10^{-130}$    | $10^{125}$        | 1 + m/2  |
| DATE         | 01/01/0001     | 31/12/9999        | 4        |
| MONEY(m,n)   | $10^{-130}$    | $10^{125}$        | 1 + m/2  |

Por último, la cláusula «APPEND» indica que en caso de direccionar la salida hacia un fichero, los datos se añadirán al final del mismo, respetando su contenido inicial.

En esta instrucción hay que especificar el «destino» del listado, pudiendo ser éste una expresión. A continuación se indican las formas de redireccionamiento a pantalla, a impresora o a un fichero.

#### a) Listado a pantalla

Cuando se desea ejecutar un listado por pantalla habrá que abrir el STREAM STANDARD de la siguiente forma:

**START OUTPUT STREAM STANDARD TO TERMINAL [UNBUFFERED] [BINARY]**

Esta apertura hacia el terminal controla que cada 24 líneas se produzca una parada del listado, indicándose mediante el carácter «^». No obstante, esta parada puede obviarse o modificarse mediante la variable de entorno PAGEPAUSE, cuyos posibles valores son:

|             |                                                                                                                                                    |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| PAGEPAUSE=N | No se detiene el listado.                                                                                                                          |
| PAGEPAUSE=S | Hace una pausa cada 24 líneas (valor por defecto).                                                                                                 |
| PAGEPAUSE=P | Hace una pausa cada «n» líneas, siendo «n» el valor indicado en la cláusula «SIZE» de la instrucción FORMAT (que equivale al tamaño de la página). |

Este tipo de listado no es bidireccional, es decir, una vez que se ha mostrado una página del listado, ésta no podrá verse de nuevo a no ser que se repita el listado.

#### **Ejemplo 2 [PSTREA2]. Listado de todos los clientes enlazados con sus provincias:**

```
database almacen

main begin
 declare cursor listado for select cliente, empresa, descripcion
 from clientes, provincias
 where clientes.provincia = provincias.provincia
 format stream standard size 24
 margins top 0, bottom 0, left 5, right 80
 page header size 3
 put stream standard column 20,
 "Listado de clientes"
```

```
 page trailer size 2
 put stream standard skip 1,column 30,"Página: ",
 current page of standard
 end format
 start output stream standard to terminal
 put stream standard every row of listado
 stop output stream standard
end main
```

Este programa declara un CURSOR que enlaza dos tablas («clientes» y «provincias»). La tabla derivada generada por dicho CURSOR será la que conforme el listado hacia la pantalla. El STREAM por defecto («STANDARD») se formatea antes de su apertura indicando el tamaño de la página (24 líneas) así como los márgenes superior, inferior, izquierdo y derecho. Asimismo, se fija la cabecera y el pie de cada página del listado, mostrando un literal y el número de la página, respectivamente. Una vez formateado el STREAM STANDARD, éste se abre con destino al terminal («TO TERMINAL»). A continuación, todas las filas que componen la tabla derivada del CURSOR se redirigen hacia el STREAM. Una vez enviada toda la tabla derivada se cierra el STREAM STANDARD. En estos momentos aparecerá el listado en la pantalla.

### IMPORTANTE

En caso de abrir un STREAM hacia el terminal («TO TERMINAL»), éste deberá ser necesariamente el STREAM por defecto, es decir, el STREAM STANDARD.

Como hemos podido comprobar en el ejemplo anterior, el listado no es bidireccional. En caso de querer movernos página a página, ya sea hacia adelante o hacia atrás, habría que redireccionar la información hacia un fichero y mostrarlo posteriormente en una ventana (WINDOW). Esta ventana será la que nos permita desplazarnos por la información de la pantalla.

**Ejemplo 3 [PSTREA3]. Este ejemplo es idéntico al anterior, pero con posibilidad de bidireccionalismo en el listado:**

```
database almacen
define
 variable proceso smallint
end define

main begin
 declare cursor listado for select cliente, empresa, descripcion
 from clientes, provincias
 where clientes.provincia = provincias.provincia
 format stream standard size 22
 margins top 0, bottom 0, left 5, right 80
 page header size 3
 put stream standard column 20,
 "Listado de clientes"
 page trailer size 2
 put stream standard skip 1,column 30,"Página: ",
 current page of standard
 end format
 let proceso = procid()
 start output stream standard to "fic" & proceso left
 put stream standard every row of listado
 stop output stream standard
 window from "fic" & proceso left as file
 24 lines 79 columns label "L i s t a d o"
 at 1,1
 rm "fic" & proceso left
end main
```

El procedimiento que emplea este programa para mostrar el listado por pantalla es la creación de un fichero, cuyo nombre estará compuesto por las letras «fic» más un número que será dinámico y único en máquina



(número de proceso UNIX devuelto por la máquina. En Windows devuelve también un número pero que no se corresponde con el proceso). Una vez que el listado ha finalizado, el fichero se muestra por pantalla en una ventana («WINDOW») con la cláusula «AS FILE». Esta ventana permite movernos página arriba y abajo y a la primera y última del listado. Por último, cuando el operador pulse la tecla asignada a la acción <quit>, la ventana desaparecerá y se borrará el fichero empleado para el listado (comando **rm**).

### ***b) Listado a impresora: selección y atributos***

Para realizar un listado hacia la impresora será necesario abrir el STREAM que se vaya a emplear (que no tendrá que ser necesariamente el STANDARD) de acuerdo a la siguiente sintaxis:

**START OUTPUT [STREAM {nombre\_stream | STANDARD}] THROUGH PRINTER  
[UNBUFFERED] [BINARY]**

La apertura de un STREAM hacia la impresora emplea la variable de entorno DBPRINT para dirigir el listado hacia cualquier impresora conectada al sistema. Por lo tanto, en función de la configuración de esta variable de entorno se empleará una impresora u otra. Los valores por defecto que toma esta variable en cada sistema operativo son los siguientes:

| UNIX/Linux | Windows         |
|------------|-----------------|
| DBPRINT=lp | set DBPRINT=PRN |

En el caso del UNIX/Linux, esta variable apunta al comando **lp** del sistema operativo, el cual indica que la salida por impresora se realizará por medio del «spooler» del sistema. Dicho «spooler», por lo general, tiene una impresora de sistema, es decir, una impresora por defecto («system default destination»). En consecuencia, si no se configura la variable DBPRINT, todos los listados que tengan como salida la impresora utilizarán la que el sistema tenga asignada por defecto. Asimismo, podrán indicarse en la definición de la variable todas las cláusulas admitidas por el comando **lp**. Por ejemplo, para seleccionar una impresora distinta de la que el sistema tiene asignada por defecto se puede emplear la cláusula «-d» de dicho comando, indicando el nombre de la impresora que se desea utilizar:

```
DBPRINT="lp -d impresora1"
```

Por otra parte, empleando la cláusula «THROUGH PRINTER» en la apertura del STREAM para crear un fichero es posible simular el redireccionamiento de la información hacia un fichero:

```
DBPRINT="cat > /tmp/fichero"
```



En Windows, el contenido por defecto de la variable DBPRINT indica que se empleará el comando **print** para la impresión a través de un puerto del sistema. La impresión por ese puerto se realizará carácter a carácter. En este caso también existe la posibilidad de seleccionar otra impresora distinta. Así, por ejemplo, en caso de tener conectada una impresora al puerto LPT2 se podría especificar:

```
set DBPRINT=LPT2
```

Además de permitir la impresión a través del puerto, es posible emplear también el Administrador de Impresión propio de este entorno. Para ello, la variable de entorno DBPRINT deberá definirse con el valor «PRINTMAN» o «PM». Por ejemplo:

```
set DBPRINT=PRINTMAN
```

pone de una función interna, denominada «putenv()», que permite asignar valores a las variables de entorno. Por lo tanto, mediante esta función se podrá gestionar la selección de la impresora por la que se desea realizar el listado. Este modo de configurar la variable DBPRINT debe gestionarse antes de la apertura del STREAM hacia la impresora.

### Ejemplo 4 [PSTREA4]. Elección de la impresora por la que se realizará el listado:

```
database almacen

define
 frame impresora
 screen
 {
 Nombre Impresora: [a]
 Tamaño de página: [b]
 Margen superior: [c]
 Margen inferior:[d]
 Margen izquierdo: [e]
 Margen derecho:[f]
 }
end screen

variables
 a = impre char
 b = pagina smallint default 66
 c = arriba smallint default 3
 d = abajo smallint default 3
 e = izqui smallint default 5
 f = dere smallint default 80
end variables

layout
 box
 label "I m p r e s o r a"
end layout
end frame
end define

main begin
 declare cursor listado for select cliente, empresa, descripcion
 from clientes, provincias
 where clientes.provincia = provincias.provincia
 input frame impresora for add
 end input
 while cancelled = true begin
 input frame impresora for add
 end input
 end
 switch
 case getctlvalue("O.S.") in ("", "WINDOWS")
 call putenv ("DBPRINT",impresora.impre)
 default
 if impresora.impre is null then call putenv ("DBPRINT","lp")
 else call putenv ("DBPRINT", "lp -d" & impresora.impre)
 end
 format stream standard size impresora.pagina
 margins top impresora.arriba, bottom impresora.abajo,
 left impresora.izqui, right impresora.dere
 page header size 3
 put stream standard column 20, "Listado de clientes"
 page trailer size 2
 put stream standard skip 1,column 30,"Página: ",
 current page of standard
 end format
 start output stream standard through printer
 put stream standard every row of listado
 stop output stream standard
end main
```

En este programa se realiza un listado por impresora, permitiendo al operador elegir la impresora final y especificar los márgenes. Esta petición de datos se realiza asignando valores por defecto a un FRAME en cada una de sus variables, que representan los márgenes de la página. Dichos valores deberán ser grabados por el operador para que tomen efecto, de lo contrario volverán a solicitarse hasta que sean aceptados.

En este programa no se controla la información introducida por el operador a nivel de márgenes. No obstante, esta comprobación se podría realizar con las instrucciones no procedurales de la sección EDITING.

Una vez aceptados los valores introducidos en el FRAME, y dependiendo del sistema operativo que se esté utilizando, la variable DBPRINT tomará un valor u otro. A continuación, se formatea la página de listado mediante la instrucción FORMAT, que en este caso emplea el tamaño y los márgenes de forma dinámica, asignando los valores introducidos por el operador en el FRAME. El siguiente paso consiste en abrir el STREAM hacia la impresora y listar la información devuelta por el CURSOR. El listado no comenzará a salir por impresora hasta que no se ejecuta la instrucción «STOP OUTPUT STREAM ...».

En los listados por impresora es muy común utilizar distintos atributos de impresión (caracteres en negrita, en cursiva, expandidos o comprimidos, subrayados, etc.). Para poder utilizar estos atributos habrá que asegurarse, en primer lugar, de que las secuencias de escape de la impresora que vamos a emplear se encuentran en el fichero «tprinter» de MultiBase, que se encuentra en el subdirectorio «etc». Para más información sobre la forma de configurar dicho fichero consulte el capítulo sobre «Configuración de Impresoras» del Manual del Administrador.

Una vez configuradas en el fichero «tprinter» las secuencias de escape correspondientes a los diferentes atributos de impresión, la forma de emplear dichos atributos será asignando un valor a la variable de entorno RPRINTER. El valor que habrá que asignar a dicha variable será el indicado en el fichero «tprinter» encabezando las secuencias de escape. Por ejemplo:

```
:f3150|proprinter
```

| # | function | key  |
|---|----------|------|
|   | normal   | \022 |
|   | bold-on  | \EE  |
|   | bold-off | \EF  |
|   | ...      |      |

En este ejemplo, el valor que tendríamos que asignar a la variable de entorno RPRINTER podría ser «f3150» o «proprinter»:

```
RPRINTER=f3150
RPRINTER=proprinter
```

No obstante, como ya hemos visto anteriormente, la selección de la impresora puede ser también dinámica. Para ello podremos emplear la instrucción OPTIONS o bien definir la sección del mismo nombre (OPTIONS) dentro de la sección GLOBAL, si se trata de un módulo principal, o LOCAL, si se trata de un módulo librería, para definir los atributos de la impresora seleccionada. Esta impresora se seleccionará mediante la cláusula «PRINTER» de la instrucción o sección, según el caso.

Una vez elegida la impresora, los atributos de ésta se invocan mediante la cláusula «CONTROL(atributo)» de la instrucción PUT.

**Ejemplo 5 [PSTREAS].** Este ejemplo es similar al anterior, pero eligiendo el tipo de impresora para emplear atributos:

```
database almacen

define
 frame impresora
 screen
{
```

```

Nombre Impresora: [a]
Tamaño de página: [b]
Margen superior:[c]
Margen inferior:[d]
Margen izquierdo: [e]
Margen derecho: [f]
Tipo Impresora:[g]
}

end screen

variables
 a = impre char
 b = pagina smallint default 66
 c = arriba smallint default 3
 d = abajo smallint default 3
 e = izqui smallint default 5
 f = dere smallint default 80
 g = tipoimpre char default "ibm3150"
end variables

layout
 box
 label "I m p r e s o r a"
end layout
end frame

end define

main begin
 declare cursor listado for select cliente, empresa, descripcion
 from clientes, provincias
 where clientes.provincia = provincias.provincia
 input frame impresora for add
 end input
 while cancelled = true begin
 input frame impresora for add
 end input
 end
 switch
 case getctlvalue("O.S.") in ("", "WINDOWS")
 call putenv ("DBPRINT", impresora.impre)
 default
 if impresora.impre is null then call putenv ("DBPRINT", "lp")
 else call putenv ("DBPRINT", "lp -d" & impresora.impre)
 end
 options
 printer impresora.tipoimpre
 end options
 format stream standard size impresora.pagina
 margins top impresora.arriba, bottom impresora.abajo,
 left impresora.izqui, right impresora.dere
 page header size 3
 put stream standard column 20, control("bold-on"),
 "Listado de clientes", control ("bold-off")
 page trailer size 2
 put stream standard skip 1, control("italic-on"),
 column 30, "Página: ", current page of standard,
 control ("italic-off")
 end format
 start output stream standard through printer
 put stream standard every row of listado
 stop output stream standard
end main

```

Este ejemplo es similar al «pstrea4», con la diferencia de que en este caso el operador elige en el FRAME el tipo de impresora que va a emplear. Con el nombre de impresora indicado, la instrucción OPTIONS con la cláusula «PRINTER» selecciona los atributos del fichero «tprinter». Estos atributos se emplean tanto en la «PAGE HEADER» como en la «PAGE TRAILER» mediante la cláusula «control()». El resto del programa es idéntico al anterior.



Por lo que respecta a Windows, si se desea emplear el Administrador de Impresión habrá que asignar el valor «PRINTMAN» o «PM» a la variable de entorno DBPRINT:

```
set DBPRINT=PRINTMAN
```

Estos mismos valores deberán asignarse también a la variable RPRINTER en el caso de querer utilizar atributos de impresión. La forma de utilizar estos atributos con el Administrador de Impresión del Windows es mediante la cláusula «control()». El aspecto del fichero «tprinter» en este caso sería el siguiente:

```
:PRINTMAN|PM
```

```
functionkey
 Negrita-on =Times_New_Roman|b|10
 Itálica-on =Times_New_Roman|i|10
 Comprimido =Times_New_Roman|l|17
 ...
```

En este caso es posible especificar una fuente («font») para que sea utilizada por defecto. Dicha fuente deberá estar definida en la sección «.CONFIGURATION» del fichero «\*.ini», que se encuentra en el subdirectorio «etc» de MultiBase y que se invoca mediante la cláusula «-ini» del comando **ctl**. Por ejemplo:

```
PRINTMANFONT=Times_New_Roman
```

Al utilizar el Administrador de Impresión del Windows cada página de listado está definida como una matriz de «l x c» (líneas x columnas), siendo «l» el número de líneas y «c» el de columnas que pueden ser escritas utilizando la fuente definida. El número de líneas y de columnas se calcula en función de la altura y anchura medias de la fuente empleada. Si no se especifica nada, CTL utilizará estos parámetros para calcular el interlineado (separación entre líneas) del listado.

NOTAS:

- 1.— En las versiones de MultiBase para Windows es posible también dirigir la impresión hacia el puerto asignando el valor «lpt1» a la variable de entorno DBPRINT.
- 2.— Para más información sobre cómo configurar los ficheros «tprinter» y «\*.ini» consulte el Manual del Administrador.

### c) Listado a fichero

Para realizar un listado hacia un fichero hay que abrir el STREAM empleando la siguiente sintaxis:

```
START OUTPUT [STREAM {nombre_stream | STANDARD}] TO fichero
[UNBUFFERED] [BINARY] [APPEND]
```

El nombre del fichero podrá ser una expresión.

Si el fichero indicado no existe, éste se creará en el directorio especificado en la apertura del STREAM que se utilice para la realización del listado. En caso de no indicarse el «path» completo el fichero se creará en el directorio en curso. En instalaciones UNIX y en redes de área local habrá que tener en cuenta respectivamente los permisos de escritura establecidos por el propio sistema operativo sobre los directorios o sobre las diferentes particiones existentes en el disco.

Por el contrario, si el nombre del fichero indicado como salida del listado coincide con el de algún fichero ya existente, éste se sobrescribirá, salvo que se hubiese indicado la cláusula «APPEND» en la sintaxis, en cuyo caso el listado se añadiría al final del contenido del fichero.

### IMPORTANTE

En los listados cuyo destino sea un fichero tenga especial cuidado a la hora de asignar el nombre del «fichero» de destino, ya que si dicho fichero existe y no se emplea la cláusula «APPEND» el contenido del fichero se sobrescribirá, perdiéndose el contenido anterior.

Si el STREAM se ha abierto con la cláusula «UNBUFFERED», cada instrucción PUT que escriba sobre el STREAM se reflejará directamente en el fichero creado en la apertura. Por contra, si no se incluye dicha cláusula el contenido del fichero se creará cuando se produzca el cierre del STREAM.

### Ejemplo 6 [PSTREA6]. Simulación de la instrucción UNLOAD:

```
database almacen

define
 frame provin like provincias.*
end frame
end define

main begin
 declare cursor descarga for select * by name on provin from provincias
 control
 on every row
 put stream standard provin.provincia left clipped,"|",
 provin.descripcion & NULL,"|",
 provin.prefijo left & NULL,"|", skip 1
 end control
 start output stream standard to "provin.unl"
 foreach descarga begin end
 stop output stream standard
 window from "provin.unl" as file
 18 lines 40 columns
 label "D e s c a r g a"
 at 1,20
 end main
```

Este programa define un FRAME que recibirá la información de cada una de las filas que componen la tabla derivada del CURSOR declarado en la sección MAIN. Dicho CURSOR define su sección CONTROL, donde indica que para cada una de las filas de su tabla derivada escribirá sobre el STREAM que se abrirá posteriormente. En la instrucción de escritura PUT las variables numéricas se ajustan a la izquierda, truncándose los blancos que queden a la derecha (mediante «CLIPPED» o bien concatenando con un «NULL»), mientras que la variable alfanumérica se concatena con un «NULL». Estas operaciones de truncado y concatenación se realizan para que las variables definidas con la cláusula «LIKE» pierdan los atributos al convertirse en una expresión. En caso contrario aparecerían las etiquetas de cada una de las columnas asignadas en el diccionario. Por último, el fichero generado con el STREAM «provin.unl» se muestra en una ventana generada por la instrucción WINDOW con la cláusula «AS FILE».

### d) Listado a un programa (sólo en versiones UNIX)

Para realizar un listado hacia un programa hay que abrir el STREAM de la siguiente forma:

```
START OUTPUT [STREAM {nombre_stream | STANDARD}]
THROUGH programa [UNBUFFERED] [BINARY]
```

Esta característica de los STREAMS sólo puede emplearse en las versiones de MultiBase sobre sistema operativo UNIX/Linux, ya que Windows carece de la posibilidad de multiproceso.

En este caso el resultado del listado se pasa como información de entrada hacia el comando «programa» indicado en la apertura del STREAM. A fin y al cabo, los STREAMS son tuberías («pipelines») del sistema operativo.

**Ejemplo 7 [PSTREA7]. Creación de un fichero y grabación de éste en un disco flexible:**

```

database almacen

define
 stream out
end define

main begin
 declare cursor listado for select cliente, empresa, descripcion
 from clientes, provincias
 where clientes.provincia = provincias.provincia
 start output stream out through "cat > clientes ; tar cv clientes"
 put stream out every row of listado
 stop output stream out
end main

```

Este programa define un STREAM («out») que se abrirá hacia un comando del sistema operativo UNIX (**cat**) para la generación de un fichero («clientes»). Una vez generado el fichero con toda la información del CURSOR («every row of listado»), éste se grabará en un disco flexible mediante el comando «**tar cv ...**». Como puede observarse, en este ejemplo se han concatenado dos comandos del sistema operativo mediante el separador «;».

**Ejemplo 8 [PSTREA8]. Empleo del comando sort del UNIX para ordenar la salida del STREAM:**

```

database almacen

main begin
 declare cursor listado for select empresa
 from clientes
 start output stream standard through "sort > /tmp/clientes"
 put stream standard every row of listado
 stop output stream standard
 window from "/tmp/clientes" as file
 20 lines 40 columns label "Clientes"
 at 1,20
end main

```

Este programa envía todas las filas del CURSOR «listado» hacia el STREAM cuyo destino es el comando **sort**. El resultado de este comando se recoge sobre un fichero («/tmp/clientes»), el cual, después de ordenarse con el comando **sort**, se muestra en una ventana («WINDOW» con cláusula «AS FILE») en la que podemos movernos hacia arriba, hacia abajo, al principio y al final.

## Redireccionamiento de la información hacia el STREAM

Una vez definido el STREAM, formateado y abierto mediante cualquiera de los procedimientos indicados en el apartado anterior, el siguiente paso consiste en conocer la forma de redireccionar la información hacia el STREAM. Para ello, CTL dispone de cuatro instrucciones específicas, que son: PUT EVERY ROW, PUT, PUT FRAME y PUT FILE.

A continuación se comentan las particularidades más significativas de cada una de estas instrucciones en el manejo de STREAMS. No obstante, para una información más detallada consulte el Manual de Referencia de MultiBase.

### Instrucción PUT EVERY ROW

Esta instrucción precisa de la declaración de un CURSOR para la realización del listado. Su finalidad es enviar todas las filas («EVERY ROW») del CURSOR declarado hacia el STREAM. La forma de presentarlo en el destino elegido en dicho STREAM es tal cual lo haría la instrucción SELECT en una ventana a través de las instrucciones «WINDOW» o «TSQL». La sintaxis de la instrucción PUT EVERY ROW es la siguiente:

**PUT [STREAM {nombre\_stream | STANDARD}] EVERY ROW OF nombre\_cursor**

En este caso no es necesario especificar ni la apertura, ni la lectura ni el cierre del CURSOR, por lo que no habrá que utilizar las instrucciones OPEN, FETCH, CLOSE o FOREACH de manejo de dicho objeto, ya que es la propia PUT EVERY ROW quien se encarga de realizar las operaciones antes citadas. Asimismo, tampoco será necesario declarar la recepción de variables en el CURSOR (cláusulas «INTO», «BY NAME» o «BY NAME ON frame»), ya que la instrucción PUT EVERY ROW no emplea dichas variables receptoras para la realización del listado

### Ejemplo 9 [PSTREA9]. Listado de los albaranes con sus fechas de todos los clientes de la base de datos:

```
database almacen

main begin
 declare cursor listado for select albaran, fecha_albaran, empresa, descripcion
 from clientes, provincias, albaranes
 where clientes.provincia = provincias.provincia
 and clientes.cliente = albaranes.cliente
 order by albaran
 format stream standard size 24
 margins top 0, bottom 0, left 2, right 80
 end format
 start output stream standard to terminal
 put stream standard every row of listado
 stop output stream standard
end main
```

Este programa declara un CURSOR compuesto por la lectura de tres tablas de la base de datos «almacen», ordenando la tabla derivada por el código del albarán («albaran»). El formateo del STREAM consiste en fijar los márgenes y el tamaño de la página. En este caso no se ha empleado ninguna instrucción no procedural para las cabeceras y pies de página. Dado que el STREAM se abre a la pantalla («terminal»), habrá que emplear el STREAM STANDARD. La siguiente instrucción (PUT EVERY ROW) envía todas las filas devueltas por el CURSOR hacia el STREAM, cerrándose éste con la instrucción STOP.

### Instrucción PUT: Posicionamiento y atributos de impresión

Esta instrucción permite enviar expresiones a la salida asociada a un STREAM, para lo cual dispone de cláusulas específicas que permiten el posicionamiento de dichas expresiones. Su sintaxis es la siguiente:

**PUT [STREAM {nombre\_stream | STANDARD}] expresión [REVERSE | UNDERLINE]  
[LABEL {"literal" | número\_ayuda}] [, ...]**

En una instrucción PUT se puede indicar una o varias expresiones, en cuyo caso éstas deberán ir separadas por comas.

La colocación de una expresión dentro de una página puede indicarse con cualquiera de las siguientes cláusulas de la instrucción PUT:

**[COLUMN columna]** Esta cláusula se posiciona en la «columna» indicada, que podrá ser una expresión. Si a continuación de la cláusula se incluye una expresión, ésta aparecerá igualmente en la «columna». No es posible escribir en una columna anterior a aquella sobre la cual estamos situados, es decir, no se puede sobrescribir.

```
put stream standard column 3, var1, column 20, var2
```

Este ejemplo significa que la variable «var1» aparecerá en la columna 3 y la variable «var2» en la columna 20 de la línea en curso.

**[SPACES blancos]** Esta cláusula incluye los espacios en blanco indicados en «blancos», pudiendo éstos ser una expresión.

```
put stream standard column 3, var1, spaces 2, var2
```



Este ejemplo indica que la variable «var1» aparecerá en la columna 3 de la línea en curso, mientras que la variable «var2» aparecerá dos espacios a la derecha desde la posición en que finalizó «var1».

**[SKIP [líneas]]**

Esta cláusula salta tantas líneas como indique «líneas», pudiendo ésta ser una expresión. En caso de que se omita este parámetro se entiende que el salto que se tiene que producir es de una línea.

```
put stream standard column 3, var1, spaces2, var2, skip 2
put stream standard column 30, "literal"
```

Siguiendo con el ejemplo anterior, entre la primera instrucción PUT y la segunda habrá dos saltos de línea, es decir, una línea en blanco entre ambas.

**[CONTROL(atributo)]**

En listados hacia impresora, esta cláusula empleará el «atributo» indicado hasta que se indique lo contrario. Dicho atributo deberá estar definido en el fichero de filtro de impresora («tprinter») y tener a su vez la variable de entorno RPRINTER con el tipo de impresora definido en dicho fichero. La asignación de un valor a esta variable también puede simularse con la instrucción OPTIONS y cláusula «PRINTER» (ver el epígrafe anterior de este mismo capítulo).

```
put stream standard column 3, control("bold-on"), var1,
 control("bold-off"), spaces 2, control("italic-on"),
 var2, control("italic-off"), skip 2
```

En este caso la variable «var1» se imprimirá en negrita y «var2» en cursiva (itálica). Las restantes instrucciones PUT utilizarán impresión normal.

**[FLUSH]**

Esta cláusula sólo tiene sentido cuando el STREAM que se emplea no es «UN-BUFFERED». Su finalidad es «volcar» la información contenida en el «buffer» de salida hasta ese momento.

```
put stream standard column 3, var1, spaces2, var2, skip 2
put stream standard column 30, "literal", flush
```

En este ejemplo, toda la información retenida en el «buffer» de escritura del STREAM se «volcará» hacia el destino indicado en la apertura.

**Ejemplo 10 [PSTREA10]. Listado de artículos agrupados por proveedor:**

```
database almacen

define
 frame rangos
 screen
{
 Desde proveedor: [a] [b
]
 Hasta proveedor: [c] [d
]
}

end screen

variables
 a = desde like proveedores.proveedor
 lookup b = empresa joining proveedores.proveedor
 check ($ is not null)
 c = hasta like proveedores.proveedor
 lookup d = empresa joining proveedores.proveedor
 check ($ is not null and $ >= desde)
end variables
```

```
layout
 box
 label "R a n g o s"
 end layout
end frame

variable articulo like articulos.articulo
variable descripcion like articulos.descripcion
variable pr_vent like articulos.pr_vent
variable proveedor like proveedores.proveedor
variable empresa like proveedores.empresa
variable telefono like proveedores.telefono
variable direccion1 like proveedores.direccion1
variable provincia like provincias.descripcion
variable prefijo like provincias.prefijo
variable sz smallint
variable salida char(1) upshift
end define

main begin
 declare cursor listado for select articulo, articulos.proveedor,
 articulos.descripcion, pr_vent, empresa, direccion1, telefono,
 provincias.descripcion provincia, prefijo
 by name
 from proveedores, articulos, provincias
 where proveedores.proveedor between $rangos.desde and $rangos.hasta
 and proveedores.proveedor = articulos.proveedor
 and proveedores.provincia = provincias.provincia
 order by proveedor
 control
 before group of proveedor
 put stream standard column 10, "Proveedor: ",
 proveedor left, column 30, empresa & NULL, skip 1
 put stream standard column 10, "Dirección: ",
 direccion1 & NULL, skip 1
 put stream standard column 10, "Teléfono: (",
 prefijo using "###", spaces 1,") ", telefono & NULL, skip
 put stream standard column 10, "Provincia: ",provincia & NULL, skip 2
 put stream standard strrepeat("-",76), skip 1
 put stream standard "Artículo", column 20,
 "D e s c r i p c i ó n", column 50,
 "Precio Venta", skip 1
 put stream standard strrepeat("-",76), skip 2
 on every row
 put stream standard articulo left clipped,
 column 20, descripcion & NULL,
 column 50, pr_vent using "##,###,###.##", skip 1
 after group of proveedor
 put stream standard skip 2, column 30,
 "T O T A L A R T I C U L O S",
 (group count) using "###,###"
 new page stream standard
 end control

 menu salida
 if cancelled = true then exit program

 input frame rangos for add
 end input
 if cancelled = true then exit program
 clear frame rangos
 format stream standard size sz
```

```

margins top 0, bottom 0, left 0, right 80
first page header size 5
 put stream standard column 30, "L i s t a d o", skip 1
 put stream standard column 36, "d e", skip 1
 put stream standard column 30, "A r t í c u l o s", skip
 put stream standard column 36, "p o r", skip 1
 put stream standard column 30, "P r o v e e d o r e s", skip
page header size 2
 put stream standard column 5, "Fecha: ", today,
 column 30, "Artículos por Proveedor",
 column 60, "Hora: ", now
page trailer size 2
 put stream standard skip 1, "Página: ",
 current page of standard
end format

switch salida
case "P"
 start output stream standard to terminal
case "F"
 start output stream standard to "fic" & procid() left
case "I"
 start output stream standard through printer
end

foreach listado begin end
stop output stream standard
if salida = "F" then
 window from "fic" & procid() left as file
 23 lines 80 columns label "L i s t a d o"
 at 1,1
end main

menu salida "E l e g i r" down line 10 column 40 no wait skip 2
option "Pantalla"
 let salida = "P"
 let sz = 24
option "Fichero"
 let salida = "F"
 let sz = 21
option "Impresora"
 let salida = "I"
 let sz = 66
end menu

```

Este programa realiza un listado hacia pantalla, impresora o fichero, según elija el operador. En primer lugar se define un FRAME para pedir el rango de los proveedores sobre los que se desea realizar el listado. Una vez comprobada la validez del rango (mediante el atributo CHECK del CTL), se definen las variables que serán receptoras de los valores devueltos por el CURSOR que se declara a continuación. El programa comienza declarando de dicho CURSOR, donde interviene su sección CONTROL con varias instrucciones no procedurales:

- «BEFORE GROUP OF proveedor» escribirá los datos del proveedor, así como una cabecera de los artículos de que dispone dicho proveedor.
- «ON EVERY ROW» escribirá cada uno de los artículos del proveedor especificado anteriormente, es decir, la línea de detalle.
- «AFTER GROUP OF proveedor» calculará el total de artículos de dicho proveedor, saltando a su vez de página.

A continuación, aparecerá un menú de tipo «Pop-Up» donde el operador podrá seleccionar la salida del listado. Posteriormente se mostrará el FRAME con los rangos de los proveedores que se desean listar. El STREAM empleado en este caso es el STANDARD, que se formatea con un tamaño de página dinámico, dependiendo de la

salida, y con unos márgenes fijos. Asimismo, esta instrucción `FORMAT` incluye instrucciones no procedurales para fijar la cabecera de la primera página («FIRST PAGE HEADER») y de las siguientes («PAGE HEADER»), así como el pie de página de todas ellas («PAGE TRAILER»).

El siguiente paso consiste en la apertura del `STREAM` hacia la salida indicada en el menú «Pop-Up» anterior. Después de su apertura se leen todas las filas del `CURSOR` y, dependiendo de la acción que se produzca, se ejecutarán las instrucciones no procedurales definidas entre el formateo del `STREAM` y la sección `CONTROL` del `CURSOR`. Por último, una vez finalizado el listado y cerrado el `STREAM`, si la salida elegida ha sido un fichero, éste se mostrará en una ventana (instrucción `WINDOW` con cláusula «AS FILE»).

Como puede comprobarse, al imprimirse las variables éstas se concatenan con «NULL» o se ajustan a la izquierda para convertirlas en expresiones. Esta operación se realiza para que no aparezcan las etiquetas definidas en el diccionario a las columnas con las que se han definido las variables mediante la cláusula «LIKE».

En caso de emplear el Administrador de Impresión del Windows, el procedimiento de posicionamiento es diferente. Por lo que respecta a listados encolumnados (por ejemplo un listado de albaranes), la técnica seguida hasta ahora consistía en rellenar con espacios en blanco hasta alcanzar la siguiente columna. Esta técnica carece ahora de sentido si en el listado se emplea alguna fuente («font») de tamaño proporcional, en el que los caracteres no tienen todos la misma anchura (el carácter «m» tiene una anchura superior al carácter «l», y éste a su vez es distinto de «espacio en blanco», etc.).

Para solucionar este problema es preciso que el posicionamiento en una columna no se haga rellenando con espacios en blanco, sino que sea fijo e independiente del tamaño de una letra. Por lo tanto, para posicionarse en una columna determinada existen dos caminos:

a) Utilizando la función «`setpmattr()`» con el parámetro «`curcol`» y el valor de la columna donde deseamos posicionarnos. Por ejemplo:

```
call setpmattr("curcol",15)
```

b) Especificando que la cláusula «`COLUMN`» de la instrucción `PUT STREAM` se refiera a columnas fijas (en caso contrario rellenaría con espacios en blanco). Por ejemplo:

```
call setpmattr("fixprtcolum", true)
...
...
put stream standard column 23, ...
```

Sin embargo, si lo que se quiere encolumnar son caracteres numéricos, lo normal es que éstos se alineen a la derecha. En este caso deberemos especificar el parámetro «`rightalign`» de la función «`setpmattr`». Por ejemplo:

```
call setpmattr("rightalign",30)
```

Asimismo, también puede conseguirse esta alineación mediante la cláusula «`COLUMN`» con un valor negativo, que indicará la posición donde acaba el texto:

```
call setpmattr("fixprtcolum", true)
put stream standard column -53, numero using "###,###"
```

Este ejemplo indica que el número comenzará a escribirse en la columna 47, representándose el último carácter numérico en la columna 53.

**Ejemplo 11 [PSTREA11].** Este ejemplo es similar a «`pstrea10`», salvo que en este caso se ha empleado el Administrador de Impresión de Windows para obtener la salida:

```
database almacen

define
 frame rangos
 screen
{
```

```

Desde proveedor: [a] [b]
Hasta proveedor: [c] [d]

}
end screen

variables
 a = desde like proveedores.proveedor
 lookup b = empresa joining proveedores.proveedor
 check ($ is not null)
 c = hasta like proveedores.proveedor
 lookup d = empresa joining proveedores.proveedor
 check ($ is not null and $ > = desde)
end variables

layout
 box
 label "R a n g o s"
end layout
end frame

variable articulo like articulos.articulo
variable descripcion like articulos.descripcion
variable pr_vent like articulos.pr_vent
variable proveedor like proveedores.proveedor
variable empresa like proveedores.empresa
variable telefono like proveedores.telefono
variable direccion1 like proveedores.direccion1
variable provincia like provincias.descripcion
variable prefijo like provincias.prefijo
end define

main begin
 declare cursor listado for select articulo, articulos.proveedor,
 articulos.descripcion, pr_vent, empresa, direccion1, telefono,
 provincias.descripcion provincia, prefijo
 by name
 from proveedores, articulos, provincias
 where proveedores.proveedor between $rangos.desde and $rangos.hasta
 and proveedores.proveedor = articulos.proveedor
 and proveedores.provincia = provincias.provincia
 order by proveedor

 control
 before group of proveedor
 put stream standard column 10, "Proveedor: ",
 proveedor left, column 30, empresa & NULL, skip 1
 put stream standard column 10, "Dirección: ",
 direccion1 & NULL, skip 1
 put stream standard column 10, "Teléfono: (",
 prefijo using "###", spaces 1,") ", telefono & NULL, skip
 put stream standard column 10, "Provincia: ",provincia & NULL, skip 2
 put stream standard strrepeat("-",76), skip 1
 put stream standard "Artículo", column 20,
 "D e s c r i p c i ó n", column 50,
 "Precio Venta", skip 1
 put stream standard strrepeat("-",76), skip 2
 on every row
 put stream standard articulo left clipped,
 column 20, descripcion & NULL,
 column -63, pr_vent using "##,###,###.##", skip 1

```

```
 after group of proveedor
 put stream standard skip 2, column 30, control("=Times_New_Roman|b|17"),
 "TOTAL ARTICULOS",
 (group count) using "###,###", control("=Times_New_Roman||10")
 new page stream standard
 end control

 call setpmattr("fixprtcolum",true)
 call putenv("DBPRINT","PRINTMAN")
 options
 printer "PRINTMAN"
 end options

 input frame rangos for add
 end input
 if cancelled = true then exit program

 clear frame rangos
 format stream standard size 66
 margins top 0, bottom 0, left 0, right 80
 first page header size 5
 put stream standard column 30, control("=Times_New_Roman|b|17"),
 "Lista de", skip 1
 put stream standard column 36, "de", skip 1
 put stream standard column 30, "Artículos", skip
 put stream standard column 36, "por", skip 1
 put stream standard column 30, "Proveedores", control("=Times_New_Roman||10"),
 skip
 page header size 2
 put stream standard column 5, "Fecha: ", today,
 column 30, "Artículos por Proveedor",
 column 60, "Hora: ", now
 page trailer size 2
 put stream standard skip 1, "Página: ",
 current page of standard
 end format
 start output stream standard through printer
 foreach listado begin end
 stop output stream standard
end main
```

Las diferencias fundamentales entre los ejemplos 10 y 11 son las siguientes:

- 1.— En este caso no existe menú para seleccionar la salida o el destino del listado.
- 2.— Al emplear el Administrador de Impresión de Windows, el tamaño de la página de salida es fijo en 66 líneas.
- 3.— Antes de comenzar el listado se fija la variable de entorno DBPRINT con el valor «PRINTMAN», indicando que se empleará el Administrador de Impresión del Windows.
- 4.— Asimismo, se establece el posicionamiento fijo de columnas («fixprtcolum»).
- 5.— El tipo de impresora («PRINTMAN») se selecciona mediante la instrucción OPTIONS y cláusula «PRINTER».
- 6.— Por último, se emplean distintos atributos de impresión utilizando la fuente «Times\_New\_Roman». Dichos atributos se invocan mediante la cláusula «CONTROL(...)».

### Instrucción PUT FRAME

La instrucción PUT FRAME permite enviar el contenido de un FRAME a la salida asociada a un STREAM. Su sintaxis es la siguiente:

**PUT [STREAM {nombre\_stream | STANDARD}] FRAME nombre\_frame**

Los semigráficos generados por los metacaracteres de la SCREEN no aparecerán impresos, por lo cual, si se desean incluir gráficos en el FRAME habrá que dibujarlos. El editor de textos de MultiBase («tword -e» — disponible únicamente en las versiones para y UNIX—) dispone de una cláusula que permite dibujar gráficos empleando caracteres semigráficos simples o dobles.

**Ejemplo 12 [PSTREA12]. Listado de etiquetas de clientes:**

```

database almacén

define
 variable sz smallint
 variable salida char(1) upshift

 frame eti
 screen
 {
 [a]
 [b]
 [c]
 [d]
 [e]

 }

 end screen

 variables
 a = empresa char
 b = nombre char
 untagged apellidos char(25)
 c = direccion1 char
 untagged distrito like clientes.distrito
 d = poblacion char
 e = descripcion char
 end variables
 end frame
end define

main begin
 declare cursor listado for select empresa, apellidos, nombre,
 direccion1, distrito, poblacion, descripcion
 by name on eti
 from clientes,provincias
 where clientes.provincia = provincias.provincia
 control
 on every row
 let eti.nombre = eti.nombre clipped && " " && eti.apellidos clipped
 let eti.poblacion = eti.distrito using "#####" &&
 " - " && eti.poblacion
 put stream standard frame eti
 end control

 menu salida
 if cancelled = true then exit program

 format stream standard size sz
 margins top 0, bottom 0, left 5, right 80
 end format

```

```
start output stream standard to "fic" & procid() left
foreach listado begin end
stop output stream standard
if salida = "P" then begin
 window from "fic" & procid() left as file
 22 lines 60 columns label "E t i q u e t a s"
 at 1,10
 rm "fic" & procid() left
end
end main

menu salida "E l e g i r" down line 10 column 40 no wait skip 2
option "Pantalla"
 let salida = "P"
 let sz = 22
option "Impresora"
 let salida = "I"
 let sz = 66
end menu
```

Este programa define un FRAME con diversas variables encerradas en una caja generada con semigráficos simples. Algunas de estas variables se declaran como «UNTAGGED» debido a que sólo se van a emplear para recibir datos del CURSOR, concatenándose posteriormente a otras variables («apellidos» y «distrito»). El CURSOR se declara con su sección CONTROL, en la que la instrucción no procedural «ON EVERY ROW» imprimirá un FRAME por cada una de las filas que componen la tabla derivada del CURSOR. Previamente a la apertura del STREAM, éste se formatea con un tamaño de página dinámico, dependiendo del «destino» elegido por el operador. Este destino se elige en un menú de tipo «Pop-Up». En caso de seleccionar la opción «Pantalla» se empleará realmente como «destino» un fichero, el cual se mostrará posteriormente en una ventana mediante la instrucción WINDOW con la cláusula «AS FILE».

### Instrucción PUT FILE

Esta instrucción permite enviar el contenido de un fichero a la salida asociada a un STREAM. Su sintaxis es la siguiente:

**PUT [STREAM {nombre\_stream | STANDARD}] FILE fichero**

El contenido del fichero es constante, y en caso de no encontrarse en el directorio en curso habrá que especificar el «path» completo de su localización.

### **Ejemplo 13 [PSTREA13] y [CARTA]. «Mailing» a todos los clientes:**

```
database almacen

define
 variable empresa like clientes.empresa
 variable nombre like clientes.nombre
 variable apellidos like clientes.apellidos
 variable direccion1 like clientes.direccion1
 variable distrito like clientes.distrito
 variable poblacion like clientes.poblacion
 variable descripcion like provincias.descripcion
end define

main begin
 declare cursor mailing for select empresa, nombre, apellidos,
 direccion1, distrito, poblacion, descripcion
 by name
 from clientes, provincias
 where clientes.provincia = provincias.provincia
 control
```



```

on every row
 put stream standard column 40, empresa & NULL, skip
 if nombre is not null then
 put stream standard column 40, nombre & NULL && " " && apellidos & NULL, skip
 put stream standard column 40, direccion1 & NULL, skip
 put stream standard column 40, distrito using "#####" &&
 " - " && poblacion & NULL, skip
 put stream standard column 40, descripcion & NULL, skip 4
 put stream standard file "carta"
 new page stream standard
end control

format stream standard size 66
 margins top 3, bottom 3, left 5, right 80
end format

start output stream standard through printer
foreach mailing begin end
 stop output stream standard
end main

```

Este programa declara un CURSOR que obtendrá todos los clientes a los que se les envía el «mailing». Por cada fila («cliente») se imprime la etiqueta del cliente en curso y por último la carta cuyo cuerpo es fijo para todos los clientes.

## Cierre del STREAM de salida

El cierre de un STREAM abierto previamente se lleva a cabo mediante la instrucción STOP, cuya sintaxis es la siguiente:

**STOP OUTPUT [STREAM {nombre\_stream | STANDARD}]**

A partir del cierre del STREAM, todas aquellas instrucciones PUT que se ejecuten producirán un error de CTL. Sin embargo, si dichas instrucciones emplean el STREAM por defecto (STREAM STANDARD), toda la información que se envíe aparecerá en pantalla.

Si un STREAM no se ha abierto con la cláusula «UNBUFFERED», en el momento que se ejecuta la instrucción STOP es cuando comienza la descarga del «buffer» empleado para el listado, enviando la información hacia el destino (dispositivo) elegido en la apertura.

### Ejemplo 14 [PSTREA14]. Listado de la tabla de formas de pago:

```

database almacen

main begin
 declare cursor listado for select formpago, descripcion from formpagos

 format stream standard size 24
 margins top 0, bottom 0, left 5, right 80
 end format

 start output stream standard to terminal
 put stream standard every row of listado
 stop output stream standard
end main

```

Este ejemplo listará por pantalla todas las filas insertadas en la tabla «formpagos». La apertura del STREAM se realiza «TO TERMINAL», por lo que dicho STREAM deberá ser el STANDARD. Una vez abierto, todas las filas del CURSOR declarado previamente se envían hacia dicho STREAM. Al ejecutar la instrucción STOP se muestra la información en pantalla.

### Escritura en binario

Como hemos visto anteriormente, un STREAM puede abrirse con la cláusula «BINARY», la cual indica que todo lo que se escriba en el STREAM será en formato binario con las longitudes correspondientes a cada tipo de dato del SQL.

**Ejemplo 15 [PSTREA15]. Demostración de la información representada en binario por cada tipo de dato SQL:**

```
define
 variable a char(6) default "Manuel"
 variable b smallint default 999
 variable c integer default 1000
 variable d time default "01:01:10"
 variable e decimal(8,2) default 1112.15
 variable f date default "27/06/1965"
 variable g money(8,2) default 1112.15
end define

main begin
 start output stream standard to "fichero"
 put stream standard column 3, "***** SALIDA NORMAL *****", skip 2
 put stream standard "CHAR(6) : ",a, skip,
 "SMALLINT : ", b, skip,
 "INTEGER : ", c, skip,
 "TIME : ", d, skip,
 "DECIMAL : ", e, skip,
 "DATE : ", f, skip,
 "MONEY : ", g, skip 3
 stop output stream standard
 start output stream standard to "fichero" binary append
 put stream standard column 3, "***** SALIDA EN BINARIO *****", skip 2
 put stream standard "CHAR(6) : ",a, skip,
 "SMALLINT : ", b, skip,
 "INTEGER : ", c, skip,
 "TIME : ", d, skip,
 "DECIMAL : ", e, skip,
 "DATE : ", f, skip,
 "MONEY : ", g, skip
 stop output stream standard
 window from "fichero" as file
 20 lines 40 columns label "F i c h e r o"
 at 1,20
 rm "fichero"
end main
```

Este ejemplo emplea un STREAM hacia un «fichero» para realizar un listado con el STREAM normal y en binario. Dicho STREAM se abre y cierra dos veces. La segunda apertura del STREAM se realiza en forma binaria («BINARY») y con concatenación («APPEND») sobre el mismo fichero generado por el listado anterior. En ambos listados la salida es la misma, si bien en el caso del STREAM binario la información que se graba en el disco no es legible. Esto es debido a que cada tipo de dato emplea su longitud.

### Manejo del «buffer» de escritura

Como ya hemos indicado anteriormente en este capítulo, la salida de la información hacia el destino («dispositivo» o «fichero») no se realiza hasta que no se cierra el STREAM. No obstante, existen dos posibilidades para manejar el «buffer» de escritura:

- Empleando la cláusula «FLUSH» de la instrucción PUT.

- Abriendo el STREAM con la cláusula «UNBUFFERED», lo que indicará que según se vaya enviando información hacia el STREAM, dicha información sea recibida a su vez por el dispositivo o fichero especificado en la apertura.

**Ejemplo 16 [PSTREA16]. Listado del juego de caracteres «GCS#2» de IBM:**

```
define
 variable i smallint
end define

main begin
 pause "STREAM NORMAL" reverse bell
 start output stream standard to terminal
 for i = 1 to 256 begin
 put stream standard character(i), spaces 5
 if i = 100 then begin
 put stream standard flush
 call sleep(5)
 end
 end
 end
 stop output stream standard

 pause "STREAM UNBUFFERED" reverse bell

 start output stream standard to terminal unbuffered
 for i = 1 to 256 begin
 put stream standard character(i), spaces 5
 end
 stop output stream standard
end main
```

Este ejemplo emplea el STREAM STANDARD para un listado con y sin «buffer». El primer listado que aparece en pantalla emplea el «buffer» para su construcción, provocándose un «volcado» del mismo cuando se han calculado los 100 primeros caracteres. Este «volcado» se realiza mediante la cláusula «FLUSH». Para comprobar que realmente se libera el «buffer» se provoca una espera de cinco segundos aproximadamente [«call sleep()»]. Cuando finaliza el listado, es decir, cuando se cierra el STREAM, aparece el total de caracteres que faltan.

Por su parte, en el segundo listado el STREAM se abre con la cláusula «UNBUFFERED», lo que significa que según se va mandando información a dicho STREAM, ésta aparecerá directamente en la pantalla sin necesidad de provocar el vaciado del «buffer».

## Otras instrucciones relacionadas con STREAM OUTPUT

Hasta este momento, las instrucciones comentadas hacían referencia a la apertura del STREAM, formateo de la página de salida, redirección de la información y cierre del STREAM. En resumen, hemos visto las instrucciones «START OUTPUT ...», «FORMAT STREAM ...», «PUT STREAM ... EVERY ROW OF cursor», «PUT STREAM ...», «PUT STREAM ... FRAME nombre\_frame», «PUT STREAM ... FILE nombre\_fichero» y «STOP STREAM ...».

Dentro de este apartado trataremos las restantes instrucciones relacionadas con los STREAMS de salida («OUTPUT»). Dichas instrucciones son «NEED STREAM ... líneas LINES» y «NEW PAGE STREAM ...».

A continuación se comentan las principales características de estas instrucciones. No obstante, para una información más detallada consulte el Manual de Referencia.

### Instrucción NEED

Esta instrucción provoca un salto de página si el número de líneas que falta hasta el final de página es menor que el indicado. Su sintaxis es la siguiente:

**NEED [STREAM {nombre\_stream | STANDARD}] líneas LINES**

El parámetro «líneas» puede ser una expresión.

Esta instrucción se emplea principalmente cuando no se desea que la descripción o el detalle de un mismo tema se separe en dos páginas distintas. Por ejemplo, en un listado de albaranes no tiene sentido que el detalle de un albarán que cabe en una página se divida en dos. En este caso, antes de comenzar la impresión de los albaranes se debería comprobar el número de líneas que se estima por cada albarán para no cortarlo.

### Ejemplo 17 [PSTREA17]. Listado de proveedores agrupados por provincias:

```
database almacen

define
 variable descripcion char(20)
 variable empresa char(25)
 variable direccion1 char(25)

 stream salida
end define

main begin
 declare cursor listado for select empresa, direccion1, descripcion
 by name
 from proveedores,provincias
 where proveedores.provincia = provincias.provincia
 order by descripcion
 control
 before group of descripcion
 need stream salida 10 lines
 put stream salida column 5, "Provincia: ", descripcion, skip 2
 put stream salida column 2, strrepeat("-",70), skip,
 column 5, "E m p r e s a", column 40,
 "D i r e c c i ó n", skip,
 column 2, strrepeat("-",70), skip 2
 on every row
 put stream salida column 5, empresa, column 40,
 direccion1, skip
 after group of descripcion
 put stream salida skip 2, column 35, "T O T A L : ",
 (group count) using "####", skip 3
 end control

 format stream salida size 22
 margins top 0, bottom 0, left 5, right 75
 page trailer size 2
 put stream salida skip 1, column 25, "Página: ",
 current page of salida
 end format
 start output stream salida to "fic" & procid() left
 foreach listado begin end
 stop output stream salida
 window from "fic" & procid() left as file
 24 lines 80 columns label "L i s t a d o"
 at 1,1
 rm "fic" & procid() left
 end main
```

En este ejemplo la instrucción NEED se emplea antes de comenzar el detalle de una provincia nueva. En este caso, si no existen 10 líneas desde donde se encuentra el cursor en la página de salida hasta el final, automáticamente se provocará un salto de página. El listado se realiza hacia un fichero, cuyo nombre será dinámico y estará compuesto por «fic» más un número aleatorio y único en máquina devuelto por la función interna de CTL «procid()». Dicho fichero se mostrará en pantalla mediante la instrucción WINDOW y la cláusula «AS FILE», produciéndose a continuación su respectivo borrado.

Instrucción NEW PAGE

Provoca un salto de página en la salida asociada al STREAM. Su sintaxis es la siguiente:

**NEW PAGE** [STREAM {nombre\_stream | STANDARD}]

El tamaño de la página está fijado por la cláusula «SIZE» de la instrucción FORMAT.

**Ejemplo 18 [PSTREA18].** Este ejemplo es similar al «pstrea17», pero incluyendo un salto de página después de cada provincia:

```
database almacen

define
 variable descripcion char(20)
 variable empresa char(25)
 variable direccion1 char(25)

 stream salida
end define

main begin
 declare cursor listado for select empresa, direccion1, descripcion
 by name
 from proveedores,provincias
 where proveedores.provincia = provincias.provincia
 order by descripcion
 control
 before group of descripcion
 put stream salida column 5, "Provincia: ", descripcion, skip 2
 put stream salida column 2, strrepeat("-",70), skip,
 column 5, "E m p r e s a", column 40,
 "D i r e c c i ó n", skip,
 column 2, strrepeat("-",70), skip 2
 on every row
 put stream salida column 5, empresa, column 40,
 direccion1, skip
 after group of descripcion
 put stream salida skip 2, column 35, "T O T A L : ",
 (group count) using "####"
 new page stream salida
 end control

 format stream salida size 22
 margins top 0, bottom 0, left 5, right 75
 page trailer size 2
 put stream salida skip 1, column 25, "Página: ",
 current page of salida
end format

start output stream salida to "fic" & procid() left
foreach listado begin end
stop output stream salida
window from "fic" & procid() left as file
 24 lines 80 columns label "L i s t a d o"
 at 1,1
rm "fic" & procid() left
end main
```

En la instrucción «AFTER GROUP OF descripcion» se incluye el salto de página incondicional, lo que significa que el detalle de cada provincia comenzará en una página nueva.

### Expresiones relacionadas con STREAMS de salida

CTL dispone de ciertas expresiones válidas para el manejo de los STREAMS de salida («OUTPUT»). Estas expresiones son las siguientes:

**CURRENT {PAGE | LINE} OF {STANDARD | nombre\_stream}**

Estas expresiones evalúan el número de página («PAGE») o línea («LINE») asociado al STREAM. Si se utilizan sin formatear el STREAM el valor que evaluarán será erróneo.

Asimismo, en caso de haber definido un formato al STREAM (instrucción FORMAT), si alguna de las expresiones anteriormente indicadas es la primera a evaluar en cualquiera de las instrucciones no procedurales de la sección CONTROL del CURSOR, el valor devuelto será igualmente erróneo. Para que dicho valor sea coherente con la línea o página en curso, habrá que enviar previamente algún carácter hacia el STREAM: un espacio en blanco, un literal vacío («»), etc. Por ejemplo:

```
database almacen

main begin
 declare cursor listado for select ...
 control
 on every row
 put stream standard "",
 current page of standard, skip
 put stream standard ...
 end control
 format stream standard size 24
 end format
 start output stream standard to terminal
 foreach listado begin end
 stop output stream standard
end main
```

#### Ejemplo 19 [PSTREA19]. Listado sin formatear de los clientes de la base de datos:

```
database almacen
main begin
 declare cursor listado for select empresa, direccion1, telefono
 from clientes
 format stream standard size 24
 page header size 2
 put stream standard
 current page of standard,
 "a pgina"
 end format
 start output stream standard to terminal
 put stream standard every row of listado
 stop output stream standard
end main
```

Este programa emplea la expresión que devuelve el número de página en curso. Dicho número se utiliza en la instrucción no procedural «PAGE HEAER», por lo que, al haberse incluido la instrucción FORMAT, aunque sea la primera expresión a enviar al STREAM no será necesario enviar ningún carácter previo.

### STREAM de entrada

Un STREAM de entrada se emplea a la hora de recoger información desde un dispositivo o un fichero. El manejo de este tipo de STREAMS conlleva las siguientes operaciones:

- Apertura del STREAM.

- Recepción de la información.
- Cierre del STREAM.

En los siguientes apartados siguen se indican las particularidades de cada uno de estos pasos.

## Apertura del STREAM

Como ya hemos indicado, un STREAM de entrada puede recibir información proveniente de diferentes fuentes. Una vez determinada esta fuente, la apertura del STREAM se lleva a cabo con la instrucción **START INPUT**, cuya sintaxis es la siguiente:

```
START INPUT [STREAM {nombre_stream | STANDARD}]
{FROM | THROUGH} fuente [[NO] ECHO] [UNBUFFERED]
[BINARY]
```

La cláusula «ECHO» se emplea por defecto en la apertura de un STREAM de salida, a no ser que se indique lo contrario («NO ECHO»). Dicha cláusula mostrará en pantalla lo que se vaya leyendo sobre las instrucciones **PROMPT FOR** o **INPUT FRAME** (palabra, línea, fichero o conjunto de palabras).

Por último, este tipo de STREAMS acepta también la cláusula «BINARY» para recibir la información en binario. Por defecto, al igual que en la escritura con los STREAMS de salida, la lectura se interpreta como una cadena de caracteres resultado de convertir según los tipos de datos de la variable o variables a cargar. Si se especifica «BINARY» se leerán los dtos en formato binario con las longitudes correspondientes a cada tipo de dato del SQL, que son las que se muestran en el siguiente cuadro:

| VALORES      |                |                   |           |
|--------------|----------------|-------------------|-----------|
| Tipo         | Mínimo         | Máximo            | nº bytes  |
| CHAR(n)      | 1 carácter     | 32.767 caracteres | n         |
| SMALLINT     | -32.767        | +32.767           | 2         |
| INTEGER      | -2.147.483.647 | +2.147.483.647    | 4         |
| TIME         | 00:00:01       | 24:00:00          | 4         |
| DECIMAL(m,n) | $10^{-130}$    | $10^{125}$        | $1 + m/2$ |
| DATE         | 01/01/0001     | 31/12/9999        | 4         |
| MONEY(m,n)   | $10^{-130}$    | $10^{125}$        | $1 + m/2$ |

En la instrucción **START INPUT** hay que especificar la «fuente» de donde provendrá la información a leer, pudiendo ser ésta una expresión. Las posibles fuentes de recepción de información son un fichero o un programa (esta última sólo en el caso de estar utilizando una versión de MultiBase para sistema operativo UNIX).

### a) Entrada de información desde un fichero

Para leer un fichero mediante un STREAM éste tendrá que abrirse de acuerdo a la siguiente sintaxis:

```
START INPUT [STREAM {nombre_stream | STANDARD}] FROM fichero
[[NO] ECHO] [UNBUFFERED] [BINARY]
```

El nombre del fichero podrá ser una expresión. Asimismo, bajo sistema operativo UNIX se podrá incluir la expresión «CLIPPED» en las expresiones alfanuméricas cuando éstas contengan blancos a la derecha.

Si el fichero no se encuentra en el directorio en curso, habrá que indicar en la expresión el «path» absoluto o relativo donde se halle dicho fichero.

En el caso del sistema operativo UNIX/Linux, así como en las versiones para red de área local, habrá que tener en cuenta los permisos de lectura establecidos por el propio sistema operativo sobre los directorios o sobre las diferentes particiones de disco que pudieran existir, respectivamente.

### Ejemplo 20 [PSTREA20] y [PROV\_TEMP.SQL]. Simulación de la instrucción LOAD:

```
database almacén

define
 variable linea char(300)
 variable separador char(1)
 variable i smallint
 variable columnas smallint default 0
 variable concatena char(200)

 frame provin like provincias.*
end frame

 stream salida
end define

main begin
 create temp table prov_temp (provincia smallint not null,
 descripcion char(20),
 prefijo smallint)
 let separador = getenv("DBDELIM")
 if separador is null then let separador = "|"
 unload to "provin.unl" select * from provincias
 start input stream salida from "provin.unl"
 prompt stream salida for linea as line
 end prompt
 display frame box at 3,1 with 5, 80 label "L í n e a e n c u r s o"
 while eof of salida = false begin
 display linea clipped at 5,5
 view
 for i = 1 to length(linea) begin
 if linea[i] = separador then begin
 if linea [i - 1] <> "\"" then begin
 let columnas = columnas + 1
 switch columnas
 case 1
 let provin.provincia = concatena
 case 2
 let provin.descripcion = concatena
 case 3
 let provin.prefijo = concatena
 let columnas = 0
 end
 let concatena = null
 end
 end
 else begin
 if linea[i - 1] = "\"" then
 let concatena = concatena clipped && "\"" && linea[i]
 else let concatena = concatena & linea[i]
 end
 end
 insert into prov_temp (provincia, descripcion, prefijo)
 values ($provin.*)
 let linea = null
 clear at 5,5
 prompt stream salida for linea as line
 end prompt
 end
 stop input stream salida
 clear frame box at 3,1
 window from select * from prov_temp
```



```

10 lines label "Carga de datos"
at 5, 10
end main

```

Para poder compilar correctamente este módulo habrá que crear previamente la tabla temporal «prov\_temp» mediante la ejecución del fichero «prov\_temp.sql» («trans almacen prov\_temp»). Este programa genera un fichero ASCII, de nombre «provin.unl», compuesto por todas las filas de la tabla «provincias». Este fichero será el que se lea mediante el STREAM, grabándose las filas sobre la tabla temporal «prov\_temp». La lectura de este fichero se realiza línea a línea, comprobando carácter a carácter por cada una de ellas dónde se encuentra el separador de campos. Este separador se extrae de la variable de entorno DBDELIM. Cuando se detecta un separador de campos se comprueba si éste es realmente un separador o bien está anulado por un «backslash» («\»). Una vez completadas todas las variables del FRAME, éste se emplea en la instrucción INSERT para grabar una fila sobre la tabla temporal antes citada. Cuando se hayan leído todas las líneas del fichero ASCII «provin.unl» se cerrará el STREAM, mostrándose una ventana con la información grabada en la tabla temporal «prov\_temp».

### b) Recepción desde un programa (sólo en versiones UNIX)

Para recoger la información devuelta por un comando del sistema operativo UNIX habrá que abrir el STREAM de la siguiente forma:

```

START INPUT [STREAM {nombre_stream | STANDARD}]
THROUGH programa [[NO] ECHO] [UNBUFFERED] [BINARY]

```

Esta característica de los STREAMS sólo puede ser utilizada en las versiones de MultiBase sobre sistema operativo UNIX/Linux, ya que Windows carece de la posibilidad de multiproceso.

En este caso, la información de salida devuelta por el comando («programa») se podrá recoger mediante la lectura de dicho STREAM con las instrucciones PROMPT FOR o INPUT FRAME.

#### **Ejemplo 21 [PSTREA21]. Lectura de todos los ficheros existentes en el directorio en curso:**

```

define
 variable lectra char(30)
 variable linea smallint default 2

 stream salida
end define

main begin
 display frame box at 1,15 with 20,50
 label "Ficheros del directorio en curso"
 start input stream salida through "ls"
 prompt stream salida for lectura as line
 end prompt
 while eof of salida = false begin
 let linea = linea + 1
 display lectura at linea,20
 view
 let lectura = null
 if linea = 18 then begin
 let linea = 2
 pause "Pulse [Return] para continuar" reverse
 clear from frame box at 1,15
 end
 prompt stream salida for lectura as line
 end prompt
 end
 stop input stream salida
end main

```

Este programa lee la información devuelta por el comando **ls** del UNIX/Linux. Dicha información consiste en los nombres de todos los ficheros existentes en el directorio en curso. Cada uno de estos nombres se encuentra en una línea diferente. Por lo tanto, la lectura de la información se realiza mediante la instrucción **PROMPT FOR** con la cláusula «AS LINE». Cada uno de los nombres leídos se muestra en una línea diferente y, en caso de llegar a la línea número 18, la presentación se para mediante la instrucción **PAUSE**, volviendo a mostrarse los siguientes nombres de ficheros desde la línea número 3.

## Recepción de información desde un STREAM de entrada

La recepción de información desde un **STREAM** de entrada, es decir, abierto con la cláusula «**INPUT**», se realiza a través de las siguientes instrucciones:

```
PROMPT STREAM nombre_stream FOR...
INPUT STREAM nombre_stream FRAME...
```

Estas instrucciones nos permitirán leer del **STREAM** de entrada de las siguientes formas:

- de palabra en palabra,
- de línea en línea,
- todo un fichero completo,
- de varias palabras en varias palabras.

Los tres primeros tipos de lectura pueden realizarse mediante la instrucción «**PROMPT STREAM** nombre\_stream **FOR** ...». Por su parte, la última forma de lectura de un **STREAM** se consigue mediante la instrucción «**INPUT STREAM** nombre\_stream **FRAME** ...» del objeto **FRAME**.

En las páginas que siguen se indican las características principales de cada una de estas instrucciones de lectura de un **STREAM**.

### Instrucción **PROMPT STREAM**

Esta instrucción permite leer de un **STREAM** abierto para la lectura de un fichero o bien de un comando (programa) del sistema operativo. Una de las formas de leer del **STREAM** con esta instrucción es de palabra en palabra. En este caso la sintaxis de la instrucción sería la siguiente:

```
PROMPT [STREAM {nombre_stream | STANDARD}] FOR variable
 [[NO] CLEAN] [atributos]
END PROMPT
```

**Ejemplo 22 [PSTREA22].** Lectura del directorio en curso de todos los ficheros con extensión «.sct», pudiendo editarlos y compilarlos posteriormente:

```
database almacen

define
 variable fichero char(20)
 stream entrada
end define

main begin
 create temp table ficheros (fichero char(18))
 prepare inst1 from "insert into ficheros values (?)"
 start input stream entrada through "lc *.sct"
 prompt stream entrada for fichero
 end prompt
 while eof of entrada = false begin
 execute inst1 using fichero
 prompt stream entrada for fichero
 end prompt
 end
 stop input stream entrada
```

```

window from select * from ficheros
 15 lines 3 columns label "F i c h e r o s"
 at 3,5
 set fichero
if cancelled = false then begin
 edit file fichero at 1,1 with 18,78
 if cancelled = false then ctlcomp fichero
end
end main

```

Este programa define un STREAM («entrada») para leer los ficheros con extensión «.sct» que se encuentren en el directorio en curso. La lectura se realiza de palabra en palabra mediante la instrucción PROMPT FOR. Esta instrucción leerá desde el primer fichero existente hasta el último, grabando todos ellos en una tabla temporal (denominada «ficheros») que se ha creado al principio del programa. El contenido de esta tabla se muestra en una ventana para poder elegir uno de los ficheros y editarlo. En caso de modificar dicho fichero, acto seguido se provocará su compilación.

No obstante, como ya indicamos anteriormente, esta instrucción permite también la lectura de un STREAM de entrada de una línea completa de un fichero o de la información devuelta por un comando (programa) del sistema operativo UNIX/Linux. En este caso, la sintaxis de la instrucción PROMPT FOR sería:

```

PROMPT [STREAM {nombre_stream | STANDARD}] FOR variable
 [[NO] CLEAN] [atributos] AS LINE
END PROMPT

```

**Ejemplo 23 [PSTREA23]. Descarga de a tabla «provincias» sobre un fichero ASCII con longitud fija en los campos y carga de ese mismo fichero sobre una tabla temporal:**

```

database almacén

define
 variable provincia like provincias.provincia
 variable descripcion like provincias.descripcion
 variable prefijo like provincias.prefijo
 variable linea char(80)
end define

main begin
 create temp table temporal (provincia smallint,
 descripcion char(20),
 prefijo smallint)
 prepare insercion from "insert into temporal (provincia,descripcion," &&
 "prefijo) values (?,?,?)"
 declare cursor listado for select * by name from provincias
 control
 on every row
 put stream standard (provincia + 0) using "<<<<<<",
 descripcion && null,
 (prefijo + 0) using "<<<<<<", skip
 end control
 start output stream standard to "provincia.unl"
 foreach listado begin end
 stop output stream standard
 window from "provincia.unl" as file
 15 lines 60 columns label "Fichero ASCII"
 at 3,10
 start input stream standard from "provincia.unl"
 prompt stream standard for linea as line
 end prompt
 while eof of standard = false begin
 let provincia = substring (linea,1,5)
 let descripcion = substring (linea,6,20)

```

```
 let prefijo = substring (linea,26,5)
 execute insercion using provincia, descripcion, prefijo
 prompt stream standard for linea as line
 end prompt
 end
 stop input stream standard
 window from select * from temporal
 15 lines label "C a r g a"
 at 1,10
end main
```

Este programa emplea un STREAM STANDARD, primero como salida y posteriormente como entrada. En primer lugar, este STREAM se emplea para hacer un listado de la información de la tabla «provincias». El STREAM genera un fichero ASCII en el que la longitud de cada registro es fija, reservando 5 bytes para el código de la provincia, 20 para la descripción y 5 para el prefijo. Una vez creado este fichero, se abre el STREAM como lectura del mismo. La lectura se realiza de línea en línea mediante la instrucción PROMPT FOR y la cláusula «AS LINE». Una vez leída una línea se parte respecto a las longitudes que ocupa cada campo. Realizada dicha partición, las variables receptoras se emplean para la inserción de una fila sobre la tabla temporal creada al comienzo del programa. Cuando se ha leído la última línea del fichero ASCII se cierra el STREAM, apareciendo una ventana con el contenido de la tabla temporal recién grabada.

Por último, la instrucción PROMPT FOR permite también la lectura del contenido de un fichero completo. En este caso su sintaxis sería la siguiente:

```
PROMPT [STREAM {nombre_stream | STANDARD}] FOR variable
[[NO] CLEAN] [atributos] AS FILE
END PROMPT
```

### Ejemplo 24 [PSTREA24]. Lectura de un fichero «.sct» sobre una variable de CTL:

```
database almacen

define
 variable fichero char(10000)
 variable nombre char(80)
 stream entrada
end define

main begin
 path walk from getenv("DBPROC") extent ".sct"
 10 lines 3 columns
 at 5,20
 set nombre
 let nombre = nombre & ".sct"
 start input stream entrada from nombre clipped no echo
 prompt stream entrada for fichero as file
 end prompt
 stop input stream entrada
 display fichero at 1,1
 pause "Pulse [Return] para continuar" reverse
 clear at 1,1
end main
```

Este programa permite al operador elegir el fichero que se desea leer en el STREAM. Para ello se muestra una ventana con los nombres de todos los ficheros con extensión «.sct» que se encuentren en cualquiera de los directorios especificados en la variable de entorno DBPROC. En el momento que se elige uno de ellos, el STREAM se abre como lectura, leyendo el fichero completo mediante la instrucción PROMPT FOR y la cláusula AS FILE. A continuación, mediante la instrucción DISPLAY, muestra en pantalla el contenido de dicho fichero.

Instrucción INPUT STREAM

Esta instrucción pertenece propiamente al objeto FRAME. Su finalidad es permitir la lectura de varias palabras de un fichero o programa especificado en la apertura de un STREAM de entrada. Esta instrucción presenta dos tipos de sintaxis:

(A)

```
INPUT [STREAM {nombre_stream | STANDARD}] FRAME nombre_frame
 [FOR {ADD | UPDATE}] [VARIABLES lista_variables]
 [AT línea, columna]
END INPUT
```

(B)

```
INPUT [STREAM {nombre_stream | STANDARD}] FRAME nombre_frame
 FOR QUERY [VARIABLES lista_variables]
 {{BY NAME | ON lista_columnas} TO variable}
 [AT línea, columna]
END INPUT
```

Como vimos en el capítulo anterior, un FRAME sirve básicamente para recoger información sobre las variables que lo componen, pudiendo mostrarlas en pantalla o enviarlas a un STREAM. Asimismo, otra de las funciones propias de un FRAME es recoger la información devuelta por la lectura de un STREAM. La lectura del STREAM se producirá por tantas palabras como variables compongan el FRAME, o bien por tantas como se indiquen en la cláusula «VARIABLES» de la instrucción «INPUT ...».

Si se emplea la instrucción «INPUT STREAM nombre\_stream FRAME» con la cláusula «FOR QUERY», la información que devuelva se recogerá en la variable indicada en la cláusula «TO» de dicha instrucción. Dicha variable contendrá un conjunto de condiciones enlazadas por el operador lógico «AND». A fin y al cabo, esta misma operación se puede realizar, pero sin STREAM, introduciendo por teclado los valores sobre cada una de las variables que componen el FRAME.

**Ejemplo 25 [PSTREA25] (sólo en UNIX). Lectura de la información devuelta por el comando «ls -l» del UNIX sobre el directorio en curso:**

```
database almacen

define
 frame directorio
 screen
 {
 Permisos: [a]
 Links: [b]
 Propietario: [c]
 Grupo: [d]
 Tamaño.....: [e]
 Mes: [f]
 Día: [g]
 Hora: [h]

 N o m b r e: [i]
 }
end screen

variables
 a = permisos char
 b = links smallint
 c = propietario char
 d = grupo char
 e = ocupa integer
 f = mes char
```

```

 g = dia smallint
 h = hora time format "hh:mm"
 i = nombre char
end variables

layout
 box
 label "Lectura del comando ls -l de UNIX"
 line 5
 column 5
 end layout
end frame

frame totaliza
screen
{
 T o t a l:[a] [b]
}
end screen

variables
 a = literal char
 b = tot inteer
end variables

layout
 box
 label "Primera Línea"
 line 2
 column 5
 end layout
end frame
end define

main begin
 display frame totaliza
 display frame directorio
 start input stream standard through "ls -l"
 input stream standard frame totaliza
 end input
 view
 while eof of standard = false begin
 view
 input stream standard frame directorio
 end input
 end
 stop input stream standard
end main
```

Este programa define dos FRAMES, uno para la lectura de la primera línea devuelta por el comando «ls -l» («total xxxx») y otro para el resto de líneas, es decir, una línea por fichero o directorio. Ambos FRAMES se muestran en pantalla antes de la apertura del STREAM. La instrucción INPUT FRAME recoge cada una de las líneas devueltas por el comando «ls -l» y muestra dicha información en pantalla mediante la instrucción VIEW, que refrescará los valores del FRAME pintado previamente.

## Lectura en binario desde un STREAM

Como hemos visto anteriormente, un STREAM de entrada puede abrirse con la cláusula «BINARY». Esta cláusula indica que todo lo que se lea del STREAM será en formato binario con las longitudes correspondientes a cada tipo de dato de SQL.

**Ejemplo 26 [PSTREA26]. Demostración de la información representada en binario por cada tipo de dato SQL y posterior lectura, también en binario, de cada uno de ellos:**

```

define
 frame tipos
 screen
 {
 CHAR(6): [a]
 SMALLINT: [b]
 INTEGER: [c]
 TIME: [d]
 DECIMAL: [e]
 DATE: [f]
 MONEY: [g]
 }
end screen

variables
 a = a char(6)
 b = b smallint
 c = c integer
 d = d time
 e = e decimal(8,2)
 f = f date
 g = g money(8,2)
end variables

layout
 box
 label "Tipos de Datos"
 line 5
 column 20
end layout
end frame
end define

main begin
 input frame tipos for add
 end input
 if cancelled = true then exit program
 clear frame tipos
 start output stream standard to "fichero" binary
 put stream standard frame tipos
 stop output stream standard
 window from "fichero" as file
 20 lines 40 columns label "F i c h e r o"
 at 1,20
 initialize frame tipos
 display frame tipos
 pause "Después de escribir el STREAM. Pulse [Return] para continuar" reverse
 clear frame tipos
 start input stream standard from "fichero" no echo binary
 input stream standard frame tipos
 end input
 stop input stream standard
 rm "fichero"
 display frame tipos
 pause "Después de leer el STREAM. Pulse [Return] para continuar" reverse
 clear frame tipos
 view
end main

```

Este programa emplea el STREAM STANDARD para escribir en primer lugar la información insertada sobre un FRAME en modo «ADD». Posteriormente, dicho STREAM se empleará para la lectura del fichero grabado en el paso anterior. En este caso, la definición del FRAME incluye varias variables, cada una con un tipo de dato SQL diferente. El operador tendrá que asignar valores sobre las variables del FRAME en la operación realizada por la instrucción INPUT FRAME. En caso de no cancelar los valores introducidos, éstos se escribirán en binario sobre un fichero denominado «fichero».

Una vez finalizada la escritura, dicho fichero se mostrará en una ventana producida por la instrucción WINDOW con la cláusula «AS FILE». En esta ventana lo único que aparecerá legible será el valor introducido sobre la variable de tipo CHAR. A continuación, se vuelve a abrir el STREAM STANDARD para la lectura del «fichero». En este caso, la instrucción INPUT FRAME se empleará para la lectura, cargando los valores introducidos anteriormente sobre cada una de las variables del FRAME. Entre un paso y otro el FRAME se inicializa con todos los valores a nulos para comprobar finalmente el buen funcionamiento de los STREAMS de lectura en binario. Por último, el fichero se borra del directorio en curso mediante la instrucción **rm**, mostrándose los valores leídos en binario sobre la SCREEN del FRAME.

### Cierre del STREAM de entrada

El cierre de un STREAM de entrada abierto previamente mediante la instrucción START se lleva a cabo con la instrucción STOP, cuya sintaxis es la siguiente:

**STOP OUTPUT [STREAM {nombre\_stream | STANDARD}]**

En el momento que un STREAM de entrada (INPUT) se cierra mediante la instrucción STOP, todas las instrucciones de lectura (PROMPT FOR o INPUT FRAME) no emplearán dicho STREAM aunque en su sintaxis se incluya la referencia. En este caso, ambas instrucciones emplearán el teclado para la introducción de datos sobre las variables.

**Ejemplo 27 [PSTREA27] (sólo en UNIX). Lectura en UNIX del directorio en curso:**

```
define
 variable directorio char(80)
end define

main begin
 start input stream standard through "pwd"
 prompt stream standard for directorio
 end prompt
 stop input stream standard
 pause directorio
end main
```

Este programa abre el STREAM STANDARD para recoger la información devuelta por el comando **pwd** del sistema operativo UNIX/Linux. La lectura se produce con la instrucción PROMPT FOR. Como la información a recibir no contiene ningún espacio en blanco entre los diferentes directorios, en este caso no es necesario emplear ninguna cláusula de lectura de línea («AS LINE») ni de fichero («AS FILE»). Por último, después del cierre del STREAM se muestra el nombre del directorio mediante la instrucción PAUSE.

### Expresiones relacionadas con STREAMS de lectura

Los STREAMS abiertos para la lectura de un fichero o de la salida de un programa de sistema operativo UNIX emplean una expresión para detectar si se ha llegado al final del fichero o de la información devuelta por el comando. Esta expresión es la siguiente:

**EOF OF {nombre\_stream | STANDARD}**



**Ejemplo 28 [PSTREAM28]. Grabación de un fichero fuente sobre una tabla de la base de datos:**

```

database almacén

define
 variable línea char(70)
 variable número smallint default 0
 variable fichero char(20)
end define

main begin
 path walk from getenv("DBPROC") extent ".sct"
 10 lines 4 columns
 label "E L E G I R"
 at 1,1
 set fichero
 if cancelled = true then exit program
 let fichero = fichero & ".sct"
 create temp table fuente (número smallint not null, lin char(80))
 primary key (número)
 prepare insercion from "insert into fuente (número,lin) values (?,?)"
 start input stream standard from fichero clipped
 prompt stream standard for línea as line
 end prompt
 while eof of standard = false begin
 let número = número + 1
 execute insercion using número, línea
 let línea = null
 prompt stream standard for línea as line
 end prompt
 end
 stop input stream standard
 window from select lin[1,70] from fuente
 where número > 0
 18 lines label "M ó d u l o"
 at 1,1
end main

```

Este programa pide al operador que elija el fichero de cualquiera de los directorios indicados por la variable de entorno DBPROC. Dicho fichero se grabará sobre una tabla temporal. En caso de no cancelar esta petición del fichero se abrirá el STREAM STANDARD de entrada desde el fichero con extensión «.sct» que se haya elegido. Este fichero se leerá línea a línea por medio de la cláusula «AS LINE» de la instrucción PROMPT FOR. Cada línea leída se grabará en una tabla temporal creada en el propio programa. Una vez alcanzado el final del fichero fuente, se cerrará el STREAM de entrada y se mostrarán las filas grabadas sobre la tabla temporal, es decir, deberá aparecer el texto del fichero fuente leído previamente.

## STREAMS de lectura/escritura

Este tipo de STREAMS es una combinación de las dos anteriores («INPUT» y «OUTPUT»). En este caso, el destino elegido para leer y escribir deberá ser un programa del sistema operativo que permita recibir y enviar información. La sintaxis de la apertura y del cierre de este tipo de STREAMS es la siguiente:

**a) Apertura:**

```

START INPUT-OUTPUT [STREAM {nombre_stream | STANDARD}]
 THROUGH programa_unix [[NO] ECHO] [UNBUFFERED] [BINARY]

```

**b) Cierre:**

```

STOP INPUT-OUTPUT [STREAM {nombre_strea | STANDARD}]

```

Las instrucciones que se podrán emplear en este tipo de STREAMS están formadas por la combinación de las explicadas en los apartados relativos a STREAMS de entrada (INPUT) y de salida (OUTPUT), es decir: PUT, PUT FILE, PUT FRAME, PUT EVERY ROW, PROMPT FOR e INPUT FRAME.

### Comunicación con un servicio remoto

A partir de la versión 2.0«release» 05 de MultiBase, los STREAMS de entrada/salida se pueden emplear también en las versiones de y Windows cuando se comporten como «cliente» en una instalación «cliente-servidor».

Con el fin de mejorar el aprovechamiento de los recursos disponibles en una red de ordenadores los STREAMS disponen de la capacidad de comunicarse con cualquier proceso UNIX remoto; es decir, un CTL ejecutando en una máquina «cliente» (, Windows o UNIX) podrá ponerse en comunicación bidireccional con cualquier proceso residente en cualquier máquina UNIX de la red, siempre y cuando este último proceso esté dado de alta como un servicio del TCP/IP.

Los STREAMS definidos con la cláusula «THROUGH» permiten indicar la dirección de red de la máquina en la que reside el proceso con el que se va a establecer la comunicación.

Para arrancar un proceso en una máquina remota bastará con especificar el nombre de la máquina y el nombre del servicio asociado a dicho proceso:

```
START INPUT-OUTPUT [STREAM {nombre_stream | STANDARD}]
THROUGH "@servidor:servicio"
```

Donde:

|          |                                                                                                           |
|----------|-----------------------------------------------------------------------------------------------------------|
| servidor | Nombre del servidor UNIX donde se encontrará el «servicio» a ejecutar.                                    |
| servicio | Nombre del «servicio» de la máquina «servidor» que se encargará de ejecutar un programa en dicha máquina. |

El «servidor» debe encontrarse en el fichero «hosts» de la máquina cliente.

El «servicio» debe encontrarse en el fichero «services», tanto de la máquina «cliente» como de la máquina «servidor». La línea a incluir en este fichero tiene la siguiente sintaxis:

```
service_name nnn/tcp comentario
```

El número «nnn» debe ser único en el fichero «services» e igual en ambas máquinas («cliente» y «servidor»).

Asimismo, el servicio en la máquina «servidor» debe estar asignado a un comando, que será el que se ejecute cuando se invoque al STREAM. Esta relación servicio-comando se especifica en el fichero «inetd.conf» de la máquina «servidor», la cual deberá tener el sistema operativo UNIX/Linux, encontrándose habitualmente el fichero indicado en el directorio «/etc». La sintaxis de la línea (**una sola**) a incluir en dicho fichero es la siguiente:

```
servicio stream tcp nowait root comando_path argumento argumento ...
```

Por ejemplo, supongamos que nos queremos poner en comunicación con un programa CTL, de nombre «buscaprv», residente en la máquina «unix134». A este servicio le vamos a llamar «ctlserver». Los ficheros de configuración del TCP/IP deberán tener las siguientes entradas:

#### a) Máquina Cliente:

- Fichero «HOSTS»:  
unix134 125.1.1.1
- Fichero «SERVICES»:  
ctlserver 12345/tcp "servidor de ctl"

**b) Máquina Servidor:**

- Fichero «SERVICES»:
 

```
ctlserv 12345/tcp "servidor de ctl"
```
- Fichero «INETD.CONF» (una sola línea):
 

```
ctlserv stream tcp nowait root /usr/ctl/bin/ctl ctl -ef /usr/ctl/etc/conf1.env buscaprv
```

Dado que el CTL necesitará una serie de variables de entorno para su ejecución, éstas se le pasarán en el fichero indicado con el parámetro «-ef» de la línea de comando, indicándole a continuación el nombre del fichero donde se definen dichas variables de entorno. En nuestro ejemplo, este fichero es «/usr/ctl/etc/conf1.env» y su contenido podría ser:

```
TRANSDIR=/usr/ctl
DBPATH=/usr/mbdemo
DBPROC=/usr/mbdemo/progs
```

A continuación se muestra un ejemplo de utilización del CTL como servidor remoto para un programa de mantenimiento de clientes. En este ejemplo, similar al de la aplicación de demostración de MultiBase, se ha supuesto que la tabla «provincias» pertenece a una base de datos remota y el resto de tablas están en la base de datos local.

El nombre de la máquina UNIX remota es «unix», y el nombre del servicio que lanza el CTL remoto es «serverctl». El ejemplo 29 será el programa que ejecuta en local y el 30 el lanzado en remoto.

**Ejemplo 29. Programa de la máquina «cliente»:**

```
database almacén

define
 variable resultado char(20)
 variable seleccionar char(500)
 variable insertar char(500)
 variable err integer
 stream serv
 frame provi
 screen
{
 Codigo Provincia: [a1]
 Descripcion: [a2]
 Prefijo: [a3]
}

end screen

variables
 a1 = provinfram smallint
 a2 = descrip char
 a3 = prefijo smallint
end variables
end frame

form
screen
{
 Codigo Cliente..... : [f1]
 Empresa..... : [f2]
 Apellidos.....: [f3]
 Nombre.....: [f4]
 Direccion1..... : f5]
 Direccion2..... : [f6]
 Poblacion.....: [f7]
 Provincia..... : [f8] [u1]
}
```

```

 Distrito..... : [f9]
 Telefono..... : [f10]
 forma de Pago..... : [a]
 Total Facturado..... : [f12]
 }

end screen

tables
 clientes
end tables

variables
 f1 = clientes.cliente required
 f2 = clientes.empresa
 f3 = clientes.apellidos
 f4 = clientes.nombre
 f5 = clientes.direccion1
 f6 = clientes.direccion2
 f7 = clientes.poblacion
 f8 = clientes.provincia
 u1 = vartable.descripcion char
 f9 = clientes.districto
 f10 = clientes.telefono
 a = clientes.formpago
 f12 = clientes.total_factura
end variables

menu mb 'Clientes'
 option a1 'Seleccionar'
 query
 option a2 'Modificar'
 modify
 option a3 'Borrar'
 remove
 option a4 'Agregar'
 add
end menu

editing
 after provincia
 let seleccionar = "select descripcion from " &&
 "provincias where provincia ="
 && provincia left
 let descripcion = exec_remote (seleccionar, "L")
 if descripcion is null or descripcion = ""
 then begin
 if yes ("No existe. Desea darla de alta","Y") = true
 then begin
 let provi.provinfram = provincia
 let provi.descrip = NULL
 let provi.prefijo = NULL
 input frame provi for update at 5,5 end input
 let insertar = "insert into provincias " &&
 "(provincia, descripcion, prefijo) "
 && "values ("&& provi.provinfram && ","&&
 && provi.descrip
 && ","&& provi.prefijo &&")"
 clear frame provi
 let err = exec_remote(insertar, "I")
 let descripcion = exec_remote(seleccionar, "L")
 end
 else begin
 let provincia = null
 end
 end
 end
 end
end editing

```

```

 next field "provincia"
 end
 end
 end editing
end form
end define

main begin
 call open_serv()
 menu form
 call close_serv()
end main

function exec_remote(stmt, oper)
begin
 put stream serv oper, skip, flush
 put stream serv stmt clipped, skip, flush
 let resultado = NULL
 prompt stream serv for resultado as line
 en prompt
 return resultado
end function

function pen_serv()
begin
 start input-output stream serv
 through "@UNIX/Linux:serverctl"
end function

function close_serv()
begin
 stop input-output stream serv
end function

```

**Ejemplo 30. Programa de la máquina «servidor»:**

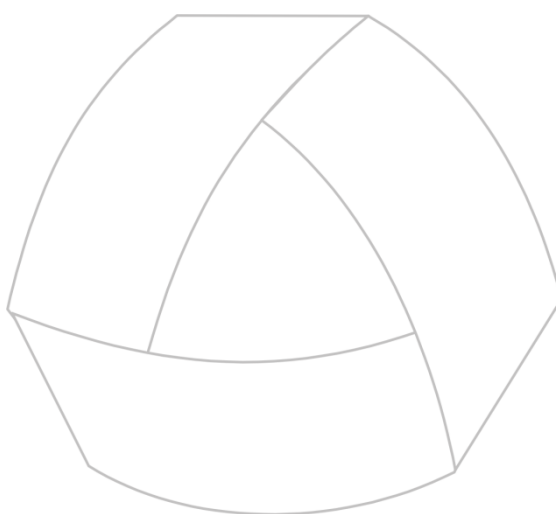
```

define
 variable directorio char(60)
 variable operacion char(1)
 variable frase char(500)
 variable descripcion char(20)
end define

main begin
 database almacen
 forever begin
 let operacion = NULL
 let frase = NULL
 prompt for operacion as line end prompt
 if eof of standard = true then
 break
 prompt for frase as line end prompt
 if frase is null then begin
 put stream standard skip, flush
 continue
 end
 prepare instr from frase clipped
 switch operacion
 case "L"
 declare cursor lectura for instr
 let descripcion = ""
 open lectura
 fetch lectura into descripcion
 close lectura
 end
 end
 end
end

```

```
 put stream standard descripcion clipped, skip, flush
 case "I"
 execute instr
 put stream standard errno, skip, flush
 end
end
end main
```



MultiBase  
MANUAL DEL PROGRAMADOR

## **CAPÍTULO 14. Tipos de menús CTL**

### Menús CTL

---

El menú es la forma habitual de presentar las distintas operaciones que puede realizar el usuario en el manejo de una aplicación. CTL dispone de dos estructuras para la definición de tres tipos de menús: la estructura «PULLDOWN» y la estructura «MENU». Mediante estas estructuras se podrán construir los siguientes tipos de menús CTL:

- Menús «Lotus» o de «Iconos» (Windows): Estructura «MENU».
- Menús «Pop-Up»: Estructura «MENU».
- Menús «Pulldown»: Estructura «PULLDOWN».

La estructura «PULLDOWN» permite englobar varias estructuras «MENU», y su presentación se realiza en una ventana en la parte superior de la pantalla donde se muestran horizontalmente las distintas opciones (también denominadas «persianas»).

En UNIX/Linux, los menús de tipo «pulldown» son cíclicos, es decir, si el número de opciones («persianas») que contiene dicho menú no cabe en el ancho de la pantalla, las opciones restantes aparecerán en pantallas sucesivas. Por contra, en el caso específico de las versiones para Windows, todas las opciones se mostrarán en la misma pantalla en tantas líneas como sea necesario.

Cada persiana de un menú «pulldown» lleva asociada una estructura «MENU» con sus propias opciones, la cual aparecerá al activar la correspondiente «persiana» del «pulldown». Dependiendo del entorno operativo en el que nos encontremos, la forma de activar una «persiana» de un menú «pulldown» será la siguiente:

#### a) En UNIX/Linux:

- En primer lugar, habrá que posicionarse sobre la «persiana» que se desea activar, para lo cual podremos utilizar las teclas de movimiento del cursor hacia derecha e izquierda.
- Una vez posicionados en la «persiana» elegida, la forma de desplegarla será mediante [RETURN] o bien las teclas asociadas a las acciones <fup>, <fdown> o <faccept>, que por lo general se corresponderán con las teclas de movimiento del cursor flecha arriba y flecha abajo y la tecla de función [F1], respectivamente.
- No obstante, se podrá desplegar directamente una «persiana» pulsando la tecla correspondiente a la letra que aparece en distinta intensidad de vídeo



#### b) En Windows:

En las versiones de MultiBase para este entorno, la forma de activar un menú «pulldown» podrá realizarse mediante el teclado, pulsando la tecla [ALT] y a continuación las teclas de movimiento del cursor o bien la tecla correspondiente a la letra que aparezca subrayada, o bien mediante el empleo del ratón.

La estructura «MENU» asociada a cada «persiana» podrá mostrar, después de elegir una de las opciones incluidas en dicha «persiana», otro menú de tipo «Pop-up» (vertical) en cualquier posición de la pantalla, o bien un menú de tipo «Lotus» o de «iconos» (horizontal).

Los menús de tipo «Lotus» en UNIX/Linux se muestran al pie de la pantalla, o bien en la línea indicada en la opción «MENU LINE» de la instrucción OPTIONS o en la cláusula «LINE» de la definición del MENU. En el caso específico del Windows, estos menús (denominados de «iconos») se representan como si se tratase de un menú de «persianas», teniendo cada una de sus opciones un icono asociado para su ejecución.

Los menús «Lotus» o de iconos podrán llamar a una función definida, a otro módulo o simplemente ejecutar instrucciones del CTL.

En los siguientes apartados se indican las características de cada uno de estos tipos de menús, así como su estructura.



## Menús «Lotus» o de «Iconos» (Windows)

Los menús de tipo «Lotus» en y UNIX (horizontales) se representan por defecto en la línea 23 ó 24 de la pantalla, dependiendo de que el terminal utilizado emplee 24 ó 25 líneas, respectivamente. No obstante, esta representación puede modificarse mediante las cláusulas «MENU LINE» de la instrucción OPTIONS, «LINE» de la definición del MENU y «AT» de la instrucción MENU. Su aspecto es el siguiente:

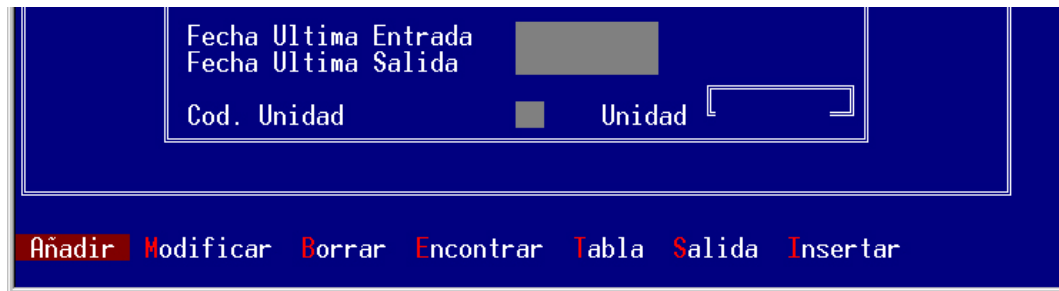


FIGURA 1. Menú «Lotus» en UNIX/Linux.



Como ya hemos comentado anteriormente, en las versiones de MultiBase para Windows estos menús se denominan de «Iconos», caracterizándose por llevar asociado un icono a cada una de sus opciones. Su aspecto es el que se muestra en la siguiente figura.

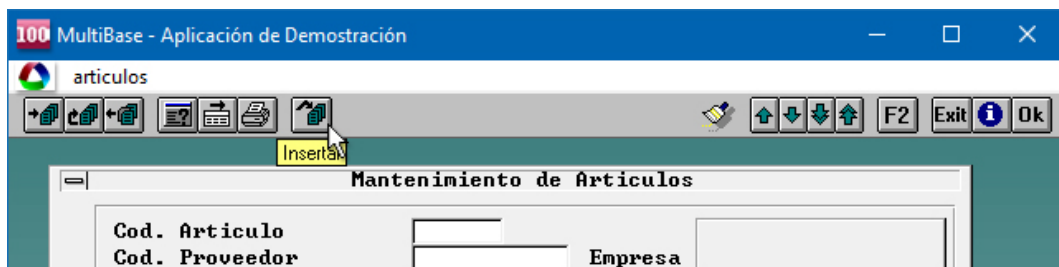


FIGURA 2. Menú de «Iconos» en MultiBase para Windows.

La definición de este tipo de menús tiene la siguiente estructura:

```
MENU nombre_menú ["Etiqueta"] [[NO] CLEAN] [LINE línea]
 [COLUMN columna] [MESSAGE número]
 [HELP número_help] [NO WAIT]
 OPTION [nombre_opción] ["Etiqueta"]
 [KEY "carácter"] [COMMENTS "literal"] [HELP número_help]
 [CLEAN]
 ...
 ...
 OPTION [nombre_opción] ["Etiqueta"]
 [KEY "carácter"] [COMMENTS "literal"] [HELP número_help]
 [CLEAN]
 ...
 ...
 [OPTION ...
 ...]
END MENU
```

Esta estructura de menús puede situarse como sección MENU de un FORM o un FRAME o bien como un menú general del módulo. En este último caso, la estructura MENU se incluirá después de la sección MAIN, si se trata de un módulo principal, o después de la sección LOCAL, si existe, en el caso de que se trate de un módulo librería. Para poder ejecutar un menú en un módulo de librería desde el módulo principal o desde cualquier otro que componga el programa, será necesario realizar una llamada a una función local a dicho módulo librería donde se haya definido el menú. Por ejemplo:

```
function llamada() begin
 menu m1
end function

menu m1 "literal"
 option "Primera opción"
 ...
 option "Segunda opción"
 ...
end menu
```

La ejecución del menú «m1» desde cualquier módulo que componga el programa se realizará mediante la siguiente instrucción:

```
call llamada()
```

Los identificadores «nombre\_menú» y «nombre\_opción» son los nombres simbólicos con los que CTL permite acceder a estos elementos mediante las instrucciones MENU y NEXT OPTION, respectivamente. La sintaxis de esta instrucción MENU es la siguiente:

```
MENU {FORM | FRAME nombre_frame | nombre_menú}
 [DISABLE lista_opciones] [AT línea, columna]
```

Como puede comprobarse en esta sintaxis, los objetos FORM y FRAME pueden emplear también el objeto MENU para realizar todas las operaciones posibles sobre ellos (la sección MENU del FORM y el FRAME se ha explicado en los capítulos correspondientes a estos objetos dentro de este mismo volumen).

La cláusula «AT» de esta instrucción MENU tiene preferencia ante las cláusulas «MENU LINE» de la instrucción OPTIONS y «LINE» de la estructura del MENU.

Un objeto MENU no puede definir submenús en su estructura. Si alguna opción de un menú precisa un submenú, es decir, un segundo nivel, éste se definirá aparte como otra estructura MENU, invocándose con la instrucción de ejecución de un menú: «MENU nombre\_menú» si el menú está definido en el módulo local, o mediante la llamada a una función si se encuentra en un módulo librería.

La ejecución de cualquier menú «MENU» finaliza cuando el usuario pulsa la tecla asignada a la acción <fquit>, cuando se ejecuta la instrucción EXIT MENU, o bien al seleccionar una opción si el MENU ha sido definido con la cláusula «NO WAIT».

A partir de la versión 2.0 «release» 05 de MultiBase, y por lo que respecta al caso del Windows, al situar el puntero del ratón sobre cualquiera de los iconos de un menú de «Iconos» aparecerá un pequeño cartel con un breve comentario que explica la acción que ejecutará dicho icono. Este cartel desaparece automáticamente cuando se separa el puntero del ratón del icono en cuestión. La presentación de estos carteles se consigue mediante la cláusula «COMMENTS» de cada opción del menú.

Las «Etiquetas» del menú y de las opciones pueden extraerse de un fichero de ayuda. Para ello habrá que emplear la cláusula «MESSAGE» y seleccionar el fichero de ayuda mediante la cláusula «HELP FILE» de la instrucción OPTIONS. La forma de indicar un número de mensaje en el fichero de ayuda se explica en el capítulo correspondiente a «Ficheros de Ayuda» dentro de este mismo manual.

NOTA: La función interna «yes()» genera un menú tipo «Lotus» con dos opciones: «SÍ» y «NO». Este menú finalizará en el momento que el operador seleccione una de ellas (cláusula «NO WAIT» implícita).

**Ejemplo 1 [PMENU1]. Listado de la tabla «provincias» con posibilidad de elegir la salida hacia pantalla, impresora o fichero mediante un menú tipo «Lotus»:**

```
database almacen

define
 variable sz smallint
 variable salida char(1) upshift
```

```

end define

main begin
 menu m1
 if cancelled = true then exit program
 declare cursor listado for select * from provincias
 format stream standard size sz
 margins top 0, bottom 0, left 5, right 75
 end format
 switch salida
 case "P"
 start output stream standard to terminal
 case "I"
 start output stream standard through printer
 case "F"
 start output stream standard to "fic" && procid() left
 end
 put stream standard every row of listado
 stop output stream standard
 if salida = "F" then
 window from "fic" && procid() left as file
 15 lines 80 columns label "L i s t a d o"
 at 1,1
 end main

 menu m1 "E l e g i r" line 3 column 2 no wait
 option "Pantalla" key "P" comments "Listado hacia Pantalla"
 let salida = "P"
 let sz = 23
 option "Impresora" key "I" comments "Listado hacia Impresora"
 let salida = "I"
 let sz = 66
 option "Fichero" key "F" comments "Listado hacia Fichero"
 let salida = "F"
 let sz = 23
 end menu

```

En primer lugar, este programa fija el menú de tipo «Lotus» en la línea 3, columna 2 de la pantalla (esto sólo tendrá sentido en y UNIX). Dependiendo de la opción elegida, el listado se enviará a la pantalla, a la impresora o a un fichero («START»). Al seleccionar una opción el menú desaparece debido a la cláusula «NO WAIT» indicada en su definición.

## Menús «Pop-Up»

Los menús de tipo «Pop-up» muestran sus opciones verticalmente (ver Figura 3). Su estructura es idéntica a la de los menús «Lotus» o de «Iconos», pero incluyendo la cláusula «DOWN». Por lo tanto, todo lo comentado en el apartado anterior para los menús «Lotus» es válido también para los menús «Pop-up». La sintaxis de este tipo de menús tiene la siguiente estructura:

```

MENU nombre_menú ["Etiqueta"] DOWN [SKIP líneas] [[NO] CLEAN]
 [LINE línea] [COLUMN columna] [MESSAGE número]
 [HELP número_help] [NO WAIT]
 OPTION [nombre_opción] ["Etiqueta"]
 [KEY "carácter"] [COMMENTS "literal"] [HELP número_help]
 [CLEAN]
 ...
 ...
 OPTION [nombre_opción] ["Etiqueta"]
 [KEY "carácter"] [COMMENTS "literal"] [HELP número_help]
 [CLEAN]

```

```

...
...
[OPTION ...
...]
END MENU

```

Este tipo de menús admite también la cláusula «SKIP» para indicar el número de líneas que se dejarán entre cada una de sus opciones.

En los menús «Pop-up» hay que tener en cuenta que las teclas asignadas a las acciones <fup> y <fdown> están siendo utilizadas por el propio menú. Por ejemplo, en el caso de emplear un menú «Pop-up» para el mantenimiento de una tabla, las teclas que se emplean para recorrer la lista en curso del FORM son las asignadas a las acciones <fdown>, <fup>, <ftop> y <fbottom>. Como podemos comprobar, aquí también se emplean las acciones <fup> y <fdown>, por lo que al existir conflicto entre ambos objetos la lista en curso del FORM sólo podría mostrar la primera y última fila de dicha lista. Para poder manejar esta lista en curso habría que emplear un bucle en el que se realizase una petición de tecla mediante la instrucción «READ KEY THROUGH FORM».

| Tablas           |
|------------------|
| Definición       |
| Renombrar        |
| Primarias        |
| Claves           |
| Indíces          |
| Permisos         |
| Chequear/Reparar |
| Borrar           |
| Documentar       |

FIGURA 3. Aspecto de un menú «Pop-up».

**Ejemplo 2 [PMENU2]. Este ejemplo es idéntico al «pmenu1» pero empleando un menú de tipo «Pop-up»:**

```

database almacen

define
 variable sz smallint
 variable salida char(1) upshift
end define

main begin
 menu m1
 if cancelled = true then exit program
 declare cursor listado for select * from provincias
 format stream standard size sz
 margins top 0, bottom 0, left 5, right 75
 end format
 switch salida
 case "P"
 start output stream standard to terminal
 case "I"
 start output stream standard through printer
 case "F"
 start output stream standard to "fic" && procid() left
 end
 put stream standard every row of listado
 stop output stream standard
 if salida = "F" then
 window from "fic" && procid() left as file
 15 lines 80 columns label "L i s t a d o"
 at 1,1
 end main

 menu m1 "E l e g i r" down skip 2 line 3 column 20 no wait
 option "Pantalla" key "P" comments "Listado hacia Pantalla"
 let salida = "P"
 let sz = 23
 option "Impresora" key "I" comments "Listado hacia Impresora"
 let salida = "I"
 let sz = 66
 option "Fichero" key "F" comments "Listado hacia Fichero"
 let salida = "F"

```

```

 let sz = 23
 end menu

```

A diferencia del ejemplo «pmenu1», en este caso se ha empleado un menú tipo «Pop-up». En dicho menú se ha indicado una línea en blanco entre cada una de sus opciones (cláusula «SKIP 2»). La última diferencia radica en el posicionamiento del menú en la pantalla, que en este caso se realiza en la columna 20.

## Menús «Pulldown»

La estructura de un menú de tipo «Pulldown» permite englobar varias estructuras MENU como las expuestas en los menús «Lotus» o de «Iconos». A partir de estas estructuras de menús (primer nivel), se podrán llamar a otros menús —que podrán ser «Lotus», «Pop-Up» o también «Pulldown»—, funciones, programas, FORMS, FRAMES, etc. La definición de un menú de tipo «Pulldown» tiene lugar después de la sección MAIN en un módulo principal o después de la sección LOCAL en un módulo librería. Su estructura es la siguiente:

```

PULLDOWN nombre [ACCESORIES]

MENU nombre_menu ["Etiqueta"] [[NO] CLEAN] [LINE línea]
 [COLUMN columna] [MESSAGE número]
 [HELP número_help] [NO WAIT]
OPTION [nombre_opción] ["Etiqueta"]
 [KEY "carácter"] [COMMENTS "literal"] [HELP número_help]
 [CLEAN]
...
...
OPTION [nombre_opción] ["Etiqueta"]
 [COMMENTS "literal"] [HELP número_help]
 [CLEAN] [KEY "carácter"]
...
...
[OPTION ...
...]
END MENU

MENU nombre_menu ["Etiqueta"] [[NO] CLEAN] [LINE línea]
 [COLUMN columna] [MESSAGE número]
 [HELP número_help] [NO WAIT]
OPTION [nombre_opción] ["Etiqueta"]
 [KEY "carácter"] [COMMENTS "literal"] [HELP número_help]
 [CLEAN]
...
...
OPTION [nombre_opción] ["Etiqueta"]
 [KEY "carácter"] [COMMENTS "literal"] [HELP número_help]
 [CLEAN]
...
...
[OPTION ...
...]
END MENU

[MENU nombre_menu ...
...
END MENU]

END PULLDOWN

```

La cláusula «ACCESORIES» implica la aparición de una primera «persiana» en el menú «Pulldown», cuyo nombre es «MB». Esta persiana contiene una serie de opciones para el manejo de las distintas aplicaciones de ofimática incluidas en las versiones de MultiBase para UNIX y . En el caso específico de las versiones para Win-

dows, dichas opciones provocan la llamada a diversas aplicaciones propias de este entorno («Calendario», «Calculadora», «Bloc de Notas», etc.). Asimismo, y bajo cualquier entorno operativo, esta persiana incluye también opciones para la definición de colores que se emplearán en MultiBase, selección de la impresora, etc.

Las opciones que componen la persiana «MB» se extraen de un fichero ASCII propio de CTL, de nombre «sysmenu». Este fichero se encuentra en el subdirectorio «msg» de MultiBase, y en él se indicarán el nombre de cada opción y el correspondiente comando que se deberá ejecutar al seleccionarla.

En las versiones de MultiBase para Windows la persiana «MB» estará siempre presente en cualquier menú «Pull-down», con independencia de que se haya especificado o no la cláusula «ACCESORIES» en la definición del menú.

La forma de ejecutar un menú de tipo «Pull-down» es mediante la instrucción PULLDOWN, cuya sintaxis es la siguiente:

**PULLDOWN** nombre\_pull-down



El aspecto de un menú «Pull-down» en Windows es el que se muestra en la siguiente figura. Como puede comprobarse, este tipo de menús en dicho entorno llevan asociado un menú de «Iconos», lo que permite ejecutar muchas de las opciones del «Pull-down» a través del ratón.

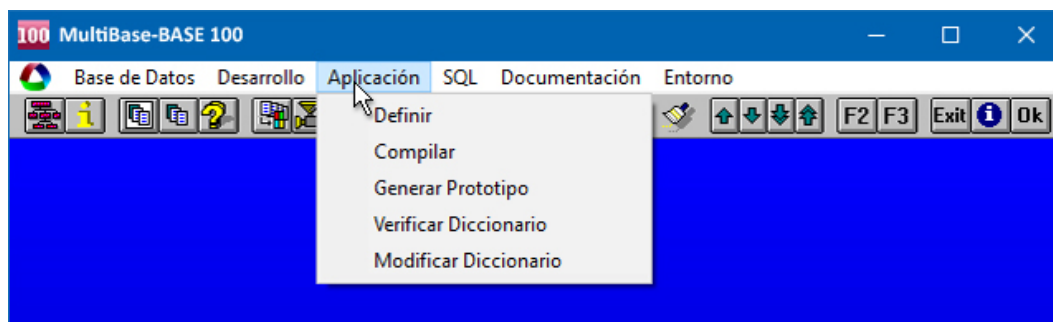


FIGURA 4. Menú «Pull-down» de MultiBase.

En el caso de que un menú que compone la estructura de un «Pull-down» precise llamar a un submenú («Lotus» o «Pop-up»), éste tendrá que especificarse fuera de la estructura del «Pull-down», es decir, en el lugar donde se vayan a ubicar dichos menús. Asimismo, en el caso de que una opción de un menú que compone la estructura de un «Pull-down» precise la llamada a otro «Pull-down», éste tendrá que especificarse fuera de la sintaxis del menú en curso, es decir, en el lugar donde se declaran los «pull-downs», menús y funciones, tanto si es un módulo principal como librería.

En caso de definir un menú «Pull-down» en un módulo librería, éste sólo podrá ejecutarse desde una función local a dicho módulo. Por lo tanto, para poder ejecutar dicho «Pull-down» habrá que ejecutar previamente la función que contiene su llamada. Por ejemplo:

```
function llamada() begin
 pulldown nombre
end function

pulldown nombre
 menu m1 "Entrada de datos"
 option "Clientes"
 ctl "clientes"
 option "Articulos"
 ctl "articulos"
 end menu

 menu m2 "Listados"
 option "Albaranes"
 ctl "l_albaran"
```

```

 option "Clientes"
 ctl "rp_cliente"
 end menu
end pulldown

```

En este caso para ejecutar dicho «Pulldown» desde cualquier módulo que componga el programa se tendrá que hacer llamando a la función «llamada()»:

```
call llamada()
```

Este tipo de menús puede generarse mediante la opción «Generar Fuente» de la persiana «Módulos/Generación» (opción «Menús»). El módulo que se genera dispondrá de una persiana por cada tipo de programa («Entrada de Datos», «Listado» y «Consultas»).

### Ejemplo 3 [PMENU3]. Menú «Pulldown» final de la aplicación «almacen»:

```

database almacen

define
 frame fr_help
 screen
 {
 ! [x
 % Ayuda % Fecha % Hora % Accesorios
 [a1] [x1] [x2] [a2] *
 *
 @
 }
 end screen

 variables
 a1 = klabel1 char
 a2 = klabel2 char
 x = titu2 char reverse
 x1 = di date format "ddd dd mmm yyyy"
 x2 = ho time
 end variables

 control
 after display of fr_help
 call valores_fr()
 end control

 layout
 line 17
 no underline
 end layout
 end frame
end define

global
 control
 on beginning
 call valores_fr()
 display frame fr_help
 on ending
 clear frame fr_help
 on action "fuser2"
 menu accesorios at 4,58
 end control
 options
 help file 'almacen'
 end

```

```
 end options
 end global

 main begin
 pulldown almacen
 while yes('Finaliza','n')=false
 pulldown almacen
 end main

 pulldown almacen accessories
 menu D1 'Entrada Datos '
 option 'Articulos'
 clear frame fr_help
 ctl 'articulos'
 display frame fr_help
 option 'Clientes'
 clear frame fr_help
 ctl 'clientes'
 display frame fr_help
 option 'Editalbar'
 clear frame fr_help
 ctl 'editalbar'
 display frame fr_help
 option 'Formpago'
 clear frame fr_help
 ctl 'formpago'
 display frame fr_help
 option 'Proveedor'
 clear frame fr_help
 ctl 'proveedor'
 display frame fr_help
 option 'Provincias'
 clear frame fr_help
 ctl 'provincias'
 display frame fr_help
 option 'Unidades'
 clear frame fr_help
 ctl 'unidades'
 display frame fr_help
 end menu

 menu R1 'Listados '
 option 'L_albaran'
 clear frame fr_help
 ctl 'l_albaran'
 display frame fr_help
 option 'Precios'
 clear frame fr_help
 ctl 'precios'
 display frame fr_help
 option 'Rp_cliente'
 clear frame fr_help
 ctl 'rp_cliente'
 display frame fr_help
 option 'Rp_provee'
 clear frame fr_help
 ctl 'rp_provee'
 display frame fr_help
 end menu

 menu O1 'Consultas '
 option 'Actuprecio'
 clear frame fr_help
```



```

 ctl 'actuprecio'
 display frame fr_help
 option 'Artlineas'
 clear frame fr_help
 ctl 'artlineas'
 display frame fr_help
 option 'Facturar'
 clear frame fr_help
 ctl 'facturar'
 display frame fr_help
 option 'Modprecio'
 clear frame fr_help
 ctl 'modprecio'
 display frame fr_help
 option 'Statcli'
 clear frame fr_help
 ctl 'statcli'
 display frame fr_help
 option 'Statgen'
 clear frame fr_help
 ctl 'statgen'
 display frame fr_help
 option 'Triple'
 clear frame fr_help
 ctl 'triple'
 display frame fr_help
 option 'Vercliente'
 clear frame fr_help
 ctl 'vercliente'
 display frame fr_help
 end menu
end pulldown

function valores_fr() begin
 let titu2 = " Menu General de la Aplicacion"
 let klabel1 = keylabel("fhhelp")
 let klabel2 = keylabel("fuser2")
 let di = today
 let ho = now
end function

menu accesorios no wait down
 option "Calculadora"
 if getctlvalue("o.s.")="WINDOWS"
 then run "calc"
 else run "tcalcu"
 end if
 option "Calendario" key "I"
 if getctlvalue("o.s.")="WINDOWS"
 then run "calendar"
 else run "tcal", "-b"
 end if
 option "Block de Notas"
 if getctlvalue("o.s.")="WINDOWS"
 then run "notepad"
 else run "tnotes", "-b"
 end if
 option "Hoja Electronica"
 if getctlvalue("o.s.")="WINDOWS"
 then run "excel"
 else run "tcalc", "-s",3,5,20,70
 end if
 end menu

```

Este menú «Pulldown» se ha generado por defecto a través de la siguiente secuencia de opciones: «Desarrollo» — «Módulos/Generación» — «Menús» — «Generar Fuente». Como puede comprobarse, cada vez que se programa una opción del menú principal se limpia el FRAME que contiene valores de teclas, fecha y hora, eje-

cutándose un programa CTL. Cuando este programa CTL finaliza se volverá a repintar la información que contiene dicho FRAME. El menú «Accesorios» indicado al final del módulo se ejecutará cuando el operador pulse la tecla asignada a la acción <fuser2>. Esta acción se recoge en la sección GLOBAL del módulo, y en ese caso se mostrará el menú «Pop-up» de «Accesorios» en la línea 4, columna 58 de la pantalla. En el momento que se ejecuta una de sus opciones, este menú desaparece. Este efecto lo provoca la cláusula «NO WAIT» indicada en su definición.

## Instrucciones relacionadas

---

Los menús explicado en este capítulo utilizan ciertas instrucciones del CTL específicas para su funcionamiento. Estas instrucciones son las siguientes:

- EXIT MENU
- EXIT PULLDOWN
- MENU nombre\_menu
- NEXT OPTION
- PULLDOWN nombre\_menu\_pulldown

En los siguientes epígrafes se comentan las características más significativas de cada una de estas instrucciones.

### Instrucción EXIT MENU

Esta instrucción fuerza la salida de un menú «Lotus» («Iconos» en Windows) o «Pop-Up». Su sintaxis es la siguiente:

#### EXIT MENU

La ejecución de esta instrucción se encuentra implícita en un menú cuando éste ha sido definido con la cláusula «NO WAIT». La ejecución de esta instrucción es equivalente a pulsar la tecla asignada a la acción <fquit>.

**Ejemplo 5 [PMENU5]. Este ejemplo es similar al «pmenu4», pero añadiendo una opción más para provocar la salida incondicional del menú:**

```
database almacen

define
 form
 tables
 provincias
 end tables

 menu manteni "Mantenimiento"
 option "Añadir"
 add
 option "Modificar"
 if numrecs <> 0 then modify
 else begin
 message "Previamente hay que consultar filas" reverse bell
 next option a4
 end
 option "Borrar"
 if numrecs <> 0 then remove
 else begin
 message "Previamente hay que consultar filas" reverse bell
 next option a4
 end
 option a4 "Consultar"
 query
 option "S a l i r"
```

```

 exit menu
 end menu

 layout
 box
 label "Mantenimiento de Provincias"
 end layout
 end form
end define

main begin
 menu form
end main

```

En este ejemplo, similar al «pmenu4», se ha incluido una opción para salir del menú sin necesidad de pulsar la tecla asignada a la acción <fquit>.

## Instrucción EXIT PULLDOWN

Esta instrucción fuerza la salida de un menú de tipo «Pull-down». Su sintaxis es la siguiente:

### EXIT PULLDOWN

La ejecución de esta instrucción es equivalente a pulsar la tecla asignada a la acción <fquit> en el «Pull-down».

**Ejemplo 6 [PMENU6]. Menú «Pull-down» del que es posible salir sin necesidad de pulsar la tecla asignada a la acción <fquit>:**

```

main begin
 pulldown almacen
end main

pulldown almacen
 menu m1 "Entrada de Datos "
 option "Clientes"
 ctl "clientes"
 option "Articulos"
 ctl "articulos"
 option "S a l i r"
 exit pulldown
 end menu

 menu m2 "Listados"
 option "Albaranes"
 ctl "l_albaran"
 option "Clientes"
 ctl "rp_cliente"
 option "S a l i r"
 exit pulldown
 end menu
end pulldown

```

En este menú «Pull-down» se ha incluido una opción «Salir» en cada una de las persianas que lo componen. Dicha opción ejecutará al activarse la instrucción EXIT PULLDOWN.

## Instrucción MENU

Esta instrucción mostrará en pantalla y activará un menú de tipo «Lotus» (o de «Iconos») o un menú «Pop-up» en caso de haber incluido la cláusula «DOWN». Su sintaxis es la siguiente:

```

MENU {FORM | FRAME nombre_frame | nombre_menú}
 [DISABLE lista_opciones] [AT línea, columna]

```

La sección MENU de un FORM o un FRAME se ha explicado en los capítulos correspondientes a estos dos objetos dentro de este mismo volumen. La forma de invocar a esta sección, dependiendo del objeto en que nos encontremos, es mediante las instrucciones «MENU FORM» y «MENU FRAME nombre\_frame», respectivamente.

La instrucción MENU se utiliza asimismo para ejecutar cualquier tipo de menú definido en un módulo. En este caso, la forma de invocar al menú será por medio de su nombre («nombre\_menú»). Esto es debido a que en CTL los menús operan como funciones especializadas.

### Instrucción NEXT OPTION

Esta instrucción indicará qué opción de un menú «Lotus» (de iconos en Windows) o «Pop-up» será la que se deba ejecutar en un determinado momento. Al ejecutar esta instrucción se mostrará en vídeo inverso su correspondiente opción. Su sintaxis es la siguiente:

**NEXT OPTION** nombre\_opción

La opción «nombre\_opción» que se desee invocar deberá tener asignado un nombre simbólico para permitir su activación.

**Ejemplo 4 [PMENU4]. Mantenimiento de la tabla «unidades» controlando que antes de modificar o borrar una fila haya que consultar previamente:**

```
database almacen

define
 form
 tables
 provincias
 end tables

 menu manteni "Mantenimiento"
 option "Añadir"
 add
 option "Modificar"
 if numrecs <> 0 then modify
 else begin
 message "Previamente hay que consultar filas" reverse bell
 next option a4
 end
 option "Borrar"
 if numrecs <> 0 then remove
 else begin
 message "Previamente hay que consultar filas" reverse bell
 next option a4
 end
 option a4 "Consultar"
 query
 end menu
 end menu

 layout
 box
 label "Mantenimiento de Provincias"
 end layout
 end form
 end define

main begin
 menu form
end main
```

Este programa mantiene la tabla «provincias», comprobando si hay alguna fila en pantalla antes de proceder a su modificación o borrado. La instrucción NEXT OPTION indica que antes de realizar una de las dos operaciones se deberá consultar (esto no significa que se ejecute la opción «Consultar», sino que únicamente ésta se mostrará en vídeo inverso, recomendando así su utilización).

## Instrucción PULLDOWN

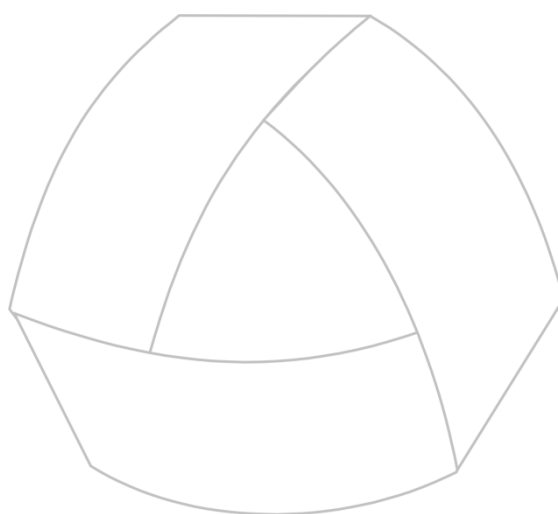
Esta instrucción mostrará en pantalla y activará un menú de tipo «Pulldown». Su sintaxis es la siguiente:

**PULLDOWN** nombre\_pulldown [ACCESORIES]

La forma de salir de un menú «Pulldown» es pulsando la tecla asignada a la acción <fquit> o bien ejecutando la instrucción EXIT PULLDOWN.

Como ya hemos comentado anteriormente, la cláusula «ACCESORIES», encargada de mostrar la persiana «MB» en el «Pulldown», no tiene efecto en las versiones de MultiBase para Windows, ya que esta persiana aparecerá siempre en cualquier «Pulldown» de dicho entorno.





MultiBase  
MANUAL DEL PROGRAMADOR

## CAPÍTULO 15. Funciones de usuario

# Introducción

---

Una función es un conjunto de instrucciones, orientado a la realización de una tarea específica, que se referencia mediante un nombre simbólico. Las funciones pueden ser:

- Internas del CTL.
- Funciones de usuario. Son aquellas funciones definidas por el programador en caso de que no exista ninguna para la realización de una tarea específica, pudiendo definir tantas como sea necesario.

Las funciones internas de CTL se explican en el capítulo 5 («Conceptos Básicos») en este mismo volumen.

Una función puede modificar los valores de las variables globales definidas en la sección DEFINE del módulo o bien devolver un resultado (un solo valor).

En los siguientes apartados indican las características, estructura y la forma de manejar y definir estas funciones de usuario.

## Estructura y características de una función

---

La definición de una función dentro de un módulo CTL tiene lugar después de la sección MAIN en un módulo principal o después de la sección LOCAL en un módulo librería.

La sintaxis de una función tiene la siguiente estructura:

```
FUNCTION nombre_función ([parámetros])
 variable_local {tipo_sql | LIKE tabla.columna}
BEGIN
 instrucciones
 ...
 ...
END FUNCTION
```

El nombre de la función («nombre\_función») será el nombre simbólico que se empleará para su ejecución. En caso de que una función reciba algún parámetro, éstos tendrán que indicarse separados por comas. Los nombres de dichos parámetros no tienen por qué estar definidos ni como variables del módulo ni como variables locales de la función.

### Ejemplo 1 [PFUNC1]:

```
main begin
 call presenta("valor", 1)
end main

function presenta(a,b) begin
 pause a
 pause b
end function
```

Como puede comprobarse, la función «presenta» se ha definido con dos parámetros («a» y «b»), los cuales operan como variables locales de la función, no siendo necesario por tanto indicar el tipo de dato SQL. De este modo, mediante dichos parámetros podrá ejecutarse la función con independencia del tipo de dato recibido.

Las variables que se definen entre el nombre de la función y la sentencia BEGIN son variables locales a la función, esto es, son reconocidas solamente dentro de dicha función. Si el nombre de una variable local coincide con el de una variable global (definida en la sección DEFINE), las operaciones que se realicen en la función sólo afectarán a la variable local y no a la global.

Como se ha visto en la sintaxis de las funciones, la definición de estas variables locales admite la cláusula «LIKE», la cual permite indicar el nombre de una tabla y una columna para que dicha variable local herede su tipo



de dato y su longitud, no así sus atributos como sucedía en la definición de variables globales del módulo (sección DEFINE).

**Ejemplo 2 [PFUNC2]. Menú «Pop-up» para seleccionar la aplicación que se desea ejecutar. Antes de ejecutar el menú de una aplicación u otra se pide una clave:**

```
define
 variable a char(1) default ""
 variable clave char(18) default ""
end define

main begin
 menu m1
end main

menu m1 "Aplicaciones" down skip 2 line 5 column 30 no wait
 option a1 "A l m a c é n"
 if passwd("almacen") = true then ctl "almacen"
 option a2 "C o n t a b i l i d a d"
 if passwd("contabilidad") = true then ctl "conta"
 option a3 "N o m i n a s"
 if passwd("nominas") = true then ctl "nominas"
end menu

function passwd(pass)
 lon smallint
 col smallint
 i smallint
begin
 let lon = length (pass)
 let col = 12
 display box at 15,1 with 3,30
 display "Password " at 16,2
 for i = 1 to lon begin
 prompt for a clean autonext at 16 ,col
 end prompt
 if cancelled = true then return false
 if actionlabel(lastkey) in ("freturn","faccept") then break
 display "***" at 16, col
 view
 if col < 79 then let col = col + 1
 let clave = clave & a
 let a = null
 end
 clear
 if clave = pass then return true
 else begin
 pause "Clave incorrecta. Pulse [Return] para continuar" reverse bell
 return false
 end
end function
```

Esta función solicita repetidamente un carácter (tantos como indique la longitud de la clave a comprobar) sobre una variable global definida como «char(1)». Al introducir cada carácter se produce automáticamente su aceptación («AUTONEXT»), limpiándose de la pantalla mediante la cláusula «CLEAN». Por cada carácter introducido se mostrará un asterisco (instrucción DISPLAY). Una vez introducidos todos los caracteres se comprobará si éstos coinciden con la clave pasada como parámetro. De no ser así se mostrará un mensaje y no se podrá ejecutar el menú final de la aplicación seleccionada.

Como ya se ha indicado, un programa CTL puede estar compuesto por varios módulos, siendo uno de ellos el principal y los restantes librerías. La construcción de un programa se realiza mediante el comando **ctlink**, indi-

cando los nombres de todos los módulos (principal y librerías) que compondrán dicho programa. Los módulos librerías pueden estar compuestos por funciones y estructuras de menús («Lotus» o de iconos, «Pop-up» y «Pulldown»). La ejecución de estos menús residentes en módulos librerías siempre tendrá que realizarse desde una función. Al ejecutar una función incluida en un módulo librería dentro de un programa, dicho módulo se carga en memoria, no descargándose hasta que finalice la aplicación. El resto de programas que compartan dicha librería no la volverán a cargar, sino que compartirán la ya existente en memoria.

Una función se incluirá en un módulo librería cuando el proceso o tarea que calcula ésta se vaya a emplear en otro programa de la aplicación. En el ejemplo anterior, si la función «password» se emplease en otros programas de la aplicación, lo lógico sería definir un módulo librería en el que se incluyese dicha función. Por tanto, los procesos que compartiesen dicha función tendrían que enlazarse con esta librería mediante el comando **ctlink**.

La forma de ejecutar una función es mediante la instrucción **CALL** o bien incluyendo su nombre dentro de una expresión.

**Ejemplos 3 y 4: [PFUNC3] y [PFUNC4] (ctlink pfunc3 pfunc4). Mantenimiento de dos FORMS en un solo programa (módulo principal y librería):**

**[PFUNC3]:**

```
database almacen

define
 form
 tables
 clientes
 end tables

 editing
 after editadd editupdate of clientes.provincia
 if yes("Provincia inexistente. Desea darla de alta","Y") = true then
 call provincias(clientes.provincia)
 else let clientes.provincia = null
 next field "provincia"
 end editing
 end form
end define

main begin
 menu form
end main
```

Mantenimiento de la tabla «clientes». Cuando se intente asignar un cliente a una provincia que no existe, el programa llamará a la función «provincias()» para ejecutar el mantenimiento de la tabla «provincias». Al encontrarse este mantenimiento en un módulo librería es por lo que su ejecución se tiene que realizar a través de una función. La llamada a dicha función incluye un parámetro que coincide con el código de la provincia que se desea dar de alta.

**[PFUNC4]:**

```
database almacen

define
 variable provin like provincias.provincia
 form
 tables
 provincias
 end tables

 editing
```

```

 before editadd of provincias.provincia
 let provincias.provincia = provin
 next field
 next field down "descripcion"
 next field up "prefijo"
 end editing
end form
end define

function provincias(a) begin
 let provin = a
 add one
 clear form
end function

```

Este módulo de librería emplea la función «provincias(a)» para ejecutar el mantenimiento de la tabla «provincias». En concreto, la instrucción ADD ONE permitirá dar de alta una fila en la tabla cada vez que se ejecute la función. El parámetro que se recoge en la función es el enviado por el módulo principal, asignándosele a la columna «provincia» para su alta antes de comenzar la edición.

## Instrucciones relacionadas

CTL dispone de dos instrucciones específicas para el manejo de funciones, CALL y RETURN. Con independencia del tipo de función de que se trate, es decir, tanto si es una función local a un módulo como si reside en un módulo librería o es una función interna del CTL, todas ellas se manejarán de la misma forma.

En los siguientes apartados se comentan las instrucciones relacionadas con el manejo de funciones, incluso con las internas del CTL.

### Instrucción CALL

Como ya hemos indicado anteriormente, una función de CTL se ejecuta mediante la instrucción CALL o bien incluyendo su nombre dentro de una expresión empleada en cualquier otra instrucción del CTL (IF, WHILE, LET, etc.). La instrucción CALL se empleará cuando la función que se desea ejecutar no devuelva ningún valor. Su sintaxis es la siguiente:

**CALL** nombre\_función([parámetro1][, parámetro2][, parámetro n])

Por el contrario, cuando la función que se va a ejecutar devuelve un valor, es decir, si incluye la instrucción RETURN en su estructura, la llamada a dicha función debe realizarse recogiendo o empleando el valor que devolverá en una expresión. En este caso, dicha llamada podrá producirse desde cualquier instrucción CTL que emplee una expresión.

Cuando la función finaliza su ejecución el programa seguirá ejecutando la siguiente instrucción a la llamada, es decir, la siguiente instrucción a CALL.

**Ejemplo 5 [PFUNC5]. Mantenimiento de la tabla «provincias».** En caso de producirse algún error por parte del operador, se mostrará en una ventana generada por una función. Asimismo, esta función también mostrará las acciones de cada una de las teclas que se pulsen a continuación del mantenimiento de la tabla:

```

database almacen

define
 form
 tables
 provincias
 end tables

```

```
menu m1 "Mantenimiento"
 option a1 "Consulta"
 query
 option "Modificar"
 if numrecs <> 0 then modify
 else begin
 call disp_msg("Debe consultar antes")
 next option a1
 end
 option "Borrar"
 if numrecs <> 0 then remove
 else begin
 call disp_msg("Debe consultar antes")
 next option a1
 end
 option "Añadir"
 add
end menu

layout
 box
 label "Provincias"
end layout
end form

end define

main begin
 menu form
 display "Pulsar una tecla para mostrar su acción" reverse at 5,1
 forever begin
 read key
 if actionlabel(lastkey) = "fquit" then break
 call disp_msg(actionlabel(lastkey))
 end
 clear at 5,1
end main

function disp_msg(m)
 l1 smallint
 l2 smallint
 men2 char(14)
begin
 let men2 = " Pulse <Intro>"
 let l1 = length(m)
 let l2 = length(men2)
 display box at 20,1 with 3,l1 + l2 + 4
 display m at 21,2, men2 reverse at 21,l1 + 4
 read key
 clear box at 20,1
 clear at 21,2
 clear at 21,l1 + 4
end function
```

Esta función recibe un parámetro, que es el mensaje que se mostrará en una caja. En primer lugar, en la operación de mantenimiento, si el operador intenta modificar o borrar un fila antes de realizar una consulta, se le avisa mediante un mensaje tratado por la función «disp\_msg()». Después de abandonar el mantenimiento se forma un bucle para la petición de teclas. Las acciones de dichas teclas se muestran mediante la ejecución de la misma función.

## Instrucción RETURN

La instrucción RETURN finaliza la ejecución de una función y permite devolver un valor hacia el programa desde el que se realizó la llamada. Su sintaxis es la siguiente:

**RETURN** [valor]

Por defecto, una función que no incluya esta instrucción finalizará cuando se ejecute la última instrucción, es decir, aquella que se encuentre antes de las palabras reservadas «END FUNCTION». Sin embargo, si una función devuelve un valor, la única forma de hacerlo será mediante esta instrucción. Normalmente, la llamada a este tipo de funciones que devuelven un valor no será mediante la instrucción CALL, sino incluyéndose en una expresión dentro de una instrucción CTL.

La sección MAIN de un módulo principal es considerada como una función, por lo que la instrucción RETURN podrá emplearse también en ella. En este caso, si el programa ha sido llamado desde otro, el programa que realiza la llamada puede recoger el valor devuelto por esta instrucción mediante la variable interna de CTL «status».

**Ejemplo 6 [PFUNC6]. Función que cuenta el número de filas que contiene una tabla de la base de datos «almacen»:**

```

database almacen

define
 frame tablas
 screen
 {
 Tabla: [a]
 Filas: [b]
 }
 end screen

 variables
 a = tabla char
 b = filas integer
 end variables

 layout
 box
 label "T a b l a"
 end layout
 end frame
end define

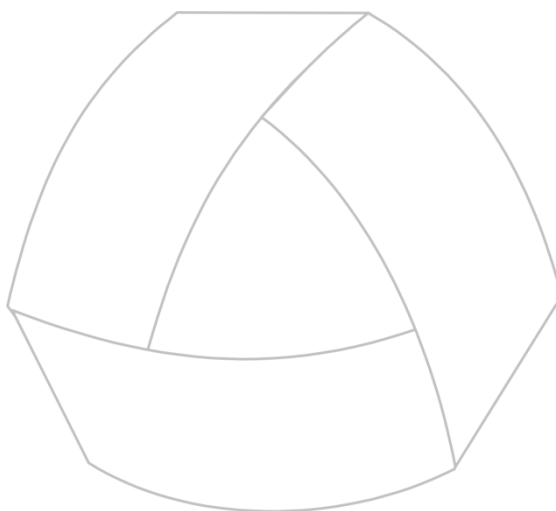
main begin
 display frame tablas
 forever begin
 window from select tablename from systables
 10 lines label "T a b l a s"
 at 7,40
 set tablas.tabla
 if cancelled = true then break
 let tablas.filas = numfilas(tablas.tabla)
 view
 end
 clear frame tablas
end main

function numfilas(tab)
 nrow integer
begin
 tsql "update statistics for table " && tab clipped

```

```
select nrows by name from systables
 where tabname = $tab
return nrows
end function
```

Este programa emplea una función para averiguar el número real de filas que tiene la tabla elegida por el operador en una ventana. Dicha función ejecuta la instrucción UPDATE STATISTICS de forma dinámica mediante la instrucción TSQL. A continuación lee la columna «nrows» de la tabla «systables» para la tabla seleccionada en la sección MAIN. El número leído en esta instrucción SELECT será el que se devuelva al programa principal. Este valor se recoge en una variable de un FRAME y se muestra automáticamente en pantalla dado que dicho FRAME también está presente.



MultiBase  
MANUAL DEL PROGRAMADOR

## CAPÍTULO 16. Ficheros de ayuda y mensajes

### Introducción

---

Los ficheros de ayuda y mensajes son ficheros ASCII que permiten incluir textos de ayuda así como cualquier tipo de mensaje que se desea que aparezca al usuario al ejecutar instrucciones tales como PAUSE, MESSAGE, DISPLAY, etc.

Un fichero de ayuda puede configurar los textos o literales que se incluyan en una sección SCREEN tanto de un FORM como de un FRAME, pudiendo asimismo incluir los literales de un menú («Lotus» o de iconos, «Pop-up» o «Pulldown») y de sus respectivas opciones.

Los ficheros de ayuda se preparan mediante cualquier editor de textos ASCII (por ejemplo «tword -e»), debiendo indicarles la extensión «.shl». La estructura de este tipo de ficheros es la siguiente:

```
.[número | COMMENTS]
líneas de texto con el mensaje asociado al número o comentario.

.[número | COMMENTS]
texto ...
...
```

Una vez escritos los números y sus textos asociados, para que el fichero sea utilizable desde CTL debe compilarse mediante la opción «Compilar» del menú «Ayudas y Mensajes» de la persiana «Desarrollo» del Entorno de Programación. Esta opción emplea el comando **thelpcomp** para la compilación. Dicho comando producirá un fichero homónimo pero con extensión «.ohl», debiendo estar éste en cualquiera de los directorios especificados en la variable de entorno MSGDIR.

Para que un programa CTL utilice un fichero de ayuda, éste tendrá que emplear la instrucción OPTIONS con la cláusula «HELP FILE» para indicar su nombre.

Los ficheros de ayuda se emplean para las siguientes tareas:

- Incluir textos de ayuda de programas.
- Incluir literales, mensajes o etiquetas.
- Incluir pantallas (SCREENS) de los objetos FORM y FRAME.
- Incluir literales empleados por un menú («Lotus» o de iconos, «Pop-up» y «Pulldown»).

En los siguientes apartados se explica la utilidad de los ficheros de ayuda respecto a estas tareas.

### Inclusión de textos de ayuda de programas

---

A diferencia de los mensajes, que se tratarán en el siguiente apartado, los textos de ayuda se activan a voluntad del usuario de la aplicación cuando éste pulsa la tecla asociada a la acción <fhelp>.

Las ayudas se pueden activar en entrada de datos, esto es, en un menú o en un formulario (FORM). Al pulsar la tecla asignada a la acción <fhelp> aparecerá en pantalla una caja, cuyo título será el tipo de elemento de diálogo, en la que se mostrarán enmarcados los tres niveles básicos de ayuda disponibles, los cuales, dependiendo del elemento de que se trate, son los siguientes:

a) «Teclas»: Muestra alternativamente una ventana con una lista de las funciones disponibles y sus teclas asociadas, según estemos en un menú o en edición de un campo (esto es, introduciendo valores). Este nivel de ayuda está disponible por defecto en el CTL, por lo que no es necesario programarlo.

b) El siguiente nivel de ayuda se denomina «Opción» o «Campo», según estemos en un menú o en edición de un campo, respectivamente. En él se mostrará el mensaje que hayamos introducido en el fichero de ayuda especificado por «HELP FILE» y asociado al número indicado mediante la cláusula «HELP». Ésta puede incluirse en una definición de MENU o como atributo de una variable global de los objetos FORM o FRAME. También se



mostrará el mensaje si ponemos su número en una «OPTION» de un menú. Dicha ayuda aparecerá en una caja con las dimensiones definidas en la sección OPTIONS mediante la cláusula «HELP BOX» o con las que por defecto proporcione CTL. El título de la caja será, en el caso de los menús, el texto indicado como texto de la opción, y «LABEL» en el caso de las variables si se especificó éste; de lo contrario aparecerá «campo».

c) «Global»: Este nivel de ayuda corresponde al mensaje especificado mediante la cláusula «HELP MESSAGE» y un número de ayuda. Éste es válido para el programa en el que se define. Tanto las dimensiones de la caja como su título («Global») son valores estándares por defecto.

Asimismo, en el FORM existe un cuarto nivel de ayuda en la operación QUERY, que muestra siempre las reglas de edición de campos para búsqueda de filas («Query by Forms»).

Todos los mensajes de ayuda que se introduzcan dentro de un fichero de ayuda serán empleados por el Documentador Automático («Tdocu») para la generación de la documentación de usuario.

NOTA: El diálogo de la pantalla de ayuda en Windows es global a todo el módulo, activándose unos mensajes u otros mediante unos botones.

#### Ejemplo 1 [PHELP1] y [AYUDA1.SHL]. Fichero de ayuda.

##### [AYUDA1.SHL]:

```
.10
Opción para añadir filas
sobre la tabla de provincias
.20
Opción para modificar la fila
en curso de la tabla de provincias
.30
Opción para borrar la fila en
curso de la tabla de provincias
.40
Opción para consultar filas
de la tabla de provincias.
.50
Mantenimiento de PROVINCIAS
.70
Programa de Mantenimiento de la tabla de PROVINCIAS
```

##### [PHELP1]:

```
database almacen

define
 form
 tables
 provincias
 end tables

 menu m1 "Mantenimiento"
 option "Añadir" help 10
 add
 option "Modificar" help 20
 modify
 option "Borrar" help 30
 remove
 option "Consultar" help 40
 query
 end menu

 layout
 box
 label 50
```

```
 end layout
 end form
 end define

 global
 control
 on beginning
 message "AYUDA: " && keylabel ("fhelp") reverse
 end control

 options
 help file "ayuda1"
 help message 70
 help box at 10,3 with 5,60
 end options
 end global

 main begin
 menu form
 end main
```

Este programa mantiene la tabla «provincias» y emplea el fichero de ayuda «ayuda1» para extraer los mensajes de ayuda del menú y del módulo. En caso de pulsar la tecla asignada a la acción <fhelp> en el menú, se podrá consultar para qué sirve cada una de las opciones de éste. Si se elige la opción «Global» aparecerá el mensaje de ayuda del programa.

## Literales, mensajes o etiquetas

---

En CTL se considera mensaje cualquier texto constante que se produzca en el curso de la ejecución de un programa, ya sea por pantalla, impresora o en un fichero de resultados. Estos textos se corresponden con cualquier cadena entrecomillada en el fuente del programa. Por lo tanto, las instrucciones de CTL que empleen dichos literales podrán utilizar los mensajes de un fichero de ayuda.

CTL dispone de una función interna, denominada «msgtext()», que, indicándole el número de mensaje, extraerá el texto asignado a dicho número en el fichero de ayuda activo, es decir, el indicado en la cláusula «HELP FILE» de la instrucción/sección OPTIONS.

**Ejemplo 2 [PHELP2] y [AYUDA2.SHL]. Extracción de mensajes y etiquetas de un fichero de ayuda.**

**[AYUDA2.SHL]:**

```
.10
Opción para añadir filas
sobre la tabla de clientes
.20
Opción para modificar la fila
en curso de la tabla de clientes
.30
Opción para borrar la fila
en curso de la tabla de clientes
.40
Opción para consultar filas
de la tabla de clientes
.50
Pulsar %s para CONSULTAR
.60
P r o v i n c i a s
.70
Mantenimiento de CLIENTES
.80
```

```

Programa de Mantenimiento de la tabla de CLIENTES
.100
FIN DEL PROGRAMA

```

**[PHELP2]:**

```

database almacen

define
 form
 tables
 clientes
 end tables

 menu m1 "Mantenimiento"
 option "Añadir" help 10
 add
 option "Modificar" help 20
 modify
 option "Borrar" help 30
 remove
 option "Consultar" help 40
 query
 end menu

 editing
 before editadd editupdate of clientes.provincia
 message msgtext(50, keylabel("fuser1"))
 on action "fuser1"
 if curfield = "provincia" then
 window from select provincia, descripcion
 from provincias
 10 lines label 60
 at 5,10
 set clientes.provincia
 end editing

 layout
 box
 label 70
 end layout
 end form
end define

global
 control
 on beginning
 message msgtext(50,keylabel("fhelp"))
 end control

 options
 help file "ayuda2"
 help message 80
 end options
end global

main begin
 menu form
 pause msgtext(100)
end main

```

Este programa emplea el fichero de ayuda para mostrar tanto los mensajes de ayuda de las opciones del menú del FORM como los del programa (comentados en el apartado anterior). Asimismo, mediante la función interna

«msgtext()» se extraen literales de dicho fichero de ayuda. Por ejemplo, antes de editar la «provincia» del cliente aparecerá un mensaje indicando la tecla que se debe activar para mostrar una ventana con todas las provincias existentes en la tabla «provincias». Dicho mensaje contiene el nombre de la tecla como «%s», lo que significa que la función «msgtext()» admite un segundo parámetro, cuyo valor, en caso de emplearse se sustituirá por la cláusula «%s» en el fichero de ayuda. Este mensaje también se empleará en la sección CONTROL para indicar cuál será la tecla que el operador deberá pulsar para obtener la ayuda del FORM y del programa. Asimismo, la etiqueta de la ventana que muestra las provincias existentes en la tabla «provincias» se extrae del fichero de ayuda mediante el número «60». Por último, al finalizar el programa se vuelve a emplear la función «msgtext()» para mostrar un mensaje que avisa de la finalización del programa.

## Literales de pantalla de un FORM/FRAME

Tanto en un FORM como en un FRAME existe la posibilidad de incluir todos los literales que aparecerán en una pantalla de dichos objetos, para lo cual deberá emplearse la cláusula «MESSAGE» acompañando a cada SCREEN del objeto en cuestión. Si los literales estuviesen definidos en la SCREEN, además de indicarse en un número del fichero de ayuda, a la hora de ejecutar el objeto dichos literales se superpondrían sobre los propios de la SCREEN.

### Ejemplo 3 [PHELP3] y [AYUDA3.SHL].

#### [AYUDA3.SHL]:

```
.10
 Cliente:

 Empresa:
 Dirección:
 Provincia:

.20
C l i e n t e s
.30
Programa de mantenimiento de CLIENTES en el que la
pantalla (SCREEN) se extrae de un fichero de ayuda
```

#### [PHELP3.SCT]:

```
database almacen

define
 form
 screen message 10
{
 %
 [a] *
 [b]
 [c]
 [d] [e]
}
end screen

tables
 clientes
end tables

variables
```

```

a = clientes.cliente required
b = clientes.empresa upshift
c = clientes.direccion1
d = clientes.provincia lookup e = descripcion
 joining provincias.provincia
end variables

layout
 box
 label 20
 end layout

end form
end define

global
 options
 help file "ayuda3"
 help message 30
 end options
end global

main begin
 menu form
end main

```

En este ejemplo, la sección SCREEN no contiene ningún literal para los identificadores incluidos entre corchetes. Estos literales se extraerán en ejecución del fichero de ayuda «ayuda3». Asimismo, de este fichero se extraerá la etiqueta de la ventana estándar del FORM, así como el «help» del programa («HELP MESSAGE»).

## Literales de menús

Los mensajes de un fichero de ayuda también pueden servir para extraer los literales empleados en los menús de tipo «Lotus» o «Iconos», «Pop-Up» o «Pulldown». El formato del fichero de ayuda en este caso es el siguiente:

```

.número
Etiqueta del menú
Etiqueta de la primera opción,[tecla],[comentario]
Etiqueta de la segunda opción,[tecla],[comentario]
Etiqueta de la tercera opción,[tecla],[comentario]
...

```

**Ejemplo 4 [PHELP4] y [AYUDA4.SHL].** Siguiendo con el ejemplo anterior («phelp3»), en este caso se ha incluido un menú «Lotus» al FORM.

**[AYUDA4.SHL]:**

```

.10
 Cliente:

 Empresa:
 Dirección:
 Provincia:

.20
C l i e n t e s
.30
Programa de mantenimiento de CLIENTES en el que la
pantalla (SCREEN) se extrae de un fichero de ayuda
.40

```

Mantenimiento

Añadir,d,Opción para añadir filas

Modificar,f,Opción para modificar filas

Borrar,,Opción para borrar filas

Consultar,,Opción para consultar filas

### [PHELP4.SCT]:

```
database almacen

define
 form
 screen message 10
 {
 %
 [a]
 *
 [b]
 [c]
 [d] [e]
 }
 end screen

 tables
 clientes
 end tables

 variables
 a = clientes.cliente required
 b = clientes.empresa upshift
 c = clientes.direccion1
 d = clientes.provincia lookup e = descripcion
 joining provincias.provincia
 end variables

 menu m1 message 40
 option
 add
 option
 modify
 option
 remove
 option
 query
 end menu

 layout
 box
 label 20
 end layout

 end form
end define

global
 options
 help file "ayuda4"
 help message 30
 end options
end global

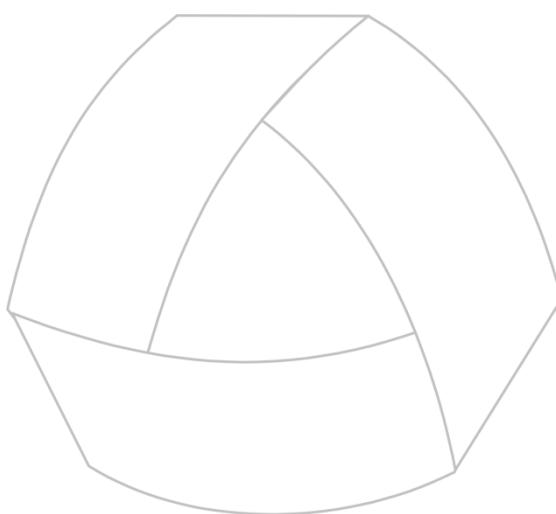
main begin
```

```
 menu form
 end main
```

Este programa, además de extraer los literales de la sección SCREEN de un fichero de ayuda, utilizará también éste para los literales del menú «Lotus» o de «Iconos». La cláusula «MESSAGE» de la sección MENU indica el número de mensaje que se empleará para este menú. En el fichero de ayuda, la primera línea corresponde con el literal que se mostrará como etiqueta del menú. A continuación, cada una de las siguientes líneas se corresponde con cada opción del MENU. El primer literal se corresponde con la etiqueta de la opción («OPTION»), el segundo es la tecla que permite activar la opción (equivale a la cláusula «KEY») y el tercero se corresponde con el comentario de la opción (equivale a la cláusula «COMMENTS»).







MultiBase  
MANUAL DEL PROGRAMADOR

## CAPÍTULO 17. Bloqueos

### Introducción

---

Un bloqueo es la limitación del acceso a una base de datos, a una tabla o a una fila, impuesta por una instrucción en un proceso determinado para evitar inconsistencia en los valores de ésta respecto de otras instrucciones en otros procesos. Los niveles de bloqueos existentes en CTL son los siguientes:

- Bloqueo de la base de datos.
- Bloqueo de una tabla.
- Bloqueo de una fila.
- Bloqueo de un número indeterminado de filas.

Dado que el sistema tiene un número máximo de bloqueos, impuesto por los parámetros del «Kernel» del UNIX/Linux, es conveniente utilizar la instrucción LOCK TABLE donde sea posible con el fin de no incrementar excesivamente la lista de bloqueos, ya que dicha instrucción inhibe los bloqueos por cada fila. Esta operación es especialmente recomendable cuando la base de datos tiene activado el tratamiento de transacciones.

En los siguientes apartados se explican los distintos casos de bloqueos expuestos anteriormente.

### Bloqueo de la base de datos

---

Para bloquear la base de datos ésta deberá seleccionarse con la cláusula «EXCLUSIVE» de la instrucción DATABASE, cuya sintaxis es la siguiente:

**DATABASE** nombre\_base\_datos [EXCLUSIVE]

Dicha cláusula provoca el bloqueo de la tabla «systables» del catálogo de la base de datos. Por lo tanto, ningún otro usuario que tenga permiso de acceso a la base de datos podrá abrirla.

En caso de ejecutar esta instrucción cuando la base de datos se encuentre abierta por otro usuario, se producirá un error de CTL, activando éste la variable interna de CTL «ERRNO».

Esta instrucción equivaldría a bloquear la tabla «systables» mediante la siguiente:

```
database base_datos;
lock table systables in exclusive mode;
```

### Bloqueo de una tabla

---

El bloqueo de una tabla de una base de datos sólo puede realizarse mediante la instrucción LOCK TABLE, cuya sintaxis es la siguiente:

**LOCK TABLE** nombre\_tabla IN {SHARE | EXCLUSIVE} MODE

Esta instrucción bloquea temporalmente una tabla hasta que se produzca su desbloqueo mediante la instrucción UNLOCK TABLE:

**UNLOCK TABLE** nombre\_tabla

La cláusula «SHARE» de la instrucción LOCK TABLE indica que la tabla «nombre\_tabla» se bloquea para el resto de usuarios a nivel de las operaciones de modificación, borrado e inserción. Sin embargo, dichos usuarios podrán leer las filas de dicha tabla sin ningún problema. Por el contrario, la cláusula «EXCLUSIVE» implica el bloqueo de la tabla respecto a todas las operaciones, incluida la de lectura de filas.

En el caso de las bases de datos transaccionales es conveniente bloquear una tabla completa en lugar de acumular bloqueos de filas, como veremos más adelante en este mismo capítulo. Por ejemplo, la instrucción LOAD

precisa del tratamiento de una transacción para la carga masiva de filas. En este caso, es preferible bloquear la tabla antes de comenzar la transacción y desbloquearla al final del proceso.

#### Ejemplo 1 [PLOCK1]:

```
main begin
 database almacen
 select "" from systables
 where tabname = "syslog"
 if found = false then begin
 close database
 start database almacen with log in "/usr/manuel/ficlog"
 end
 unload to "provincias.unl" select * from provincias
 create temp table carga (provincia smallint,
 descripcion char(20),
 prefijo smallint)
 lock table carga in exclusive mode
 begin work
 load from "provincias.unl" insert into carga
 if errno = 0 then commit work
 else rollback work
 unlock table carga
 window from select * from carga
 10 lines label "Carga masiva"
 at 1,1
 close database
 start database almacen with log in ""
end main
```

Este programa selecciona la base de datos «almacen» y comprueba si existe una fila en la tabla «systables» identificada por «syslog» en la columna «tabname». Si dicha fila existe indicará que la base de datos es transaccional. Por contra, si no existe la base de datos no será transaccional, en cuyo caso se convertirá a transaccional. A continuación se descarga la tabla «provincias» para cargar sus mismas filas en una nueva tabla, denominada «carga». Antes de comenzar la carga masiva se bloquea la tabla «carga» en modo exclusivo para que nadie pueda realizar ninguna operación sobre ella.

Posteriormente, comenzará la transacción (BEGIN WORK) cargándose las filas existentes en el fichero «provincias.unl». En caso de no haberse producido ningún error, la transacción finaliza correctamente (COMMIT WORK), dándose como válida. En caso contrario se deshacerá todo lo cargado mediante la instrucción ROLLBACK WORK. A continuación, la tabla empleada para la carga masiva de datos se desbloquea, presentándose en una ventana todas las filas cargadas en la transacción. Por último, la base de datos se cambia de nuevo a no transaccional mediante la instrucción START DATABASE.

## Bloqueo de una fila

El bloqueo de una fila se encuentra implícito en el FORM y en el tratamiento de «cursores» declarados con la cláusula «FOR UPDATE».

En el caso del FORM, en versiones para red local (Windows), en instalaciones «cliente-servidor» o en sistemas UNIX, si dos usuarios acceden a una misma fila para modificar algún valor de sus columnas, el primero de ellos que haya seleccionado la opción de «modificar» será el que bloquee la fila, mientras que al segundo le aparecerá un mensaje de error indicando que la fila ya se encuentra bloqueada por otro usuario.

En el caso del CURSOR, si éste se declara con la cláusula «FOR UPDATE» y no se incluye la cláusula «NOWAIT», cada fila que se lea mediante las instrucciones FETCH o FOREACH quedará automáticamente bloqueada para el

resto de usuarios. En el caso de incluir la cláusula «NOWAIT» en la declaración del CURSOR, para detectar si una fila está o no bloqueada por otro usuario habrá que manejar la variable interna «LOCKED».

### Ejemplo 2 [PLOCK2]. Actualización del precio de venta de cada uno de los artículos de la tabla «articulos»:

```
database almacen

define
 frame arti
 screen
 {
 %
 Código Artículo[a] *

 Descripción[b]

 Precio Venta[c]

 }
 end screen

 variables
 a = articulo like articulos.articulo
 b = descripcion like articulos.descripcion
 c = pr_vent like articulos.pr_vent reverse
 end variables

 editing
 after editupdate of pr_vent
 exit editing
 end editing

 layout
 box
 label "A r t í c u l o s"
 end layout
 end frame
end define

main begin
 declare cursor precio_venta for select articulo, descripcion, pr_vent
 by name on arti
 from articulos
 for update
 foreach precio_venta begin
 input frame arti for update variables pr_vent at 7,10
 end input
 if cancelled = false then
 update articulos set pr_vent = $arti.pr_vent
 where current of precio_venta
 end
 clear frame arti
 end main
```

Este programa declara un CURSOR para la actualización del precio de venta de todos los artículos (cláusula «FOR UPDATE»). Por cada una de las filas contempladas en el CURSOR se realiza una actualización del precio de venta en la variable del FRAME mediante la instrucción INPUT FRAME con cláusula «FOR UPDATE». Debido a esta cláusula la fila recién leída queda bloqueada para el resto de usuarios que tengan acceso a la base de datos. En caso de no cancelar la operación de actualización del artículo en curso, éste se actualizará con la instrucción UPDATE del SQL con la cláusula «WHERE CURRENT OF cursor». Esta cláusula internamente emplea el

«ROWID» para la actualización de la fila en curso del CURSOR. Por último, después de haber actualizado los precios de todos los artículos, el FRAME se elimina de la pantalla mediante la instrucción CLEAR FRAME.

## Bloqueo de un número indeterminado de filas

El bloqueo de un número indeterminado de filas sólo puede conseguirse con tratamiento de transacciones. Si se están controlando transacciones (instrucción BEGIN WORK) y se actualiza una fila, ésta quedará bloqueada para el resto de usuarios hasta que finalice dicha transacción, es decir, hasta que se ejecute la instrucción COMMIT WORK o ROLLBACK WORK.

Si en el ejemplo anterior («plock2») se hubiesen empleado transacciones, todas las filas tratadas por el CURSOR habrían quedado bloqueadas hasta que hubiese finalizado la transacción.

**Ejemplo 3 [PLOCK3]. Ejemplo similar al anterior pero con tratamiento de transacciones:**

```

database almacen

define
 frame arti
 screen
 {
 %
 Código Artículo[a]
 *

 Descripción[b]

 Precio Venta[c]
 }
 end screen

 variables
 a = articulo like articulos.articulo
 b = descripcion like articulos.descripcion
 c = pr_vent like articulos.pr_vent reverse
 end variables

 editing
 after editupdate of pr_vent
 exit editing
 end editing

 layout
 box
 label "A r t í c u l o s"
 end layout
 end frame

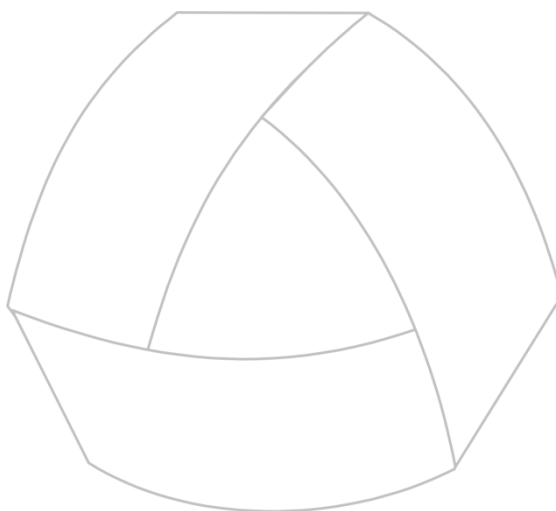
 variable sw smallint default 0
end define

main begin
 select "" from systables
 where tabname = "syslog"
 if found = false then begin
 close database
 start database almacen with log in "/usr/manuel/ficlog"
 end
end

```

```
declare cursor precio_venta for select articulo, descripcion, pr_vent
 by name on arti
 from articulos
 for update
begin work
foreach precio_venta begin
 input frame arti for update variables pr_vent at 7,10
 end input
 if cancelled = false then begin
 update articulos set pr_vent = $arti.pr_vent
 where current of precio_venta
 if errno <> 0 then begin
 let sw = 1
 break
 end
 end
end
if sw = 0 then commit work
else rollback work
clear frame arti
end main
```

En este ejemplo se comprueba si la base de datos «almacen» es o no transaccional. Para ello se comprueba la existencia o no de una fila en la tabla «systables» cuyo identificador de tabla sea «syslog». En caso de no ser transaccional, dicha base de datos se convierte mediante la instrucción START DATABASE. Por último, todas las filas tratadas en el CURSOR se bloquean de forma automática al declararlo con la cláusula «FOR UPDATE» y comenzar el tratamiento de transacciones (BEGIN WORK). Si no existe ningún error («ERRNO») en la actualización de ninguna fila tratada por el CURSOR, la transacción finalizará correctamente con la instrucción COMMIT WORK. Sin embargo, en caso de que la actualización de alguna fila produzca error, la transacción finalizará con la instrucción ROLLBACK WORK deshaciendo todas las actualizaciones previas.



MultiBase  
MANUAL DEL PROGRAMADOR

## CAPÍTULO 18. Transacciones

### Introducción

---

Una transacción es el conjunto de operaciones relativas a recuperación y actualización sobre la base de datos que se realizan de forma unitaria e indivisible, esto es, o se realizan totalmente o no se realizan. En otras palabras, es el modo de agrupar varias instrucciones DML (INSERT, DELETE y UPDATE) y asegurar que dicho grupo se ejecute completamente.

Para poder contemplar transacciones en una base de datos, ésta tiene que ser una base de datos transaccional. En los siguientes epígrafes se indican las operaciones necesarias para crear una base de datos transaccional y, además, la forma de convertir una base de datos que no es transaccional en transaccional y viceversa.

### Creación de una base de datos transaccional

---

Para crear una base de datos transaccional hay que emplear la instrucción CREATE DATABASE del CTSQL con la opción «WITH LOG IN», que indica el fichero transaccional. La sintaxis de la instrucción es la siguiente:

```
CREATE DATABASE base_datos [WITH LOG IN "fichero_log"]
[COLLATING "fichero_ordenación"]
```

Además del directorio que representa la base de datos («.dbs»), en el directorio en curso o en cualquiera de los indicados en la variable de entorno DBPATH se generará el fichero especificado en «fichero\_log». El nombre de «fichero\_log» tiene que incluir el «path» completo. Este «fichero\_log» no tiene formato ASCII, por lo que **NUNCA** deberá ser editado. La única forma de manejar su información es mediante las instrucciones del CTSQL que se comentan en este capítulo.

Ejemplo:

```
create database almacen with log in "/usr/manuel/ficlog"
```

Esta instrucción genera el fichero «/usr/manuel/ficlog» para recoger todas las operaciones de manejo y mantenimiento sobre las tablas de la base de datos. Asimismo, en la tabla «systables» del catálogo de la base de datos existe una fila correspondiente a la identificación del fichero transaccional. La información que se guarda para este fichero transaccional en esta tabla es la siguiente:

```
select * from systables
where tabname = "syslog"
```

|          |                    |
|----------|--------------------|
| tabname  | syslog             |
| owner    | ctldemo            |
| dirpath  | /usr/manuel/ficlog |
| tabid    | 150                |
| rowsize  | 0                  |
| ncols    | 1                  |
| nindexes | 0                  |
| nrows    | 0                  |
| created  | 25/05/1994         |
| version  | 0                  |
| tabtype  | L                  |
| nfkeys   | 0                  |
| part1    | 0                  |
| part2    | 0                  |
| part3    | 0                  |
| part4    | 0                  |
| part5    | 0                  |
| part6    | 0                  |
| part7    | 0                  |
| part8    | 0                  |



Este fichero transaccional («fichero\_log») tiene que existir mientras la base de datos se encuentre activa, o bien hasta que se sustituya por otro fichero mediante la instrucción `START DATABASE`. En caso de perder el fichero de transacciones, la base de datos no podrá abrirse jamás. En este supuesto, debemos conocer el «path» completo donde residía dicho fichero transaccional y crearlo, aunque sea vacío.

```
$ > /usr/manuel/ficlog
```

NOTAS:

- 1.— La creación de una base de datos transaccional no puede realizarse desde ninguna de las opciones del entorno de desarrollo. La única posibilidad es mediante la ejecución de la instrucción indicada anteriormente.
- 2.— El fichero transaccional crecerá constantemente, pudiendo ocupar un espacio considerable en el disco duro.

## Convertir una base de datos en transaccional y viceversa

Para convertir en transaccional una base de datos que no lo es hay que emplear la instrucción `START DATABASE` del CTSQL. Esta instrucción no se encuentra representada por ninguna de las opciones del entorno de desarrollo de MultiBase, y su sintaxis es la siguiente:

```
START DATABASE base_datos WITH LOG IN "fichero_log"
```

Al igual que sucedía en la instrucción `CREATE DATABASE`, el nombre del fichero transaccional («fichero\_log») tendrá que incluir el «path» completo donde se ubicará.

Esta instrucción sólo puede realizarla el propietario de la base de datos, o bien el usuario que tenga permiso «DBA» sobre ella. Asimismo, esta operación se tiene que realizar con la base de datos cerrada y sin que nadie la tenga seleccionada. Por lo tanto, antes de ejecutar esta operación habrá que especificar la instrucción `CLOSE DATABASE`:

**CLOSE DATABASE**

En caso de que deseásemos convertir una base de datos transaccional en no transaccional habría que ejecutar igualmente la instrucción `START DATABASE`, pero con el nombre del fichero transaccional vacío (válido únicamente a partir de la versión 2.0 de MultiBase):

```
start database base_datos with log in ""
```

En versiones de MultiBase anteriores a la 2.0, la única forma de eliminar las transacciones de una base de datos era eliminando la fila grabada en la tabla del catálogo «systables». Por ejemplo:

```
main begin
 database almacen exclusive
 if errno <> 0 then exit program
 delete from systables
 where tablename = "syslog"
 close database
end main
```

Como ya se ha indicado anteriormente, la ejecución de estas instrucciones sólo puede realizarla el propietario de la base de datos, o aquel que tenga permiso «DBA» sobre la misma.

## Cómo cambiar de fichero transaccional

Como ya hemos comentado anteriormente, la instrucción `START DATABASE` convierte en transaccional una base de datos que no lo es y viceversa. Del mismo modo, esta instrucción se emplea también para cambiar de fichero transaccional. Estos ficheros transaccionales crecen con bastante facilidad, por lo que se recomienda

cambiar de fichero transaccional cuando éste ocupe un espacio considerable en el disco (por ejemplo, cada vez que se realice una copia de seguridad).

Todas las recomendaciones expuestas en el epígrafe anterior para la instrucción `START DATABASE` son igualmente válidas en este caso.

NOTAS:

1.— Cuando se cambia de fichero transaccional y se realiza una copia de seguridad, el fichero transaccional antiguo debe incluirse en ella. Esta recomendación es importante si alguna vez hay que recurrir a la recuperación de datos (instrucción `ROLLFORWARD DATABASE`) desde dicho fichero transaccional.

2.— Cuando un fichero transaccional se encuentra en la copia de seguridad, dicho fichero debe eliminarse del disco, ya que posiblemente ocupe bastante espacio.

## Control de transacciones

Una vez que la base de datos seleccionada dispone de un fichero transaccional es posible controlar las transacciones. Una transacción consiste en controlar un determinado proceso, de tal forma que, en el hipotético caso de que éste finalizase de manera anómala, tuviésemos la posibilidad de deshacer dicha operación. Para ello hay que marcar explícitamente el principio y el fin de dicho proceso (grupo de instrucciones) con las siguientes instrucciones del CTSQL:

|                      |                                                                                                   |
|----------------------|---------------------------------------------------------------------------------------------------|
| <b>BEGIN WORK</b>    | Inicia la transacción.                                                                            |
| <b>COMMIT WORK</b>   | Finaliza la transacción haciendo definitivos los cambios realizados por el proceso en cuestión.   |
| <b>ROLLBACK WORK</b> | Finaliza la transacción deshaciendo los cambios realizados en la base de datos por dicho proceso. |

La consecuencia práctica de estas instrucciones es que en el fichero transaccional «log file» se registrarán tanto el comienzo de la transacción como su finalización. Esta operación permite lo siguiente:

- Que el programador decida en base a condiciones de error de determinadas instrucciones la realización o no de las modificaciones incluidas en la transacción como un todo.
- Que si ocurre algún fallo del sistema durante el proceso de la transacción, al faltar la marca de cierre de la misma se pueda producir una vuelta a un estado consistente de la base de datos.

Así pues, mediante este mecanismo se puede establecer un nuevo nivel de integridad de datos de las tablas bajo control del programador en operaciones tales como borrado de datos obsoletos de tablas relacionadas por un enlace («join»), asegurando además el cumplimiento de todas las instrucciones implicadas.

Una transacción puede suponer el bloqueo de todas las filas implicadas en las operaciones que reciben este tratamiento, siendo liberadas al finalizarla (instrucciones `COMMIT WORK` y `ROLLBACK WORK`) y cerrando todos los «cursores» abiertos durante la misma. Dicho bloqueo no es automático, sino que, para cada una de dichas filas hay que provocar una actualización (`UPDATE`), aunque ésta sea absurda (por ejemplo, actualizar una fila dejándole el mismo valor. A partir de este momento dicha fila estaría bloqueada para el resto de usuarios que tuviesen acceso a la base de datos).

### Ejemplo 1 [PTRANS1]:

```
database almacen
define
 variable porcentaje smallint
end define
```

```

main begin
 prompt for porcentaje label "Tanto por ciento de incremento"
 end prompt
 begin work
 update articulos set pr_vent = pr_vent * (1 * $porcentaje / 100)
 if errno <> 0 then rollback work
 else commit work
 end main

```

Este programa pedirá por pantalla el tanto por ciento que se desea incrementar a todos los artículos grabados en la tabla «articulos». A continuación comienza la transacción (BEGIN WORK) y, en caso de no producirse ningún error («ERRNO»), se grabará la actualización; en caso contrario el precio de los artículos quedará igual que antes de comenzar la actualización.

#### NOTAS:

1.— Téngase en cuenta que, en función de cada fabricante, cada sistema operativo UNIX tiene un número máximo de bloqueos («tunable parameters»), por lo que habrá que ajustar éste o utilizar, en su caso, la instrucción LOCK TABLE, que inhibe el bloqueo de fila.



2.— En Windows sólo existe la posibilidad de bloqueo en las versiones para red de área local. Este bloqueo se controla mediante el comando **share**.

## Recuperación de datos

Como ya se ha indicado anteriormente, la instrucción START DATABASE asigna un nuevo fichero transaccional a la base de datos indicada en su sintaxis. Esta operación debe realizarse cada vez que se obtenga una copia de seguridad de la base de datos. Dado que las operaciones de actualización, inserción y borrado sobre las tablas de la base de datos se reflejan sobre dicho fichero transaccional, en caso de pérdida de información en la base de datos aquella podría recuperarse. La instrucción que se encarga de esta operación es la siguiente:

**ROLLFORWARD DATABASE** base\_datos [TO fichero\_resultado]

Mediante la ejecución de esta instrucción sería posible la reconstrucción de la base de datos partiendo de la información registrada en un fichero transaccional sobre una copia de seguridad. La instrucción ROLLFORWARD DATABASE sólo recuperará aquellas transacciones que hayan finalizado con COMMIT WORK y no con ROLLBACK WORK.

La base de datos indicada en esta instrucción («base\_datos») tiene que ser transaccional. El fichero indicado en la cláusula «TO» («fichero\_resultado») sirve para almacenar el resultado de la ejecución de dicha instrucción.

A continuación se expone un supuesto práctico con el procedimiento lógico a la hora de asignar ficheros transaccionales al realizar diariamente copias de seguridad.

#### • Viernes:

El fichero transaccional que se encuentra activo en la base de datos el viernes es: «/usr/ctldemo/viernes»

#### • Lunes:

```
start database almacen with log in "/usr/ctldemo/lunes";
```

Hacer copia de seguridad de la base de datos junto con el fichero transaccional antiguo «/usr/ctldemo/viernes».

Procesos de actualización, inserción y borrado sobre las tablas de la BD.

#### • Martes:

```
start database almacen with log in "/usr/ctldemo/martes";
```

Hacer copia de seguridad de la base de datos junto con el fichero transaccional antiguo «/usr/ctldemo/lunes».

Procesos de actualización, inserción y borrado sobre las tablas de la BD.

- **Miércoles:**

start database almacen with log in "/usr/ctldemo/miercole";

Hacer copia de seguridad de la base de datos junto con el fichero transaccional antiguo «/usr/ctldemo/martes».

Procesos de actualización, inserción y borrado sobre las tablas de la BD.

¡CAÍDA DEL SISTEMA CON PÉRDIDA DE INFORMACIÓN DE ALGUNAS TABLAS!

Recuperar la copia de seguridad del «martes». Inicialmente, parece que todo el trabajo del «miércoles» se ha perdido.

Ejecutar:

rollforward database almacen

Con esta instrucción se recuperarán todas las operaciones realizadas en la base de datos que se encuentren registradas en el fichero transaccional «miercole».

En principio, la base de datos queda en estado normal de explotación, habiendo recuperado todo el trabajo del miércoles excepto la última transacción (BEGIN WORK), que no finalizó por caída del sistema.

- **Jueves:**

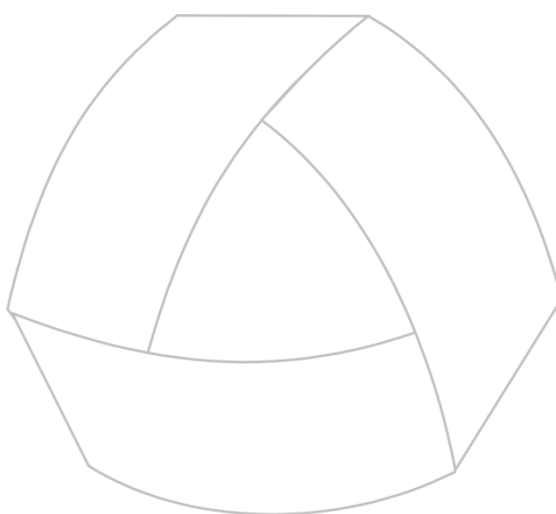
start database almacen with log in "/usr/ctldemo/jueves";

Hacer copia de seguridad de la base de datos junto con el fichero transaccional antiguo «/usr/ctldemo/miercole».

Procesos de actualización, inserción y borrado sobre las tablas de la BD.

etc., etc.

NOTA: Si se incluyen las instrucciones BEGIN WORK, COMMIT WORK, ROLLBACK WORK y ROLLFORWARD DATABASE en un programa CTL, se producirá un error si la base de datos en curso no es transaccional. Dicho error podría eliminarse preparando las instrucciones (mediante la instrucción PREPARE) y ejecutándolas con EXECUTE. En este caso, la instrucción de control de errores WHENEVER podría eliminar el mensaje de error producido por no estar la base de datos preparada para trabajar con transacciones.



MultiBase  
MANUAL DEL PROGRAMADOR

## **CAPÍTULO 19. Control de errores e interrupciones**

### Introducción

---

La instrucción **WHENEVER** permite controlar tanto las interrupciones provocadas por el usuario como las derivadas de otros procesos y errores durante la ejecución de programas.

Las instrucciones de SQL no activarán la acción de la **WHENEVER** a no ser que ésta incluya alguna variable «host». Para poder emplear la instrucción **WHENEVER** sobre las instrucciones de este tipo, éstas deberán incluirse en la sintaxis de una **PREPARE**. Dicha instrucción se ejecutará a continuación mediante **EXECUTE**.

La instrucción **WHENEVER** asocia un tratamiento de error (conjunto de instrucciones) a la aparición del mismo, esto es, cuando la variable interna de CTL «errno» es distinta de cero, o a un tratamiento de interrupción del programa, generado por la tecla correspondiente a la acción <break> del teclado o la señal «sigint» del sistema operativo UNIX («kill»).

Al realizar un listado, si el operador pulsa la tecla o combinación de teclas asignada a la ruptura de procesos en cada sistema operativo, automáticamente preguntará si se desea continuar. Este control interno es producido por una instrucción **WHENEVER** implícita.

NOTA: La instrucción **WHENEVER** nunca detectará avisos de error («warnings»).

La sintaxis completa de esta instrucción es la siguiente:

**WHENEVER {ERROR | INTERRUPT} [SILENT] [NO RESET]  
{IGNORE | CALL función(parámetros) | GOTO etiqueta | DEFAULT | EXIT}**

A continuación se indican las diferentes sintaxis admitidas por la instrucción **WHENEVER**:

#### 1.— Ignoración de errores:

**WHENEVER {ERROR | INTERRUPT} [SILENT] [NO RESET] IGNORE**

Esta sintaxis indica que se ignorará el posible error o la interrupción del programa por parte del usuario.

**Ejemplo [PWHENEV1]. Ignoración de errores al ejecutarse instrucciones de control de transacciones sobre una base de datos no transaccional:**

```
database almacen

main begin
 prepare inst1 from "begin work"
 prepare inst2 from "commit work"
 prepare inst3 from "rollback work"
 whenever error silent ignore
 execute inst1
 update articulos set pr_vent = pr_vent * 1.05
 if errno <> 0 then execute inst3
 else execute inst2
end main
```

Si la base de datos no es transaccional, al ejecutar cualquiera de las instrucciones de control de transacciones se producirá un error imposible de detectar con la instrucción **WHENEVER**, ya que aquéllas no admiten variables «hosts». En el ejemplo anterior, las instrucciones de control de transacciones se han preparado y ejecutado respectivamente con las instrucciones **PREPARE** y **EXECUTE**. En este caso, los errores que se produzcan en la ejecución de dichas instrucciones sí serán detectados por la instrucción **WHENEVER**, pero serán ignorados y no se mostrarán en la pantalla.

El siguiente ejemplo es una copia del anterior, si bien en este caso la instrucción **WHENEVER** no se activará nunca en caso de existir algún error, por ejemplo si la base de datos no es transaccional:

```

database almacen

main begin
 whenever error silent ignore
 begin work
 update articulos set pr_vent = pr_vent * 1.05
 if errno <> 0 then rollback work
 else commit work
end main

```

## 2.— Llamada a una función:

En caso de producirse un error o una interrupción del programa por parte del usuario, el control del programa se pasa a la ejecución de la función indicada en la cláusula «CALL». Esta cláusula también puede combinarse con la «GOTO», produciéndose un salto hacia la «etiqueta» especificada. La sintaxis de la instrucción WHENEVER en este caso es la siguiente:

**WHENEVER {ERROR | INTERRUPT} [SILENT] [NO RESET] CALL función()**

**Ejemplo [PWHENEV2.SCT]. Detección del error producido por la imposibilidad de abrir la base de datos «almacen» al estar bloqueada por otro usuario:**

```

define
 frame arti
 screen
 {
 %
 Código Artículo: [a]
 *

 Descripción: [b]
 Precio Venta: [c]
 }
end screen

variables
 a = articulo integer
 b = descripcion char
 c = pr_vent money(16,2)
end variables

control
 after update of arti
 if newvalue = true then
 update articulos set pr_vent = $arti.pr_vent
 where articulos.articulo = $arti.articulo
 end control
end control

editing
 after editupdate of pr_vent
 exit editing
end editing

layout
 box
 label "A r t í c u l o s"
 line 5
 column 20
end layout
end frame
end define

```

```
main begin
 prepare inst1 from "database almacen"
 whenever error silent call errores()
 execute inst1
 forever begin
 window from select articulo, descripcion, pr_vent from articulos
 15 lines label "A r t í c u l o s"
 at 5,10
 set arti.articulo, arti.descripcion, arti.pr_vent
 from 1,2,3
 if cancelled = true then break
 input frame arti for update variables pr_vent
 end input
 clear frame arti
 end
end main

function errores() begin
 if errno = -1107 then begin
 if yes("Base de datos bloqueada, reintentar","Y") = true then execute inst1
 else exit program
 end
 end
end function
```

Este programa selecciona la base de datos «almacen» para la actualización, mediante un FRAME, del precio de venta de los artículos seleccionados por el operador. A la hora de seleccionar la base de datos, si ésta se encuentra bloqueada por otro usuario, la instrucción WHENEVER lo detecta, ejecutándose una función para el tratamiento del error. En este caso se pregunta al usuario si desea reintentar la apertura. En caso afirmativo se vuelve a ejecutar la instrucción preparada (DATABASE) hasta que pueda seleccionarse o bien hasta que el usuario desista en su intento. Una vez seleccionada la base de datos aparecerá una ventana con todos los artículos existentes. En caso de que el usuario elija uno de ellos, éste se carga en un FRAME, pudiendo entonces modificar el precio de venta en dicha estructura.

### 3.— Salto a una etiqueta:

En caso de producirse un error o una interrupción del programa por parte del usuario, el control del programa saltará hasta donde se encuentre definida la «etiqueta». La sintaxis de la instrucción WHENEVER en este caso es la siguiente:

**WHENEVER {ERROR | INTERRUPT} [SILENT] [NO RESET] GOTO etiqueta**

La «etiqueta» se define de la siguiente manera:

**ERROR etiqueta:**

Esta definición se puede situar en cualquier lugar donde se pueda colocar una instrucción.

**Ejemplo [PWHENEV3]. Apertura de un STREAM de lectura y cierre sin realizar ninguna operación en caso de no existir el fichero indicado para la lectura:**

```
define
 variable linea char(80)
end define

main begin
 whenever error silent goto cierre
 start input stream standard from "facturas"
 prompt stream standard for linea as line
 end prompt
 error cierre: stop input stream standard
end main
```



En este caso, si el fichero «facturas» no existe en el directorio en curso se producirá un error. Este error no se presentará en pantalla al haber incluido en la instrucción WHENEVER la cláusula «SILENT». Además, el STREAM no realizará ninguna lectura, es decir, la instrucción PROMPT FOR no se ejecutará jamás debido a la cláusula «GOTO», que redirecciona el control del programa al cierre de dicho STREAM.

#### 4.— Llamada a una función y salto a una etiqueta:

La combinación de los dos puntos anteriores implica el tratamiento de un error o una interrupción por una función de usuario y su posterior salto a una etiqueta definida en el módulo. Su sintaxis es la siguiente:

**WHENEVER {ERROR | INTERRUPT} [SILENT] [NO RESET] CALL función() GOTO etiqueta**

**Ejemplo [PWHENEV4]. Detección de un error producido por la apertura de un STREAM al no existir el fichero del que se intenta realizar la lectura:**

```
define
 variable linea char(100)
end define

main begin
 whenever error silent call trata_error() goto fin
 start output stream standard from "fichero"
 prompt stream standard for linea as line
end prompt
error fin:
 stop input stream standard
end main

function trata_error() begin
 if errno = 2 then
 pause "Fichero inexistente. Pulse [Return]"
 reverse bell
end function
```

Este programa abre un STREAM de entrada para la lectura de un fichero. En caso de que el «fichero» a leer no exista en el directorio en curso se producirá un error. Este error es captado por la instrucción WHENEVER, la cual ejecuta una función de usuario. En esta función de control de errores se consulta el número de error devuelto por la apertura del STREAM, mostrándose un mensaje en pantalla indicando que el «fichero» no existe en caso de producirse el código de error «2» («errno=2»). Una vez que el usuario pulsa [RETURN] se salta hacia la etiqueta «fin» definida en la instrucción STOP.

#### 5.— Acción por defecto:

La acción por defecto que se ejecuta cuando se produce un error o una interrupción es la salida del programa, es decir, EXIT. Si se utiliza la cláusula «DEFAULT» se puede detectar un error o una interrupción antes de que finalice el programa. La sintaxis de la instrucción WHENEVER en este caso es la siguiente:

**WHENEVER {ERROR | INTERRUPT} [SILENT] [NO RESET] DEFAULT**

**Ejemplo [PWHENEV5]. Tratamiento de diversos tipos de error con diferentes instrucciones WHENEVER:**

```
database almacén

main begin
 whenever error call trata_error()
 prepare inst1 from "esto debería ser una instrucción SELECT"
 call declarar()
 whenever error default
 call mostrar()
 pause "Fin del programa"
end main

function trata_error() begin
```

```
 pause "El error producido es: " && errno
 end function

 function declarar() begin
 whenever error goto salida
 declare cursor listado for inst1
 prepare inst2 from "delete from tabla_noexiste"
 execute inst2
 pause errno
 error salida: return
 end function

 function mostrar() begin
 prepare inst3 from "update tablanoexiste set col = 1"
 execute inst3
 window from cursor listado
 10 lines label "L i s t a d o"
 at 1,1
 end function
```

Dependiendo de la función que se ejecute en cada momento, el programa anterior emplea diversos controles de errores. En primer lugar, el módulo prepara una instrucción («inst1») con un literal que indica que debería ser una instrucción SELECT, ya que posteriormente se declara un CURSOR. Al no tratarse de una instrucción de SQL se produce un error al ejecutarla, el cual es detectado con la primera instrucción WHENEVER, ejecutándose la función «trata\_error». A continuación se declara el CURSOR con dicha instrucción preparada previamente (la que produjo el error). Tanto esta declaración como la siguiente instrucción, TSQL, que ejecuta una DELETE errónea, se controlan con una WHENEVER que salta a la etiqueta «salida». En el momento que se ejecuta la instrucción TSQL se producirá el salto a dicha etiqueta. Por último, se emplea la instrucción WHENEVER con la cláusula DEFAULT, la cual reinicializa el tratamiento de errores dejándolo por defecto. Es decir, en el momento que se ejecuta la función «mostrar», la instrucción TSQL producirá un error, ya que la tabla donde se intenta realizar la actualización (UPDATE) no existe, finalizando el programa a continuación.

Si en el ejemplo anterior se hubiera especificado la cláusula «NO RESET» en la instrucción «WHENEVER ERROR NO RESET GOTO SALIDA», el programa hubiera ejecutado también la instrucción WHENEVER especificada al principio del módulo, es decir, «WHENEVER ERROR CALL trata\_error()».

### 6.— Salida del programa:

La cláusula «EXIT» abandona la ejecución del programa en el momento que se detecta un error o una interrupción por parte del operador. La sintaxis de la WHENEVER en este caso es la siguiente:

**WHENEVER {ERROR | INTERRUPT} [SILENT] [NO RESET] EXIT**

La cláusula «EXIT» se encuentra implícita en la «DEFAULT», ya que se trata de la opción por defecto del CTL. En el momento que se produce un error o una interrupción finaliza el programa.

**Ejemplo [PWHENEV6]. Detección de un error producido en una instrucción PREPARE, saliendo del programa en el momento que se produzca:**

```
database almacen

define
 frame provin like provincias.*
end frame
variable optimiza char(200)
end define

main begin
 whenever error silent exit
 forever begin
 input frame provin for query by name to optimiza
```

```

end input
if cancelled = true then break
prepare inst1 from "select * from provincias where " &&
 optimiza clipped
declare cursor listado for inst1
window from cursor listado
 10 lines label "L i s t a d o"
 at 1,1
end
end main

```

Este módulo define un FRAME para la petición de condiciones al usuario. Estas condiciones se almacenan en la variable «optimiza» resultante de la instrucción INPUT FRAME. En el momento que esta variable no contenga una condición válida, la instrucción PREPARE producirá un error, que será el que active la instrucción «WHENEVER ERROR», produciéndose la salida del programa.

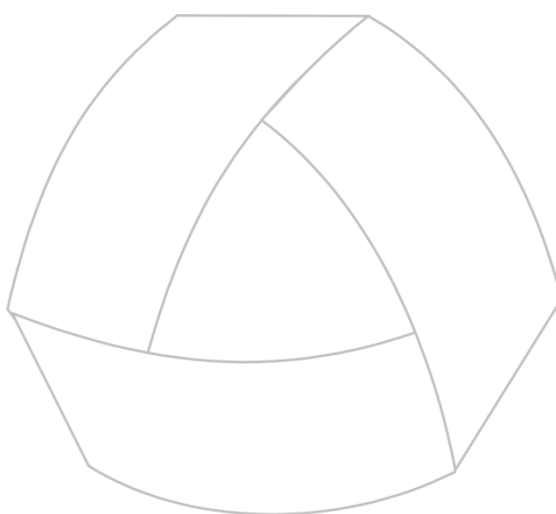
## Propagación del error

Cuando se define una instrucción de control de error, así como una posible interrupción, hay que tener presente el ámbito de ejecución de dicho control, esto es, qué instrucciones quedan bajo ese control. La instrucción WHENEVER tiene validez en el bloque en el que se define y en todos los dependientes de éste, es decir, no actuará en el bloque que llamó al que contiene la WHENEVER, pero sí en aquellos a los que éste pueda llamar durante su ejecución. Se entiende como bloque el conjunto de instrucciones incluidas en alguno de los siguientes elementos CTL:

|                  |                                                                                                                                    |
|------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>MAIN</b>      | Función principal y de entrada a un programa.                                                                                      |
| <b>FUNCTION</b>  | Función específica del usuario en un módulo.                                                                                       |
| <b>MENU</b>      | En cualquiera de las opciones que componen un menú de CTL.                                                                         |
| <b>CONTROL</b>   | Sección CONTROL de un FORM o un FRAME.                                                                                             |
| <b>EDITING</b>   | Sección EDITING del FORM o FRAME.                                                                                                  |
| <b>ON KEY</b>    | Cualquiera de las teclas definidas en la sección CONTROL de la GLOBAL, FORM, FRAME o en PROMPT FOR.                                |
| <b>ON ACTION</b> | Cualquiera de las acciones definidas en la sección CONTROL de la GLOBAL, FORM, FRAME o en PROMPT FOR.                              |
| <b>CURSOR</b>    | Párrafos «ON EVERY ROW», «ON LAST ROW», «BEFORE GROUP» y «AFTER GROUP» de la sección CONTROL de la directiva DECLARE de un CURSOR. |
| <b>FORMAT</b>    | Párrafos «FIRST PAGE HEADER», «PAGE HEADER» y «PAGE TRAILER» de la instrucción FORMAT de un STREAM.                                |

Vemos pues que un tratamiento de error o interrupción está asociado al bloque donde se define. Asimismo, podemos activar varios tratamientos de error propagando el mismo mediante la cláusula «NO RESET» al bloque que llamó al que contiene la definición.





MultiBase  
MANUAL DEL PROGRAMADOR

## **CAPÍTULO 20. Particularidades de la versión para Windows**

### Introducción



La versión de MultiBase para Windows dispone de ciertas particularidades no contempladas en las versiones correspondientes a los restantes entornos operativos sobre los que funciona la herramienta. Estas particularidades sirven fundamentalmente para comunicar el lenguaje de cuarta generación de MultiBase (CTL) con otras aplicaciones Windows, y son las siguientes:

- Utilización del portapapeles de Windows.
- Empleo de librerías dinámicas (DLLs).
- Empleo de intercambio dinámico de datos (DDEs).

En los siguientes epígrafes se comenta en detalle las características de estas particularidades específicas de MultiBase para Windows.

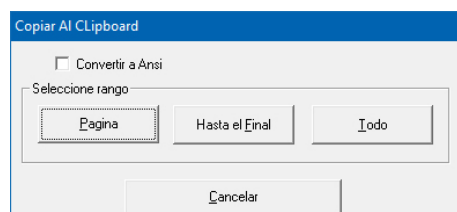
### Utilización del portapapeles



Cada vez que aparezca en pantalla la ventana provocada por la ejecución de la instrucción WINDOW de CTL (ver figura izquierda) se podrá copiar la información contenida en la ventana de MultiBase al Portapapeles («Clipboard») del Windows. Para ello, utilice la tecla correspondiente a la acción <fcopy> (normalmente la tecla [F6]; si no lo es, pulse la tecla asignada a la acción <fhelp> para ver cuál es la correspondiente a dicha acción). Una vez ejecutada aparecerá la ventana de diálogo que se muestra en la figura de la derecha, la cual permitirá copiar al Portapapeles bien el contenido de la pantalla, bien desde la página que se muestra hasta el final o bien toda la información de todas las páginas.



Ventana provocada por la ejecución de la instrucción WINDOW.



Cuadro de diálogo para copiar al Portapapeles de Windows.

Asimismo, en el caso de pulsar la tecla asociada a la acción <fcopy> sobre un campo de un FORM o un FRAME o sobre la petición de valores sobre una variable mediante la instrucción PROMPT FOR, el valor que contenga dicho campo se copiará al Portapapeles, pudiendo ser utilizado posteriormente en cualquier otra aplicación Windows. Para copiar todo el contenido de la variable que se está editando en el FORM o FRAME o en la instrucción PROMPT FOR, el cursor de la pantalla tiene que estar situado en la primera posición de dicha variable. De este modo, al pulsar la tecla para copiar al portapapeles, CTL sólo copiará desde la posición del cursor hasta el final de la variable que se esté editando.

Una vez que la información de una variable en edición ha sido traspasada al Portapapeles de Windows, dicha información podrá ser incluida en cualquier aplicación de este entorno a través de la opción «Pegar». Para comprobarlo, abra el tratamiento de textos «Write» y ejecute dicha opción.

## Empleo de librerías dinámicas (DLLs)



La versión de MultiBase para Windows permite el empleo de librerías dinámicas (DLLs) en programas CTL. Para poder emplear estas librerías dinámicas es preciso configurar la sección «.DLL» del fichero de inicialización de MultiBase (fichero con extensión «.ini», por defecto «MB.INI»). La forma de elegir un fichero de inicialización u otro es mediante la cláusula «-ini» del comando **ctl** (para más información sobre de la configuración de este fichero y la forma de seleccionarlo para la ejecución de un programa CTL consulte los capítulos «Otros Ficheros de Administración» y «Comandos MultiBase» en el Manual del Administrador).

El aspecto de la sección «.DLL» en el fichero de inicialización es el siguiente:

```
.DLL
LIB mbdib.dll
DoKey (WORD, WORD, LPSTR)
ShowBmp (WORD, LPSTR, WORD, WORD, WORD, WORD) return WORD
CreateDibWin(WORD , LPSTR, WORD , WORD , WORD , WORD) return WORD
MoveDibWin(WORD, WORD, WORD, WORD, WORD) return WORD
DestroyDibWin(WORD) return WORD
DrawDibWin(WORD, LPSTR, LPSTR) return WORD
LoadDibWin(WORD, LPSTR) return WORD
ShowDibWin(WORD, LPSTR)
ForceShow(WORD)

LIB mbtext.dll
ShowTxt (WORD, LPSTR, WORD, WORD, WORD, WORD) return WORD
CreateTxtWin(WORD , LPSTR, WORD , WORD , WORD , WORD) return WORD
MoveTxtWin(WORD, WORD, WORD, WORD, WORD) return WORD
WordTxtWin(WORD, LPSTR) return WORD
DestroyTxtWin(WORD) return WORD
DrawTxtWin(WORD, LPSTR, LPSTR) return WORD
LoadTxtWin(WORD, LPSTR) return WORD
ShowTxtWin(WORD, LPSTR)

LIB utildll.dll
OpenPort (LPSTR, LPSTR) return WORD
WritePort (WORD, LPSTR, WORD) return WORD
ReadPort (WORD, WORD) return LPSTR
ClosePort (WORD) return WORD
Chrono (WORD, WORD) return DWORD
```

Una vez incluidas las librerías dinámicas en el fichero de inicialización, la forma de ejecutarlas en un programa CTL es mediante la función interna «dllfun()», cuya sintaxis es la siguiente:

```
dllfun(nombre_función, param1, ..., paramx)
```

NOTA: La generación de librerías dinámicas se puede realizar en cualquier otro lenguaje de programación que lo permita (C, Visual Basic, etc.). Por lo tanto, el tratamiento de DLLs supone la utilización de funciones de otro lenguaje en el código de MultiBase.

**Ejemplo [DLLS.SCT] y [DLL\_LIB.SCT]. Estadísticas de ventas con distintos gráficos:**

```
database almacen define
 variable hg smallint
 variable i smallint
 variable current_frame char(4)
 variable intro smallint
 frame f_line
 screen
{
% Opciones

[a] Recto
```

```

 [b] Curva
 *
 }

 end screen

 variables
 a = line_straight char default " "
 b = line_curve char default "x"

 untagged line_param char(20)
 end variables

 control
 after display of f_line
 call update_line_options()
 end control

 editing
 before editupdate of f_line
 if line_curve = "x" then next field "line_curve"
 before line_straight line_curve
 message "Seleccionar el tipo de línea con el ratón"
 before line_straight line_curve
 if evalvar(curfield) <> "x" then begin
 let line_straight = " "
 let line_curve = " "
 call letvar(curfield,"x")
 call update_line_options()
 call do_chart()
 end
 end editing

 layout
 line 14
 column 5
 end layout
end frame

frame f_pie
screen
{
%

 Opciones

 [a] 2D
 [b] 3D

 [c] Tanto por ciento
 *
}

end screen

variables
 a = pie_2d char default " "
 b = pie_3d char default "x"
 c = pie_percent char default " "
 untagged pie_param char(2)
 untagged pie_perc smallint
end variables

control
 after display of f_pie

```



```

 call update_pie_options()
 end control

 editing
 before editupdate of f_pie
 if pie_3d = "x" then next field "pie_3d"
 before pie_3d pie_2d pie_percent
 message "Seleccionar el tipo de pie con el ratón"
 before pie_2d pie_3d
 if evalvar(curfield) <> "x" then begin
 let pie_2d = " "
 let pie_3d = " "
 call letvar(curfield,"x")
 call update_pie_options()
 call do_chart()
 end
 before pie_percent
 call letvar(curfield, evalcond(pie_percent, "=", "x", " ", "x"))
 call update_pie_options()
 call do_chart()
 if pie_2d = "x" then next field "pie_2d"
 else next field "pie_3d"
 end editing
 end editing

 layout
 line 14
 column 5
 end layout
end frame

frame f_bar
screen
{
%
 Opciones

 [a] 2D Tamaño ...:[s]
 [b] 3D
 [c] Grupo
 [d] Profundidad
 [e] Pila
 *
}

end screen

variables
 s = bar_size smallint check($ > 0) default 50
 a = bar_2d char default " "
 b = bar_3d char default " "
 c = bar_group char default " "
 d = bar_deep char default "x"
 e = bar_stacked char default " "
 untagged bar_param char(20)
end variables

control
 after display of f_bar
 call update_bar_options()
 end control

 editing
 before bar_size
 message "Cambiar tamaño o seleccionar tipo de barra con el ratón"

```

```

 after bar_size
 if newvalue = true then begin
 call update_bar_options()
 call do_chart()
 next field "bar_size"
 end
 before bar_2d bar_3d bar_group bar_deep bar_stacked
 if evalvar(curfield) <> "x" then begin
 let bar_2d = " "
 let bar_3d = " "
 let bar_group = " "
 let bar_stacked = " "
 let bar_deep = " "
 call letvar(curfield, "x")
 call update_bar_options()
 call do_chart()
 end
 next field "bar_size"
end editing

layout
 line 14
 column 5
end layout
end frame

frame f_chart
screen
{
?
 %
 A B C D E
 *
1 [a1 |b1 |c1 |d1 |e1]
2 [a2 |b2 |c2 |d2 |e2]

? $

? ?

Tipo[t]

 $
 $
}

end screen

variables
 a1 = value_a1 decimal(6) required default 1
 b1 = value_b1 decimal(6) required default 1.3
 c1 = value_c1 decimal(6) required default 4.5
 d1 = value_d1 decimal(6) required default 2
 e1 = value_e1 decimal(6) required default 23.4
 a2 = value_a2 decimal(6) required default 3.2
 b2 = value_b2 decimal(6) required default 3
 c2 = value_c2 decimal(6) required default 2.5
 d2 = value_d2 decimal(6) required default 3
 e2 = value_e2 decimal(6) required default 4.5

```

```

t = chart_type char default "BAR"
end variables

menu m_f1 "Chart"
 option "Barra"
 let chart_type = "BAR"
 call disp_option_frame()
 call do_chart()
 option "Línea"
 let chart_type = "LINE"
 call disp_option_frame()
 call do_chart()
 option "PIE"
 let chart_type = "PIE"
 call disp_option_frame()
 call do_chart()
 option "Editar Valores"
 call rm_option_frame()
 input frame f_chart for update
 end input
 if cancelled = true then call do_chart()
 call disp_option_frame()
 option "Opciones CHART"
 switch chart_type
 case "BAR"
 input frame f_bar for update
 end input
 case "LINE"
 input frame f_line for update
 end input
 case "PIE"
 input frame f_pie for update
 end input
 end
 if cancelled = true then call do_chart()
end menu

control
 after display of f_chart
 if intro = 1 then begin
 call disp_option_frame()
 call do_chart()
 let intro = 0
 end
 before update of f_chart
 call disp_option_frame()
 after update cancel of f_chart
 call rm_option_frame()
end control

editing
 before editupdate of chart_type
 next field up "value_e2"
 next field down "value_a1"
 after editupdate of value_a1 value_b1 value_c1 value_d1 value_e1
 value_a2 value_b2 value_c2 value_d2 value_e2
 if newvalue = true then begin
 call chart_data(ascii (curfield[7,7]) - ascii("a") + 1,
 curfield[8,8], evalvar(curfield))
 call do_chart()
 end
end editing

```

```
 layout
 box
 label "Caracteres Gráficos"
 lines 22
 columns 77
 line 2
 column 2
 end layout
 end frame
end define

global
 control
 on beginning
 initialize frame f_bar with default
 initialize frame f_line with default
 initialize frame f_pie with default
 initialize frame f_chart with default
 call init_values()
 on ending
 call chart_quit()
 call rm_option_frame()
 end control
 end global

main begin
 let intro = 1
 menu frame f_chart
end main

function disp_option_frame() begin
 if current_frame <> chart_type or current_frame is null then begin
 call rm_option_frame()
 switch chart_type
 case "LINE"
 display frame f_line
 case "BAR"
 display frame f_bar
 case "PIE"
 display frame f_pie
 end
 let current_frame = chart_type
 end
end function

function rm_option_frame() begin
 switch current_frame
 case "BAR"
 clear frame f_bar
 case "LINE"
 clear frame f_line
 case "PIE"
 clear frame f_pie
 end
 let current_frame = ""
end function

function update_bar_options() begin
 switch
 case bar_2d = "x"
 let bar_param = "2D"
 case bar_3d = "x"
 let bar_param = "3D"
```

```

 case bar_deep = "x"
 let bar_param = "DEEP"
 case bar_group = "x"
 let bar_param = "GROUP"
 case bar_stacked = "x"
 let bar_param = "STACKED"
 end
end function

function update_line_options() begin
 if line_curve = "x" then let line_param = "CURVE"
 else let line_param = "STRAIGHT"
end function

function update_pie_options() begin
 if pie_3d = "x" then let pie_param = "3D"
 else let pie_param = "2D"
 if pie_percent = "x" then let pie_perc = true
 else let pie_perc = false
end function

function do_chart() begin
 let hg = chart_init()
 call chart_window(1,"",0,38,13,39,11)
 call dllfun("TTSet",hg,"GRAPHTITLE", "Sales")
 switch chart_type
 case "BAR"
 call chart_bar(bar_size, bar_param)
 case "PIE"
 call chart_pie(pie_param, pie_perc)
 case "LINE"
 call chart_line(line_param)
 end
end function

function init_values()
 x_cnt smallint
 y_cnt smallint
begin
 call chart_data(-1,-1,-1)
 for x_cnt = 1 to 5
 for y_cnt = 1 to 2
 call chart_data(x_cnt, y_cnt, evalvar("value_" &&
 character(ascii("a") + x_cnt - 1) && y_cnt using "#"))
 end
 end
end function

```

**Programa [DLL\_LIB.SCT]:**

```

define
 variable chart_handle smallint default -1
 array chart_data[256] decimal

 variable max_row smallint
 variable max_col smallint
end define

function chart_init() begin
 if chart_handle is not null and chart_handle >= 0 then begin
 call dllfun("TTDeleteGraph",chart_handle)
 call dllfun("TTQuitGraph")
 end
 let chart_handle = dllfun("TTNewGraph", gethwnd(), gethinstance())
 return chart_handle
end function

```

```
function chart_quit() begin
 if chart_handle is not null and chart_handle >= 0 then begin
 call dllfun("TTDeleteGraph",chart_handle)
 call dllfun("TTQuitGraph")
 end
end function

function chart_window(w_type, title, unit, x, y, w, h)
 wx smallint
 wy smallint
begin
 if unit = 0 then begin
 let x = xtopixels(x)
 let w = xtopixels(w)
 let y = ytopixels(y,TRUE)
 let h = ytopixels(h, FALSE)
 end
 let wx = getwinparm("x")
 let wy = getwinparm("y")
 call dllfun("TTSet", chart_handle, "WINPOS", wx + x && wy + y && w && h)
 if w_type = 0 then call dllfun("TTSet", chart_handle, "WINSTYLE","CHILD")
 else call dllfun("TTSet", chart_handle, "WINSTYLE","NOCAPTION")
 if title is not null then call dllfun("TTSet", chart_handle, "WINTITLE", title)
 else call dllfun("TTSet", chart_handle, "WINTITLE", "")
end function

function chart_data(rw, cl, vl)
 i smallint
begin
 if rw = -1 then begin
 let max_row = 0
 let max_col = 0
 for i = 1 to 256
 let chart_data[i] = 0
 end
 end
 else begin
 let chart_data[(rw - 1) * 16 + cl] = vl
 let max_row = evalcond(rw, ">", max_row, rw, max_row)
 let max_col = evalcond(cl, ">", max_col, cl, max_col)
 end
end
end function

function chart_bar(bar_w, bar_t)
 tparam char(20)
begin
 call create_chart_values(max_col)
 if bar_w is not null then let tparam = bar_w using "####" && " " && bar_t
 else let tparam = bar_t
 call dllfun("TTDraw", chart_handle, "BAR", tparam)
end function

function chart_line(l_param) begin
 call create_chart_values(max_col)
 call dllfun("TTDraw", chart_handle, "LINE", l_param)
end function

function chart_pie (pie_t, pie_perc)
 tparam char(20)
begin
 call create_chart_values (1)
 let tparam = pie_t
 if pie_perc = true then let tparam = tparam clipped && " PERCENT"
```

```

 call dllfun("TTDraw", chart_handle, "PIE", tparam)
 end function

 function create_chart_values(cols)
 x smallint
 y smallint
 val_str char(5000)
 begin
 let val_str = ""
 for y = 1 to max_row begin
 let val_str = val_str clipped && " " && y using "##"
 for x = 1 to cols
 let val_str = val_str clipped && " " && chart_data[(y - 1) * 16 + x]
 end
 call dllfun("TTSet", chart_handle, "DataSet", max_row using "##"
 && " " && cols using "##" && val_str)
 end
 end function

```

## Protocolo de intercambio dinámico de datos (DDE)



El protocolo de comunicaciones entre diferentes aplicaciones Windows (DDE, «Dynamic Data Exchange») permite a los programadores acceder desde el lenguaje de cuarta generación de MultiBase (CTL) a cualquier servidor DDE de Windows.

Frente a la utilización del portapapeles y de las librerías dinámicas (que cumplen igualmente este cometido), la incorporación del mecanismo DDE supone una gran mejora en el intercambio de datos entre aplicaciones.

DDE es un protocolo, basado en mensajes, para intercambiar datos entre distintos programas Windows. Gracias a esta nueva funcionalidad los programadores podrán acceder desde el CTL a cualquier servidor DDE de Windows, integrando en una arquitectura cliente-servidor las aplicaciones desarrolladas con MultiBase y los diferentes paquetes del mercado que incluyen el DDE entre sus características (Microsoft Excel, Microsoft Word, Lotus, etc.).

MultiBase para Windows dispone de cuatro funciones que sirven de interface entre el lenguaje de cuarta generación CTL y el protocolo DDE:

- **DDEinitiate**(aplicación, tipo\_comunicación): Esta función abre dinámicamente un canal de comunicación para el intercambio de datos y retorna el número de canal abierto. Para que esta comunicación sea posible es preciso haber ejecutado previamente la aplicación con la cual se desea enlazar. Esto se realizará con la instrucción «RUN aplicación» del CTL.
- **DDEexecute**(canal, comando): Provoca la ejecución de un comando en la aplicación con la que previamente se ha iniciado la comunicación. Esta función dispone de unos códigos para comprobar si la comunicación se efectúa de manera correcta o no.
- **DDEpoke**(canal, item, data): Esta función se encarga de enviar datos a la aplicación con la que mantenemos la comunicación. Tanto en esta función como en la «DDEexecute» los parámetros a enviar dependen de la aplicación con la que deseamos mantener la comunicación. En estas funciones los valores a traspasar en los parámetros «comando», «item» y «data» dependen de lo admitido por la propia aplicación con la que comunica.
- **DDEterminate**(canal): Fin de la comunicación con la aplicación.

Como conclusión, para obtener rendimiento del protocolo DDE hay que conocer perfectamente la aplicación «servidor» a la cual enviamos y solicitamos información (Microsoft Excel, Microsoft Word, Lotus, etc.).

### Ejemplo 1. Función que comunica con la aplicación «MONOLOG» para emitir sonidos:

```

function comuni(s)
 canal smallint
begin

```

```
let canal = DDEinitiate ("MONOLOG","TALK")
call DDEpoke (canal, " ", s)
call DDEterminate (canal)
end function
```

**Ejemplo 2. Ejemplo de comunicación DDE utilizando WordPerfect como servidor DDE y MultiBase como cliente. La prueba consiste en abrir un fichero WordPerfect e imprimirlo:**

```
define
variable canal integer {* Identificador del canal de comunicación *}
variable resultado integer {* Resultado de la instruccion DdeExecute() *}
end define

main begin
 run "c:\wpwin60\wpwin.exe /fl c:\docs\docu.wpd"
 let canal = DDEinitiate("WPWin60_Macros", "Commands")
 if canal is not null and canal != 0 then begin
 let resultado = DDEexecute(canal, "PrintFullDoc()")
 if resultado != 1 then
 pause "Se ha producido un error."
 else
 pause "Pulse una tecla para continuar, una vez finalizado."
 end
 call DDEterminate(canal)
 end
end main
```